CUTE · PTX ISA · TMA · FP8 ATTENTION

swizzle<3,4,3>

m16n8k8.f32

CUDA 知乎

知乎专栏精选

优秀文章合集

cp.async.bulk

ldmatrix.x4

LAYOUT · TENSOR · TILEDCOPY · 指令集 · 内存模型 · 精度优化

e4m3 · e5m2

wgmma.mma_async

忠实整理 · 图文完整 · 仅用于开源学习

整理：LEETCUDA PROJECT

2026 年 11 月

CUDA 知乎优秀文章合集

知乎优秀 CUDA/CuTe 系列文章忠实整理

郑重声明

本书收录知乎多位作者（**reed**（专栏「CUDA 高性能编程」）、**竹熙佳处**（CuTe 教程系列）、**frankshi** 等）的 CUDA/CuTe 系列优秀文章，进行**忠实整理，除修正原文中明显的公式、图号与拼写笔误，对确证的技术性错误做最小订正，补全缺失的图注与参考文献外，不对原文内容做任何修改**；各章章首均标注作者与原文出处。

本书**仅用于开源学习**，**不得用于任何商业用途**。

全部文章著作权归原作者所有，转载请注明原文出处。

2026 年 11 月

整理说明

本书系统整理了知乎上多位作者的 CUDA/CuTe 系列优秀文章，共 32 篇，按系列与主题分四个部分编排：reed 的 CuTe 系列（13 篇）、竹熙佳处的 CuTe 教程系列（7 篇）、reed 的「NVidia GPU 指令集架构」系列（8 篇），以及 GPU 架构 · 内存 · Attention 精度主题单篇（4 篇，reed 3 篇与 frankshi 1 篇），同一部分内按系列依赖/发表顺序编排。整理过程中**忠实保留原文内容与图片**；除修正原文中明显的公式、图号与拼写笔误，对确证的技术性错误做最小订正，补全缺失的图注与参考文献外，不对原文内容做任何修改，仅做 Markdown 到 LaTeX 的排版映射（标题层级、代码块、公式、图片位置与图注均与原文一致）；各章章首标注作者与原文链接。

本书仅用于开源学习，不得用于任何商业用途。

reed 系列文章来源（13 篇）

章节	原文
01	cute 之 Layout https://zhuanlan.zhihu.com/p/661182311
02	cute Layout 的代数和几何解释 https://zhuanlan.zhihu.com/p/662089556
03	cute 之 Tensor https://zhuanlan.zhihu.com/p/663093816
04	cute 之 MMA 抽象 https://zhuanlan.zhihu.com/p/663092747
05	cute 之 Copy 抽象 https://zhuanlan.zhihu.com/p/666232173
06	cute 之简单 GEMM 实现 https://zhuanlan.zhihu.com/p/667521327
07	cute 之 Swizzle https://zhuanlan.zhihu.com/p/671419093
08	cute 之 GEMM 流水线 https://zhuanlan.zhihu.com/p/665082713
09	cute 之高效 GEMM 实现 https://zhuanlan.zhihu.com/p/675308830
10	cute 之 Hopper 前言 https://zhuanlan.zhihu.com/p/1948048832794427610
11	cute 之 Hopper MBarrier https://zhuanlan.zhihu.com/p/1962636004235153810
12	cute 之 Hopper TMA https://zhuanlan.zhihu.com/p/1985678344352731952
13	cute 之 TMA Descriptor 编码与隐藏的第 21bit https://zhuanlan.zhihu.com/p/2037200219700449995

竹熙佳处系列文章来源（7 篇）

章节	原文
01	写给大家看的 CuTe 教程：tiled copy https://zhuanlan.zhihu.com/p/1930389542784964333
02	写给大家看的 CuTe 教程：tiled mma https://zhuanlan.zhihu.com/p/1937145378446226159
03	写给大家看的 CuTe 教程：Layout Compose & Inverse https://zhuanlan.zhihu.com/p/1962625273636845008
04	写给大家看的 CuTe 教程: Layout Product & Divide https://zhuanlan.zhihu.com/p/1971945267294111573
05	写给大家看的 CuTe 教程：TMA Copy https://zhuanlan.zhihu.com/p/2003198909405763007
06	写给进阶开发的 CuTe 笔记：tiled mma 的 permutationMNK 参数 https://zhuanlan.zhihu.com/p/1973526710105419953
07	写给大家看的 CuTe 教程：async pipeline https://zhuanlan.zhihu.com/p/2024832248386528275

reed「NVidia GPU 指令集架构」系列（8 篇）

章节	原文
01	NVidia GPU 指令集架构-前言 https://zhuanlan.zhihu.com/p/686198447
02	NVidia GPU 指令集架构-寄存器 https://zhuanlan.zhihu.com/p/688616037
03	NVidia GPU 指令集架构-Load 和 Cache https://zhuanlan.zhihu.com/p/692445145
04	NVidia GPU 指令集架构-浮点运算 https://zhuanlan.zhihu.com/p/695667044
05	NVidia GPU 指令集架构-整数运算 https://zhuanlan.zhihu.com/p/700921948
06	NVidia GPU 指令集架构-比特和逻辑操作 https://zhuanlan.zhihu.com/p/712356884
07	NVidia GPU 指令集架构-Warp 级和 Uniform 操作 https://zhuanlan.zhihu.com/p/712357647
08	NVidia GPU 指令集架构-程序控制和原子操作 https://zhuanlan.zhihu.com/p/712357443

GPU 架构 · 内存 · Attention 精度（4 篇）

章节	原文
01	GPU 内存模型之 Proxy（reed） https://zhuanlan.zhihu.com/p/2080760925481592736
02	Blackwell 双 die 拓扑：SM 与地址的 die 映射及跨 die 访存代价（reed） https://zhuanlan.zhihu.com/p/2051639750541546085
03	FP8 Attention 中的精度优化：逆序计算与缩放因子选择（reed） https://zhuanlan.zhihu.com/p/2038747362596737621
04	CUDA shared memory 避免 bank conflict 的 swizzling 机制解析（frankshi） https://zhuanlan.zhihu.com/p/4746910252

目录

整理说明 **2**

第一部分　reed 系列（CUDA 高性能编程） **5**

第一章　cute 之 Layout **6**

1.1 一维向量的表示 . . . 6
1.2 二维矩阵的表示 . . . 7
1.3 有层次的 Layout（Hierarchy Layout） . . . 8
1.4 常量 Shape（编译时 Shape） . . . 12
1.5 变量 Shape（运行时 Shape） . . . 12
1.6 总结 . . . 12
1.7 参考 . . . 12

第二章　cute Layout 的代数和几何解释 **13**

2.1 基本属性 . . . 13
2.2 坐标（coordinate） . . . 14
2.3 切片（slice） . . . 14
2.4 补集（complement） . . . 16
2.5 乘法（product） . . . 16
2.6 除法（divide） . . . 17
2.7 复合函数和逆（composition & inverse） . . . 19
2.8 总结 . . . 19
2.9 参考 . . . 19

第三章　cute 之 Tensor **21**

3.1 Tensor 的生成 . . . 21
3.2 Tensor 维度信息查询 . . . 21
3.3 Tensor 的访问 operator()/operator[] . . . 22
3.4 Tensor 的 Slice . . . 22
3.5 Tensor 的 Take . . . 22
3.6 Tensor 的 flatten . . . 22
3.7 Tensor 的层级合并 coalesce . . . 22
3.8 Tensor 的主轴层级化 group_modes . . . 23
3.9 Tensor 的划分 logical_divide/tiled_divide/zipped_divide . . . 23
3.10 Layout 的乘积 logical/zipped/tiled/blocked/raked . . . 23
3.11 Tensor 的局部化切块 local_tile . . . 23
3.12 Tensor 的局部数据提取 local_partition . . . 23
3.13 Tensor 数据类型转换 recast . . . 25
3.14 Tensor 内容的填充 fill 和清除 clear . . . 25
3.15 Tensor 的线性组合 axpby . . . 25
3.16 Tensor 的打印 print . . . 25
3.17 特殊矩阵 . . . 25
3.18 使用 Tensor 实现 Vector Add 示例： . . . 25
3.19 总结与讨论 . . . 27

第四章 cute 之 MMA 抽象 **28**
4.1 NVidia Tensor Core 简介 . . . 28
4.2 cute 的 MMA 核心数据结构及其相互关系 . . . 29
4.3 MMAOperation . . . 30
4.4 MMA_Traits . . . 31
4.5 TiledMMA . . . 31
4.6 ThrMMA . . . 32
4.7 cute::gemm . . . 32
4.8 总结 . . . 33
4.9 参考 . . . 33

第五章 cute 之 Copy 抽象 **34**
5.1 CUDA 的存储层次和数据加载路径 . . . 34
5.2 高效的 ldmatrix 指令 . . . 35
5.3 cute Copy 抽象及其相互关系 . . . 36
5.4 CopyOperation . . . 36
5.5 Copy_Traits . . . 38
5.6 Copy_Atom . . . 38
5.7 TiledCopy . . . 38
5.8 ThrCopy . . . 39
5.9 cute::copy . . . 39
5.10 总结 . . . 40
5.11 参考 . . . 40

第六章 cute 之简单 GEMM 实现 **41**
6.1 BLAS 约定和深度学习约定 . . . 41
6.2 NVidia cuBLAS 类库 . . . 42
6.3 Tensor 表示 . . . 42
6.4 以 C 矩阵为中心的任务划分策略 . . . 44
6.5 TiledMMA：主机端选择指令，设备端将分块划分到线程 . . . 45
6.6 Loop Over K . . . 48
6.7 实验性能对比 . . . 49
6.8 总结 . . . 49
6.9 参考 . . . 49

第七章 cute 之 Swizzle **50**
7.1 局部性原理和 Shared Memory . . . 50
7.2 共享内存读取（ldmatrix 指令） . . . 52
7.3 Shared Memory 写入 . . . 53
7.4 异或运算的封闭性和双射性 . . . 53
7.5 Swizzle 抽象 . . . 54
7.6 Thread Block Swizzle . . . 55
7.7 总结 . . . 55
7.8 参考 . . . 55

第八章 cute 之 GEMM 流水线 **57**
8.1 RISC 硬件流水线 . . . 57
8.2 GEMM 软件流水线（Tile 间） . . . 58
8.3 Tile 内的流水线 . . . 59
8.4 异步拷贝和 MultiStage 流水线 . . . 60

8.5 总结 . . . 61
8.6 参考 . . . 61

第九章 cute 之高效 GEMM 实现 62
9.1 计算高效 . . . 62
9.2 访存高效 . . . 64
9.3 算法高效 . . . 65
9.4 尾阶段高效（Epilogue） . . . 67
9.5 试验设置和结果 . . . 67
9.6 总结和讨论 . . . 69
9.7 启发式算法 . . . 69
9.8 参数相容性 . . . 69
9.9 其它方面 . . . 69
9.10 总结 . . . 70
9.11 参考 . . . 70

第十章 cute 之 Hopper 前言 71
10.1 背景与动机 . . . 71
10.2 系列内容规划 . . . 71

第十一章 cute 之 Hopper MBarrier 72
11.1 计算和数据搬运的桥梁 . . . 72
11.2 MBarrier . . . 73
11.3 使用示例 . . . 77
11.4 总结 . . . 77
11.5 参考 . . . 77

第十二章 cute 之 Hopper TMA 78
12.1 SIMT 编程模型中的数据搬运 . . . 78
12.2 高效的异步 Tensor 内存拷贝加速单元 . . . 80
12.2.1 TMA 描述符（TMA Descriptor） . . . 80
12.2.2 发起拷贝任务 . . . 82
12.2.3 完成机制 . . . 83
12.2.4 Swizzle 机制 . . . 83
12.2.5 Async 单元和可见性保障 . . . 85
12.2.6 其他方面 . . . 86
12.3 cute 封装 . . . 86
12.4 总结 . . . 87
12.5 参考 . . . 87

第十三章 cute 之 TMA Descriptor 编码与隐藏的第 21bit 88
13.1 TMA Descriptor 的完整二进制布局 . . . 88
13.2 整体结构 . . . 88
13.3 Control Word 的 bitfield 布局 . . . 89
13.4 开源参考实现 . . . 89
13.5 bit21 的设置规则 . . . 90
13.6 bit21 的硬件语义：实验验证 . . . 90
13.7 实验设计 . . . 90
13.8 实验结果 . . . 90
13.9 硬件行为模型 . . . 91

13.10 当性能优化遇到动态 Shape . . . 92
13.11 规避方法 . . . 92
13.12 方法一：创建模板时强制清零 bit21 . . . 92
13.13 方法二：在 `tensormap.replace` 修改 shape 时联动处理 bit21 . . . 92
13.14 结论 . . . 93
13.15 参考 . . . 93

第二部分　竹熙佳处系列（CuTe 教程） 94

第十四章　写给大家看的 CuTe 教程：tiled copy 95
14.1 更新 . . . 95
14.2 动机 . . . 95
14.3 Tiled copy 是为了解决什么问题? . . . 96
14.4 CuTe Tiled copy 的设计哲学 . . . 96
14.4.1 Cuda 如何实现 tiled copy . . . 96
14.4.2 CuTe 中如何实现 tiled copy . . . 98
14.4.3 CuTe make_tiled_copy 解读 . . . 99
14.4.4 CuTe partition 解读 . . . 102
14.4.5 进阶：CuTe Copy_Atom 解读 . . . 102
14.5 总结 . . . 104

第十五章　写给大家看的 CuTe 教程：tiled mma 105
15.1 前序知识 . . . 105
15.2 动机：从 cuda 视角理解的 tiled mma . . . 105
15.3 CuTe Tiled mma 的使用 . . . 107
15.4 Tiled mma 的构造 . . . 108
15.5 Tiled mma partition . . . 111
15.6 Tiled mma gemm 执行 . . . 111
15.7 Tiled mma 延伸：s2r / r2s Tiled copy . . . 112
15.8 进阶：CuTe Tiled mma 的实现细节 . . . 112
15.9 总结 . . . 114

第十六章　写给大家看的 CuTe 教程：Layout Compose & Inverse 115
16.1 动机 . . . 115
16.2 Layout 的定义 & 作用 . . . 115
16.3 Layout 的复合 compose . . . 116
16.4 Layout 的逆 inverse . . . 118
16.5 Layout compose & Inverse 实战 . . . 120
16.6 结论 . . . 122

第十七章　写给大家看的 CuTe 教程: Layout Product & Divide 123
17.1 动机 . . . 123
17.2 Layout 的乘法 Product . . . 124
17.3 Product 用法案例：tile_to_shape . . . 126
17.4 Layout 的除法 Divide . . . 127
17.5 Divide 用法案例：local_partition & local_tile . . . 131
17.6 总结 . . . 132
17.7 Reference . . . 132

第十八章 写给大家看的 CuTe 教程：TMA Copy **134**
18.1 动机 . . . 134
18.2 用法 . . . 135
18.3 CuTe 中的坐标表示: arithTuple . . . 137
18.4 MBarrier . . . 139
18.5 make_tma_copy 梳理 . . . 140
18.6 TMA proxy & fence . . . 142
18.7 总结 . . . 143

第十九章 写给进阶开发的 CuTe 笔记：tiled mma 的 permutationMNK 参数 **144**
19.1 前言 . . . 144
19.2 动机 . . . 144
19.3 如何使用 PermMNK . . . 144
19.4 总结 . . . 147
19.5 Reference . . . 147

第二十章 写给大家看的 CuTe 教程：async pipeline **148**
20.1 动机 . . . 148
20.2 基本用法 . . . 149
20.3 AsyncPipeline：mbarrier 的封装 . . . 149
20.4 PipelineState：流水线的状态机 . . . 151
20.5 总结 . . . 153

第三部分 NVidia GPU 指令集架构（reed） **154**

第二十一章 NVidia GPU 指令集架构-前言 **155**

第二十二章 NVidia GPU 指令集架构-寄存器 **158**
22.1 Load-Store 架构 . . . 158
22.2 寄存器表达的程序状态机 . . . 159
22.3 通用寄存器 . . . 160
22.4 特殊寄存器 . . . 160
22.5 Predicate 寄存器 . . . 160
22.6 Uniform 寄存器 . . . 161
22.7 总结 . . . 162
22.8 参考 . . . 162

第二十三章 NVidia GPU 指令集架构-Load 和 Cache **163**
23.1 NVidia GPU 的计算单元和存储层级 . . . 163
23.2 GPU 中的 Cache 和 Shared Memory 机构 . . . 164
23.3 数据 Load 指令 . . . 165
23.4 广播语义的常量 Cache . . . 165
23.5 寄存器 reuse 和 Prefetch . . . 166
23.6 总结 . . . 166
23.7 参考 . . . 166

第二十四章 NVidia GPU 指令集架构-浮点运算 **167**
24.1 浮点数的表示 . . . 167
24.1.1 浮点数基础 . . . 167
24.1.2 浮点数的圆整 . . . 169

24.1.3 浮点数特殊值 . . . 169
24.1.4 浮点数的 denormal . . . 169
24.2 NVidia GPU 浮点运算指令 . . . 169
24.2.1 加法和乘法 . . . 169
24.2.2 乘加 FMA（Fused Multiply Add） . . . 170
24.2.3 随路取负和绝对值 . . . 171
24.2.4 低精度浮点指令 . . . 171
24.2.5 类型转换 . . . 171
24.2.6 超越函数 . . . 172
24.2.7 除法 . . . 172
24.3 总结 . . . 173
24.4 参考 . . . 173

第二十五章 NVidia GPU 指令集架构-整数运算 174
25.1 常用整数类型及其表示 . . . 174
25.2 NVidia GPU 对不同整数类型的支持 . . . 174
25.3 有符号整数的补码表示和算术位移 . . . 175
25.4 NVidia GPU 整数指令集 . . . 176
25.4.1 整数加法 . . . 176
25.4.2 整数乘法 . . . 176
25.4.3 整数位移 . . . 177
25.4.4 整数除法和取余 . . . 177
25.4.5 整数转浮点数 . . . 177
25.4.6 整数转 Packed 整数 . . . 177
25.4.7 整数绝对值 . . . 178
25.4.8 整数点积 . . . 178
25.4.9 整数最大最小 . . . 178
25.4.10 整数比较产生条件 . . . 178
25.4.11 Tensor Core 整数指令 . . . 178
25.5 总结 . . . 178
25.6 参考 . . . 179

第二十六章 NVidia GPU 指令集架构-比特和逻辑操作 180
26.1 无类型的寄存器表示 . . . 180
26.2 POPC 指令 . . . 180
26.3 FLO 指令 . . . 181
26.4 BREV 指令 . . . 181
26.5 SGXT 指令 . . . 181
26.6 BMSK 指令 . . . 182
26.7 PRMT 指令 . . . 182
26.8 LOP3 指令 . . . 183
26.9 总结 . . . 185
26.10 参考 . . . 185

第二十七章 NVidia GPU 指令集架构-Warp 级和 Uniform 操作 186
27.1 SIMT 体系下的 Tensor 计算 . . . 186
27.2 Warp Level 指令 . . . 188
27.2.1 DMMA 指令 . . . 188
27.2.2 HMMA 指令 . . . 190
27.2.3 IMMA 指令 . . . 190

27.2.4 SHFL 等 . . . 190
27.3 Uniform 指令 . . . 191
27.4 总结 . . . 192
27.5 参考 . . . 192

第二十八章 NVidia GPU 指令集架构-程序控制和原子操作 193
28.1 高级编程语言和底层指令 . . . 193
28.2 指令层的控制流 . . . 194
28.3 原子操作 . . . 195
28.4 总结 . . . 196
28.5 参考 . . . 197

第四部分 GPU 架构 · 内存 · Attention 精度 198

第二十九章 GPU 内存模型之 Proxy 199
29.1 C++ 内存模型 . . . 199
29.2 契约、实现与数据竞争 . . . 199
29.3 happens-before . . . 199
29.4 acquire / release / relaxed . . . 200
29.5 fence . . . 201
29.6 Proxy 的硬件基础 . . . 201
29.7 Coherence 与 consistency . . . 201
29.8 Proxy 与访存方法 . . . 201
29.9 四个基本构件 . . . 202
29.10 Proxy 与线程的正交 . . . 202
29.11 GPU 上的同步原语与可见性 . . . 203
29.12 三类原语的分工 . . . 203
29.13 Scope 作为同步条件 . . . 204
29.14 Fence 家族 . . . 204
29.15 mbarrier 的 sem / scope 规则 . . . 205
29.16 完成状态与内存可见性 . . . 205
29.17 TMA 完成语义 . . . 205
29.18 典型数据边 . . . 206
29.19 TMA 写入与 tcgen05.mma 消费 . . . 206
29.20 完成信号与发起粒度 . . . 207
29.21 TMA 写入与 generic 读取 . . . 207
29.22 跨 CTA 的 mbarrier 交接 . . . 208
29.23 异步响应缓冲区复用 . . . 209
29.24 CLC 官方示例中的同步关系 . . . 209
29.25 同步判定 . . . 210
29.26 常见错误 . . . 211
29.27 总结 . . . 211
29.28 参考 . . . 211

第三十章 Blackwell 双 die 拓扑：SM 与地址的 die 映射及跨 die 访存代价 213
30.1 一、Blackwell 双 die 的拓扑与硬件特点 . . . 213
30.2 二、测量原语：一次干净的冷 HBM 访问延迟 . . . 214
30.3 三、SM 到 die：干净的 74/74 划分 . . . 214
30.4 四、地址到 die：一个三层确定性结构 . . . 216

30.5 五、跨卡：结构普适，标号逐卡 . . . 219
30.6 六、die 亲和放置与只读带宽 . . . 219
30.7 七、工程可用性与局限 . . . 221
30.8 总结 . . . 221
30.9 参考 . . . 221

第三十一章 FP8 Attention 中的精度优化：逆序计算与缩放因子选择 223
31.1 Attention Sink 与 P 的精度退化 . . . 223
31.1.1 FP8 E4M3 的精度结构 . . . 223
31.1.2 Attention Sink 与 P 塌陷 . . . 224
31.2 逆序计算策略 . . . 224
31.3 缩放因子 256 的选取 . . . 225
31.3.1 二次幂缩放使乘除运算 bit-exact . . . 225
31.3.2 dp(S) 锯齿波在二次幂中锁定 S = 256 . . . 225
31.3.3 实现层补充：精度优势就是工程优势的全部 . . . 226
31.4 主流实现对比与实验验证 . . . 226
31.4.1 主流 FP8 Attention 实现的设计选择 . . . 226
31.4.2 实验：两项优化的有效性验证 . . . 227
31.5 总结 . . . 228
31.6 参考 . . . 228

第三十二章 CUDA shared memory 避免 bank conflict 的 swizzling 机制解析 229
32.1 背景 . . . 229
32.2 矩阵转置基于 shared memory 的 naive 实现 . . . 229
32.3 使用 Padding . . . 231
32.4 swizzling 机制 . . . 232
32.4.1 4.1 逻辑位置和物理位置 . . . 232
32.4.2 4.2 swizzling 中的映射函数 . . . 234
32.4.3 4.3 一对一映射 . . . 234
32.4.4 4.4 映射前后的取值范围保持不变 . . . 235
32.4.5 4.5 列内任意两行在映射后的 x 值不同 . . . 235
32.4.6 4.6 swizzling 机制的代码应用实践 . . . 235
32.5 TMA 中的 swizzling 机制 . . . 237
32.6 Cutlass 中的 swizzling 使用 . . . 238
32.7 参考资料 . . . 238

第一部分

reed 系列（CUDA 高性能编程）

第一章　cute 之 Layout

作者：reed（知乎 @reed-84-49，专栏「CUDA 高性能编程」）　原文：https://zhuanlan.zhihu.com/p/661182311

计算机中的内存是一维的线性地址空间，而数学计算问题所要处理的空间经常是高维的。如 GEMM（General Matrix Multiplication）问题的数学计算体系是二维计算空间，Deep Learning 计算体系是三维以上的计算空间 (batch, height, width, channel, etc.)。如何高效的表达高维计算空间，如何高效便捷的将计算所要求的高维空间映射到一维空间变得越来越重要。

历史上对该问题的探究可以分为三个阶段：

- 第一阶段 BLAS 的 row/col-major + leading dimension 描述阶段；
- 第二阶段 Tensor 的 shape + stride 阶段；
- 第三阶段为 Hierarchy Tensor 阶段。

二十世纪七八十年代开发的 BLAS（Basic Linear Algebra Subprograms）库中描述矩阵时面对的问题多为二维问题，其通过引入行优先、列优先的概念来刻画一维存储结构和二维逻辑结构的映射关系。为了做某个维度的地址对齐、padding 等，引入了 leading dimension 的概念，其可以描述第二个维度上数据的连续性关系。二十一世纪初 Deep Learning 在模式识别等领域暂露头角，各种框架应运而生，在 Deep Learning 计算中为了提升计算密度 batch 化成了重要手段，这时高维数据的描述成为必须，相关人员在 blas 中 leading dimension 基础上进行了扩展，引入了 shape 和 stride 描述体系。shape stride 描述体系可以很好的刻画一维存储结构和高维逻辑结构的映射关系，并且使得某些操作只需要改变 shape stride 描述无需对数据实体进行移动，在 stride 的帮助下通过简单的 coordinate 和 stride 点积既可以完成逻辑地址和一维索引的映射。shape stride 描述可以表达简单的高维数据，但是其对于具体的维度的轴必须是单调的，在数据的连续性方面要求比较严格。2023 年层级的 Tensor（Hierarchy Tensor）描述被提出，其在 shape stride 描述的基础上添加了具体的轴的层级描述（Graphene Tensor IR）[1]，使得数据在某个具体的轴上可以有更为丰富的表达。对于现代 GPU 计算体系而言，Tensor 的分块和复杂映射都可以通过该层级体系（描述 + 代数）来表达和推导，其构建了复杂逻辑空间和硬件排列映射的一条途径。

Tensor 的层次化描述体系在 Graphene Tensor IR 中进行了详细的描述；在具体实现方面，NVidia 开源的 cutlass 模板中也利用同样的思路进行 Tensor 描述和计算。为了更好的抽象和利用编译时优化，cutlass 利用了 C++17 实现了 cute，它定义了层次化 Tensor 体系和其之上的代数计算，然后基于该 cute 描述体系实现了最新 hopper 架构之上的矩阵运算。Tensor 是数据的表达，其表达一个相对独立且有结构的数据体，而 Tensor 内的数据排布则由 Layout 来表达。本文将关注这种映射结构，即 Layout。本文在行文组织结构上：首先以示例的形式介绍一维向量的表示、二维矩阵的表示，然后介绍层次化的 Tensor 表示，再后介绍 cute 中常用的编译时和运行时形状描述，最后本文总结 Layout 的功能。

简单地讲，引入有层次的描述（Layout）代数来表达计算空间和一维地址空间的映射问题。Layout 是一个数据排列的描述体系，其可以实现将逻辑坐标映射到索引坐标（offset 表示）。Layout 包含 Shape 和 Stride 两部分。其中 Shape 描述排列的分块层次和结构。Stride 描述块内或块间的数据排列连续性。Shape 和 Stride 都是层级的嵌套表示。也就是说 Shape 可以包含 Int 数据和 Shape。Shape 和 Stride 需要有相同的层次关系。

在介绍有层次的 Layout 之前，我们先回顾无层次的 Layout 描述，也就是 shape 和 stride 描述的高维 tensor。

1.1　一维向量的表示

Shape: (8), Stride: (1) 表示该排列包含 8 个逻辑位置，在逻辑位置和物理（数据）做映射的时候每一个元素之间的差为 1，如图 1 所示其描述了 0-7 总计 8 个数字，其计算逻辑为 index_physical = index_logical * stride;

Figure 1. shape = 8, stride = 1 的逻辑空间和物理空间映射关系

Shape: (8), Stride: (2) 表示该排列包含 8 个逻辑位置，每一个位置的坐标为自然序列 0-7，如图 2 所示，其对应物理位置映射时的公式为 index_physical = index_logical * stride; 这时候我们发现逻辑空间和物理空间的大小是不一样的，在 cute 体系下，逻辑空间可以被称作 domain，而代表存储的物理空间称作 codomain，也就是 size(tensor) = 8, cosize(tensor) = 15.

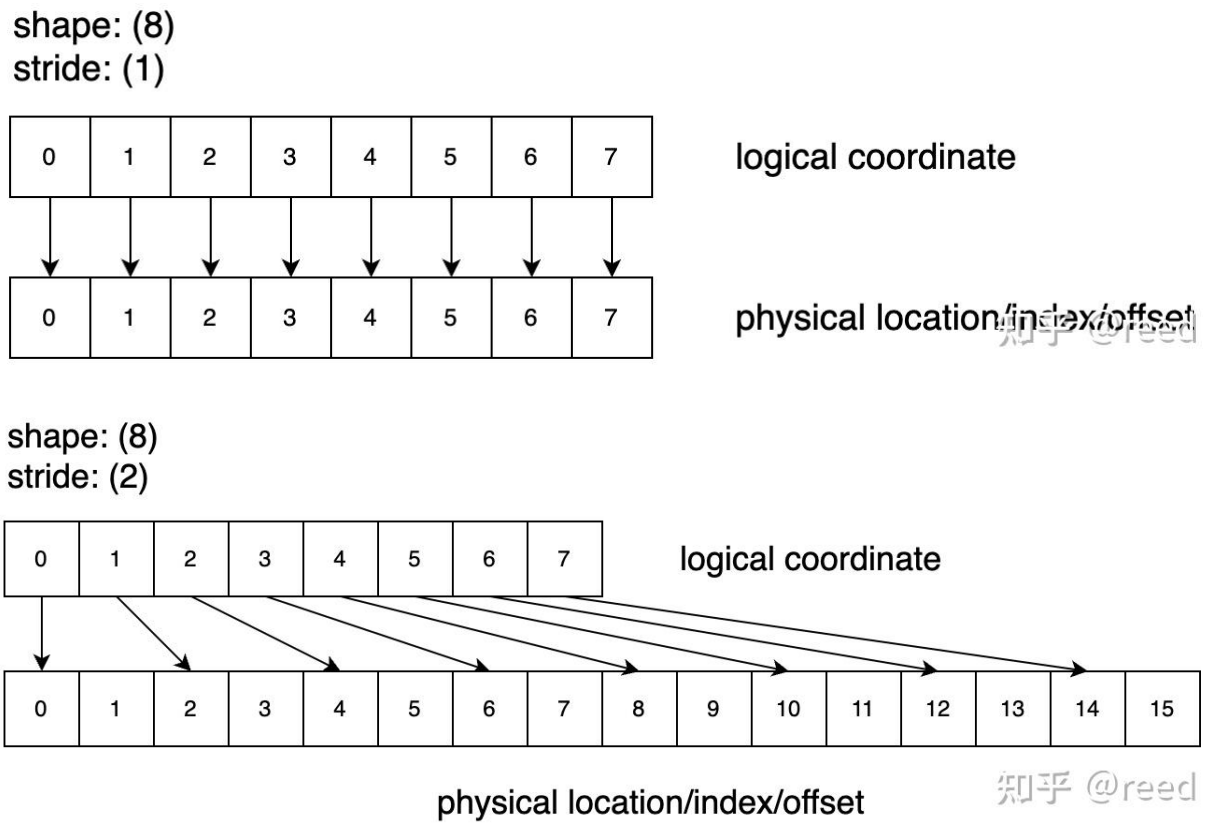


Figure 2. shape = 8，stride = 2 的逻辑空间和物理空间的映射关系

Shape: (8), Stride: (0) 表示逻辑上我们需要的 8 个数据都来自于同一个存储位置 0，如图 3 所示，所有的元素都指向同一个物理位置，通过图示也可以看到该 layout 的 cosize 为 1。

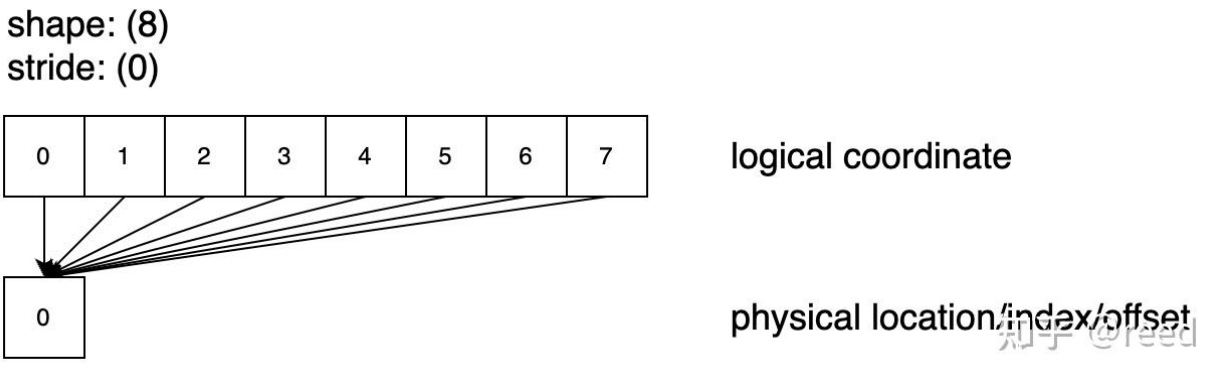


Figure 3. shape = 8, stride =0 的逻辑空间和物理空间的映射关系

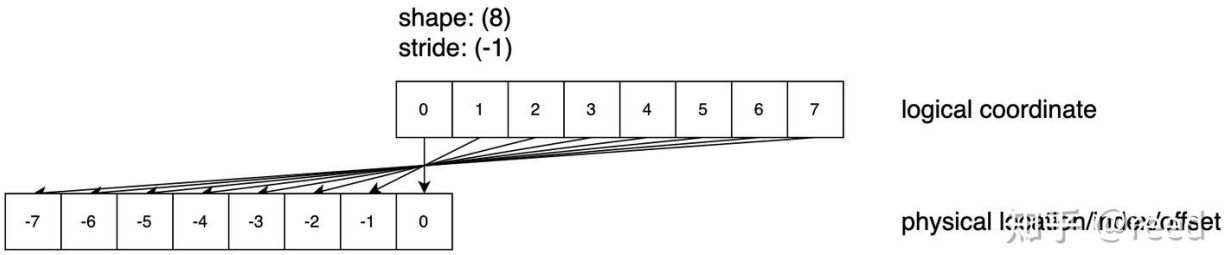
Shape: (8), Stride: (-1) 通过 Stride 设置为-1 可以实现对数据的反向访问，即顺序是 reverse 的，这种 case 很少用到。

Figure 4. shape = 8, stride = -1 的逻辑空间和物理空间的映射关系

以上一维空间的示例，呈现了 Tensor 在其 shape 不变的情况下，通过 stride 的改变可以描述 tensor 中的各个元素在物理空间中的位置。我们在使用 Tensor 时候，关注的是其逻辑的大小，而通过 stride 则将这个逻辑空间和实际存储的物理空间进行了关联。并且在计算层面，我们可以看到其始终满足 index = coordinate * stride；

1.2 二维矩阵的表示

Shape: (3, 4), Stride: (1, 3)，和一维向量比较类似，此次的二维空间指的是 Tensor 的逻辑空间，其存储结构依然可以是一维的，二维空间的列优先描述可以表达为 shape(3, 4), stride(1, 3), 如图 5，shape 中的 3，4 分别表示矩阵的行数和列数，stride 中的 1，3 分别表示元素沿着行增加 1 则物理存储相对的加 1，而其中的 3 则表示如果元素沿着列增加



1 则其在存储空间的位置需要增加 3。

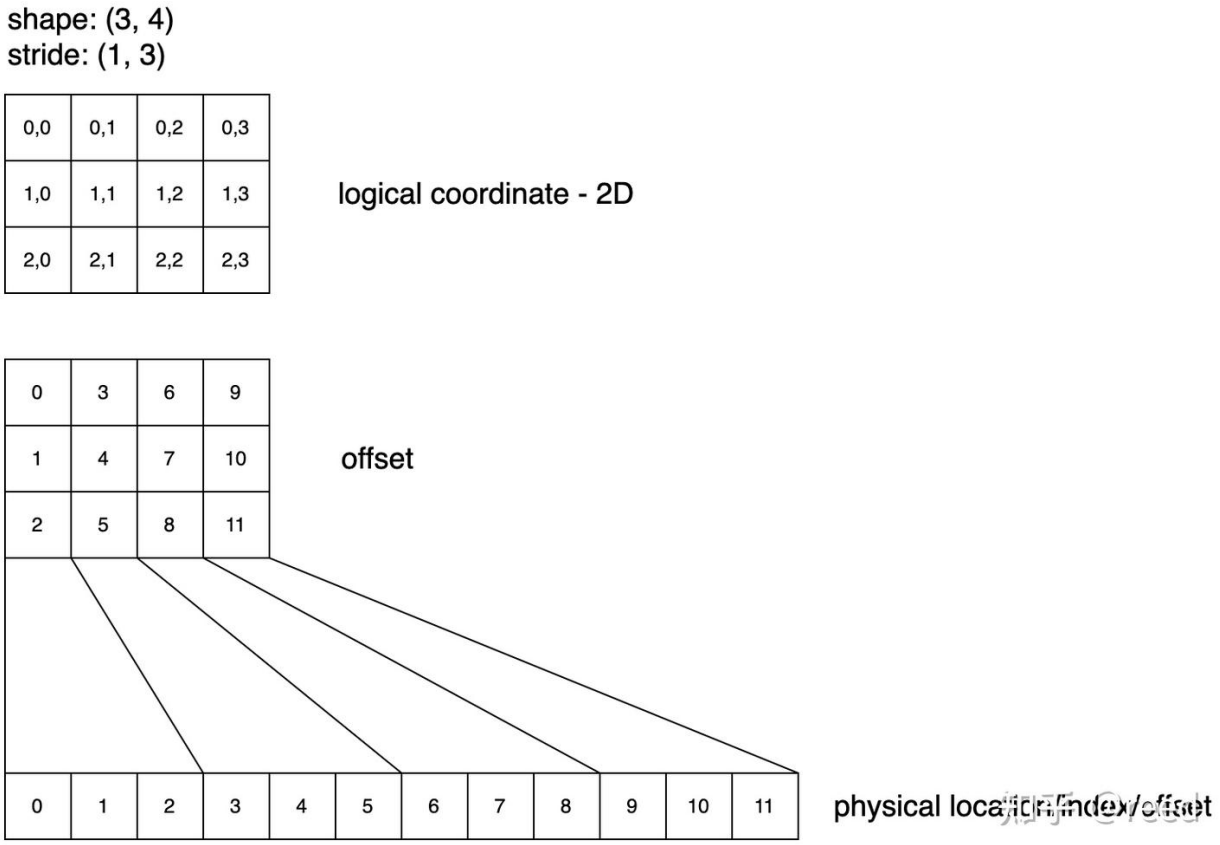


Figure 5. shape = (3,4) stride = (1, 3) 的逻辑空间和物理空间的映射关系

Shape: (3, 4), Stride: (4, 1)，和上面类似，Shape 不变，而 stride 由 (1，3) 变为 (4, 1) 则存储结构变为行优先，即在物理存储数据的时候先存储每一行，行内元素存储的优先级高。

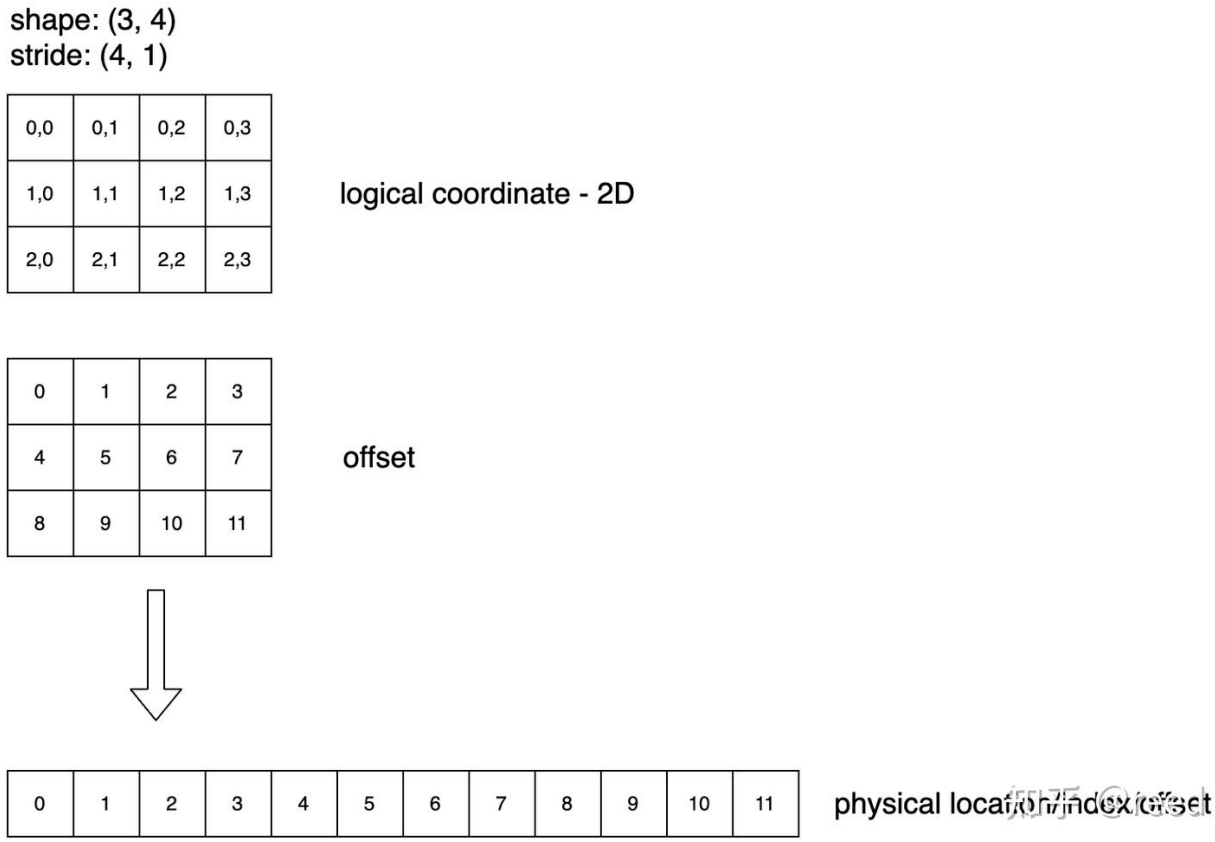
Figure 6. shape = (3, 4) stride = (4, 1) 的逻辑空间和物理空间的映射关系

二维矩阵的描述和一维类似，shape 表示其逻辑形状，stride 表示具体的某个元素和物理空间的映射时的间隔量。逻辑空间到物理空间的映射通过点积来完成。从二维空间我们可以很容易的扩展到高维空间，如深度学习常用的（N，H，W，C），其映射关系依然利用点积公式：

$$index_{physical} = coordinate * stride = \sum_i coordinate_i \cdot stride_i$$

1.3 有层次的 Layout（Hierarchy Layout）

以上介绍的一维向量和二维矩阵描述描述被深度学习框架所广泛采用，在 torch 中我们可以访问 tensor 的 shape 属性和 stride 方法（注意方法调用需要使用带括号形式，即 stride()）来获取对应的信息。我们不难发现，上面的 shape 和 stride 描述其限制了 Tensor 的每一个轴只能有一个 stride 值，也就是说，整个 tensor 在某一个维度上的连续性关系是不能变的，更形象地描述则为：Tensor 不可以分块。我们将这样轴的连续性不可变更，体现为 Tensor 不可以分块的描述称为单调 Tensor 描述。而当我们处理复杂的 Tensor 计算问题时，尤其是如 NVidia 硬件引入的指令计算时，这种表示



是不充分的。由此则引入了有层次的 Tensor 描述，即 Hierarchy Layout。简单地理解，有层次的 Tensor（或者 Layout）就是以原有的单调 Tensor 所描述的小块作为基础单元，将其组成 Tensor。这样原有的小块是 Tensor，小块作为单元的外部组织也是 Tensor，实现了 Tensor 套 Tensor，这就是所谓的有层级的 Tensor 了。而其坐标到实际物理位置的映射关系则就是其 Layout，Tensor 有了层次，Layout 也就有层次了。

Figure 7. 有层次的逻辑空间和其 offset 计算（引用自 ASPLOS23 Graphene IR）

如图 7-a/b，它们表示了传统的列优先和行优先的 Layout，可以认为它们是一层的 Tensor；而图 7-c/d 用之前的 shape 和 stride 描述没有办法表示如此复杂的情况（存在不单调的轴）。此时，我们可以将图 7-c 图看作两个层级的 Tensor，如图 8 所示，其中内层 Tensor 为红框所标注的部分，外层 Tensor 为以红框作为元素的外层 Tensor，内层 Tensor 的 shape 和 stride 描述可以表示为 shape: (4, 2), stride: (2, 1)，外层 Tensor 的 Layout（以元素为单位）可以表示为 shape: (1, 4), stride: (0, 8). 两层 Tensor 合并后，表示整体表示为 4 行，8 列，其中列方向为两个层次，即 8=2x4 列，表示为 (2, 4) 列，其中 2 表示内层的 Tensor 维度为 2，4 表示列方向上外层 Tensor 有 4 个元素。即层级的 shape 表示为 (4, (2, 4)) 第一个 4 表示有 4 行，第二个 4 表示外层 Tensor 重复 4 列，2 表示内层 Tensor 为 2 列。我们清楚了 shape 中的各个元素的含义后，stride 的形式要与性状保持一致，即 stride 要满足 (x, (y, z)) 的形式，根据 x 位置对应的含义，表示内层（红框内矩阵）矩阵的行方向的间隔，则有 x = 2；y 为内层红框内列方向的间隔，有 y = 1；z 则表示红框和红框之间横向的间隔，可以用第二个红框的左上角元素 8 和第一个红框对应的左上角元素 0 相减得到，即 z = 8 - 0 = 8, 这样我们得到有层次的 Tensor 的 shape 和 stride 描述：shape: (4, (2, 4)), stride: (2, (1, 8))。

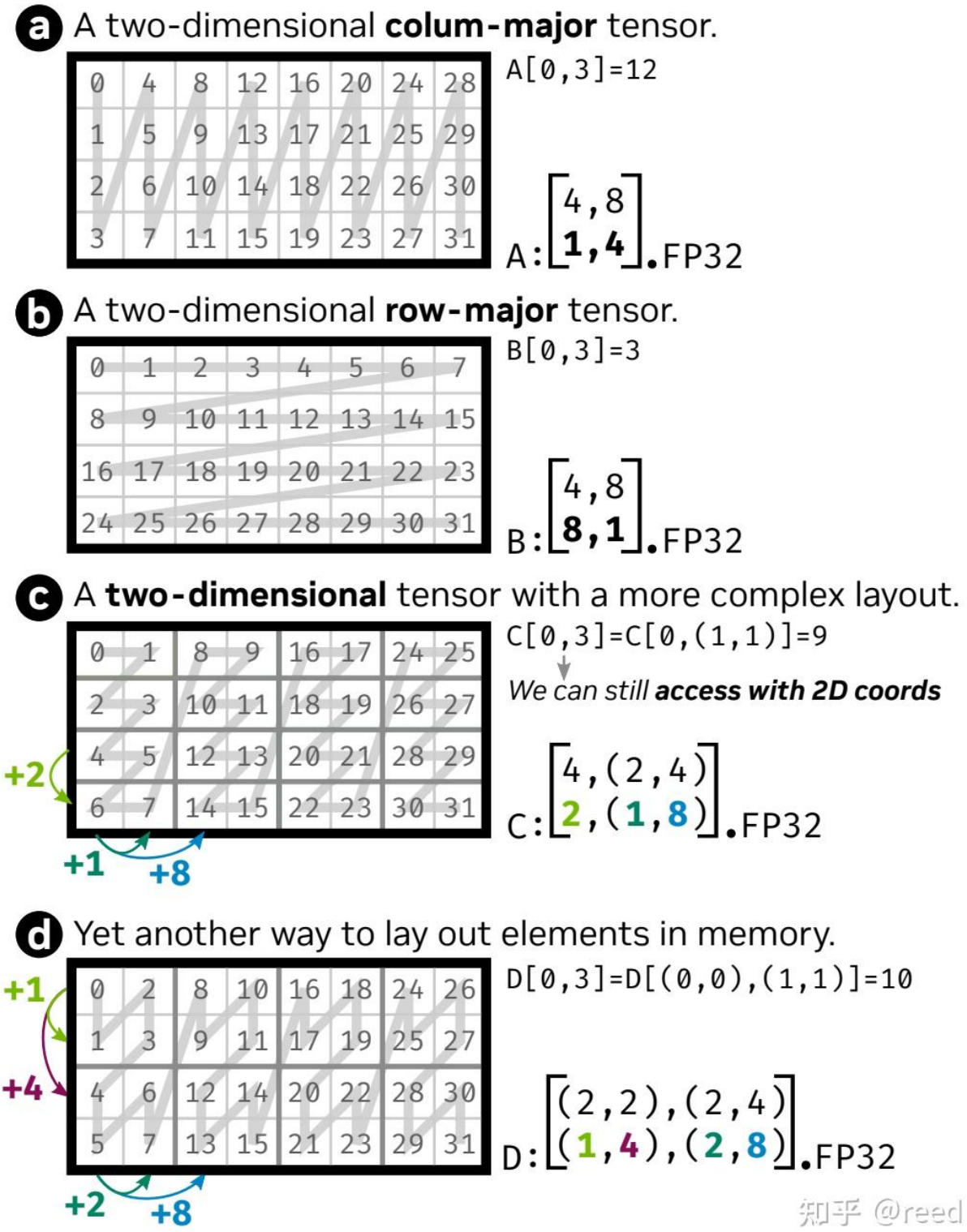
Figure 8. 图 7-c Tensor 的层次分解

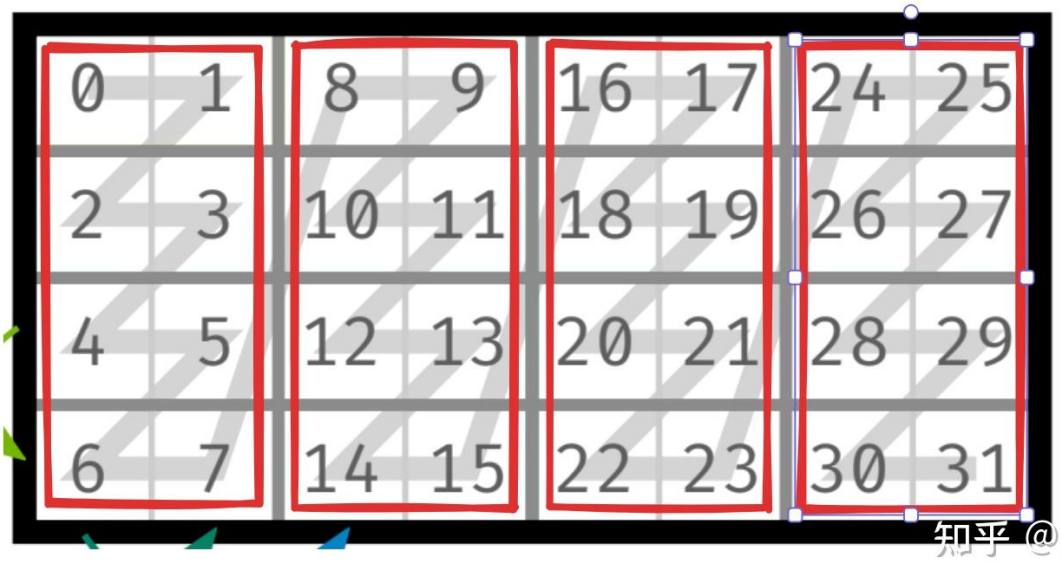
图 7-d 的层次分解和上面类似，其也是两层的结构（如图 9）：内层的 Tensor 为红框所标注，shape: (2, 2), stride: (1, 2)；外层的 Tensor 为绿线所标注，shape: (2, 4), stride: (1, 2)；合并后的层次化表示 shape: ((2, 2), (2, 4))，其中的数字 2，2，2，4 分别表示内层 Tensor 行数，外层 Tensor 行数，内层 Tensor 列数，外层 Tensor 列数。同样的根据以上数据所表示的轴的信息，可以得到 stride: ((1, 4), (2, 8))。

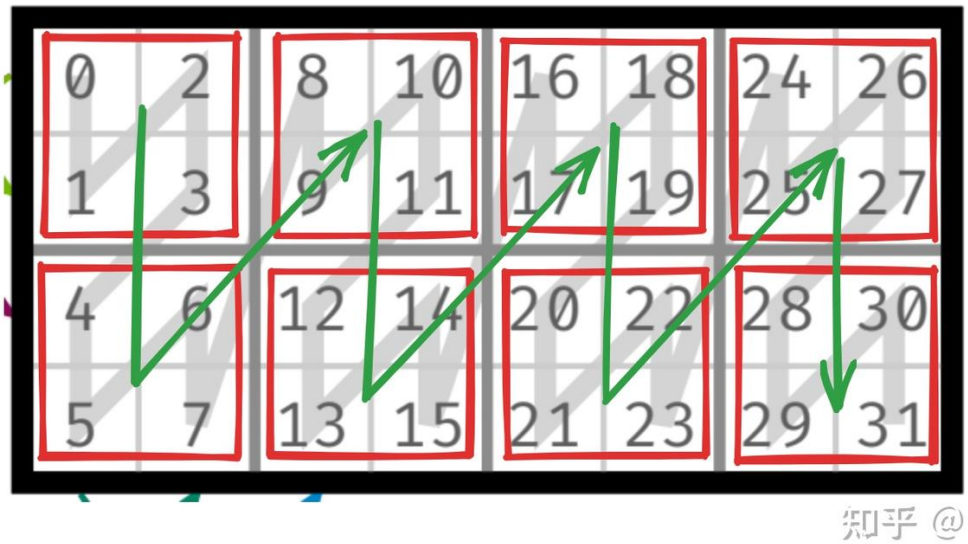
Figure 9. 图 7-d Tensor 的层次分解

shape 和 stride 给出了 Tensor 的逻辑空间描述和映射到物理空间时的间隔描述，当根据逻辑空间的位置获得物理空间位置时，依然和传统的 Tensor 计算规则一样，采用 coordinate 和 stride 点积即可，只不过坐标也采用 hierarchy 表达即可。

通过以上两个有层次 Tensor 的 shape 和 stride 的分解和组合，我们可以完成有层次的 Tensor 的组合，其突破了传统的 Tensor 轴只能有一个 stride 的限制，可以表达更丰富的 Tensor（有层次的 Tensor）。


a A two-dimensional colum-major tensor.
A[0,3]=12
A:[4,8 / 1,4].FP32
b A two-dimensional row-major tensor.
B[0,3]=3
B:[4,8 / 8,1].FP32
c A two-dimensional tensor with a more complex layout.
C[0,3]=C[0,(1,1)]=9
We can still access with 2D coords
C:[4,(2,4) / 2,(1,8)].FP32
+2
+1
+8
d Yet another way to lay out elements in memory.
D[0,3]=D[(0,0),(1,1)]=10
D:[(2,2),(2,4) / (1,4),(2,8)].FP32
+1
+4
+2
+8
知乎 @reed


0 1 8 9 16 17 24 25
2 3 10 11 18 19 26 27
4 5 12 13 20 21 28 29
6 7 14 15 22 23 30 31
知乎 @reed


0 2 8 10 16 18 24 26
1 3 9 11 17 19 25 27
4 6 12 14 20 22 28 30
5 7 13 15 21 23 29 31
知乎 @reed

以上介绍了有层级的 Layout 描述，具体的在 cute 实现时，其提供了 make_shape 和 make_stride 接口，并且 make_shape 的参数可以接受 make_shape 的结果，以此来完成有层次的 shape 和 stride 构建，为了提升效率，shape 和 stride 可以区分为常量 shape 和变量 shape。

1.4 常量 Shape（编译时 Shape）

常量 Shape，即编译时 Shape，可以通过在编译时完成坐标的映射或者推导，而减少运行时的计算量。对于需要编译时确定的量，需要用编译时常量来实现，如矩阵分块的大小其依赖了寄存器空间申请则必须使用这种类型 shape，其书写形式为 Int<K>{}，其中 Int 表示编译时常量，K 表示具体的数值，表示以前面的类型来构造一个对象（常量对象），实例如下：

```
auto shape = make_shape(Int<2>{}, Int<3>{});
auto shape1 = make_shape(shape, Int<3>{});
```

1.5 变量 Shape（运行时 Shape）

有些维度信息是运行时决定的，如 vector 长度等，而且不需要编译时决策，则采用运行时计算模式，值得注意的是示例代码中的 2，3 虽然是常数，但是在 cute 的约定里，该形式表示变量。

```
auto shape = make_shape(2, 3);
auto shape = make_shape(m, n);
```

1.6 总结

Layout 的本质是函数，其可以实现由一种坐标系统变换到一个表示偏移量的标量，超参为 Tensor 的逻辑 shape 和 stride，即

$$offset = Layout^{shape}_{stride}(coordinate)$$

1.7 参考

- https://dl.acm.org/doi/abs/10.1145/3582016.3582018
- https://github.com/NVIDIA/cutlass/tree/main/media/docs/cute

第二章　cute Layout 的代数和几何解释

作者：reed（知乎 @reed-84-49，专栏「CUDA 高性能编程」）　原文：https://zhuanlan.zhihu.com/p/662089556

前面文章“cute 之 Layout[1]”通过回顾 shape 和 stride 描述体系介绍了有层次的 Layout 的基础概念。本文将围绕 Layout 进行更全面的介绍，包含：Layout 的基本属性和 Layout 的运算。这些运算是定义在以 shape 和 stride 为参数之上的代数运算，同时为了更直观和形象的展示这些操作，本文会通过几何的形式呈现这些运算。几何是设计和思考模型，代数是实现和计算形式。

2.1　基本属性

如图 1，展示了一个有层次的 Layout，可以看到它的组成分为两个层级，其中小的 Tensor 采用同样的颜色表示，图中的左上角 0 到 5 展示了一个内层的 Tensor，然后沿着纵向和横向重复该 Tensor，其中行方向重复 4 次，列方向重复 5 次。得到的 Hierarchy Tensor 的 shape: ((2, 4), (3, 5))，同样我们可以得到其 stride: ((3, 6), (1, 24))。Layout 的本质是函数其数学描述可以参考

Definition 1.3 Let $f : X \to Y$ be a function.

- The set X is called the **domain** of f.
- The set Y is called the **codomain** of f.

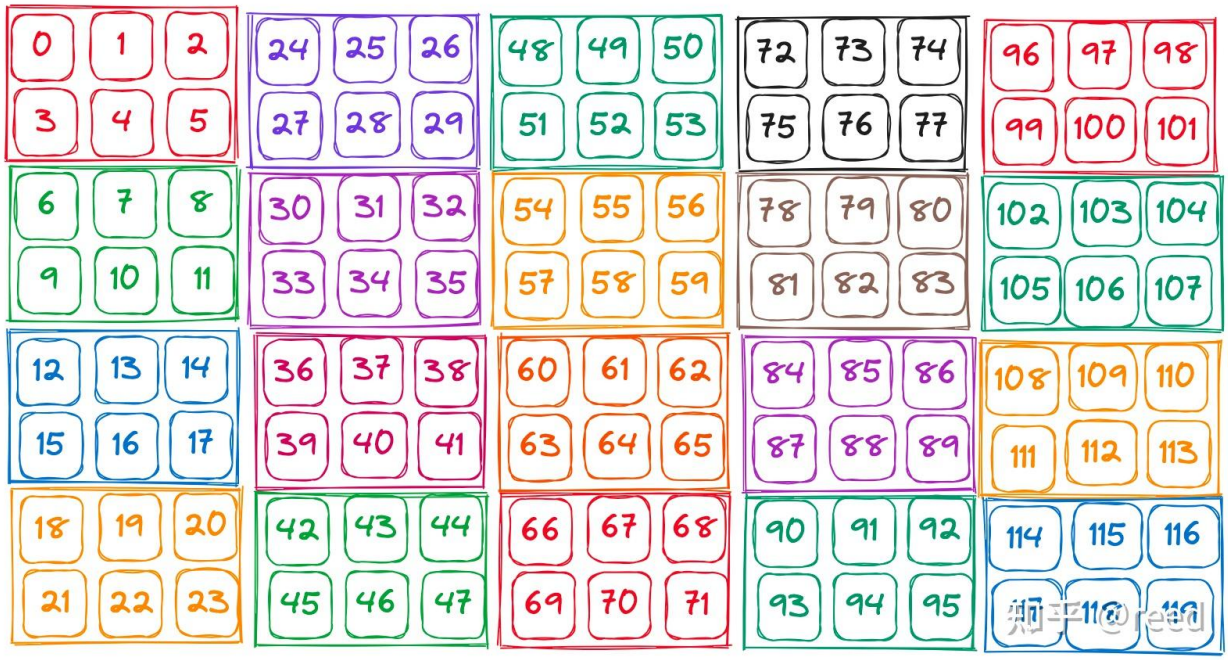


图 2.1: Figure 1. shape 为 ((2, 4), (3, 5))，stride 为 ((3, 6), (1, 24)) 的 Layout 示意图

针对图 1 的 Layout，有如下基本属性，列举为如下表格

shape	stride	size	rank	depth	coshape	cosize
((2,4),(3,5))	((3,6),(1,24))	120	2	2	120	120

其中 shape，stride 表示 Layout 的逻辑形状和每一个维度上在地址空间上步长；size 表示该逻辑空间的大小其数值上等于 shape 维度的积即 $size = \prod_{i=0}^{nshape-1} shape_i$ ，rank 表示该 Layout 的秩，其数值上等于 shape 中第一层的元素

[1] https://zhuanlan.zhihu.com/p/661182311

的个数（括号表示一个维度合并），图示的维度为 2，其中（2，4）贡献一个维度，（3，5）贡献一个维度；depth 表示嵌套的深度，常规的非嵌套 Tensor 定义为 1，则此处图示的 Tensor 的深度为 2；coshape，cosize 表示 codomain 的空间大小和占用的空间大小，此处为 120，如果 stride 存在不紧凑的情形，则 cosize 可能会大于 domain 域的 size。

2.2 坐标（coordinate）

Layout 是有层次的，那么访问这种有层次的 Layout 所需的坐标自然也是有层次的。对于只有一层的 Tensor 的访问，可以通过指定行坐标和列坐标来实现，如 `auto coord1 = make_coord(0, 1);`其表示构建一个一层的坐标，其中行坐标为 0，列坐标为 1。如需构建有层次的坐标，则可以通过将 make_coord 的结果进行嵌套，形如如下格式

```
auto row_coord = make_coord(1, 3);
auto col_coord = make_coord(2, 4);
auto coord = make_coord(row_coord, col_coord);
```

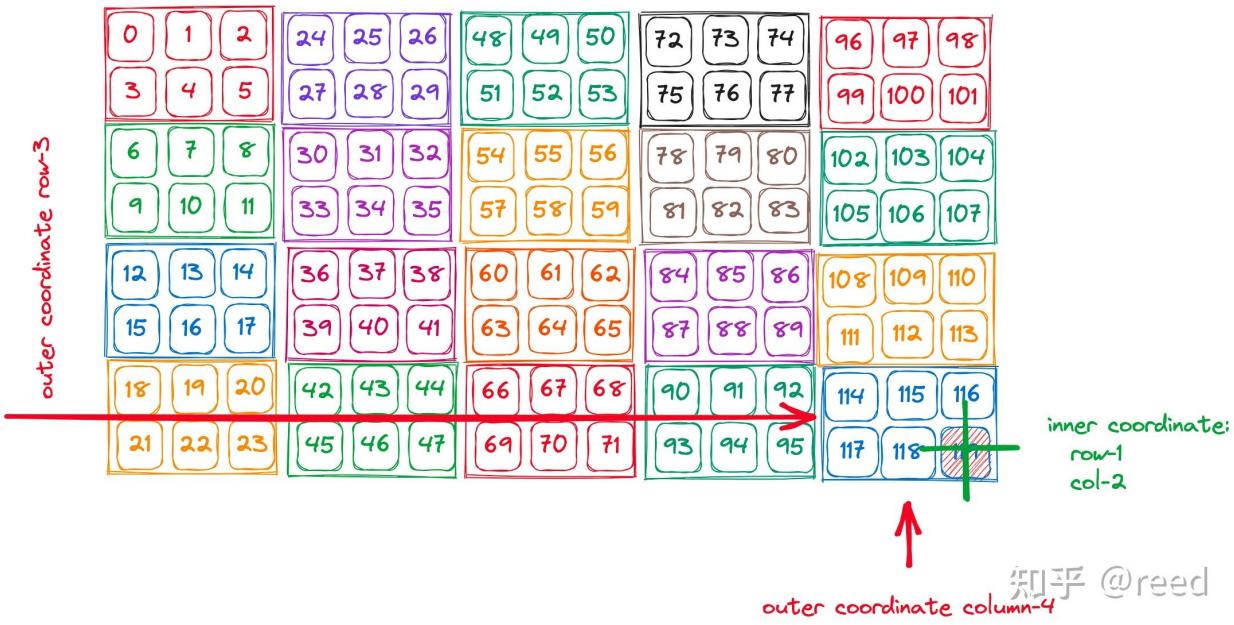


图 2.2: Figure 2. coord: ((1, 3), (2, 4)) 所表示的位置和访问层次

其实现了对 Layout(实际为 Tensor) 的有层次访问，其中的 row_coord 中的 1，3 分别表示在内层和外层 Tensor 的行方向的坐标，col_coord 中的 2，4 表示在内层和外层 Tensor 的列方向的坐标，该坐标可以简记为 `coord: ((1, 3), (2, 4))`。如图 2，其表示了对该坐标的访问。

除了具体的坐标值，cute 提供了用于对某个维度全选的 Underscore 类型和对应的变量_；该变量类似于 python 或 fortran 语言中的冒号 (:)，可以用来表示某个维度的全部选定，该功能在下一小节的 Layout 切片中被大量使用。

2.3 切片（slice）

通过提供坐标可以访问到某一个具体坐标上的数值，很多时候我们除了需要访问单个位置之外，还需要只固定一些坐标位置，而其他位置进行全选，来完成 Tensor 的切片操作 (slice)。这种需求可以通过之前坐标章节提到的 Underscore 类型来完成。具体地，我们可以 slice 第一列数据，或者一行数据，或者其中的第一层 Tensor。如图 3 其展示了不同的 slice 的效果，函数调用形式如代码片段所示：

```
auto layout_out = slice(coord, layout_in);
```

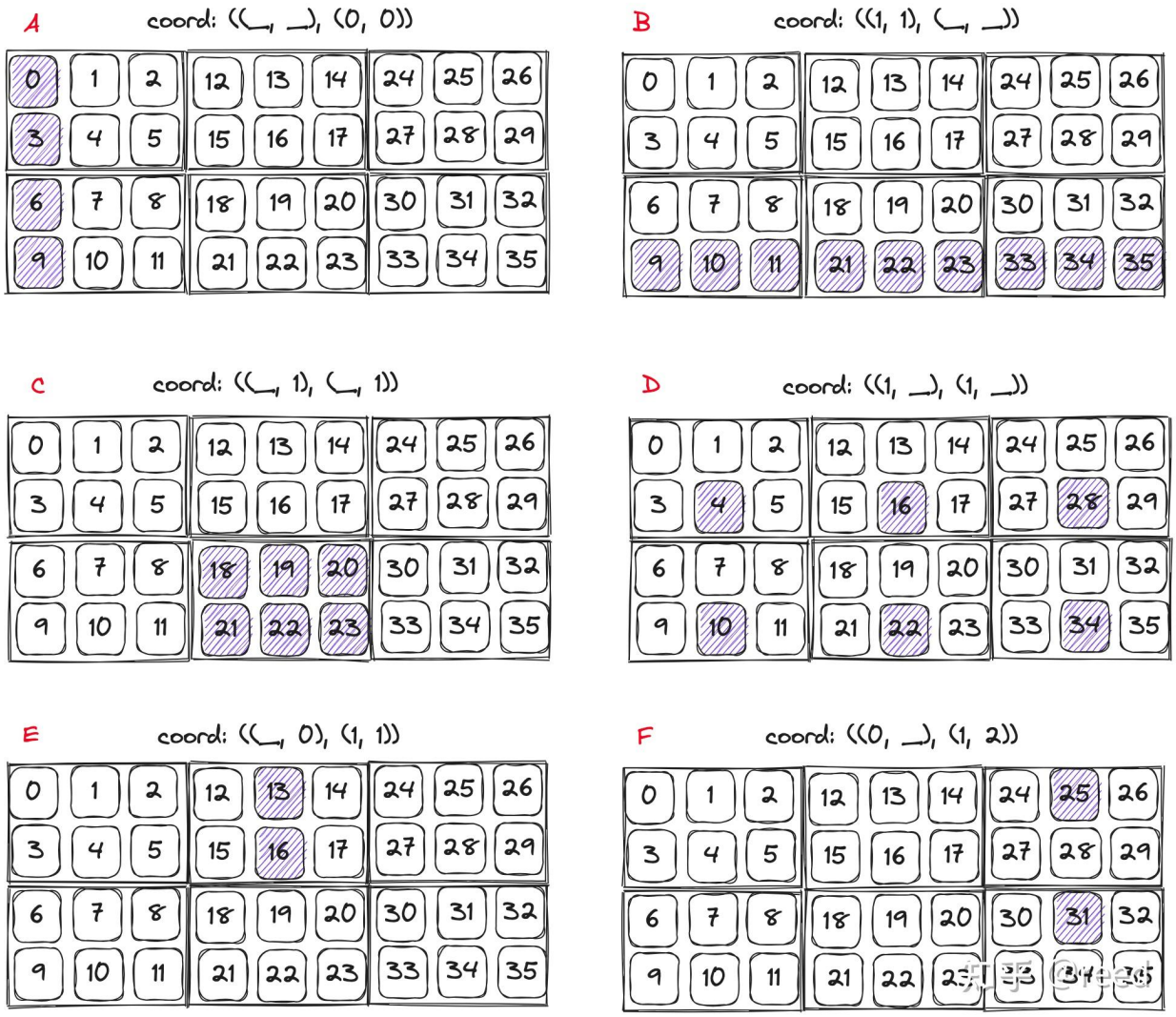


图 2.3: Figure 3. Layout 的切片运算示意图

2.4 补集（complement）

Layout 的本质是函数，函数的本质是集合，Layout 定义了从 domain 到 codomain 的投影，当 codomain 存在不连续时，则存在空洞的位置，如图 4 所示，这时候我们可以构造一个 Layout2 能够填充上 codomain 的空洞位置，此时我们构造的 Layout 则为原 Layout 的补集，同时为了表示的简洁性，补集会被压缩为最小表示，周期性重复的部分会被约掉。

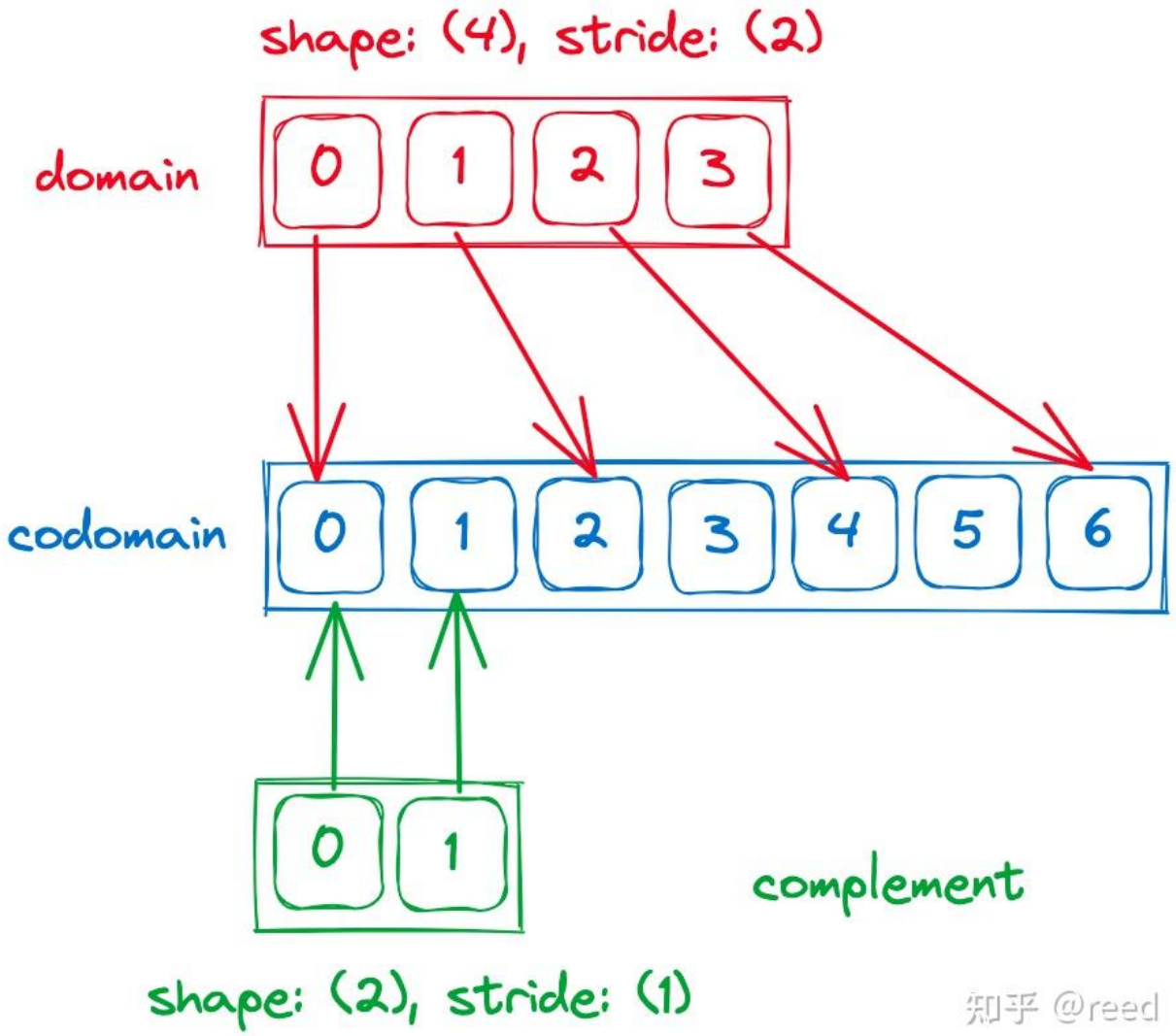


图 2.4: Figure 4. Layout 的补集

2.5 乘法（product）

实数域乘法的语义是将某个量重复若干次。Layout 乘法计算中同样遵循该语义：重复某个 Tensor 若干次。由于 Tensor 的表示是有高维数据，所以其上的乘法在实现上也有多个，但本质不变。cute 中定义的乘法包含如下 5 个

```
logical_product, tiled_product, zipped_product, blocked_product, raked_product
```

我们有两个 Layout 对其进行相乘，其中第一个 shape:(x, y)，第二个 shape: (z, w)，则其乘积的 shape: (x, y, z, w)。同时我们约定各种乘法的层级顺序，如 shape: ((x,y), (z, w))，本质上而言 x, y, z, w 的顺序和层级关系并不重要，核心是我们对乘积进行访问的时候需要按照乘约定的顺序即可。只是在现实实现中我们要遵循一种约定，方便我们后期的使用，于是我们约定了各个乘法的顺序和层级如下

乘法模式	乘积的 shape
logical	((x, y), (z, w))
zipped	((x, y), (z, w))
tiled	((x, y), z, w)
blocked	((x, z), (y, w))
raked	((z, x), (w, y))

如图 5 所示，Layout 乘法在集合上可以认为是按照图示的形式进行的，在 Layout x Layout 的计算中，首先将 Layout1 按照 Layout2 的顺序进行重复，原来 Layout2 中的位置被 Layout1 所占据，然后将其中的内层数据 Layout1 按照列优先的顺序排成一列，外层 Layout 按照列优先排列后表示为最终矩阵的列。

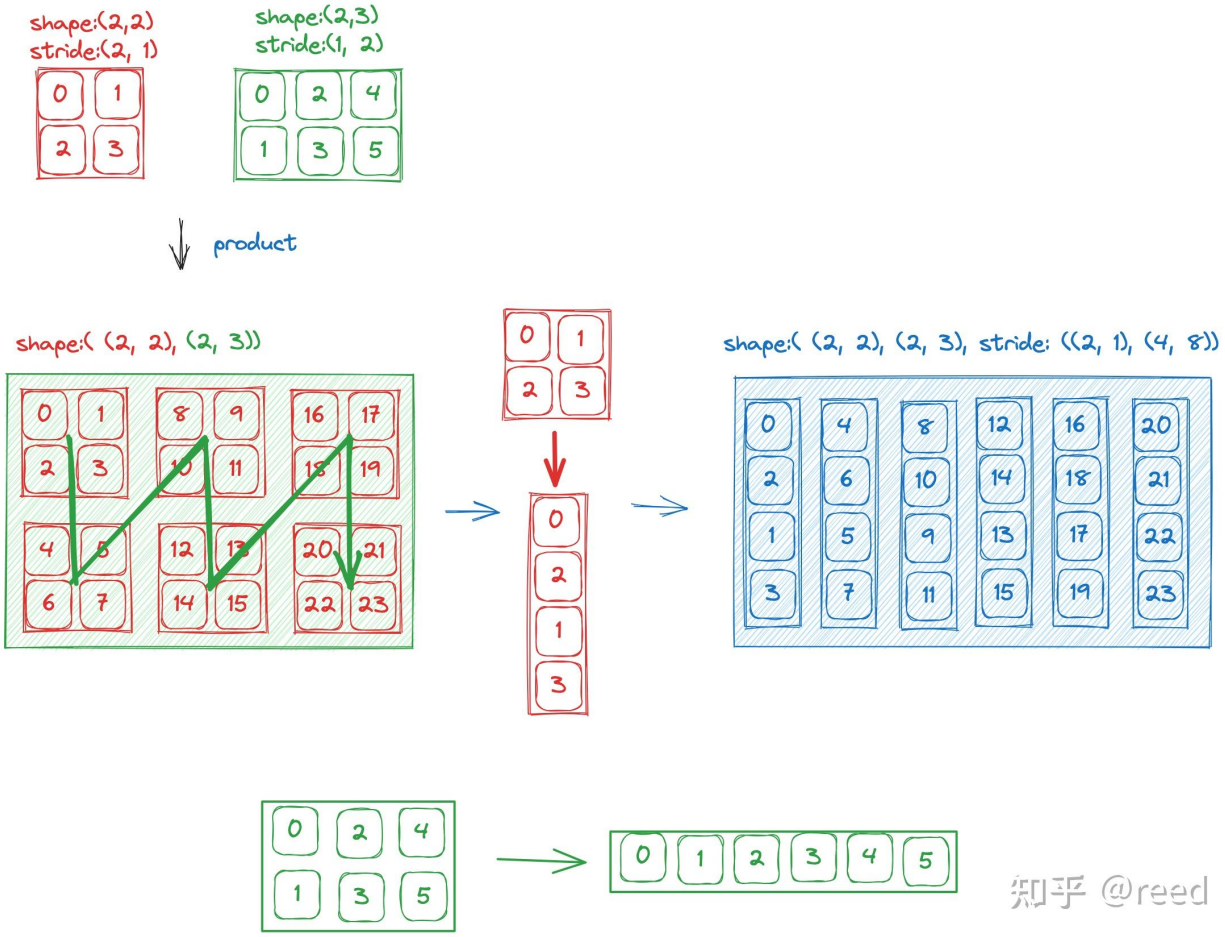


图 2.5: Figure 5. Layout 乘法的几何解释

2.6　除法（divide）

除法是乘法逆运算，实数域上的除法表示被除数能被除数分多少份。如 $10 \div 5 = 2$，Layout 上的除法也有类似的逻辑，但 Layout 除法和实数域不同的是 Layout 除法的结果是一种划分层次，但不是被划分的结果。如果用实数域的公式来表达则为 $10 \div 5 = (5, 2)$。其中作为结果的括号中的 5 表示被分解的块的大小，2 表示可以被分解多少次。如图 6 所示，其展示了 Layout 的除法，首先按照列优先将被除数 Layout 转化为一维表示，同时按照列优先将除数 Layout 展开为一维表示，结果中第一维度为除数 Layout 的大小，根据其中的 Layout 数值从被除数中取值作为输出的结果。如（0，6，12，18）序列按照（0，2，1，3）的顺序进行取值后得到（0，12，6，18）序列，重复后续位置，并且表示为列行模式，则得到图中右侧表示。其 shape 表示中 rank-0 表示分块时每一个块的大小，rank-1 表示能被分割多少块。

在 cute 中，提供了三个 Layout 除法函数，分别为 logical、tiled、zipped。它们的整体逻辑是一致的，只是在最后的维度（rank）表示上有略微差别。

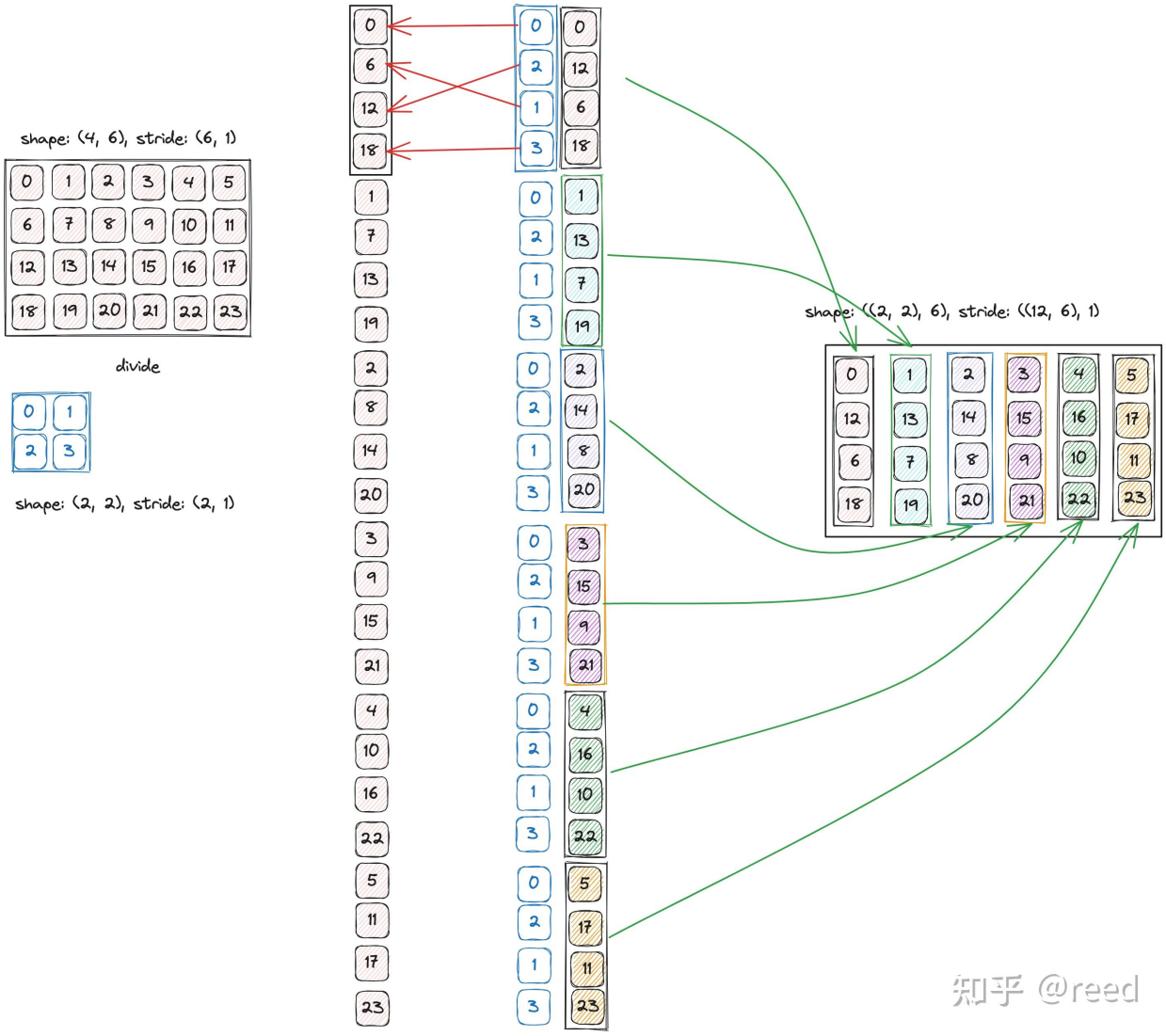


图 2.6: Figure 6. Layout 除法的几何解释

2.7 复合函数和逆（composition & inverse）

Layout 的本质是函数，Layout 的复合即函数复合，此处我们引用复合定义如下，

> **Definition 1.5** Let $f : X \to Y$ and $g : Y \to Z$ be functions. The **composition** of g and f, written $g \circ f$ is the function $g \circ f : X \to Z$ such that $(g \circ f)(x) = g(f(x))$.

同时，函数域上我们引用单位函数的定义如下

> One important function from a set X to itself is the **identity function** id_X, which does nothing: it is defined by $\mathrm{id}_X(x) = x$ for all $x \in X$.

在有了复合函数和单位函数的定义之后，我们引入左逆和右逆的定义如下

> **Definition 1.6** Let $f : X \to Y$ and $g : Y \to X$ be functions.
>
> - We say that g is a **left inverse** to f, and that f is a **right inverse** to g, if $g \circ f = \mathrm{id}_X$.
> - We say that f is **invertible** if there is a function $h : Y \to X$ such that $f \circ h = \mathrm{id}_Y$ and $h \circ f = \mathrm{id}_X$.

几何化的 right_inverse 表示如图 7 所示，首先将 Layout 按照列优先对 coord 和对应的 value 进行排列，采用 two line notation 表示形式。根据右逆的计算逻辑，要找到一个排列能够使得其满足输入 value 得到单位排列（此处刚好为 coord），则按照 value 进行排列得到的 coord 即为上面要求的排列，如图中橙色排列所示，其 coord 就是我们需要的 right inverse，将其转换为 shape 和 stride 形式如图左下角结果。

2.8 总结

我们介绍了 Layout 的基本属性和常用代数方法，并通过几何图形的形式对 Layout 常用的运算进行了相应的解释，通过该文章，我们可以对 Layout 有基本的认识，并且能用 Layout 计算完简单的 Layout 变换。更详细的差异和结果可以直接通过 cute 进行实验来得到。

2.9 参考

- https://zhuanlan.zhihu.com/p/661182311
- https://www.ucl.ac.uk/~ucahmto/0007_2021/1-2-functions.html

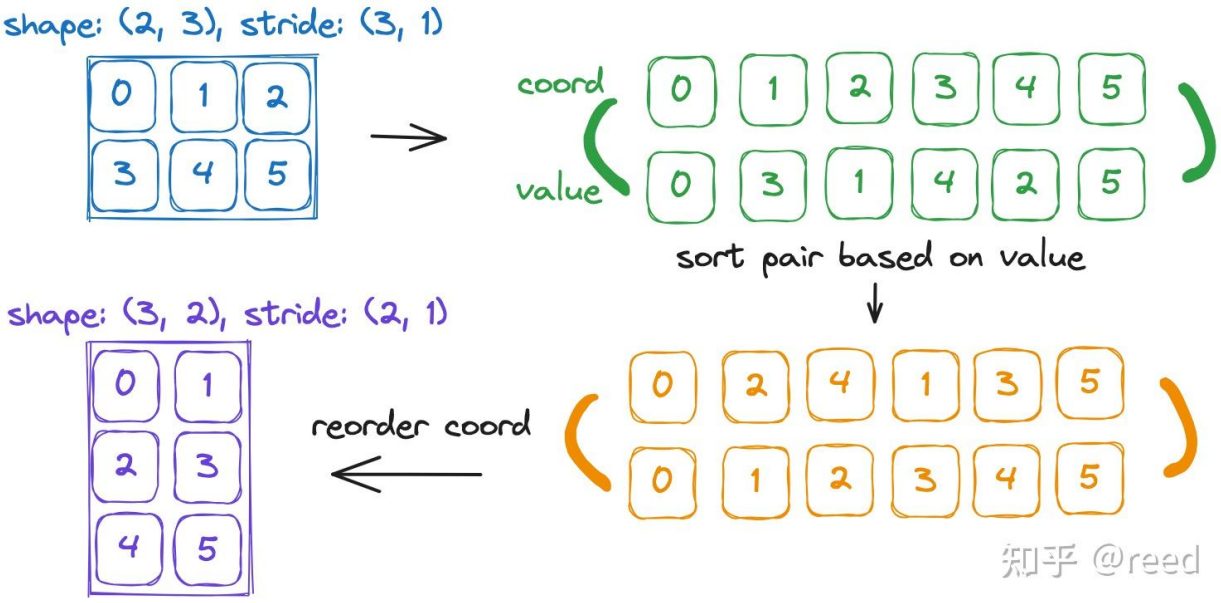


图 2.7: Figure 7. right inverse 的几何解释

第三章　cute 之 Tensor

作者：reed（知乎 @reed-84-49，专栏「CUDA 高性能编程」）　原文：https://zhuanlan.zhihu.com/p/663093816

前面的章节介绍了 cute Layout[1]以及 Layout 的代数和几何解释[2]，Layout 描述了数据的排列和底层存储位置关系，但 Layout 并没有指定存储。Tensor 就是在 Layout 的基础上包含了存储，即 Tensor = Layout + storage，数据存储的具体表现上可以是指针表达的数据或则是栈上数据（GPU 上表现为寄存器）。cute 中的 Tensor 并不同于深度学习框架中的 Tensor（如 Pytorch、Tensorflow），深度学习框架中的 Tensor 更强调数据的表达实体，通过 Tensor 实体与实体之间的计算产生新的 Tensor 实体，即多份数据实体，cute 中的 Tensor 更多的是对 Tensor 进行分解和组合等操作，而这些操作多是对 Layout 的变换（只是逻辑层面的数据组织形式），底层的数据实体一般不变更。也就是说深度学习框架中的 Tensor 是通过 Tensor 产生新 Tensor，cute 中是对数据表达形式的变换，底层数据一般不变更，只变更表达的形式，这个表达形式的变更是通过之前文章介绍的 Layout 上的运算实现的。之所以有这些差别是因为深度学习框架中的 Tensor 是用来表达数据实体，cute 中的 Tensor 是偏向描述的实体。

本文介绍了 cute 中 Tensor 的相关操作，通过阅读本文，开发者可以了解 cute 中 Tensor 的常用方法，熟悉 Tensor 方法的语义，为后续通过 Tensor 进行操作和变换提供基础知识。在文章结构方面，本文首先介绍 cute 中 Tensor 的常用函数，然后以一个向量求和的例子展示如何使用 cute Tensor 工具，最后文章对 Tensor 的特性和使用进行了总结。

3.1　Tensor 的生成

```
// 栈上对象：需同时指定类型和Layout，layout必须是静态shape
Tensor make_tensor<T>(Layout layout);

// 堆上对象：需指定pointer和Layout，layout可动可静
Tensor make_tensor(Pointer pointer, Layout layout);

// 栈上对象，tensor的layout必须是静态的
Tensor make_tensor_like(Tensor tensor);

// 栈上对象，tensor的layout必须是静态的
Tensor make_fragment_like(Tensor tensor);
```

通过 make_tensor 可以方便的生成 Tensor，常用的 Tensor 构造方式有两种，一种是栈上对象，如上形式 1；第二种是堆上对象，通过指定 pointer 来实现；线性地址指针 pointer，layout 描述 pointer 指向的数据的 layout，其可以是多 hierarchy 的。其中 pointer 可以通过`make_gmem_ptr, make_smem_ptr`来生成。其中栈上的对象必须是静态的，堆上对象可以是动态也可以是静态，不存在动态的栈上结构，整体如下表格：

	Static	Dynamic
Heap non-owning	make_tensor(ptr, make_shape(Int<M>{}, Int<N>{}));	make_tensor(ptr, make_shape(M, N));
Stack owning	make_tensor(make_shape(Int<M>{}, Int<N>{})); make_tensor_like(tensor);make_fragment_like(tensor);	NA

3.2　Tensor 维度信息查询

全局`size`函数可以获取 tensor 的元素的个数。其重载了比较运算，可以直接和整数类型进行比较。同时 Tensor 也提供了面向对象的方式来获取各个属性信息，其成员函数和全局函数如下：

```
// 成员函数
Tensor::layout();
```

[1] https://zhuanlan.zhihu.com/p/661182311
[2] https://zhuanlan.zhihu.com/p/662089556

```
Tensor::shape();
Tensor::stride();
Tensor::size();

// 全局函数，可以获取完整信息，或者通过<>获取某一个维度
auto cute::layout<>(Tensor tensor);
auto cute::shape<>(Tensor tensor);
auto cute::stride<>(Tensor tensor);
auto cute::size<>(Tensor tensor);
auto cute::rank<>(Tensor tensor); // (1, (2, 3)) => rank 2
auto cute::depth<>(Tensor tensor);
```

3.3 Tensor 的访问 operator()/operator[]

Tensor 可以通过**括号运算符**来实现对数据的访问（可读 + 可写），坐标可以使用一维的，也可以使用有层次的表示形式，具体的如下：

```
Tensor tensor = make_tensor(ptr, layout);
auto coord = make_coord(irow, icol);

tensor(0) = 1;
tensor(1, 2) = 100;
tensor(coord) = 200;
```

也可以通过`data`函数直接获取数据存储空间的地址

```
Tensor::data();
```

3.4 Tensor 的 Slice

可以通过_来筛选（slice）特定轴，和 Layout 的表达形式一致，具体的代码如下：

```
Tensor tensor = make_tensor(ptr, make_shape(M, N, K)); // MxNxK
Tensor tensor1 = tensor(_, _, 3); // MxN,k =3
```

3.5 Tensor 的 Take

通过 take 函数提取 take 出 [B, E) 的轴上的数据

```
Tensor tensor = make_tensor(ptr, make_shape(M, N, K));
Tensor tensor1 = take<0, 1>(tensor);
```

3.6 Tensor 的 flatten

将层次的 layout 展开为一层（不是将 M，N，K 展开为 MNK）

```
Tensor tensor = ...;  // M, N, K
Tensor tensor1 = flatten(tensor);  // M, N, K
```

3.7 Tensor 的层级合并 coalesce

层级中如果存在空间可连续的坐标，即 stride 意义下不存在空隙，则进行合并

```
Tensor tensor = make_tensor(ptr, make_shape(M, N));
Tensor tensor1 = coalesce(tensor);
```

3.8 Tensor 的主轴层级化 group_modes

将 [B,E) 的主轴层级化为一个新的层级

```
Tensor tensor = ...; // 1, 2, 3, 4
Tensor tensor1 = group_modes<B, E>(tensor); // B=1,E=3 => 1, <2, 3>, 4
```

3.9 Tensor 的划分 logical_divide/tiled_divide/zipped_divide

divide 是将 tensor 进行按照 tile 参数的大小进行划分，具体的可以参考前序文章”cute Layout 代数和几何解释” 中的除法的解释：

```
Tensor tensor = ...;
Tensor tensor1 = logical_divide(tensor, tile);
Tensor tensor2 = zipped_divide(tensor, tile);
Tensor tensor3 = tiled_divide(tensor, tile);
```

3.10 Layout 的乘积 logical/zipped/tiled/blocked/raked

Tensor 上没有定义乘法，只有 Layout 上有乘法，其表达了**重复**的语义：

```
Layout  = ..;
Tile tile = ...;
Layout tensor1 = logical_product(layout, tile);
Layout tensor2 = zipped_product(layout, tile);
Layout tensor3 = tiled_product(layout, tile);
Layout tensor4 = blocked_product(layout, tile);
Layout tensor5 = raked_product(layout, tile);
```

3.11 Tensor 的局部化切块 local_tile

该函数是 Tensor 中用户可以使用到的重要的函数，可以通过 tile 方法对 tensor 进行分块，通过 local_tile 可以实现从大的 tensor 中切取 tile 块，并且通过 coord 进行块的选取，以下代码展示了将维度为 MNK 的张量按照2x3x4的小块进行划分，取出其中的第(1, 2, 3)块。

```
Tensor tensor = make_tensor(ptr, make_shape(M, N, K));
Tensor tensor1 = local_tile(tensor, make_shape(2, 3, 4), make_coord(1, 2, 3));
```

如上图 1 所示，A Tensor 表达了 4 行 6 列的行优先的数据，分块的 tile 大小为 2x2, local_tile 会将 A 矩阵按照 tile 的单位进行分块，然后根据坐标选取出 (1, 1) 位置的分块，如图则取出了（1，1）位置的 tensor 得到右下角的结果。

3.12 Tensor 的局部数据提取 local_partition

local partition 和 local tile 类似，先将大的 Tensor 按照 tile 大小进行分块，分块后每一块取出 coordinate 指定的元素形成新的块，代码形式和具体实例参考图 2:

```
Tensor tensor = make_tensor(ptr, ...);
Tensor tensor1 = local_partition(tensor, tile);
```

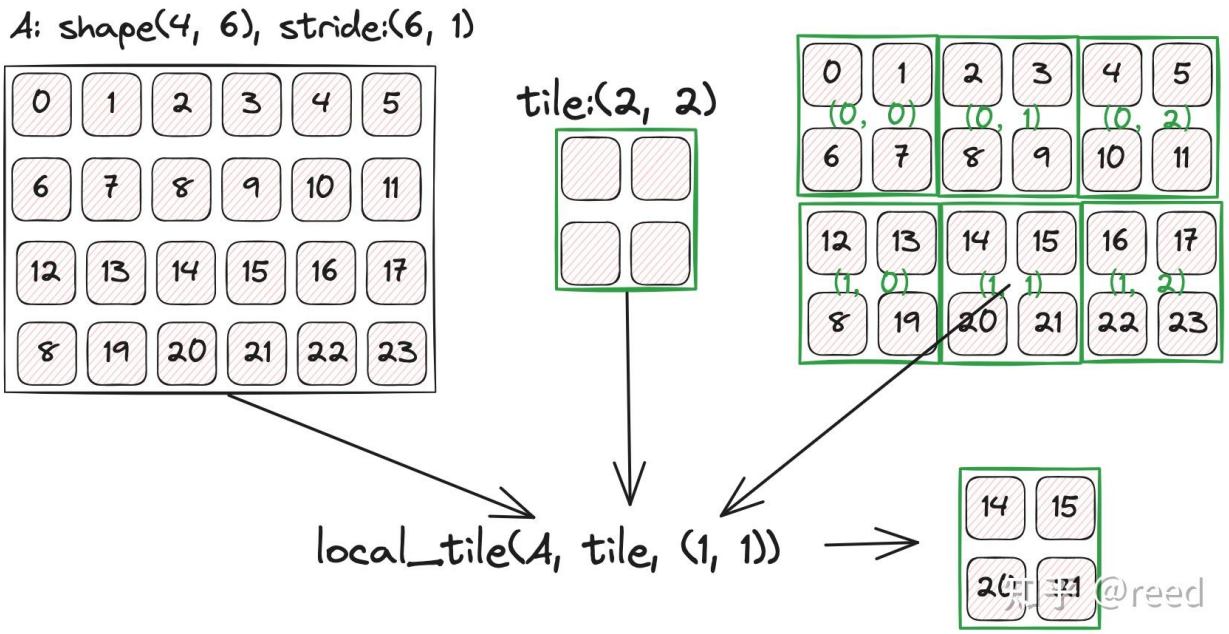


图 3.1: Figure 1. local tile 的几何解释

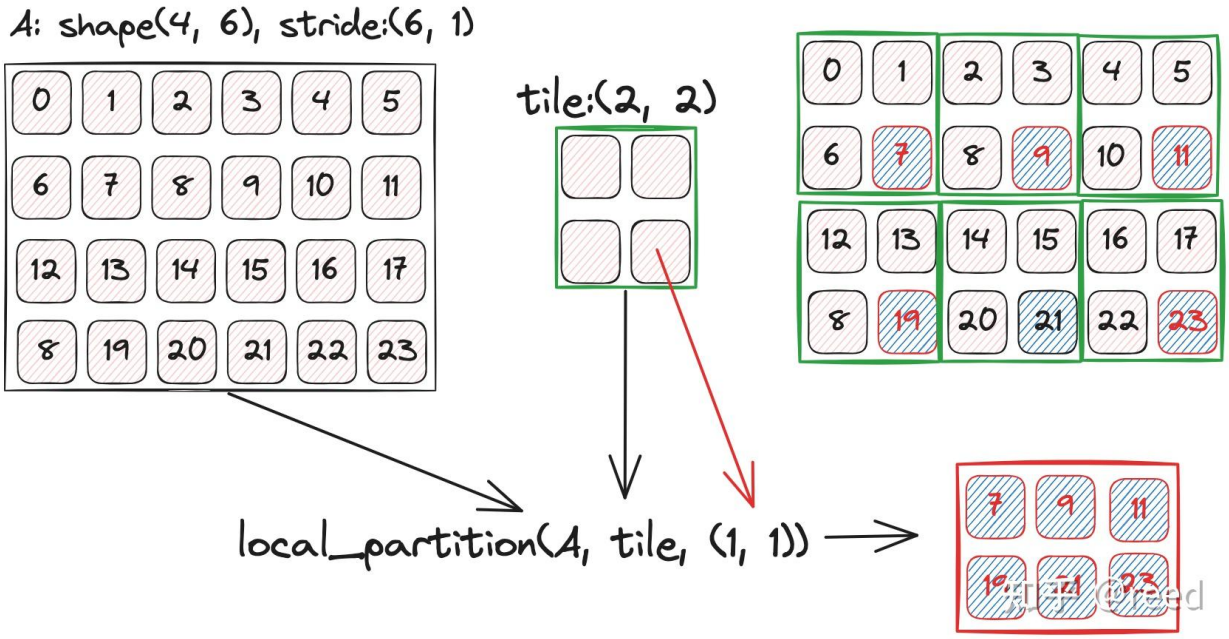


图 3.2: Figure 2. local partition 的几何解释

3.13 Tensor 数据类型转换 recast

Tensor 表达了特定的数据类型的数据，可以对其进行数据类型的重新解释，然后形成新的 tensor，类似于 C++ 中的 reinterpret_cast 语义，代码形式如下：

```
Tensor tensor = make_tensor<float>(make_shape(...));
Tensor tensor1 = recast<NewType>(tensor);
```

3.14 Tensor 内容的填充 fill 和清除 clear

`clear()`, `fill()` 可以对 tensor 进行元素级别的清除和填充操作

```
Tensor tensor = make_tensor(...);

clear(Tensor);  // 使用T{}类型的默认构造函数对tensor进行赋值
fill(Tensor tensor, T value);  // 将Tensor中的值填充为value值。
```

3.15 Tensor 的线性组合 axpby

对两个 tensor 进行线性组合运算`y = a*x + b*y`：

```
Tensor x = make_tensor(...);
Tensor y = make_tensor_like(x);

axpby(a, x, b, y);
```

3.16 Tensor 的打印 print

可以通过全局 print 方法来完成对 tensor 的打印来 debug 和展示 Tensor 的信息，print 会打印 tensor 存储空间所在的位置和 shape、stride 信息，而 print_tensor 除了以上信息还会打印 Tensor 中具体的每一个数值：

```
Tensor tensor = make_tensor(...);
print(tensor);
print_tensor(tensor);
```

3.17 特殊矩阵

构造只有形状没有类型的 tensor，用于一些特定变换：

```
Tensor tensor = make_identity_tensor(shape);
```

3.18 使用 Tensor 实现 Vector Add 示例：

以上回顾了 Tensor 的常用操作和方法，下面我们通过具体的实例展示如何使用 Tensor 工具实现 CUDA kernel 来完成针对 half 类型数据的向量化的 $z = ax + by + c$ 的计算。

对于该向量类问题，如果有丰富的 CUDA 开发的经验，我们可以通过如下优化手段提供高效的实现：

- 单个线程处理多个数据，通过数据预取和指令并行，提升数据读取效率、提升执行单元的流水线效率；
- 对 global 内存进行大字长读写，减少数据 IO 所需要的指令数目，减少调度开销，提升程序运行效率；
- 使用 Half2 类型，减少 half 类型引入的 PRMT 指令的转换和开销；

- 使用 FMA（fused multiply accumulate）指令完成计算，减少 FMUL、FADD 指令数，提升计算精度；

具体地，我们实现了如下函数，

```
// z = ax + by + c
template <int kNumElemPerThread = 8>
__global__ void vector_add_local_tile_multi_elem_per_thread_half(
    half *z, int num, const half *x, const half *y, const half a, const half b, const half c) {
  using namespace cute;

  int idx = threadIdx.x + blockIdx.x * blockDim.x;
  if (idx >= num / kNumElemPerThread) { // 未处理非对齐问题
    return;
  }

  Tensor tz = make_tensor(make_gmem_ptr(z), make_shape(num));
  Tensor tx = make_tensor(make_gmem_ptr(x), make_shape(num));
  Tensor ty = make_tensor(make_gmem_ptr(y), make_shape(num));

  Tensor tzr = local_tile(tz, make_shape(Int<kNumElemPerThread>{}), make_coord(idx));
  Tensor txr = local_tile(tx, make_shape(Int<kNumElemPerThread>{}), make_coord(idx));
  Tensor tyr = local_tile(ty, make_shape(Int<kNumElemPerThread>{}), make_coord(idx));

  Tensor txR = make_tensor_like(txr);
  Tensor tyR = make_tensor_like(tyr);
  Tensor tzR = make_tensor_like(tzr);

  // LDG.128
  copy(txr, txR);
  copy(tyr, tyR);

  half2 a2 = {a, a};
  half2 b2 = {b, b};
  half2 c2 = {c, c};

  auto tzR2 = recast<half2>(tzR);
  auto txR2 = recast<half2>(txR);
  auto tyR2 = recast<half2>(tyR);

#pragma unroll
  for (int i = 0; i < size(tzR2); ++i) {
    // two hfma2 instruction
    tzR2(i) = txR2(i) * a2 + (tyR2(i) * b2 + c2);
  }

  auto tzRx = recast<half>(tzR2);

  // STG.128
  copy(tzRx, tzr);
}
```

其中 template 行通过编译时常量指定每个线程处理 8 个数据，避免运行时常量不能利用寄存器（寄存器不可以寻址）所带来的 Local Memory 问题。由于一个线程处理 8 个元素，并且单个数据的大小为 sizeof(half) = 2, 这样一个线程所需要的数据量为 8 x 2 = 16byte，该大小的数据可以通过 LDG.128 指令实现一条指令完成对数据从全局内存到寄存器到加载；

其中函数声明行通过 ___global___ 关键字指明该函数为 cuda kernel 函数，且通过 const 等信息提示了输入输出数据；

using 行引入 cute 的名字空间，以便使用 cute 名字空间中的 Tensor 工具和其他函数；

idx 和 if 判断行，通过使用 cuda 提供的内建 threadIdx、blockIdx、blockDim 变量来定位线程在网格中的位置；

Tensor tz、tx、ty 行，通过利用 make_tensor 接口将 kernel 参数中的裸指针和维度信息包装成 tensor 表达；

Tensor tzr、txr、tyr 行，通过 local_tile 方法实现对 tz/tx/ty Tensor 的分块和选择（利用 idx），在这之后我们只需要关注我们需要处理的局部的 tensor 即可，无需关注大的全局的 tensor。同时在进行局部切块的时候，我们使用 Int<>{} 形式实现形状的编译常量表示，避免了运行时的量及其会引入的 local memory；

Tensor txR、tyR、tzR 行通过 make_tensor_like 接口实现栈上 tensor 的定义（GPU 表现为寄存器空间）；

copy 行通过调用 cute 提供的 copy 函数实现全局内存数据读入到寄存器空间，此处会生成 LDG.128 指令；

half2 a2、b2、c2 行，重复系数 a、b、c 构造 half2 类型的系数，以利用的 HFMA2 的指令完成后续计算；

auto tzR2 等行通过 recast 指令实现连续的 half 类型到 half2 类型的转换，以便能利用更高效的 HFMA2 指令；

pragma 及后续 for 行实现了多个元素的 z = ax + by + c 的计算，并且通过括号将该计算通过两个 HFMA2 指令实现，如果没有括号，则其会生成 HMUL2 + HMUL2 + HADD2 + HADD2 指令（由于乘法不满足结合律，且 IEEE 规定了浮点数计算的顺序需按照代码书写顺序）；

最后我们将结果 cast 回来并通过 copy 接口将计算结果存储到全局内存；

值得注意的是，Tensor tz; Tensor tzr; 行看似是生成 Tensor 但实际其并没有涉及到全局内存到读写（并没有 Tensor 被拷贝），只是利用 Layout 进行 tensor 的表达和变换，数据实体没有移动，只有在 copy 的时候才有实际的数据读写。这也回应了文章开头的对 cute 中 Tensor 和深度学习框架中的 Tensor 不一样，大部分时间，我们在 cute 中使用 Tensor 只是使用 Tensor 的逻辑语义和变换，并没有实质的触发 Tensor 的搬运。同时我们也看到我们使用 Tensor 语义和工具能够更形象化的表达我们的逻辑，方便我们的思考，而 CUDA 的优化思路和技巧并不会因为 Tensor 的引入而变简单或困难。Tensor 只是工具，可以方便我们的表达，至于深层次的优化思路那还是对经验的挑战。

3.19 总结与讨论

本文介绍 cute 中 Tensor 常用的方法，并通过该 Tensor 工具完成了向量乘加法运算。Tensor 是对数据表达的一种很好的抽象，我们可以利用这些抽象简化我们的思考模型（使我们的思考可以更抽象），降低编程困难，cute 中提供的 Tensor 更多的是对 Tensor 的表达和变换，由 Tensor 生成 Tensor 只是 Layout 表达的变换并不产生数据实体的搬运（如 local_tile）。Tensor 表达虽然提供了很多方便，但也只限于表达的高效和便捷，如何对程序进行优化，Tensor 表达的引入并没有提供额外帮助，它仍然需要我们从别的途径来获得。

即便如此，” 表达和抽象” 依然无比重要，那正如：伽罗瓦如果没有群这一表达工具，就难以解决多项式根的问题；杨振宁没有群这一工具就难以构思举世的杨-米尔斯理论。

抽象和工具让我们可以在更高的维度上思考。在后续文章中我们将基于 Tensor(Layout)这种抽象进行 MMA(matrix multiply accumulate) 和 COPY 的介绍。

第四章　cute 之 MMA 抽象

作者：reed（知乎 @reed-84-49，专栏「CUDA 高性能编程」）　原文：https://zhuanlan.zhihu.com/p/663092747

Layout 和 Tensor 是描述数据逻辑排列和数据的抽象工具，在 Tensor 抽象之上我们可以完成矩阵乘法。前序文章我们介绍了 Layout[1]、Layout 解释[2]和 Tensor[3]，本文将介绍 cute 中利用 Tensor Core 完成矩阵乘法（MMA = matrix multiply accumulate）所需要的数据结构抽象。在介绍这些抽象之前，我们先对 NVidia GPU 上的 Tensor Core 作简单介绍。

4.1　NVidia Tensor Core 简介

深度学习的出现极大的提升了对算力需求，尤其是矩阵类算力的需求，NVidia 为了应对 Google 推出的 TPU 并抢占算力市场，于 2017 年推出 Volta 架构装配 Tensor Core 的显卡。在推出 Tensor Core 架构之前，矩阵类计算多诉诸 SIMT 的 CUDA Core 算力和传统的 CPU 上的 SIMD 算力。Tensor Core 是专门针对矩阵计算而设计的硬件单元，其可以高效的实现小块的矩阵乘法，公式为`D = A x B + C`，如国内使用算力主力 Ampere 架构的显卡 A100 所装配的 Tensor Core，其 fp16 指令（mma.sync.m16n8k16）可以完成 16x8x16（MNK 描述方法，即 A 矩阵的维度为 16x16, B 矩阵为 16x8, C 矩阵为 16x8）的半精度的矩阵乘法。其效率要比传统的 CPU 和 CUDA Core 高很多，所以对算力需求极高的深度学习计算（如卷积、矩阵乘法、注意力）也多是使用 Tensor Core 来完成。

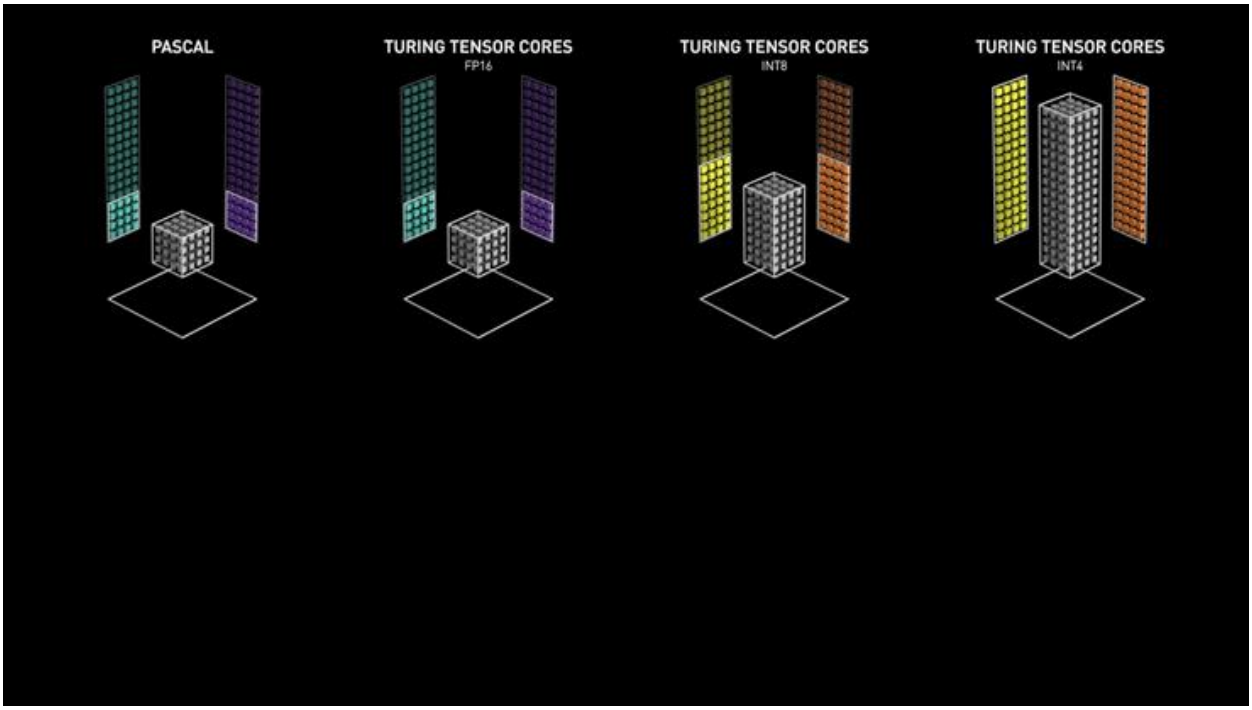


图 4.1: Figure 1. Tensor Core 对不同计算精度的效率动画（引用参考 1）

Tensor Core 在 Volta、Turing、Ampere 架构上输入输出数据都为和 CUDA Core 共享的寄存器，在 Hopper 架构上，为了得到更好的带宽，计算所需要的输入数据可以直接存放在共享内存上，更详细的信息可以参考关于 Tensor Core 数据存储空间的讨论[4]。Tensor Core 支持不同精度的计算，不同精度的计算效率也有区别，其动画效果如图 1，各代架构对不同计算精度的支持如图 2。Tensor Core 提供了高算力，我们有两种形式可以使用 Tensor Core，第一种是利用 NVidia 提供的矩阵计算库 cublas 和深度学习库 cudnn，它们封装了常用的矩阵类计算和深度学习计算所需要的函数，

[1] https://zhuanlan.zhihu.com/p/661182311
[2] https://zhuanlan.zhihu.com/p/662089556
[3] https://zhuanlan.zhihu.com/p/663093816
[4] https://www.zhihu.com/question/587780273/answer/2929756314

	Hopper	Ampere	Turing	Volta
Supported Tensor Core precisions	FP64, TF32, bfloat16, FP16, FP8, INT8	FP64, TF32, bfloat16, FP16, INT8, INT4, INT1	FP16, INT8, INT4, INT1	FP16
Supported CUDA® Core precisions	FP64, FP32, FP16, bfloat16, INT8	FP64, FP32, FP16, bfloat16, INT8	FP64, FP32, FP16, INT8	FP64, FP32, FP16, INT8

知乎 @reed

图 4.2: Figure 2. 不同架构支持的计算精度（引用参考 1）

以 SDK 的形式提供能力；第二种是通过 CUDA 编程提供的特定的接口和 PTX 汇编实现。对于第二种形式，CUDA 编译器 NVCC 提供了 wmma（warp matrix multiply accumulate）和 mma（matrix multiply accumulate）两种形式，第一种形式是通过提供`fragment`数据表示和特定的`load_matrix_sync()`、`store_matrix_sync()`、`mma_sync()`数据加载存储和计算 API 来触发对 Tensor Core 的编程。另一种是通过 PTX 汇编实现，其数据直接面向寄存器表示，计算则是通过`mma.sync`类的函数实现。wmma 形式对数据和 API 都进行了相应的抽象，编程相对简单，但对指令的控制也相对粗糙。mma 形式的编程直接面向寄存器表示和汇编指令，难度较大，容易出错，但是可以实现精细的控制从而达到更高的计算效率。

4.2 cute 的 MMA 核心数据结构及其相互关系

cute 作为高性能的原语表示和抽象，其直接面向 mma 实现，对数据和计算进行很好抽象的前提下，依然保留了精细控制能力。对于矩阵计算类任务，cute 提供了 MMA 的抽象。其主要包含的核心结构：`MMAOperation`、`MMA_Traits`、`MMA_Atom`、`TiledMMA`、`ThrMMA`。这些结构共同完成对 MMA 的抽象和实现，这些结构是实现矩阵计算的基础。具体地，

- MMAOperation 提供了指令层的封装，针对不同的 GPU 架构，NVidia 提供了不同的指令来使用 Tensor Core，MMAOperation 即对这些硬件指令的封装同时提供了公共的`fma`方法，供框架层调用，终端用户从既定的架构中选择适合的即可；
- 了解 MMA_Traits，必须先了解 C++ 中 traits 的概念。在 C++ 程序设计中 traits 被翻译为萃取，其表达对类型的提取。这种解读非常抽象，这里我提供一种更简洁的理解：traits 是函数抽象，这个函数接受的输入是数据类型而不是传统的变量或对象，同时这个函数可以返回多个属性。对于使用者而言，这些信息是使用者所需要的，但是又不是原始的类型所必须的。也就是说有些信息不属于类型，但是这部分信息对于调用者却需要，这样我们引入 traits，其承接类型到该类型使用者之间的桥梁作用。MMA_Traits 就是这个桥梁。MMAOperation 的使用者是 MMA_Atom，但是 MMA_Atom 所需要的信息又比 MMAOperation 本身提供的要多，这时引入 MMA_Traits 则填补上它们之间的这个空隙。
- MMA_Atom 自然的如其名字中的 Atom 所表达的 “原子”，这是硬件提供的能执行的矩阵乘法的最小的单位，其能完成一个特定规格 (如 MNK) 的矩阵乘法问题 `D = A x B + C`。
- 在 MMA_Atom 之上进行扩展，形成更大的矩阵乘法能力，即 TiledMMA，它应当是原子的整数倍。这种扩展可以是执行单元层面的也可以是对 Atom 重复执行，因为两种扩展都能提供更大的矩阵乘法。
- TiledMMA 提供的是逻辑表达，而具体到 CUDA 编程实现的时候，我们的代码只能是线程级别的，即我们在 CUDA kernel 中只能写每一个线程执行的指令，不能写其级别的指令，即便我们知道有些指令是 warp level 的，但是我们能 code 的，都是线程级别的指令。这样，我们在矩阵乘法问题上也需要将前面的逻辑的矩阵块分块，然后写出线程级别的代码。ThrMMA 就是完成这个工作：将逻辑的矩阵根据提供的线程号（`threadIdx.x`）来获得自己这个线程的任务（可以是 warp level 的等）。
- 在 ThrMMA 得到各个线程的计算任务后，各个线程同时调用 cute::gemm 函数完成各个线程级别的任务下发，最终所有线程等结果体现为大块的`D = A x B + C`的任务完成计算。

以上 MMA 核心结构的关系如图 3，表现为三个大的抽象层：硬件和指令抽象、逻辑抽象、和 CUDA 编程级指令。

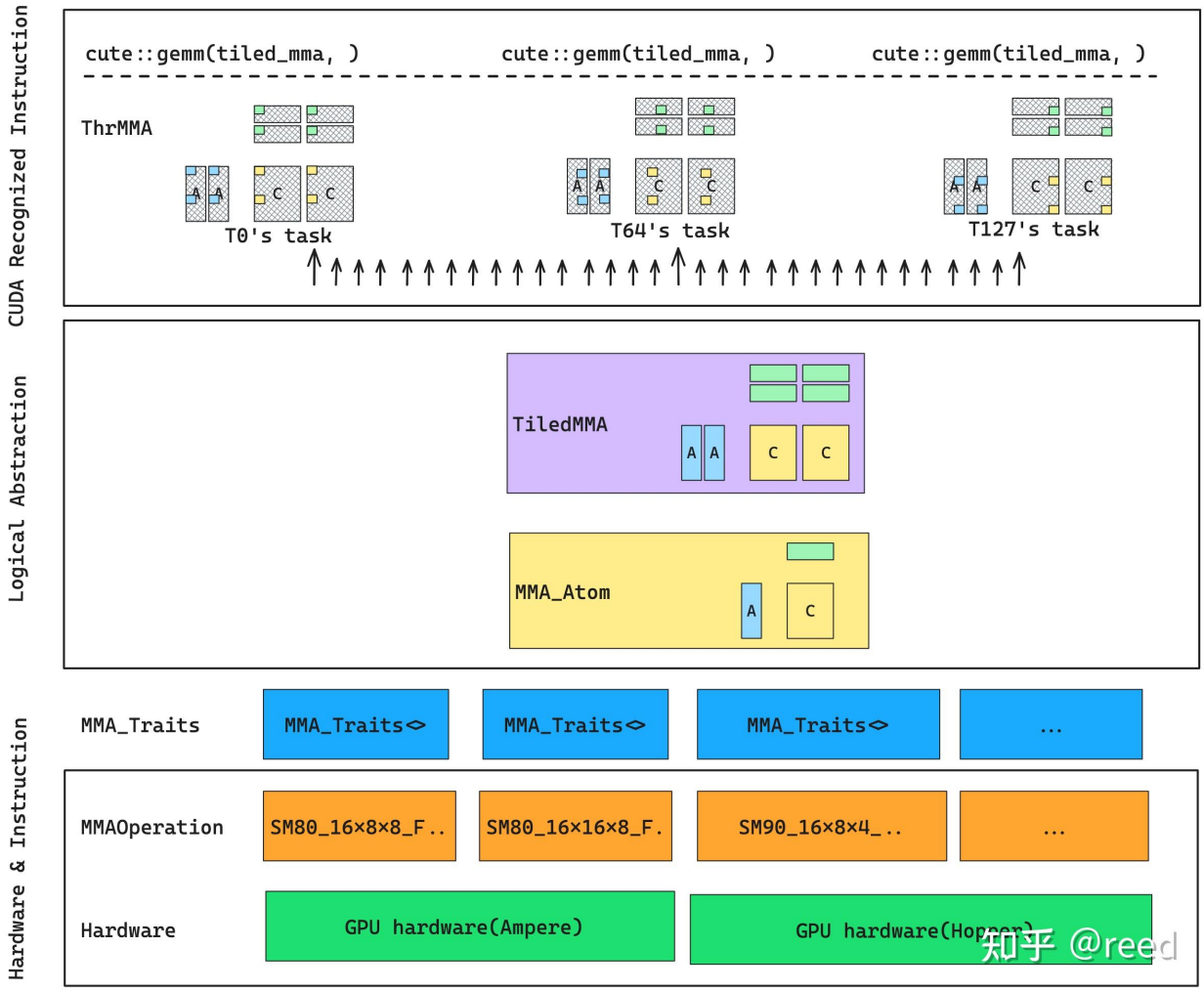


图 4.3: Figure 3. cute MMA 核心结构和其相互关系

4.3 MMAOperation

operation 通过指令实现 D = AB + C 计算，该 operation 需要设定 A/B/C/D 的操作数的类型。其中 fma 的参数形式依赖于 D/A/B/CRegisters 的类型和数据量。即，如果 DRegister 的类型为 float[2], 则 fma 的接口中最前面的两个参数为 float 输出。如`SM75_16x8x8_F32F16F16F32_TN` 表示 SM75 算力的 Turing 架构的 MMA，16x8x8 表示矩阵的 MNK 大小，F32F16F16F32 表示 D、A、B、C 的数据类型分别为 float32、float16、float16、float32。T 表示 A 矩阵为行优先，B 矩阵为列优先（blas 中约定 normal 矩阵为列优先，T 表示 transpose，即对列优先的矩阵进行转置则为行优先），

```
struct SM75_16x8x8_F32F16F16F32_TN {
  using DRegisters = float[4];
  using ARegisters = uint32_t[2];
  using BRegisters = uint32_t[1];
  using CRegisters = float[4];

  // Register asm fma
  CUTE_HOST_DEVICE static void
  fma(float         & d0, float         & d1, float      & d2, float      & d3,
      uint32_t const& a0, uint32_t const& a1,
      uint32_t const& b0,
      float    const& c0, float    const& c1, float const& c2, float const& c3)
  {
```

```
    asm volatile("mma.sync.aligned.m16n8k8.row.col.f32.f16.f16.f32" ...);
  }
};
```

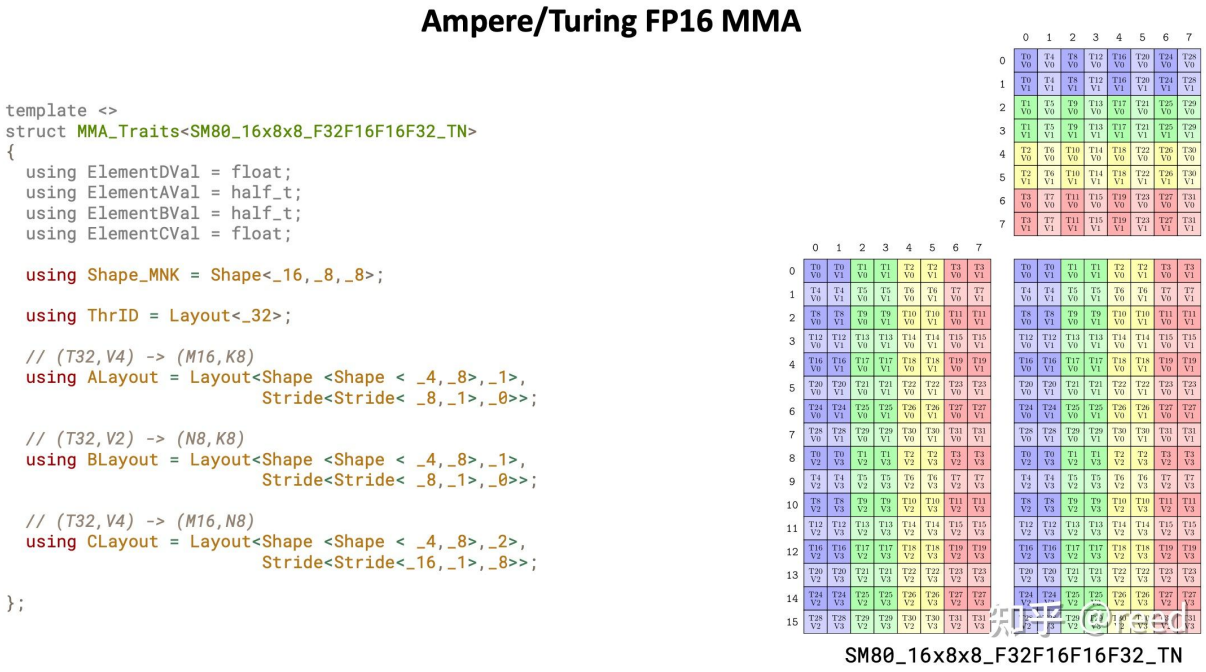


图 4.4: Figure 4. MMA_Traits 提供给 TiledMMA 的信息（引用自参考 3）

4.4　MMA_Traits

针对特定的 MMAOperation 类型，定义其相关的辅助**类型**或**值**给 MMA_Atom 使用，用以完成块状的矩阵乘法，其需要提供出的类型信息如下，

```
using ElementDVal =  // Logical D-value type
using ElementAVal =  // Logical A-value type
using ElementBVal =  // Logical B-value type
using ElementCVal =  // Logical C-value type

using ElementAFrg =  // A-type consumed by MMA  (if ommitted, same as ElementAVal)
using ElementBFrg =  // B_type consumed by MMA  (if ommitted, same as ElementBVal)
using ElementCFrg =  // C_type consumed by MMA  (if ommitted, same as ElementCVal)

using Shape_MNK =    // Logical MxNxK shape of the MMA

using ThrID     =    // Logical thread id (tid) -> tidx

using ALayout =      // (Logical thread id (tid), Logical value id (vid)) -> Flat MK-coord
using BLayout =      // (Logical thread id (tid), Logical value id (vid)) -> Flat NK-coord
using CLayout =      // (Logical thread id (tid), Logical value id (vid)) -> Flat MN-coord
```

4.5　TiledMMA

TiledMMA 整体表达了矩阵在 MNK 空间维度如何通过 Atom 组织而来，其结构内部定义了很多函数，这些函数提供了对给定计算块的划分能力，但是这部分逻辑终端用户早期可以不用太多关注 (这些)，只需要关注如下两个 API 即可，第一个是 TiledMMA 的模版参数，第二是 TiledMMA 提供的 get_thread_slice 函数。模版参数表达了 TiledMMA 在 MMA_Atom 上的扩展逻辑：`AtomLayoutMNK`表示 M N K 方向上分别重复几次 Atom，这种重复会要求更多的执行线

程，`ValueLayoutMNK`表述了对该 Atom 在 M N K 方向上重复几次，这里的重复是通过重复计算来完成的。get_slice、get_thread_slice 函数通过给定线程 id 则获取线程对应到 ThrMMA 结构，形式如下

```
template <class MMA_Atom,
          class AtomLayoutMNK   = Layout<Shape<_1,_1,_1>>,
          class ValLayoutMNK    = Layout<Shape<_1,_1,_1>>,
          class PermutationsMNK = Tile<Underscore,Underscore,Underscore>>
struct TiledMMA : MMA_Atom {
  ...;
  ThrMMA get_slice(ThrIdx thr_idx);
  ThrMMA get_thread_slice(ThrIdx thr_idx);
  ...;
}
```

cutlass3.4 版本中更新了该接口去掉了 ValLayoutMNK，具体的参数解读可以参考 cute 核心作者 Cecka 的解释[5]。

4.6 ThrMMA

该结构由 TiledMMA 根据具体的线程 id 分解而来（ThrMMA 即 Thread MMA），其描述了线程级实现`D = A x B + C`任务时的功能抽象，主要是如下`partition`类函数和`partition_fragment`类函数。其中`partition`函数完成对逻辑 Tensor 针对该线程的划分，即 Tensor 参数提供大块的逻辑矩阵单元，其返回值返回该线程需要进行的任务的 Tensor 描述。如 Tensor C 为 BLK_M x BLK_N，则 partition_C 可以得到线程级别的任务，维度为（MMA, MMA_M, MMA_N），MMA 表达了 TiledMMA 一次能计算的单元，MMA_M, MMA_N 表达了 M 方向和 N 方向需要分块数量。`partition_fragment`类函数是按照`partition`类函数返回的 Tensor 形状生成的对应的寄存器表示。

```
ThrMMA {
  Tensor partition_C(Tensor C);
  Tensor partition_A(Tensor A);
  Tensor partition_B(Tensor B);
  Tensor partition_fragment_C(Tensor C);
  Tensor partition_fragment_A(Tensor A);
  Tensor partition_fragment_B(Tensor B);
}
```

4.7 cute::gemm

cute::gemm 是线程完成 MMA 计算的函数，其核心接口如下，这里 D、A、B、C 接收的 Tensor 即为 ThrMMA 所划分出来的 Tensor，

```
void gemm(TiledMMA &mma, Tensor& D, Tensor const& A, Tensor const& B, Tensor const& C);
```

cute::gemm 和其他组件的相互关系如表格所示。

功能	MMA
指令 + 存储类型	MMAOperation
逻辑类型和形状要求	MMA_Traits
原子能力	MMA_Atom
块状能力 (多个原子能力)	TiledMMA
线程级能力	ThrMMA
数据拆分 API	ThrMMA::partition_A/B/C()
触发功能执行	cute::gemm(tiled_mma, thr_d,...);

[5] https://github.com/NVIDIA/cutlass/discussions/1345

4.8 总结

cute 提供了 MMA 能力来完成`D = A x B + C`的矩阵乘法运算，其针对指令封装，适配层，原子能力、块状 MMA、线程划分和执行进行了抽象，形成了`MMAOperation`、`MMA_Traits`、`MMA_Atom`、`TiledMMA`、`ThrMMA`、`cute::gemm`数据结构和函数，我们通过这些结构能够完成逻辑块状矩阵乘法的划分和执行。这些抽象通过软件分层设计使得各层次独立，我们不必关注底层细节，只需要从提供的模块中组合我们的逻辑即可，同时抽象的解偶设计，使得我们可以专注于顶层逻辑而降低对底层细节的要求。

后续我们会介绍`Copy`类结构抽象，来实现数据从不同存储结构的搬运，在介绍`Copy`之后我们将结合 MMA 和 Copy 能力完成一个简单的矩阵乘法，然后在这个版本之上进行一步步优化，最终得到 SOTA 的矩阵计算。

4.9 参考

- NVIDIA Tensor Cores: Versatility for HPC & AI[6]
- 大佬们，A100 显卡上的 tensorcore 有自己的私有寄存器吗？[7]
- https://www.cs.utexas.edu/users/flame/BLISRetreat2023/slides/Thakkar_BLISRetreat2023.pdf

QST What is PermutationMNK in TiledMMA in CUTLASS 3.4 changes? · NVIDIA/cutlass · Discussion #1345[8]

[6] https://www.nvidia.com/en-us/data-center/tensor-cores/
[7] https://www.zhihu.com/question/587780273/answer/2929756314
[8] https://github.com/NVIDIA/cutlass/discussions/1345

第五章　cute 之 Copy 抽象

作者：reed（知乎 @reed-84-49，专栏「CUDA 高性能编程」）　原文：https://zhuanlan.zhihu.com/p/666232173

前面文章介绍了 cute 中的 MMA 抽象[1]，通过 MMA 我们可以利用 Tensor Core 完成寄存器上的`D = AxB + C`计算。对于 GPU 编程而言，输入数据一般来被存储在全局内存之上。这就涉及到如何高效地将全局内存上的数据运到 Tensor Core MMA 计算所要求的寄存器上的问题。数据搬运的数学描述是 `D = S`，其中 D 和 S 是前文中介绍的 Tensor 结构[2]，分别表示目标 Tensor（Destination）和源 Tensor（Source）。它们一般存储在不同的存储层次上，它们有各自的 Layout 描述[3]。本文我们将围绕该问题，重点介绍 cute 在数据搬运方面的数据结构的抽象。在文章结构上，本文首先介绍 CUDA GPU 中的存储层次和矩阵计算相关的 ldmatrix 指令，然后总体介绍 cute 中 Copy 相关的数据结构和函数的抽象以及各个结构之间的相互关系；然后我们依次对各个数据结构的核心函数和成员进行介绍，最后我们总结 cute 对 Copy 的抽象。

5.1　CUDA 的存储层次和数据加载路径

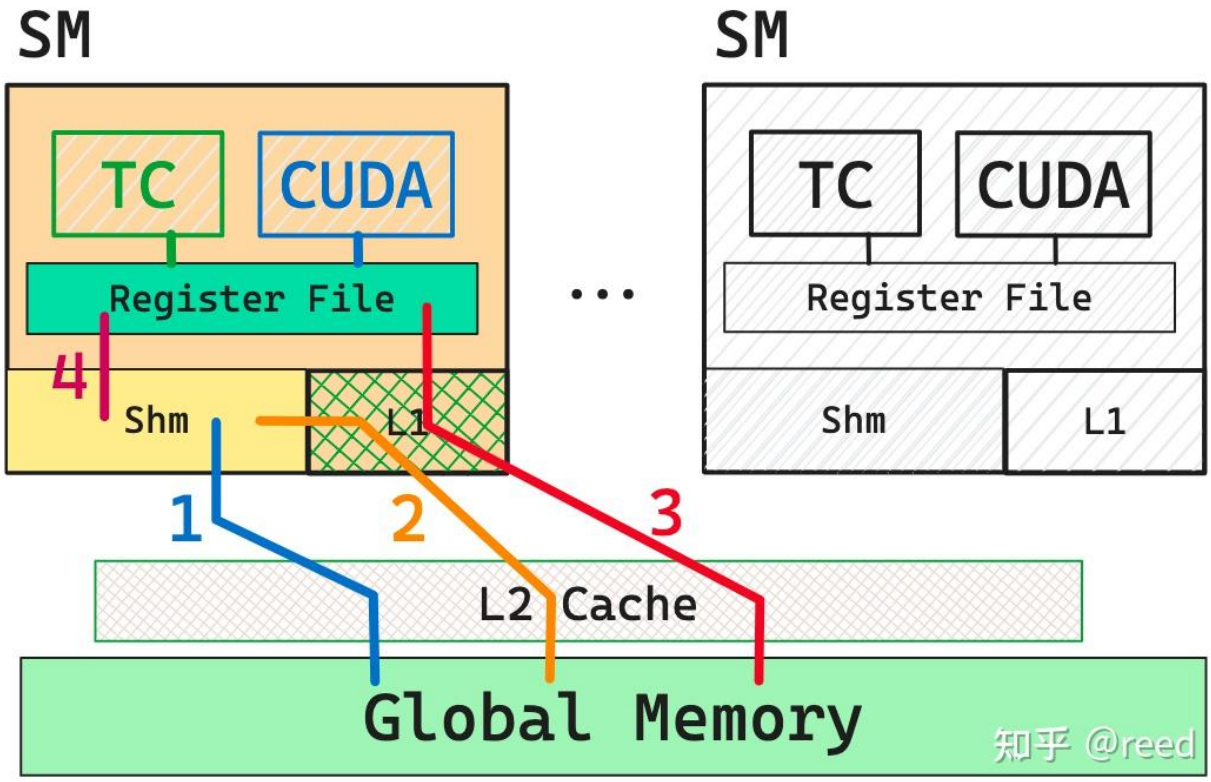


图 5.1: Figure 1. GPU 的存储层次和数据搬运路径

经典的 GPU 存储层次如图 1 所示，主要有：片外存储结构-全局内存（global memory）；片上 SM（stream multi-processor）内的 shared memory 和 L1 data cache，以及寄存器堆（register file）；以及在全局内存和 SM 之间的 L2 Cache。具体地，全局内存（图中标记为绿色 global memory）容量最大，在数据中心级 A100 显卡中存储空间可以达到 80GB，采用 HBM2e 技术实现，最高带宽可达 2TB/s；再往内层为 L2 Cache，在 A100-80GB 上为 40MB，带宽可达 20TB/s; 再向内层为片上 (on-chip) 存储 shared memory 和 L1 data cache，shared memory 和 L1 data cache 共享 192KB 的空间，可以配置 shared memory 和 L1 的大小，最大可将 shared memory 配置为 164KB。离计算单元 Tensor Core 和 CUDA

[1] https://zhuanlan.zhihu.com/p/663092747
[2] https://zhuanlan.zhihu.com/p/663093816
[3] https://zhuanlan.zhihu.com/p/661182311

Core（图中分别标记为 TC 和 CUDA）更近的存储结构为寄存器堆（图中标记为 Register File），计算单元计算所需要的数据必须来自寄存器（Ampere 及之前架构如此，Hopper 架构的 Tensor Core 可以直接读取存储在 shared memory 数据进行计算），是 GPU 中最快的存储结构，一个线程最多使用 255 个 32bit 的寄存器。对于一个用 GPU 进行计算的问题，其原始数据来自于全局内存，可以经过三条路径到达核心计算单元上（Tensor Core 或 CUDA Core）。第一条路径，如图中路径 1 所示，从 global 内存经过 L2 到达 shared memory（L1 bypass），然后从 shared memory 到达寄存器；第二条路径，如图中路径 2 所示，从 global 内存经过 L2 到达 L1 然后到达 shared memory，再从 shared memory 到达寄存器；第三条路径从 global 内存经过 L2 到达 L1，然后到达寄存器。其中路径 1 和 2 只在 Ampere 及其之后的架构才支持。Ampere 之前的架构从全局内存到寄存器只支持路径 3，从全局内存到共享内存只能先通过 3 将数据加载到寄存器，然后通过路径 4 存储到共享内存。这些存储结构我们可以编程控制的部分为全局内存共享内存和寄存器，L1 Cache 和 L2 Cache 是缓存机构，我们可以控制其 bypass 与否，同时我们也可以通过 PTX 指令 modifier 控制 L2 cache 的数据预取行为。

5.2 高效的 ldmatrix 指令

在矩阵计算的优化中，非常重要的一个技术是通过数据分块实现数据复用，数据复用可以减少对低层级存储器的访问数据量，提升数据访问效率继而提升总体计算效率。在 GPU 的实现中，可编程的数据复用发生在共享内存部分，即用户通过编程手段将部分数据加载到共享内存然后复用共享内存中的数据，实现数据由共享内存到寄存器，然后实现更高效的计算。前面 MMA 章节我们介绍了 Tensor Core 的基础信息，如果我们认真研究 Tensor Core 的汇编指令我们不难发现，参与计算 warp 内的线程只持有矩阵的部分数据，这些数据保存在线程的私有寄存器中（SIMT 架构中，可以认为寄存器为线程所私有），warp 内的所有线程的寄存器共同组成完整的矩阵计算数据。这是 NVidia 在 SIMT 架构下实现 warp level 的 Tensor Core 计算的创新实践。如图 2 所示，在 SIMT 意义下每个线程持有两个数据（如 float16，可以表达为一个寄存器），warp 内的 32 个线程共同构成 64 个数据，形成 8x8 的 warp level 的小矩阵，供 Tensor Core 计算用。

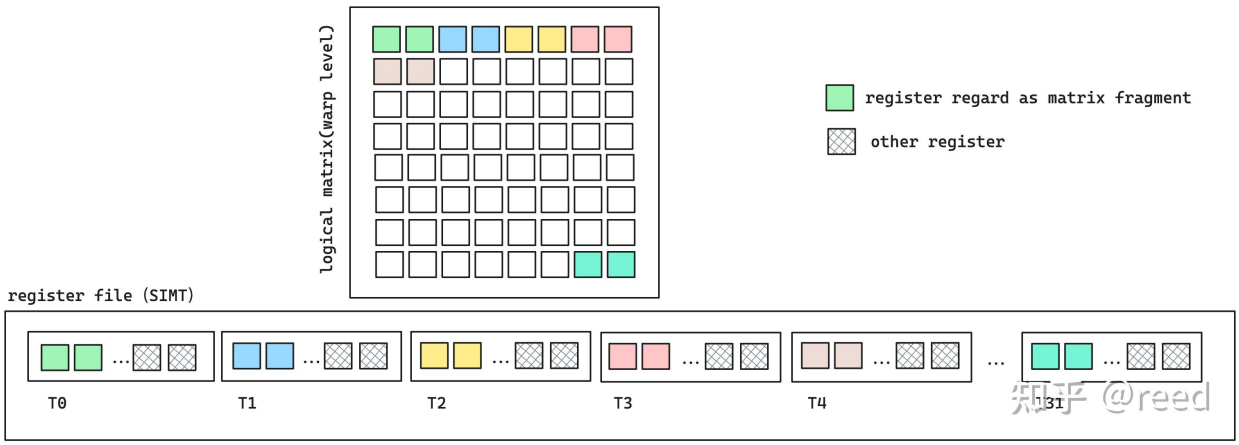


图 5.2: Figure 2. SIMT 寄存器协作构成 warp level 矩阵

通过各个线程提供的寄存器可以完成 warp level 的矩阵表示和存储，利用 Tensor Core 则可以完成高效的存储在寄存器上的矩阵计算。就数据从共享内存到寄存器的加载方面而言，可以通过 SIMT 意义下的 LDS（load shared）来完成，但是由于数据是分布在不同的线程的寄存器上连续性方面不友好。为了更极致的性能 NVidia 从 Turing 架构开始提供了专门针对这种场景的加载指令 ldmatrix。如图 3，其展示了 SIMT 形式的模式加载矩阵数据和 ldmatrix 协作式加载矩阵数据的对比，ldmatrix 协作式加载可以通过线程提供共享内存的地址（提供 16Byte 数据）完成数据的加载然后将数据分配到 warp 中各个线程的寄存器中，实现了跨越 SIMT 寄存器边界的写出，而如果是 SIMT 形式的加载，则只能使用更窄的数据位宽，这也就需要更多的指令完成等量数据的读写，同时 ldmatrix 由于是单线程提供 16Byte 的数据地址，warp 内所有线程可以提供 16Byte x 32 = 512Byte 的数据到寄存器的加载，单指令实现 16x16 float16 矩阵的加载，减少指令数提高调度效率，同时其可以在写出时合并矩阵转置能力（可以参考 tensorcore 中 ldmatrix 指令的优势是什么?[4]）。通过 ldmatrix 可以实现 warp level 共享内存到寄存器的数据搬运，自然地，cute 对这种数据搬运提供了对应的抽象能力。

[4] https://www.zhihu.com/question/600927104/answer/3029266372

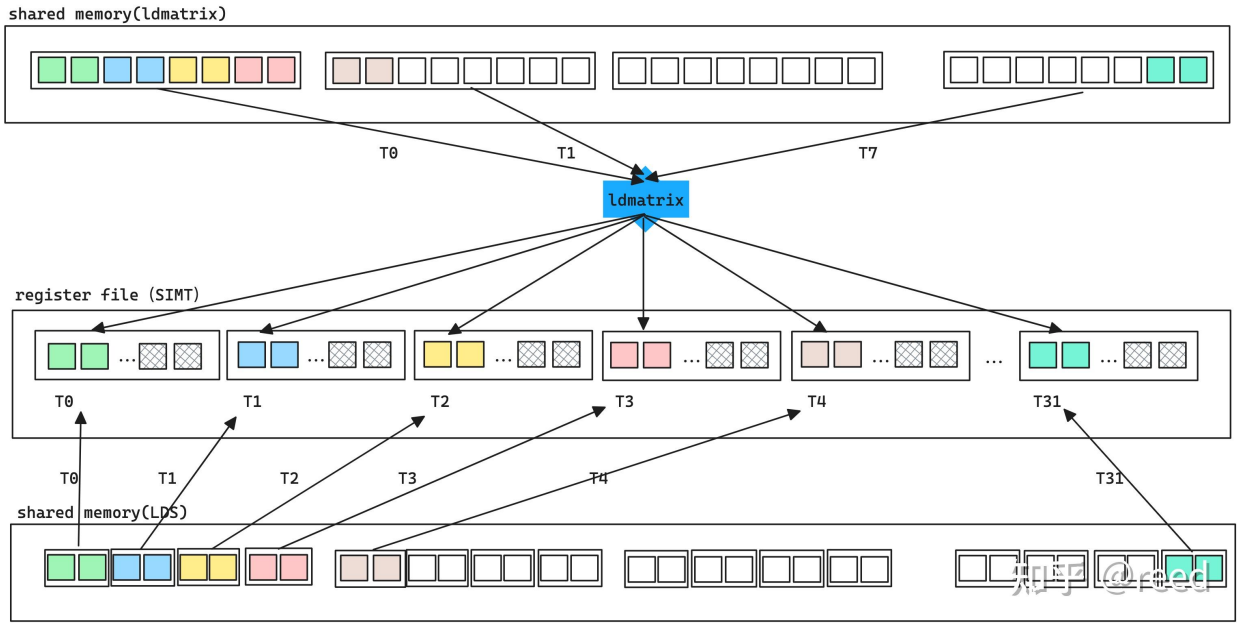


图 5.3: Figure 3. SIMT 形式加载矩阵数据和 ldmatrix 协作式加载矩阵的对比

5.3 cute Copy 抽象及其相互关系

和 MMA 类似，cute 对数据搬运提供了对数据搬运的数据结构抽象，主要包括`CopyOperation`、`Copy_Traits`、`Copy_Atom`、`TiledCopy`、`ThrCopy`和拷贝函数`cute::copy`。这些结构和函数共同完成对 GPU 各个层级存储之上的数据进行搬运的抽象和实现，具体地，

- CopyOperation 提供了指令级的数据搬运的封装，NVidia 在不同的硬件架构、不同的存储层次之间数据搬运提供了不同的指令，如前文提到的`ldmatrix`和`LDS`等，还有针对 Ampere 架构的`cp.async`等，我们在使用时只需要根据我们的硬件支持的指令情况和需要搬运的内存层次来选择已经提供的 Operation 即可；
- Copy_Traits 和 MMA_Traits 类似，提供了 CopyOperation 类型没有提供，但是其使用者 Copy_Atom 却需要的起到桥梁作用的信息；
- Copy_Atom 提供了指令级别不可分割的数据搬运的拷贝能力；
- TiledCopy 是对 Copy_Atom 的能力的封装通过重复拷贝执行单元的个数（增加执行线程）或者做多次的拷贝实现对原子能力的重复；
- TiledCopy 提供的是逻辑上的拷贝的概念，在具体的 kernel 执行之时，为了复合 CUDA 的编程范式，需要写成线程级别的指令，ThrCopy 可以实现将大块的数据根据 TiledCopy 所描述的划分规则，通过提供当前线程的线程号 threadIdx.x 对大块的 Tensor 进行划分，得到当前线程为了完成`D = S` 拷贝所需要该线程做的任务；
- cute::copy 在 ThrCopy 提供了当前线程的任务之后，便可以通过 copy 函数触发具体的数据搬运指令。

如图 4 所示，在硬件和拷贝方向之上提供了指令抽象 CopyOperation，再往上形成 D = S 的拷贝逻辑抽象，包含指令级别所能完成的拷贝原子能力 Copy_Atom 和对 Atom 重复后得到的 TiledCopy 能力，在逻辑之上针对具体的线程划分出具体的线程级的任务，通过 cute::copy 函数触发相应的拷贝任务，所有线程共同完成 Tensor 到 Tensor 的拷贝。下面我们针对每一个抽象层次，具体的介绍各个数据结构和抽象的细节。

5.4 CopyOperation

Operation 封装了特定硬件支持拷贝能力。它一般通过 PTX 汇编指令（或 CUDA 实现）来完成，实现指令集的拷贝能力抽象，其定义了源数据类型和目标数据类型以及个数，同时提供 copy 函数供框架层调用，示例如下，源寄存器为一个 uint128_t(128bit 数据)，目标寄存器为一个 uint32_t 的数据：

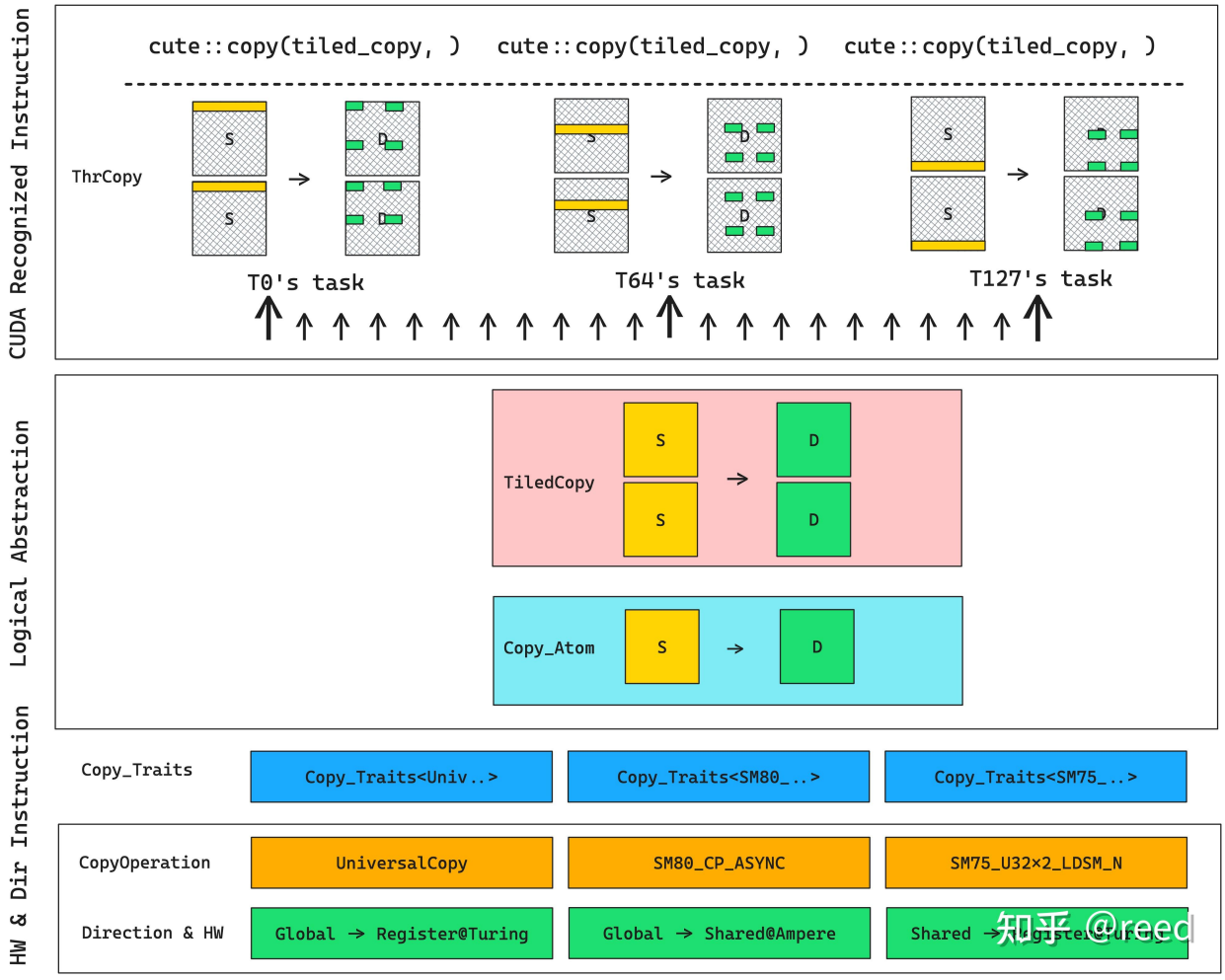


图 5.4: Figure 4. cute Copy 核心结构和其相互关系

```
struct SM75_U32x1_LDSM_N {
  using SRegisters = uint128_t[1];
  using DRegisters = uint32_t[1];

  void copy(uint128_t const& smem_src, uint32_t& dst) {
    asm volatile ("ldmatrix.sync. ...\n");
  }
};
```

5.5 Copy_Traits

traits 补充了 CopyOperation 的信息，如其提供了执行 operation 所需要的线程数，源数据和目标数据的 Layout 排布情况，其描述了线程和数据的存放关系，即通过线程号和寄存器号可以得到数据的逻辑位置，同时提供 RefLayout 供线程级的数据拆分时实现 retile 能力，具体的定义如下，

```
struct Copy_Traits<SM75_U32x1_LDSM_N> {
  // Logical thread id to thread idx (warp)
  using ThrID = Layout<_32>;

  // Map from (src-thr,src-val) to bit
  using SrcLayout = Layout<Shape <Shape <  _8,_4>,_128>,
                           Stride<Stride<_128,_0>,  _1>>;
  // Map from (dst-thr,dst-val) to bit
  using DstLayout = Layout<Shape <_32,_32>,
                           Stride<_32, _1>>;

  // Reference map from (thr,val) to bit
  using RefLayout = DstLayout;
};
```

5.6 Copy_Atom

Atom 将 Operation 和 Traits 进行封装和抽象，定义了内部数据类型，供形成 TiledCopy 和后续的 ThrCopy 分解任务时提取信息，如其继承来自 Traits 的线程情况和数据 Layout 情况，提供 call 方法实现对底层指令的调用入口，

```
struct Copy_Atom<Copy_Traits<Args...>, T>
  : Copy_Traits<Args...>
{
  using Traits = Copy_Traits<Args...>;

  // Bit and Thr layouts from the Copy_Traits
  using ThrID        = typename Traits::ThrID;
  using BitLayoutSrc = typename Traits::SrcLayout;
  using BitLayoutDst = typename Traits::DstLayout;
  using BitLayoutRef = typename Traits::RefLayout;

  using ValType = T;

  void call(Tensor<TS,SLayout> const& src, Tensor<TD,DLayout>& dst);
};
```

5.7 TiledCopy

tiled 抽象通过对 Atom 能力进行重复得到更大的块的拷贝能力，对 Atom 的重复可以通过提供线程-存储的 Layout 来提供，也可以直接通过提供 Atom 能力和 MMA 中的 tiled_mma 实现，如`make_tiled_copy_A/B/C`，因为 MMA

已经提供了计算D = AxB + C时所需要的数据划分能力，当然这些函数是针对寄存器表达能力的，具体的模版参数和形参如下。除了描述对 Atom 的重复方式外，TiledCopy 提供的核心函数是get_slice和get_thread_slice，其可以实现将逻辑 Tensor 的拷贝能力根据线程的 id 得到每一个线程的 Layout 描述的拷贝任务，所以以上两个函数的返回对象为 ThrCopy：

```
template <class Copy_Atom,
          class LayoutCopy_TV,  // (tid,vid) -> coord   [Need not be 2D...]
          class ShapeTile_MN>   // coord space
struct TiledCopy : Copy_Atom {
  ThrCopy get_slice(ThrIdx const& thr_idx);
  ThrCopy get_thread_slice(ThrIdx const& thr_idx);
};

CUTE_HOST_DEVICE
auto make_tiled_copy_A(Copy_Atom<Args...> const& copy_atom,
                   TiledMMA           const& tiled_mma)
```

5.8 ThrCopy

thread copy 是线程级别的拷贝的抽象，其通过 TiledCopy 调用get_slice方法而得到，其核心函数为partition_S/D和retile_S/D，其中 S 和 D 分别表示 source 和 destination，partition 表示对一个大的逻辑 Tensor 进行划分得到当前线程的拷贝所需要的源 Tensor 和目标 Tensor，而 retile 系列的函数表示其输入的数据已经是当前的线程的私有的数据了，但是其可能不满足拷贝所要求的形状，需要将其变换到拷贝所支持的形状，形式如下代码：

```
template <class TiledCopy, class ThrIdx>
struct ThrCopy {
 auto partition_S(Tensor&& stensor);
 auto partition_D(Tensor&& dtensor);
 auto retile_S(Tensor&& stensor);
 auto retile_D(Tensor&& stensor);
};
```

5.9 cute::copy

copy 函数是拷贝的实际执行函数，调用该函数会触发线程级别的拷贝的发生，完成线程指令的执行，实现 src 到 dst 的数据拷贝指令，实现逻辑上的D = S。我们用块状逻辑对数据进行拷贝的时候可能遇到边界处理的情况，这时可以通过 copy_if 实现对某些数据拷贝的 mask，从而避免非法的数据访问，其函数原型如下，

```
void copy(TiledCopy const& copy, Tensor const& src, Tensor& dst);
void copy_if(TiledCopy const& copy, PrdTensor const& pred, Tensor const& src, Tensor& dst);
```

cute::copy 和其他组件的相互关系如表格所示。

功能	Copy
指令 + 存储类型	CopyOperation
逻辑类型和形状要求	Copy_Traits
原子能力	Copy_Atom
块状能力 (多个原子能力)	TiledCopy
线程级能力	ThrCopy
数据拆分 API	ThrCopy::partition_S/D(), ThrCopy::retile_S/D()
触发功能执行	cute::copy(tiled_copy, thr_s, thr_d);

5.10 总结

cute 提供了 Copy 能力来完成实现`D = S`Tensor 的搬运，其通过对指令、适配层、原子能力、块状能力和线程级别的抽象分别形成了数据结构`CopyOperation`、`Copy_Traits`、`Copy_Atom`、`TiledCopy`、`ThrCopy`和`cute::copy`，在这些抽象的帮助下，我们可以实现逻辑 Tensor 由一个存储单元向另一个存储单元的拷贝，而不用过多的关注指令细节。这种能力和 MMA 一道共同构成 cute 下矩阵乘法的基础。在 Copy 和 MMA 抽象的帮助下我们可以在逻辑层次构造数据的搬运`D = S`和矩阵乘法`D = A x B + C`。

至此我们已经讲解了 cute 中的 Layout、Tensor、MMA、Copy 抽象，我们已经可以完成一个 GEMM（GEneral Matrix Multiplaction）问题了。通过 Tensor 和 Layout 抽象我们可以实现对计算矩阵的分块；基于 Copy 抽象，我们可以完成块状矩阵 A、B 数据从 global 内存到寄存器的加载；通过 MMA 抽象我们可以利用 Tensor Core 完成寄存器上小块矩阵的乘法运算；再次通过 Copy 抽象，我们可以将寄存器上的结果拷贝到 global 内存，完成完整的 GEMM 运算。下一篇文章我们将利用 Layout、Tensor、MMA、Copy 能力完成一个简单的矩阵乘法。

5.11 参考

- https://docs.nvidia.com/cuda/parallel-thread-execution/index.html#warp-level-matrix-instructions-for-mma
- https://github.com/NVIDIA/cutlass/blob/main/include/cute/algorithm/copy.hpp
- tensorcore 中 ldmatrix 指令的优势是什么？[5]

[5] https://www.zhihu.com/question/600927104/answer/3029266372

第六章　cute 之简单 GEMM 实现

作者：reed（知乎 @reed-84-49，专栏「CUDA 高性能编程」）　原文：https://zhuanlan.zhihu.com/p/667521327

前面的文章我们对 cute 提供的核心数据结构和抽象进行了一系列的介绍，包括 Layout、Tensor、MMA 和 Copy。本文我们将利用这些概念和抽象实现一个简单版本的矩阵乘法。在介绍具体实现之前，我们首先介绍 BLAS 约定和深度学习对矩阵乘法的需求和约定，然后介绍 cuda 上的矩阵库 cuBLAS 和矩阵乘法任务到硬件执行单元的划分策略，最后我们从 Tensor 构建、切块、划分，执行、拷贝等方面结合代码实现具体介绍了矩阵乘法的实现，同时我们对该版本的实现选取了实际场景中遇到的问题规模进行了实验并对比了该实现的效率。

6.1　BLAS 约定和深度学习约定

在科学计算和数值分析领域，经常需要解决矩阵特征值、线性方程（代数）等问题，形成了如 EISPACK、LINPACK、LAPACK 数学求解库。这些库解决的是偏上层的数学问题，它们共同依赖了更加底层的、更基础的数学库 BLAS（Basic Linear Algebra Subprograms）。BLAS 总共分为三个层次：第一层是向量操作，如 $\boldsymbol{y}=\alpha\boldsymbol{x}+\boldsymbol{y}$ 计算问题；第二层向量-矩阵操作，如 $\boldsymbol{y}=\alpha\boldsymbol{A}\boldsymbol{x}+\beta\boldsymbol{y}$ 计算问题；第三层矩阵-矩阵操作，如 $\boldsymbol{C}=\alpha\boldsymbol{A}\boldsymbol{B}+\beta\boldsymbol{C}$ 。

$\boldsymbol{C}=\alpha\boldsymbol{A}\boldsymbol{B}+\beta\boldsymbol{C}$ 就是我们常说的 GEMM（GEneral Matrix Multiplication）问题，它也是 BLAS 第三层次要求解的问题。其中的广义来自于 α β 可以是任意数值，$\boldsymbol{A}$ 和 $\boldsymbol{B}$ 可以是转置的也可以是非转置的，数值精度方面可以是单精度浮点数、双精度浮点数等。BLAS 早期实现的时候采用的是数值计算领域的 Fortran 语言，在 Fortran 语言中二维数组是列主序的（列优先），所以在 BLAS 的 API 中针对 AB 矩阵的转置情况描述而言，列优先称为 Normal（简写为 N），行优先则称为 Transpose（简写为 T），值得注意的是转置与否是针对 A B 矩阵而言的，$\boldsymbol{C}$ $\boldsymbol{D}$ 矩阵必须是列优先的。同时为了区分不同的数值精度，BLAS 规定了不同的简写，如单精度浮点为 s（缩写自 single precision），双精度浮点为 d（缩写自 double precision），单精度复数为 c（缩写自 complex），双精度浮点复数记为 z。所以在附加上数据类型（如单精度浮点）之后函数名字就由 gemm 变为了 sgemm（single precision general matrix multiplication）。同时 BLAS 约定了矩阵的维度，表示为 $\boldsymbol{D}_{\boldsymbol{m},\boldsymbol{n}}=\alpha\boldsymbol{A}_{\boldsymbol{m},\boldsymbol{k}}\boldsymbol{B}_{\boldsymbol{k},\boldsymbol{n}}+\beta\boldsymbol{C}_{\boldsymbol{m},\boldsymbol{n}}$ ，

其中的 m，n 表示行列数目，k 为 reduce 轴上的数据维度。

深度学习中也有矩阵乘法所表达的网络层（如 Pytorch 中 Linear 层、Tensorflow 中的 Dense 层），但是神经网络中该层的公式可以表达为 $y=x\boldsymbol{A}+b$，其中 x 为 input，维度一般为（N, Cin）y 为 output 维度为（N, Cout），b 为 bias（一般是 boardcast 语义，此处我们忽略掉）。我们在忽略掉 bias 同时采用 BLAS 的矩阵形式描述，则其公式可以变更为 $\boldsymbol{D}=\boldsymbol{A}\boldsymbol{B}$ ，我们可以发现深度学习中需要的也是矩阵乘法的语义，和 BLAS 类似但是又不完全相同，不同点一深度学习中的公式没有 BLAS 公式中的 α，不同点二深度学习中公式没有 BLAS 中的加法右侧项，不同点三输出的 D 是行优先的而非 BLAS 中的列优先。除了公式层面的差异外，深度学习中的层为了计算效率会选用比传统的 BLAS 中的数据类型更低的精度如半精度和整数量化精度。

由于深度学习的输出要求的是行优先的，而 BLAS 中的约定中输出矩阵 D 是列优先的，因此当我们借助 BLAS 类函数完成深度学习的矩阵乘法时，需要对 BLAS 的公式进行转置变换，即

$\boldsymbol{D}^{\boldsymbol{T}}=\alpha\boldsymbol{B}^{\boldsymbol{T}}\boldsymbol{A}^{\boldsymbol{T}}+\beta\boldsymbol{C}^{\boldsymbol{T}}$ ，同时设置 α 为 1，β 为 0。由于转置矩阵 AB 的顺序也发生了变化。使用 BLAS 实现深度学习矩阵乘法的伪代码调用如下

```
// blas_gemm(D, A, B, trans_a, trans_b, m, n, k, alpha, beta) API specification;
//  D = alpha op(A) op(B) + beta C;
// op can be 'N'ormal or 'T'ranspose
// output:
//    D(m, n) column major
// input:
//    A(m, k) column major
//    B(k, n) column major
//    C(m, n) column major

// D(m, n) row major
// A(m, k) row major
```

```
// B(k, n) column major
void linear_layer(T *D, const T *A, const T *B, int m, int n, int k) {
  T alpha = 1;
  T beta = 0;
  blas_gemm(D, B, A, 'T', 'N', n, m, k, alpha, beta);
}
```

6.2 NVidia cuBLAS 类库

NVidia 使用 GPU 进行了 BLAS 加速实现，形成了 cuBLAS 加速库，cuBLAS 除了实现 BLAS 中的 API（Application Programming Interface）还扩展实现了多 Batch 支持、多 GPU 支持和针对深度学习加速的混合精度、低精度实现。对于我们深度学习中常用半精度（half precision）的 gemm 类似于之前的 sgemm，在 cuBLAS 中提供了 hgemm（half precision general matrix multiplication）。本质而言 cuBLAS 是一个大的 kernel 仓库，其中存储了 NVidia 针对不同数据类型，计算精度、矩阵维度、地址对齐等维度综合考虑的高度优化的 kernel。当用户调用其进行矩阵运算时，cuBLAS 会使用特定的方法从这些 kernel 中选择出一个针对该问题规模和约束的性能较好的实现版本。除了 cuBLAS 外，NVidia 同时针对不同功能需求和性能追求提供了 cuBLASLt 库等，如图 1 所示，其对比了 NVidia 提供的 BLAS 类库的 API 复杂度和融合后处理能力。

API	API complexity	Called from	Fusion support	Matrix sizes for maximum performance
cuBLAS (since CUDA 6.0)	Low	Host	None	Large (global memory)
cuBLASXt (since CUDA 6.0)	Low	Host	None	Very Large (multi-GPU, global memory)
cuBLASLt (since CUDA 10.1)	Medium	Host	Fixed set	Medium (global memory)
cuBLASDx (targeting 2023 EA)	Medium/High	Device	User ops	Small (shared memory)

知乎 @reed

Table 1. Comparison of various cuBLAS APIs. In general, the higher the API complexity the more suitable the API is for kernel developers.

图 6.1: Figure 1. NVidia 提供的 BLAS 类的库对比（引用自参考 5）

cuBLASLt 库是 cuBLAS 库的基础上提供针对人工智能和机器学习更灵活的应用，它允许用户指定更复杂的输入输出类型、计算类型（解决混合精度的多类型指定问题）等，同时它还提供了启发式算法，允许用户选择底层实现的性能更好的 kernel。后续的文章中我们的实验会利用 cuBLASLt 的这种选择 kernel 能力，选择候选 kernel 中最好的那一个作为我们的 baseline。本文中实验我们使用 cuBLAS 选定的 kernel 作为 baseline。

6.3 Tensor 表示

如图 2 所示，本文我们研究深度学习中常用的 C = AB 的矩阵乘法问题，其中矩阵 A、B 存储在 GPU 的全局内存中，输出 C 矩阵将会被存储在全局内存中；维度方面 A 为 m 行 k 列，B 为 k 行 n 列，输出 C 为 m 行 n 列；数据存储 Layout 方面，A 为行优先，B 为列优先，C 为行优先。数据类型方面 A、B、C 都为 16bit 的半精度浮点数，在 cuda 中表示为 half 类型（cute 封装为`cute::half_t`类型）。我们将矩阵 ABC 的信息形成如下表格，同时将 row/column major 表示为 stride 形式，填充上指针变量名称

矩阵	指针变量	存储位置	shape	stride	数据类型
A	Aptr	global memory	(m, k)	(k, 1)	half
B	Bptr	global memory	(k, n)	(1, k)	half
C	Cptr	global memory	(m, n)	(n, 1)	half

我们将 ABC 表示为 Tensor 形式，则可以写出如下 kernel 代码

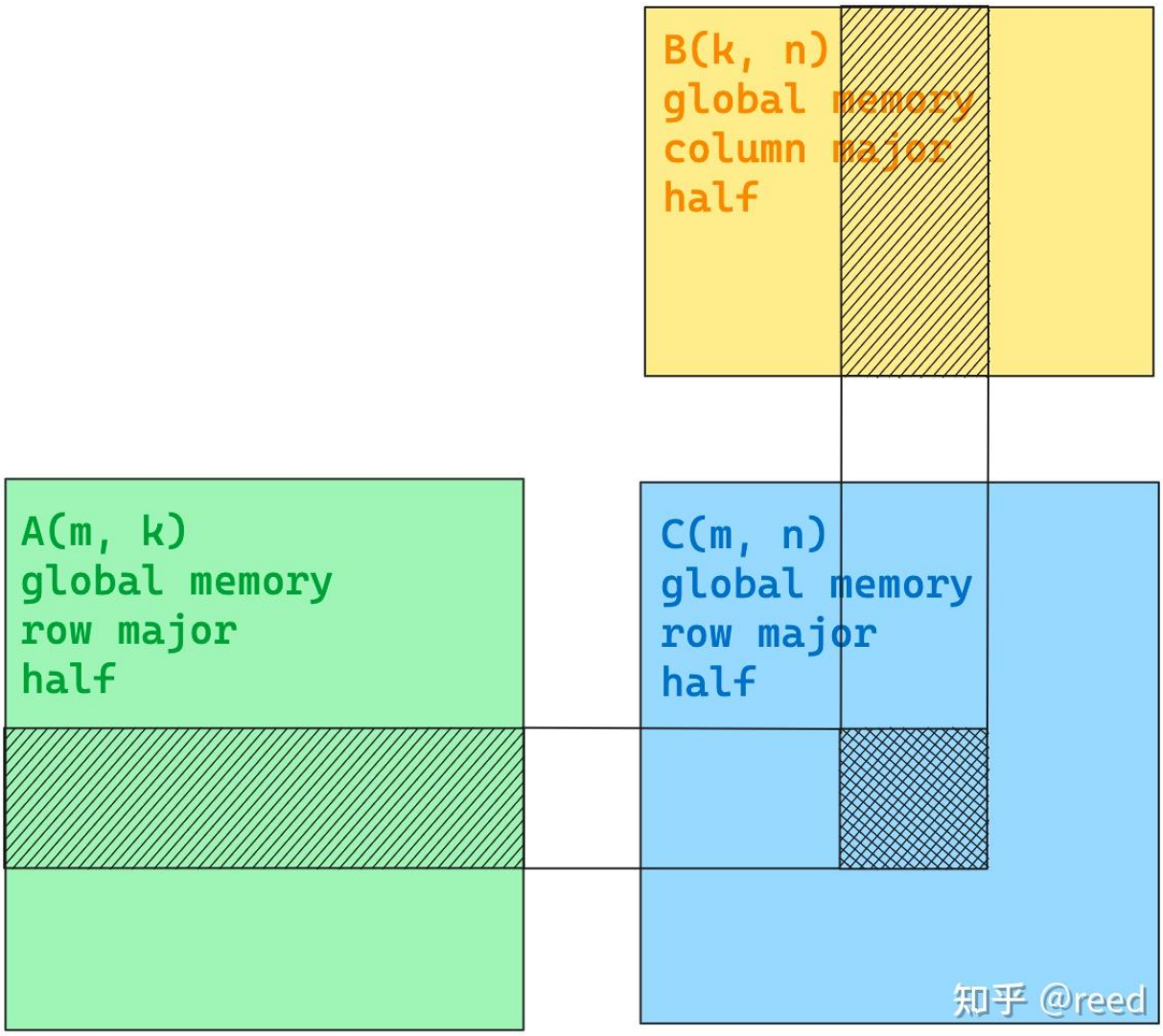


图 6.2: Figure 2. 矩阵乘法问题的 Tensor 表示及其属性

```cpp
template <typename T>
__global__ void gemm_simple(T *Cptr, const T *Aptr, const T *Bptr, int m, int n, int k) {
  Tensor A = make_tensor(make_gmem_ptr(Aptr), make_shape(m, k), make_stride(k, Int<1>{}));
  Tensor B = make_tensor(make_gmem_ptr(Bptr), make_shape(n, k), make_stride(k, Int<1>{}));
  Tensor C = make_tensor(make_gmem_ptr(Cptr), make_shape(m, n), make_stride(n, Int<1>{}));
}
```

其中`make_gmem_ptr()`函数用于标识 tensor 指针的存储层次，方便后续使用时能从指针中取出该指针的存储层次。同时，我们修改了 Tensor B 的矩阵表示形式把它的形状表示 (n, k)，对应的 stride 为 (k, 1)；这样后续的循环的时候我们就可以写成 reduce 的形式。为了编译时决策和优化，我们把 stride 中的连续维度 1 表示为编译时常量的形式，即`Int<1>{}`，这样后续对矩阵进行操作的时候如果需要用到 stride 的计算则可以利用编译时的决策和优化减少不必要的运行时运算。

6.4 以 C 矩阵为中心的任务划分策略

GPU 中有多个 SM（Stream Multiprocessor）我们编程时以 grid、block 的软件层级进行编程来利用这些 SM，在矩阵计算中，我们以输出矩阵 C 作为划分 thread block 的单元进行任务拆分，也就是说一个 thread block 完成 C 矩阵中的一个小块（TileC）的计算任务，如图 3 所示，我们定义矩阵 TileC 的大小为 kTileM、kTileN，分别表示小块矩阵包含元素的行数目和列数目，根据块状矩阵乘法的公式完成 TileC 的计算需要 A 矩阵中的绿色高亮部分和 B 矩阵的黄色高亮部分，它们的形状分别为 (kTileM, k) 和 (kTileN, k)。我们对 AB 矩阵的 k 轴按照 kTileK 的大小进行分块，则可以将 TileC 矩阵表达为 AB 矩阵块的点积运算（其中 $num_tile_k = k/kTileK$ 为 k 轴的分块数）

$$TileC = \sum_{itile=0}^{num_tile_k-1} TileA_{itile} TileB_{itile}^{T}$$

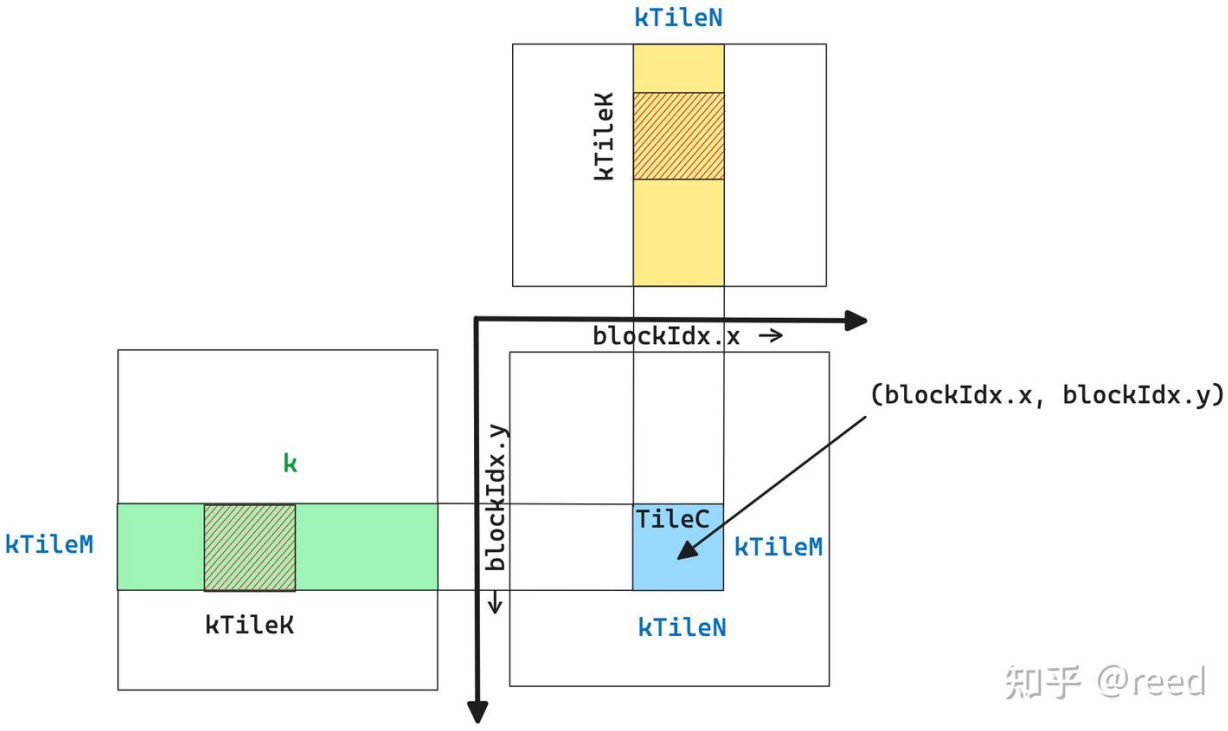


图 6.3: Figure 3. sliced-k 模式的 C 矩阵为中心的任务划分方法

如此，我们在 k 轴上移动 kTileK 得到 AB 上的小块 $TileA_{itile}$ 和 $TileB_{itile}$ 将他们相乘的积累加到 TileC 上，便可以得到 TileC 的计算结果。这种沿着 k 轴移动的策略称 sliced-k 方法。这样我们使用一个 block（坐标如图 blockIdx.x, blockIdx.y）便可以完成 C 矩阵中一个小块的完整的计算。通过 block 维度的扩展，如图所示的矩阵 C 中 M 轴方向的 blockIdx.y 和 N 轴方向的 blockIdx.x 的扩展，便可以完成整个 C 矩阵的计算。由此我们可以计算出完成整个 C 矩阵所需要的 grid 维度：`grid.x = N / kTileN, grid.y = M / kTileM`（此处暂时不考虑不可整除的情形）。根据上面的计算过程我们可以继续完善代码，

```cpp
template <typename T, int kTileM, int kTileN, int kTileK>
__global__ void gemm_simple(T *Cptr, const T *Aptr, const T *Bptr, int m, int n, int k) {
  Tensor A = make_tensor(make_gmem_ptr(Aptr), make_shape(m, k), make_stride(k, Int<1>{}));
  Tensor B = make_tensor(make_gmem_ptr(Bptr), make_shape(n, k), make_stride(k, Int<1>{}));
  Tensor C = make_tensor(make_gmem_ptr(Cptr), make_shape(m, n), make_stride(n, Int<1>{}));

  int ix = blockIdx.x;
  int iy = blockIdx.y;

  Tensor gA = local_tile(A, make_tile(Int<kTileM>{}, Int<kTileK>{}), make_coord(iy, _));
  Tensor gB = local_tile(B, make_tile(Int<kTileN>{}, Int<kTileK>{}), make_coord(ix, _));
  Tensor gC = local_tile(C, make_tile(Int<kTileM>{}, Int<kTileN>{}), make_coord(iy, ix));
}

int main() {
  ...
  dim3 grid(n / kTileN, m / kTileM);
  ...
}
```

如我们上面所述，我们在模版参数中给定划块的超参 kTileM、kTileN、kTileK，同时使用 Tensor 章节中介绍`local_tile`方法对矩阵进行固定大小的分块，同时通过指定坐标的的全 slice 方法，得到当前 thread block 要处理的 Tensor gA, gB, gC。和构造 Tensor A 时类似，我们在进行 Tensor 分块的时候也将编译时能确定的量进行了`Int`<>化，用以指示该维度信息是编译时常量，编译器可以在编译阶段完成必要的路径决策和优化计算，避免运行时的开销。同时我们在主函数中给出了 grid 的大小（如上代码）。值得注意的是，经过 local_tile 函数我们得到的 gA, gB, gC 的维度信息如下表格

Tensor	shape
gA	(kTileM, kTileK, num_tile_k)
gB	(kTileN, kTileK, num_tile_k)
gC	(kTileM, kTileN)

先选定 TileC，然后沿着 k 轴移动小块进行累加求和的策略为 sliced-k，它对于 m、n 维度较大的场景（m n 分块所需要的 block 数目足以填充所有的 SM）比较有效。对于 k 比较大，而 m、n 比较小的场景，由于 m、n 较小而我们根据 C 来划分 thread block，这时需要的 thread block 数目比较小，当这个数目无法填充所有的 SM 时，则存在很多 SM 无任务，而有任务的 SM 需却又需要循环多次的问题，这时候可以考虑将 k 轴拆分成多段，每一段都计算一个 TileC 结果，最后再通过额外的累加过程将多段的结果进行求和，这种模式的任务划分方法成为 split-k 方法。如图 4，把 k 拆分成两段，由不同的计算单元来完成不同段的计算，如此则得到多份 C，最后将多个 C 进行累加求和得到最终结果。该方法在特殊场景下有用，而且实现不困难，本文暂不实现该策略。

除了 sliced-k、split-k 策略在任务划分方面还有 stream-k 方法，stream-k 方法作者们指出 sliced-k 或者 split-k 方法都是静态的划分任务，在划分的任务数目和 SM 执行单元不能整除的时候，总会存在某轮（wave）计算中有 SM 空闲的问题。stream-k 则是抛弃以任务为中心的划分逻辑，而是变成了以计算资源为核心的分配任务方式，使得 SM 的任务量基本相当，如图 5，其展示了假设只有 4 个 SM 的情况下，不同任务划分逻辑的差异，其中 stream-k 对计算资源的利用效果最好，具体可以参考发表在 PPoPP'23 上的 poster。现阶段 cuBLAS 中的 kernel 依然多为 sliced-k 和 split-k 实现。本文暂不实现该策略。

6.5 TiledMMA：主机端选择指令，设备端将分块划分到线程

前面经过把 C++ pointer 封装成 Tensor，然后利用`local_tile`将 Tensor 划分成小块，我们便可以得到一个 thread block 需要处理的任务。这时，假设我们通过前面 MMA 章节构造了一个 TiledMMA 能力，则借助其方法我们便可以通过 ThrMMA 的 partition_A/B/C 方法实现对 TileA、TileB、TileC 的划分，通过`partition_fragment_A/B/C`便可以构造矩阵乘所需要的寄存器表示。通过`cute::gemm`方法便可以完成线程级别寄存器表示的矩阵乘法。具体的 kernel 代码为

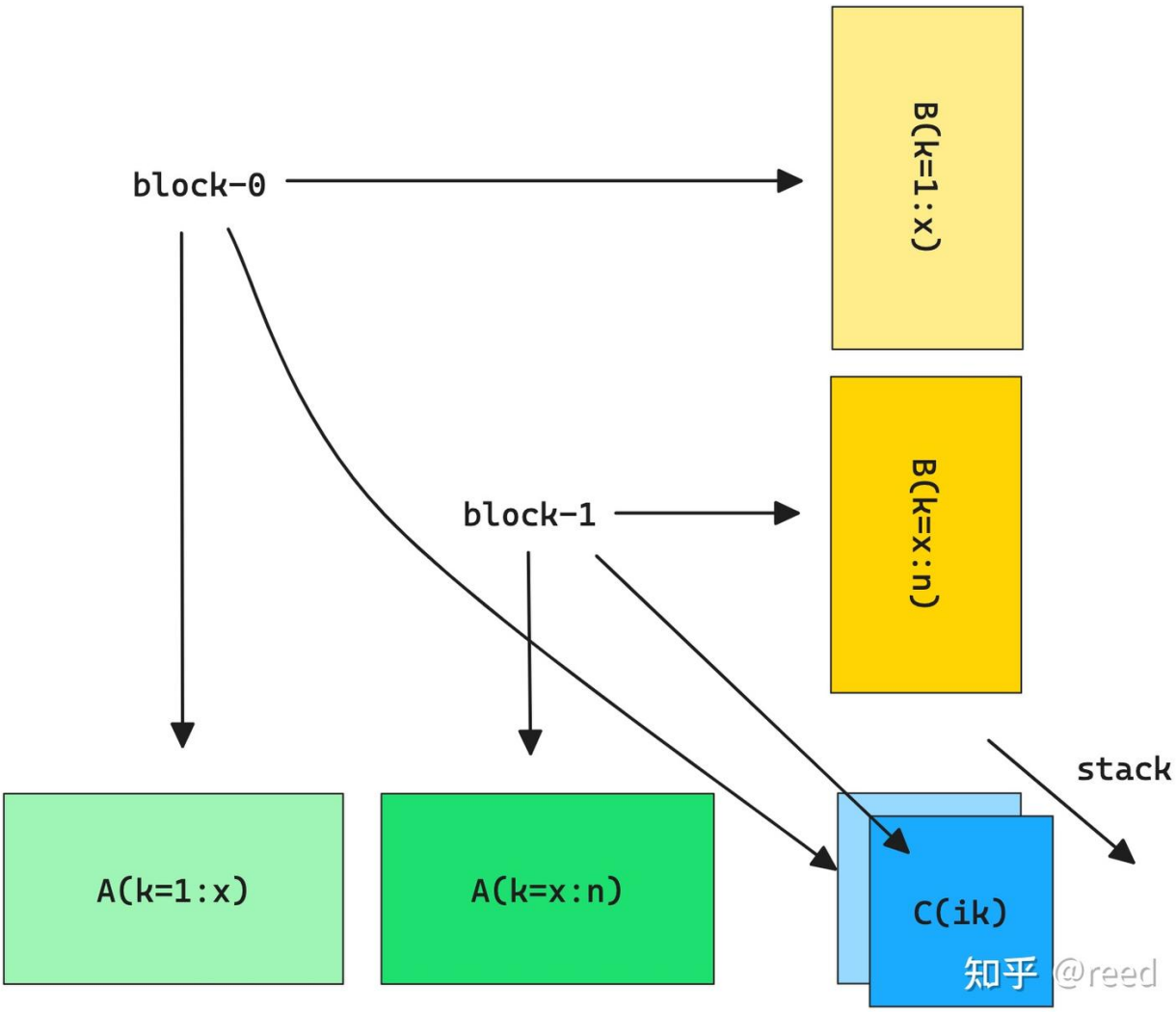


图 6.4: Figure 4. split-k 策略的计算逻辑

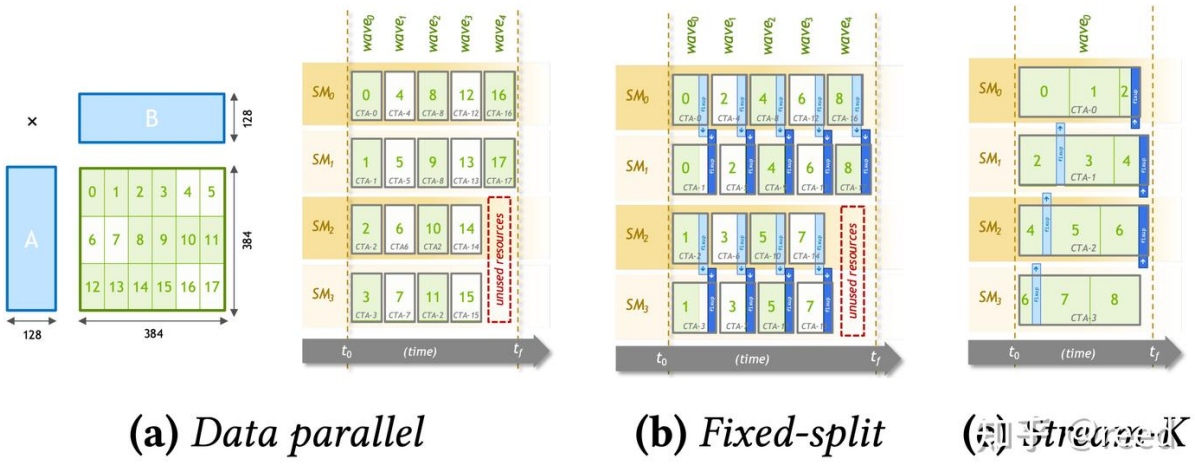


图 6.5: Figure 5. stream-k 策略任务划分逻辑（引用自 PPoPP23: Stream-K）

```cpp
template <typename T, int kTileM, int kTileN, int kTileK, typename TiledMMA>
__global__ void gemm_simple(T *Cptr, const T *Aptr, const T *Bptr, int m, int n, int k) {
  Tensor A = make_tensor(make_gmem_ptr(Aptr), make_shape(m, k), make_stride(k, Int<1>{}));
  Tensor B = make_tensor(make_gmem_ptr(Bptr), make_shape(n, k), make_stride(k, Int<1>{}));
  Tensor C = make_tensor(make_gmem_ptr(Cptr), make_shape(m, n), make_stride(n, Int<1>{}));

  int ix = blockIdx.x;
  int iy = blockIdx.y;

  Tensor gA = local_tile(A, make_tile(Int<kTileM>{}, Int<kTileK>{}), make_coord(iy, _));
  Tensor gB = local_tile(B, make_tile(Int<kTileN>{}, Int<kTileK>{}), make_coord(ix, _));
  Tensor gC = local_tile(C, make_tile(Int<kTileM>{}, Int<kTileN>{}), make_coord(iy, ix));
  //  gA(kTileM, kTileK, num_tile_k)
  //  gB(kTileN, kTileK, num_tile_k)
  //  gC(kTileM, kTileN)

  TiledMMA tiled_mma;
  auto thr_mma = tiled_mma.get_slice(threadIdx.x);
  auto tAgA = thr_mma.partition_A(gA);  // (MMA, MMA_M, MMA_K, num_tile_k)
  auto tBgB = thr_mma.partition_B(gB);  // (MMA, MMA_N, MMA_K, num_tile_k)
  auto tCgC = thr_mma.partition_C(gC);  // (MMA, MMA_M, MMA_N)

  auto tArA = thr_mma.partition_fragment_A(gA(_, _, 0));  // (MMA, MMA_M, MMA_K)
  auto tBrB = thr_mma.partition_fragment_B(gB(_, _, 0));  // (MMA, MMA_N, MMA_K)
  auto tCrC = thr_mma.partition_fragment_C(gC(_, _));     // (MMA, MMA_M, MMA_N)

  clear(tCrC);
}

int main() {
  ...
  using mma_op = SM80_16x8x16_F16F16F16F16_TN;
  using mma_traits = MMA_Traits<mma_op>;
  using mma_atom = MMA_Atom<mma_traits>;

  auto MMA = decltype(make_tiled_mma(mma_atom{},
                      make_layout(Shape<_2, _2, _1>{}),
                      make_layout(Shape<_1, _2, _1>{})));
  dim3 block(size(MMA{}));
  dim3 grid(n / kTileN, m / kTileM);
  ...
}
```

其中**get_slice**函数能够将 TiledMMA 能力根据具体的线程 id 得到每一个线程所需要的 layout 信息。利用 partition 函数实现对 gA gB gC 的矩阵在线程级别的分解，经过 partition 后得到的维度信息为 (MMA, MMA_M, MMA_K, num_tile_k)，其中 MMA 表示 TiledMMA 一次能做的矩阵运算所需要的数据，MMA_M、MMA_K 表示 (kTileM, kTileK) 按照 TiledMMA 能力去划分的时候，M 方向和 K 方向需要重复多少次才能完成该矩阵乘法，即 M K 方向需要循环多少遍 TiledMMA 才能完成计算，而 num_tile_k 则为将 gA 的维度自然带入下来。也就是说 partition_A 的逻辑本质上是将 Tensor 的前两维进行划分，划分后得到一个三维的结果，其中第一维度表示 TiledMMA 单次所能处理的数据，接下来的两维表示两个方向上的重复，而如果被 partition 的维度再多，则后续的维度自然的继承下来即可。partition_fragment类函数和前面的类似，只是其返回的是寄存器声明。同时我们注意到对于**partition_fragment_A/B**，我们输入的 gA 时保留了前两个维度，对第三个维度我们选择了位置 0，这就等价于传递给**partition_fragment_A/B**的是形状为 (kTileM, kTileK)、(kTileN, kTileK) 的 Tensor。自然的其返回的结果形状也是类似的：第一维表示 TiledMMA 单次能力所需的数据，接下来的两维表示对 TiledMMA 能力在 M 方向和 K 方向的重复次数。在得到 TileC 的寄存器表示后，我们使用 clear 方法将其初始化为 0，以备后面进行矩阵乘法时的累加操作。

在 main 函数中，我们选择使用 Ampere 架构提供的 16x8x16 的 Tensor Core 矩阵乘法指令，数据精度和计算精度都为 fp16。然后通过**MMA_Traits**得到 mma_traits，继续将 traits 转换成**MMA_Atom**。我们知道 SM80 的 Tensor Core 执行

是 warp level 的，也就是说这个 MMA_Atom 是 32 个线程，我们对MMA_Atom能力通过增加线程的方式进行 M、N 方向的重复，同时我们让 B 矩阵 C 矩阵使用更多寄存器在 N 方向扩展 2 次，得到 main 函数中的 MMA 类型。这样，我们便可以得到 TiledMMA 需要 32x2x2 = 128 线程，其能处理的矩阵的大小: M = 16 x 2 x 1 = 32, N = 8 x 2 x 2 = 32, K = 16 x 1 x 1 = 16, 即 TiledMMA 能处理的 MNK 为 32x32x16。

6.6 Loop Over K

有了 TiledMMA 的数据划分之后，我们调用cute::gemm即可以完成 C[kTileM, kTileN] =A[kTileM, kTileK] B[kTileN, kTileK]利用 Tensor Core 进行矩阵乘的能力，我们对其进行这个块沿着 k 方向循环便可以得到最终的矩阵计算结果，实现如下

```
template <typename T, int kTileM, int kTileN, int kTileK, typename TiledMMA>
__global__ void gemm_simple(T *Cptr, const T *Aptr, const T *Bptr, int m, int n, int k) {
  Tensor A = make_tensor(make_gmem_ptr(Aptr), make_shape(m, k), make_stride(k, Int<1>{}));
  Tensor B = make_tensor(make_gmem_ptr(Bptr), make_shape(n, k), make_stride(k, Int<1>{}));
  Tensor C = make_tensor(make_gmem_ptr(Cptr), make_shape(m, n), make_stride(n, Int<1>{}));

  int ix = blockIdx.x;
  int iy = blockIdx.y;

  Tensor gA = local_tile(A, make_tile(Int<kTileM>{}, Int<kTileK>{}), make_coord(iy, _));
  Tensor gB = local_tile(B, make_tile(Int<kTileN>{}, Int<kTileK>{}), make_coord(ix, _));
  Tensor gC = local_tile(C, make_tile(Int<kTileM>{}, Int<kTileN>{}), make_coord(iy, ix));
  //  gA(kTileM, kTileK, num_tile_k)
  //  gB(kTileN, kTileK, num_tile_k)
  //  gC(kTileM, kTileN)

  TiledMMA tiled_mma;
  auto thr_mma = tiled_mma.get_slice(threadIdx.x);
  auto tAgA = thr_mma.partition_A(gA);  // (MMA, MMA_M, MMA_K, num_tile_k)
  auto tBgB = thr_mma.partition_B(gB);  // (MMA, MMA_N, MMA_K, num_tile_k)
  auto tCgC = thr_mma.partition_C(gC);  // (MMA, MMA_M, MMA_N)

  auto tArA = thr_mma.partition_fragment_A(gA(_, _, 0));  // (MMA, MMA_M, MMA_K)
  auto tBrB = thr_mma.partition_fragment_B(gB(_, _, 0));  // (MMA, MMA_N, MMA_K)
  auto tCrC = thr_mma.partition_fragment_C(gC(_, _));     // (MMA, MMA_M, MMA_N)

  clear(tCrC);

  int num_tile_k = size<2>(gA);
#pragma unroll 1
  for(int itile = 0; itile < num_tile_k; ++itile) {
    cute::copy(tAgA(_, _, _, itile), tArA);
    cute::copy(tBgB(_, _, _, itile), tBrB);

    cute::gemm(tiled_mma, tCrC, tArA, tBrB, tCrC);
  }

  cute::copy(tCrC, tCgC);
}

int main() {
  ...
  using mma_op = SM80_16x8x16_F16F16F16F16_TN;
  using mma_traits = MMA_Traits<mma_op>;
  using mma_atom = MMA_Atom<mma_traits>;

  auto MMA = decltype(make_tiled_mma(mma_atom{},
                     make_layout(Shape<_2, _2, _1>{}),
```

```
                      make_layout(Shape<_1, _2, _1>{})));
  dim3 block(size(MMA{}));
  dim3 grid(n / kTileN, m / kTileM);
  gemm_simple<T, kTileM, kTileN, kTileK, MMA>(Cptr, Aptr, Bptr, m, n, k);
  ...
}
```

通过 gA 我们可以获取 k 方向的 tile 需要循环的次数 num_tile_k，然后利用 sliced-k 模式循环 k 方向的 tile，通过`cute::copy`实现全局内存到寄存器的直接 copy，数据拷贝到寄存器后通过`cute::gemm`完成 Tile 块的矩阵乘法。在循环结束后再次利用`cute::copy`实现寄存器到全局内存的写出。`cute::copy`在不指定`Copy_Atom`时采用 UniversalCopy 实现，即简单的 cuda 语言层面的 T d = s 形式。到此，我们已经可以使用 TiledMMA 和 cute::copy 实现简单的矩阵乘法。

6.7　实验性能对比

我们选定了一个实际深度学习推理中会使用到的矩阵的大小 (M, N, K) = (81920, 256, 256)，选定 (kTileM, kTileN, kTileK) = (128, 128, 32)，在 RTX 3090 显卡上进行了相应的实验，其中操作系统版本为 Ubuntu 20.04.6 LTS，CUDA 驱动版本为 535.113.01，编译器 NVCC 版本为 V11.7.64。我们实现的 kernel 和 cuBLAS 选定的 kernel 分别调用 100 次取 kernel 平均的耗时，得到如下表格，实现及测试代码见https://github.com/reed-lau/cute-gemm。

方法	平均耗时 (us)（实验 100 次取平均）
gemm_simple	218
`ampere_h16816gemm_256x128_ldg8_stages_32x3_tn`	138

我们看到 cuBLAS 中选择的 kernel 为 ampere 架构的实现，我们的简单实现版本比 cuBLAS 中的实现还有很大差距，在后续的文章中我们会通过在该实现上进行优化最终达到甚至超越这个实现效率。

6.8　总结

本文我们介绍了 cuBLAS 的矩阵乘法约定和深度学习中的约定，介绍了矩阵任务向硬件模型划分任务的常用方法，最后通过 cute 提供的 Layout、Tensor，TiledMMA 和 Copy 功能实现了简单的 gemm，针对特定的规格进行了性能对比实验，结果显示该简单实现可以达到 cuBLAS 的 60% 的效率。后续我们会在该版本基础上进行进一步优化，最终实现 SOTA 的 gemm 能力。

6.9　参考

http://history.siam.org/pdfs2/Dongarra_%20returned_SIAM_copy.pdf

https://pytorch.org/docs/stable/generated/torch.nn.Linear.html

https://www.tensorflow.org/api_docs/python/tf/keras/layers/Dense

https://developer.nvidia.com/cublas

https://developer.nvidia.com/blog/new-cublas-12-0-features-and-matrix-multiplication-performance-on-nvidia-hopper-gpus

https://dl.acm.org/doi/10.1145/3572848.3577479

第七章　cute 之 Swizzle

作者：reed（知乎 @reed-84-49，专栏「CUDA 高性能编程」）　原文：https://zhuanlan.zhihu.com/p/671419093

前面的文章我们介绍了 GEMM 中的流水线技术[1]，流水线的核心是将拷贝[2]和计算[3]并行或者说是将数据加载隐藏在计算过程中。矩阵计算中的数据加载是从全局内存到共享内存然后到寄存器。共享内存作为中间的媒介可以减少矩阵计算时对全局内存的访问数据量，从而提升计算访存比。共享内存为了提升访问的并行性采用多 bank 结构，这也造成了编程时的困难，cute 通过提供 swizzle 抽象简化了逻辑空间和多 bank 存储空间的映射的复杂度。本文首先介绍了 shared memory 的多 bank 存储结构，之后介绍了矩阵计算中的 ldmatrix 指令对逻辑空间和存储空间的要求，再次我们介绍了异或运算的特性和 Swizzle 抽象，最后我们简单介绍了 Thread Block Swizzle 和对本文进行了总结。

7.1　局部性原理和 Shared Memory

局部性原理（Principle of Locality）是计算机科学的基石之一，它包括空间局部性和时间局部性。其中的空间局部性（也叫数据局部性）是指对数据的使用会限制在一个相对临近的存储空间中。Cache 是针对空间局部性的很好的解决方案，但是 Cache 的数据更新和替换逻辑一般会实现在硬件中，表现为不可编程。在 SIMT（single Instruction Multiple Thread）编程模式下，线程私有的寄存器提供了线程级别的存储能力，有时线程间需要交换一些数据来协同的完成特定任务，为了追求更好的数据局部性和实现线程间的数据共享，提供可编程的、线程间可共享的 Cache 就显得尤其重要。CUDA 在硬件 SM（Stream Multiprocessor）上提供了 Shared Memory 存储机构，同时软件上通过提供了相应的读写接口和同步源语言来实现其读写、同步和可见性，这样线程块内的线程便可以通过共享内存完成数据共享，同时对于线程块公共使用的数据便可以存储在其中达到线程块级别的可编程的数据局部性。

由于 Shared Memory 是为线程块服务的，所以其必须能支持线程块内的线程并行的对其进行访问（包含数据读取和写入），为了保障 Shared Memory 存储结构在多线程并发读写下的效率（更低的 Latency 和更高的 Throughput），其硬件被实现为多 bank 的模式，每个 bank 都是可以独立寻址的存储空间，bank 之间可以并行的读写数据，相互之间不会影响。在 NVidia 的架构中，shared memory 包含 32 个 bank，bank 中可寻址的基本单元为 4byte，如图 1 所示，每个 bank 为黑框所包含的单元，用户看到的地址空间为箭头所示的方向，即相邻的 4byte 占用不同的 bank。如图 2，当 32 个线程同时访问 32 个不同的 bank 时，各个 bank 是并行执行的，其效率是最高的，即 32 个线程并发的访问 32 个 bank 中不同颜色的单元，是可以并行的，值得注意的是其中的线程编号（如图 2 中的 T0 所示）和 bank 中的行位置并没有连续性要求。如图 3，如果某两个线程 T0、T2 要同时访问相同 bank-2 的不同地址，则这两次访问会被排队执行，即先访问该 bank 的一个地址，然后再访问第二个地址，这样两次访问在发射任务维度上（产生访问请求指令）时间维度上是并行的，但是在真正 bank 读写数据在时间维度上是串行的。这就是所谓的 bank conflict。由于一个 bank 上有两次冲突，这种情况称为二路冲突（two-way conflict）。

为了减少指令数，我们在进行 kernel 优化时会采用向量化的读写指令（也叫大字长读写），如以 128bit 的形式读写共享内存，此时线程需要访问的单位数据量为 16byte，32 个线程需要访问的数据量为 16byte x 32 = 512byte。完整的 512byte 需要 4 个 phase 才能完成访问，第一 phase，T0-T7 无 bank conflict 的访问所有 bank，第二 phase，T8-T15 无 bank conflict 的访问所有 bank，第三 phase，T16-T23 无 bank conflict 的访问所有 bank，第四 phase，T24-T31 无 bank conflict 的访问所有的 bank。这种情况也可以看作是：shared memory 基本单元为 16byte，总 bank 数为 8，冲突与否的分析不再是 32 线程，而变成 4 个 phase 中的不同线程。如果采用 64bit 的访问形式，则相应的基本单元可以看作是 8byte，总 bank 数目为 16，冲突与否的条件变成两个 phase 内的线程是否冲突。整体上 shared memory 空间可以看作二维存储空间，其中列方向表示 bank 情况，行方向表示自由定义的大小。值得注意的是冲突与否是通过内存访问事务级别来判定的，具体的可以参考 NVidia 开发者论坛的讨论[4]。

[1] https://zhuanlan.zhihu.com/p/665082713
[2] https://zhuanlan.zhihu.com/p/666232173
[3] https://zhuanlan.zhihu.com/p/663092747
[4] https://forums.developer.nvidia.com/t/how-to-understand-the-bank-conflict-of-shared-mem/260900

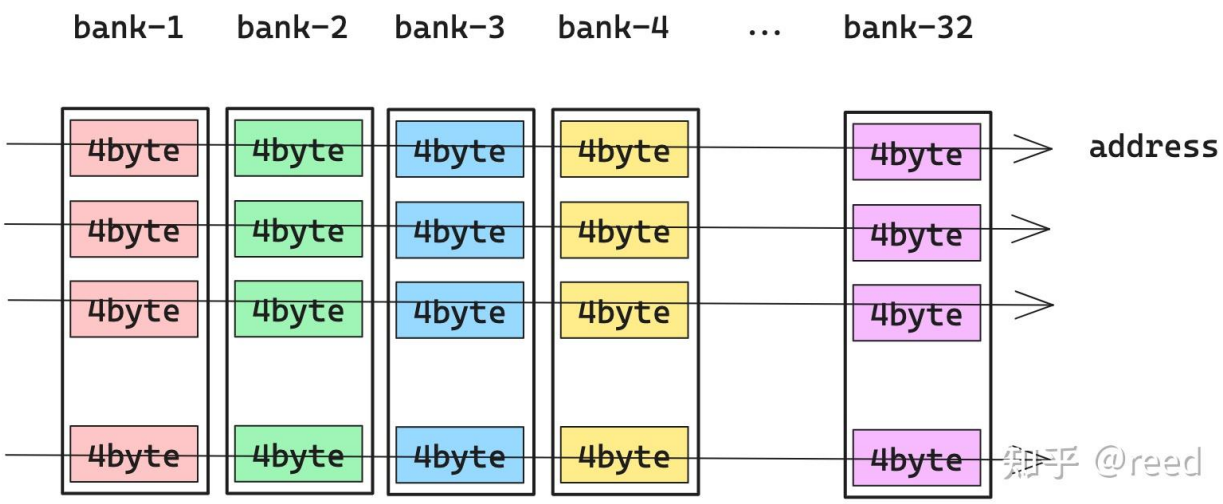


图 7.1: Figure 1. 共享内存 bank 结构和地址连续方向

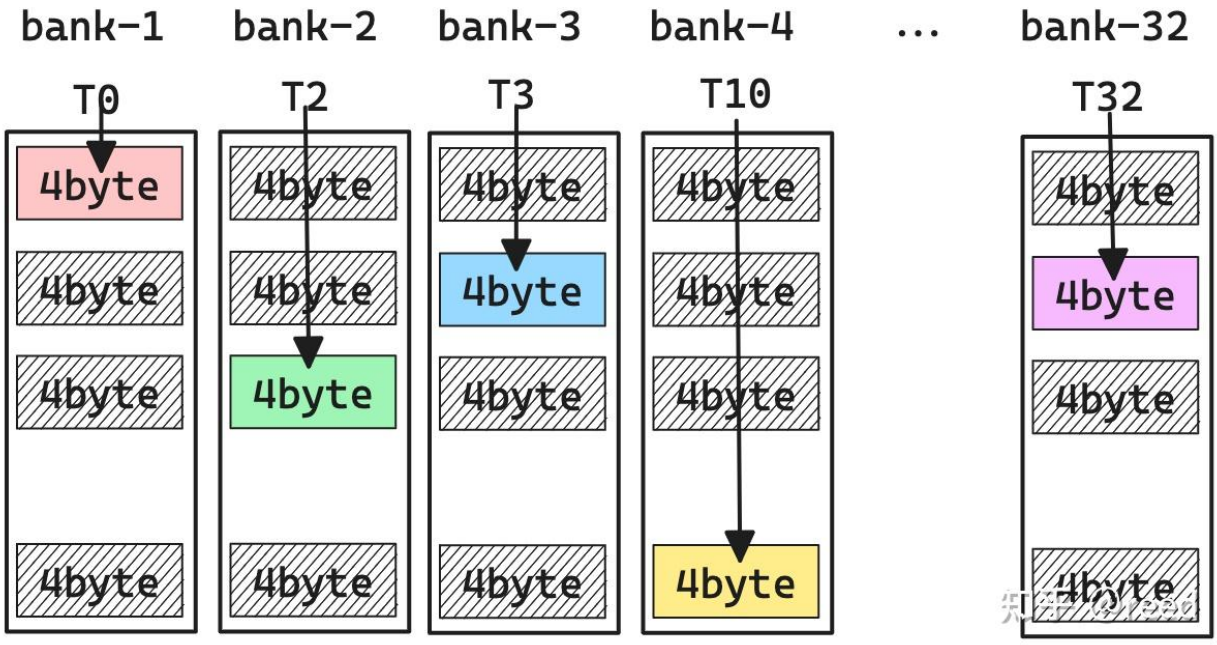


图 7.2: Figure 2. 无 bank conflict 的共享内存访问模式

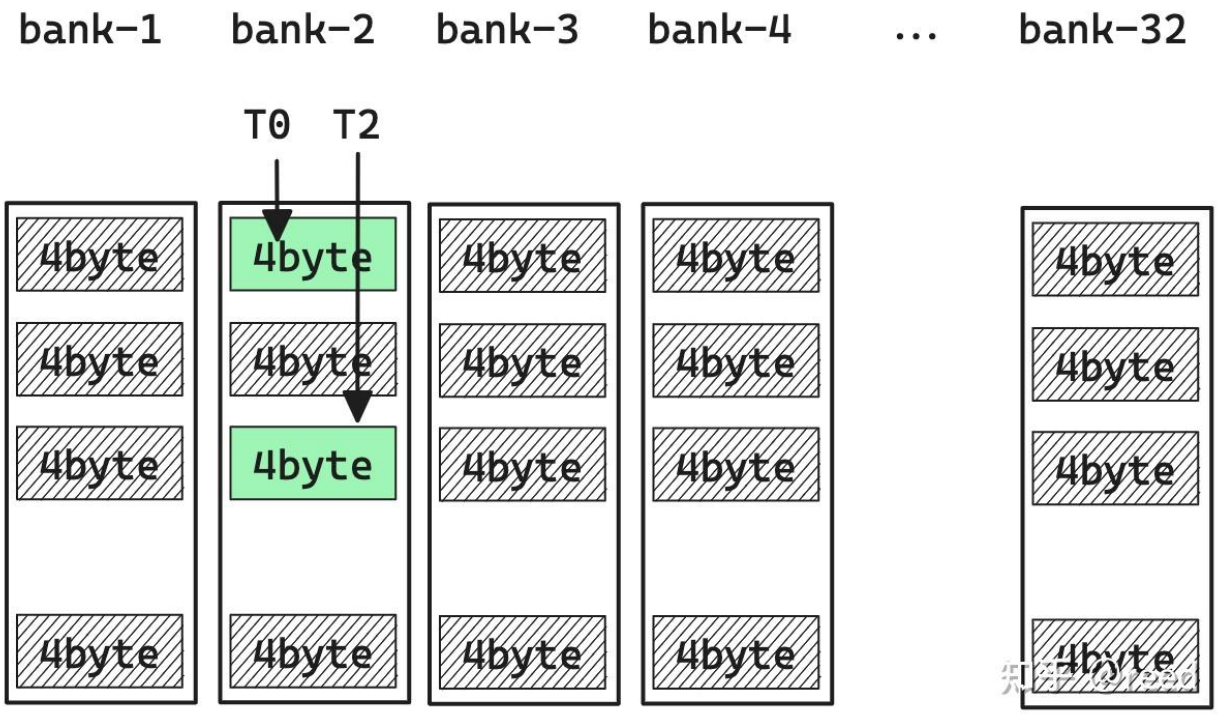


图 7.3: Figure 3. 两路冲突的共享内存访问模式

7.2 共享内存读取（ldmatrix 指令）

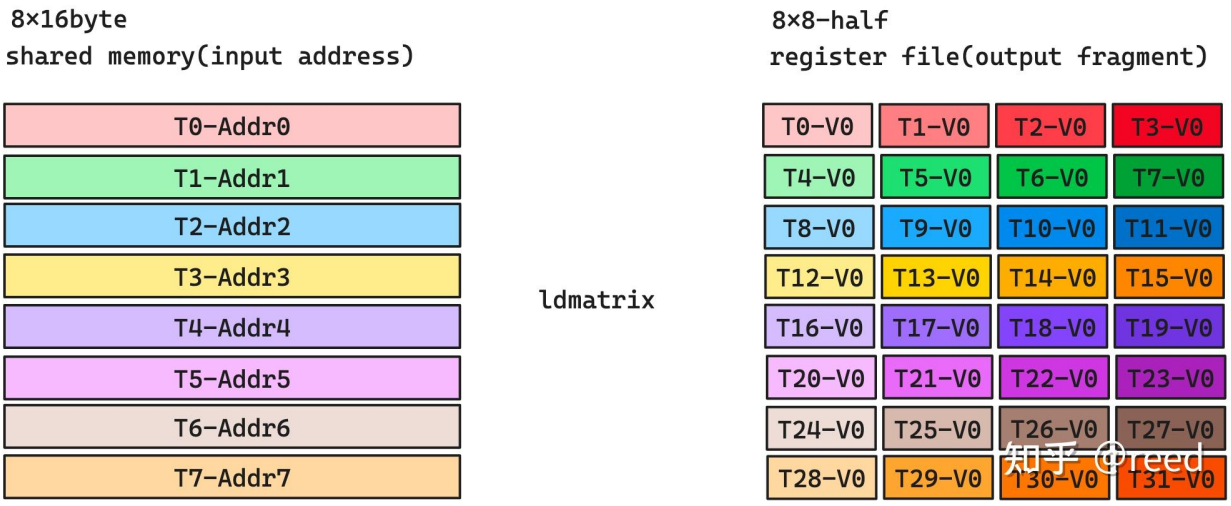


图 7.4: Figure 4. ldmatrix 输入和输出数据

在 GEMM 流水线中，利用 Tensor Core 可以完成特定规格的矩阵计算乘计算（如 $D_{16\times8} = A_{16\times16}B_{16\times8} + C_{16\times8}$ ），其中矩阵数据 A、B、C、D 是通过 warp 内的所有线程提供一部分寄存器共同表示的。如图 4 中右侧的 register file 所示，其表示 32 个线程 T0-T31 每一个线程提供一个寄存器 V0（4byte），共同表示形状为 8x8 half 类型的矩阵块，多个 8x8 的块可以构成更大的 16x16，16x8 的块。前序文章已经介绍过，这部分数据可以利用 ldmatrix 指令通过 warp level 实现。针对一个 8x8-half 的寄存器表示的作为输出的矩阵块，ldmatrix 其输入要求为 8 个 shared memory 地址，每个地址指向一个 16byte 共享内存中的数据，其中 T0-Addr0 指向的 16byte 数据经过 ldmatrix 会被分派到 T0-T3 的 V0 寄存器中。T1-Addr1 指向的数据会被分派到 T4-T7 的 V0 寄存器中。通过 ldmatrix 指令便可以实现矩阵数据从共享内存到寄存器到加载，前面的介绍我们知道共享内存是有 bank 结构的，并且按照 16byte 的形式进行读取，所以 T0-T7 读取该数据时会被作为一个独立的 phase，这就要求所有的 16byte 表示的 8 个数据必须分布在不同的 bank，才能确保读取共享内存数据时不产生 bank conflict。图 5 展示了一种 ldmatrix 时无 bank conflict 的布局形式。

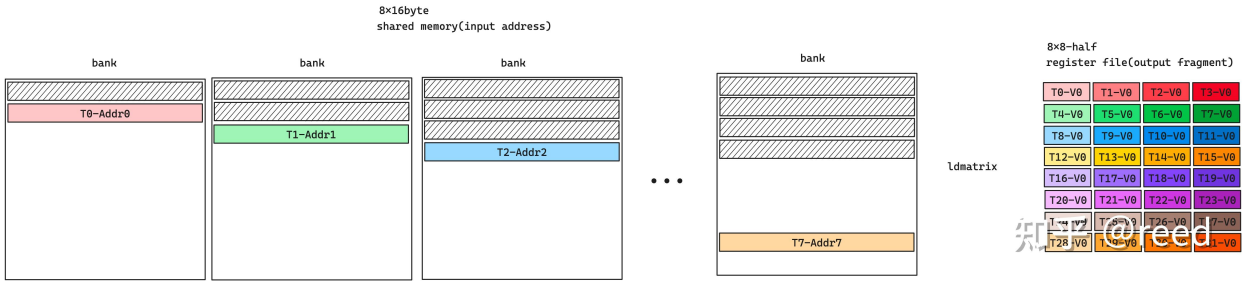


图 7.5: Figure 5. ldmatrix 指令无 bank conflict 时的 bank 占用情况

从数学逻辑上看，8x8-half 的寄存器数据表示连续的矩阵块，共享 8x16byte 的内存也有很好空间局部性的矩阵块，但是从共享内存的存储逻辑上看，为了避免读取时的 bank 冲突，其必须分配在不同的 bank 中。所以其横向位置在共享内存排列时不是简单的逻辑向下排列，而需要横向（bank 方向）错开来避免 bank conflict。

7.3　Shared Memory 写入

在 GEMM 流水线中数据的起点是全局内存，如图 6 所示，矩阵乘法所需要的寄存器数据来自于共享内存，共享内存数据来自于全局内存，数学逻辑上寄存器表示的数学空间和全局内存的位置是对应的。但是共享内存由于有 bank 的存在，其块状数据在共享内存存储时不是简单行列排列。需要根据 ldmatrix 的要求来避免冲突，这样从全局内存读取数据后写入共享内存时，也需要按照逻辑要求进行存储空间的映射。同时在全局内存向共享内存加载时，为了提升全局内存的读取效率需要考虑合并访存和 L2 Cache line 的情况，一般会要求其线程沿着线性地址的空间顺序排列，如图 T0->Tn 所示。也就是说我们在做全局内存到共享内存数据搬运时，思考模型是逻辑空间，而执行时需要考虑存储空间以避免 bank conflict。

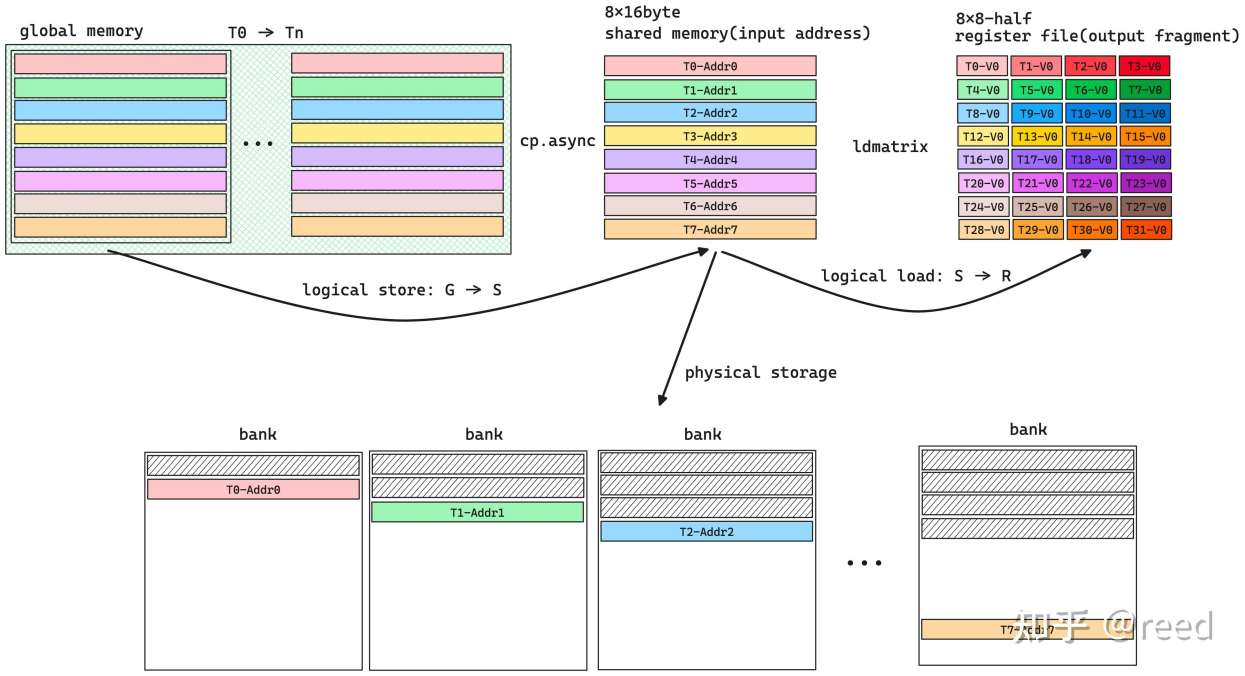


图 7.6: Figure 6. 数据加载的逻辑空间和物理空间

7.4　异或运算的封闭性和双射性

计算机异或指令（符号表示为 ^）接收两个输入，对于 1bit 数据，如果输入的 bit 相同则输出 0，如果 bit 不同则输出 1。对于多 bit 数据，则针对各个位置进行 1bit 异或操作。如 5^3 = b0101 ^ b0011 = b0110 = 6。异或计算满足交换律，结合律。同时对于集合 $S = \{x, x \in [0, 2^n - 1]\}$ 中的任意两个元素，异或所形成的输出满足封闭性。如图 7，我

们可以发现该结果满足双射性（bijective），以上这些性质可以通过集合理论来进行严谨的证明。

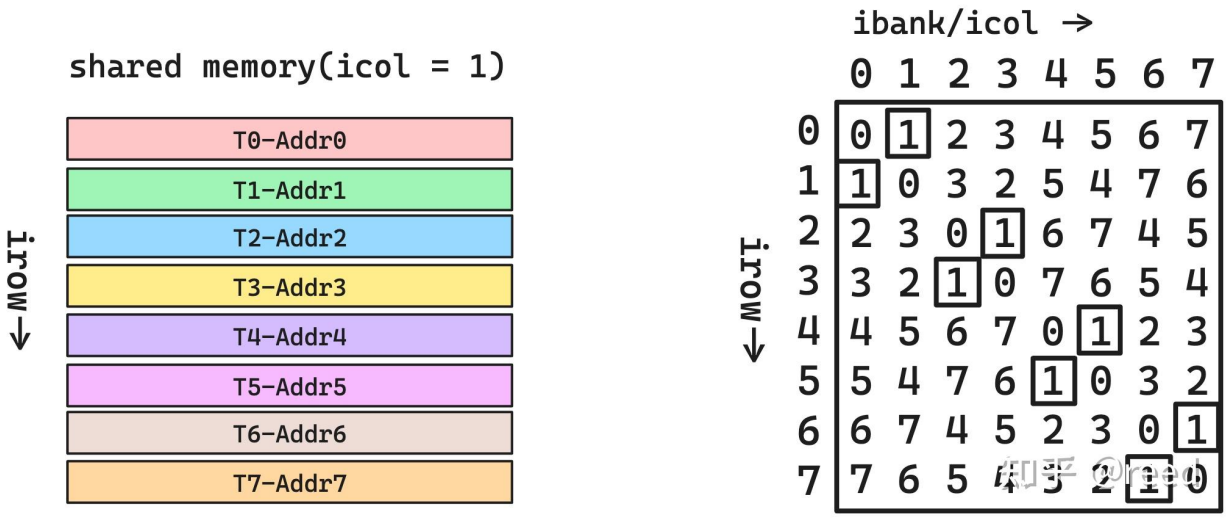


图 7.7: Figure 7. 使用异或避免共享内存 bank 冲突

图 7 中左侧的逻辑矩阵可认为是 icol 为 1 的共享内存，其在共享内存对应一个 bank，即矩阵的逻辑位置为 $(irow = [0,7], icol = 1)$，我们可以通过对 column 进行异或得到新的 column 作为 bank 值，即 $(irow = [0,7], ibank = irow \wedge icol)$。如图 7 中右侧黑框标注的部分，可以看到这些数据被分配到了不同的 bank 中，在读写时可以避免 bank conflict。

7.5 Swizzle 抽象

cute 中通过 swizzle 抽象来实现共享内存 bank conflict 的冲突解决。通过前面的描述我们知道，在整个计算体系中，我们需要的是二维的逻辑空间来描述矩阵块，但是为了避免共享内存的冲突，我们在共享内存存储数据时需要的是物理空间。回顾之前的介绍我们知道描述逻辑空间我们可以使用 Layout（本质是函数）[5]，而为了避免 bank 冲突，cute 中定义了 swizzle 抽象，swizzle 的本质也是函数，swizzle 作用在 layout 上，即函数作用在函数上，复合函数复合的定义。Layout 的作用是给定坐标返回 offset，而 swizzle 的作用则是给定 offset 返回 bank conflict free 的 offset。即 $offset_{bank_conflict_free} = Swizzle(Layout(coord))$。为了达成这个目的，Swizzle 定义了三个参数: B、M、S。它们共同表达描述一维坐标向二维空间映射的三个层次。当我们把一个一维度坐标转换成二维坐标时，我们首先将一维中连续的几个元素作为新空间中的基础元素，然后描述该二维空间有多少行和列。其中一维坐标中连续的 2^M 个元素构成二维空间中最基本的元素，2^S 表示新的二维空间中有多少列，2^B 表示新的二维空间中有多少行。

如图 8 所示，当 B = 1, M= 1，S = 2 时，M 描述了一维坐标连续的两个元素构成一个二维空间的元素，S 描述了形成二维空间后的元素的个数，B 描述了形成二维空间的行数，如此我们便得到了图 2-D(a)，其二维空间包含二行四列，基本单元包含两个元素。然后我们对二维空间中的列坐标和对应的行坐标进行异或得到新的列号 (icol = irow ^ icol)，形成 2-D(b)。如果一维坐标映射后超过了 2^B 的大小，则超出的部分的行号从 0 开始记，但是 offset 上要加上前面的所有的元素个数。

在实际操作时，如我们有一块 half 类型，shape:(8, 32), stride: (32, 1) 的共享内存，我们定义 Swizzle<3, 3, 3> 作用到该 shared memory Layout 上，形成 A = Composition(Swizzle<3, 3, 3>{}, Layout<Shape<8, 32>, Stride<32, 1>>{}); 则 Layout 中有效的 offset 为 0–255（cosize = 256）。Swizzle 中 M 为 3，所以 8 个元素形成一个新的最小的元素，即 8x2byte = 16byte；Swizzle 中 S 为 3，所以 2D 空间中一行包含 8 个元素，则有 8x16byte = 128byte，128byte 为 shared memory 无 conflict 访问所有 bank 的最大宽度；Swizzle 中 B 为 3，则 2D 空间 irow 更新的间隔为 8。如此则实现了将一个逻辑的空间向 2D 的 shared memory 空间的映射，其中列的宽度为 128byte 占满所有的 bank，行列异或后得到新的列号，避免了在 bank 方向（亦即 icol 方向）的冲突。

[5] https://zhuanlan.zhihu.com/p/661182311

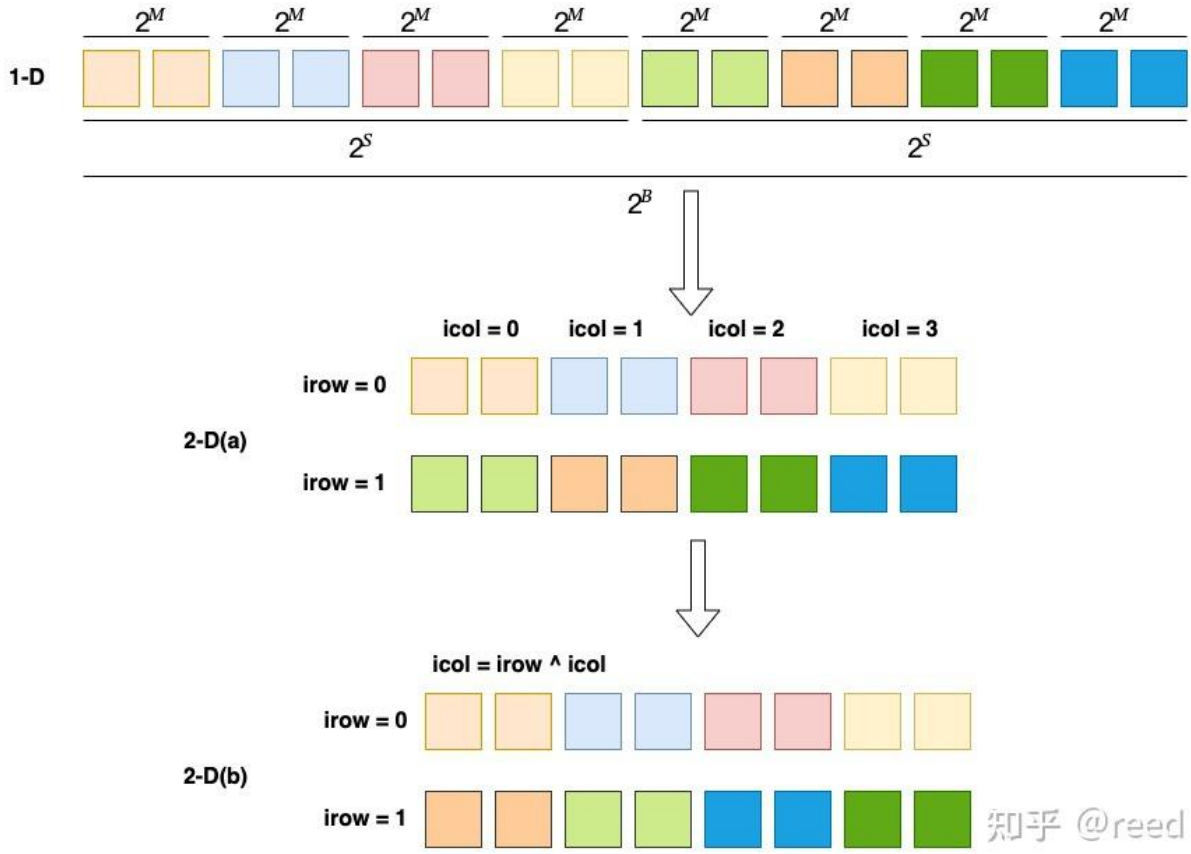


图 7.8: Figure 8. Swizzle 的计算逻辑

7.6 Thread Block Swizzle

除了避免共享内存冲突的 swizzle 外，cute（cutlass）中还有另一种 swizzle，为 thread block swizzle，在以 C 为中心的任务划分模式中，如果没有 Thread Block Swizzle，则任务块会按照线性的行优先或者列优先的顺序分配给所有的执行单元（如图 9 中 SM0-3，假设硬件只有 4 个 SM），进行 Thread Block Swizzle 后，可以形成如图 9 右侧所示的任务划分关系，在某些场景下，其可以提升 L2 Cache 的命中率，数学上表现为在相同的元素能覆盖更大的面积，同时这部分面积 (A、B) 能够很好的被 L2 缓存住，具体的可以参考 cutlass 中的 thread block swizzle 实现。

7.7 总结

本文介绍了 shared memory 的 bank 结构和特征，以及通过行列异或实现 bank 的交错的方法。继而介绍了 cute 中 Swizzle 的抽象，最后介绍了优化 L2 命中率的 Thread Block Swizzle 方法。至此矩阵乘法优化的理论性部分已经全部介绍完毕，接下来的文章我们会基于这些方法完成高效的矩阵乘法。

7.8 参考

- https://en.wikipedia.org/wiki/Locality_of_reference
- https://on-demand.gputechconf.com/gtc/2018/presentation/s81006-volta-architecture-and-performance-optimization.pdf
- https://patentimages.storage.googleapis.com/44/ed/9d/7b0c401348f57b/US8108625.pdf
- https://patentimages.storage.googleapis.com/e7/27/98/de3cbfc7ab3b8c/US7680988.pdf
- https://github.com/NVIDIA/cutlass/blob/main/include/cutlass/gemm/threadblock/threadblock_swizzle.h

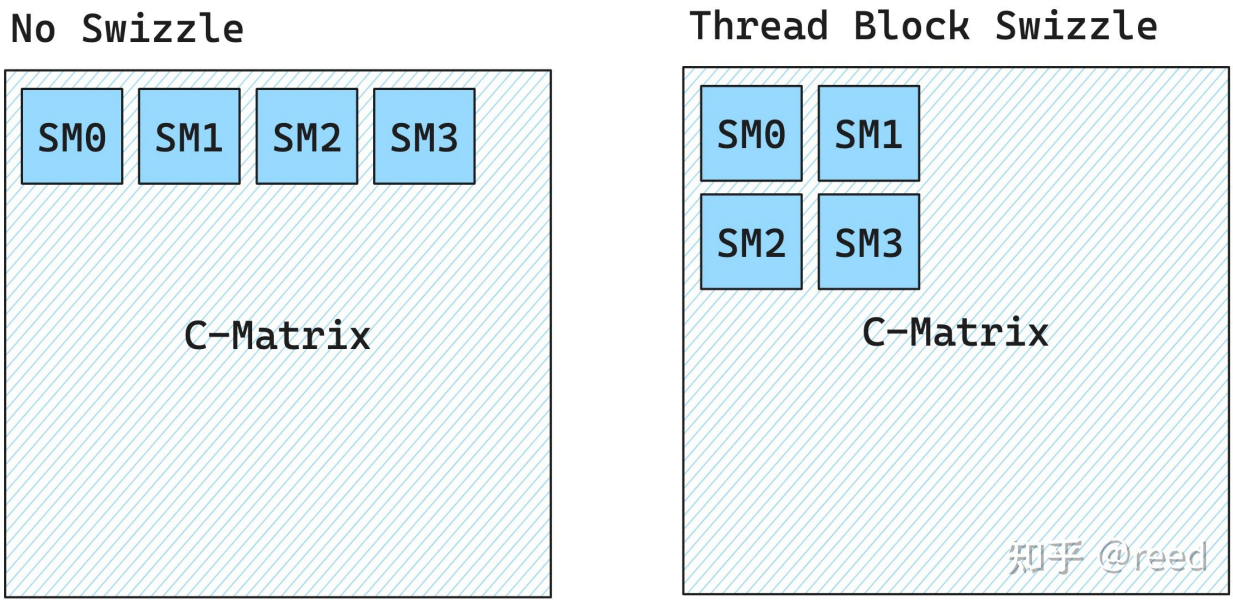


图 7.9: Figure 9. Thread Block Swizzle

- How to understand the bank conflict of shared_mem[6]

[6] https://forums.developer.nvidia.com/t/how-to-understand-the-bank-conflict-of-shared-mem/260900

第八章　cute 之 GEMM 流水线

作者：reed（知乎 @reed-84-49，专栏「CUDA 高性能编程」）　原文：https://zhuanlan.zhihu.com/p/665082713

前面文章我们介绍了 cute 的 Copy 抽象、MMA 抽象，基于这些抽象我们进行了简单的 GEMM 实现。从逻辑上而言，cute 的介绍已经结束了，但是对于我们要完成的 GEMM 运算而言还有很重要的一项优化需要考虑，那就是如何高效的、并行的利用 GPU 中的数据加载和计算单元，亦即如何组织 cute 中的 Copy 抽象和 MMA 抽象以完成高效的 GEMM 计算。这部分内容属于 GEMM 的策略部分，不是 cute 的功能范畴，但是为了这系列文章的标题的对称性，我们依然将题目取为"cute 之 GEMM 流水线"，我们需要知道的是，本质而言流水线部分本身不属于 cute，而是 GEMM 的优化策略。文章结构方面，本文首先通过回顾经典的 RISC 硬件实现的指令流水线引入流水线对性能提升的作用，然后由类比介绍了 GEMM 算法常用的软件流水线（Tile 间和 Tile 内），其后介绍了 NVidia Ampere 架构提供的异步拷贝指令和 MultiStage 流水线，最后文章总结了 GEMM 流水线和 cute 的关系。

8.1　RISC 硬件流水线

在当今的处理器微架构中，流水线技术是提升指令并行的核心技术。流水线处理器是指将每一个指令的执行过程分为多个阶段(Stage)，并且允许不同指令的不同阶段可以被同时处理。以经典的 RISC(Reduced Instruction Set Computer)流水线为例，一条指令的执行被分为五个阶段：取指（IF = Instruction Fetch），从指令缓存中根据程序运行的位置（PC = program counter）取出一条要执行的指令；译码（ID = Instruction Decode），将取出的二进制编码分解成要进行的运算类型、源寄存器和目的寄存器；执行（EX = EXecute），执行单元执行特定的运算；访存（MEM = MEMory），如果指令有对内存的访问需求，该阶段则负责相应的内存读写；写回（WB = Write Back），将执行单元的执行结果和（或）内存的访问结果写出到目的寄存器。

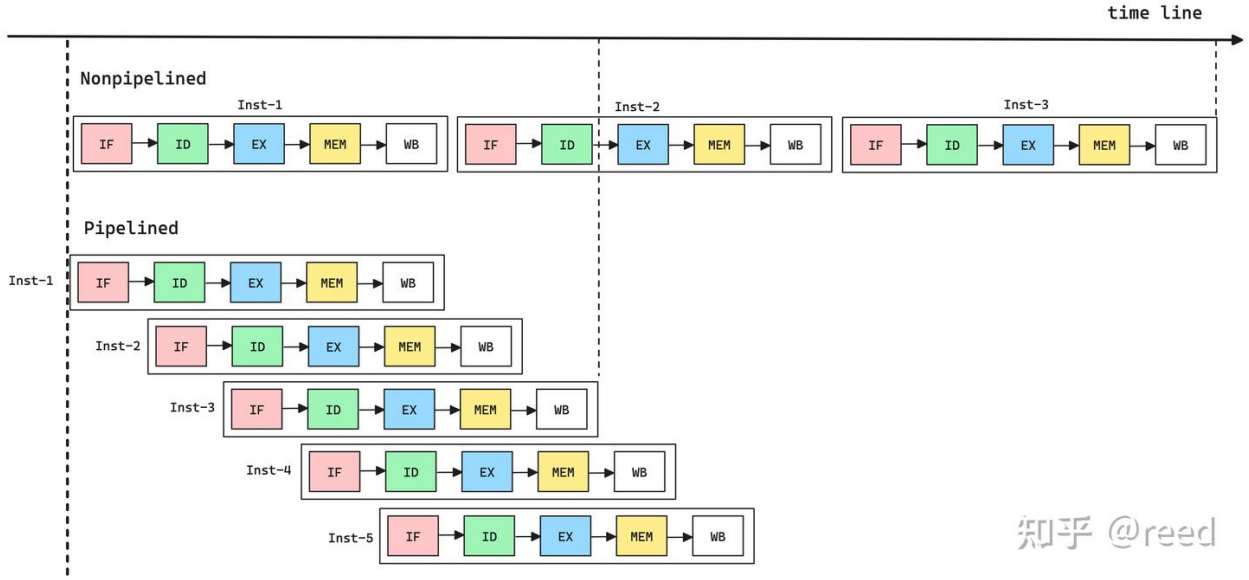


图 8.1: Figure 1. 流水线和非流水线处理器指令执行的时间分析

图 1 对比了非流水线和流水线架构执行指令时的时间对比，可以看到非流水线结构执行每一条指令都需要执行所有的阶段，其执行三条指令（Inst-1, Inst-2, Inst-3）所需要的时间如图上半部分所示。在流水线结构下，第一条指令在做取指后进入译码阶段，这时候第二条指令则可以进入取指阶段，后续的指令阶段也是类似的可以产生重叠。如图中的下半部分所示，以流水线的形式执行三条指令所需的时间要比非流水线的模式小很多。流水线模式提升了指令执行中的各个阶段的不同单元的使用率，使得每一个时刻每一个单元都能充分利用，而不是非流水线结构中一个时间点只有一部分单元运行，而其他时间都在空闲等待。

8.2 GEMM 软件流水线（Tile 间）

我们看到指令流水线通过硬件的设计逻辑可以提升各个单元的利用情况，提升并行度既而提升运行效率。对于 GEMM 问题而言，我们也可以采用这种思路利用软件编程的过程的来实现更好的并行。

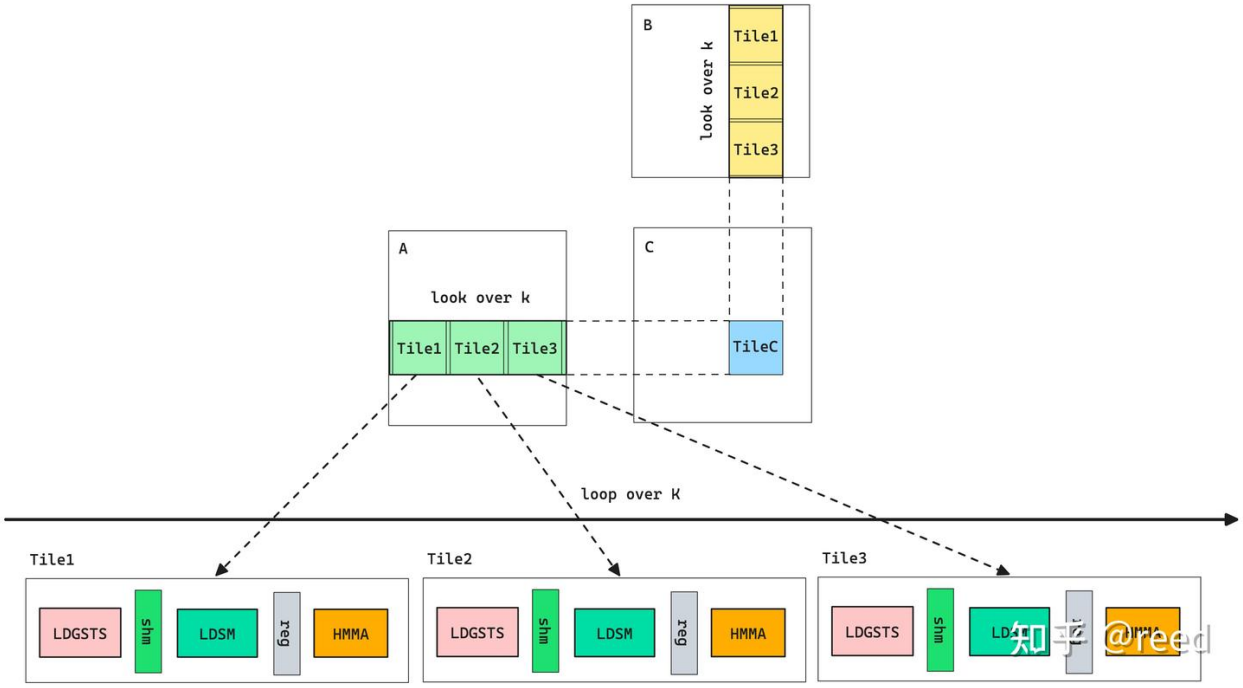


图 8.2: Figure 2. 循环 k 模式的矩阵乘法的指令构成

如图 2 所示，一个典型的 sliced-k 模式的 GEMM 实现中，通过循环 K 轴方向的 tile 来累加得到最终的 CTile 的结果。则类似 RISC 中的流水线模式，我们将计算每一个 Tile 的矩阵乘积作为一个基本的单元（如 RISC 中的指令），则该指令的执行类比 RISC 流水线可以划分为多个阶段，数据加载到共享内存（LDGSTS = LoaD Global STore Shared memory）；数据加载到寄存器（LDSM = LoaD Shared Matrix）；块状矩阵乘法运算（MMA = Matrix Multiply Accumulate）。其中第一个阶段的输出数据存放在共享内存中，第二个阶段的数据存放在寄存器中。和 RISC 中的流水线类似，我们把一个 Tile 的计算过程分成三个阶段，如果各个阶段可以重叠则效率可以极大的提升，于是有了流水线思路优化的 GEMM 执行效果，如图 3 所示，

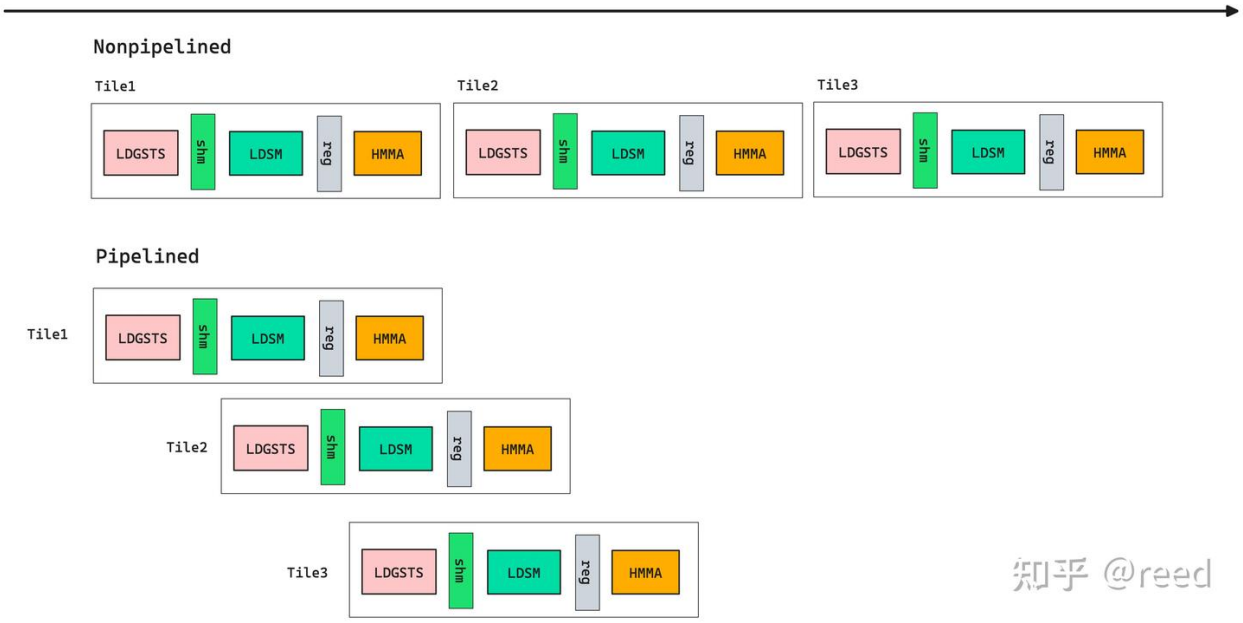


图 8.3: Figure 3. 非流水线模式和流水线模式下的 GEMM 执行逻辑

这样三个阶段的执行便可以并行起来，通过流水线使得各个单元能够同时工作，提高了各个单元的利用效率，包括全局内存到共享内存的数据加载、共享内存到寄存器的数据加载、矩阵计算，提升 GEMM 的运行效率。

8.3 Tile 内的流水线

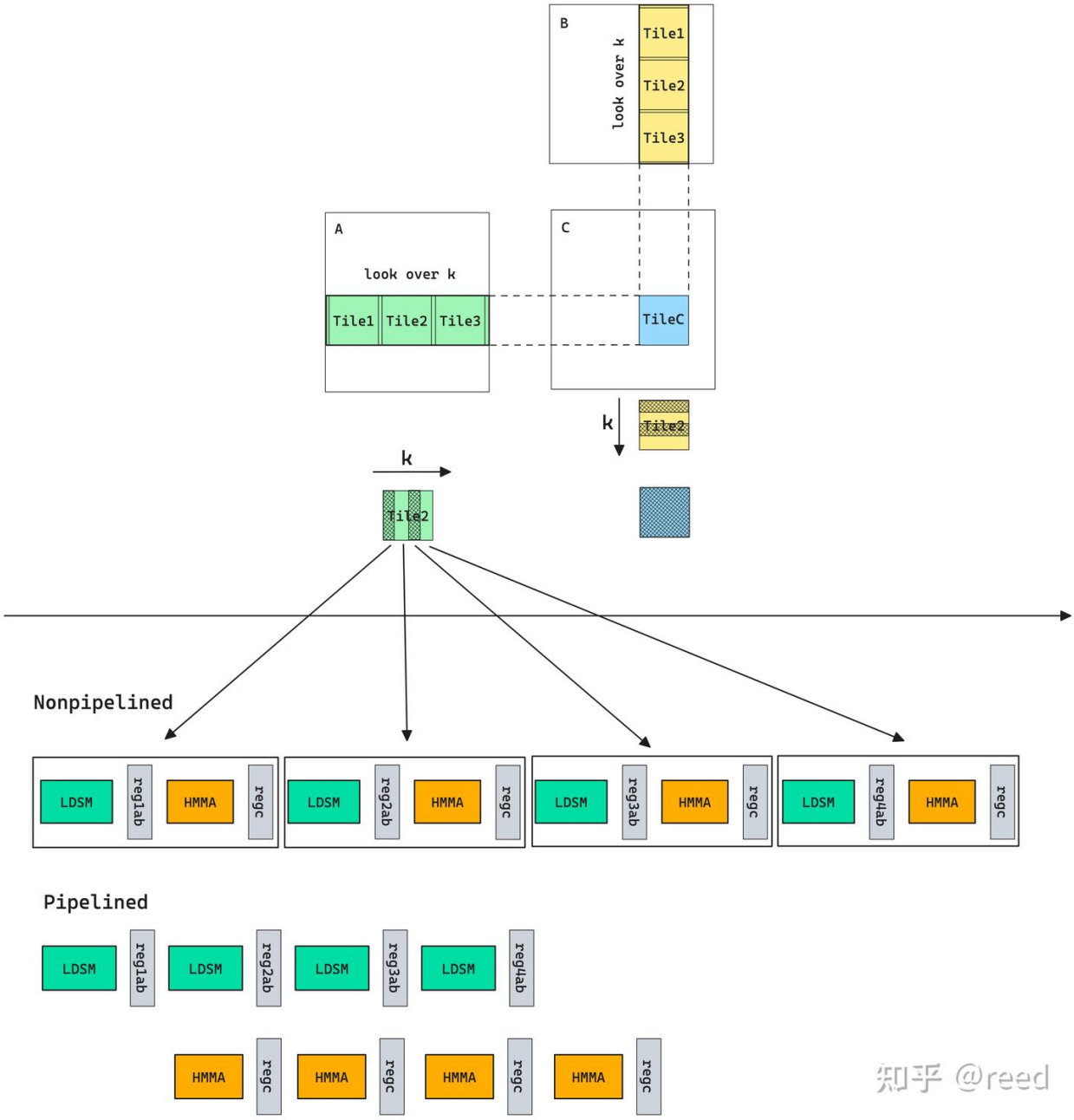


图 8.4: Figure 4. GEMM Tile 内的流水线模式

对于矩阵乘法中的分块模式 Tile 内也可以使用流水线模式（本文称为 tile 内小 k 循环），如图 4 所示，对于 Tile 级别的矩阵乘，一般一个 Tile 内包含的矩阵大小需要若干个指令（MMA_Atom 中的指令）才能完成矩阵乘法，并且各个矩阵乘法的输入数据相互独立，所以我们可以将数据加载和计算组成流水线模式以提高数据加载单元和计算单元的利用率，如此便可以形成 Tile 内的流水线模式（二级流水线），如图中 pipelined 标记的部分，其可以通过重叠数据加载和计算提高 Tile 内完成矩阵计算的整体效率。

8.4 异步拷贝和 MultiStage 流水线

为了提升数据加载效率，NVidia 在 Ampere 架构的 GPU 中提供了异步拷贝指令cp.async（SASS 汇编为 LDGSTS = LoaD Global Store Shared）。该异步拷贝指令可以异步地完成全局内存到共享内存的数据加载。在 Ampere 架构之前，全局内存到共享内存数据的加载必须经过寄存器，所以在寄存器层面会产生数据依赖，由于 GPU 的顺序执行机制和 scoreboard 的依赖解决方法（in-order issue, in-order execute），使得全局内存到共享内存的数据有寄存器依赖引入的 stall。而 Ampere 下提供的 cp.async 则克服了这个约束，直接实现全局内存到共享内存的加载，由于数据是异步加载的，即指令发射出去后便可以执行后续指令而无需等待，所以该架构提供了 commit 和 wait 机制来做显式的同步。其中 commit 用于标记事件的同步点，wait 用于同步到特定同步点，保证某个同步点之前的数据都已经拷贝完成。

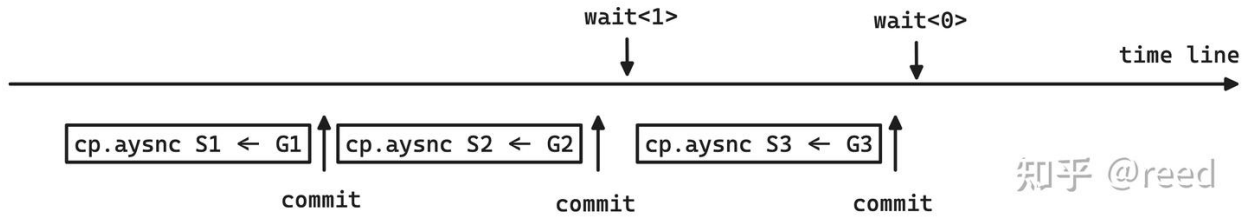


图 8.5: Figure 5. 异步拷贝机制

如图 5 所示，我们通过cp.async指令提交了三个全局内存到共享内存到拷贝任务，同时通过 commit 提交了三个事务点和两个 wait，其中wait<1>表示可以允许最多有一个未完成的异步事务（G2 -> S2），即wait<1>执行结束能够确保 G1 到 S1 的拷贝已经完成，wait<0>表示允许有零个未完成的事务。也就是其会等待之前所有的 commit 的任务都完成，即 G1->S1, G2->S2, G3->S3 全部已经完成。

有了异步数据拷贝指令我们便可以完成全局内存到共享内存的异步加载，即完成矩阵 A、B Tile 的加载，再整合 Tile 间和 Tile 内的流水线，我们可以得到 GEMM 计算的 MultiStage 流水线模型，如图 6 所示，

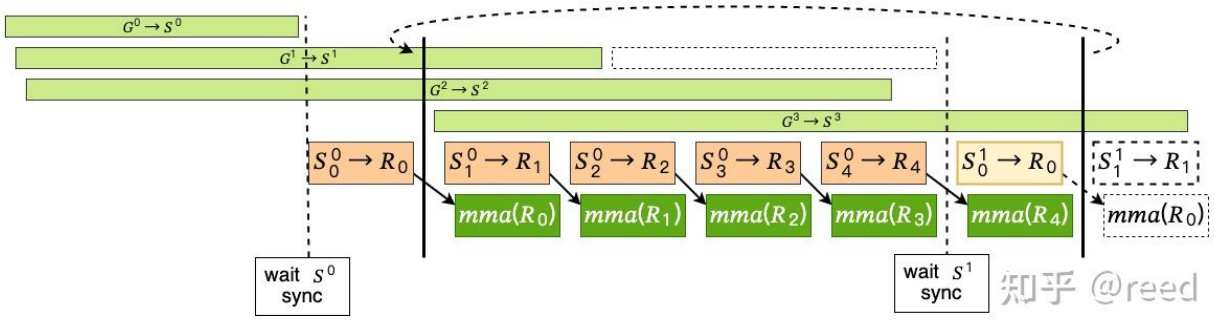


图 8.6: Figure 6. MultiStage 流水线模型

其中浅绿色的 $G^i \to S^i$ 表示全局内存到共享内存到异步数据加载其对应的大小为 Tile 的大小，其合并表示 TileA 和 TileB 的数据加载（即 tile 循环），棕色的 $S_j \to R_j$ 表示共享内存到寄存器到数据加载，其对应 Tile 内的小矩阵的数据加载，也是合并的表示 A B 的加载（即 tile 内的小 k 循环）；深绿色 $mma(R_i)$ 表示寄存器上的矩阵乘法计算，亦 tile 内的小 k 循环。mma 的边界有两条黑色边界线，两边界线通过曲线虚线连接，其表示 tile 内小 k 循环的起点和终点，即黑线之间完成 tile 内的矩阵乘法；曲线虚线表示完成一个 tile 的计算之后继续进行下一个 tile 的计算。在第一个 tile 开始计算之前（即第一个黑色实线边界之前）对于 multistage 实现（图示 kStage 为 4），需要将 stage - 1 个异步的全局内存到共享内存到加载任务发射出去（G0->S0, G1->S1, G2->S2），同时为了能够读取第一个 Tile 的内容，则在所有异步任务发射之后，我们 wait S0 完成，wait 之后表示数据已经到达了共享内存，在进入 tile 的小 k 循环之前我们首先从 S0 中取出 ik = 0 的矩阵计算所需要的数据到寄存器 R0(第一条黑色虚线和第一条黑实线之间)，这时，我们已经有了第一个矩阵计算所需要的数据了，于是我们进入 tile 内的小 k 循环，进入循环我们需要执行三个动作：1. 发射异步读取新的 Tile 数据 G3->S3，2. 从共享内存读取下一个小 k 矩阵乘法所需要的数据 R1，3. 执行第一个小 k 的矩阵运算。其中共享内存写出的数据和 mma 所需数据依赖关系通过其中的箭头表示。进入小 k 循环后重复上面的 2、3 步骤即可以流水线的完成数据加载和计算在最后一个小 k 循环之前，我们需要读取下一个 tile 中的第一个小 k 的数据（共享内存到寄存器），但是此时下一个 tile 的数据（全局内存到共享内存）需要通过 wait S1 保证数据加载完成，所以在最后

一个小 k 循环之前需要插入对 S1 的异步事务等待，等待结束后我们便可以像之前进入小 k 循环之前一样在进入下一个循环（tile 循环）之前加载共享内存的数据到寄存器，值得注意的是此处的共享内存已经不是当前 tile，而是下一个 tile，即 S1。读完 R0 之后，完成最后一个小 k 的 mma 计算，如此便完成了 tile 内的小 k 循环，小 k 循环结束便重复下一个 tile 的计算，最终完成 tile 循环。

如上便是 multi stage 的 GEMM 流水线（Tile 间多级，Tile 内二级），其中 multi 表示多个，具体的个数则为 shared memory 中间 buffer 的个数。即 stage 为 5 的 GEMM 流水线指有五个 shared memory buffer 的流水线设计，其中每个 buffer 可以存放一个 Tile 的数据（包含 TileA 和 TileB）。在 Tile 循环开始之前需要先发射 stage - 1 个全局内存到共享内存的加载，然后在循环中加载下一个 Tile，如此循环使用以上 stage 个 buffer 完成所有数据到加载。在没有异步拷贝支持的 GPU 架构中，寄存器的依赖关系和 syncthread 的全局影响，决定了其最多只能有两个 memory buffer（实质是寄存器 buffer），一个用于当前数据的计算，一个用于后续数据的加载，这也是常说的双缓存机制（double buffer），双算缓存机制可以认为是 multi stage 中 stage 为 2 的一个特例。

合适的 stage 大小本质是数据加载能力和矩阵计算能力的 balance，其由 Tile 大小和硬件 latency 决定，在具体选择时可以通过 micro-benchmark 来获取相应的指令的 latency 来正向设计，也可以通过具体环境试验 tuning 得到。后面的文章中，我们会利用这种软件流水线方式实现更高效的 GEMM。

8.5 总结

我们回顾上面的流水线建立过程不难发现，软件流水线的本质是：合理的控制数据搬运和计算的大小和执行顺序，使得它们背后的硬件单元能够更充分的并行执行。cute 提供了这两者的抽象：Copy 和 MMA，按照流水线的思路去组织这些 Copy 和 MMA 即可以完成高效的矩阵乘法。也就是说 cute 是工具，如何更好的使用这些工具来达到更好的硬件利用则属于设计，其已经超越了 cute 的范畴。后续文章我们会使用 cute 提供的 Copy 和 MMA 抽象，基于本文讨论的流水线模式实现高效的 GEMM。

8.6 参考

https://en.wikipedia.org/wiki/Classic_RISC_pipeline

https://link.springer.com/book/10.1007/978-3-031-01729-2

https://github.com/NVIDIA/cutlass/blob/main/include/cutlass/gemm/collective/sm80_mma_multistage.hpp

第九章　cute 之高效 GEMM 实现

作者：reed（知乎 @reed-84-49，专栏「CUDA 高性能编程」）　原文：https://zhuanlan.zhihu.com/p/675308830

前面的文章我们介绍了 cute 中的 Layout 抽象、Tensor 抽象、MMA 抽象、Copy 抽象、Swizzle 抽象和流水线技术。本文将组合利用这些抽象和技术形成高效的矩阵乘法。为了更好地实现高效的矩阵乘法，本文从多个维度介绍了高效的实现方法。在文章结构上，本文首先介绍了计算高效，然后介绍了访存高效，再后介绍了算法高效，最后介绍了尾阶段高效。我们基于这些高效方案利用 cute 实现了高效的矩阵乘法，并和 cuBLAS、cuBLASLt 进行了性能对比，对比结果表明我们的实现达到了 SOTA 水平。文章最后讨论了搜索 kernel 的启发式算法和参数相容性问题并对文章进行了总结。

9.1　计算高效

GEMM 中的核心计算部分是块状的矩阵乘法，针对输入为半精度类型（half precision），Accumulator 为半精度类型的计算任务，Ampere 架构提供了 Tensor Core 上的如下计算指令 `mma.sync.aligned.m16n8k8.row.col.f16.f16.f16.f16` 和 `mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16`，cute 将这两条指令抽象为 MMA_Operation：`SM80_16x8x8_F16F16F16F16_TN`和 `SM80_16x8x16_F16F16F16F16_TN`，当我们的问题规格较大时，我们尽可能选用计算量更大的指令，这样同一条指令产生的计算任务就更多，可以减少指令数目，提升调度效率。

选定计算指令之后，我们便可以通过`MMA_Traits`，如图 1，在 MMA_Operation 的基础上补充上后续计算所需要的其它信息，如矩阵计算的形状、该指令所需要的协作线程（此处为 32 线程）、A、B 矩阵的寄存器 Layout 分布情况。有了 MMA_Traits 之后，我们便可以将其进一步封装为`MMA_Atom`，其利用 Traits 提供的信息，提供数据划分所需要的信息和 Operation 的执行功能。MMA_Atom 描述了矩阵计算的原子能力（单条指令的计算能力，最小能力），我们通过增加更多的线程、每个线程做多次任务则可以将计算的规格增大，如此则有了 TiledMMA，TiledMMA 针对每一个线程则被分裂为 ThrMMA，TiledMMA 和 ThrMMA 利用 MMA_Atom 提供的信息，能够实现对矩阵块的划分。调用相应的`cute::gemm`函数即可以完成矩阵乘法计算。

此时，我们便可以得到如下主机端代码

```
using mma_op = SM80_16x8x16_F16F16F16F16_TN;
using mma_traits = MMA_Traits<mma_op>;
using mma_atom = MMA_Atom<mma_traits>;

static constexpr int kMmaEURepeatM = 2;
static constexpr int kMmaEURepeatN = 2;
static constexpr int kMmaEURepeatK = 1;

static constexpr int kMmaVRepeatM = 1;
static constexpr int kMmaVRepeatN = 2;
static constexpr int kMmaVRepeatK = 1;

using MMA_EU_RepeatT = decltype(make_layout(make_shape(
    Int<kMmaEURepeatM>{}, Int<kMmaEURepeatN>{}, Int<kMmaEURepeatK>{})));
using MMA_V_RepeatT = decltype(make_layout(make_shape(
    Int<kMmaVRepeatM>{}, Int<kMmaVRepeatN>{}, Int<kMmaVRepeatK>{})));

using MMA =
    decltype(make_tiled_mma(mma_atom{}, MMA_EU_RepeatT{}, MMA_V_RepeatT{}));
```

其中前三行选择了 MMA 指令形成了 Atom 能力，然后定义了对该 Atom 能力的重复方法（包括线程重复和寄存器重复），它们分别形成各自重复的 Layout，然后利用 make_tiled_mma 接口形成更大块的矩阵乘法描述。设备端的代码如下：

```
TiledMMA tiled_mma;
auto thr_mma = tiled_mma.get_slice(idx);
auto tCrA = thr_mma.partition_fragment_A(gA(_, _, 0));  // (MMA, MMA_M, MMA_K)
```

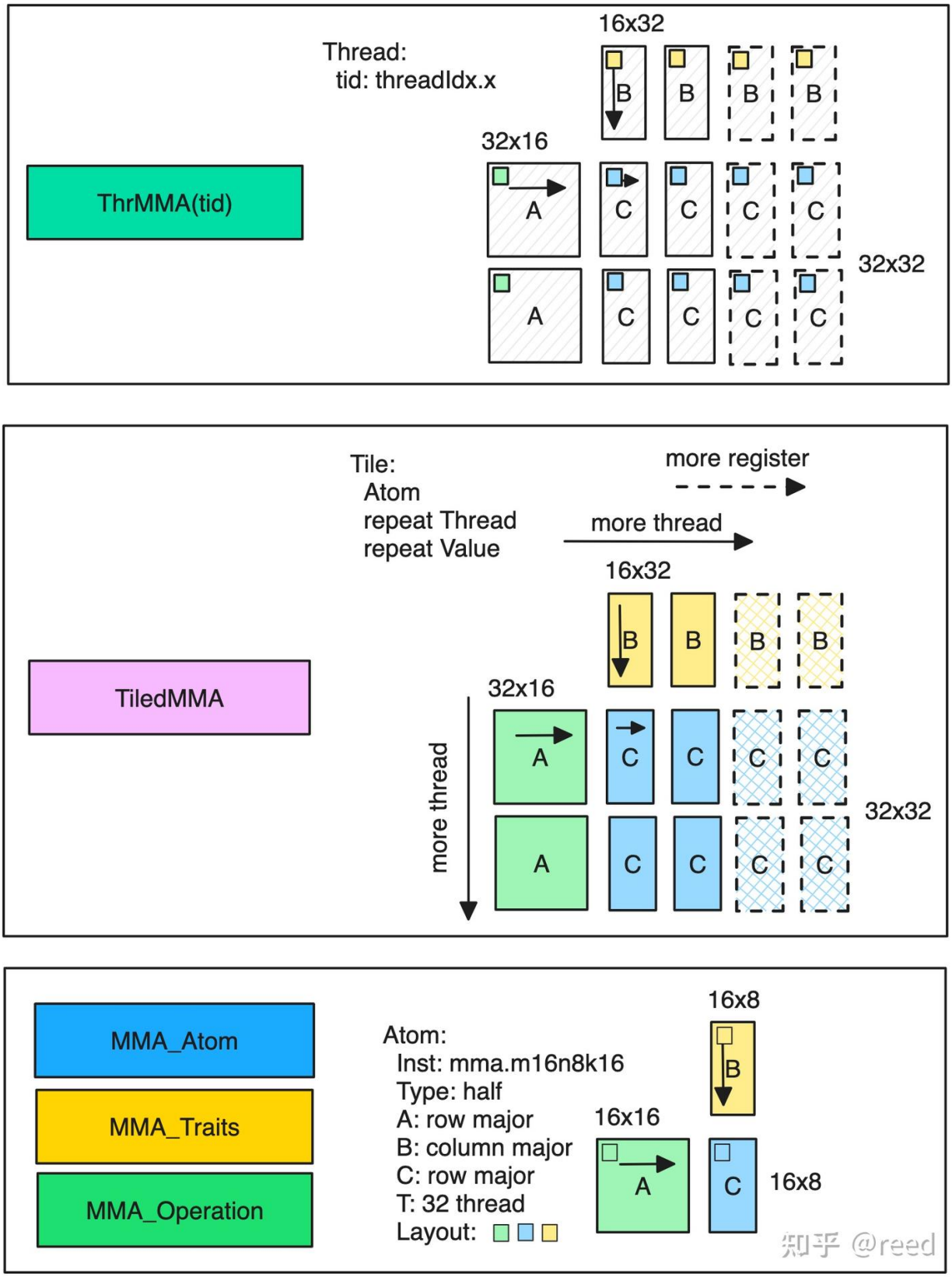


图 9.1: Figure 1. MMA 能力层次和各层的主要功能

```
  auto tCrB = thr_mma.partition_fragment_B(gB(_, _, 0));  // (MMA, MMA_N, MMA_K)
  auto tCrD = thr_mma.partition_fragment_C(gD);           // (MMA, MMA_M, MMA_N)
```

给 TiledMMA 提供线程号，则获得具体线程的数据划分能力，对给定的数据块进行划分，得到线程级的数据描述。

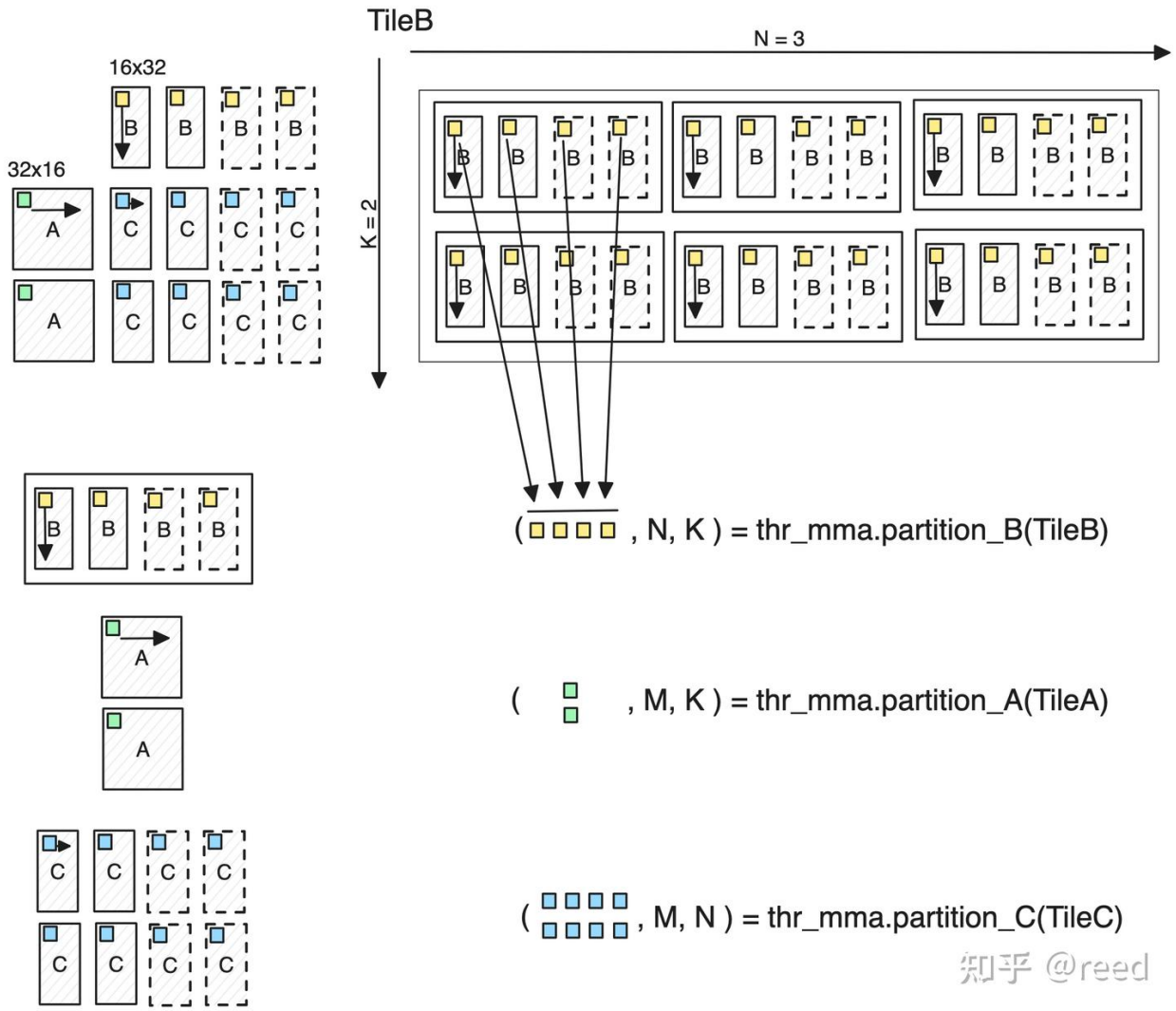


图 9.2: Figure 2. partition_A/B/C 逻辑示意图

如图 2 所示，其展示了 ThrMMA 提供的`partition_A/B/C`和`partition_fragment_A/B/C`函数的计算逻辑，给定一个静态大小的 Tensor TileB（被划分维度为`Int`<>编译时常量），则 thr_mma 可以对其进行划分，其中划分的逻辑为：以 TiledMMA 中描述的矩阵大小对目标 Tensor 进行周期性分块，对高亮的部分进行选取形成新的矩阵，其中第一个维度为 TiledMMA 中单个线程的数据描述，第二个维度和第三个维度为行方向和列方向需要重复的次数。如果 TileB 的维度比两维高，则高出的部分继承到 N,K 维度之后。类似地，A/C 的划分采用同样的逻辑。

9.2 访存高效

整个 GEMM 计算体系中数据的在到达 Tensor Core 进行计算之前需要包含：全局内存到共享内存，共享内存到寄存器，在流水线章节我们介绍过全局内存到共享内存的异步拷贝方法，和共享内存到寄存器的`ldmatrix`指令。在 cute 中，针对全局内存到共享内存，我们和选择 MMA 能力类似，选择 cute 已经定义好的抽象能力即可，此处我们选择`SM80_CP_ASYNC_CACHEGLOBAL`（以`cute::uint128_t`为模板参数），该指令可以实现全局内存到共享内存的异步拷贝，同时 CACHEGLOBAL 指示了数据只在 L2 做 Cache，对 L1 则做 bypass。于是我们可以形成如下主机端代码：

```
using g2s_copy_op = SM80_CP_ASYNC_CACHEGLOBAL<cute::uint128_t>;
using g2s_copy_traits = Copy_Traits<g2s_copy_op>;
using g2s_copy_atom = Copy_Atom<g2s_copy_traits, T>;

```

```
using G2SCopyA =
    decltype(make_tiled_copy(g2s_copy_atom{},
                             make_layout(make_shape(Int<32>{}, Int<4>{}),
                                         make_stride(Int<4>{}, Int<1>{})),
                             make_layout(make_shape(Int<1>{}, Int<8>{}))));
using G2SCopyB = G2SCopyA;
```

和 MMA 时的make_tiled_mma类似，Copy 抽象提供了 make_tiled_copy 能力，其通过指定线程和数据的重复方法将 Atom 能力扩展到块状能力。数据拷贝时可以区分 AB 矩阵的不同拷贝方法，我们此处选用同样的 Copy 能力。设备端代码如下

```
G2SCopyA g2s_tiled_copy_a;
auto g2s_thr_copy_a = g2s_tiled_copy_a.get_slice(idx);
auto tAgA_copy = g2s_thr_copy_a.partition_S(gA);  // (CPY, CPY_M, CPY_K, k)
auto tAsA_copy =
    g2s_thr_copy_a.partition_D(sA);  // (CPY, CPY_M, CPY_K, kStage)
```

和图 1 中的 MMA 层级类似，Copy 时先将 TileCopy 通过指定线程号得到线程级的 Copy 能力抽象 ThrCopy。也和图 2 中的 MMA 划分类似，ThrCopy 抽象提供了partition_S/D函数，其实现将大块的矩阵划分到线程维度上。经过 partition_S/D 划分的数据维度为 (E, M, K)，E 表示该线程要做的数据大小（包含分布），M、K 表示由于给定的被划分的块需要在纵轴和横轴上需要重复的次数。如果被划分的 Tile 的维度大于 2，则多出的维度附加到 (E, M, K) 维度之后。

对于共享内存到寄存器的拷贝，cute 提供了对 ldmatrix 指令的封装，主机端代码和设备端代码分别如下：

```
// shared memory to register copy
using s2r_copy_op = SM75_U32x4_LDSM_N;
using s2r_copy_traits = Copy_Traits<s2r_copy_op>;
using s2r_copy_atom = Copy_Atom<s2r_copy_traits, T>;

using S2RCopyAtomA = s2r_copy_atom;
using S2RCopyAtomB = s2r_copy_atom;
```

设备端代码：

```
auto s2r_tiled_copy_a = make_tiled_copy_A(S2RCopyAtomA{}, tiled_mma);
auto s2r_thr_copy_a = s2r_tiled_copy_a.get_slice(idx);
auto tAsA = s2r_thr_copy_a.partition_S(sA);  // (CPY, CPY_M, CPY_K, kStage)
auto tCrA_view = s2r_thr_copy_a.retile_D(tCrA);  // (CPY, CPY_M, CPY_K)
```

其中主机端选择 ldmatrix 指令的 x4 模式，形成 Atom 抽象，设备端通过make_tiled_copy_A函数借助 tiled_mma 抽象出共享内存到寄存器的 TileCopy。和前面的全局内存到共享内存 TileCopy 不同的是，这里直接利用 tiled_mma 的信息形成块状拷贝，原因是 TiledMMA 包含了计算所需要的数据描述，所以对于以它作为目标的 Copy 而言，tiled_mma 自然也是精准描述这部分数据的，所以就不需要用户额外指定 Copy_Atom 到 TileCopy 的信息，而是直接从 MMA 能力中获得，其一定程度上可以避免了独立设置的不一致性问题。由于 MMA 时已经声明了寄存器存储空间，此处直接对其进行线程级小块的 retile 即可，不再是大块到小块的 partition。

9.3　算法高效

前面两个章节介绍了计算的高效和访存的高效，如何将这两个步骤组合起来也是 GEMM 性能的关键要素，这部分我们称为算法的高效。其主要包含两个部分：1. 分块；2. 流水线。这两部分内容我们在简单 GEMM 实现和流水线中都有介绍，我们此处就不再赘述。对于分块部分，我们主机端和设备端的代码如下：

```
static constexpr int kTileM = kTileM_;
static constexpr int kTileN = kTileN_;
static constexpr int kTileK = kTileK_;
static constexpr int kStage = kStage_;
```

设备端：

```
  // slice the tensor to small one which is used for current thread block.
  Tensor gA = local_tile(A, make_tile(Int<kTileM>{}, Int<kTileK>{}),
                         make_coord(iy, _));  // (kTileM, kTileK, k)
  Tensor gB = local_tile(B, make_tile(Int<kTileN>{}, Int<kTileK>{}),
                         make_coord(ix, _));  // (kTileN, kTileK, k)
  Tensor gD = local_tile(D, make_tile(Int<kTileM>{}, Int<kTileN>{}),
                         make_coord(iy, ix));  // (kTileM, kTileN)

  // shared memory
  auto sA = make_tensor(make_smem_ptr(Ashm),
                        SmemLayoutA{});  // (kTileM, kTileK, kStage)
  auto sB = make_tensor(make_smem_ptr(Bshm),
                        SmemLayoutB{});  // (kTileN, kTileK, kStage)
```

分别定义了分块的大小和设备端如何通过 Tensor 抽象和 local_tile 将矩阵进行分块。

流水线方面，为了做 multi stage 流水线，需要在共享内存分配时指定流水线级数，同时设备端需要做必要的数据加载和计算重叠方案。主机端代码如下

```
  static constexpr int kShmLoadSwizzleM = 3;
  static constexpr int kShmLoadSwizzleS = 3;
  static constexpr int kShmLoadSwizzleB = 3;

  using SmemLayoutAtom = decltype(composition(
      Swizzle<kShmLoadSwizzleB, kShmLoadSwizzleM, kShmLoadSwizzleS>{},
      make_layout(make_shape(Int<8>{}, Int<kTileK>{}),
                  make_stride(Int<kTileK>{}, Int<1>{}))));
  using SmemLayoutA = decltype(
      tile_to_shape(SmemLayoutAtom{},
                    make_shape(Int<kTileM>{}, Int<kTileK>{}, Int<kStage>{})));
  using SmemLayoutB = decltype(
      tile_to_shape(SmemLayoutAtom{},
                    make_shape(Int<kTileN>{}, Int<kTileK>{}, Int<kStage>{})));
```

其定义了共享内存的 Layout，其中 Swizzle 用来避免 bank conflict，其详细介绍可以参考前序文章 cute 之 Swizzle 抽象，其中 kStage 表示流水线的级数。

核心设备端代码由两个 for 循环组成，具体如下，

```
  // loop over k: i. load tile, ii. mma
  int ntile = k / kTileK;
#pragma unroll 1
  for (int itile = 0; itile < ntile; ++itile) {
    int nk = size<2>(tCrA);

#pragma unroll
    for (int ik = 0; ik < nk; ++ik) {
      int ik_next = (ik + 1) % nk;

      if (ik == nk - 1) {
        cp_async_wait<kStage - 2>();
        __syncthreads();

        ismem_read = (ismem_read + 1) % kStage;
      }

      // shm -> reg s[itile][ik + 1] -> r[ik + 1]
      cute::copy(s2r_tiled_copy_a, tAsA(_, _, ik_next, ismem_read),
                 tCrA_view(_, _, ik_next));
      cute::copy(s2r_tiled_copy_b, tBsB(_, _, ik_next, ismem_read),
                 tCrB_view(_, _, ik_next));

```

```
      if (ik == 0) {
        if (itile_to_read < ntile) {
          cute::copy(g2s_tiled_copy_a, tAgA_copy(_, _, _, itile_to_read), tAsA_copy(_, _, _, ismem_write))
    ;
          cute::copy(g2s_tiled_copy_b, tBgB_copy(_, _, _, itile_to_read), tBsB_copy(_, _, _, ismem_write))
    ;

          ++itile_to_read;
          ismem_write = (ismem_write + 1) % kStage;
        }

        cp_async_fence();
      }

      cute::gemm(tiled_mma, tCrD, tCrA(_, _, ik), tCrB(_, _, ik), tCrD);
    }  // for ik
  }    // itile
```

外层循环对 Tile 做循环，内层循环对 Tile 内做 k 循环，同时在 ik == 0 和 ik == nk -1 的时候发射了后续 kStage-1 的 Tile 的全局内存到共享内存的数据加载和针对即将读取的共享内存的数据同步。

9.4　尾阶段高效（Epilogue）

通过上面的计算、加载、算法，我们便可以得到矩阵乘之后的块状的数据，这些数据表示为线程内寄存器，由于数据存储，将这些数据存储出去并不平凡，如图 3 所示，如果将寄存器数据直接写出，则在全局地址空间中会产生内存地址的不连续，这将导致存储时需要更多的内存事务，并且不能使用向量化存储指令（STG.128）。

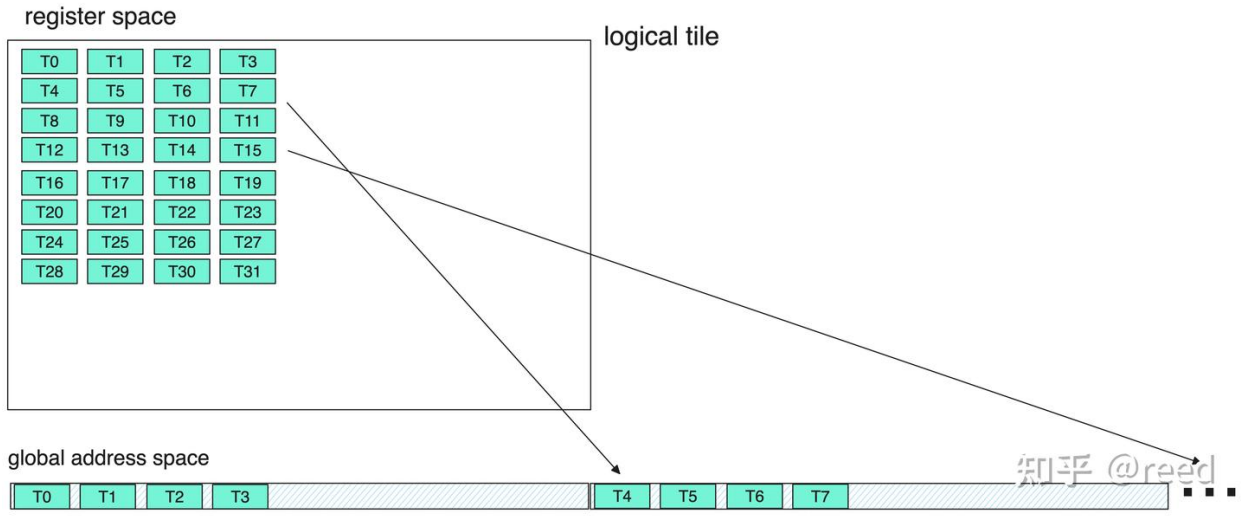


图 9.3: Figure 3. 寄存器堆直接存储至全局内存引入的不连续

针对这个问题，cute 中（实质为 cutlass 中），专门提供了 Epilogue 来通过共享内存作为中间媒介。先将寄存器数据存储到共享内存，然后再从共享内存中以更连续、更高位宽的形式存储到全局内存中去。PACT'20 Fireiron 文章有对该问题的详细探讨，可以参考之。

本文通过共享内存实现高效的 TileC 的存储，具体代码可以参考 github 上的实现，整体过程如图 4 所示。

9.5　试验设置和结果

根据上面介绍的优化方法，我们利用 cute 实现了高效的 Multi-Stage GEMM，代码地址为https://github.com/reed-lau/cute-gemm。我们针对具体业务中使用的问题规格进行了性能测试。具体地问题规格为：M=81920, N=256, K=256, 其中 A 矩阵为行优先，B 矩阵为列优先，C 矩阵为行优先，输入输出和计算数据类型均为 half。测试环境方面，显卡为 NVIDIA GeForce RTX 3090，操作系统为 Ubuntu 20.04.6 LTS，NVidia 驱动版本为 535.113.01，

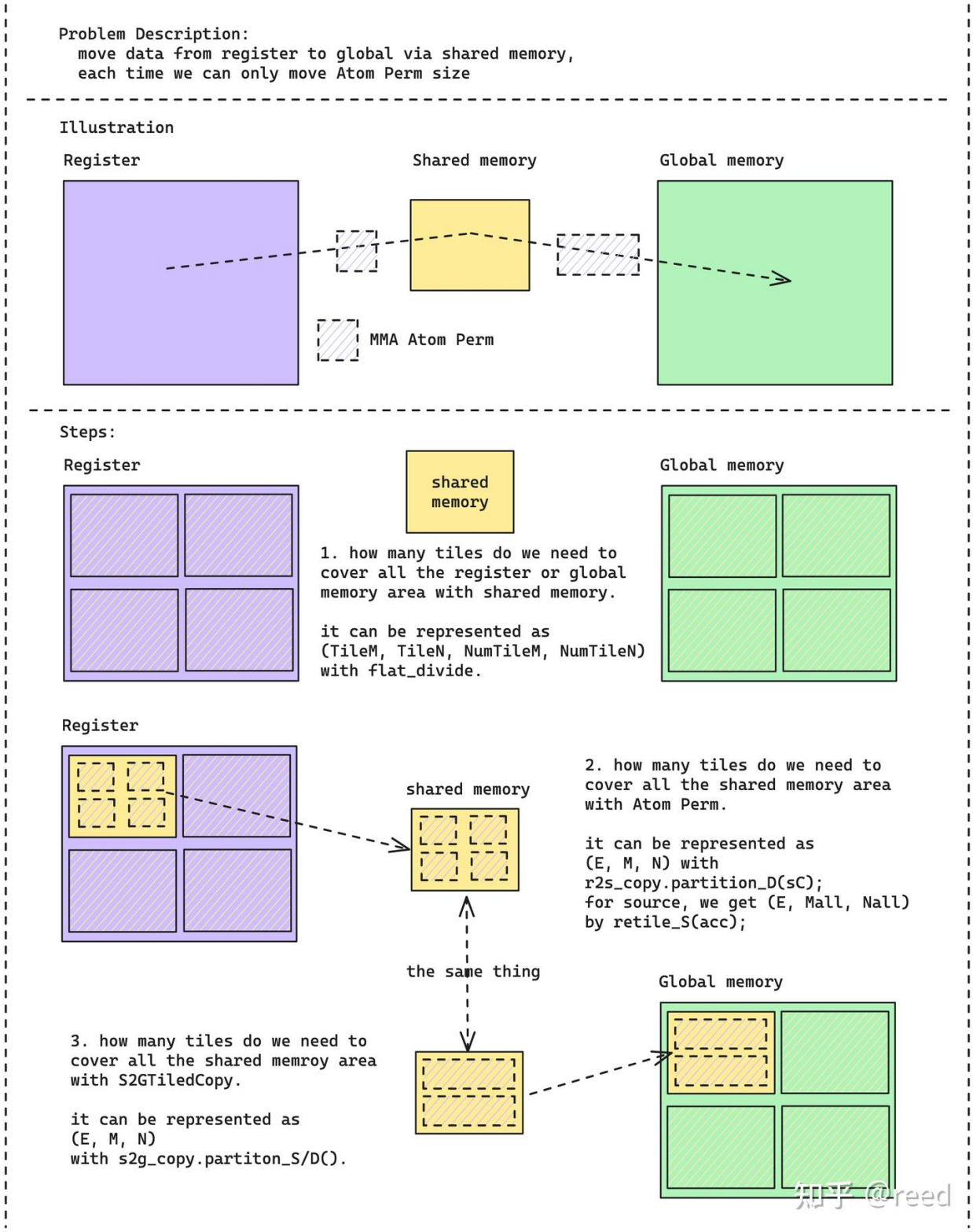


图 9.4: Figure 4. Epilogue 中 Accumulator 寄存器通过共享内存实现到全局内存的数据搬运

NVCC 编译器版本为 V11.7.64。我们对比了 cuBLAS 和 cuBLASLt 的调优的 kernel，其 cuBLAS 和 cuBLASLt 版本号为 111001。我们使用 cuBLASLt 的启发式方法选择尽可能多的 kernel 进行对比。

使用 Nsight Compute 对 kernel 运行时间进行精准的测量，同时为了避免 L2 Cache 缓存数据对效率评估的影响，我们主动设置`--cache-control=all`选项来保证在进行 kernel profiling 的时候清空历史 cache 数据，通过设置`--clock-control=base`选项来锁定 GPU 运行的频率，避免动态调频对性能测量的影响。

cuBLASLt 经过启发式算法选出 5 个 kernel，针对上面提到的问题规格进行了 11 次试验，我们统计了这 11 次的均值、方差和中位数，结果如下表格：

所属库	kernel 名称	均值 (us)	方差 (us)	中位数 (us)
our-impl	gemm_multi_stage	130.1	0.4	130.1
cuBLAS	`ampere_h1688gemm_128x128_ldg8_stages_32x1_tn`	153.4	2	154.1
cuBLASLt	`ampere_h16816gemm_128x64_ldg8_tn`	154.0	0.5	154.1
cuBLASLt	`ampere_h16816gemm_256x128_ldg8_stages_32x3_tn`	154.2	0.6	154.2
cuBLASLt	`ampere_h1688gemm_128x128_ldg8_stages_32x1_tn`	153.4	2	154.1
cuBLASLt	`ampere_h1688gemm_128x128_ldg8_tn`	136.5	0.7	136.5
cuBLASLt	`cutlass_80_tensorop_h16816gemm_128x256_32x3_tn_align2`	183.4	0.7	183.4

表格中的数据可以看出均值和中位数相当，所以测量结果相对稳定。我们看到我们的实现为 130.1 us，cuBLAS 和 cuBLASLt 的最好结果为 136.5us。针对该问题规格，我们的实现达到了 SOTA 的结果。

9.6 总结和讨论

9.7 启发式算法

回顾前文或者代码，我们不难发现代码中的很多参数是可以调整的，如矩阵分块的大小 kTileM、kTileN、kTileK，流水线级数 kStage 等。针对不同的问题规格适当的调整这些参数能得到更好的效果。逻辑上，我们可以用理论计算来设计公式实现不同问题规模对这些参数的选择；我们也可以对不同的问题规格进行一些离线的穷举试验，分析这些参数维度空间中的分布数据，分析这些数据得到不同问题规格和这些优化参数的映射关系，归纳出特定的启发式算法，使得我们给定一个问题时能够选择出一个不错的 kernel。本文只针对一个特定的 case 做优化，所以不涉及这部分内容，甚至可以直接参考 cuBLAS 选中的 kernel 对这些参数的设置情况。

9.8 参数相容性

前面的代码介绍，我们发现无论是 MMA 还是 Copy 每一个操作都需要设置很多参数，这些参数有的是描述 layout，有的是描述线程数，有的描述寄存器重复，有的是描述计算块的大小。这些参数并不是完全自由的，它们之间会存在某种约束，这种约束可以是共享内存大小、可以是寄存器多少、可以是线程块大小、可以是划分方法，这是我们调优时调整参数需要考虑的。

9.9 其它方面

CUDA 优化还有很多方面，如在流水线章节的最后部分我们介绍的提升 L2 命中率的 Thread Block Swizzle 方法，本文中的 N 维度不大，所以并没有使能；设置`__launch_bounds__`给编译器提供更多的优化提示本文也并未探究。本文的实现没有考虑数据地址非对齐和 Tile 非整除情形，这些在实际使用时都是有限制或者需要考虑的。同时 cutlass 的核心部分也已借助 cute 进行了 collective 和 epilogue 的抽象，我们也可以使用它来实现我们的矩阵计算，并且它们有更好的通用性。

9.10 总结

本文从计算高效、访存高效、算法高效、尾阶段高效这几个方面介绍了矩阵乘的高效实现方法。并且对核心代码进行了细致的解读。文章针对特定 case 将本文实现和 cuBLAS、cuBLASLt 进行了对比，实验结果表明，使用 cute 实现的 GEMM 方法可以达到 SOTA 的效果。

掌握 cute 的 Layout、Tensor、MMA、Copy 抽象可以帮助我们快速的构建矩阵计算能力，我们可以借助这种能力更高效的解决其它更复杂的问题（如 Flash Attention）。

这是该 cute 系列的最后一篇，希望该系列文章能够帮助大家快速掌握 cute。后续我将再选个 topic 和大家共同探讨高性能计算、高性能系统，希望我们很快又能见面。

9.11 参考

PACT'20 Fireiron

NVidia cutlass Epilogue

第十章　cute 之 Hopper 前言

作者：reed（知乎 @reed-84-49，专栏「CUDA 高性能编程」）　原文：
https://zhuanlan.zhihu.com/p/1948048832794427610

10.1　背景与动机

之前的 “cute 之 [1][2][3][4][5][6][7][8][9]” 系列文章我们重点介绍了 cute 中的核心概念和抽象，并且基于这些概念和抽象，按照自低向上的方式逐步构建起了利用 cute 实现 GEMM 计算的核心框架，这些内容主要是以 Ampere 架构为基础进行的介绍。2022 年 3 月，NVidia 在 GTC 大会上发布了 Ampere 的后继硬件架构-Hopper 架构 [10]，并于同年 10 月份发布了其针对 AI 计算市场的旗舰级产品 H100。如今已是 2025 年 11 月，经过 3 年的时间，该架构下的芯片已然成为各大数据中心的算力输出的核心产品，其构筑了当今 AI 计算的最核心力量。相较于前序的 Ampere 架构，Hopper 架构上增加了很多功能 feature，其编程范式也发生了一系列变化，这些 feature 和新范式组合在一起共同组成了核心矩阵计算能力。刚好我也由于工作上的原因有机会接触到 Hopper 架构的产品，所以形成此系列文章，一方面作为我个人学习认知的总结和沉淀，另一方面也分享给各位奋斗、挣扎 [11] 在 CUDA 进阶之路上的同仁们，希望它能澄清 Hopper 上的编程范式和结构，希望这个系列能够帮助到你们。

10.2　系列内容规划

总体构思上大概分为五篇来介绍该内容：

- cute 之 Hopper MBarrier
- cute 之 Hopper TMA
- cute 之 Hopper WGMMA
- cute 之 Hopper 硬件流水线
- cute 之 Hopper 高效 GEMM 实现

第十一章　cute 之 Hopper MBarrier

作者：reed（知乎 @reed-84-49，专栏「CUDA 高性能编程」）　原文：
https://zhuanlan.zhihu.com/p/1962636004235153810

Hopper 架构上提供了更高效的计算单元和数据搬运单元，通过数据依赖协调这两个单元的执行进度十分重要，为此 Hopper 架构上提供了 MBarrier 功能单元，本文将重点介绍 MBarrier 相关的功能，数据表示和状态转移，并以实验的形式呈现该状态转移。文章结构上，首先回顾了 NVidia GPU 上的 Tensor Core 和数据搬运单元，由此引入了 GPU 上的同步单元，然后重点介绍了 MBarrier 的核心功能、数据结构表示和其状态转移，最后我们给出了文中讲解的示例代码并对文章进行了总结。

11.1　计算和数据搬运的桥梁

在高性能处理器架构体系中，有两个核心点，一个核心是计算，另一个核心是数据搬运。处理器的推陈出新始终是在围绕更高效的计算核心和更高效的数据搬运引擎在演进。

NVidia GPU 的演进自然也在这个范式之内，如图 1 所示，从 Volta 架构开始，GPU 的处理器单元 SM（Stream Multiprocessor）上装配了 Tensor Core 来提高计算能力，尤其是矩阵计算能力，从 Turing 架构开始提供了`ldmatrix`指令以提升共享内存（shared memory）向寄存器（register）进行数据搬运的能力，Ampere 架构上提供了`cp.async`指令实现全局内存（global memory）到共享内存异步数据搬运的能力，在 Hopper 架构中针对计算部分，其装配了具有异步执行能力的 WGMMA（WarpGroup Matrix Multiply-Accumulate）Tensor Core，针对数据搬运部分，其装配了独立的数据搬运 TMA（Tensor Memory Accelerator）单元。

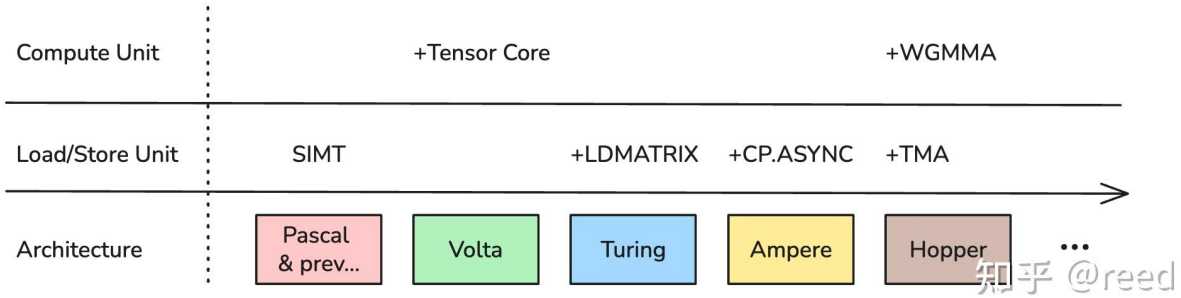


图 11.1: Figure-1. Compute and Load/Store unit evolution

Hopper 架构之前的 Tensor Core 单元，在进行矩阵乘加计算时其指令的源操作数和目的操作数都是寄存器（如图 2 中`mma`指令），其指令执行周期也是固定的，Hopper 架构的 Tensor Core 计算单元除了可以完成 mma 指令，还可以连续的四个 warp 组成 warp group 共同完成更大规格的矩阵乘法，这是使用的指令为 warp group 层级的指令，即 WGMMA 指令，该指令逻辑上的输入矩阵 A 除了可以存储在寄存器上，还可以存储在共享内存上，B 矩阵必须存储在共享内存上，C 矩阵为输出矩阵，其存储在寄存器上。不同于同步的 MMA 指令，该 WGMMA 指令是异步的，需要配合额外的 commit 和 wait 指令完成计算并且确保结果的可见性。由于其计算规格更大，并且多个 warp 协同完成，而且支持更低的输入精度（如 FP8）其计算效率更高。

Hopper 除了提供了高效的 WGMMA 能力完成矩阵计算，同时还提供了高效的数据拷贝引擎 TMA，它可以实现全局内存和共享内存之间异步高效数据搬运。这样便可以实现 TMA 完成全局内存到共享内存的加载，异步的 Tensor Core 利用 WGMMA 实现从共享内存读取数据进行矩阵运算，运算结束后将结果输出到全局内存。如图 3 所示，一个典型的 GEMM（General Matrix Multiplication）实现会循环的从全局内存读取数据写入到共享内存，同时 Tensor Core 循环的读取共享内存数据进行矩阵运算，将结果写入寄存器。可以看到异步的数据搬运单元 TMA 和计算单元 Tensor Core 通过中间介质共享内存完成数据依赖的解耦，逻辑上 Tensor Core 并不需要知道共享内存中数据是怎么获得的，其只需要知道其计算依赖的数据是不是就绪，如果 Tensor Core 所依赖的数据在共享内存中已经就绪，其就可以开始计算，如

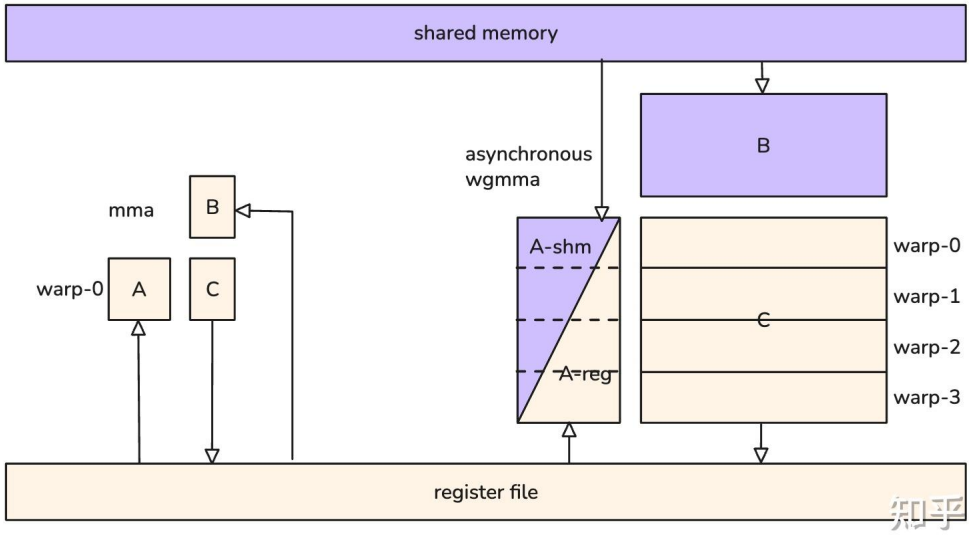


图 11.2: Figure-2. Hopper tensor core instruction operand source location

果共享内存中不间断的有数据就绪，则 Tensor Core 可以不间断的进行运算，达到 Tensor Core 的最大使用效率，同时针对异步的 TMA 而言，其也不需要关注共享内存的数据被谁消耗，其只需要关注有没有可以用来写入的共享内存空间，如果这块空间没有被依赖则其可以写入新的数据以供 Tensor Core 来使用，通过共享内存作为中间数据交换点，可以解耦数据加载和计算，使得单边点性能达到最高，同时使用共享内存作为中间缓冲区，可以更好的平衡由于数据加载抖动引起的计算性能波动。

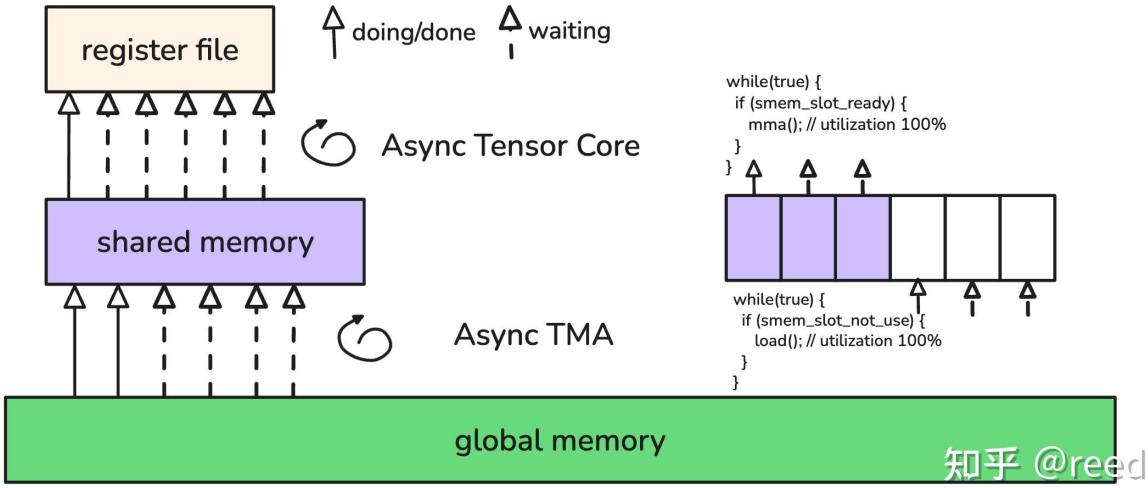


图 11.3: Figure-3. A typical GEMM data dependency and decoupled by shared memory

以上编程范式在传统的算法中表现为生产者消费者模型，在一个典型的 C++ 实现中，其通过锁和条件变量实现，为了更好的适配这种编程范式，使得 Tensor Core 和 TMA 能够更好的发挥各自的性能，Hopper 架构上提供了能够协调线程到达，数据到达和等待的高效的同步机制，即 MBarrier。

11.2 MBarrier

传统的 barrier（栅栏）在 NVidia GPU 体系上实现为独立的硬件单元，如常用的 CUDA 线程同步函数`__syncthreads()`，其有两条语义，第一是执行同步，第二是内存可见性屏障。对于执行同步而言，即所有参与线程在调用`__syncthreads()`后都会等待其他线程，只有当所有线程到达该同步点后才能继续执行后续的指令，即其可以表达为两个步骤，i. 线程到达，ii. 等待其他线程到达；对于可见性而言，`__syncthreads()`能够确保该调用之前的任意线程发起的共享内存写入操

作对该调用之后的所有线程可见，借用 C++ 内存模型表示，可以简单的表达为在线程同步之前调用内存 release() 语义，在同步之后施加 acquire 语义，简单的可以将__syncthreads()拆解如下

```
void __syncthreads() {
  memory_release();
  thread_arrive();
  wait_all_threads();
  memory_acquire();
}
```

除了上面的线程块（block）内所有线程都需要参与的同步机制__syncthreads(),CUDA 还通过 PTX 提供了 named barrier[1]同步机制bar.sync a b; 其中 a 表示 named barrier 的 id，每一个线程块可以使用 0 到 15 共 16 个有名的 barrier，b 表示线程块内有多少线程需要参与。相较于__syncthreads()的所有参与线程都需要同步，named barrier 可以实现部分线程的同步，可以实现更精细的同步控制，其核心逻辑可以拆解如下：

```
void bar_sync(int id, int count) {
  memory_release();
  thread_arrive(id);
  wait_threads(id, count);
  memory_acquire();
}
```

更进一步地，为了提高 barrier 的控制能力和细节，NVidia 提出了 mbarrier，它缩写自 memory barrier，意为在内存上的 barrier，这里的内存为共享内存，概念上它不再是有限个数的硬件单元，而是可以和共享内存的空间一样大（如图 4 所示）。它实现了更精细的语义控制，提供独立的 arrive 和 wait 能力，除此之外 mbarrier 还将 TMA 异步拷贝的完成机制也合并到其中，当 TMA 拷贝完成时能够改变该 mbarrier 的状态，这样使用统一的 wait 就可以等待 TMA 异步拷贝完成。

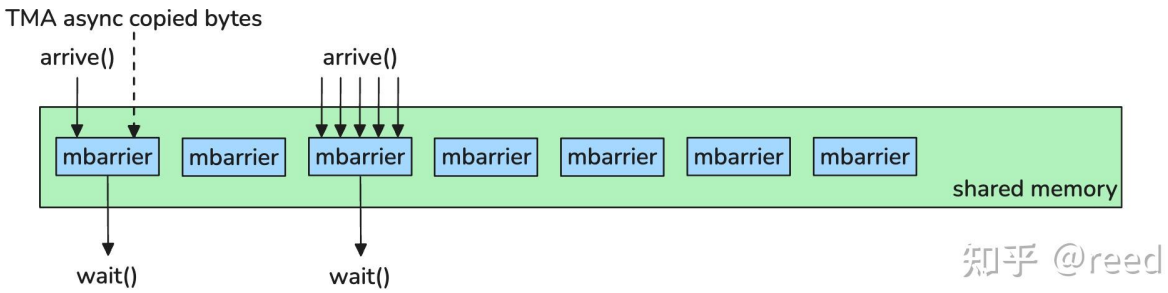


图 11.4: Figure-4. mbarrier semantic

mbarrier 使用 SM 上的共享内存作为存储后端，为了提高其操作效率，硬件层面提供 cache 机构对其加速，只在初始化和销毁 barrier 的时候 cache 内容才会写回（write back）共享内存，其他操作均可以发生在 cache 内，并不向共享内存写回，所以虽然从指令层面看，barrier 对应的 SASS 指令 SYNCS 操作对象为共享内存，但是其操作本身并不用向共享内存写回，其效率要远高于共享内存操作。

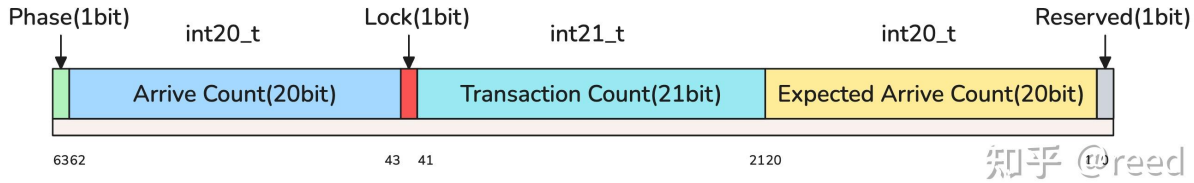


图 11.5: Figure-5. MBarrier fields

mbarrier 在数据表示上表达为一个 64bit 的整数类型数据，其内部各个域的定义如图 5 所示，整体分为六个域，分

[1]https://link.zhihu.com/?target=https%3A//docs.nvidia.com/cuda/parallel-thread-execution/%23parallel-synchronization-and-communication-instructions-bar

别是最低位的一比特的保留域，第一比特到第二十比特的 Expected Arrive Count 域，第二十一比特到第四十一比特的 Transaction Count 域，第四十二比特位置的 Lock 域，第四十三比特到第六十二比特的 Arrive Count 域和第六十三比特位置的 Phase 域。

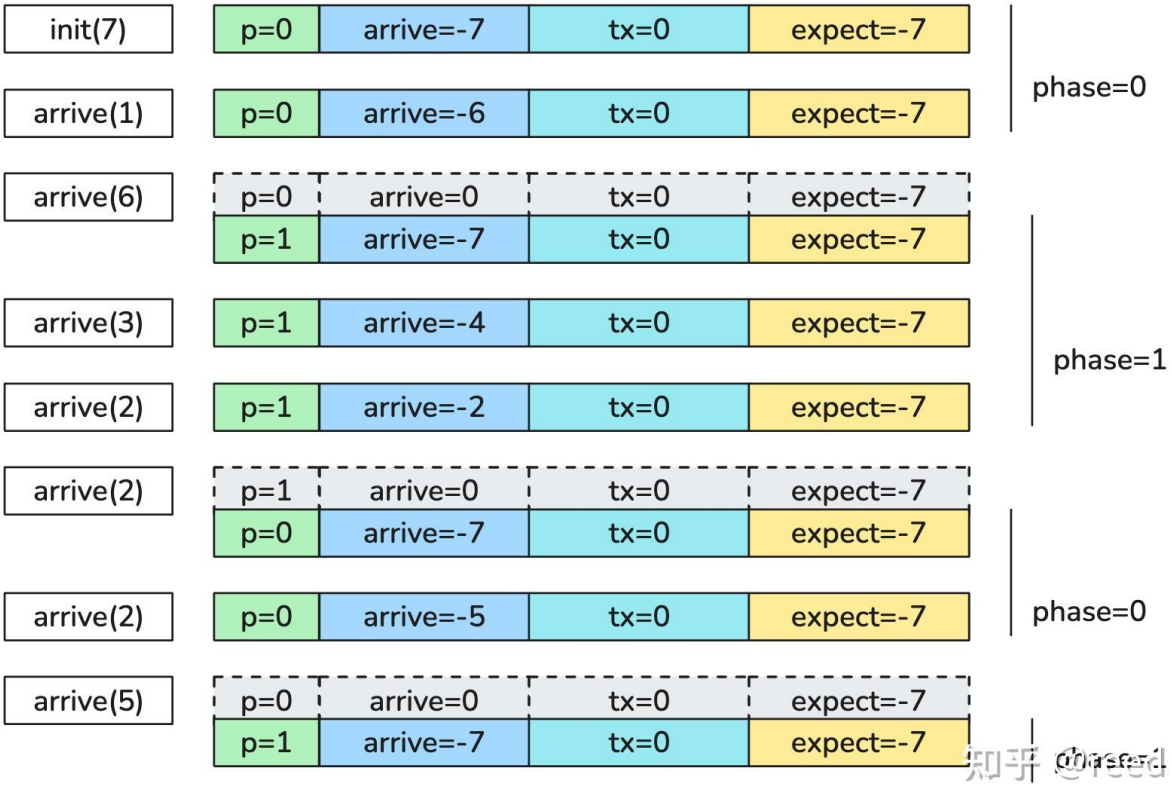


图 11.6: Figure-6. mbarrier state update with arrival

具体地，在 mbarrier 初始化的时候，Expected Arrive 和 Arrive Count 域会被设置为负数表示的初始化数值，如初始化 mbarrier 为 7，即等待 7 个到达的 arrive，则 Expected Arrive Count 和 Arrive Count 都会被设置为-7，表示需要等待 7 个到达，并且这两个域都是二十比特，高位为符号位，表示为有符号整数类型`int20_t`，表示正数使用原码，表示负数时使用补码，如-7 表示为`b11111111111111111001`，初始化时 Transaction Count 域会被设置为 0，该域和 Arrive Count 域类似，也是有符号的数据，只是 bit 位数比其多一，整体表示为数据类型`int21_t`，初始化时 Lock 域会被设置为 0，表示是一种正常的状态，Phase 域在初始化的时候被设置为 0 表示当前为 phase 0，其只有一比特表示，所以其是只有 0 1 两种状态，使用 0 1 两种状态来完成对 mbarrier 的状态维护和复用，具体的理论可以参考并行计算领域的 Sense Reversing Barrier 部分。如图 6，当有线程调用 arrive(n) 的时候则 Arrive Count 域会加上 n，当加上 n 之后如果 Arrive Count 域刚好等于 0，则表示该 mbarrier 完成，则其 phase 自动切换到下一个状态（如由 0 切换到 1，或由 1 切换到 0），同时 Arrive Count 域自动重置为 Expected Arrive Count，表示在新的 phase 重新需要等待新的 arrive 到来。如果加上 n 之后不能到达 0，则表示这次到达并不能完成该 phase，则 phase 不变，只是 Arrive Count 加上 n，需要等待后续的 arrive 到来才能完成该 phase，如果一次 arrive 的数量 n 加上 Arrive Count 后大于 0，则此 mbarrier 出错，Lock 设置为 1 进入错误锁定状态。当 phase 切换后（如图中的灰色表示），对该 mbarrier 的 wait 条件是可以满足的，则调用 wait(phase) 线程可以继续，否则 wait 线程会等待 Arrive Count 到达 0 才可以继续执行，值得注意的是初始化状态，即`p=0,arrive=-7,tx=0,expect=-7`，也可以认为是由`p=1,arrive=0,tx=0,expect=-7`切换而来的状态，所以在该状态进行 wait(phase=1) 是满足的。

以上介绍了 mbarrier 状态随着 arrive 事件各个 field 的更新过程，下面我们介绍由 transaction count 相关的状态转换，如图 7，mbarrier 初始化时设置为 3，则 arrive count 和 expected arrive count 都会设置为-3 表示需要等待 3 次 arrive，同时 transaction count（tx）会被初始化为 0，phase 会被初始化为 0。此时我们设置 mbarrier 需要等待的 transaction bytes，如图示例，我们设置了 1024bytes，设置后则 tx 更新为-1024，表示需要等待 1024bytes 数据的到来，和前面介绍的类似，我们可以通过 arrive 来完成到达，同时我们会把 mbarrier 设置给 TMA 的拷贝单元，当 TMA 完成拷贝时，其会自动更新 mbarrier 的 tx，如图中虚线框 async_complete 所示，当 TMA 完成数据拷贝时，其会通知

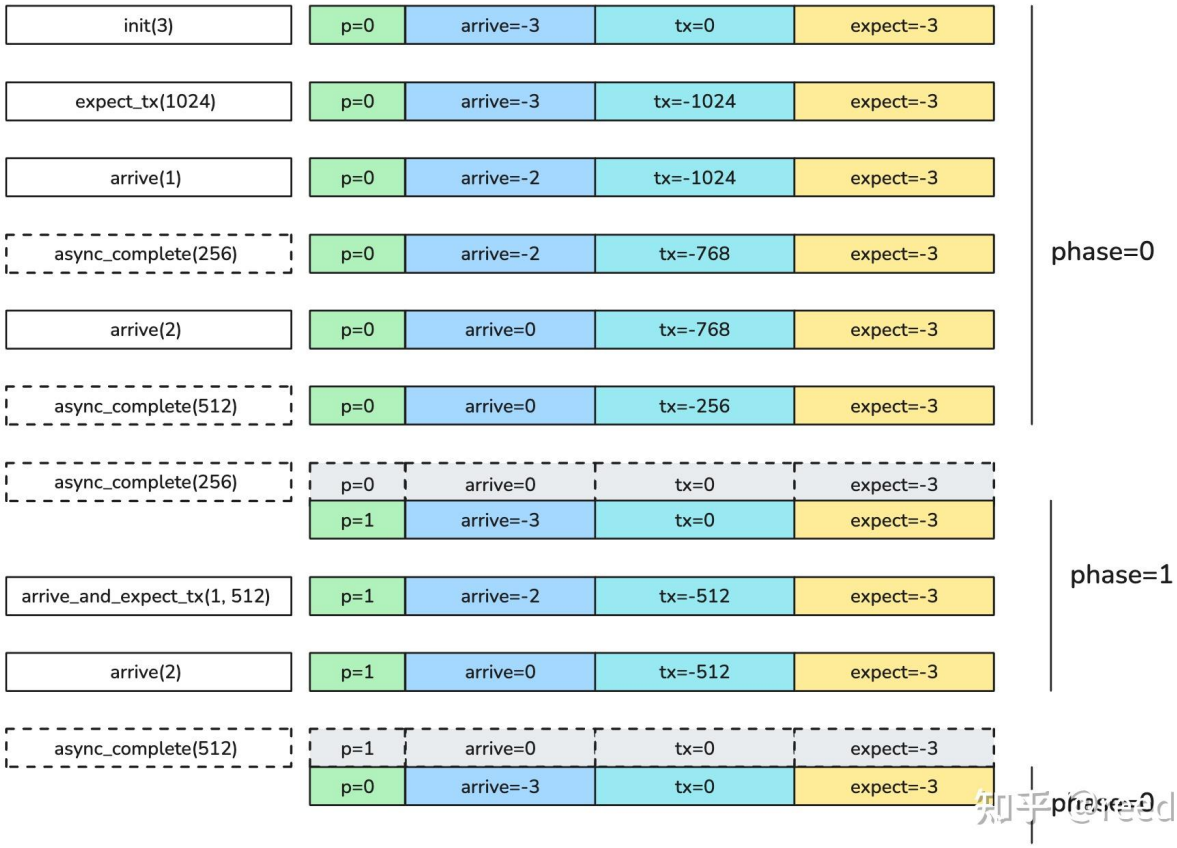


图 11.7: Figure-7. mbarrier state update with asynchronous transaction

mbarrier 数据到达，从而更新 mbarrier 的 tx 字段会加上到达的数据量，当 tx 到达后如果 arrive 字段和 tx 字段同时到达 0，则 phase 完成反转，Arrive Count 字段会重置为 Expect Arrive Count 的值，同时 Transaction Count 域会被重置为 0，这时可以对其进行新一轮等待数据进行设置，来进行对 mbarrier 的重新复用，图中的灰色框表示该 phase 的完成，此时对该 mbarrier 进行 wait 相应的 phase 则可以完成，如果在该位置之前进行等待则需要等 arrive 和 tx 事件完成方可等待完成，在 phase 反转之后对上一 phase 进行等待也可以立即完成。mbarrier 同时提供了指令能够同时指定 arrive 和 tx，其能原子的实现对该数据的更新，减少操作 mbarrier 的指令调用次数。除了对 mbarrier 直接进行 wait，还可以通过 PTX 提供对 test 指令来实现对 mbarrier 状态的探测。

mbarrier 提供的 wait 可以指定内存可见性 scope，这样当 wait 成功时，可以确保整个 scope 是可以看见其内存副作用的，避免了需要所有线程 wait。

CUDA 以 PTX 的形式提供了 mbarrier 的能力，cute 针对 mbarrier 进行了函数封装（位于 cute/arch/copy_sm90_desc.hpp），常用的有

```
void
initialize_barrier(uint64_t& smem_barrier,                 // 64 bits user-managed barrier in smem
                   int thread_count = 1)                   // Thread count expected to arrive/wait on this barrier
void
set_barrier_transaction_bytes(uint64_t& smem_barrier,      // 64 bits user-managed barrier in smem
                              uint32_t bytes)              // Number of bytes transfered by per TMA transaction
void
wait_barrier(uint64_t& smem_barrier,                       // 64 bits user-managed barrier in smem
             int phase_bit)                                // Current phase bit the barrier waiting to flip
void
arrive_barrier(uint64_t& smem_barrier)                      // 64 bits user-manged barrier in smem
```

还有一部分能力被封装在了 cutlass/arch/barrier.h 中。

11.3 使用示例

针对以上 mbarrier 的状态转移过程，通过代码进行了验证，由于通常情况下 mbarrier 是被缓存在 cache 中的，并且不会写回共享内存，正常情况下我们无法通过读取共享内存来获取 mbarrier 的数据，但我们可以通过调用`mbarrier.inval`指令来销毁 mbarrier，这会使得 cache 中的数据向共享内存写回，这时我们就可以通过读取共享内存（LDS）来获取 mbarrier 的状态了。具体实验代码参见https://github.com/reed-lau/cute-gemm/tree/main/mbarrier[2]。

11.4 总结

本文介绍了 Hopper 上硬件加速的事务型内存同步机制 MBarrier，重点介绍了其核心的数据表示和状态转移，它作为 Hopper 上高效的同步机制是实现生产者-消费者模式的必要组件，是 TMA 和 Tensor Core 形成流水线的核心组件，同时本文通过代码示例展示了 mbarrier 的状态转移过程。

11.5 参考

https://patents.google.com/patent/US20230289242A1/en[3]

https://docs.nvidia.com/cuda/parallel-thread-execution/#parallel-synchronization-and-communication-instructions-bar[4]

https://mattchung.me/blog/2020/09/18/making-sense-of-the-sense-reversing-barrier-synchronization/[5]

https://en.cppreference.com/w/cpp/atomic/memory_order.html[6]

[2] https://link.zhihu.com/?target=https%3A//github.com/reed-lau/cute-gemm/tree/main/mbarrier

[3] https://link.zhihu.com/?target=https%3A//patents.google.com/patent/US20230289242A1/en

[4] https://link.zhihu.com/?target=https%3A//docs.nvidia.com/cuda/parallel-thread-execution/%23parallel-synchronization-and-communication-instructions-bar

[5] https://link.zhihu.com/?target=https%3A//mattchung.me/blog/2020/09/18/making-sense-of-the-sense-reversing-barrier-synchronization/

[6] https://link.zhihu.com/?target=https%3A//en.cppreference.com/w/cpp/atomic/memory_order.html

第十二章　cute 之 Hopper TMA

作者：reed（知乎 @reed-84-49，专栏「CUDA 高性能编程」）　原文：
https://zhuanlan.zhihu.com/p/1965678344352731952

在 Hopper 架构之前所有的数据拷贝都是多个线程共同完成的，它包含每个线程完成大致相似的地址计算，每一个线程最多单次完成 16 字节的数据加载，通过多次循环完成更大规格的数据搬运，但是在 Hopper 及其之后的架构中，为了提高数据的加载效率，减少复杂且容易出错的加载流程，NVidia 在 Hopper 架构中将数据搬运任务抽象成独立的硬件单元 TMA（Tensor Memory Accelerator）。借助 TMA 的能力，Hopper 上可以实现高效的数据搬运。本文首先介绍了传统的 SIMT 编程模型以及该模型下的数据搬运，然后重点介绍了 Hopper 上的 TMA 能力和编程模型的变更，详细介绍了它的 TMA 描述符、发起拷贝任务等特点，最后文章介绍了 cute 对 TMA 能力的封装和抽象。

12.1　SIMT 编程模型中的数据搬运

CUDA 从一开始就选定了 SIMT 编程模型，从编程逻辑角度由宏观到微观看，每一个程序（kernel）由一个线程网格（grid）组成，每一个线程网格由一个或多个线程块（block，也称为 thread block）组成，每一个线程块由一个或多个线程（thread）组成（如图 1）。线程网格具有一定的局部性，在线程网格内的多线程之间可协同的完成计算和同步。

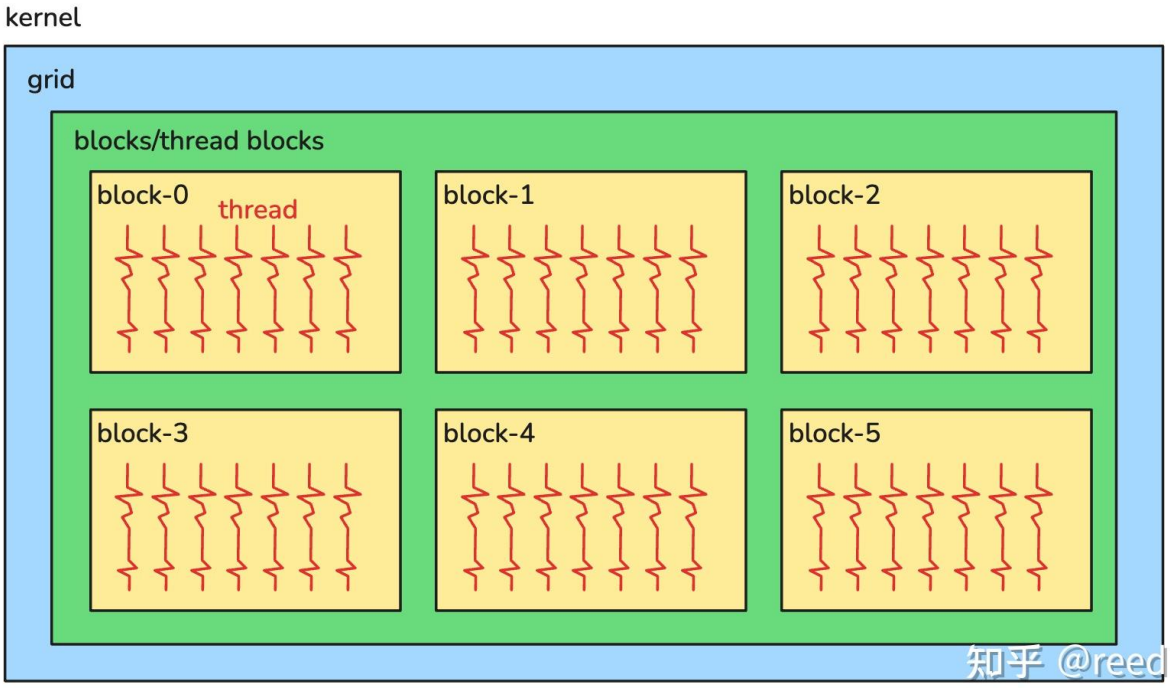


图 12.1: Figure 1. SIMT Programming Model

编程模型是软件模型，其本质是对硬件结构和能力的抽象。SIMT 编程模型之所以这样抽象，是由 NVidia 硬件的设计模式决定的。如图 2 所示，NVidia GPU 硬件架构整体分为四层，最内层的执行单元为带有 Predication 能力的 SIMD 核心，其可以在单个 cycle 内向量化的完成浮点运算，图中 FP32 核心可以在一个周期内完成 32 个 32bit 的符合 IEEE 标准的浮点乘法、加法、乘加运算，可以通过 Predicate 来决定该 lane 是否执行。和 FP32 功能单元类似，NVidia 硬件还提供了整数计算单元（INT32）、64bit 浮点计算单元（FP64）、矩阵计算单元（TensorCore）、数据加载单元（LSU）和特殊函数单元（SFU，可以完成正弦、余弦、倒数、开平方、E 指数、对数等计算），这些不同计算类型的单元组合的在一起形成一个多功能计算能力，同时组合上 Warp 调度器（warp scheduler）和分派单元（dispatch unit）共同形成 subcore，其可以实现 warp 级别调度和执行能力。4 个 subcore 组合在一块形成更大的计算能力，同时附加上 L1 数据缓存（L1 data cache）和共享内存（shared memory）形成流处理器（Stream Multiprocessor，简称 SM），在 SM 上会

有 Barrier 硬件单元能够协同多个 subcore 的数据同步和执行进度，共享内存提供本地局部的数据复用。多个 SM 组合在一起同时附加上 L2 缓存（L2 Cache）和高带宽内存（High Bandwidth Memory）形成 GPU 设备（device）。

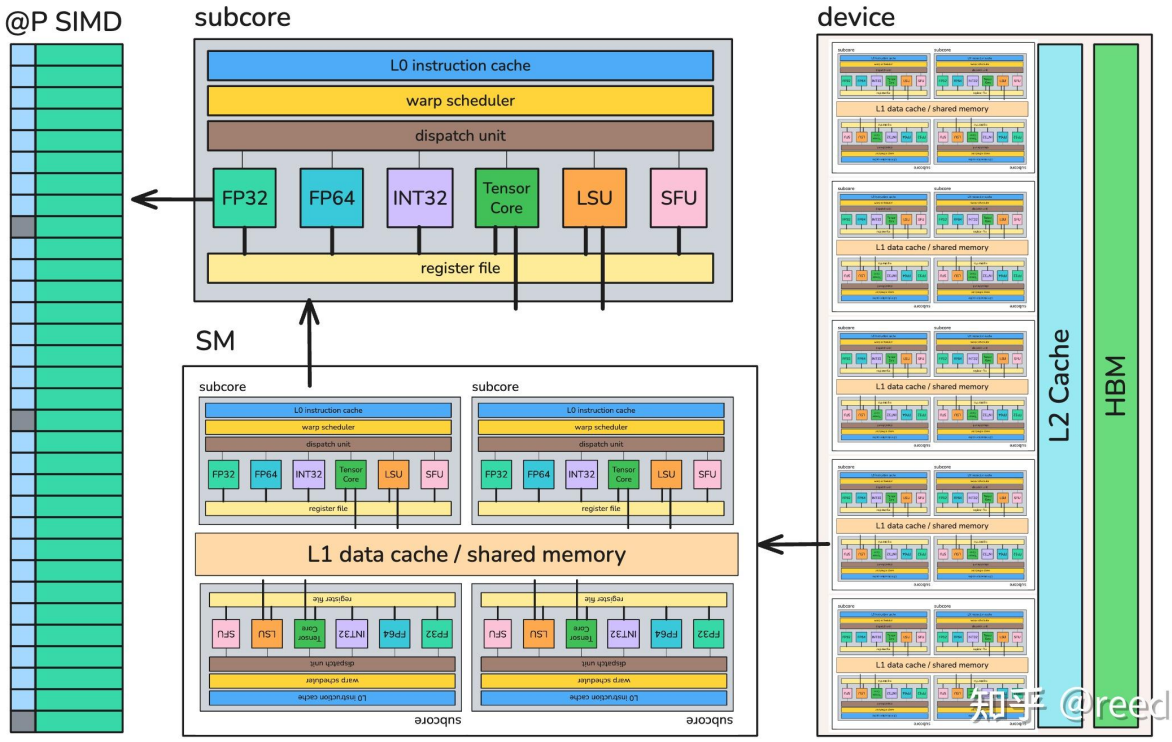


图 12.2: Figure 2. NVidia GPU Hardware Architecture Abstraction

软件上的 Grid 是硬件 Device 的抽象，并且通过调度单元使得数量有限的 SM 可以支撑远超其规格的 grid 大小。软件上的 block 是硬件 SM 的抽象，block 内包含多个 warp，block 内线程间可以通过共享内存共享数据，可以做线程同步，本质则是硬件在 SM 层级提供了可编程的 cache 机构 shared memory，同时提供了执行同步的 barrier 能力。当一个 block 使用的资源不多的时候，一个 SM 可以承载多个并行的 block 来提高资源利用效率。软件上的 warp 是对硬件 subcore 的抽象，单个 warp 内可以进行并行的计算，可以完成多种类型指令的计算任务。而带 Predicate 的 SIMD 核心则是线程的抽象，通过 Predicate 来 mask 掉某些不需要执行的 lane，从而实现 SIMT 语义。

当我们使用 GPU 完成一个特定问题的求解时，本质是将任务拆分为可并行的更小的子任务，这些任务最终被分配到一个并行执行的 FP32 或者 INT32 等核心上执行。具体到每一个线程，其根据其所在的 block 号和 thread 号（CUDA 中的 blockIdx 和 threadIdx）来映射一个问题域的很小的计算逻辑。针对数据计算和加载都是这样的。整体可以表示为 $T = P(\text{threadIdx}, \text{blockIdx})$ 。根据任务的划分使用 FP32/INT32/LSU 等来完成对数据的加载和计算。

以上便是 SIMT 模型下 GPU 的工作逻辑，针对于深度学习场景，其计算面对的输入输出通常是高维的 Tensor，而在任务划分后所需要完成的数据拷贝也是高维空间中的一块数据，如常用的 GEMM 类计算，在 Ampere 架构上每一个线程会负责的任务有：

- 将数据加载到共享内存（使用 INT32）
- 读取共享内存数据读取到寄存器
- 使用 TensorCore 完成矩阵乘加计算
- 将结果写出到共享内存
- 读取共享内存数据到寄存器
- 将寄存器数据写出到全局内存

不难发现以上 GEMM 计算相关的内存操作中涉及大量的共享内存的操作，原因是使用共享内存可以提高数据复用，减少对全局内存的访问量，减少对低带宽通路的占用，继而提升计算效率。但是针对全局内存和共享内存之间的读取和写入而言，其会涉及到大量的地址计算和繁琐的处理，以读取全局内存写入共享内存为例，其需要使用 INT32 单元计算

全局内存上作为输入的数据的地址，使用 INT32 单元计算共享内存输出的地址，同时由于共享内存的 bank 结构，需要引入避免 bank 冲突的逻辑，也会消耗整数和逻辑 ALU 指令，除了做 bank 冲突处理，矩阵计算一般是按照块进行的，对于边界不能整除的场景（tile quantization），经常需要对读取进行 mask 掉，对于输出的共享内存部分则需要填充上零，也会引入额外的指令消耗。值得注意的是，在 SIMT 编程模型下，这些操作需要所有的线程参与。并且处理这类问题很容易出错，有统计表明，CUDA 开发者 90% 的时间是在处理数据访问相关的问题，换句话说 CUDA 类程序员大部分时间是在处理 illegal memory 相关的问题。

12.2 高效的异步 Tensor 内存拷贝加速单元

针对 SIMT 意义下这种硬件计算浪费、编程错误率高、开发效率低的问题，NVidia 在 Hopper 架构上给出了全局内存和共享内存之间数据拷贝的软硬件协同的高效的解决方案。硬件方面提供独立的张量内存加速单元（Tensor Memory Accelerator 简称 TMA），它将数据拷贝相关功能进行抽象并高度集成，形成独立的硬件单元，使其能够完成从一维到五维的全局内存和共享内存之间的大块 Tensor 语义的拷贝，拷贝的同时能够配置避免共享内存 bank 冲突的 Swizzle 模式，同时可以配置超过边界（Out Of Bound，OOB）时的填充模式，为了提高拷贝和计算可并行的效率，这些拷贝是异步执行的，即 SM 在发起拷贝任务后可以立即进行其他的任务，线程并不会因为拷贝而被阻塞。软件方面也打破了之前 SIMT 的编程范式，不再需要所有的线程参与地址的计算和数据读取，而是只需要一个线程即可以发起大块的 Tensor 的拷贝任务。

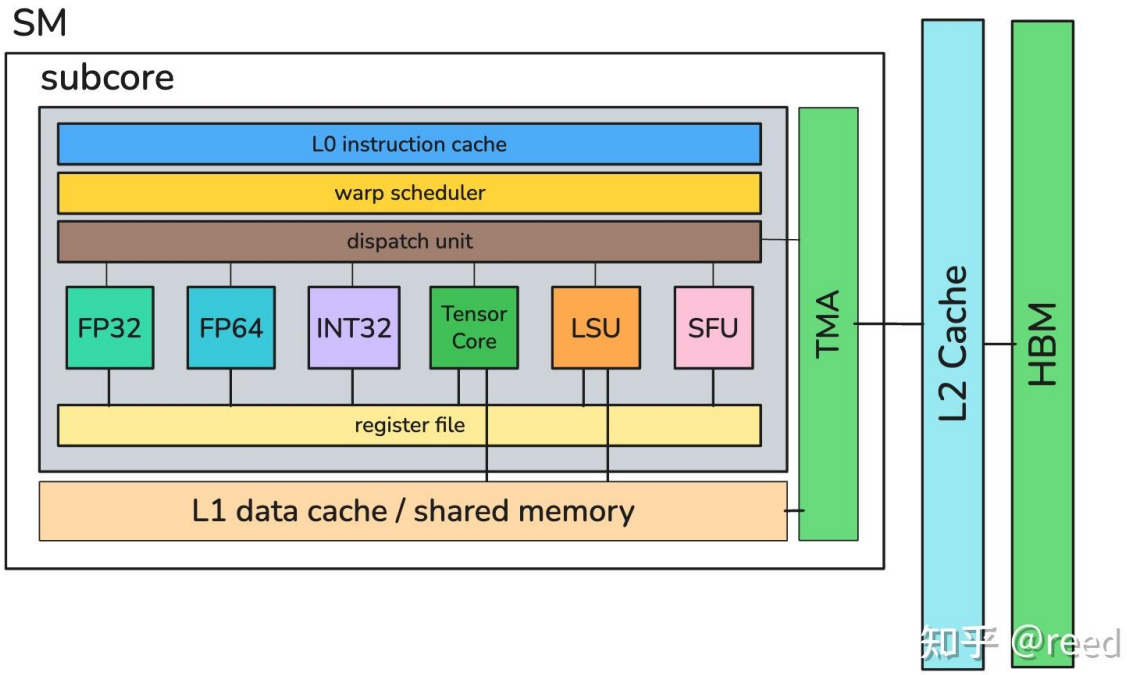


图 12.3: Figure 3. TMA Connection Illustration

如图 3 所示，TMA 在硬件结构上和 SM 一一对应，其被布置在 SM 内部，Dispatch 单元可以向 TMA 单元发送数据拷贝指令，该指令发送后 TMA 则可以异步的进行数据拷贝，完成全局内存到 SM 内 Shared Memory 上的数据搬运。

12.2.1 TMA 描述符（TMA Descriptor）

使用 TMA 完成全局内存到共享内存到数据搬运需要两个阶段，第一阶段为定义 TMA 描述符，第二阶段为实施 Copy 阶段（Copy Operation）。TMA 描述符（TMA Descriptor）是一段内存结构其描述了完成拷贝所需要的必要信息，如图 4 所示，TMA 描述符中包含全局内存的描述信息，输入 Tensor 的全局内存地址，Tensor 的 Shape 和 Stride 表示；以及单次拷贝的拷贝块的大小表示（如图 CopyBox），和共享内存上 Tensor 的 swizzle 信息和 oob 信息。通过描述符中的信息 TMA 则可以获得输入 Tensor 的完整精准的描述以及实施拷贝时的需要拷贝的大小和数据量，同时知晓在向共享内存写入时应该如何做 Swizzle。在拷贝实施阶段，即 SM 发射了拷贝任务，则需要 SM 向 TMA 提供 TMA 描述符

位置信息，TMA 单元根据该位置读取上述信息，还需要提供要拷贝的块的起点位置（如图 xy 所示）；当以共享内存作为目标时，还需要提供共享内存的地址和作为拷贝完成机制的 mbarrier 的地址。

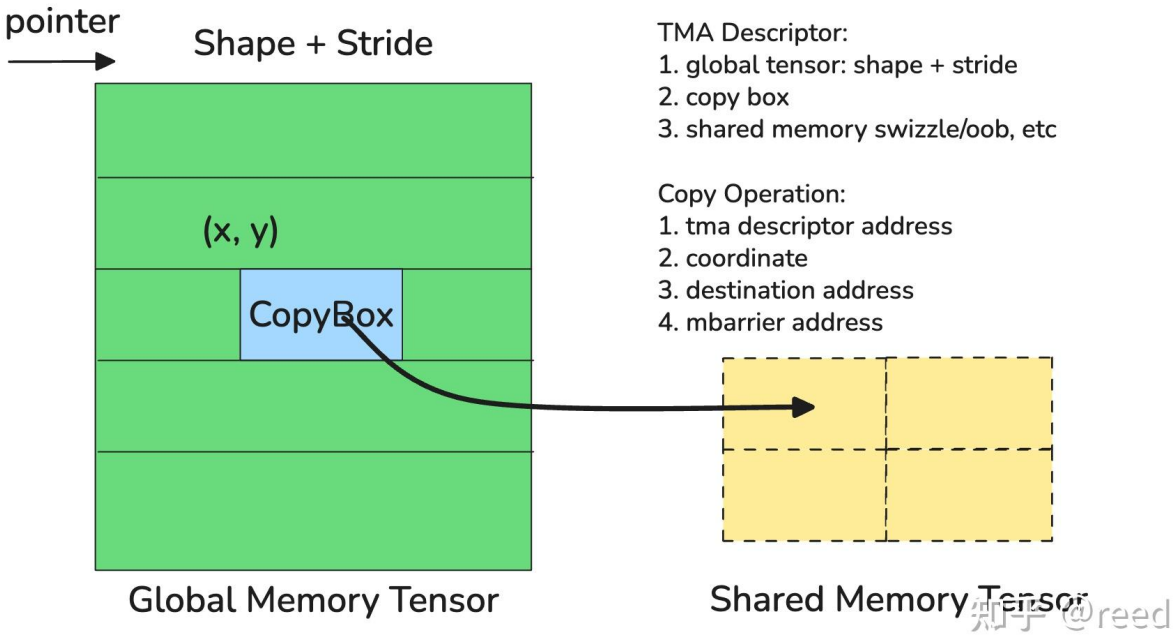


图 12.4: Figure 4. TMA operation mechanism

对于 TMA 描述符的构建，CUDA 提供了特定的 Driver API `cuTensorMapEncodeTiled`如下所示，其通过输入内存描述等信息，构建了 TMA 的描述符（tensorMap），在使用 TMA 完成拷贝时需要将该描述符传递给 CUDA Kernel。

```
CUresult cuTensorMapEncodeTiled (
      CUtensorMap* tensorMap,
      CUtensorMapDataType tensorDataType,
      cuuint32_t tensorRank,
      void* globalAddress,
      const cuuint64_t* globalDim,
      const cuuint64_t* globalStrides,
      const cuuint32_t* boxDim,
      const cuuint32_t* elementStrides,
      CUtensorMapInterleave interleave,
      CUtensorMapSwizzle swizzle,
      CUtensorMapL2promotion l2Promotion,
      CUtensorMapFloatOOBfill oobFill);
```

目前 TMA Descriptor 传递给 Kernel 有三种形式。第一种使用`const __grid_constant__` 修饰符通过 Kernel Parameter 传值的方法传递给 kernel，如下

```
__global__ void tma_desc_demo1(const __grid_constant__ CUtensorMap map);
```

第二种使用 CUDA 运行时 API`cudaMemcpyToSymbol` 将描述符拷贝到`__constant__`修饰的 device 变量上。

```
// device
__constant__ CUtensorMap device_tma_desc;

// Host
CUtensorMap host_tma_desc;
cudaMemcpyToSymbol(device_tma_desc, &host_tma_desc, sizeof(host_tma_desc));
```

第三种则是将 TMA Descriptor 存储在全局内存上，CUDA Kernel 内部直接向 TMA 提供该描述符地址即可

```
__global__ void update_tma(CUtensorMap *desc) {
  // update TMA Descriptor fields
```

```
  // update global memory shape / stride
  // update copy box etc.
}
__global__ void tma_desc_demo3(CUtensorMap *desc) {
  // pass desc to TMA operation
}
```

以上三种形式适用于不同的场景，第一种适用于从 host 端可以确定全局内存 Tensor 的情形，如固定长度的 linear 层对应的 gemm，第二种和第一种类似但是需要多出一个拷贝类的 API 调用；第三种为需要动态修改全局内存中 Tensor 的维度等信息时采用的模式，如大模型中 MoE（Mixture of Experts）层中的 GroupGEMM 实现时，矩阵的 m 轴是动态变化的，这时一般采用第三种形式。不论是哪种形式，其本质都是一个大小为 128 字节的描述信息，只是这个信息是相对固定的还是在运行时会动态更新和变化。就效率方面而言，前两种方法 CUDA 驱动层能够确保描述符是在更高效的缓存系统中，所以 TMA 获取描述符时效率更高一些，第三种形式需要 TMA 单元从 HBM 经由 L2 Cache 来获取该描述符，会有一定的 overhead，此时可以通过软件编程的角度在恰当的位置对描述符进行预取，使用的 PTX 指令和对应的 SASS 如下

```
// PTX
prefetch{.tensormap_space}.tensormap [a];

.tensormap_space =          { .const, .param };

// SASS
UTMACCTL.PF [UR4] ;  // Uniform TMA Cache Control PreFetch
```

上面的 SASS 指令也揭示了 TMA 单元为了提升操作效率会有 TMA Descriptor 的 Cache 机构。TMA 在进行拷贝相关的任务时会优先从本地的 cache 机构中查找描述符信息，如果命中，则直接发起相应的拷贝任务，如果不能命中则会从更低级的存储机构中读取。

12.2.2 发起拷贝任务

TMA 描述符精准的描述了全局内存的情况以及具体拷贝时的基本块，在具体的需要拷贝时可以通过 PTX 提供的指令实现，以全局内存到共享内存为例，具体指令为

```
cp.async.bulk.tensor.dim.dst.src{.load_mode}.completion_mechanism{.level::cache_hint}
                                  [dstMem], [tensorMap, tensorCoords], [mbar] {, cache-policy}

.dst =                  { .shared::cta }
.src =                  { .global }
.dim =                  { .1d, .2d, .3d, .4d, .5d }
.completion_mechanism = { .mbarrier::complete_tx::bytes }
.load_mode =            { .tile, .im2col }
.level::cache_hint =    { .L2::cache_hint }
```

其中 cp 来自 copy 的缩写，表示其为数据搬运指令，async 表示该指令为异步指令，指令执行后并不代表 copy 完成，而只是代表 copy 任务被发起，bulk 表示为大块内存拷贝区别于 Ampere 架构中的`cp.async`指令（Ampere 中的 cp.async 指令依然是 SIMT 范式的指令），tensor 用以指示是面向 tensor 的 copy，即由 Tensor 到 Tensor 到块拷贝，dim 用以指示 tensor 的维度信息（亦即 rank 信息）只支持一维到五维，dst 和 src 用以指定 copy 的目的和源内存类型，分别是 thread block 中共享内存和全局内存，load_mode 用以指示 TMA 的加载模式，除了本文重点介绍的 tile 模式，还有针对卷积运算（convolution）的 im2col 模式，tile 模式即块到块的模式，输入输出等大小，completion_mechanism 表示完成机制，此处为 mbarrier::complete_tx::bytes，表示完成机制为 mbarrier，当拷贝完成时 TMA 负责将拷贝的字节数写入到 mbarrier 中的 transaction bytes 字段中（参考 cute 之 Hopper MBarrier），同时通知其完成，level::cache_hint 表示使用的 L2 的 cache 的驱逐策略，如 evict_norm、evict_first、evict_last，根据数据复用的时间、空间局部性，可以选择恰当的 L2 驱逐策略，以提升 L2 的利用效率。

以上介绍了 TMA 发起拷贝的 modifier 属性，具体操作数方面，有 dstMem 用于指定输出的地址，示例这里指共享内存的地址，tensorMap 和 tensorCoords 用以指定 TMA Descriptor 的地址和拷贝块的坐标，这里会根据 modifier 中 dim 选择的不同而输出的参数数目不同，mbar 用以指定完成时的通知机制 mbarrier 的地址，cache-policy 为可选项，其

用以指定 L2 的驱逐策略。

特别的针对一维的场景，可以不指定复杂的 TMA Descriptor，而是指定全局内存的地址和 size 来完成大块数据的搬运，针对全局内存到共享内存拷贝，PTX 提供的异步拷贝指令如下

```
// global -> shared::cta
cp.async.bulk.dst.src.completion_mechanism{.level::cache_hint}
                      [dstMem], [srcMem], size, [mbar] {, cache-policy}

.dst =                  { .shared::cta }
.src =                  { .global }
.completion_mechanism = { .mbarrier::complete_tx::bytes }
.level::cache_hint =    { .L2::cache_hint }
```

在 modifier 方面整体和 tensor 版本类似，只是不再需要 tensor 和 dim modifier，参数方面也只需要指定 global memory 的地址和 size 即可，可以避免相对复杂的 TMA Descriptor 构建。虽然一维的拷贝不需要 TMA Descriptor 但是其整体工作范式依然是 TMA 模式而非传统的 SIMT 模式，其只需要一条指令即可以完成大块数据的拷贝，而无需所有线程参与拷贝任务。

12.2.3 完成机制

从以上的指令介绍可以看到，针对全局内存到共享内存采用了 mbarrier 作为完成机制，而针对共享内存到全局内存方向的拷贝则采用 bulk group 机制，即 commit/wait 机制，其类似 Ampere 中的 cp.async，能够将之前发起的异步拷贝整体提交，提交之后通过 wait 指令可以实现对这些提交的完成确认，不同源和目的内存的完成机制如下图（引用自 NVidia PTX 文档）。

.completion-mechanism	.dst	.src	Completion mechanism
.mbarrier::...	.shared::cta	.global	mbarrier based
	.shared::cluster	.global	
	.shared::cluster	.shared::cta	
.bulk_group	.global	.shared::cta	Bulk async-group based

知乎 @reed

图 12.5: Figure 5. TMA completion mechanism(From NVidia PTX ISA-9.1, chapter-9.7.9.25.4.1)

12.2.4 Swizzle 机制

TMA Descriptor 创建时为了避免共享内存的读取和或写入的冲突可以指定 Swizzle 模式，常用的有以下四种 `CU_TENSOR_MAP_SWIZZLE_NONE`，`CU_TENSOR_MAP_SWIZZLE_32B`，`CU_TENSOR_MAP_SWIZZLE_64B`，`CU_TENSOR_MAP_SWIZZLE_128B`

图 6 展示了 shared memory 的 bank 情况，此处最小的单元表示为 16 字节，可以认为 shared memory 有 8 个 bank，不同的 bank 对应不同的颜色表示。对于不做 Swizzle 的情况，即 SWIZZLE_NONE, 横向 128 字节为不同的 bank，纵向不做 swizzle 故而每行的 bank 颜色和第一行一致。针对 SWIZZLE_128B 场景，即 SWIZZLE 的边界为 128B，第二行的 128B 需要按照 16B 的列号和行号进行异或得到 swizzle 后的 bank 号，即`ibank = irow ^ icol`。SWIZZLE_64B 的模式时，可以认为是 SWIZZLE_128B 的第一行分为前后两个部分，第二部分，即后 64B 折返到第二行形成两行，第三行和第四行则可以认为是 SWIZZLE_128B 的第二行的前后一半排列成两行。SWIZZLE_32B 以此类推。整体计算逻辑如下

```
int banks[sizeh][sizew];
for (int irow = 0; irow < sizeh; ++irow) {
  for (int icol = 0; icol < sizew; ++icol) {
    int ioffset = irow * sizew + icol;
    int irow1 = ioffset / 8;
```

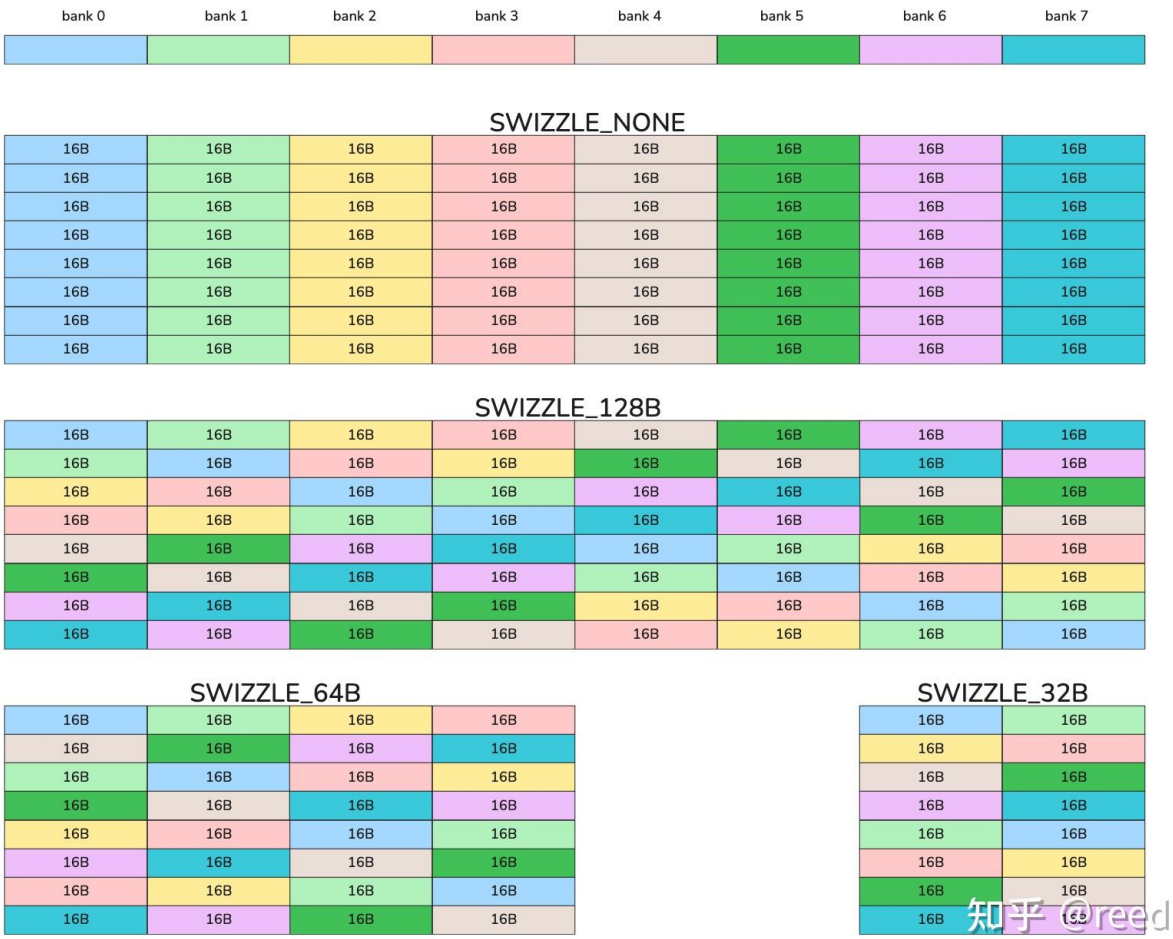


图 12.6: Figure 6. Swizzle Pattern for TMA

```
    int icol1 = ioffset % 8;
    banks[irow][icol] = irow1 ^ icol1;
  }
}
```

12.2.5 Async 单元和可见性保障

通过上面的介绍我们了解到 TMA 可以异步的读写共享内存,传统的 LSU 也可以读写共享内存(即通过对__shared__内存进行读取和写入)，但是 TMA 和 LSU 不共享操作流水线，这就造成 TMA 和 LSU 对 shared memory 操作的可见性方面需要额外的机制才能保障。

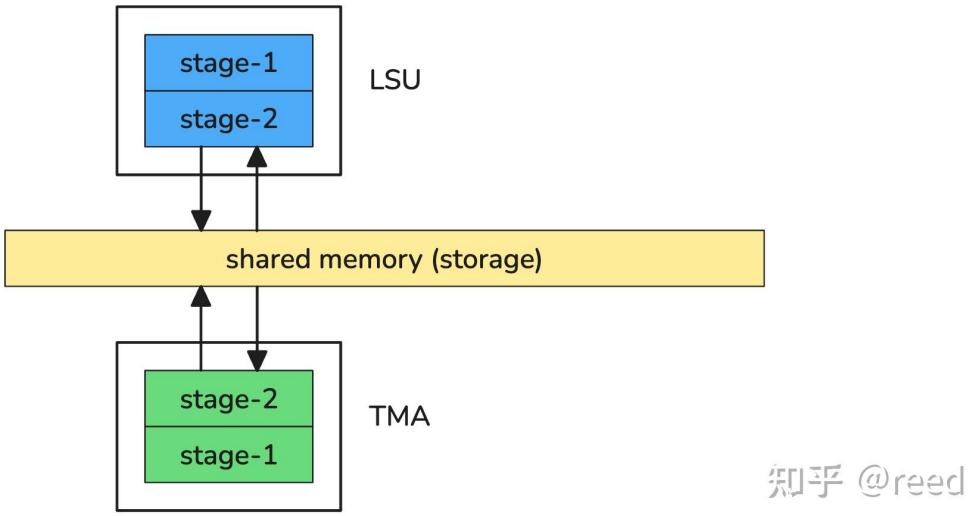


图 12.7: Figure 7. LSU and TMA has its own pipeline

TMA 读取 global 的数据写入到 shared memory 中，此时通过 mbarrier 的 wait 机制可以确保线程块看到 TMA 读取的结果，通过 LSU 读取共享内存可以得到正确的结果。如果是通过 LSU 向共享内存写入数据，但是通过 TMA 读取，如果不进行额外操作则 TMA 可能读到错误的结果。如下示例，通过 LSU 先向共享内存写入数据然后调用线程块同步函数__syncthreads，同步之后调用单线程的 TMA 读取共享内存，在时序上看似先向共享内存写入的数据，然后发起了 TMA 读取共享内存，但是在 TMA 发起后真正读取数据的时候，LSU 对共享内存的写入可以依然在流水线上，并没有真正完成，这也不能保证对 TMA 对可见，所以 TMA 有可能读到错误的数据，如果在 syncthreads 后是一个普通的共享内存的读，则其由于其走相同的流水线，能够确保读取到正确的数据。

```
__shared__ float sdata[1024];

sdata[idx] = idx;
__syncthreads();

if(elected) {
  TMA::copy(sdata, ...);
}
```

针对以上场景，则需要显式的通过 fence 指令来确保内存事务的一致性，更新后的代码为

```
sdata[idx] = idx;
asm volatile("fence.proxy.async.shared::cta;\n");
__syncthreads();

if(elected) {
  TMA::copy(sdata, ...);
}
```

其中 fence 指令行能够确保 fence 之前的内存事务被追踪，确保 TMA 能够看到该内存屏障事务，__syncthreads能

够确保线程在执行层的同步，而不是可见性的同步，这样 TMA 发起的拷贝任务会等待 fence 事务完成后才能进行，能够得到正确结果，一定程度上可以认为有了 fence 语句之后，TMA 可以看到 LSU 的写入完成并且会等待写入完成后才能开始其对共享内存的读取，所以能够得到正确的结果。

12.2.6 其他方面

除了上面介绍的 TMA 的基本的概念和用法，TMA 还能提供 Cluster 内的 Multicast 能力，即读取一份数据可以组播给 cluster 内的多个 block；除了上面提到的 tile 的拷贝能力，TMA 还提供了针对 convolution 计算模式的 image2column 能力，完成复杂的坐标变换和数据排列。同时在 TMA 读取共享内存写出全局内存的时候，还可以在写出时合并 Reduce 能力，这部分能力可以在具体的使用场景中参考相关文档来使能。

之前提到了 TMA 提供了对 TMA Descriptor 对预取能力，TMA 还提供了对全局内存中要读取的数据进行预取的能力，使得程序员能够在计算过程中对需要用到的数据进行预取，以提升访存效率。

由于 TMA 是单线程发起范式，在一个 block 和 warp 内有一个线程即可以完成大块数据的拷贝，所以在指令设计方面它属于 Uniform 指令的范畴，自然地它的参数也为 Uniform 寄存器，能够减少对 general 寄存器的使用。同时为了辅助这种单线程发起拷贝模式，NVidia 提供了`elect.sync` PTX 指令来更好的实现 leader 线程的选取。

由于是大块的内存访问，出于对效率的考虑，TMA 访问的内存会有对齐的要求，具体的对齐要求如下（参看 NV 的 CUDA 编程手册）

Table 23: Alignment requirements for multi-dimensional bulk tensor asynchronous copy operations.

Address / Size	Alignment
Global memory address	Must be 16 byte aligned.
Global memory sizes	Must be greater than or equal to one. Does not have to be a multiple of 16 bytes.
Global memory strides	Must be multiples of 16 bytes.
Shared memory address	Must be 128 byte aligned.
Shared memory barrier address	Must be 8 byte aligned (this is guaranteed by `cuda::barrier`).
Size of transfer	Must be a multiple of 16 bytes.

知乎 @reed

图 12.8: Figure 8. Alignment Requirement for TMA（from NV CUDA Programming Guide）

12.3 cute 封装

针对 TMA 能力，不管是 TMA Descriptor 的创建、device 端对拷贝的发起还是 TMA Descriptor 的预取、预访问 Tensor 的预取，以及 fence 能力等，cute 都做了很好的封装，我们后续在完成 Hopper 上高效 GEMM 实现时也将使用这些封装。核心封装在 cute 的/arch/copy_sm90_desc.hpp 文件和 cute/arch/copy_sm90_tma.hpp 文件中，简单摘录如下，在 cute/arch/copy_sm90_desc.hpp 文件中有对 TMA Descriptor 的预取的封装

```
void prefetch_tma_descriptor(TmaDescriptor const* desc_ptr);

enum class CacheHintSm90 : uint64_t {
  EVICT_NORMAL = 0x1000000000000000,
  EVICT_FIRST = 0x12F0000000000000,
  EVICT_LAST = 0x14F0000000000000,
};
```

在 cute/arch/copy_sm90_tma.hpp 有对发起 copy 能力的封装，包含 LOAD、STORE、REDUCE、1D 能力、IM2COL 和 MULTICAST 能力：

```
struct SM90_TMA_LOAD{_MULTICAST}[_1D/2D/3D/4D/5D]/PREFETCH
struct SM90_TMA_LOAD_IM2COL{_MULTICAST}[_3D/4D/5D]
struct SM90_TMA_STORE[_1D/2D/3D/4D/5D]
struct SM90_TMA_REDUCE_ADD[_1D/2D/3D/4D/5D]
struct SM90_BULK_COPY_G2S/S2G
```

12.4 总结

SIMT 的编程范式中需要每一个线程参与数据的读写，其需要处理复杂的边界、针对共享内存需要处理由于共享内存 bank 结构带来而引入的 swizzle 优化，这会消耗大量的整数计算指令，同时开发效率也会因此而降低，针对这些问题，将上面的这些计算和处理需求封装成独立的硬件单元 TMA，其能够高效的完成数据在共享内存和全局内存之间的拷贝，同时可以通过配置方式处理 swizzle 模式，oob 模式，减少了地址的相关计算，提高了编程的效率。这是 NVidia 对 SIMT 编程范式的一次突破。如果要使用 TMA 能力，需要构造 TMA Descriptor 和发起拷贝两个部分，以全局内存向共享内存拷贝为例详细介绍了相关的指令，并且介绍和解读了开发中可能会经常遇到的可见性问题。cute 对该能力有相对全面且完整的封装，利用好 cute 可以很好的利用其 TMA 的能力，完成数据的高效拷贝。后续我们将介绍 Hopper 上另一个重要功能单元 WGMMA，最后我们将利用 TMA 完成高效的数据加载，利用 WGMMA 完成高效的矩阵运算、利用 MBarrier 完成高效的协同，借助这三者的力量完成 Hopper 架构上高效的矩阵计算。

12.5 参考

cute 之 Hopper MBarrier[1]

cute 之 GEMM 流水线[2]

cute 之 Swizzle[3]

cute 之 Layout[4]

https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/async-copies.html#using-the-tensor-memory-accelerator-tma[5]

https://docs.nvidia.com/cuda/parallel-thread-execution/#data-movement-and-conversion-instructions-bulk-copy[6]

https://patents.google.com/patent/US20230289292A1/en[7]

https://resources.nvidia.com/en-us-hopper-architecture/nvidia-h100-tensor-c[8]

[1] https://zhuanlan.zhihu.com/p/1962636004235153810

[2] https://zhuanlan.zhihu.com/p/665082713

[3] https://zhuanlan.zhihu.com/p/671419093

[4] https://zhuanlan.zhihu.com/p/661182311

[5] https://link.zhihu.com/?target=https%3A//docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/async-copies.html%23using-the-tensor-memory-accelerator-tma

[6] https://link.zhihu.com/?target=https%3A//docs.nvidia.com/cuda/parallel-thread-execution/%23data-movement-and-conversion-instructions-bulk-copy

[7] https://link.zhihu.com/?target=https%3A//patents.google.com/patent/US20230289292A1/en

[8] https://link.zhihu.com/?target=https%3A//resources.nvidia.com/en-us-hopper-architecture/nvidia-h100-tensor-c

第十三章　cute 之 TMA Descriptor 编码与隐藏的第 21bit

作者：reed（知乎 @reed-84-49，专栏「CUDA 高性能编程」）　原文：
https://zhuanlan.zhihu.com/p/2037200219700449995

在之前的文章”cute 之 Hopper TMA” 中，我们介绍了 TMA（Tensor Memory Access）的基本概念和使用方式。TMA 用一个 128 字节的 opaque descriptor 描述 tensor 布局，硬件自动完成地址计算、越界 fill、swizzle 等工作。其中” 越界自动 fill” 是 TMA 对程序员最友好的承诺——你不需要手写边界 clamp，只要 tile 坐标超出 globalDim，硬件就会把越界元素填成 zero 或 NaN。

我们在研究 cuTensorMapEncodeTiled 的编码逻辑、并在不同驱动版本间做对比验证时，在 control word 的 bit[21] 位置发现了一个行为差异。在我们测试的 580 版本驱动中，该 bit 会根据 tensor 数据量是否达到 128KB 而被设置；而在我们测试的 530 版本驱动中，该 bit 始终为 0。进一步的实验表明，当 bit21=1 时，TMA 硬件会跳过 OOB 边界检查，直接对越界 tile 计算地址并发起内存请求——如果目标地址恰好落在未映射的虚拟地址区域，就会触发 illegal memory access。

本文分三个部分展开：首先完整呈现 TMA descriptor 的 128 字节二进制布局（这是理解 bit21 位置的前提）；然后通过精确控制 GPU 虚拟地址映射的实验，验证 bit21 对 OOB 行为的影响；最后讨论这个未文档化 bit 在 tensormap.replace 动态修改场景下引发的实际问题及规避方法。

13.1　TMA Descriptor 的完整二进制布局

cuTensorMapEncodeTiled 构造的 CUtensorMap 是一个 128 字节、128 字节对齐的 opaque 结构体。NVIDIA 官方将其定义为 opaque type，没有公开内部编码方式。通过对大量参数组合下的输出做 byte-level 对比分析，结合 TMA 指令对 descriptor 各字段的访问行为，我们还原了完整的二进制布局。

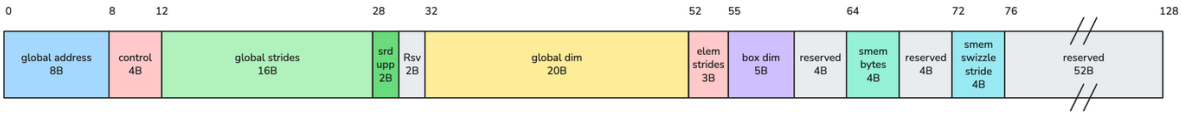


图 13.1: Figure 1. TMA Descriptor Offset and Fields

13.2　整体结构

```
Offset   Size    Field
------   ----    ----------------------------------------
 0- 7    8B      globalAddress (64-bit device pointer)
 8-11    4B      Control Word (bitfield, 见下文详解)
12-27   16B      globalStrides[0..3] / 16 (各 32-bit, little-endian)
28-29    2B      stride upper bits (每个 stride 的高 4 位, packed as nibbles)
30-31    2B      reserved (=0)
32-51   20B      globalDim[0..4] - 1 (各 32-bit)
52-54    3B      elementStrides (每维 3 bits, packed)
55-59    5B      boxDim[0..4] - 1 (各 8-bit)
60-63    4B      reserved (=0)
64-67    4B      smem box total bytes
68-71    4B      reserved (=0)
72-75    4B      smem swizzle stride
```

```
76-127  52B      reserved (=0)
```

几个关键字段的说明：

globalAddress (bytes 0-7): tensor 在 device global memory 中的起始地址。TMA 硬件以此为基地址，加上 coord × stride 计算每个 tile 的实际访问地址。这是 descriptor 中唯一的指针字段。

globalStrides (bytes 12-29): 除最内层维度外的各维 stride，以字节为单位除以 16 后存储。采用 36-bit 编码（32-bit lower + 4-bit upper nibble），支持最大 2^{40} B（约 1TB）的 stride 值。最内层维度的 stride 隐含为 elemSize，不编码在 descriptor 中。

globalDim (bytes 32-51): 各维度大小减 1 后存储为 32-bit 值。TMA 硬件用此做越界判断——当 tile 的起始坐标超出此值时认为越界。这也是 `tensormap.replace` 最常修改的字段。

smem box total bytes (bytes 64-67): 一次 TMA tile load 搬运到 shared memory 的总字节数。计算方式为 `product(boxDim[i] / elemStride[i])` × `baseSize`，其中 `baseSize` 在 interleave 模式下为 interleave 粒度（16B 或 32B），否则为 elemSize。

13.3　Control Word 的 bitfield 布局

Control word（bytes 8-11）浓缩了 descriptor 的模式配置信息。通过逐 bit 变化各 API 参数并观察输出变化，我们确认了以下布局：

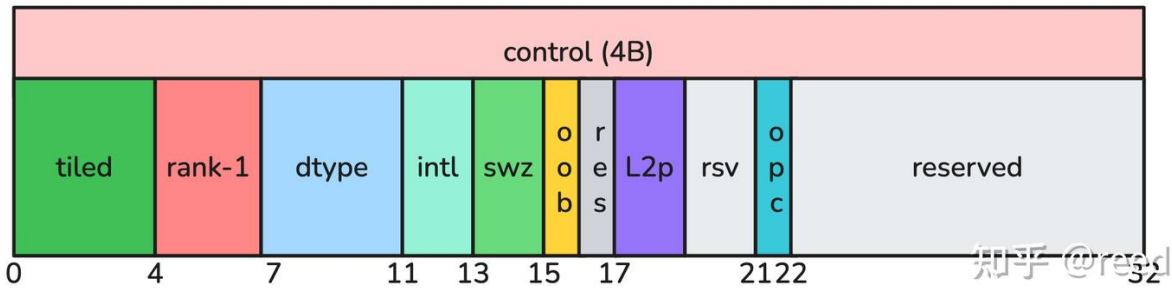


图 13.2: Figure 2. Control Word bitfield

```
bit 位置        含义
----------      ----------------------------
bits[ 3: 0]    tensor type (0 = tiled)
bits[ 6: 4]    tensorRank - 1
bits[10: 7]    internal dtype code (4 bits, 与 API enum 值有重映射)
bits[12:11]    interleave (0=none, 1=16B, 2=32B)
bits[14:13]    swizzle (0=none, 1=32B, 2=64B, 3=128B)
bit [15]       oobFill (0=zero, 1=NaN)
bit [16]       reserved (=0)
bits[18:17]    l2Promotion (0=none, 1=64B, 2=128B, 3=256B)
bits[20:19]    reserved (=0)
bit [21]       OOB protection control <- 本文主角
bits[31:22]    reserved (=0)
```

需要强调的是，以上布局完全基于实验还原，不来自任何 NVIDIA 公开文档。NVIDIA 未公开 CUtensorMap 的内部结构。

13.4　开源参考实现

为了方便开发者理解和调试 TMA descriptor，我们提供了 `cuTensorMapEncodeTiled_impl.h`一个纯软件的参考实现，具备两项核心能力：

正向编码：给定相同的 API 参数，生成与 driver 580 输出 byte-level 一致的 128 字节 descriptor（含 bit21 逻辑）。我们在多组测试向量上验证了与官方驱动的完全一致性。

解析打印：给定任意 CUtensorMap，将其内部字段完整解码并格式化打印——包括 globalAddress、各维度 dim/stride/box、control word 中的每个 bitfield。

这在调试 `tensormap.replace`后的 descriptor 状态时尤其有用：你可以直接将修改前后的 descriptor dump 出来，逐字段比对，快速定位不一致。

13.5 bit21 的设置规则

在我们测试的两个驱动版本中，bit21 的行为如下：

Driver 530: bit21 始终为 0，无论输入参数如何变化。

Driver 580: bit21 根据以下条件设置：

```
bit21 = (product(globalDim[0..rank-1]) * elemSize >= 131072)  // >= 128KB
```

dtype	dims	total bytes	bit21 (driver 580)
UINT8(fp8)	[256, 511]	130816	0
UINT8(fp8)	[256, 512]	131072	1
FLOAT16	[256, 256]	131072	1
FLOAT32	[128, 128]	65536	0
FLOAT32	[128, 256]	131072	1
FLOAT32	[32, 32, 32]	131072	1
FLOAT32	[32, 32, 31]	126976	0

阈值精确卡在 $2^{17} = 131072$ 字节。我们没有确认这个行为具体从哪个驱动版本开始引入——仅确认在 580 中存在、在 530 中不存在。

13.6 bit21 的硬件语义：实验验证

知道了 bit21 的设置规则后，关键问题是：它控制什么硬件行为?

13.7 实验设计

为了隔离 bit21 的影响，我们需要构造一个实验，使得除 bit21 外所有条件完全相同，然后观察 TMA 对 OOB tile 的处理差异。

实验构造的核心挑战在于内存映射。cudaMalloc 的 suballocator 内部会映射远大于请求大小的虚拟地址区域（我们实测 cudaMalloc(2MB) 的有效映射达到 4MB 以上）。这意味着常规分配下 OOB 地址大概率仍在有效映射内——TMA 读到” 垃圾数据” 而不是 fault，无法区分” 跳过检查直接读了” 还是” 检查后 fill 了”。

因此我们使用 cuMemCreate + cuMemMap 精确控制映射边界：在连续的虚拟地址空间中只映射中间 1 个 page（2MB），前后都是 unmapped 区域。将 tensor 数据（100 行 × 256 列 = 25KB）放在 mapped 区域的末尾，使得 OOB tile（coord_y=128）的计算地址一定落入末尾的 unmapped 区域。

实验流程分两个 kernel 完成：

Kernel 1（update）：将模板 descriptor 拷贝到 shared memory → `tensormap.replace` 修改 address 和 dim[1]=100 → `tensormap.cp_fenceproxy` 写回 global memory

Kernel 2（load）：`fence.proxy.tensormap` acquire → TMA load at coord_y=128（OOB: 128 > dim[1]=100）

两组实验的唯一差异是创建模板时的 dims——[256, 512] vs [256, 511]——前者导致 bit21=1，后者 bit21=0。

13.8 实验结果

两个场景中 `tensormap.replace` 后的实际状态完全相同——globalDim[1]=100，base address 相同，OOB 坐标相同，目标物理地址相同（均落入 unmapped 区域）。唯一的差异是 control word 中的 bit21。

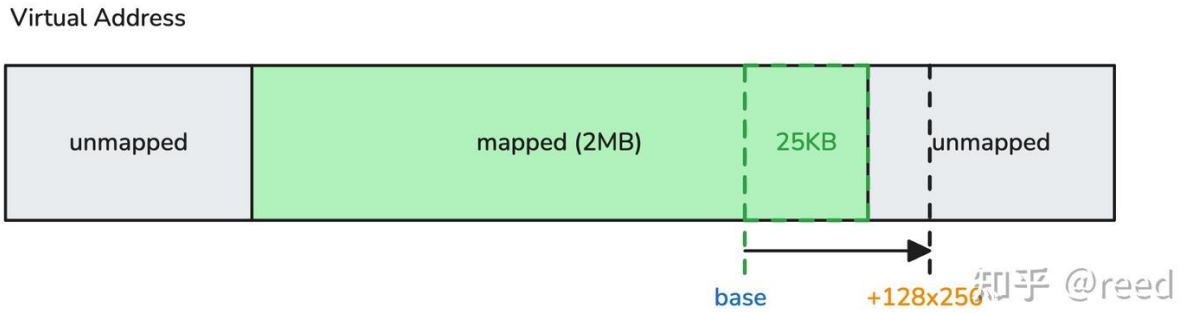


图 13.3: Figure 3. Memory Layout and OOB Address

模板 dims	bit21	OOB TMA Load 结果
[256, 512] = 128KB	1	illegal memory access
[256, 511] < 128KB	0	zero fill, 无 fault

13.9　硬件行为模型

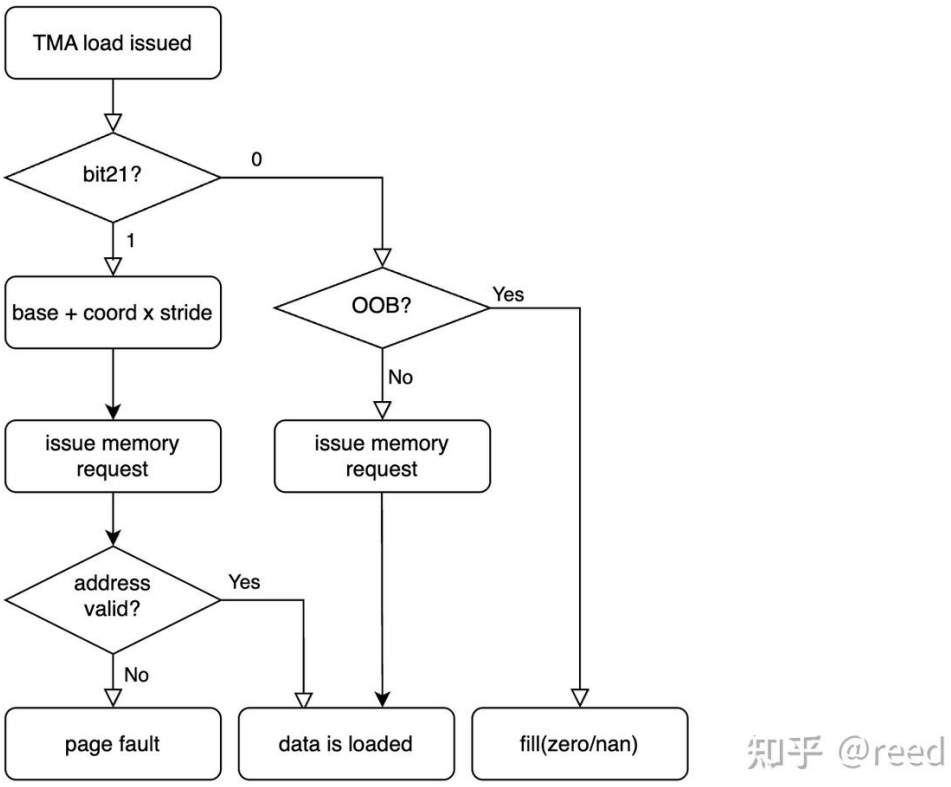


图 13.4: Figure 4. TMA OOB Path for 21st Bit in Control Word

基于实验结果，我们得出 bit21 的硬件行为模型：

bit21=0: TMA 前端在发起任何内存事务之前，先比较 tile 坐标与 globalDim。如果 coord ≥ globalDim，该 tile 被判定为 OOB，硬件在本地生成 fill value（zero 或 NaN，取决于 oobFill 配置），不产生任何内存请求。即使目标地址完全不可访问，也绝对安全。

bit21=1: TMA 跳过上述前端检查，直接计算 globalAddress + coord × stride 并向内存子系统发起请求。如果该地址有效（落在已映射的 VA 范围内），TMA 会读到该位置的数据——可能是正确数据，也可能是不相关的垃圾。如果该地址无效（未映射），则触发 page fault，表现为 illegal memory access。

bit21 本质上是一个性能优化开关：对于大 tensor，跳过逐 tile 的边界检查可以减少 TMA 前端延迟。猜测 NVIDIA 的设计假设是——大 tensor 的实际内存分配通常有足够的对齐和 padding，正常使用中 OOB tile 的地址不会落在 page

边界之外。

13.10 当性能优化遇到动态 Shape

理解了 bit21 的语义后，它引发实际问题的条件也就清楚了。

NVIDIA 提供了 `tensormap.replace` 指令，允许 kernel 在 device 上动态修改 descriptor 的 globalDim、globalAddress 等字段——这是处理 variable-length sequence（如 LLM 推理中的 group gemm、variable-length attention）的标准做法。问题在于：tensormap.replace 修改 dim 时不会联动更新 control word 中的 bit21。

这意味着以下使用序列会产生一个自相矛盾的 descriptor：

1. 用较大的 max shape 创建模板 descriptor → 因为 total ≥ 128KB，bit21 被设为 1
2. 运行时通过 `tensormap.replace`将 dim 缩小为实际值 → bit21 仍为 1（replace 不更新 control word）
3. TMA load 遇到 OOB tile → bit21=1 → 跳过检查 → 计算地址

在大多数场景下这不会 fault——因为 cudaMalloc 映射的 VA 范围远大于实际分配，OOB 地址通常仍然”可访问”（只是读到垃圾）。但在以下条件同时满足时会触发 illegal memory access：模板 tensor 总大小恰好 ≥ 128KB（bit21=1）；运行时某些 batch/group 的实际 dim 远小于模板；对应的 OOB 地址刚好越过了实际的 page 映射边界（memory pool allocator 的分配边界、cuMemCreate 精确映射等场景）。

这不是误用 API。用户通过 NVIDIA 提供的标准 API 路径正确编程，得到的 descriptor 却进入了一个语义不一致的状态：globalDim 字段说”tensor 只有 100 行”，bit21 却告诉硬件”这个 tensor 很大，不需要 OOB 保护”。CuTe 中的 `tma_descriptor_cp_fence_release` 和 `tma_descriptor_fence_acquire`也不涉及 bit21 的同步更新。

bit21 的引入使得 OOB 检查变为有条件的，但 tensormap.replace 的实现未能跟进这一变化——它更新了参与 OOB 判定的 globalDim 字段，却没有联动更新控制 OOB 检查是否执行的 bit21。这是一个优化引入后接口语义未完整适配的问题。

13.11 规避方法

13.12 方法一：创建模板时强制清零 bit21

在调用 cuTensorMapEncodeTiled 后、将 descriptor 交给 kernel 使用之前，在 host 侧手动清除 bit21：

```
// 创建 descriptor
cuTensorMapEncodeTiled(&tmap, ...);
// 强制清除 bit21，确保 OOB 保护始终开启
uint32_t control;
memcpy(&control, (uint8_t*)&tmap + 8, 4);
control &= ~(1u << 21);
memcpy((uint8_t*)&tmap + 8, &control, 4);
```

这种方法简单直接，代价是大 tensor 可能损失一点 TMA 前端性能（需要逐 tile 检查边界）。对于绝大多数应用场景，这个开销可以忽略。

13.13 方法二：在 `tensormap.replace` 修改 shape 时联动处理 bit21

如果需要保留 bit21 的性能优化（大 tensor 跳过检查），可以在每次 replace dim 后根据新的实际 tensor 大小重新决策 bit21。由于 tensormap.replace 不支持直接修改 control word，需要在 shared memory 中手动 patch 对应字节后再通过 `tensormap.cp_fenceproxy` 写回：

```
// 在 smem 中完成 replace 后，根据新 dims 更新 bit21
uint64_t new_total = (uint64_t)new_dim0 * new_dim1 * elemSize;
uint32_t* ctrl_ptr = (uint32_t*)((uint8_t*)&smem_tmap + 8);
if (new_total >= 131072)
```

```
    *ctrl_ptr |= (1u << 21);
else
    *ctrl_ptr &= ~(1u << 21);
// 然后执行 cp_fenceproxy 写回 gmem
```

13.14　结论

TMA descriptor control word 的 bit[21] 是一个未文档化的硬件标志。在我们测试的 driver 580 中，当 product(globalDim) × elemSize ≥ 128KB 时被设置为 1；在我们测试的 driver 530 中始终为 0。

bit21=1 时，TMA 硬件跳过 tile 坐标的越界检查（OOB fill 机制被禁用），直接计算并访问目标地址。

`tensormap.replace` 指令修改 globalDim 时不联动更新 bit21，可能导致 descriptor 进入"dim 小但 bit21=1" 的不一致状态，在 OOB 地址恰好落入未映射区域时触发 illegal memory access。

规避方式为：创建 descriptor 后强制清零 bit21，或在 replace 修改 dim 后根据新 tensor 大小手动更新 bit21。

本文涉及的全部代码更新在https://github.com/reed-lau/cute-gemm/tree/main/tma-desc包括：

`cuTensorMapEncodeTiled_impl.h`：参考实现（正向编码 + 字段解析打印，与 driver 580 输出 bit-level 一致）

`test_bit21_oob.cu`：bit21 OOB 行为的最小复现

main.cc + tests.txt：52 组测试向量的全量回归验证

13.15　参考

https://docs.nvidia.com/cuda/cuda-driver-api/group__CUDA__TENSOR__MEMORY.html
https://github.com/NVIDIA/cutlass/blob/main/include/cute/arch/copy_sm90_desc.hpp
https://patents.google.com/patent/US20230289292A1

第二部分

竹熙佳处系列（CuTe 教程）

第十四章　写给大家看的 CuTe 教程：tiled copy

作者：竹熙佳处（知乎专栏「CuTe 教程」）　原文：https://zhuanlan.zhihu.com/p/1930389542784964333

14.1　更新

本篇文章的后续，写给大家看的 CuTe 教程：tiled mma[1] 已发布，有兴趣的读者可以在本文读完后查看

14.2　动机

NV GPU 编程体系下，tensor core 的使用越来越重要。那么如何利用好 tensor core 进行编程呢？在 CuTe 出现之前，我们有几种选择：

- cuda + ptx 指令
- cutlass 模版库
- Triton

其中“cuda + ptx 指令”的组合足够直白，掌控粒度也比较细，通常能够做到性能最优，然而其带来的代价就是开发者需要对 ptx 指令较为熟悉，并且存在跨架构时的兼容问题。

Cutlass 模板库，能够提供高效 Gemm 融合后处理（i.e. epilogue）的能力，并且能够隐藏大部分实现细节（e.g. 指令选择，流水排布，etc），参考官方 example 就可以比较快速写出一个性能不错的 gemm kernel，然而，其缺点在于除了其提供的模板之外不再有能力写 kernel，例如，flash atten v2 就无法用 Cutlass 完成。

Triton，编程最简单，也能有比较好的功能扩展能力，然而其性能本质上是依赖其后端编译过程的，因此很难做到对性能的极致掌控。

而 CuTe 的出现，其实是希望改进“cuda + ptx 指令”这条路线，做到保障 kernel 性能的同时，隐藏掉 ptx 封装的过程以及坐标运算过程中的诸多细节，并且做到跨架构时的兼容（仅需少量代码修改，如 copy/mma 指令的选择）。同时，cutlass3.0 也是用 CuTe 重构过的，因此我们可以认为，熟练掌握 CuTe，可以写出与 cutlass 同样质量的 kernel 代码，而不失灵活性。

更进一步的，CuTe 的表达能力以及其深层的数学体系很完备，能够同时兼顾传统 cuda core 编程体系以及 tensor core 编程体系，甚至有在类 gpu 的 ai 芯片上兼容 / 扩展的潜力。

然而，代价是什么？

其伴随而来的学习曲线实在过于陡峭，笔者在很长一段时间都对于很多基础概念都是一知半解，写相关的代码也只是在 @reed[2] 大师以及 tri Dao 的代码上做一些修改，经过相当一段时间的摸索后总算有些粗浅的认知。在这个过程中，我逐渐意识到其这里面的难点很大一部分在于没有一个足够 junior 的教程能够让开发者能够对 CuTe 的基本能力有一个直白的认知：cutlass CuTe 官方的 tutorial ，只能说很难评；reed 大师的系列文章[3]写的很好，高屋建瓴，干货满满。然而我认为其对初步接触的 CuTe 的同学来说仍然有些复杂。

我认为，有很多概念其实可以解释的更直观，也可以结合我们已有的 cuda 经验来更好的理解。还有就是很多变量的命名，一些辅助函数，本身也包含了很多的信息，也值得和大家分享下。接下来这篇文章，我们以 tiled copy 为例，共同探索一下 CuTe 的用法。

[1] https://zhuanlan.zhihu.com/p/1937145378446226159
[2] https://www.zhihu.com/people/2600b0313c76a175aa9e09a235382b79
[3] https://www.zhihu.com/people/reed-84-49

14.3 Tiled copy 是为了解决什么问题？

在现在的 gemm 体系下，通常会需要在一块大的数据中（e.g. 行数为 M，列数为 N），切分出一个小块的数据，此即 Tile 这个概念的来源。我们可定义这个 Tile 的行数为 kTileM，列数为 kTileN。

对于一个 threadblock 而言，通常就是需要将这个 tile data 整体从某个 src-memory 中 copy 到某个 dst-memory 中。如，global to shared（以下表示为 g2s），shared to register (以下表示为 s2r)，shared to global（以下表示为 s2g) 等。

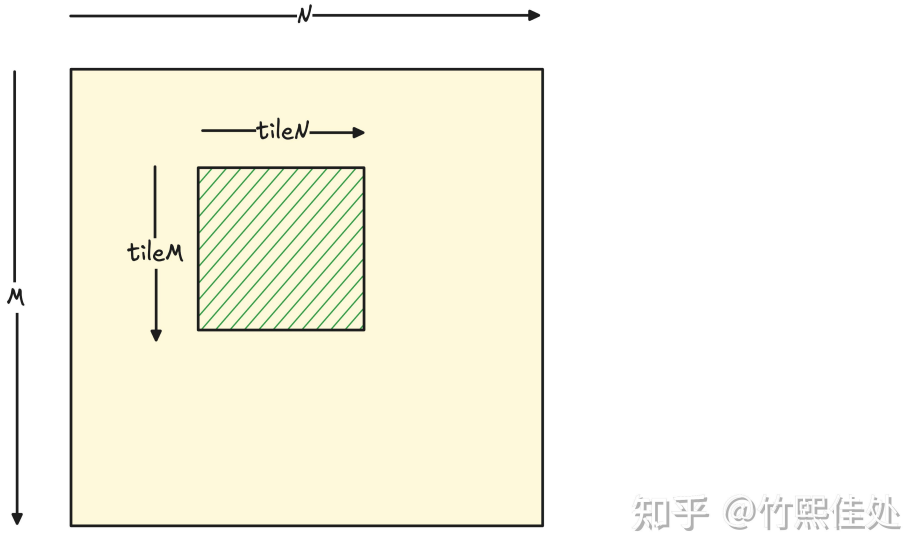


图 14.1: Figure1. 对数据分 tile，tileM 和 tileN 分别为我们需要 copy 的 tile 的行和列

而 Tiled copy 本质上是说，我们有一个 threadblock，有一个（kTileM, kTileN）的数据块，怎么去完成这个 copy。为画图方便，接下来的章节中我们以 4 threads 为一个 threadblocks （正常来说会选择 128 threads），完成一个（4，4）大小的 tile data 的 copy 为例。

14.4 CuTe Tiled copy 的设计哲学

14.4.1 Cuda 如何实现 tiled copy

以 g2s 为例，首先我们可以思考，当我们用纯粹 cuda 的方式，来完成一块 2d tile data 的 g2s copy 时，我们的思路是什么?

答案是我们通常会考虑，每个 thread 按照自己的 thread id，计算好自己的 src (i.e. global memory) 地址，以及 dst (i.e. shared memory) 地址，然后赋值过去。当然，我们考虑合并访存原则，也就是尽可能保证 “连续的 thread 读 / 写连续的地址”，我们通常会要求连续的 thread 尽可能排成一行，然后每个 thread 能够 copy 1 个 data-element，然后当每个 thread copy 完 1 个数据后，如果这个 Tile data 还没有被 copy 完成，我们进行第 2 轮 copy，然后是第 3 轮，第 4 轮。

进一步的，然后我们考虑优化访存效率：大字长访存，也就是说，我们通常会考虑每个 thread 每轮次会 copy 多个 data-elements，我们假设为 2。

按照这个思路，我们可以写出代码如下：

```
template <int kTileM, int kTileN>
__global__ void g2s_tiled_copy(float* global_data) {
    // 为画图方便，以 4x4 的 data tile 为例

    __shared__ float shared_data[kTileM * kTileN];

    int n_threads_in_block = 4;
    // 为画图方便，以 4 threads 为 1 block 为例

```

```
    int tid = threadIdx.x;

    int total_elements = kTileM * kTileN;

    int n_threads_along_TileN = kTileN / 2;
    // 在列方向上分配多少个 thread
    int n_threads_along_TileM = n_threads_in_block / n_threads_along_TileN;
    // 在行方向上分配多少个 thread

    int i_row_thread = tid / n_threads_along_TileN;
    // 在行方向上的 thread-id
    int i_col_thread = tid % n_threads_along_TileN;
    // 在列方向上的 thread-id

    int n_elements_each_loop = n_threads_along_TileN * n_threads_along_TileM * 2;
    // 每次 copy 每个 thread 需要 copy 2 个 float

    int n_copy_loops = total_elements / n_elements_each_loop;
    // 完成这次的 tiled copy 需要多少轮次

    for (int i_loop = 0; i_loop < n_copy_loops; ++i_loop) {

        float* shared_data_i_loop = shared_data + i_loop * n_elements_each_loop;
        float* global_data_i_loop = global_data + i_loop * n_elements_each_loop;
        // 按轮次整体偏移指针

        // 实际执行 g2s copy，每个 thread 每次 copy 1 个 float2
        *(float2*)(&shared_data_i_loop[i_row_thread * kTileN + i_col_thread * 2])
                = *(float2*)(&global_data_i_loop[i_row_thread * kTileN + i_col_thread * 2]);
    }
    __syncthreads();
}
```

我们把这个 thread 和第 0 和 1 轮次的 data-element 的映射关系以 Fig.2 的形式展示出来，为方便理解，我们认为我们的 data tile 里每一个位置都存了一个不同元素。为方便表示，我们认为第一轮 for-loop 访问的都是小写字母'a' / 'b' / 'c' / ...，第二轮访问的都是大写字母'A' / 'B' / 'C' / ...。

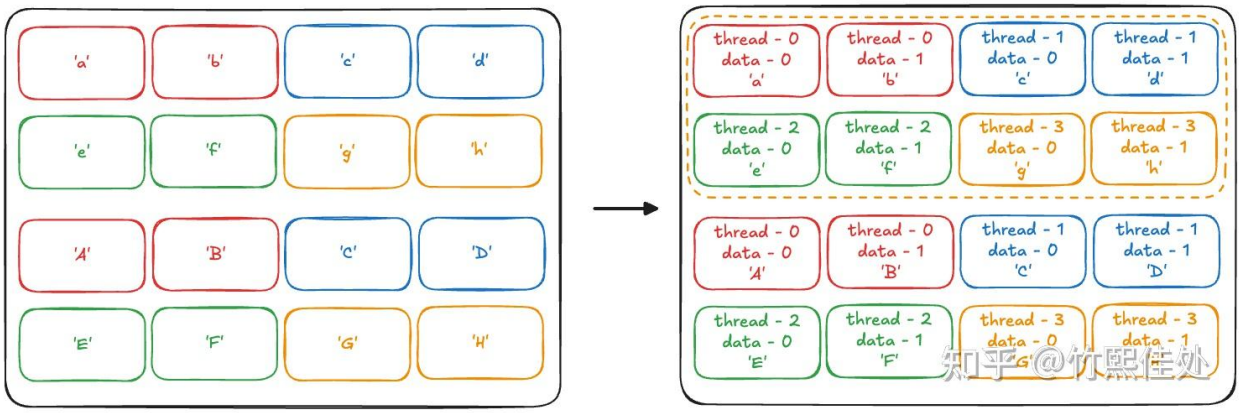


图 14.2: Figure2. 左图为 data tile 中存的元素；右图为 thread 和 data tile 的映射关系

至此，我们完成了 cuda 版本的 thread 和 data tile 的映射，有了这个表，我们实际上能够知道，给定任意一个 data tile 的 i-row 和 i-col，我们一定能知道这个是由“第几个 thread 负责 copy，会存在 thread 中的第几个 data-element”，也就是 thread-id 以及 data-id。

这对我们写代码有帮助吗？是，但不完全是。因为实际上我们写代码的时候，通常不是考虑“我的这些数据里，每一个数据由哪个 thread 放在哪个 data-element 中？”，而是“我发起了一些 thread，每个 thread 具体访问到了哪个数据”，也就是说，**我们希望给定一个 thread-id 以及 data-id，能拿到具体数据的坐标**。因此，我们需要对已有的表格做一次“inverse”，也就是说我们希望这个表格做如 Fig.3 的变化：

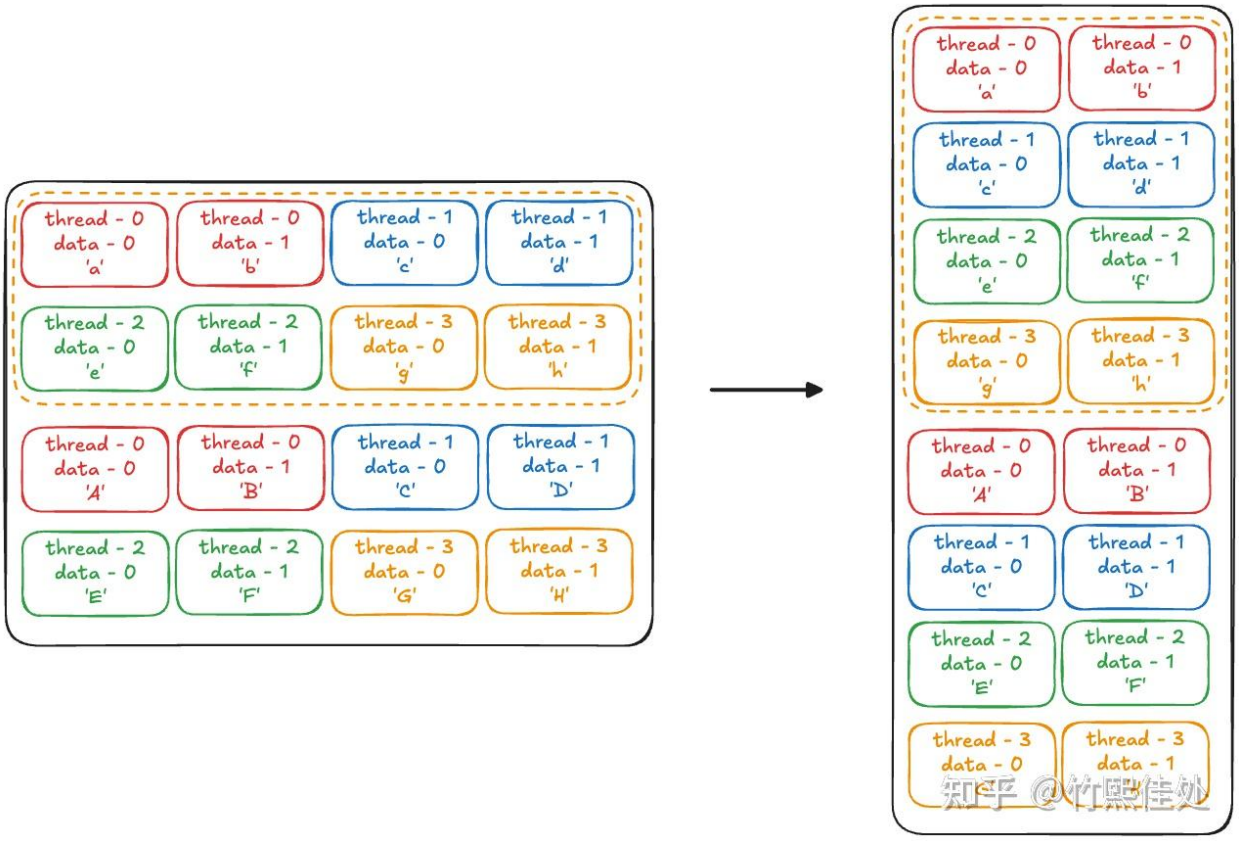


图 14.3: Figure.3 thread 和 data tile 的映射关系，变换为 thread-id 为行和 data-id 为列的形式

这样我们给定这个表格的行号（thread-id）和列号（data-id），就可以拿到原始 data 了。

这个表格在我们上述的简单例子中用处没有很大，然而，当我们考虑引入更多优化逻辑（如，避免 bank conflict 的 swizzle 逻辑，s2r 中的 ldmatrix 逻辑）的时候，复杂的地址计算会变得难以思考，此时这个表格就变得很重要了，因为不管坐标计算有多复杂，最后会统一变成这样一个表格，i.e. 一种映射。

14.4.2 CuTe 中如何实现 tiled copy

我们先展示一下 CuTe 写这样一个 tiled copy 最简单的例子

```
template <int kTileM, int kTileN>
__global__ void g2s_tiled_copy(const float *input) {
  // 为画图方便，以 4x4 的 data tile 为例，即，kTileM = 4, kTileN = 4

  int tid = threadIdx.x;

  __shared__ float shm[kTileM * kTileN];

  auto g_input_tile = make_tensor(make_gmem_ptr((float *)input),
                                  make_shape(Int<kTileM>{}, Int<kTileN>{}),
                                  make_stride(Int<kTileN>{}, Int<1>{}));
                                  // (kTileM, kTileN)

  auto s_tensor = make_tensor(make_smem_ptr((float *)shm),
                                  make_shape(Int<kTileM>{}, Int<kTileN>{}),
                                  make_stride(Int<kTileN>{}, Int<1>{}));
                                  // (kTileM, kTileN)

  using g2s_copy_op = UniversalCopy<cute::uint64_t>;
  using g2s_copy_traits = Copy_Traits<g2s_copy_op>;
  using g2s_copy_atom = Copy_Atom<g2s_copy_traits, float>;

```

```
  auto thr_layout = make_layout(make_shape(Int<2>{}, Int<2>{}),
                                make_stride(Int<2>{}, Int<1>{}));
  auto val_layout = make_layout(make_shape(Int<1>{}, Int<2>{}));

  auto tiled_copy_g2s =
      make_tiled_copy(g2s_copy_atom{}, thr_layout, val_layout);

  auto thr_copy_g2s = tiled_copy_g2s.get_slice(tid);
  auto tgA_g2s = thr_copy_g2s.partition_S(g_input_tile);
  auto tsA_g2s = thr_copy_g2s.partition_D(s_tensor);

  copy(tiled_copy_g2s, tgA_g2s, tsA_g2s);

  __syncthreads();
}
```

在这个例子中，我们首先将 input 和 shm 封装为 g_tensor 以及 s_tensor，然后调用 make_tiled_copy 函数定义了一个 tiled_copy_g2s 对象，接着调用 get_slice 函数将这个对象与 tid 进行一个绑定，接下来调用这个对象的 partition_S / partition_D 函数对 g_tensor 以及 s_tensor 进行 partition，相当于每个 thread 拿到自己要读/写的数据块信息，最后调用一次 copy 函数即可完成。

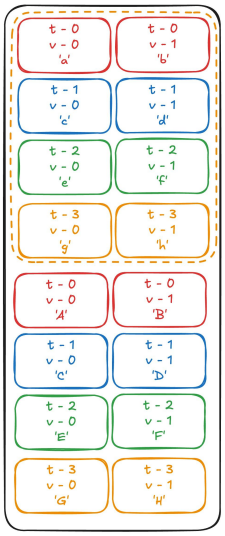
其中，make_tiled_copy 定义时需要三个参数，我们首先高度概括的总结一下，读者有个印象即可：

- Copy-atom：描述我们用什么 copy 指令 (ldg, ldg128, ldmatrix 等)，同时也包含完成一次这样指令所需要的 thread 以及数据的对应信息。
- Thread-layout：描述所有 thread 按照什么方式排布到整个 tile 中
- Value-layout：描述单个 thread 读 or 写访问的数据元素的排布方式

14.4.3　CuTe make_tiled_copy 解读

为了解释上面这些参数的含义，以及为什么 tiled copy 需要这些参数，我们接下来尝试理解一下，CuTe 中如何去构建出来我们在上一节中提到的 Fig.3。

首先，在 CuTe 的命名体系下，thread 被命名为 T，data-element 被命名为 V (value)，因此对应的 thread-id 和 data-id 就被称为 t-id 和 v-id，因此上面的图可以被修改为：



知乎 @竹熙佳处

图 14.4: Figure.4 thread 和 data tile 的映射关系（thread-id 为行和 data-id 为列）以 CuTe 的方式命名

进一步的，我们不再考虑数据本身，而是用一个数据坐标指代其中每一个元素，这个数据坐标其实是原始 m 行 n 列 data tile 的坐标（m-id, n-id），我们约定一种方式将这个二维坐标映射成 1D 坐标 mn = func0(m-id，n-id)。这个过程

是可以反向的，给定这样一个 mn，我们一定能反向找到（m-id, n-id），进而拿到我们需要的数据。那么这个 func0 是如何实现的呢？实际上这是 CuTe 通过构造出的一种特殊变换来完成的（所谓的 raked_product，我们先忽略细节，暂且认为其是由我们指定的 T-layout & V-layout 决定的一个变换），并且保证其是可逆且一一映射的。

具体的，针对我们目前的例子，这个函数会被定义为如下公式，其中 2 为 Fig.3 中的左图中橙色框圈出的部分的行数：

$$mn = \text{func0}(m\text{-id}, n\text{-id}) = 2 \times n\text{-}id + m\text{-}id$$

同理，（t-id, v-id）也可以被映射为 1D 坐标，4 为一次 atom copy 中参与运算的 thread 数：

$$tv = \text{func1}(t\text{-id}, v\text{-id}) = 4 \times v\text{-}id + t\text{-id}$$

我们无需记住上述的函数细节，只需要知道：

- 这个函数是根据我们给定的 T-layout & V-layout 决定的；
- 作用是可以从 2D 坐标映射为 1D 坐标，1D 坐标也可以转换为 2D；
- 可逆，即，给定输入可以得到输出，通过一个 inverse 变换，可以实现给定输出给定输入。

至此，我们完成了（m-id, n-id）和（t-id, v-id）之间的映射关系。实际上这个映射关系的功能，就是给定行坐标和列坐标得到一个新的坐标，在 CuTe 表达体系中，把这种坐标映射的操作称为 layout，**这个 layout 的输入是 t-id 和 v-id，因此我们把这个 layout 命名为 TV-layout**。同理，我们把第一个表格也理解为 layout，其作用是给定 data 的 m-id 和 n-id，返回 t-id 和 v-id。也就是说**这个 layout 的输入是 m-id 和 n-id，因此我们可以把这个 layout 命名为 MN-layout**。

至此，我们可以用 Fig. 5 来描述这个过程：

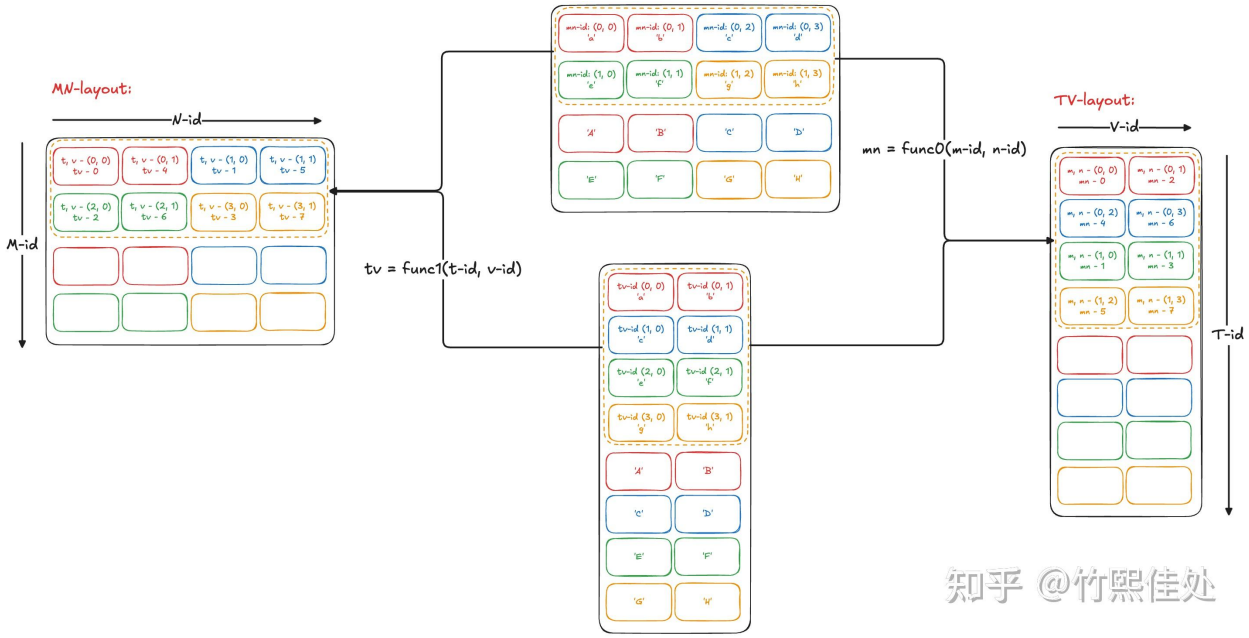


图 14.5: Figure.5 CuTe 中 TV-layout 和 MN-layout 的构造方式示意图. TV-layout 是一个行为 t-id 列为 v-id 的 layout，以（t-id, v-id）为输入坐标能索引到的值是 mn，也就是 (m-id, n-id) 的 1D 形式。同理，MN-layout 是一个行为 m-id 列为 n-id 的 layout，以（m-id, n-id）为输入坐标能索引到的值是 tv.

事实上，我们看 **make_tiled_copy 的具体实现，其实就是在完成 TV-layout 和 MN-layout 的构建，并将 TV-layout 信息保存在 tiled_copy 对象中**。对应 CuTe 中 make_tiled_copy 函数的代码，本质上就是在完成我们上面描述的过程：

```
template <class... Args,
          class ThrLayout,
          class ValLayout = Layout<_1>>
CUTE_HOST_DEVICE
auto
make_tiled_copy(Copy_Atom<Args...> const& copy_atom,
```

```
                ThrLayout          const& thr_layout = {},     // (m,n) -> thr_idx
                ValLayout          const& val_layout = {})     // (m,n) -> val_idx
{
  // Take the raked_products to compute the Layout_MN
  // (M,N) -> (thr_idx, val_idx)
  auto layout_mn = raked_product(thr_layout, val_layout);
  // (thr_idx, val_idx) -> (M,N)
  auto layout_tv = right_inverse(layout_mn).with_shape(make_shape(size(thr_layout), size(val_layout)));
  // Tiler for extracting relevant elements
  // (M,N) -> tensor coord
  auto tiler = product_each(shape(layout_mn));

  return make_tiled_copy_impl(copy_atom, layout_tv, tiler);
}
```

我们可以用 cute 提供的 print_latex 函数，把 make_tiled_copy 中计算出来的 mn-layout 和 tv-layout 打印出来看看，我们能看到这个 layout 其实和我们自己画出来的是同样的语意。例如，在我们的例子中，我们构建的 TV-layout 和 MN-layout 对比 CuTe 打印输出的 TV-layout 和 MN-layout 如 Fig.6 所示：

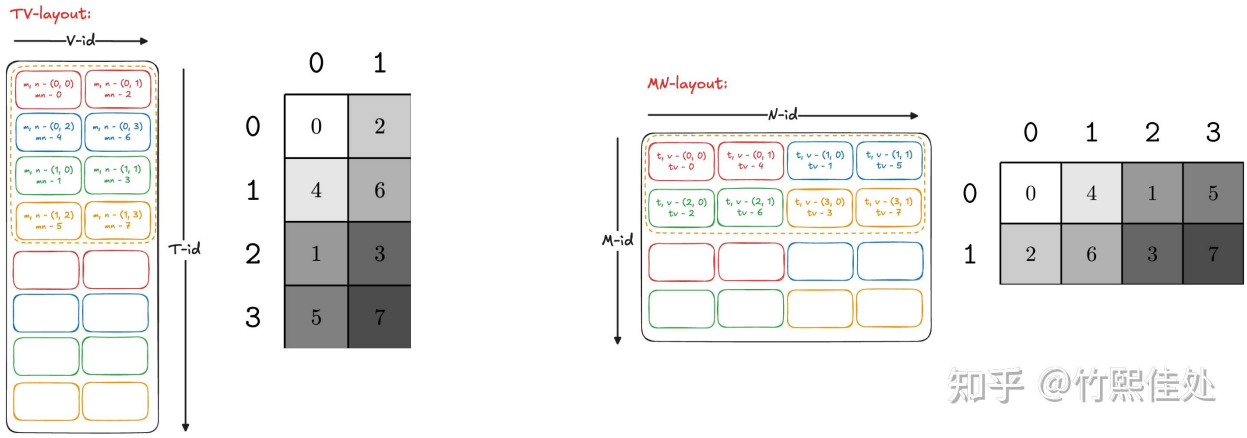


图 14.6: Figure.6 我们构建出的 TV-layout 和 MN-layout 与 CuTe 打印出来的 TV-layout 以及 MN-layout 对比

接下来，我们回顾一下我们在第一节中是如何得到 TV-layout 的：

- 首先，根据我们的 tile data，思考我们怎样划分 thread，即，在每一行上有多少 thread，能够分多少行，每一个 thread 又一次需要负责多少个 value；
- 从数据角度来看能够构建 (m-id, n-id) 到 (t-id, v-id) 的表格，即，MN-layout；
- MN-layout 转换为 TV-layout，即，输入输出对调

那么具体到 CuTe 的写法上，

- 如何划分 thread，这对应 CuTe 中的 T-layout 怎么定义。比如，对应到我们的例子，我们希望一行有 2 个 thread，总共 4 threads 可以划分 2 行，那么 T-layout 可以定为 (2, 2)。同理，我们观察到每个 thread 会 copy 几个元素，在我们的例子里，每个 thread copy 2 个连续的元素，因此 V-layout 可以定义为 (1, 2)；
- 我们观察我们要构建出来的 MN-layout，实际上如果对 CuTe layout 运算比较熟悉，是能够想到这里是用我们定义的 T-layout 和 V-layout 进行一个 product 运算来产生这样一个 layout 的。具体的，在 CuTe 中有一个 raked_product 可以完成我们的需求；
- MN-layout 转换为 TV-layout，通过一个 layout-inverse 即可完成。

上述过程中，我们实际上只考虑了单次循环中发生的事情，每个循环中，所有的 thread 会参与一次 copy，每次 copy v 个 element，这样的一个轮次被称为一个 threadblock 层级下的 atom copy。一个 tiled copy 实际上是对这种 atom 做循环，就像 Code.1. 中我们自己定义的 i-loop。虽然我们的例子中这个循环只有 1D，但是实际使用中，这个循环通常是 2D 的，因此我们经常可以看到 CuTe 代码中会有类似这样的注释：

```
auto tgA_g2s = thr_copy_g2s.partition_S(g_input_tile);  // (CPY_ATOM, CPY_M, CPY_N)
```

这里的 CPY_M, CPY_N 就代表着对 atom 做 2d 循环的次数。至此，我们可以有个初步印象，“**atom 之外，皆是循环**”，我们先记住这句话，后面看看能否有更深入的理解。

至此，我们基本梳理完了 make_tiled_copy 函数中做的事情。我们结合上面的理解，重新看看我们如何定义 make_tiled_copy 中的三大参数：

- Copy-atom：因为我们 1 次要 copy 2 个 element，并且我们的 data type 是 float，因此我们选择 UniversalCopy<cute::uint64_t> 作为 copy 指令，并且通过 traits 出单条指令所需要的 thread-value layout 构建出一个 Copy-atom；
- Thread-layout：我们总共有 4 个 thread，每次 copy 2 个连续的 element，考虑 data tile 每行有 4 个元素，因此每行分配 2 threads，总共可以分 2 行，因此设为 (2, 2)；
- Value-layout：每个 thread 单次 copy 2 个连续的 element，因此设为 (1, 2)。

值得一提的是，make_tiled_copy 在构建和 mma 相关的 copy 时，（例如：s2r / r2s copy），有着另一种用法，因为涉及到 tiled_mma，本文暂且不展开，后续我们在 tiled_mma 章节来继续探索。

14.4.4 CuTe partition 解读

在 make_tiled_copy 的定义没有问题后，我们看看如何更进一步的理解 partition 里面的设计。

Partition 的目的即是，将一个大块的 tensor 拆分成多个小 tensor，每个 thread 都获取一个小 tensor，我们称之为 thr-tensor。那么每个 thread 如何知道自己需要得到哪个 thr-tensor 呢？或者说，如何知道自己需要哪些 data?

我们惊喜的发现，我们所需要的正是我们之前构建出来的 TV-layout，我们只需要给定一个 tid，然后取出 TV-layout 这一行的所有列对应的 mn 坐标，其对应的 element 即是我们所需要 thr-tensor。

以及，由于我们上面说到的，我们构建出来的 TV-layout 其实只是一次 atom copy 的 TV-layout。为了完成整个 tile 的 partition，在 CuTe 的实现中，首先是将一个 tile 平均拆分成（CPY_M, CPY_N）个 atom 大小，然后构建出对应的 atom-TV-layout，进而拿到整个 tile 的 TV-layout。

然后再取出 TV-layout，赋予一个 tid，然后取出所有的 value，组成一个 thr-tensor，这就是 partition 的过程。不得不感慨这个逻辑的确是很严谨的。

整个过程可以描述为 Fig.7

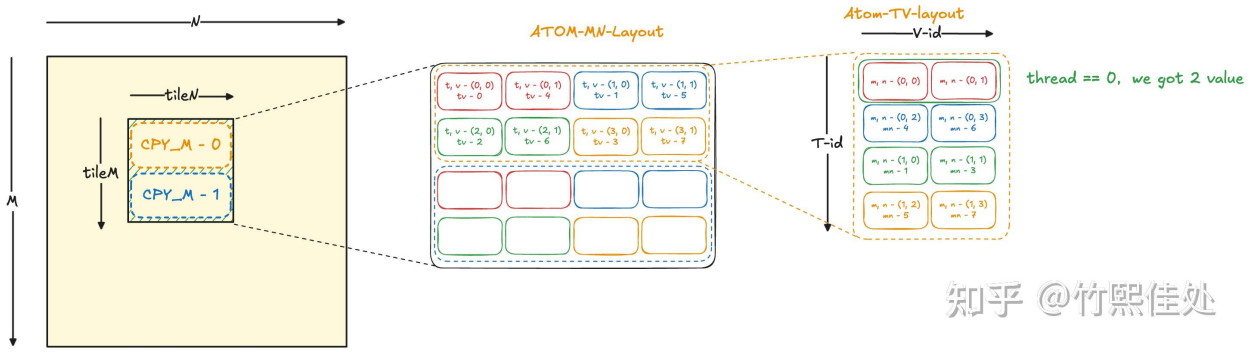


图 14.7: Figure.7 tiled copy 过程描述：首先分 tile，然后构建 TV-layout，最后 partition 给定一个 tid，取出所有的 value，即是我们所需要的 thr-tensor

14.4.5 进阶：CuTe Copy_Atom 解读

进一步扩展，我们可以思考一下这样设计出来的 tiled copy 为什么能适配各种各样的 copy 场景?

在上面的过程中，最核心的步骤就是获得这个 tile data 对应的 tv-layout：给定任意一种 copy 指令，无论是单 thread 完成还是 warp 完成，一旦能构建出来 tv-layout，我们就能给定 tid 拿到 thr-tensor 了。

那么如何能够获得这个 tile data 的 TV-layout？我们在之前强调过了：“atom 之外，皆是循环”，对于 tile data 来说，只是在循环 atom 罢了。因此，我们关注这一小块的 atom 区域对应的 atom-tv-layout 是如何构建的即可。

那么，atom-tv-layout 的构建，需要哪些信息呢？在之前的例子中，我们是用 t-layout 和 v-layout 做 raked_product 得到的，那么这个方法有什么问题呢？

我们注意，在我们构建 g2s copy 的时候，我们似乎默认了我们构建出来的 atom-tv-layout 对于 src (即，global) 和 dst (即，shared) tensor 来说是一样的，这在 g2s 这种简单场景下是成立的。然而在实际使用中，我们也会遇到 src 和 dst 所需要的 tv-layout 不一致的情况，（例如 ldmatrix，每个 thread 读到的数据实际上会分发给不同 thread），这时候难道我们需要分别传入 src-atom-tv-layout 和 dst-atom-tv-layout 吗？

直观上确实如此，然而 CuTe 的作者有着另一种解决思路：**让 src-atom-tv-layout 和 dst-atom-tv-layout 通过 layout 代数运算可以相互转换。我们将 src-atom-tv-layout 和 dst-atom-tv-layout 封装在构建 copy_atom 中，并且这个相互转换的方式可以就隐藏在我们所用的 Copy_Atom 之中**。

也就是说，Copy atom 需要做到如下功能：

- 有 copy 能力，也就是给定一个 src & dst 地址，能完成一次数据的搬移，此即 copy_op 能带来的信息；
- 可以指定 src-atom-tv-layout / dst-atom-tv-layout 以及相互转换的能力，此即 copy_traits 能带来的信息。

具体来说，在 Atom 中除了指定 src-layout / dst-layout，也会指定一个隐藏的中间 layout：ref-layout，其一定是 src 或 dst 中的一个。Src-layout <-> ref-layout <-> dst-layout，彼此用 layout inverse + compose 变换可以做到相互转换，通过这种转换，即可做到只关注 ref-layout，不太需要关注 ref 到 src-layout / dst-layout 是怎么变换的。

我们观察 Copy_atom 中 Copy_tratis 的实现代码，其实就是在描述这样一件事，感兴趣的朋友可以自行查阅源码：

```
/**
 * concept Copy_Traits
 * {
 *   using ThrID     =    // Logical thread id (tid) -> tidx
 *
 *   using SrcLayout =    // (Logical src thread id (tid), Logical src value id (vid)) -> bit
 *   using DstLayout =    // (Logical dst thread id (tid), Logical dst value id (vid)) -> bit
 *   using RefLayout =    // (Logical ref thread id (tid), Logical ref value id (vid)) -> bit
 * };
 *
 * The abstract bit ordering of the Copy_Traits (the codomain of SrcLayout, DstLayout, and RefLayout)
 * is arbitrary and only used to construct maps
 *   (ref-tid,ref-vid) -> (src-tid,src-vid)
 *   (ref-tid,ref-vid) -> (dst-tid,dst-vid)
 * in TiledCopy. The Layout_TV in TiledCopy is in accordance with the RefLayout of a Traits, then mapped
    to
 * the Src or Dst (tid,vid) representation on demand.
 *
 */

template <class CopyOperation, class... CopyOpArgs>
struct Copy_Traits
{
  static_assert(dependent_false<CopyOperation>, "Copy_Traits not implemented for this CopyOperation.");
};

template <class S, class D>
struct Copy_Traits<UniversalCopy<S,D>>
{
  // Logical thread id to thread idx (one-thread)
  using ThrID = Layout<_1>;

  // Map from (src-thr,src-val) to bit
  using SrcLayout = Layout<Shape<_1,Int<sizeof_bits<S>::value>>>;
  // Map from (dst-thr,dst-val) to bit
  using DstLayout = Layout<Shape<_1,Int<sizeof_bits<D>::value>>>;
```

```
36
37   // Reference map from (thr,val) to bit
38   using RefLayout = SrcLayout;
39 };
```

14.5 总结

本文以 tiled copy 为基础，讲解了 tiled_copy 的基础用法，以及其底层的核心概念：TV-layout，并以此为基础尝试探讨了 CuTe 的设计哲学。

梳理完后，我们发现，CuTe 的设计逻辑是非常完备的，面对越来越复杂的 GPU 硬件体系，我们需要一套完备的编程模式可以兼容未来可能存在的任何 copy / mma 能力，CuTe 即是 NV 提出的这样一种编程模式，事实也证明其确实很好的处理了历代以来的 NV GPU。

然而，CuTe 的代码确实需要一些耐心去梳理，这篇文章其实是一年前 reed 带领我们探索 CuTe 时就发给笔者的任务，直到最近笔者才真正开始慢慢领悟到其精髓，笔者将这次梳理的过程记录下来，也希望能够给大家学 CuTe 带来一点帮助。

第十五章　写给大家看的 CuTe 教程：tiled mma

作者：竹熙佳处（知乎专栏「CuTe 教程」）　原文：https://zhuanlan.zhihu.com/p/1937145378446226159

15.1　前序知识

在上一篇写给大家看的 CuTe 教程：tiled copy 我们介绍了 CuTe 中的 tiled copy 的使用方法，并引出了 CuTe 中的核心概念 TV-layout，本文的阅读同样也需要用到该概念。为方便阅读，我们在此简单回顾一下：

CuTe 体系中最核心的概念：TV-layout。其作用是：给定一个 thread-id & value-id，一定能找到一个数据的坐标（这个数据可能在上文中 copy 指令的 src 或是 dst，也可以是本文中 mma 指令中的输入输出寄存器中），进而可以拿到对应数据；

将 TV-layout 输入输出 inverse，即是 MN-layout，其作用是：给定一个数据的位置坐标 m-id & n-id，一定能找到其对应的 thread-id & value-id，通常用来展示数据视角下，每个 thread 会对应到哪些数据。

对更多细节感兴趣的读者可以先读一下 tiled-copy 章节。

15.2　动机：从 cuda 视角理解的 tiled mma

Tiled mma 是 CuTe 中除 tiled copy 之外另一大核心组件，其目的是在解决“如何正确调用 tensor core 完成矩阵乘”这样一个问题。

在传统 cuda 代码中，想要调用 tensor core 通常使用 mma 指令，而 mma 指令要求多个 threads 协同调用。我们以目前较为流行的 Ampere 架构 (sm80) 的 16x8x16 fp16 mma 指令为例，其主要功能是调用 tensor core 完成一次 m16n8k16 的矩阵乘法 C = A * B。具体的，这条指令以一个 warp 为单元，从这个 warp 中的每个 thread 中的寄存器中读入 A 矩阵 16x16，读入 B 矩阵 8x16，调用 tensor core 完成计算，然后把结果 C 矩阵写出到这个 warp 中每个 thread 中的寄存器 C 矩阵中。

我们注意到，在构成 A/B/C 矩阵时，每个 thread 贡献一部分数据，且为了正确输出结果，我们需要知道每个 thread 去贡献哪一部分数据，并将其按固定顺序排布在寄存器中。具体的，我们以 16x8 的 C 矩阵为例，其需要的数据排布如 Fig.1 所示：

进一步的，我们刚刚思考的是一个 warp 完成一条 16x8x16 的 mma 指令，实际上我们在真实场景中所需的矩阵乘法可能是远大于这个尺寸的，例如，128x128x16，那么如何去扩展到这个规格上呢，简单的想法就是在行方向上扩展 8 次，16 * 8 -> 128，列方向上扩展 16 次，8 * 16 -> 128。扩展的方式可以是：

并行维度：一个 threadblock 中起多个 warps，每个 warp 都执行一个 16x8x16 的矩阵乘，共同完成一个 threadblock 级别的 mma；

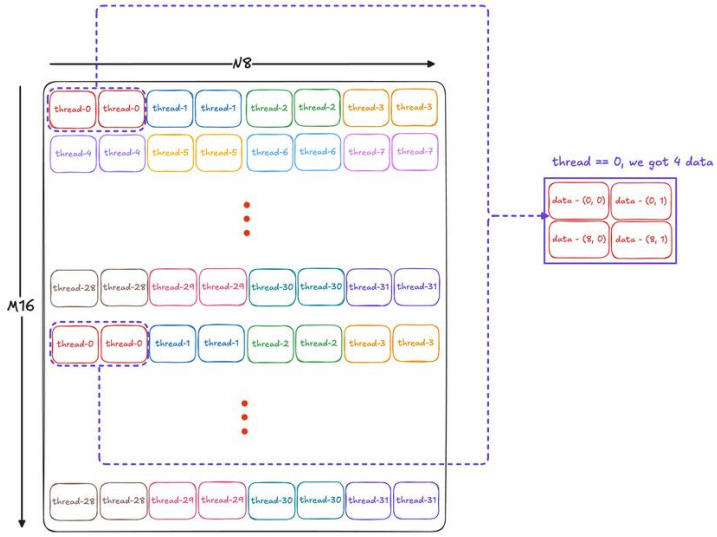
串行维度：这个 threadblock 循环执行多次 threadblock 级别矩阵乘来完成一个更大的 mma。

例如，我们首先指定一个 threadblock 包含 4 warps，在行方向上分配 2 个，列方向上分配 2 个，那么这个 threadblock 就可以完成一次 32x16x16 mma 了；然后我们发现我们实际想完成的 tiled mma 是 128x128x16，因此我们需要再将整个 threadblock 进行行方向循环 4 次和列方向循环 8 次。

当然，我们考虑到 threadblock 一次处理的数据在列方向上最好更大一点，以方便大字长访存，所以我们把 8 再拆解为 2x4，这样一个 threadblock 可以认为一次处理 32x32x16 mma，在列方向上循环 4 次。此即 tiled mma 完成的事情，我们可以把这个重复的过程总结为 Fig.2。

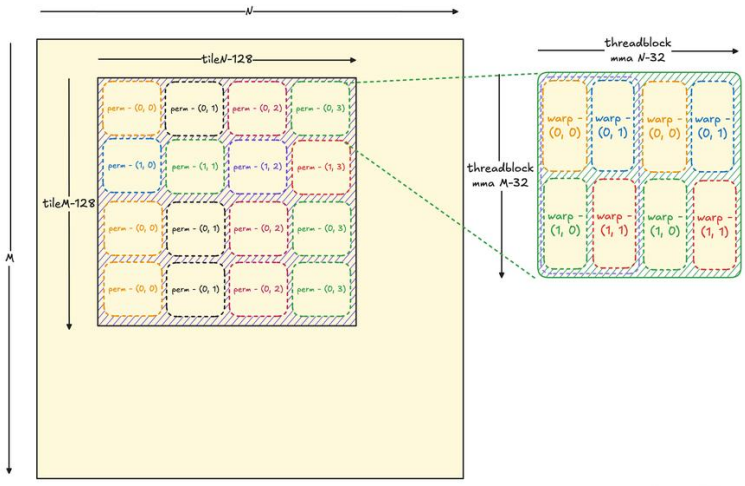
我们可以看到，对于 mma 指令，thread 和 data 的对应关系并不平凡，每次我们调用不同 mma 指令（例如，我们想调用更小规格的 fp16 指令 16x8x8，或是 int8 的指令 16x8x32）的时候，我们都需要去 ptx 手册上查类似 Fig.1 对应的数据排布，然后按照这个排布把寄存器分配好，才能完成一次正确调用 tensor core。进一步的，我们需要考虑 Fig.2 中的 threadblock mma 在 M 和 N 维度的循环，这个过程也容易让开发者迷失在各种坐标运算的细节中。

除此之外，使用 cuda + ptx 的方式调用 mma 还有一个潜在的风险，就是在架构更新的时候，tensor core 所需的数据排布可能发生重大变化（例如，volta 架构所需的排布就与 ampere 所需的排布不同），这时候很多代码都需要重写。我



知乎 @竹熙佳处

图 15.1: Figure.1 m16n8k16 mma 指令 C 矩阵（16x8）的 thread-data 映射关系，以 thread0 为例，其在输出 C 矩阵时，会负责 4 个 data，分别对应坐标 (0, 0)，(0, 1)，(8, 0)，(8, 1)。



知乎 @竹熙佳处

图 15.2: Figure.2 threadblock-level-mma 与 tile data 的切分关系。其中每个 warp 一次调用 mma 指令可以输出 16x8 的 C 矩阵，一个 threadblock 包含 4 个 warp，并让每个 warp 重复调用 2 次 mma 指令，可以得到 32x32 的 C 矩阵。将这个 32x32 的块进一步重复调用，可以得到 tileM x tileN 的数据块。

们猜测 nv 内部写 cublas 库的工程师可能就是受其所扰，才会开始构建 CuTe 这样一个表达体系来兼容不同 tensor core 指令以及对应的 thread 和 data 映射关系。

至此我们可以认为，CuTe 封装 tiled mma 的动机，是希望能够给定任意一个 mma 指令，我们能很方便的推导出其从 data tile 中找到每个 thread 所需要的部分，进而完成一次正确的 mma 指令调用。从这个动机出发，我们一起来梳理一下 tiled mma 的用法以及实现细节。

15.3　CuTe Tiled mma 的使用

我们首先来看 CuTe 定义 tiled mma 的一个简单例子，并逐一解释几大模块，为专注于 tiled mma，我们暂不引入 tiled copy，而改用最普通的 copy 来完成数据 g2r / r2g 的 copy（在这里读者可以尝试思考一下，为什么不引入 tiled copy 的情况下，copy 指令依然能正确找到要读的数据，提示：从 TV-layout 的角度思考）：

```
template <int kTileM, int kTileN, int kTileK>
__global__ void my_tiled_mma(__half *Cptr, const __half *Aptr,
                             const __half *Bptr, int m, int n, int k) {

  int tid_in_block = threadIdx.x;
  int i_tile_n = blockIdx.x;
  int i_tile_m = blockIdx.y;

  auto A = make_tensor(make_gmem_ptr(Aptr), make_shape(m, k),
                       make_stride(k, Int<1>{}));
  auto B = make_tensor(make_gmem_ptr(Bptr), make_shape(n, k),
                       make_stride(k, Int<1>{}));
  auto C = make_tensor(make_gmem_ptr(Cptr), make_shape(m, n),
                       make_stride(n, Int<1>{}));

  auto gA = local_tile(A, make_tile(Int<kTileM>{}, Int<kTileK>{}),
                       make_coord(i_tile_m, _));
  auto gB = local_tile(B, make_tile(Int<kTileN>{}, Int<kTileK>{}),
                       make_coord(i_tile_n, _));
  auto gC = local_tile(C, make_tile(Int<kTileM>{}, Int<kTileN>{}),
                       make_coord(i_tile_m, i_tile_n));

  // 0. 构造 tiled mma
  using mma_op = SM80_16x8x16_F16F16F16F16_TN;
  using mma_traits = MMA_Traits<mma_op>;
  using mma_atom = MMA_Atom<mma_traits>;

  static constexpr int kMmaEURepeatM = 2;
  static constexpr int kMmaEURepeatN = 2;
  static constexpr int kMmaEURepeatK = 1;

  using mma_atom_shape = mma_traits::Shape_MNK;
  static constexpr int kMmaPM = 32; // 32
  static constexpr int kMmaPN = 32; // 32
  static constexpr int kMmaPK = 16; // 16

  using MMA_EU_RepeatT = decltype(make_layout(make_shape(
      Int<kMmaEURepeatM>{}, Int<kMmaEURepeatN>{}, Int<kMmaEURepeatK>{})));
  using MMA_P_T = Tile<Int<kMmaPM>, Int<kMmaPN>, Int<kMmaPK>>;

  using MMA = decltype(make_tiled_mma(mma_atom{}, MMA_EU_RepeatT{}, MMA_P_T{}));

  MMA tiled_mma;

  // 1. tiled mma partition
  auto thr_mma = tiled_mma.get_slice(tid_in_block);
  auto trA = thr_mma.partition_fragment_A(gA(_, _, 0));
```

```
  auto tgA = thr_mma.partition_A(gA(_, _, 0));

  auto trB = thr_mma.partition_fragment_B(gB(_, _, 0));
  auto tgB = thr_mma.partition_B(gB(_, _, 0));

  auto trC = thr_mma.partition_fragment_C(gC);
  auto tgC = thr_mma.partition_C(gC);

  cute::copy(tgA, trA);
  cute::copy(tgB, trB);

  // 2. gemm 执行
  cute::gemm(tiled_mma, trA, trB, trC);

  cute::copy(trC, tgC);
}
```

15.4 Tiled mma 的构造

```
using mma_op = SM80_16x8x16_F32F16F16F32_TN;

using mma_traits = MMA_Traits<mma_op>;
using mma_atom = MMA_Atom<mma_traits>;

static constexpr int kMmaEURepeatM = 2; // 2-M-warp
static constexpr int kMmaEURepeatN = 2; // 2-N-warp
static constexpr int kMmaEURepeatK = 1; // always 1

static constexpr int kMmaPM = 32; // Perm-M
static constexpr int kMmaPN = 32; // Perm-N
static constexpr int kMmaPK = 16; // always 16 when using 16x8x16 mma op

using MMA_EU_RepeatT = decltype(make_layout(make_shape(
      Int<kMmaEURepeatM>{}, Int<kMmaEURepeatN>{}, Int<kMmaEURepeatK>{})));
using MMA_P_T = Tile<Int<kMmaPM>, Int<kMmaPN>, Int<kMmaPK>>;

using MMA = decltype(make_tiled_mma(mma_atom{}, MMA_EU_RepeatT{}, MMA_P_T{}));
```

和 tiled_copy 一样，其由三大元素构成：

mma_atom：描述我们用什么 mma 指令（16x8x16 fp16，16x8x32 int8 等），同时也包含完成一次这样指令所需要的 atom-TV-layout。

EU-layout：EU 指的是 execute unit，根据架构不同，实际上是指 1/4-warp (sm70) / warp (sm80) / warp-group (sm90)。也就是执行一条 mma 指令所需要的执行器粒度。如果我们查看 tiled mma 的构造过程就可以看到，CuTe 实现时是把 EU_layout 和 mma atom 中的 threads layout 做了一个 tiled_product 的，也即是把执行器 layout 转换为具体的 thread layout 了。

permutation：指定了一次 threadblock 能做的 mma 块大小，tiled mma 其实就是把这 permutation 块在 M 和 N 维度上重复多次。而 permutation 内部由多个 mma op 结果组成，这些所有的结果是由两个层面上的重复完成的，里层是按照 EU-layout 并行分的，外层是把每一个 EU 串行重复执行多条指令。

具体的，我们可以将 Tile -> permutation -> atom 的关系可以展现为 Fig.3

进一步的，我们借用 cute::print_latex 函数将我们构造出来的 tiled mma 打印出来如 Fig.4，其打印的其实就是 MN-layout，即每个给定一个行列坐标（M-id, N-id），能找到对应的 T-id 和 V-id，对于我们理解 thread - data 的映射关系是比较直观的。Fig.4 可以进一步验证我们构造出来的切分模型是正确的。

另一个例子是 flash atten v2 中 QK 矩阵乘，其为了让同一行的数据都放在同一个 warp 里以达到 warp 内循环算 softmax 的效果，其设计 EU-layout 以及 permutation 把 warp 按竖着排列。

```
using mma_op = SM80_16x8x16_F32F16F16F32_TN;
```

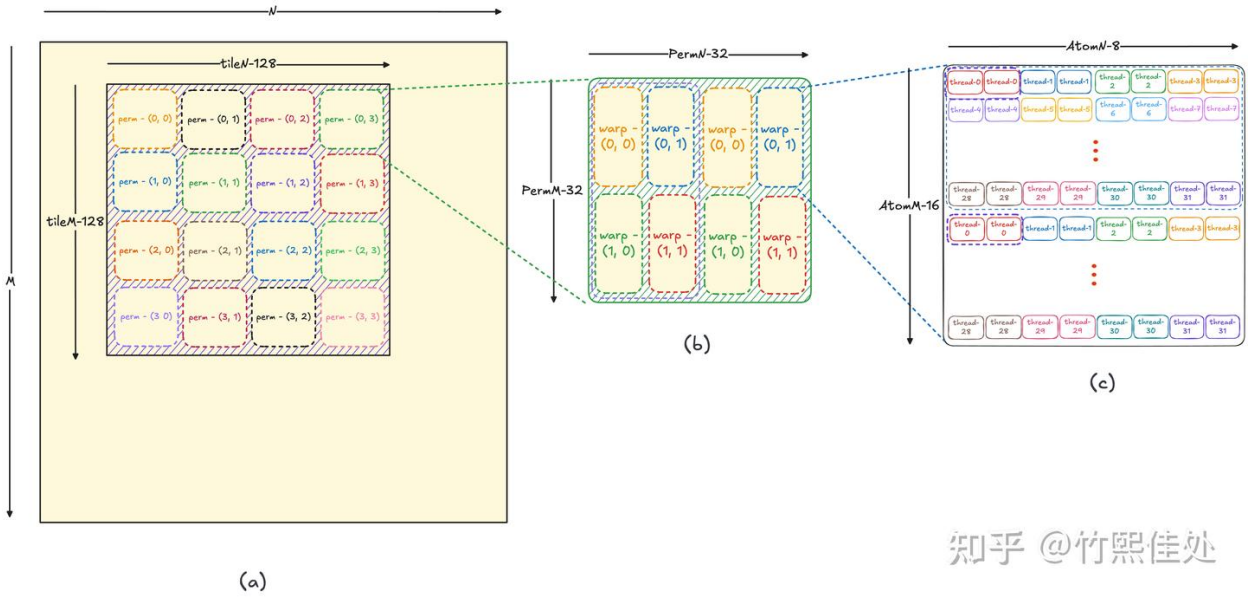


图 15.3: Figure.3 Tiled mma 中 Tile / permutation / atom 三层切分关系。tileM * tileN 会被切分为多个 permutation，而 permutation 由多个 atom 组合，permutation 包含了 warp 的并行重复，也包含指令的串行重复

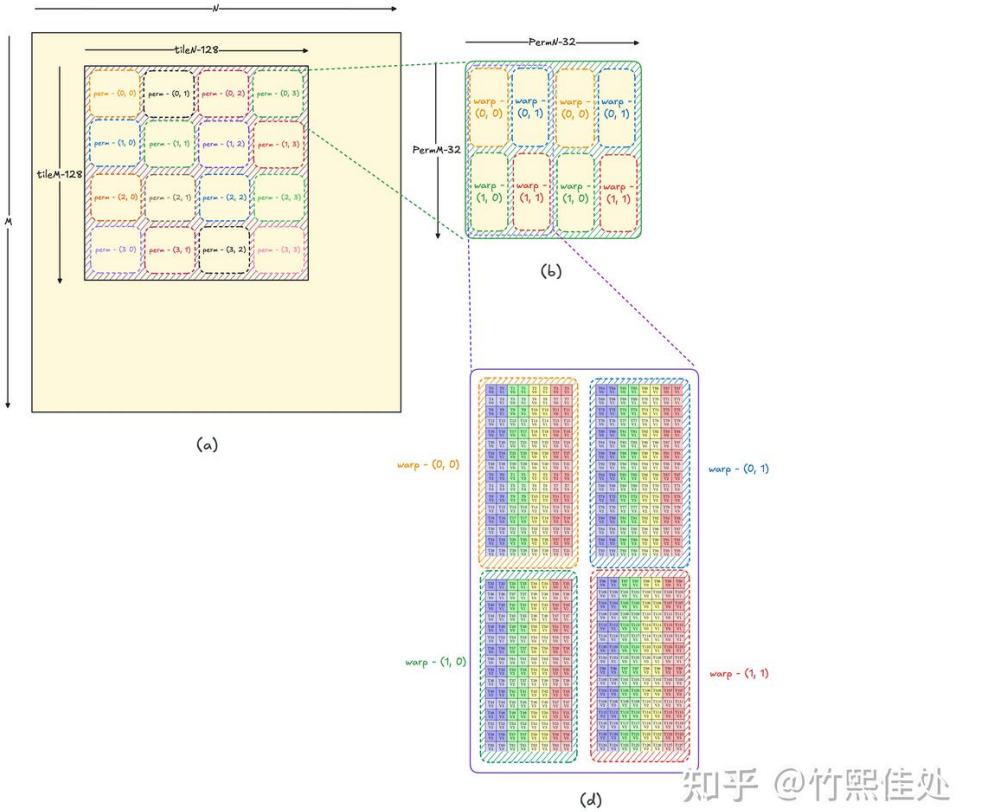


图 15.4: Figure.4 Tiled mma 中 Tile / permutation 对比 print_latex 打印出来的 tiled mma。可知 tiled mma 是先并行排布 warp 构成一个 threadblock 32x16x16 的 mma，再串行排布指令完成一个 permutation 大小的 mma。在 CuTe 中默认排布是 col-major，因此 warp(0,1) 实际对应的是 thread 64~95

```
2
3  using mma_traits = MMA_Traits<mma_op>;
4  using mma_atom = MMA_Atom<mma_traits>;
5
6  static constexpr int kMmaEURepeatM = 4; // 4-M-warp
7  static constexpr int kMmaEURepeatN = 1; // 1-N-warp
8  static constexpr int kMmaEURepeatK = 1; // always 1
9
10 static constexpr int kMmaPM = 64; // Perm-M
11 static constexpr int kMmaPN = 32; // Perm-N
12 static constexpr int kMmaPK = 16; // always 16 when using 16x8x16 mma op
13
14 using MMA_EU_RepeatT = decltype(make_layout(make_shape(
15       Int<kMmaEURepeatM>{}, Int<kMmaEURepeatN>{}, Int<kMmaEURepeatK>{})));
16 using MMA_P_T = Tile<Int<kMmaPM>, Int<kMmaPN>, Int<kMmaPK>>;
17
18 using MMA = decltype(make_tiled_mma(mma_atom{}, MMA_EU_RepeatT{}, MMA_P_T{}));
```

那么其 C 矩阵对应的 tile -> permutation 可以按 Fig.5 描述：

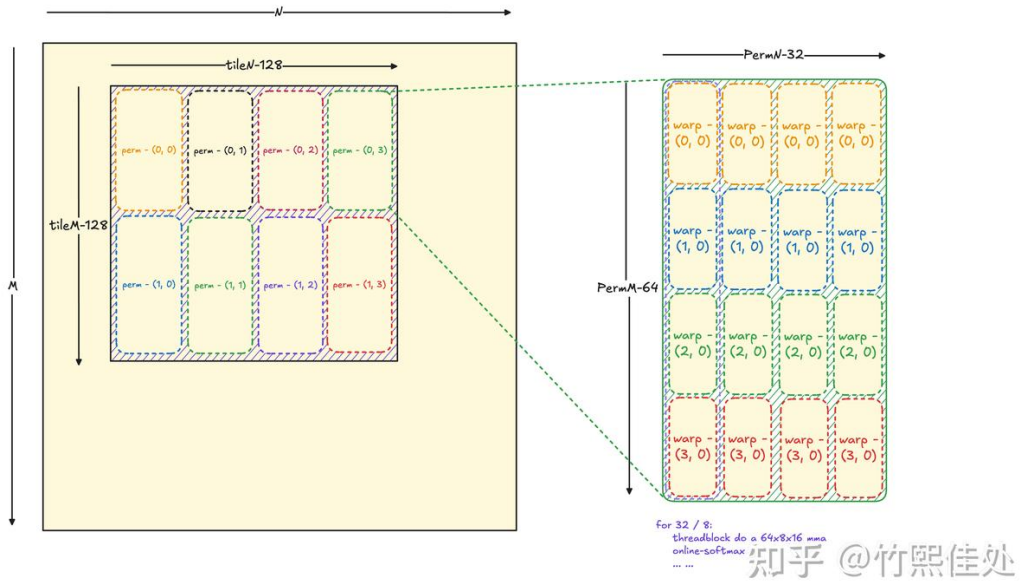


图 15.5: Figure.5 flash atten v2 中 Tiled mma 中 Tile / permutation 切分方式。为了保证一个 warp 能计算得到到一行中的所有数据（为了后续做 softmax），其将 warp 排布改成了 4x1

在此，读者可能有疑问，有 EU-layout 并且加上 16x8 的 atom 之后，就已经有重复 mma 来划分 tileM x tileN 的能力了，为什么还需要一个 permutation 的维度呢？其实是因为一个 permutation 块在后续的 s2r / r2s tiled copy 的时候是作为一个整体去参与 copy 的，为了做到更大字长的 copy（如，ldsm.x4），通常 permutation 的 N 会设为 warpN * 16，而一次 atom 实际上只有 warpN * 8。

另外，读者可能注意到，Permutation 似乎只是用来规定 threadblock mma 的 tile size，那为什么不直接叫 tile 呢？这是因为在一些高级用法上，我们想做到 tiled 中的多条 16x8x16 mma 指令算的数据的位置重新排列，例如，我们希望 warp 连续做多个 16x8x16 mma，这时候我们可以通过构造 Permutation 中的 stride 来完成，这就是为什么叫 “Permutation”。详细的内容在我们进阶篇的教程中有单独的文章进行了介绍：

写给进阶开发的 CuTe 笔记：tiled mma 的 permutationMNK 参数

感兴趣的读者可以跳转阅读。在一般场景下，我们将其当作 tile size 理解即可。

15.5　Tiled mma partition

接下来是 partition 过程。我们知道 partition 的目的是，将一个大块的 tensor 拆分成多个小 tensor，每个 thread 都获取一个小 tensor，我们称之为 thr-tensor。以 C 矩阵为例，我们从 Fig.1 和 Fig.2 的过程中了解到，一个 thread 所能拿到的数据取决于两个层次，其一是最原子的 16x8 中可以拿到的 4 个数据，其二是在 M，N 维度上循环而产生的多份数据。因此，对一个 tileM == 128，tileN == 128 的数据进行 partition 为例，partition 后得到的 thr-tensor layout 对应到我们的图解中，可以表示为 Fig.6。

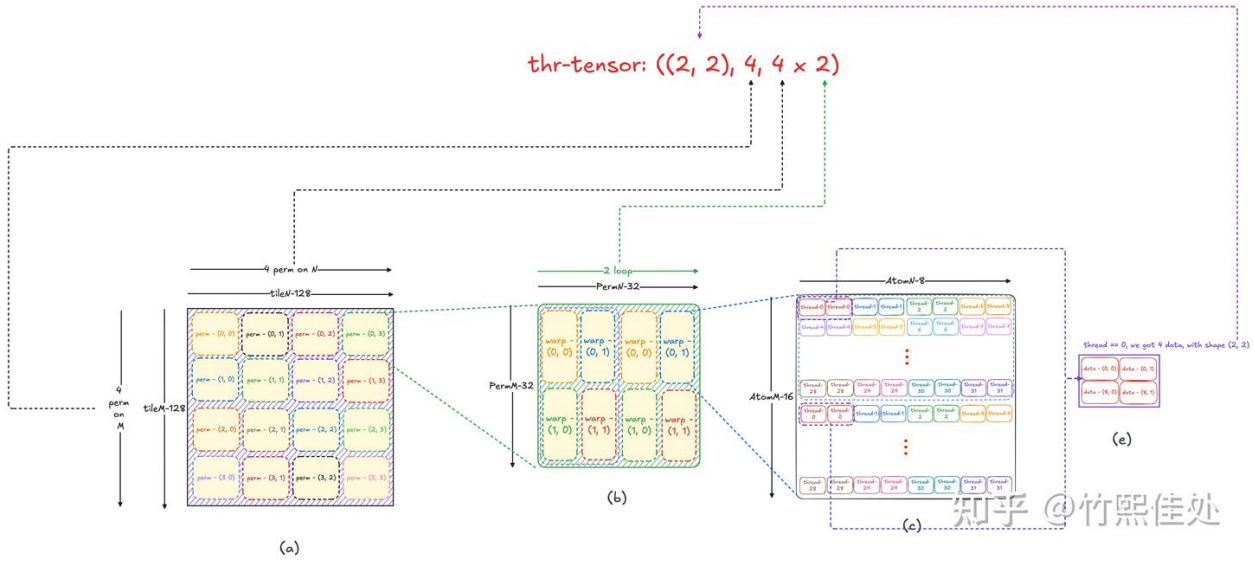


图 15.6: Figure.6 tiled mma partition 后的 thr-tensor 的 shape 解释。(2, 2) 代表一个 mma 指令的 16x8 C 矩阵被分配到 thread 后的 tensor，第二维 4 代表 M 维度上的重复，第三维 8 代表 N 维度上的重复。8 可以拆解为 4x2，4 代表 tile 到 permutation 的重复，2 代表 threadblock 整体做的指令重复

我们在代码中打印一下 code1 中 tensor 的 layout，可以看到其 shape 与图示一致，针对 tgC 和 trC，其 stride 不一样，我们给定 m = 128, n = 128, kTileM = 128, kTileN = 128，打印结果如下：

```
gC: (_128,_128):(128,_1)
trC: ((_2,_2),_4,_8):((_1,_2),_4,_16)
tgC: ((_2,_2),_4,_8):((_1,1024),4096,_16)
```

15.6　Tiled mma gemm 执行

```
cute::gemm(tiled_mma, trA, trB, trC);
```

这一步实际上只是在循环调用 mma 指令。唯一需要注意的是 gemm 函数实际上是可以拆解为多次的 threadblock 级别的 mma 的，也就是说，其最少可以完成一个 threadblock 执行 32x16x16，其他维度都是 for loop 完成的，因此上述代码也可以写成：

```
#pragma unroll
  for(auto i_m = 0; i_m < size<1>(trC); i_m += 1){
#pragma unroll
    for(auto i_n = 0; i_n < size<2>(trC); i_n += 1){
      cute::gemm(tiled_mma, trA(_, i_m, _), trB(_, i_n, _), trC(_, i_m, i_n));
    }
  }
```

15.7 Tiled mma 延伸：s2r / r2s Tiled copy

在我们上述的例子中，为专注在 tiled mma 上，我们省略了大部分优化的逻辑，然而在现实问题中，我们通常需要更多优化手段，例如使用 shared mem 来存放 A/B tile，这就涉及到我们之前介绍的 tiled copy。我们知道 g2s / s2g 的 tiled copy 通常是由我们自己定义 T-layout 和 V-layout 来构建，而涉及到 mma 相关的 s2r / r2s tiled copy 所需的 TV-layout 并不平凡，由我们自己来定义显然有些难度。

好在，mma 的 tv-layout 除了能用来支持 mma 指令，我们还能用其来构建 s2r 和 r2s 的 tiled copy，这是因为我们已知 mma 所对应的 tv-layout（即，r 对应的 tv-layout），通过 s2r / r2s 的 copy atom 中隐含的 src-layout <-> ref-layout <-> dst-layout 的代数运算，可以推导出来其所需要的 shared tensor 的 tv-layout，进而知道 shared tensor 上每个位置应该 copy 哪些数值。mma 和 copy 居然还能组成 combo！不得不说是一个很精巧的设计。

例如，我们通过如下方式可以构造出和我们的 tiled mma 绑定的 s2r / r2s tiled copy。以 C 的 r2s 为例，代码如下：

```
extern __shared__ T shm_data[];

T *Cshm = shm_data;
using SmemLayoutC = decltype(make_layout(make_shape(Int<32>{}, Int<32>{})));
auto sC = make_tensor(make_smem_ptr(Cshm), SmemLayoutC{});
using R2SCopyAtomC = Copy_Atom<UniversalCopy<int>, cute::half_t>;

// 从 tiled mma 延伸出 r2s tiled copy
auto r2s_tiled_copy_c = make_tiled_copy_C(R2SCopyAtomC{}, tiled_mma);
auto r2s_thr_copy_c = r2s_tiled_copy_c.get_slice(idx);
auto trC_r2s = r2s_thr_copy_c.retile_S(trC);
auto tsC_r2s = r2s_thr_copy_c.partition_D(sC);

cute::copy(r2s_tiled_copy_c, trC_r2s, tsC_r2s);

__syncthreads();
```

其中大部分接口与正常 tiled copy 一致，唯一需要说明的是 retile_S，其作用本质上是将 thr-tensor 进行 reshape，将其变换到 copy 所支持的形状，数据本身不发生变化。其实也挺合理的，因为对于寄存器来说变换 layout 是无开销的，怎么变都是一块连续空间。

15.8 进阶：CuTe Tiled mma 的实现细节

我们回顾我们调用 tensor core 完成 mma 的过程：每个 thread 提供一些数据，调用 mma 指令后，得到一些数据。也就是说，我们实际上是在思考，每个 thread 要提供哪些数据（A & B），以及计算完后会获得哪些数据（C）。更具体的，我们希望给出 thread id，就能拿到 tile A/B/C 数据中的一部分。我们惊喜的发现，我们实际上就是需要一个 TV-layout：即，给定一个 t-id & v-id，我们一定能找到 tileM x tileN 这个数据中的坐标，进而通过 partition 拿到对应的数据，组成 thr-tensor。

那么 Tiled mma 如何构建 tv-layout? 我们在 copy 的过程中，会尝试用 thread-layout 以及 value-layout 来构建，因为 copy 场景下是比较好理解 thread 和 data 的映射关系的。

然而在 mma 这种复杂映射下，我们其实是很难去想明白怎样的 thread-layout 以及 value-layout 能 product 出来一个合适的 TV-layout 的。幸运的是，mma 指令所需要的 TV-layout 是固定的，即，我们调用一条 mma 指令的时候，就知道一个 warp 中每个 thread 所对应的数据是怎么样的了。

因此，CuTe 通过在 mma atom 中包含一个 warp 对应的 atom-tv-layout，再用这个 atom-tv-layout 不断重复去获得整个 tiled data 对应的 TV-layout 即可。这样使用者就不需要像 tiled copy 过程中一样，考虑怎么去定这个 thread layout 和 value layout 了，也屏蔽了不同架构下需要的 mma tv layout 的差别，不得不说是一个很优雅的设计。

理解完上述的过程，我们再观察 CuTe 其如何构造 thr-tensor，也就是 partition 过程。其核心代码其实就是在做一件事，即将输入 tensor 的 MN layout，通过 layout divide 拆解到最内层，直到一个 mma-atom 能映射到 atom-MN-layout（对应注释中的 AtomM, AtomN），然后将这个 atom-MN-layout 转变为 atom-TV-layout，这样整个 tensor 的 TV-layout 就被转换出来了。

```
  template <class CTensor>
```

```
  CUTE_HOST_DEVICE constexpr
  auto
  partition_C(CTensor&& ctensor) const
  {
    auto thr_tensor = make_tensor(static_cast<CTensor&&>(ctensor).data(), this->thrfrg_C(ctensor.layout())
    );

    auto thr_vmn = make_coord(get<0>(thr_vmnk_), make_coord(get<1>(thr_vmnk_), get<2>(thr_vmnk_)));
    return thr_tensor(thr_vmn, make_coord(_, repeat<rank<1,1>(thr_tensor)>(_)));
  }

  // Tile a tensor or a layout from shape
  //   (M,N,...)
  // to shape
  //   ((ThrV,(ThrM,ThrN)),(FrgV,(RestM,RestN,...)))
  // where
  //   ThrV:  The threads local to an MMA. layout<0>(ThrLayoutVMNK): ThrV -> thread_idx
  //   ThrM:  The threads tiled in M.      layout<1>(ThrLayoutVMNK): ThrM -> thread_idx
  //   ThrN:  The threads tiled in N.      layout<2>(ThrLayoutVMNK): ThrN -> thread_idx
  //   FrgV:  The values local to an MMA.
  //   RestM: The values tiled in M.
  //   RestN: The values tiled in N.
  template <class CTensor>
  CUTE_HOST_DEVICE constexpr
  auto
  thrfrg_C(CTensor&& ctensor) const
  {
    CUTE_STATIC_ASSERT_V(rank(ctensor) >= Int<2>{});
    // Reorder the tensor for the TiledAtom
    auto t_tile = make_tile(permutation_mnk<0>(),
                            permutation_mnk<1>());
    auto t_tensor = logical_divide(ctensor, t_tile);                 // (PermM,PermN)

    // Tile the tensor for the Atom
    auto c_tile = make_tile(make_layout(size<0>(AtomShape_MNK{})),
                            make_layout(size<1>(AtomShape_MNK{})));
    auto c_tensor = zipped_divide(t_tensor, c_tile);                 // ((AtomM,AtomN),(RestM,RestN))

    // Transform the Atom mode from (M,K) to (Thr,Val)
    auto tv_tensor = c_tensor.compose(AtomLayoutC_TV{},_);           // ((ThrV,FrgV),(RestM,RestN))

    // Tile the tensor for the C-threads
    auto thr_tile = make_tile(_,
                              make_tile(make_layout(size<1>(thr_layout_vmnk_)),
                                        make_layout(size<2>(thr_layout_vmnk_))));
    auto thr_tensor = zipped_divide(tv_tensor, thr_tile);            // ((ThrV,(ThrM,ThrN)),(FrgV,(RestM,
    RestN)))

    return thr_tensor;
  }
```

在明白 tiled mma partition 的原理之后，我们对于一个 tensor partition 前后的 layout 是多少，相信也是非常自然的了。

至于为什么先并行再串行的循环，我想是因为 thread 的连续排布肯定是对数据加载更友好的，虽然单 warp 做到连续排布就已经能够合并访存了，但是 threadblock 级别的连续排布可以更进一步提高 cache 属性。而且，考虑到 volta 架构是以 1/4-warp 为执行器的，为了保证一个 warp 访问的数据连续，先并行执行器排布明显是更优的选择。

最后，值得注意的是，tiled mma 的大小实际上可以不和 g2s/s2g tiled copy 一致，例如，我们定义一个 64x64x16 tiled mma，实际对应的 tiled copy 规格可能是 128x128x16 的，后续我们需要再在两个方向上进行循环。之所以这样设计，是因为在 mma 问题中寄存器通常是受限的稀缺资源，通过这种设计可以让我们更自由的控制寄存器的行为，例如，

我们可以计算一部分 C（例如 64x64），然后先输出到 shm 中，然后再复用 64x64 的寄存器来计算下一块 C，从而大大降低寄存器的用量，进而把 tile size 开到更大。而且数据的写出以及 mma 的计算也可以并行起来。更特别的，tiled mma 定义时其实并不考虑 tileM/tileN/tileK，而是只考虑一个 threadblock 能处理的 mma，也就是我们例子中的 32x32x16，其他维度其实是在 tensor partition 过程中用 divide 得到的。

15.9 总结

我们在本文中梳理了 tiled mma 的基础用法，包括 make_tiled_mma 构造 tiled mma 对象，以及 partition 对所需数据进行 thread 与 value 映射。我们对其实现原理也进行了探索，我们发现其核心仍然是我们在 tiled copy 一文中提到的 TV-layout。进一步的，tiled mma 提供的 TV-layout 可以被转换为 s2r / r2s copy 所需的 TV-layout，因而可以由 tiled mma 延伸出 s2r / r2s tiled copy。

至此，我们认为，CuTe 的两大部分 tiled mma 和 tiled copy 都梳理完毕了，在此基础上完成一个高效 gemm 已经不是太难的事情，需要做的只是配合 tileK for 循环，并且排布好 load-compute-store 的 pipeline 即可。具体实现推荐 @reed 大师的这篇文章 cute 之高效 GEMM 实现，以及其给出的高效 gemm 实现 https://github.com/reed-lau/cute-gemm，有兴趣的读者可以深入研究。

最后，我们逐渐意识到，CuTe 的所有操作，就是围绕 TV-layout 在进行各种各样的变换，包括 inverse，product，divide，composite 等。因此，如果想要更进一步的从原理上理解 CuTe 的设计哲学（例如，我们想知道，为什么我们将 T-layout & V-layout 进行一个 raked_product 就可以得到 TV-layout，为什么 TV-layout inverse 之后就可以变换成 MN-layout，etc），对其 layout 代数逻辑的理解是无可避免，接下来的文章里，我们来尝试探索一下这些代数逻辑是怎么一回事，以及其设计动机是什么。

第十六章　写给大家看的 CuTe 教程：Layout Compose & Inverse

作者：竹熙佳处（知乎专栏「CuTe 教程」）　原文：https://zhuanlan.zhihu.com/p/1962625273636845008

（全文经浏览器取回，作者保留所有权利；本书仅作学习引用与写作风格参照）

16.1　动机

在本专栏此前的两篇文章中，我们梳理了 CuTe 中的两大模块：tiled copy 和 tiled mma 以及对应的 thread-value 的映射关系。

以这两大模块为基础，实际上就可以完成调用 tensor core 来完成矩阵计算的流程了。然而我们知道，只会简单的使用这些函数并无法做到自定义高性能 CuTe kernel。

例如，在 flash atten v2/v3 中，Q @ K.t 的结果 S 矩阵，首先需要做一个 online softmax 得到 P，然后以 P 为输入完成 P @ V。其中可以发现，P 的数据分布是按照 mma-C 矩阵来排列的，而紧接着的 P@V 又要求其作为 mma 的输入按照 mma-A 矩阵的数据分布来排列。这就需要我们对 layout 进行变换，其目的是为了让我们的数据按照我们希望的形式进行排列，以适应不同的接口要求（cute::gemm）。

另一个例子是，w4a8 混合精度 gemm，由于 load 数据 (int4) 与实际计算所需的计算 int8/fp8 数据类型不一致，load 数据之后的 layout 也需要变换；更多的例子还包含，2:4 稀疏 gemm，implict im2col gemm 等等，也都需要对 layout 变换有一定认知才能实现。

更细节的，CuTe 的所有上层接口如 make_tiled_copy/mma，partition，retile 等等逻辑，都来自 layout 的变换，了解 layout 变换的逻辑，不仅有助于我们使用 CuTe 的时候更得心应手，实际也具备将这样一套代数系统迁移到任意类 gpu 芯片上的潜力，例如 @ 摇光 的这篇文章尝试用 CuTe 为 AMD GPU 增加了一个 copy traits & op。

进一步的，国产芯片有可能基于这套体系来开发一个类 Cutlass 模版库吗？我认为也是有可能的，实际上我们观察 NV 每一代 GPU 的演进，从 hopper 的 wgmma 到 blackwell 的 umma，实际上也能发现，其趋势就是将 tensor core 和 cuda core 更多作为独立且异步的单元来使用，这个设计其实和目前很多国产芯片的 NPU + vector core 策略是趋同的。

而 NV 的每一代芯片在 CuTe 这套表达体系下依旧能适配的很好，颇有一种一统天下包罗万象的气势。那么对于国产芯片，其能适配到什么程度呢？或许只有在我们更深入理解 CuTe layout 代数的原理之后，才能回答这个问题。

那么，是时候来探索一下 CuTe 体系中最神秘的 layout 代数了。虽然我们把 layout 说的如此重要，但是相信我，正如很多宏观物理现象最终可以被表述为牛顿三公式一样，cutlass 庞大的代码规模下，其蕴含的 layout 代数，只需要高中级别的数学知识即可理解。

16.2　Layout 的定义 & 作用

NV GPU 体系下，gemm kernel 常常需要对 Tile 数据进行读写，我们访问数据时，实际上是通过 base ptr 加一个位置偏移来获得的。因此最基础版本的 layout，实际上就是为了解决逻辑上的 (x, y) 坐标转变为一个物理的指针的偏移位置这样一个问题。

带着这样的目的，我们给出 Layout 的定义：Layout 由 shape 和 stride 构成，给定一个输入坐标 (x, y)，可以获得一个 1D 的数值，我们记作 offset。Layout 本质上可以理解为一个函数，这个函数的计算只是整数的乘法和加法，所需要的参数就是我们给定的 shape 和 stride，即，计算指针偏移 offset 的过程，即是坐标和我们定义 layout 给定的 stride 的对应位置相乘并相加。

为不失一般性，CuTe 中将 base_ptr 这类可以通过加偏移来访问数据的实体称为 Engine，类似 c++ 中 iterator。Tensor / Engine / Layout 的关系以及作用，如 Fig1. 所示。

由于 layout 是一个函数，那么我们参考高中数学中函数的一些基础操作，就能理解 layout 代数实际上是在干什么了。

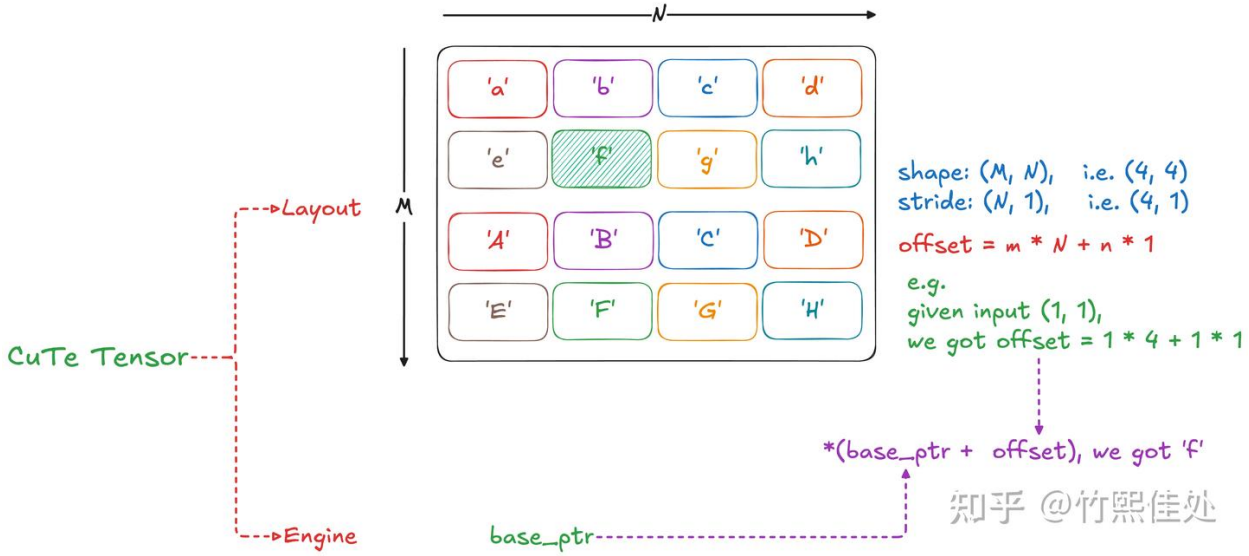


图 16.1: Figure1. 一个 2 维 Tensor 的例子，其由一个 base_ptr (i.e. engine) 和一个 layout 组成。layout 本质上是一个以 2 维坐标（m, n）为输入，1 维 offset 为输出的一个函数，这个 offset 配合 base_ptr 得到我们实际想要访问的数据

16.3 Layout 的复合 compose

我们回顾一下函数的复合的定义：给定一个函数，y = f(x) ，再给定一个函数 z = g(y) ，那么 g 复合 f 可以理解为两次连续的映射，(x <-> y <-> z)，即，z = g(f(x)) ，f 为内层函数。

同样的，对于 layout 来说，给定 layout A：(m, n) <-> offset_1，layout B: (p, q) <-> offset_0，其复合过程也就可以理解为连续两次映射。我们记作：

layoutC = compose(layoutA, layoutB) = layoutA(layoutB) 。

那么给定一个内层函数 layoutB 的输入 (p, q)，我们来看看如何得到 compose 后 layoutC 对应的输出。首先我们知道，offset_0 = p * s_p + q * s_q，(其中 s_p 、s_q 为 layout B 的 stride)，这一步计算是平凡的，接下来就是 offset_0 作为输入传给 layout A，但是 layout A 的输入需要是 2D 的 (m, n)，怎么办?

事实上，很多初学者会在这里感到困惑，就是没注意到一个 CuTe 中有点反直觉的关键信息：即，对于任意维度的 layout 来说，其输入总是可以兼容 1D 坐标的，这个 1D 坐标会根据这个 layout 的 shape，按照 col-major 的方式来转换为多维坐标，并且这个转换和 stride 无关。

以 layoutA 为例，其 shape 为 (M, N)：

1D 坐标 offset_0 可通过如下公式转换为 2D 坐标：m = offset_0 % M，n = offset_0 / M ；再将转换后的 (m, n) 传入 layoutA，计算得到 offset_1 = m * s_m + n * s_n（其中 s_m 、s_n 为 layoutA 的 stride)。

整个 compose 的计算过程可以展示为 Fig.2。

我们打印出 layoutA/B/C，可以验证 layoutC 与我们构造的结果一致。

```
auto layoutA = make_layout(make_shape(4, 4), make_stride(4, 1));
auto layoutB = make_layout(make_shape(8, 2), make_stride(2, 1));
auto layoutC = composition(layoutA, layoutB);

if (cute::thread0()) {
  print_latex(layoutA); print("\n"); // % Layout: (4,4):(4,1)
  print_latex(layoutB); print("\n"); // % Layout: (8,2):(2,1)
  print_latex(layoutC); print("\n"); // % Layout: ((2,4),(2,1)):((8,1),(4,1))
}
```

在理解 compose 的语义即是连续映射后，我们可以进一步观察到，因为 layoutC 的输入来源于 layoutB，因此其最后产生的输出个数一定与 layoutB 是一致的，因此我们可以理解为 layoutC 的 shape 在形式上与 layoutB 的 shape 相同；但是其 stride 会变得比较奇怪，甚至需要通过拆分 shape 中的维度来匹配 stride。例如，上述变换后的 layoutC，实际上是需要写成 ((2,4),(2,1)):((8,1),(4,1)) 的。

Layout compose 在很多地方都可以用到。例如，我们在之前的 tiled copy 一文中提到的 TV-layout：给定 (t-id, v-id)

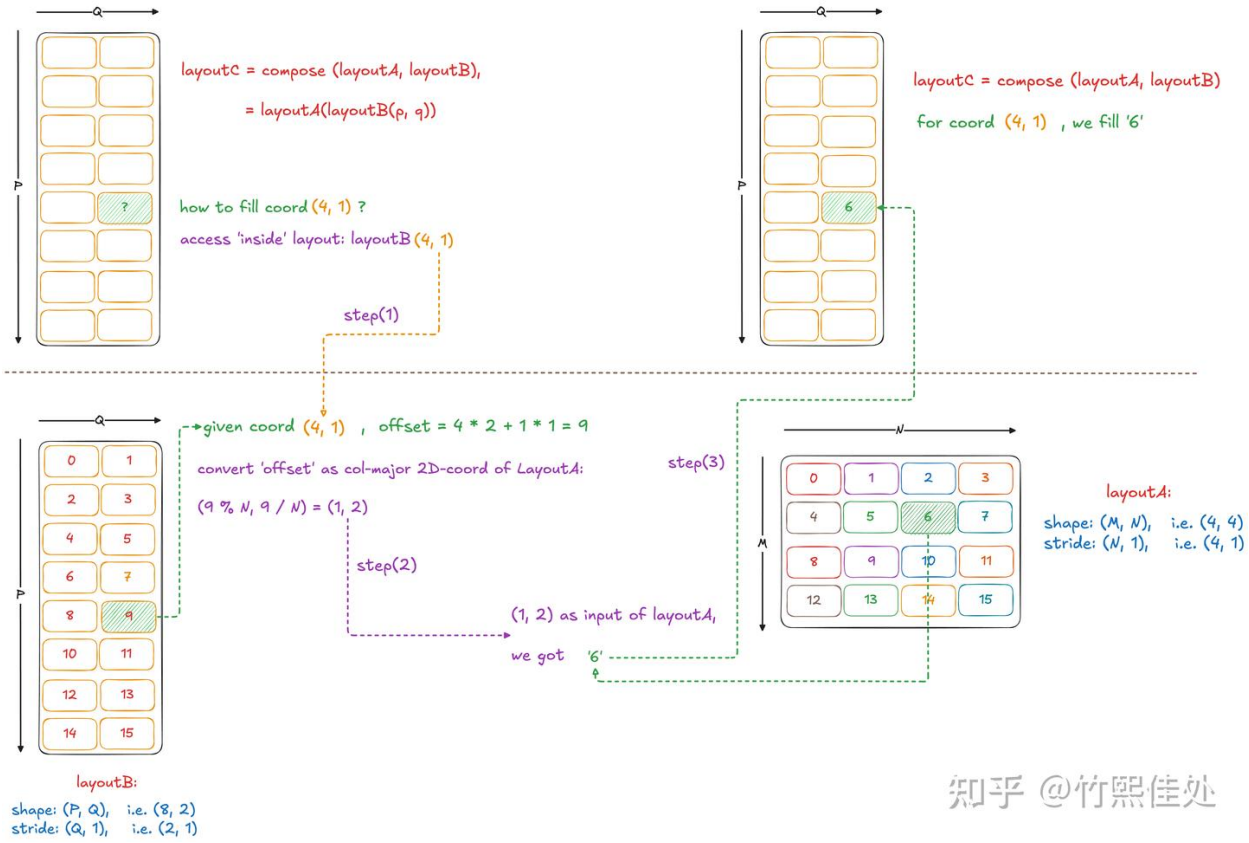


图 16.2: Figure2. layoutC = layoutA(layoutB) 的计算过程。对该式子中的内层 layoutB 的一个输入 (4, 1) 为例，首先计算出通过 layoutB 的 offset0 = 9；然后将其按照 col-major 方式转换为 2D 坐标，再输入到 layoutA 中得到 offset1 = 6，即是 layoutC(4, 1) 所需要的输出

得到 (m-id, n-id)，这个 (m-id, n-id) 只是逻辑上的坐标，我们想要真实拿到实际能访问的物理地址，必须给其引入一个 tensor 本身的 layout，完成 (m-id, n-id) <-> offset = m-id * s_m + n-id * s_n 的转换。

对 tensor 来说，其 layout 代表的是 (m, n) <-> global offset 的映射。因此，可以通过一个 compose(tensor.layout(), tv-layout) 来让每个 thread 找到自己需要的数据，其中蕴含的映射关系是 (t, v) <-> (m, n) <-> global/shared offset。

这样的设计，既可以让我们变换 layout 时只关注坐标空间，又可以在实际使用中，通过 compose 不同的 tensor layout，让同一个 (m-id, n-id) 可以正确访问 global / shared / reg tensor 中的数值。正如我们可以脱离实际问题来分析函数，又可以在计算完成后把数字应用回现实世界。

另一个例子就是为了避免 g2s copy 时 bank conflict 的 swizzle 的 compose。在朴素 cuda 实现中，我们引入 swizzle 逻辑时，需要关注我们读到的 global data 放到 shared data 哪个位置。但是引入了 layout compose，我们就可以做到对 swizzle 后的 tensor 仍然用正常逻辑下的 (m-id, n-id) 的坐标体系去访问到正确数据，虽然实际的物理地址已经改变了，但是我们不再 care 了。

具体来说，swizzle 代表的是一个 offset <-> offset' 的一个映射，我们对一个 shm tensor 做 swizzle，蕴含的映射是：(m, n) <-> offset <-> offset' ，因此能看到我们总是 swizzle 作为 A-layout 去做 compose 的，如下所示：

```
    static constexpr int kSwizzleB = 3;
    static constexpr int kSwizzleM = 3;
    static constexpr int kSwizzleS = 3;
    using SmemLayoutAtomC = decltype(composition(Swizzle<kSwizzleB, kSwizzleM, kSwizzleS>{},
                                    make_layout(make_shape(Int<8>{}, Int<64>{}), make_stride(Int<64>{},
    Int<1>{})));
```

当然定义 swizzle 的 BMS 3 个参数怎么填，每个参数代表的是什么含义，又是需要相当篇幅来解释了，我们在此先推荐一篇介绍其基本原理，并给出了相关代码可以完成自动推导的文章：布局代数实战：Swizzle 自动推导 (@melonedo)，后续我们找时间来详细梳理。

16.4 Layout 的逆 inverse

我们回顾一下函数的逆的定义：给定一个函数，y = f(x)，其逆函数 g 可以做到给出 f 的输出 y 得到其输入 x，即 x = g(y)。对于 layout 来说，inverse 操作可以完成输入和输出的调换。

因此，给定 layout A：(m, n) <-> offset_1，其 inverse 过程即是给定 offset_1 可以得到 (m, n)。又因为 offset_1 可以 col-major 转换为新的多维坐标，实际上就可以做到 offset，view as (p, q) 的操作，进而构建出 offset <-> (p,q) 这样一个新的映射。当然这个 view as (p, q) 的过程，需要进行一次 with_shape 操作，本质上就是一次 compose。整个过程如 Fig.3 所示。

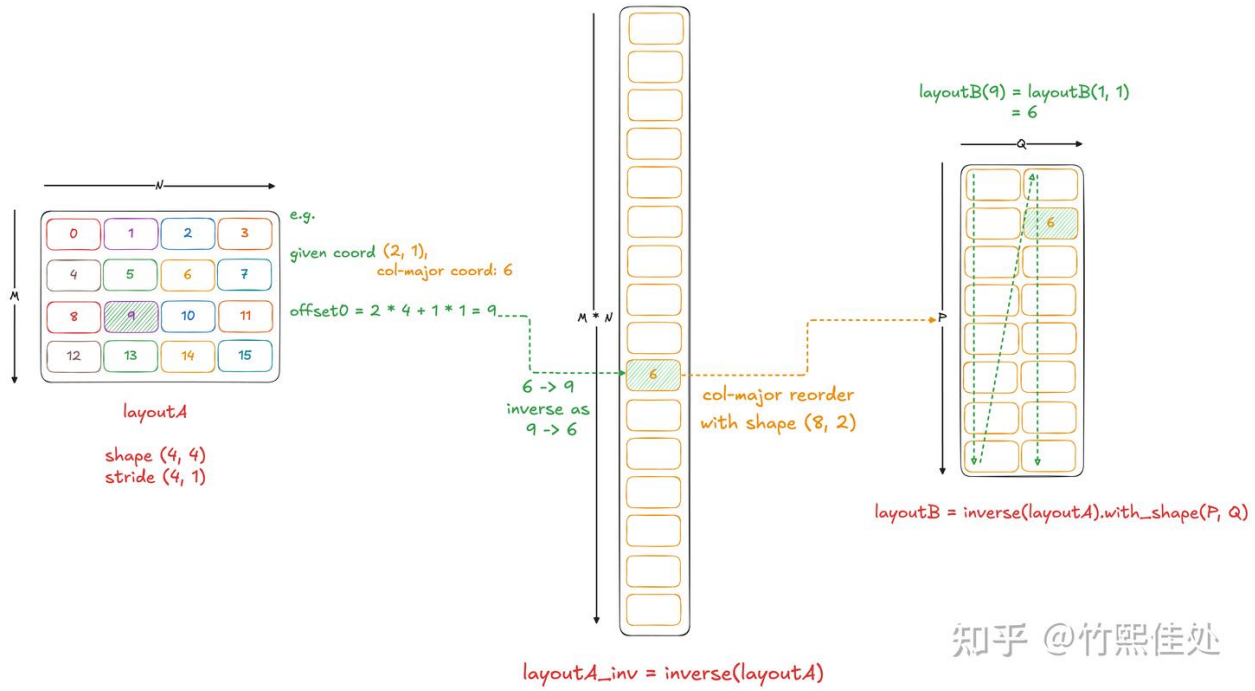


图 16.3: Figure3. layoutB = left_inverse(layoutA).with_shape 的计算过程。给定 layoutA 中的一个输入 (2, 1)，其 col-major 1D 坐标为 6；我们首先通过 layoutA 得到 1D offset = 9，然后将输入输出调换为 9 to 6；然后引入 with_shape 将其排列成 layoutB 的 shape

我们同样可以通过如下代码验证：

```
auto layoutA = make_layout(make_shape(_4{}, _4{}), make_stride(_4{}, _1{}));
auto layoutA_inv = left_inverse(layoutA);
auto layoutA_inv_with_shape = layoutA_inv.with_shape(make_shape(_8{}, _2{}));

if (cute::thread0()) {
  print_latex(layoutA); print("\n");
  print_latex(layoutA_inv); print("\n");
  print_latex(layoutA_inv_with_shape); print("\n");
}
```

另外，我们观察到 inverse 操作总是会带一个 with_shape 这样的操作，只单纯做 inverse 其实也能得到一个 layout (即，我们例子中的 layoutA_inv)，但是这个 layout 的 shape 其实并不重要，我们只需要理解成原始 layout 输入和输出的调换组成的一组集合即可。

通常 inverse 后面会有一个进一步变换的步骤，才能让 inverse 的 layout 成为一个有语义的 layout。带上 with_shape，则可以让这列数字按照 col-major 排列成我们指定的 shape。事实上，如果我们查看 CuTe 中 with_shape 函数的代码，也能看到 with_shape(P, Q) 就是将 inverse 后的 layout，进一步 compose 一个 (P, Q) : (1, P) layout。

```
template <class OtherShape>
CUTE_HOST_DEVICE constexpr
auto
with_shape(OtherShape const& shape) const {
  return composition(*this, make_layout(shape));
```

```
    }
```

除了 with_shape，我们也可以 compose 其他 layout 来完成具备语义的各种操作，我们在实战章节中跟随例子进行解读。

了解清楚 inverse 语义后，我们来看 CuTe 的具体函数。CuTe 对 inverse 提供了 left_inverse & right_inverse 两个函数。二者在给定的 layout 为一一映射且连续的场景下（即，每一个输入都对应一个不同的输出，且输出空间是连续的），left_inverse 等价于 right_inverse。事实上，CuTe 代码中用到 inverse 的场景绝大多数都是这样一一映射且连续的，因此大部分情况下 right_inverse 和 left_inverse 可以相互替换。

那么如果我们给定的 layout 不是一一映射，即，多个输入都对应一个相同的输出时（但要求输出空间连续），如，broadcast 访问场景，其 inverse 其实是会丢失一部分信息的。虽然通过 left_inverse & right_inverse 也能得到一个 layout，但是其只能说保留了一部分信息，如 Fig.4 (a) 所示，inv_layout 并不能保留 layout 的所有输入。

如果我们给定的 layout 是一一映射，但是输出空间不连续，即，每个输入都对应一个不同的输出，但输出空间不连续，如，跨步访问场景，这时候 right_inverse 只会把第一段连续输出的 layout 进行一次 inverse，left_inverse 则保留了完整的原始 layout 的输出到输入的映射，所以一般来说 left inverse 会保留更多的信息。

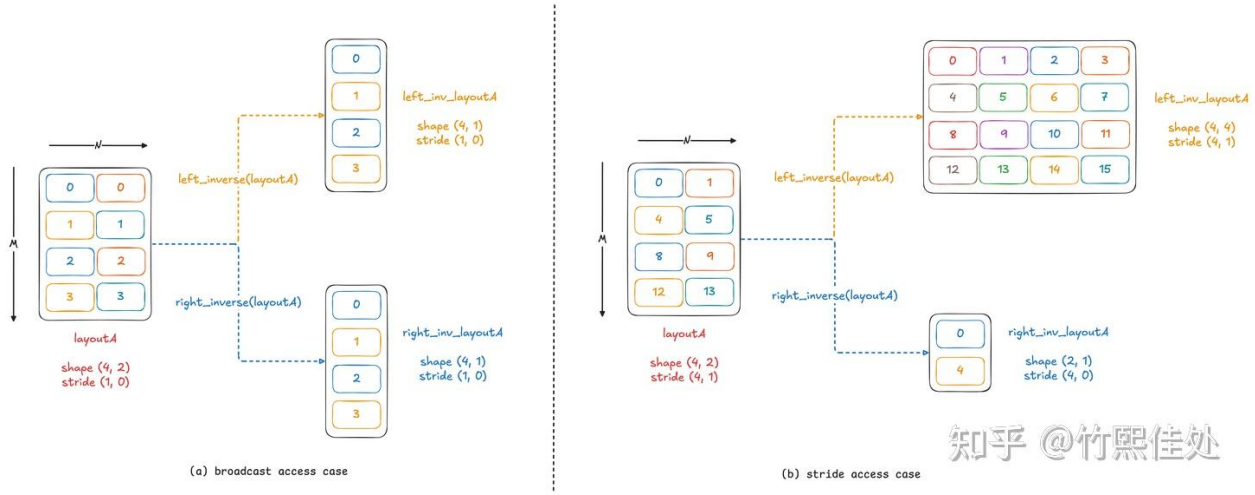


图 16.4: Figure4. broadcast access 以及 stride access 下，left_inverse & right_inverse 的输出对比。可知，在 broadcast 场景下，二者结果一致但会丢失一部分原 layout 的映射关系；stride 场景下，left_inverse 构造出了原 layout 的整个陪域的映射关系，right_inverse 只构造了第一段连续值域的映射关系

总结来说，left inverse 是对 layout 的所有潜在输出（即，陪域）做 inverse，right inverse 则是在 layout 实际的“一部分”输出做 inverse，且这个“一部分”是实际输出空间（即，值域）中连续的一段。

当我们想要对一个非紧凑或非连续的 layout 进行 inverse 操作时，需要想清楚目的是什么，再选择是用 left 还是 right。另外需要强调一点，当我们给的 layout stride 不是编译期常量时，left_inverse 会报错，而 right_inverse 则会跳过这些非编译期常量的维度来构造 inverse layout，笔者暂时没有想清楚为什么要这样设计，有了解的朋友欢迎在评论区讨论。

虽然我们在本文开头说，只需要高中级别的数学知识就能读懂本篇文章，但是考虑到有些读者可能想要进一步挖掘 CuTe 之下的数学原理，我们也简单提一下：left inverse 和 right inverse 的概念其实来源于集合论，并且可以证明在两个集合满足双射的前提下，left inverse 等价于 right inverse；当集合仅满足单射时（即，一一映射，但是并不是每个输出都能找到输入），只存在 left inverse；而仅满足满射时（即，每个输出都必然能找到一个输入，但有可能多个输入对应一个输出），只存在 right inverse。

而求一个集合的逆的过程，可以通过 two-line notation <-> swap two-line <-> 排序的方法来完成，感兴趣的同学可以参考国外的代数笔记。@reed 大师在其对 layout 代数的介绍中也简单提及了这个方法。事实上，我们观察 CuTe 实现 inverse 函数的过程中就用到了排序，原因就在此。

```
    template <class Shape, class Stride>
    CUTE_HOST_DEVICE constexpr
    auto
    left_inverse(Layout<Shape,Stride> const& layout)
    {
```

```
    // Flatten and filter shape-1
    auto clayout = coalesce(layout);
    auto lstride = wrap(clayout.stride());
    auto lshape  = wrap(clayout.shape());

    // Prefix product of the shape
    auto preprod_shape = cute::fold(lshape, cute::tuple<_1>{}, [](auto c, auto vi) { return append(c, vi
*back(c)); });

    // Sort by strides
    static_assert(is_static<decltype(lstride)>::value, "Left inverse requires static strides.");
    using Sorted = detail::SortByKey<decltype(lstride), tuple_seq<decltype(lstride)>>;
    auto sorted_seq = typename Sorted::val_type{};

    // ...
  }
```

16.5 Layout compose & Inverse 实战

在笔者刚开始接触 CuTe 时，常常会困惑于一件事：这些 layout 代数究竟有什么用？大家对于 CuTe 官方文档吐槽也主要因为其完全没有前后联系的抛出一堆概念和代数运算，然后指望用户自己从庞大的代码库中理解其用法，人为制造出一条陡峭到天上的学习曲线。

作为一篇写给大家看的教程，我们不急于输出更多的概念，先结合 Flash atten v2 中 A-layout 到 C-layout 转换这个例子，来感受一下 layout compose & inverse 的用法和其蕴含的潜力。

我们知道在 Flash atten v2 的实现中，核心是 P = Q @ K.T 以及 O = P @ V 两个矩阵连乘（为简化说明，我们先忽略 softmax 的运算），在这个过程中 P 即是 QK 矩阵乘的输出，此时其符合 mma accumulator tensor 的 layout；也是 PV 矩阵乘的输入，此时其又需要符合 mma A tensor 的 layout。我们知道，tiled mma 的 layout 要求 A 和 C tensor 的 layout 是不同的，直接将 C 作为 A tensor 输入到 cute::gemm 函数中会编译报错。那么需要怎么对 C 进行转换呢？

我们从 C = A @ B.T 这样一个简单的例子，延伸出一个简化的矩阵连乘来说明。即，我们想完成 C = A @ B.T，D = C @ B.T 这样一个计算：

```
  template <typename T, int kTileM, int kTileN, int kTileK, typename TiledMMA>
  __global__ void simple_kernel(T *Cptr, const T *Aptr, const T *Bptr, int m,
                                int n, int k) {
    auto A = make_tensor(make_gmem_ptr(Aptr), make_shape(m, k),
                         make_stride(k, Int<1>{}));
    auto B = make_tensor(make_gmem_ptr(Bptr), make_shape(n, k),
                         make_stride(k, Int<1>{}));
    auto C = make_tensor(make_gmem_ptr(Cptr), make_shape(m, n),
                         make_stride(n, Int<1>{}));

    int ix = blockIdx.x;
    int iy = blockIdx.y;

    auto gA = local_tile(A, make_tile(Int<kTileM>{}, Int<kTileK>{}), make_coord(iy, _));
    auto gB = local_tile(B, make_tile(Int<kTileN>{}, Int<kTileK>{}), make_coord(ix, _));
    auto gC = local_tile(C, make_tile(Int<kTileM>{}, Int<kTileN>{}), make_coord(iy, ix));
    //  gA(kTileM, kTileK, num_tile_k)
    //  gB(kTileN, kTileK, num_tile_k)
    //  gC(kTileM, kTileN)

    TiledMMA tiled_mma;
    auto thr_mma = tiled_mma.get_slice(threadIdx.x);
    auto tgA_g2r = thr_mma.partition_A(gA); // (MMA, MMA_M, MMA_K, num_tile_k)
    auto tgB_g2r = thr_mma.partition_B(gB); // (MMA, MMA_N, MMA_K, num_tile_k)
    auto tgC_g2r = thr_mma.partition_C(gC); // (MMA, MMA_M, MMA_N)

```

```
    auto trA_mma = thr_mma.partition_fragment_A(gA(_, _, 0)); // (MMA, MMA_M, MMA_K)
    auto trB_mma = thr_mma.partition_fragment_B(gB(_, _, 0)); // (MMA, MMA_N, MMA_K)
    auto trC_mma = thr_mma.partition_fragment_C(gC(_, _));    // (MMA, MMA_M, MMA_N)
    auto trD_mma = thr_mma.partition_fragment_C(gC(_, _));    // (MMA, MMA_M, MMA_N)

    clear(trC_mma);
    clear(trD_mma);

    int num_tile_k = size<2>(gA);
#pragma unroll 1
    for (int itile = 0; itile < num_tile_k; ++itile) {
      cute::copy(tgA_g2r(_, _, _, itile), trA_mma);
      cute::copy(tgB_g2r(_, _, _, itile), trB_mma);

      // first we compute C = A @ B.T, it's fine
      cute::gemm(tiled_mma, trC_mma, trA_mma, trB_mma, trC_mma);

      auto trC_as_A_mma = trC_mma;

      // TODO: here we need to convert trC as trC_as_A,
      // because trC_mma has shape of ((2, 2), MMA_M, MMA_N)
      // while trA_mma needs shape of ((2, 2, 2), MMA_M, MMA_N / 2)
      // how to convert layout?

      // therefore, we got compile error: layout not acceptable for cute::gemm
      cute::gemm(tiled_mma, trD_mma, trC_as_A_mma, trB_mma, trD_mma);
    }
  }
```

以 16x8x16 fp16 mma atom 为例，trC 作为 accumulator layout 为 ((2, 2), MMA_M, MMA_N)，而下一次 gemm 所需要其作为 A-tensor，需要其 layout 为 ((2, 2, 2), MMA_M, MMA_N / 2)。

二者的数据本身是完全一致的，因此我们需要对 trC layout 进行一次变换。熟悉 flash atten v2 的读者可能已经发现，Tri-dao 在其实现中使用了 logical_divide 对 layout 在几何层面上进行了一次重排，我们将其过程拆解出来如下：

```
      // tri-dao method: logic-divide
      auto acc_layout_div = logical_divide(
          trC_mma.layout(),
          Shape<Underscore, Underscore, _2>{}); // ((2, 2), MMA_M, (2, MMA_N / 2)))
      auto a_layout = make_layout(
          make_layout(get<0>(acc_layout_div), get<2, 0>(acc_layout_div)),
          get<1>(acc_layout_div), get<2, 1>(acc_layout_div));
      // (((2, 2), 2), MMA_M, MMA_N / 2)
      auto trC_as_A_mma = make_tensor(trC_mma.data(), a_layout);

      // result correct
      cute::gemm(tiled_mma, trD_mma, trC_as_A_mma, trB_mma, trD_mma);
```

然而，reed 大师早已看穿了一切。

更优雅的，我们可以利用 inverse + compose 来从代数变换的角度上完成这个变换。其原理为：

我们已知 trC layout 是一个 (x_acc, y_acc) <-> offset_0 的映射，且其是一个连续的一一映射，因此我们可以对其求 inverse（right & left 均可），得到 offset_0 <-> (x_acc, y_acc) 的映射；

然后，我们 compose 一个 (x_a, y_a) <-> offset_0 的映射，即，trA layout，则可以得到 (x_a, y_a) <-> offset_0 <-> (x_acc, y_acc) 的映射；

最后，因为我们的目标是得到 (x_a, y_a) 在 trC 空间中的 offset_1，因此我们还需要 compose 一个 trC_mma tensor 的 layout，完成 (x_a, y_a) <-> offset_0 <-> (x_acc, y_acc) <-> offset_1 变换，整个过程我们展示为如下代码：

```
      // reed method: layout-algebra
      auto C_tensor_for_partition = make_tensor_like(gC(_, _));
```

```
        auto trC_as_A_layout = thr_mma.partition_A(C_tensor_for_partition).layout();
        auto trC_as_C_layout = thr_mma.partition_C(C_tensor_for_partition).layout();

        auto acc_layout_inv = left_inverse(trC_as_C_layout);
        // (x_acc, y_acc) -> offset0 => (sorted) offset0 ->  (x_acc, y_acc)

        auto a_layout_algebra = acc_layout_inv.compose(trC_as_A_layout);
        // (x_a, y_a) -> offset0 -> (x_acc, y_acc)

        // trC: (x_acc, y_acc) -> offset1, compose as B
        auto trC_as_A_mma = trC_mma.compose(a_layout_algebra);
        // (x_a, y_a) -> offset0 -> (x_acc, y_acc) -> offset1

        // result correct
        cute::gemm(tiled_mma, trD_mma, trC_as_A_mma, trB_mma, trD_mma);
```

我们建议感兴趣的读者可以尝试跑通这个例子，并验证一下 tri-dao 和 reed 两者方法的结果，并尝试在更多的 mma 规格上测试，来感受 layout 代数所带来的语义上的便利性。

16.6 结论

我们在本文中梳理了 CuTe layout 代数中 compose 以及 inverse 的计算过程以及用法，其基本思想是将 layout 视为一种函数，因此我们可以借助函数中的 compose & inverse 的逻辑来理解 CuTe 的代数体系。

进一步的，我们结合 flash atten v2 中 A-C-layout 转换例子，也能看到利用代数方法能让我们完成一些自定义 layout 变换，真正做到随心所欲写 CuTe，而不止于调用 cutlass 现成的功能。

layout 代数在 cutlass 官方文档中一直是一个让人十分困惑的存在，即使有 reed 大师的指导，笔者在构思这篇文章的过程中，依旧经历了多轮反复的修正自己认知的过程，所谓温故知新的乐趣就是如此吧，我们将这个思考的过程分享出来，希望帮助更多的同学掌握 CuTe。

接下来的文章我们将继续探讨 layout 代数中的 product 和 divide 逻辑，之所以将其分在不同的章节中，是因为我们认为 product 和 divide 无法从函数的角度推导，而是更倾向于一种几何形式的构造。

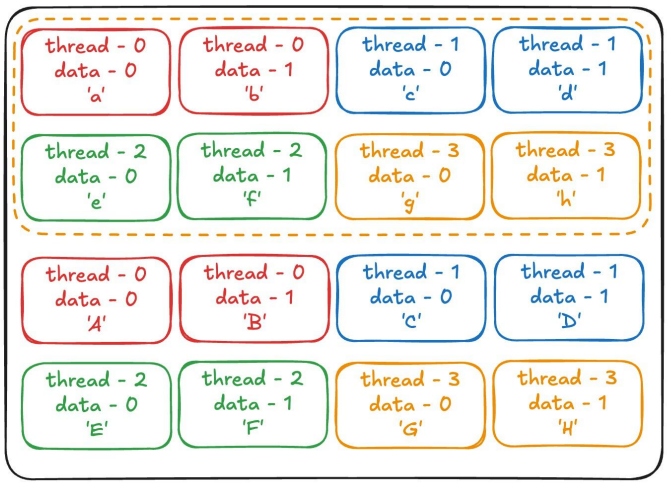
第十七章　写给大家看的 CuTe 教程：Layout Product & Divide

作者：竹熙佳处（知乎专栏「CuTe 教程」）　原文：https://zhuanlan.zhihu.com/p/1971945267294111573

17.1　动机

在之前的文章中，我们梳理了 Layout 的两种代数方法：**Compose** 与 **Inverse** [1]。本文我们将继续介绍 Layout 代数中另外两个重要的代数变换：**Product** 与 **Divide**。

在介绍这两个操作之前，我们先思考一下 CuTe 引入 Product 和 Divide 的目的是什么。在介绍 Tiled Copy 的文章 [2] 中，我们提到了需要构建 Thread 与 Data 之间的映射关系。我们以 4 个 Thread 组成的 thread block 拷贝 4×4 数据块为例，如 Fig.1 所示：



知乎 @竹熙佳处

图 17.1: Figure.1 tiled copy 中 thread 与 data 的映射关系。以 4 thread copy 4x4 data 为例，每个 thread 大字长访存，单次 copy 2 个 data，并且在行方向上重复 2 次。

我们编写 CUDA Kernel 来完成 tiled copy 过程的过程，本质上是在确定一个 Thread Block 每次所访问的数据范围，然后通过重复扩展，直至不重不漏的覆盖整个数据块。

也就是说，我们实际上是将“单个 Thread Block 访问行为”按照某种“模式”进行重复。读过本专栏其他文章的读者会发现，这种“重复”的行为在我们介绍 Tiled Copy 和 Tiled MMA[3] 中曾反复出现，为此我们提到过一个 CuTe 中的重要观点：**Atom 之外，皆是重复**。

现在我们将“模式”与“重复”的理念引入到 CuTe Layout 中，就可以发现：我们可以指定一个小 Layout 作为一种模式，将这个模式**重复**多次可以形成更大的 Layout，这就是今天要介绍的 **Layout Product**。

除了将 Layout 扩大，自然的，我们也需要具备从一个大 Layout 中按照某种模式抽取出一个小 Layout 的能力。例如，在访问大数据块时，我们需要让每个 Thread Block 知道自己每一轮要访问的数据。也就是说，我们指定一个小 Layout 作为模式，将大 Layout **拆解**成若干部分，这即 **Layout Divide**。

我们观察 CuTe 上层的函数，如 partition / local_tile / tile_to_shape 等，其实都是 product 以及 divide 的变体。更进一步，make tiled copy / mma 等核心函数中，product & divide 也是被反复用到，并配合我们在上一篇文章中提及的 compose & inverse，构成了 CuTe 体系中千变万化的代数变换。

值得一提的是，如果说 compose 和 inverse 还可以借助数学中的函数复合以及逆函数的概念进行理解，product 和 divide 的思想则更是总结为一个图形上的理解，因此接下来我们更多借助图像的方式，尽可能形象的来说明这两个 CuTe 中最难说明白的代数变换。

17.2 Layout 的乘法 Product

Product 的目的是将一个 layout 扩大。给定：
LayoutC = product(LayoutA, LayoutB)
其计算过程可以描述为：

- 将 A 补全为一个连续紧密的 A′；
- 参考 A 的“模式”，从 A′ 中选择出一系列 layout 集合，我们将这些集合记为 **A-like Layouts**。
- 按照 **Col-Major 方式**访问 B，得到一个 offset 的序列，我们按照 B 的 offset 定义的顺序，将 A-like Layouts 按照一定的规律排列在一起，形成最终的 C。

这里的“一定的规律”，则是 CuTe 中定义的 logical_product / blocked_product / raked_product / zipped_product / tile_product 等 Product 函数具体决定的。

例如，如果把每个 A-like Layouts 视为 C 的一列，按顺序排列在一起，那么就是 CuTe 中的 **logical_product**。

我们以图形的方式解释 logical_product 的过程，如 Fig.2 所示：

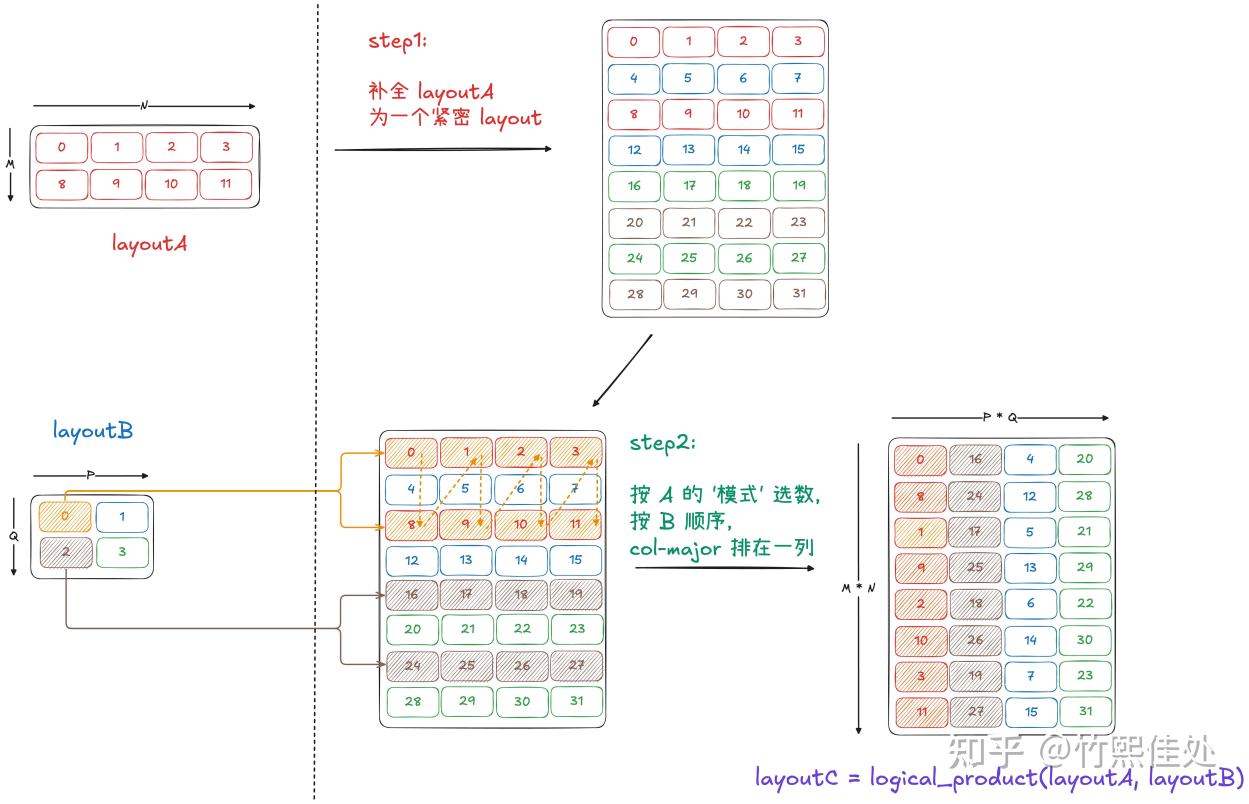


图 17.2: Figure.2 logical_product 计算过程：首先将 A 补全到紧密 layout，再从紧密 layout 中找出 A-like layouts，最后按照 B 的 col-major 顺序取 A-like layouts，按列排在一起。

其他 Product 变体函数的基本语义与 logical_product 是一致的，各个函数的区别只是在最后形成 C 时将 shape 以及 stride 维度给调换了。例如，除了 logical_product，在 cutlass 中比较常见的 product 函数还有：

- **blocked_product**：其填充 A 并提取 A-like Layouts 的过程和 logical_product 的过程一致。最后构成 C 时，不再将 A-like Layouts 排成一列，而是尝试把 *A*-like Layouts **作为一个“block”聚集在一起**，这个过程如 Fig.3 所示。事实上，这种排列方式也和我们直觉上的 layout 重复更契合，所以其在 cutlass 中被广泛使用。

- **raked_product**：其过程与 blocked_product 类似，只是在最后按“block”聚集的时候，是优先排成类似 B 的形式。如 Fig.4 所示。

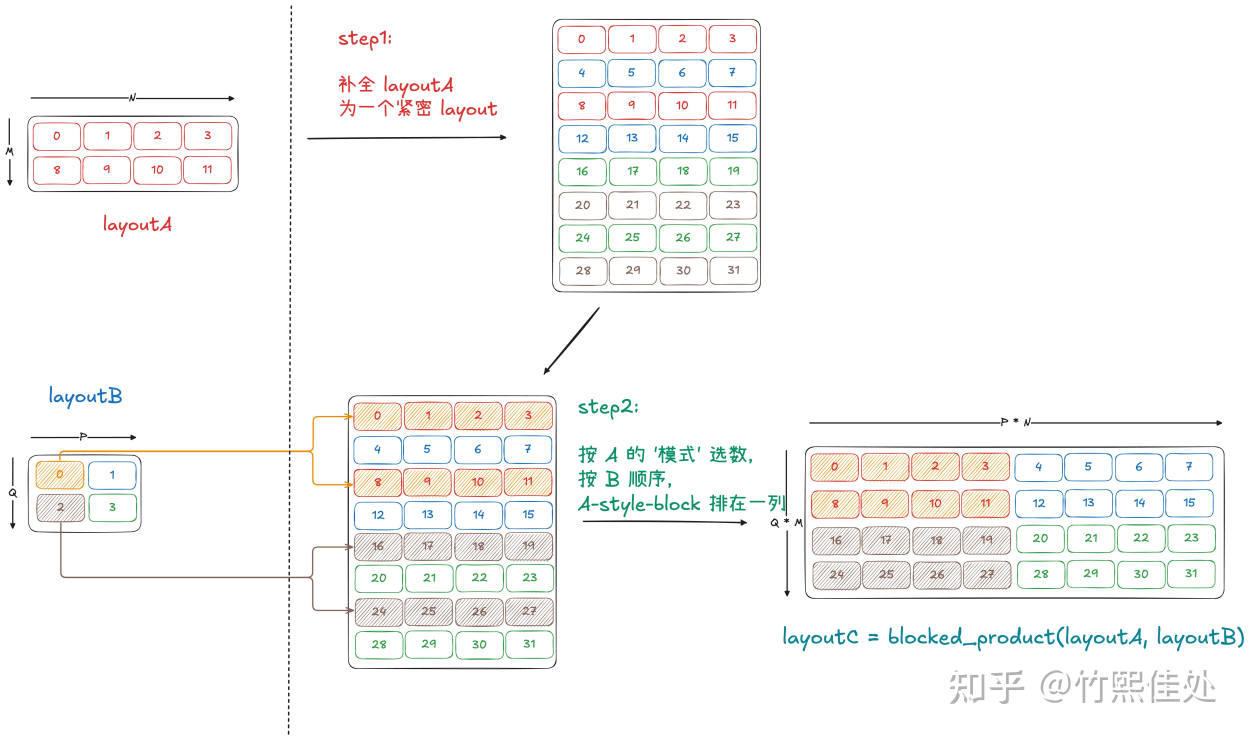


图 17.3: Figure.3 blocked_product 计算过程：首先将 A 补全到紧密 layout，再从紧密 layout 中找出 A-like layouts，最后按照 B 的 col-major 顺序取 A-like layouts，按 block 形式排在一起。

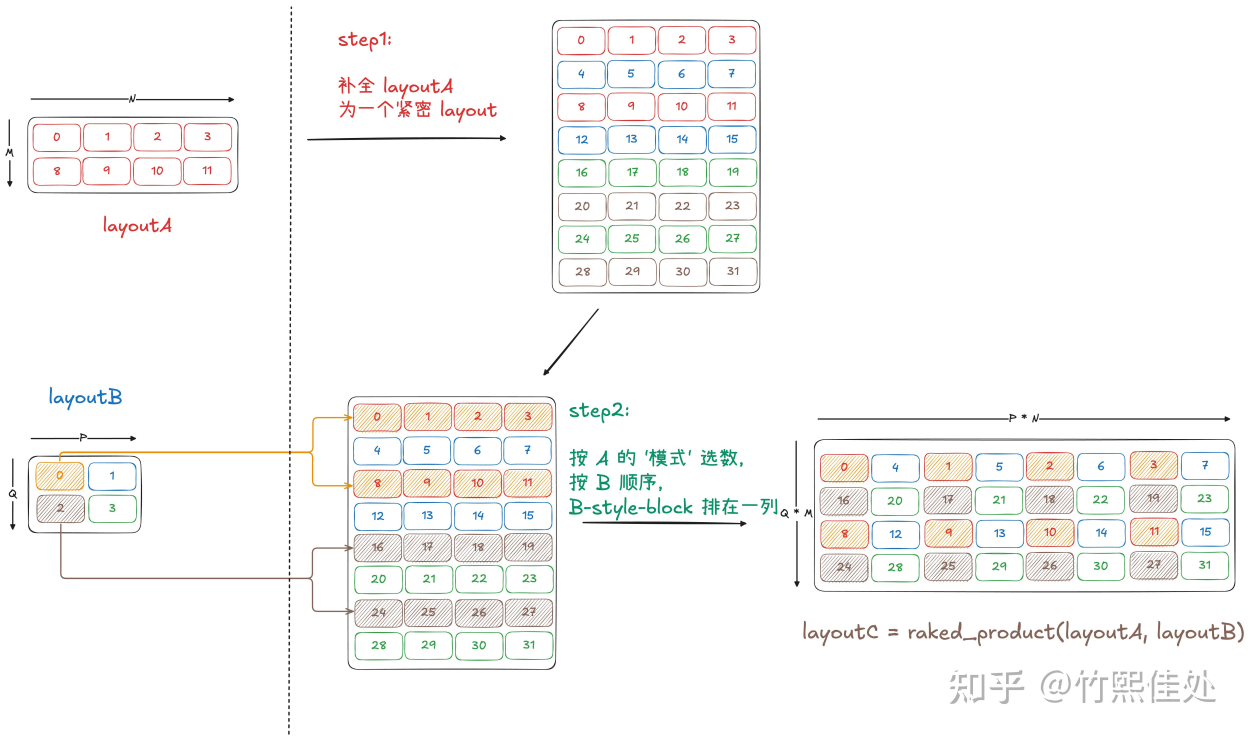


图 17.4: Figure.4 raked_product 计算过程：首先将 A 补全到紧密 layout 再从紧密 layout 中找出 A-like layouts，再按照 B 的 col-major 顺序取 A-like layouts，按 block 形式排在一起，最后进行一次重排，形成 raked 形状。

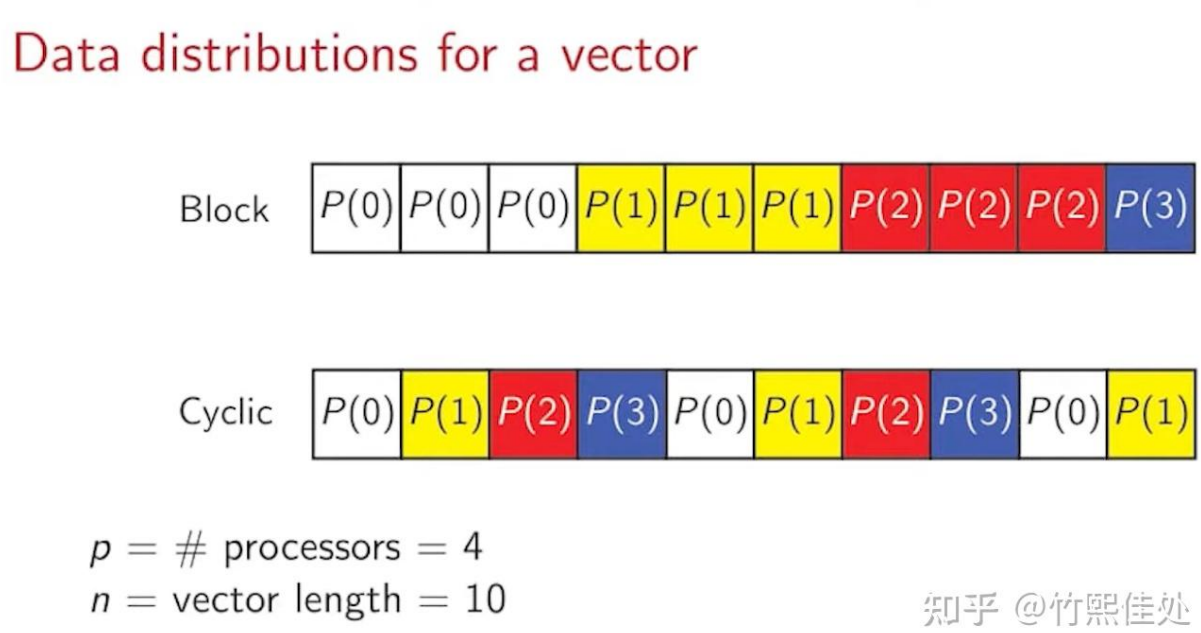


图 17.5: Figure.5 并行计算理论中 block 分布以及 cyclic 分布。

直觉上，raked_product 类似在 blocked_product 的基础上，把 C 做了一次一个隔一个的排列（rake 中文即是耙子，类似耙钉的排列，还怪形象的）。当然归根溯源，Raked 这个命名，实际上是来源于并行计算理论中的 cyclic 分布，通常用来均衡各个处理器上的任务数，如 Fig.5 [4]。

此外，product 还支持多维的形式，即：

LayoutC = product(LayoutA, (LayoutB$_0$, LayoutB$_1$, ...))

其中 (LayoutB$_0$, LayoutB$_1$, ...) 不再是一个单独的 layout，而是一个多个 layout 组成的 tuple，通常被称为 **Tiler**。

借助这种 Tiler 形式，可以做到 B$_i$ 之间不相互耦合。笔者感觉这类场景对于 product 来说其实没有实际应用的场景，在 divide 部分用的比较多。因此，我们看到 CuTe 文档中虽然提到了几种 tiler 相关的 product 函数，但它们在 cutlass 中较少被直接调用；实际上 cute 内部就在使用，例如 make_fragment_like 与 TiledMMA 的 thr_layout_vmnk 均基于 tiled_product。出于严谨，我们列出各函数对应的区别，读者有个印象即可：

```
Layout Shape : (M, N, L, ...)
Tiler Shape  : <TileM, TileN>

logical_product : ((M,TileM), (N,TileN), L, ...)
zipped_product  : ((M,N), (TileM,TileN,L,...))
tiled_product   : ((M,N), TileM, TileN, L, ...)
flat_product    : (M, N, TileM, TileN, L, ...)
```

回顾 Layout Product 的过程，其实就是**在尝试将 A layout 的模式进行重复，以覆盖一个紧密排列的空间**。我们其实能感觉到，这和我们平时写 cuda 代码的时候是很类似的。以 tiled copy 为例，首先我们规定了一个 thread block 选取的 data tile 作为一个“模式”，然后我们将这个模式重复，直到填满整个紧密排列的内存空间，不重不漏的完成所有数据的 copy。

17.3 Product 用法案例：tile_to_shape

Product 在 cutlass 中被广泛用于 layout 构建，以 blocked_product 为例，其有一个常用的变体：**tile_to_shape** 函数，区别在于 blocked_product 关注的是 layoutA 被重复多少次，tile_to_shape 关注重复后能达到多大的 shape。其具体代码如下：

```
template <class Shape, class Stride,
          class TrgShape, class ModeOrder = LayoutLeft>
CUTE_HOST_DEVICE constexpr
```

```
auto
tile_to_shape(Layout<Shape,Stride> const& block,
              TrgShape             const& trg_shape,
              ModeOrder            const& ord_shape = {})
{
  CUTE_STATIC_ASSERT_V(rank(block) <= rank(trg_shape), "Rank of layout must be <= rank of target shape.");
  constexpr int R = rank_v<TrgShape>;

  auto padded_block = append<R>(block);

  auto block_shape  = product_each(shape(padded_block));
  auto target_shape = product_each(shape(trg_shape));

  // Assert proper division
  if constexpr (is_static<decltype(target_shape)>::value) {
    CUTE_STATIC_ASSERT_V(evenly_divides(target_shape, block_shape),
                         "tile_to_shape: block shape does not divide the target shape.");
  }

  auto product_shape = ceil_div(target_shape, block_shape);

  return blocked_product(padded_block, make_ordered_layout(product_shape, ord_shape));
}
```

通过 tile_to_shape，我们可以拿一个小的 atom layout 来填满我们目标 shape 来获得的新 layout，这样我们在 atom layout 上做的变换，就能通过重复作用到整个目标 layout。例如，我们使用 shared memory swizzle 的时候，通常会写出如下代码：

```
using SmemLayoutAtom = decltype(composition(
      Swizzle<kShmLoadSwizzleB, kShmLoadSwizzleM, kShmLoadSwizzleS>{},
      make_layout(make_shape(Int<8>{}, Int<64>{}),
                  make_stride(Int<64>{}, Int<1>{}))));
using SmemLayoutA = decltype(
      tile_to_shape(SmemLayoutAtom{},
                    make_shape(Int<128>{}, Int<64>{}, Int<kStage>{})));
```

其本意即是来由 SmemLayoutAtom 通过 tile_to_shape 来构建大的 SmemLayoutA，这样只要我们保证 atom layout 中的 swizzle 能做到 bank-conflict-free，就能借由 product 扩散到整个 shared memory 的 bank-conflict-free。

17.4　Layout 的除法 Divide

如果说 Product 的逻辑是：通过重复一个“模式”来扩大一个 Layout。那么，Divide 的逻辑就是：通过选择“模式”来**拆解**一个大 Layout。

我们仍然以 Tiled Copy 为例：在乘法的例子中，我们知道一个 Thread block 负责的 Layout，可以通过重复来覆盖更大的 Layout。

在 Thread block 之下，我们还需要具体让每一个 Thread 知道自己负责的数据的坐标。这时候，就需要我们对 Thread block 负责的 Layout 进行**拆解**。

因此，**Layout Divide** 的计算过程即是：

给定一个大的待分解的 LayoutA ，然后给出一个基础的“模式” LayoutB ，二者做 divide：

LayoutC = divide(LayoutA, LayoutB)

代表我们通过在 A 上去找 B **的 offset** 决定的 B **-like layouts**，最后再把这些 B-like layouts 组合在一起，得到 C。当 tiler 整除 A 各维时，这**确保原本的 A 被不重不漏地拆分**；不整除时 Rest 维取 ceil，末块会覆盖到 A 之外。

以 **logical_divide** 为例，我们用图示来描述这个过程，如 Fig.6：

值得一提的是，当 tiler 整除 A 各维时，对 A 做 divide 得到的 C 与 A 的 Size 相同，只是映射关系改变。不整除时，Rest 维取 ceil，例如 logical_divide((10):(1), (4):(1)) 得到 $(4, 3)$，Size 为 $12 > 10$，多出的位置是越界元素；`local_tile` 的尾部 tile 正是靠这个 ceil 覆盖工作。这一点在直觉上和初等数学的 A / B 不同，可以类比成保留商和除数的整数除

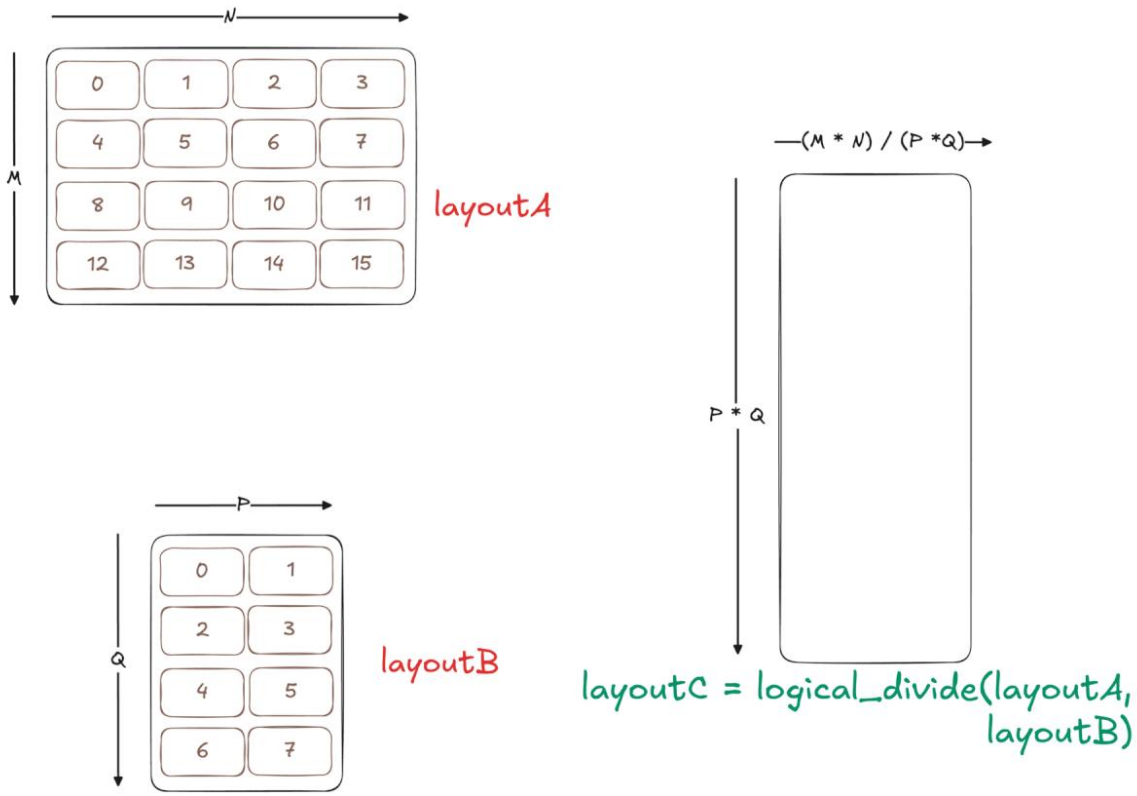


图 17.6: Figure.6 logical_divide 计算过程：按照 B col-major 访问得到 offset，映射到 A 中找出 B-like layouts，然后将所有 B-like layouts 按列排在一起。

法，如 $10/5 = (5, 2)$，但余数不为零时结果会向上取整。

与 Product 类似，CuTe 中也定义了多种不同的 divide 函数，包括 logical_divide、zipped_divide、flat_divide、tile_divide。它们之间的区别仍然在于：最如何选择和排列那些被拆解出来的 **B-like Layouts**。

与 Layout Product 不同，divide 的一维形式我们用得相对较少。相反，它的**多维形式**反而更容易理解；在多维形式中，我们通常把 LayoutB 指定为一个 **Tiler**，即：

LayoutC = divide(LayoutA, (LayoutB$_0$, LayoutB$_1$, ...))

在这种形式下，我们可以将 A 的 **Shape** 和我们希望选择的“模式”完全解耦开，真正做到我们想要指定行或者列要按照什么“模式”去选择数据。

什么意思呢，我们来做个对比：

在一维的场景中，如果我们给定的 A 的 shape 发生了改变，这时即使 B 保持完全一致，我们按照 B 的 offset 去找出来的 B-like Layouts 也会随之发生变化。而这，正是我们不希望看到的。

我们以 divide 操作中最常用的 zipped_divide 为例。当同一个 B 分别作用在具有不同 shape 的 A 和 A$'$ 上时，其结果的差异性，如 Fig.7 中所展示的。

作为对比，我们讨论二维场景下的情况。二维 divide 通常会被表达为以下形式：

LayoutC = divide(LayoutA, TilerB)

其中 TilerB 是一个由 2 个彼此独立的 Layout 所构成的 tuple：

TilerB = (LayoutB$_0$, LayoutB$_1$)

在这种情况下，divide 的过程随之调整为：将 A 的每一个维度，分别对这个 TilerB 中的对应 LayoutB$_i$ 执行 divide 操作，以完成数据的选取。

通过这种机制，我们可以确保对 TilerB 的定义不受到 LayoutA 的 shape 影响。即，当保持 TilerB 不变，并将其作用于两个具有不同 shape 的 LayoutA 和 LayoutA$'$ 上时，其选择的模式便能保持一致。具体的计算过程，如 Fig.8 所示。

因此，我们可以总结一个使用上的惯例：divide **操作通常侧重于使用多维形式，其中以二维形式最为常见。而** product **操作则多使用一维形式。**

除了前面提到的 zipped_divide 之外，常见的多维 divide 变体还包括：

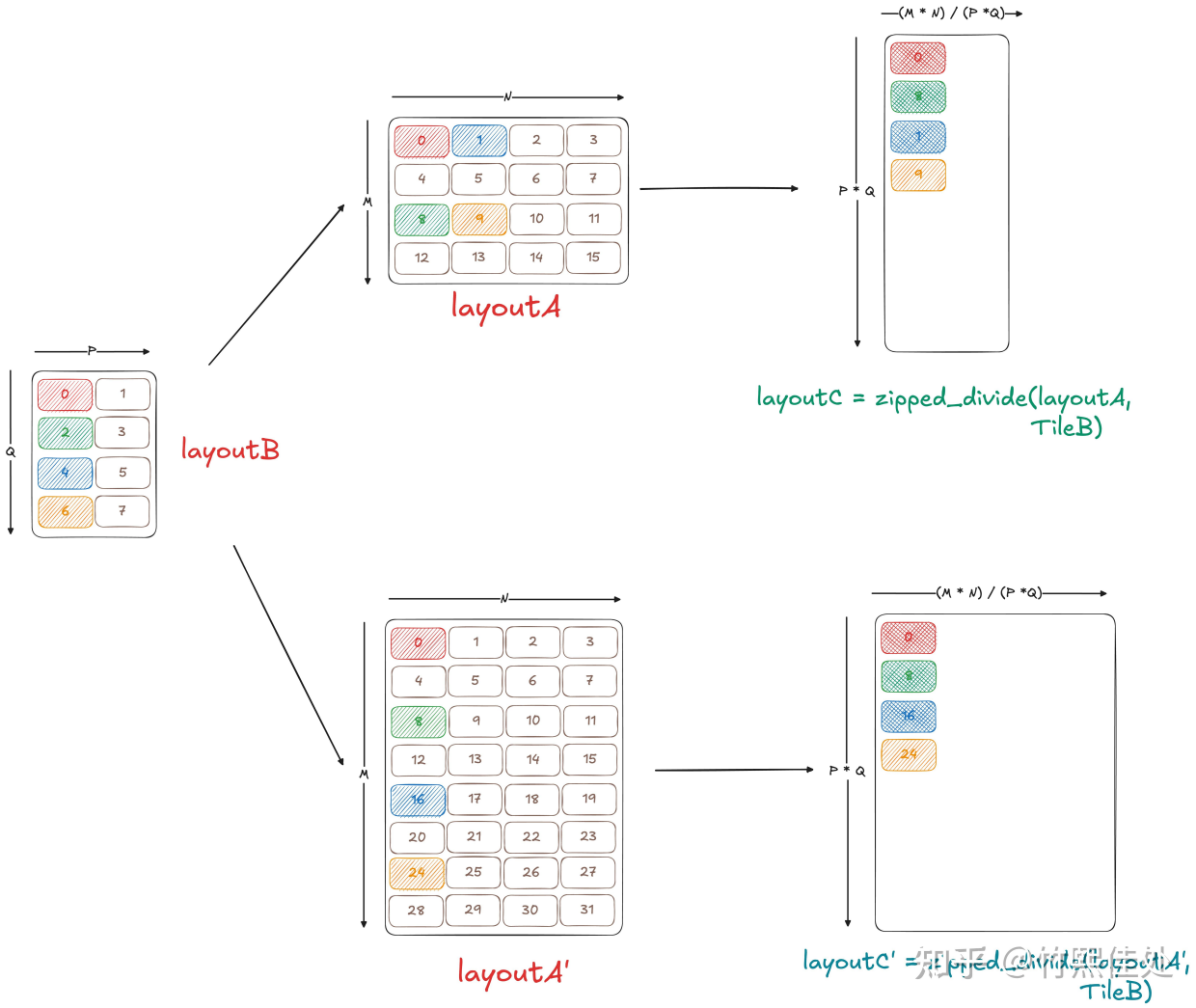


图 17.7: Figure.7 一维 zipped_divide 计算过程，相同 layoutB 作用在不同 shape layoutA 上的对比，我们选择的“模式”发生了改变。

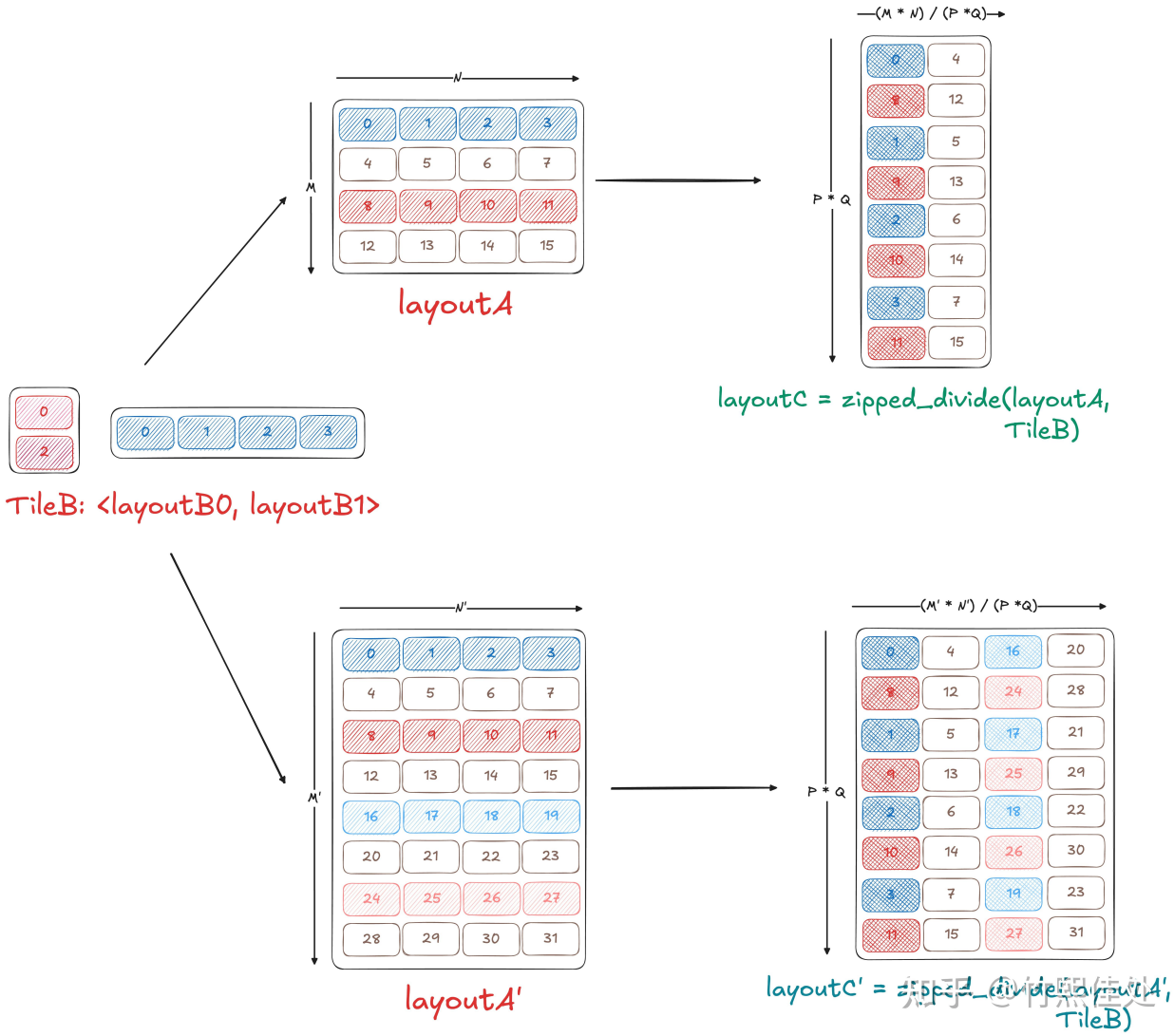


图 17.8: Figure.8 二维 zipped_divide 计算过程，相同 layoutB 作用在不同 shape layoutA 上的对比，可以看到我们选择的“模式”具有一致性。

logical_divide, flat_divide, tile_divide

这些函数在根据 TilerB 从 layoutA 中选取数据的逻辑上是完全一致的。

它们之间的区别主要体现在最后组成 layoutC 的阶段：各个函数会通过调整 layoutC 元素的顺序，或者调整括号的加减，从而影响最终输出 layoutC 的 shape。具体对比，如下表所示：

```
Layout Shape : (M, N, L, ...)
Tiler Shape  : <TileM, TileN>

logical_divide : ((TileM,RestM), (TileN,RestN), L, ...)
zipped_divide  : ((TileM,TileN), (RestM,RestN,L,...))
tiled_divide   : ((TileM,TileN), RestM, RestN, L, ...)
flat_divide    : (TileM, TileN, RestM, RestN, L, ...)
```

17.5　Divide 用法案例：local_partition & local_tile

我们注意到，如果我们将 Fig.8 中的 TilerB 中的 $(\text{LayoutB}_0, \text{LayoutB}_1)$ 设定为两个紧密排列的 layout，那么我们就能发现 zipped_divide 之后的 layoutC，**每一列就是** layoutA **中连续的小一块元素，每一行就是** layoutA **中块内相同位置的元素**，如 Fig.9 所示：

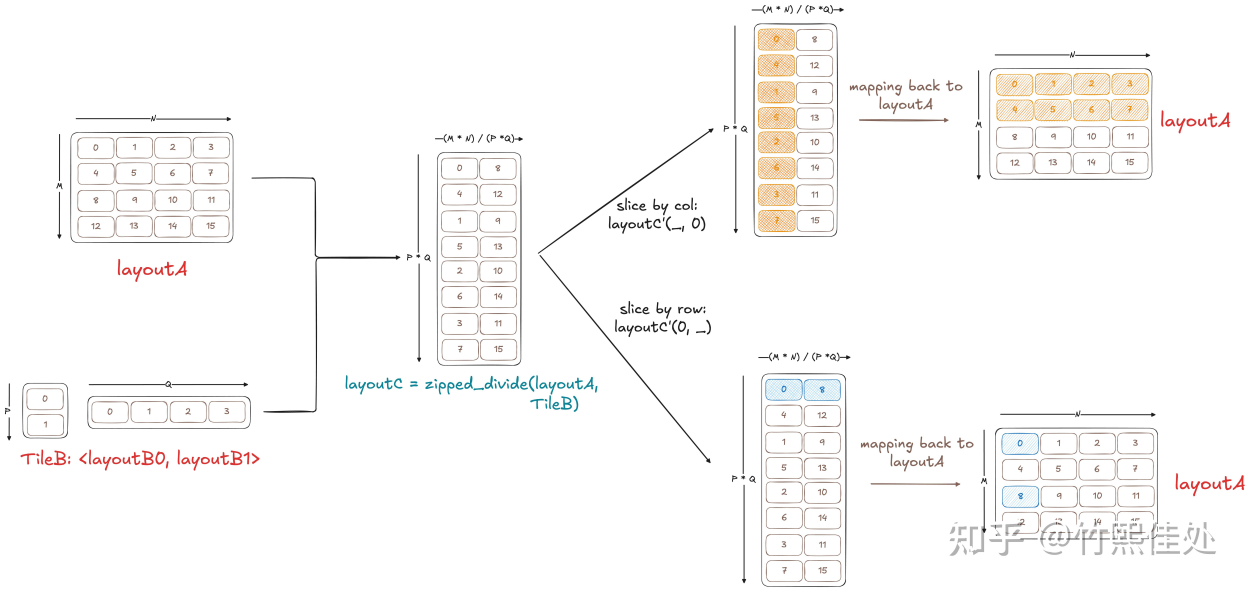


图 17.9: Figure.9 zipped_divide 的使用案例。可以看到对结果 layoutC 做 slice by col（取一列，固定块索引），可以得到 A 中连续的小一块元素；做 slice by row（取一行，固定块内位置），可以得到每一块块内相同位置的元素。

对于一些实际使用过 CuTe 的读者来说，可能立刻会联想到一个熟悉的场景：“这个功能，不就和我们在 tiled copy 操作中用到的 local_tile 和 local_partition 的功能是一样的吗?”

没错，这并非巧合。实际上当我们去观察 local_tile 和 local_partition 这两个函数（也叫 inner_partition 和 outer_partition）的具体内部实现时，我们会发现，它们正是通过 zipped_divide 来完成核心的数据重排和分块逻辑的。divide 的类似用法遍布整个 CuTe 的代码中，可以说是最常用的代数变换了。

```
// ...
local_tile(Tensor    && tensor,
           Tiler const& tiler,   // tiler to apply
           Coord const& coord)   // coord to slice into "remainder"
{
  return inner_partition(static_cast<Tensor&&>(tensor),
                         tiler,
                         coord);
}

// ...
```

```
inner_partition(Tensor    && tensor,
                Tiler const& tiler,
                Coord const& coord)
{
  auto tensor_tiled = zipped_divide(static_cast<Tensor&&>(tensor), tiler);
  constexpr int R0 = decltype(rank<0>(tensor_tiled))::value;
  // ...
}

// ...
local_partition(Tensor                     && tensor,
                Layout<LShape,LStride> const& tile,    // coord -> index
                Index                  const& index)   // index to slice for
{
  static_assert(is_integral<Index>::value);
  return outer_partition(static_cast<Tensor&&>(tensor),
                         product_each(shape(tile)),
                         tile.get_flat_coord(index));
}

// ...
outer_partition(Tensor    && tensor,
                Tiler const& tiler,
                Coord const& coord)
{
  auto tensor_tiled = zipped_divide(static_cast<Tensor&&>(tensor), tiler);
  constexpr int R1 = decltype(rank<1>(tensor_tiled))::value;
  // ...

}
```

17.6 总结

至此，我们已经完整地介绍了 layout 代数中的核心操作：product 和 divide。结合我们在上一篇中讨论的：compose 和 inverse，事实上，这四个基本操作就构成了 CuTe layout 代数的核心骨架。

阅读 Cutlass 代码的过程中，你会发现 layout 代数会以各种变体形式反复出现。除了我们本文中提到的 tile_to_shape、local_partition、local_tile 等，更常见的情况是，一个实际的功能函数，往往是多个代数变换步骤的组合。

例如，在 tiled copy 中，我们能看到 raked_product、inverse 和 compose 的联合使用。在 tiled mma 中，则用到了多次的 logical_divide、zipped_divide 和 compose 等操作。

熟练掌握这些基本的 layout 代数操作，能够让我们对 CuTe 这套工具的使用更加得心应手。一旦融会贯通，你就能够在理解高性能算子实现的道路上，觉醒一种 **“通透世界”** 的能力，清晰地看透复杂数据排布背后的逻辑，成为真正的高性能算子大师。

本系列其他文章导引：

写给大家看的 CuTe 教程：tiled copy

写给大家看的 CuTe 教程：tiled mma

写给大家看的 CuTe 教程：Layout Compose & Inverse

17.7 Reference

[1] 写给大家看的 CuTe 教程：Layout Compose & Inverse[1]

[2] 写给大家看的 CuTe 教程：tiled copy[2]

[1] https://zhuanlan.zhihu.com/p/1962625273636845008

[2] https://zhuanlan.zhihu.com/p/1930389542784964333

[3] 写给大家看的 CuTe 教程：tiled mma[3]
[4] 并行计算理论中 block/cyclic 数据分布的参考资料（原文未给出具体出处）

[3] https://zhuanlan.zhihu.com/p/1937145378446226159

第十八章　写给大家看的 CuTe 教程：TMA Copy

作者：竹熙佳处（知乎专栏「CuTe 教程」）　原文：https://zhuanlan.zhihu.com/p/2003198909405763007

18.1　动机

Nvidia 在 Hopper 架构 [1] 之中引入了非常多的新特性，其中 **Tensor Memory Accelerator (TMA)** 作为新的 load & store 单元是一个有着深远影响的改动：硬件层面上，从 Hopper 到 Blackwell 架构都延续并强化了 TMA 设计，甚至在 RTX5080 (sm120，blackwell) 这类非计算卡上都被保留了，可以预期后续多代 GPU 都会延续这个设计；软件层面上，其引入正式强化了 **warp-specialized (WS)** 编程范式的效率，目前正成为 NV tensor core kernel 的标准写法（如 CUTLASS 3.x），足以说明其重要性。

我们观察 NV GPU 架构的演进趋势，可以看到的是越来越高的 IO 带宽以及越来越强的 tensor core。一方面，IO 带宽的增加需要让每个 SM 保持足够大量的数据在 load 过程中（i.e. more bytes in flight，littles-law[2]）才能达到理论带宽上限；另一方面，tensor core 的算力要发挥出来，也需要时刻保持 tensor core 所需的数据是 ready 的状态，否则计算单元将处于 stall 状态，kernel 的计算效率不可能高。

这对于传统的 load 指令提出了挑战。即使我们使用最大字长的指令 **LDG.128**，一个 warp 单次 load 的数据也只能达到 16Bytes * 32 = 512 Bytes。由于一个 SM 驻留的 warp 数量（Occupancy）是有上限的，因此其能达到的 bytes in flight 往往不足以填满显存带宽。

这时候可以考虑连续多次发射 load 指令（即，ILP），来让更多数据处于 flight 状态。然而增加到一定程度后，就引入了新问题：每条 `LDG` 指令需要消耗寄存器来暂存数据，而寄存器资源的稀缺性反过来限制了 SM 中驻留的 warp 数量，导致 bytes in flight 再次受限，此消彼长，带宽利用率依旧上不去。更不用说在 GEMM / Attention 场景下，寄存器资源多被用于存储 Tensor Core 的累加结果，本就捉襟见肘。

另一个痛点是，频繁的 load 指令发射占用了 warp scheduler 的发射带宽，且 load 伴随的地址计算也会加重 ALU 的负担，进一步挤压了能达到的 load 带宽。

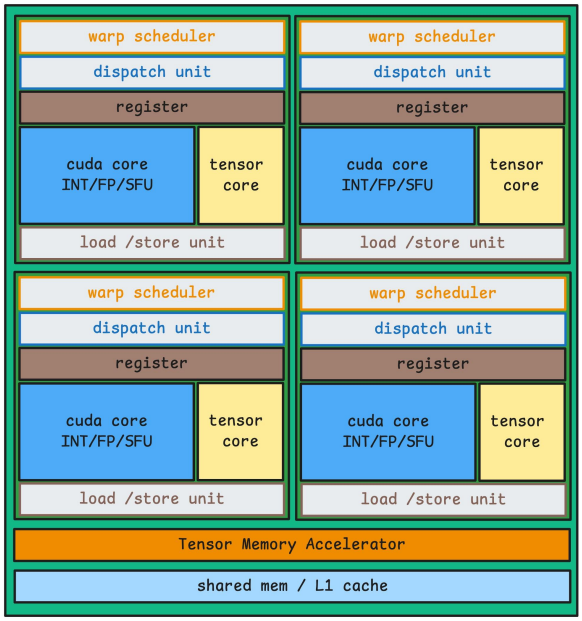
于是，机智的 NV 工程师提出了新的解法：

- 既然多条指令 load 效率低，那么把单条指令能 load 的数据量增大；
- 寄存器资源受限，那么 Bypass Register，直接将数据 load 到空间相对更充裕的 shared memory 中；
- 地址计算消耗 ALU，那让地址计算在专用硬件上完成。

因此，在 Hopper 架构中，TMA 应运而生：

- 其可以做到一条 **cp.async.bulk.tensor** 指令 load 一整块数据到 shared memory。TMA 作为一个专有硬件单元，可以自行完成地址计算，甚至自带越界 (OOB) 处理，相当贴心。
- 同时，TMA 作为 Global Memory 到 Shared Memory 之间的传输单元，是完全异步执行的。即，其具体什么时候完成 Copy 需要通过 **mbarrier** 来同步，在使用 Shared Memory 的数据前必须 wait barrier 以确保数据正确性。
- 为了针对性优化 Tensor Core 计算中常用的 Tile data 加载，TMA 被设计为支持多维数据访问，即，**对于 TMA 来说内存空间不再是一个连续的 1 维地址空间，而是一个多维 box 空间，支持 1D~5D 的数据访问**。

有了 TMA，我们就可以达到极高的 load 效率，在 Memory-bound kernel 中尽可能跑满带宽，在 Compute-bound kernel 中通过更好的 Overlap 让 Tensor Core 零等待，尽可能达到算力上限。在硬件上，TMA 作为一个独立硬件单元，物理位置紧邻 Shared Memory，每个 SM 标配一个 TMA 单元，如 Fig.1 所示。



知乎 @竹熙佳处

图 18.1: Figure.1 Hopper SM 架构图；每个 SM 均配有一个 TMA，其位置位于 Shared mem 之上

18.2　用法

那么如何才能使用 TMA 呢？首先我们以 2D load 为例，高度概括地总结一下其基本流程。

Host 端：构造 TMA Descriptor

在 Host 端完成 TMA Copy Descriptor 的构造，并将其作为 Kernel 参数传入。语义上，它包含：

- **Global Memory 信息**：Base Pointer、整体 Layout (Shape & Stride)；
- **Box 信息**：每次 Copy 搬运的数据块（Box）的 Shape & Stride；
- **控制信息**：Swizzle 模式、OOB（越界）处理模式、L2 Cache 策略等。

这个 Descriptor 本质上是底层硬件使用的一段连续的 128 Bytes 数据，具体的字段对用户不透明，在不同 arch 以及驱动版本都有着不同的含义。

Kernel 端：发起 Copy 指令

在 Kernel 内通过调用 **cp.async.bulk** 指令发起 Copy，同时提供：

- **Descriptor**：即 Host 端传进来的对象；
- **Mbarrier**：用于控制是否 copy 完成的对象；
- **Dst Pointer**：Shared Memory 的目标地址；
- **Coordinates**：要访问的数据块在 Global Tensor 中的逻辑坐标，例如 2D，就需要 2 个数值 crd0 & crd1 来代表坐标，以 Descriptor 中的 Base Pointer 为起始。

然后等待 mbarrier 到达预期状态，Copy 结束，Shared Memory 数据准备就绪。这个过程我们可以描述为 Fig.2。

以上过程，Old-school 选手可能会使用 cuda 提供的 API，来指定对应参数后创建一个 CUtensorMap 结构体作为 descriptor。然后用 inline ptx 的方式调用 **cp.async.bulk.tensor** 指令，以及相关的 **mbarrier** 指令来等待 barrier 完成。如果不想深度依赖 CuTe layout 体系，也可以使用 CuTe 已有的封装好的 ptx 函数，deepgemm 就是借用的这种方式。

```
CUTE_HOST_DEVICE static void
  copy(void const* desc_ptr, uint64_t* mbar_ptr, uint64_t cache_hint,
       void      * smem_ptr,
       int32_t const& crd0, int32_t const& crd1) {
```

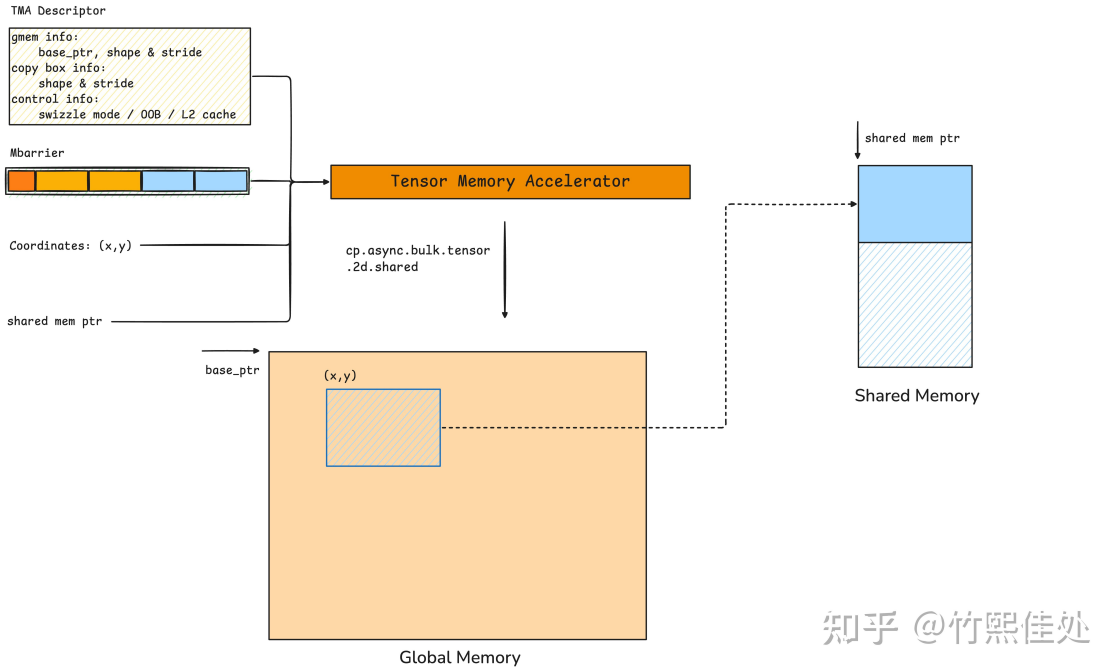


图 18.2: Figure 2. 2D TMA copy 过程示意图；在 Host 端完成 TMA Descriptor 的构造，在 Kernel 中完成 TMA Copy，每次 Copy 完成一个 Box 大小的数据搬运，依靠 Mbarrier 来标记 Copy 完成

```
    uint64_t gmem_int_desc = reinterpret_cast<uint64_t>(desc_ptr);
    uint32_t smem_int_mbar = cast_smem_ptr_to_uint(mbar_ptr);
    uint32_t smem_int_ptr  = cast_smem_ptr_to_uint(smem_ptr);
    cutlass::arch::synclog_emit_tma_load(__LINE__, gmem_int_desc, smem_int_mbar, smem_int_ptr);
    asm volatile (
      "cp.async.bulk.tensor.2d.shared::cluster.global.mbarrier::complete_tx::bytes.L2::cache_hint"
      " [%0], [%1, {%3, %4}], [%2], %5;"
      :
      : "r"(smem_int_ptr), "l"(gmem_int_desc), "r"(smem_int_mbar),
        "r"(crd0), "r"(crd1), "l"(cache_hint)
      : "memory");
  }
```

更简单的，CuTe 已经提供了足够简洁的上层函数如 make_tma_copy / get_tma_tensor / copy 函数来完成上述步骤，我们遵循 CuTe 的使用方法即可，在此我们引用 coflex 的经典文章 [3] 中的代码段（原代码有一处实参笔误，引用时已修正）来展示其用法。

TMA copy descriptor 的构造如下代码：

```
template <typename T, int CTA_M, int CTA_N>
void host_fn(T* data, int M, int N) {
  using namespace cute;

  // 1. Create the GMEM tensor
  auto gmem_layout = make_layout(make_shape(M, N), LayoutRight{});
  auto gmem_tensor = make_tensor(make_gmem_ptr(data), gmem_layout);

  // 2. Create the SMEM layout (Atom layout for TMA)
  auto smem_layout = make_layout(make_shape(CTA_M, CTA_N), LayoutRight{});

  // 3. Create the TMA object (contains the Tensor Map)
  auto tma_load = make_tma_copy(SM90_TMA_LOAD{}, gmem_tensor, smem_layout);

  // 4. Invoke the kernel
  tma_load_kernel<T, CTA_M, CTA_N, decltype(tma_load),
                                   decltype(gmem_layout),
```

```
                                          decltype(smem_layout)><<<dim3{M / CTA_M, N / CTA_N, 1}, 1>>>(
    data, data, tma_load, gmem_layout, smem_layout);
}
```

copy 执行代码如下：

```
template <typename T, int CTA_M, int CTA_N, class TmaLoad, class GmemLayout,
          class SmemLayout>
__global__ void tma_load_kernel(T* g_out, const T* g_in,
                                __grid_constant__ const TmaLoad tma_load,
                                GmemLayout gmem_layout,
                                SmemLayout smem_layout) {
  using namespace cute;
  constexpr int tma_transaction_bytes = CTA_M * CTA_N * sizeof(T);

  __shared__ T smem_data[CTA_M * CTA_N];
  __shared__ uint64_t tma_load_mbar;

  // Construct SMEM & GMEM tensor
  auto smem_tensor = make_tensor(make_smem_ptr(smem_data), smem_layout);
  auto gmem_tensor = make_tensor(make_gmem_ptr(g_in), gmem_layout);

  if (threadIdx.x == 0) {
    auto gmem_tensor_coord = tma_load.get_tma_tensor(shape(gmem_tensor));

    auto gmem_tensor_coord_cta = local_tile(
        gmem_tensor_coord, Tile<Int<CTA_M>, Int<CTA_N>>{},
        make_coord(blockIdx.x, blockIdx.y));

    initialize_barrier(tma_load_mbar, /* arrival count */ 1);
    set_barrier_transaction_bytes(tma_load_mbar, tma_transaction_bytes);

    auto tma_load_per_cta = tma_load.get_slice(0);
    copy(tma_load.with(tma_load_mbar),
         tma_load_per_cta.partition_S(gmem_tensor_coord_cta),
         tma_load_per_cta.partition_D(smem_tensor));
  }
  __syncthreads();
  wait_barrier(tma_load_mbar, /* phase */ 0);
  // after this line, the TMA load is finished
}
```

如果只是想了解如何使用，那么上面的例子抄下来改一改就可以了，这其实也是对 CuTe 熟悉之后的好处：能够对照 cutlass 快速开发算子，尽管对硬件细节没有那么了解。但是本专栏的读者朋友，大概不会满足于如此粗浅的了解，接下来我们更深入的了解一下 CuTe 中关于 TMA 的细节。

18.3　CuTe 中的坐标表示：arithTuple

首先，我们思考 TMA copy 和普通的 SIMT copy 有什么不同：我们可以注意到，**SIMT copy 本质上是每个 thread 每条指令消耗一个 1d offset，而 TMA copy 则是消耗一个起始坐标**。我们可以看到上述代码中的 **get__tma__tensor** 函数就是构建了这样一个 gmem_tensor_coord，其在 CuTe 中定义如下：

```
auto get_tma_tensor(GShape const& g_shape) const {
    static_assert(is_congruent<decltype(g_shape), decltype(aux_params_.g_stride_)>::value);
    return make_coord_tensor(make_layout(g_shape, aux_params_.g_stride_));
  }
```

为什么这里不是一个普通的 make_tensor 就可以了呢？因为 CuTe Layout 的主要功能是用来把坐标转换成 1d offset，而在 TMA 场景下，我们需要保留原始的 ND 坐标语义，不需要转换成 1d offset，因此常规的 Layout 语义在这里是不匹配的。

为此，CuTe 只能打个补丁，引入了一个新的概念来表示纯粹的坐标语义，此即 **arithTuple**。我们先结合一个例子来理解 arithTuple 如何表示坐标：

```
auto a = make_tensor(make_inttuple_iter(0,0),
                     make_shape (     4,      5),
                     make_stride(E<0>{}, E<1>{}));
print_tensor(a);
/*
ArithTuple(0,0) o (4,5):(_1@0,_1@1):
  (0,0)  (0,1)  (0,2)  (0,3)  (0,4)
  (1,0)  (1,1)  (1,2)  (1,3)  (1,4)
  (2,0)  (2,1)  (2,2)  (2,3)  (2,4)
  (3,0)  (3,1)  (3,2)  (3,3)  (3,4)
*/
```

可以看到，我们仍然用 make_tensor 创建了 tensor，只不过这个 tensor 比较特殊，其 “base_ptr” 不再指向一个内存地址，而是一个 tuple，用来表示起始坐标 (0, 0)。然后对于其 “layout” 部分，我们也用 shape & stride 表达：其中 shape 的表示是平凡的；stride 则需要注意——由于需要表示 2D 的位置变化，我们需要指定一个 “2D stride”。

就这个例子而言，我们希望在每一列上，逐行增加第一个维度的坐标，因此第一个维度 stride 设为 (1, 0)；同理，我们希望在每一行上逐列增加第二个维度的坐标，因此第二个维度 stride 设为 (0, 1)。语义上，我们可以理解成 make_stride((1, 0), (0, 1))。但为了避免歧义并简化表达，CuTe 将 (1, 0) 和 (0, 1) 这类基础步长定义为 **Basis Element，表示为 E<0>{} 和 E<1>{}**。这其实和普通 Layout 中用 Int<1>{} 作为 stride 是类似的。

具体的，这个 E<0>{} 代表的是 “第 0 个位置为 1，其他位置为 0 ” 的 tuple；E<1>{} 代表的则是 “第 1 个位置为 1，其他位置为 0”。从其 print 结果来看，(_1@0, _1@1) 其实就是在表达这样一个语义：**_1@0 代表第 0 位置的 stride 为 1（@0 代表 at 0 维度），_1@1 代表第 1 位置的 stride 为 1**。E 可以乘以一个整数表示 stride 的扩大，如我们把例子中的 stride 改为 (5 * E<0>{}, 2 * E<1>{})，其打印结果如下：

```
auto a = make_tensor(make_inttuple_iter(0,0),
                     make_shape (     4,      5),
                     make_stride(5 * E<0>{}, 2 * E<1>{}));
print_tensor(a);
/*
ArithTuple(0,0) o (4,5):(5@0,2@1):
  (0,0)  (0,2)  (0,4)  (0,6)  (0,8)
  (5,0)  (5,2)  (5,4)  (5,6)  (5,8)
  (10,0)  (10,2)  (10,4)  (10,6)  (10,8)
  (15,0)  (15,2)  (15,4)  (15,6)  (15,8)
*/
```

有了基于 arithTuple 的坐标表示，我们就可以着手构建 TMA copy 相关的函数了。我们知道普通的 CuTe Tensor 是由一个 base_ptr + layout 构成的，那么将 base_ptr 替换为一个 base 坐标，Layout 也替换为对坐标的 Shape & Stride 描述，这就构成了 TMA Tensor。

那么这种特殊的 Tensor，是否仍然适用于我们在普通 Tensor 上定义的各种函数，如 Partition & Tile? 其 Layout 是否也能做到通过 Inverse & Compose & Product & Divide [4][5] 灵活变换呢?

答案是肯定的。事实上我们从前面的代码中也可以看到，对 gmem_tensor_coord_cta 仍然可以用 **local_tile** 以及 **partition_S** 函数来进行数据划分。这在大体上保证了不同架构（SIMT vs TMA）的 Kernel 在写法和逻辑上的统一。

另外一个需要注意的点，是 TMA Box 的起始坐标，不一定需要是和 Box Size 对齐的。比如 Box Size 是 (16, 64)，起点可以是 (0, 8) 这样的位置，只要满足起点对应到 global mem 上的物理地址是 16 Bytes 对齐即可。这也是处理 variable-len Attention 或是 Group GEMM 下的常用特性，通过坐标的 domain_offset 函数即可完成起始位置的偏移。

顺带一提，在 CuTe 中，arithTuple 除了用来表示 TMA 坐标，也可以用来构建 Identity Tensor，进而用于处理 Tiled Copy 的边界判定或是 Attention 中 Mask 的逻辑，也是非常实用的特性。

18.4　MBarrier

TMA 是一个异步 copy 单元，其 copy 的完成是无法预测的，需要显式 wait barrier 达到某个特定状态，才能认为 copy 得到的数据可用。那么这个 barrier 具体是如何发挥作用的呢？对应的 CuTe 函数是如何修改 / 检查 barrier 的状态呢？

实际上，我们可以把 barrier 认为就是一个 `uint64_t` 的变量，通常声明在 shared memory，其特殊之处在于 64 个 bit 位都被赋予了特定语义。例如，我们把相关代码抽出来看，`tma_load_mbar` 就是一个 barrier。

```
constexpr int tma_transaction_bytes = CTA_M * CTA_N * sizeof(T);
  __shared__ uint64_t tma_load_mbar;
  if (threadIdx.x == 0) {
    initialize_barrier(tma_load_mbar, /* arrival count */ 1);

    set_barrier_transaction_bytes(tma_load_mbar, tma_transaction_bytes);

    copy(tma_load.with(tma_load_mbar),
         tma_load_per_cta.partition_S(gmem_tensor_coord_cta),
         tma_load_per_cta.partition_D(smem_tensor));
  }
  __syncthreads();
  wait_barrier(tma_load_mbar, /* phase */ 0);
```

那么这 64-bit 都包含哪些信息呢？在 NV open AI day 的技术分享中，其展示了 barrier 各字段语义如 Fig.3：

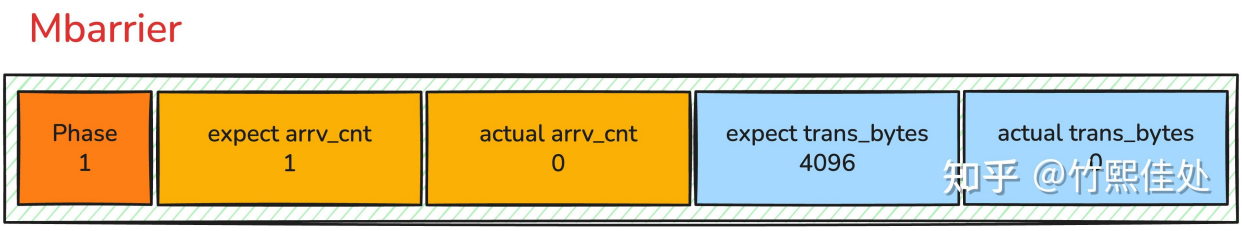


图 18.3: Figure 3. Barrier 包含的语义表示。包含一个 1bit 翻转用的 Phase 段，预期的 arrive count 以及 transaction bytes，以及实际被执行的 arrive count 以及 transaction bytes。实际 transaction bytes 则只能通过 tma 指令来改变。

具体来说，语义上，barrier 这些字段中包含我们可以指定的两个初始量：

- **Expect arrv_cnt**：代表期望有多少个 producer 线程到达（arrive），通过 initialize_barrier 函数设置。
- **Expect trans_bytes**：代表期望本次传输需要完成多少 bytes，通过 set_barrier_transaction_bytes 设置。

以及随着 copy 的进行，我们可以动态改变的量：

- **Actual arrv_cnt**：代表**当前已到达的次数**。代码中 lane 0 发起 copy 前会进行 arrive 操作，该数值 +1。
- **Actual trans_bytes** (只有 TMA 硬件能改)：代表实际传输了多少 bytes，tma 每完成一部分 bytes 的 copy，就会更新这个字段。
- **phase**：1 bit，初始化为 0，且是一个可以在 0->1 / 1->0 之间翻转的状态量。所谓 `wait_barrier` 就是在等待其产生变化。

TMA 在传输完成后，barrier 状态若同时满足：**Actual arrv_cnt == Expect arrv_cnt** 且 **Actual trans_bytes == Expect trans_bytes**，那么这个 phase 位会发生翻转。

例如，初始 phase 为 0，TMA copy 完成后，phase 发生 0 -> 1 的翻转，那么 wait_barrier(barrier, 0) 这个阻塞就被解除了，这条指令之后的指令（如 mma）就可以开始执行了。

值得注意的是，如果我们 wait_barrier(barrier, 1)，那么其在 phase 为 0 时，会直接解除阻塞（因为 0 != 1），只有等到 phase 为 1 时才开始阻塞，并且等待 phase 1 -> 0 的翻转发生后，再解除阻塞。

更细节的 barrier 各字段的精准定义，我们推荐 @reed[1] 的这篇文章：cute 之 Hopper MBarrier[2] 值得一提的是，CuTe 中对于 barrier 的使用，通常会被封装到 pipeline 的结构体 [6] 中，来提供在 multi-stage 场景下的 barrier 状态的管理，只不过在没有文档的情况下，对这个结构体的理解又是非常繁重的心智负担，我们后续再单独出文章进行总结。当然，即使不使用 pipeline，只要理清楚 barrier 的状态变化，我们在 multi-stage 场景下在依旧能用好 barrier，deepgemm 就是一个很好的例子，reed 的 hpc-ops 库 [7] 也都是这样使用的。

18.5 make_tma_copy 梳理

了解完 TMA copy 在 kernel 中如何发起后，我们再回头来看看 TMA 在 host 端的 copy descriptor 是如何构建出来的。然而在代码层面，我们可以看到核心函数 make_tma_copy() 调用一次就完成了，但是其底层代码远比我们想象的要复杂得多。甚至可以将其当作一个微型优化器来理解。

其输入：

- copy_atom
- global tensor
- shared layout

输出一个**带有最优 TMA descriptor 的 TiledCopy 对象**。

那么这个“最优”是怎么理解的呢？这里我们需要了解一下 TMA 作为一个硬件单元的特点：

- **维度限制**：支持 1D~5D 的 Box Copy，高于 5D 会报错。
- **Box 大小限制**：单次 TMA Box 的每个维度元素个数受硬件指令字段限制（8-bit），必须小于等于 256。
- **效率要求**：在 copy 相同 bytes 的数据时，一次 copy 大块数据比多次 copy 小份数据发射指令更少，因此 copy 效率更高。

而我们在使用 CuTe 时，有时会无法避免地写出来非常高维的 Tensor，例如在描述 attention 中 QKV tensor 时，(batch, seqlen, num_head, head_dim) 就已经占据了 4 个维度了，再加上我们需要叠加 Swizzle 引入的 shm atom 切分，这就 6 个维度了，完全不够用；类似的情况还有 conv、group gemm 算子等等。

既然硬件不支持 >5D 的 Box，那只有在代码层面来对高维数据进行合并；而且考虑到发起的 copy 越少越好，这个合并应该是一个贪心的过程，且在合并的过程中也要检查不能超出硬件对 Box Size 的 256 限制。其具体的逻辑涉及到非常多 CuTe 的代数变换以及多个辅助函数，我们总结其主要函数的调用逻辑为如下伪代码：

```
#输入：
# copy_atom: SM90_TMA_LOAD / SM90_TMA_LOAD_MULTICAST / SM90_TMA_STORE;
# global tensor,
# shared layout,
# 输出：
# 带有合并维度 TMA descriptor 的 tiled copy 对象
def make_tma_copy(copy_atom, gtensor, slayout, ...):
    # 1. 去除 swizzle 逻辑，只取 shape
    smem_layout_pure = get_nonswizzle_portion(slayout)

    # 2. 编译期合并维度 & 构建 TMA descriptor & 检查 256 限制
    def make_tma_copy_atom(gtensor, smem_layout_pure):

        # 2.1 compile-time 合并维度: eg. ((_8,_16),(_64,_2)) -> ((_64,_128),_2)
        tma_gbasis = construct_tma_gbasis(gtensor, smem_layout_pure):
            # 尝试将 shm layout 映射回 global layout，方便下一步合并连续的维度
            tile_gstride = gtensor.compose(right_inverse(smem_layout_pure)).layout()

```

[1] https://www.zhihu.com/people/2600b0313c76a175aa9e09a235382b79

[2] https://zhuanlan.zhihu.com/p/1962636004235153810

```
            # 按规则合并维度，并检查合并后的维度 <= 256
            tma_gstride = coalesce_256(tile_gstride)

            # 返回合并后的维度信息
            return tma_gstride

        # 2.2 按照合并好的维度构建 TMA descriptor & 必要时将超过 5d 的维度合并成 5d
        tma_desc = make_tma_copy_desc(gtensor, tma_gbasis):

            # C. run-time 计算超过 5d 维度的 Stride 最大公约数 (GCD)，进而将其压缩成一个维度
            fill_tma_gmem_shape_stride(gtensor, tma_gbasis, output=tma_desc)

            # D. 可选：根据 multicast 属性切分 tma box size
            if is_multicast:
                smem_box_shape = ceil_div(smem_box_shape, multicast)

            # E. 调用 CUDA 驱动 API，生成 TMA descriptor
            cuTensorMapEncodeTiled(tma_desc, ...)

            return tma_desc

        # 3. 返回封装好的 tiled copy 对象
        return TiledCopy(tma_desc, layout_TV)
```

初读 CuTe 中的这段代码，笔者也感到非常困惑，我们用 TMA 只不过想完成一个 Tile 数据的 copy，至于引入这么多复杂逻辑吗？但是经过思考我们发现，这些逻辑其实就是为了应对复杂 Layout 场景下（如引入 Swizzle），自动生成我们上述提到的“最优” copy 方案。

举个例子，我们在对一个简单的 (128, 128) bf16 的 Tile 进行 swizzle 的切分后，其 shape 变为了 ((8, 16), (64, 2))。按照最朴素的想法，应该设置 Box Size 为 (8, 64) 然后发起 16x2 次 copy。但是实际上经过 make_tma_copy 的优化，你会发现其打包成了 **(128, 64)**，大大减少了指令发射数量。

为什么可以这样合并呢？因为 (8, 64) 的 atom layout 按照 col-major 排列，那么在 M 维度上的 16 个 atom 实际上在内存上是连续的，**construct_tma_gbasis()** 会贪心地将其合并起来。

那么，为什么不在 N 维度上继续贪心扩展到 128 呢？这是因为 **Swizzle Mode** 决定了 TMA Box 在连续维度的最大物理宽度。对于 bf16 (2 Bytes) 来说，128 Bytes / 2 = 64 个元素。

TMA supports interleaved layouts like NC/8HWC8 where C8 utilizes 16 bytes in memory assuming 2 byte per channel or NC/16HWC16 where C16 uses 32 bytes. When `interleave` is CU_TENSOR_MAP_INTERLEAVE_NONE and `swizzle` is not CU_TENSOR_MAP_SWIZZLE_NONE, the bounding box inner dimension (computed as `channelsPerPixel` multiplied by element size in bytes derived from `tensorDataType`) must be less than or equal to the swizzle size.

- CU_TENSOR_MAP_SWIZZLE_32B requires the bounding box inner dimension to be <= 32.
- CU_TENSOR_MAP_SWIZZLE_64B requires the bounding box inner dimension to be <= 64.
- CU_TENSOR_MAP_SWIZZLE_128B* require the bounding box inner dimension to be <= 128. Additionally, `tensorDataType` of CU_TENSOR_MAP_DATA_TYPE_16U6_ALIGN16B requires `interleave` to be CU_TENSOR_MAP_INTERLEAVE_NONE.

CUDA 官方文档对 TMA swizzle mode 的约束：使用 CU_TENSOR_MAP_SWIZZLE_32B/64B/128B 时，bounding box 内维（inner dimension）的字节数需分别不超过 32/64/128（原文此图无题注，为整理者所补）。

这个限制会体现在 SMEM Layout 上，并由此传递给了 make_tma_copy() 函数，最终生成了符合硬件约束的最优 Box Size。

TMA 除了可以做到简单的数据 copy，还可以做到一次 load 数据写出到多个 SM 上的 shared mem 上（要求这些 SM 在同一个 cluster 上）。例如，我们在 gemm kernel 这类存在数据复用的场景，可以做到一个 TileA 广播给 SM0 和 SM1 使用，这样对于 SM0 和 SM1 来说，只需要各自读取 1/2 TileA 的数据并 multicast 给对方，就可以正常完成后续的 gemm 操作了，能大大减少 load 的数据量，是一个非常实用的功能。

在 CuTe 中，其做法就是将 TileA 的上半部分交给 SM0 copy，TileA 的下半部分交给 SM1 copy。这时候虽然 SM0 和 SM1 的 tma 各自只 load 了 1/2 TileA 数据，但是每个 SM 都收到了 TileA 的数据量，所以我们设置 transaction bytes 也仍然是完整 TileA 的量，原则就是每个 sm 并不关心自己的 TMA 发了多少数据的 copy，只管自己接收到了多少数据。

读者可能有疑问，为什么不直接拆分为左右各一条 tma copy 然后 multicast？我猜测这里是为了通用性考虑，因为 tile size 可能是 64 的奇数倍，例如 192，这时候如果做左右拆分，会有一条 copy 指令只有一个 SM 在做，另一个 SM 在等待，且不说性能会不如均分（max(2, 1) > mean(2, 1)），可维护性也是个大问题。

我们可以将上述 make_tma_copy 描述过程展示如 Fig.4。

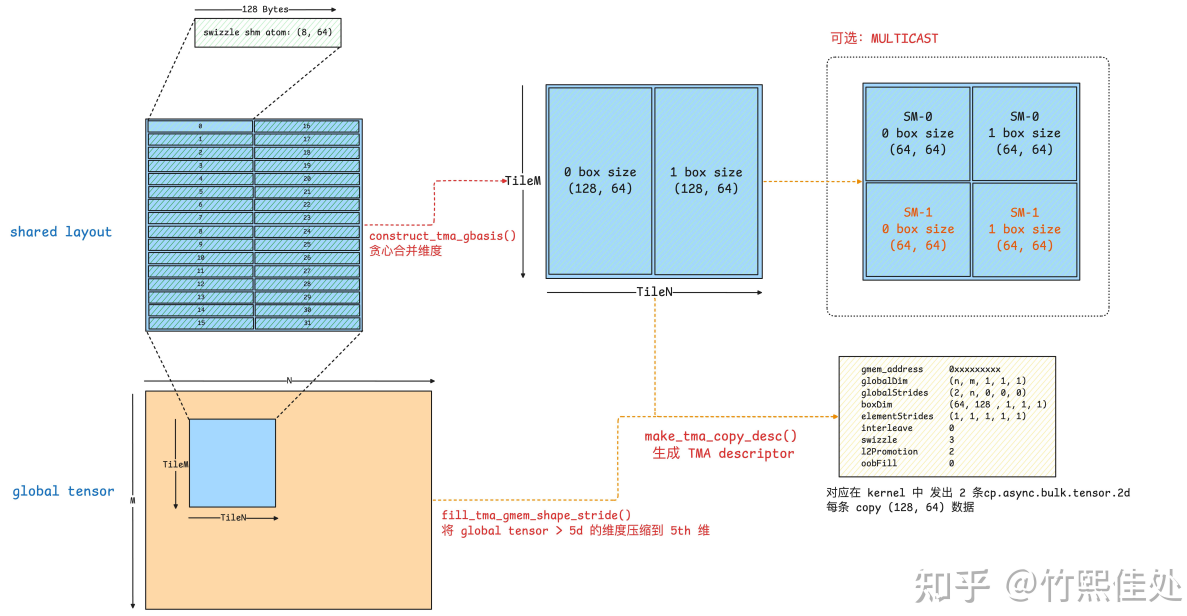


图 18.4: Figure 4. make_tma_copy() 执行流程。包含贪心合并 TMA 维度，descriptor 的生成，multicast 处理等流程，得到最优的 copy 方案。

在上述的介绍中，我们提到 **fill_tma_gmem_shape_stride()** 这个函数会对超过 5D 的数据进行压缩，在此我们给读者朋友一个挑战：

尝试思考一下其是如何做这个压缩的（为什么用到 GCD 最大公约数），这个压缩并不完美，其潜在问题是什么。如果读者对这个部分也能思考清楚，可以认为你对 TMA 有了相当清楚的认知，我们在评论区给出答案，也欢迎大家给出自己的看法。

18.6 TMA proxy & fence

通过 TMA 来 load / store shared mem，和正常 sts 以及 lds 指令访问 shared mem 所走的数据通路（后者走 Load Store Unit, LSU）是不同的，但是其访问的数据又有可能是同一段地址。这时两个访问方式就被称为两个 Proxy：正常走 LSU 的 proxy 被称为 **Generic Proxy**，TMA 走的 proxy 被称为 **Async Proxy**。

不同 proxy 访问同一块地址会出现内存可见性的问题，因为硬件之间无法即时同步这个地址的读写状态，因此要加一个 **fence** 来保障跨 Proxy 的内存可见性。

例如，在 GEMM Kernel 中，通常在计算完成需要输出结果到 Global Memory 时，会先从 Register Copy 到 Shared Mem 中（不管调用的是 stmatrix 还是 sts 指令，其对应的硬件单元都是 LSU），然后再经由 TMA 发起从 Shared Mem 到 Global Mem 的 Copy。这里为了保证 TMA 搬运的数据是 LSU 刚刚写入的最新的数据，必须要在两者之间加一个 TMA Fence，其指向的 ptx 代码如下：

```
// Issue a shared memory fence for async operations
CUTLASS_DEVICE
void fence_view_async_shared() {
#if CUDA_BARRIER_ENABLED
    cutlass::arch::synclog_emit_fence_view_async_shared(__LINE__);
    asm volatile (
        "{\n\t"
        "fence.proxy.async.shared::cta; \n"
        "}"
        ::
        : "memory");
#else
    CUTLASS_NOT_IMPLEMENTED();
#endif
```

```
15 }
```

这里值得注意的是，**fence 和 sync (___syncthreads 或是 named barrier sync) 是两个不同维度的原语**。

`__syncthreads` 仅保证了 Generic Proxy 下的线程同步和内存一致性（即，确保所有线程都已经把数据写进 shared mem），但它**并不保证**跨 Proxy 的可见性。因此，如果只做 sync 不做 fence，TMA 单元可能看不到 LSU 刚刚写入的数据；反之，如果只做 Fence 不做 Sync，TMA 发起时可能其他线程还没写完数据。

所以，正确的流程通常是：先 **sync**（确保数据就绪），再 **fence**（确保 TMA 可见），最后发起 **TMA Store**。那么 sync 的时候为什么不把 fence 一起做了呢？因为这个操作只有在涉及跨 Proxy（如 STS 和 TMA 混用）的特定场景下才需要。如果在所有 sync 中都强制加入 fence，在大量不需要跨 Proxy 交互的场景下（例如纯 CUDA Core 计算）会引入不必要的开销，得不偿失。

18.7　总结

本文介绍了从 Hopper 架构开始引入的 **Tensor Memory Accelerator (TMA)，其出现是为了充分利用 GPU 越来越高的 IO 带宽，确保 IO bound 和 Compute bound kernel 都能达到预期性能**。具体的，其实现了仅需一条指令即可将整块 Tile 搬运至 Shared Memory，从而释放了带宽潜力。

然而，代价是什么？

TMA 的出现深刻改变了 GPU 的编程范式，并引入了一系列新概念，如 descriptor，barrier，proxy 等等，对于一个从 Ampere 架构下切换过来的开发者来说是一个不小的心智负担。

好在我们可以借助 **CuTe** 来驾驭这一硬件特性。因此，我们在本文中重点介绍了 **CuTe 坐标体系**，**MBarrier 的 Phase 翻转机制**，并重点解读了 **make_tma_copy 函数**，其能够针对硬件的维度限制和 Box Size 限制进行贪心的维度合并来最大化带宽，并能够很好的处理 multicast 场景。最后我们讨论了保证 Generic/Async Proxy 内存可见性的 **fence** 机制，确保写出的 kernel 的数据正确性。

笔者的一个感受就是，CuTe 的核心代码确实难读，但是好处是其经过开源的检验，并有 CUTLASS / Flash Atten 这样可以借鉴的高质量代码库。笔者曾经参与维护过一套手写 PTX 搭建的 TMA + WGMMA 类算子，在叠加上各种异步优化之后代码变得非常难维护，经常发现查了一天 bug 发现是某行有个坐标算错了，或是某个 barrier 没同步好。

人的智慧应当用于构建更宏大的系统，而非消耗在底层的比特泥沼之中。**“君子生非异也，善假于物也。”** 愿诸君善用利器，从繁琐的指令细节中解放出来，去追求更具价值的架构创新。**Save code, live long**。

第十九章　写给进阶开发的 CuTe 笔记：tiled mma 的 permutationMNK 参数

作者：竹熙佳处（知乎专栏「CuTe 教程」）　原文：https://zhuanlan.zhihu.com/p/1973526710105419953

19.1　前言

在之前的写给大家看的 CuTe 教程：tiled mma[1][1] 文章中，我们介绍了 permutation MNK （以下记作 PermMNK）作为 tiler 的作用，但是并没有介绍其真正 “permutation” 的功能。

在文章中我们提到，在 cutlass repo 的 issue: What is PermutationMNK in TiledMMA in CUTLASS 3.4 changes?[2] [2] 中，CuTe 的作者 Cris Cecka[3] 进行了解答，其可以做到让每个 thread 做 mma 时可以更灵活的指定自己所负责的数据。但是笔者觉得，其介绍仍然延续了 CuTe 官方文档类似的问题：缺少足够的上下文以及一步步 reasoning 的过程，导致理解起来仍然非常不直观。在本文中，我们尝试用更直白的语言，帮助读者一同理解这个参数的含义以及用法。

值得注意的是，虽然 permutation 的功能在绝大多数使用 CuTe 的场景下都用不到，但是其在一些特殊场景下可能能有奇效。因此本系列文章主要面向 cute 进阶开发者收录于写给进阶开发者的 CuTe 教程[4][3]，作为写给大家看的 cute 专栏[5] [4] 的补充，主要用来解释 CuTe 中不易理解但具有潜力的细节。读者朋友们有感兴趣的内容可以在评论区提出，我们在后续的文章中进行选择梳理。

19.2　动机

我们以 ampere-style fp16 mma 为例，考虑一个场景：一个 warp 完成 2 次 16x8x16 的 mma，得到 16x16 的 C，然后 store 到 global mem。在一些场景下，例如 M & N 很大，K 很小的场景下，这时候 STG 指令的发射可能构成瓶颈，因此我们希望 store 的时候，用到更大字长的 STG，如 STG.128 来提升 store 的效率。

但是如我们在 tiled mma 篇章中介绍的，mma 指令决定了我们的 thread 存放的数据最多只能写出连续的两个 value，如 Fig.1 所示，如果是 float 数值，最大能用的 STG 指令只能是 STG.64。

这时候，我们希望 thread0 要是能得到 C 矩阵中的连续的 4 个 value 就好了。这其实就是 permutationMNK 被提出的动机。类似的情况还包含 w4a8 混合精度 gemm，fp8 attention（QK 乘之后的 P 矩阵作为 fp8 mma 输入，需要 4 个数连续），也可以尝试用这个方法。

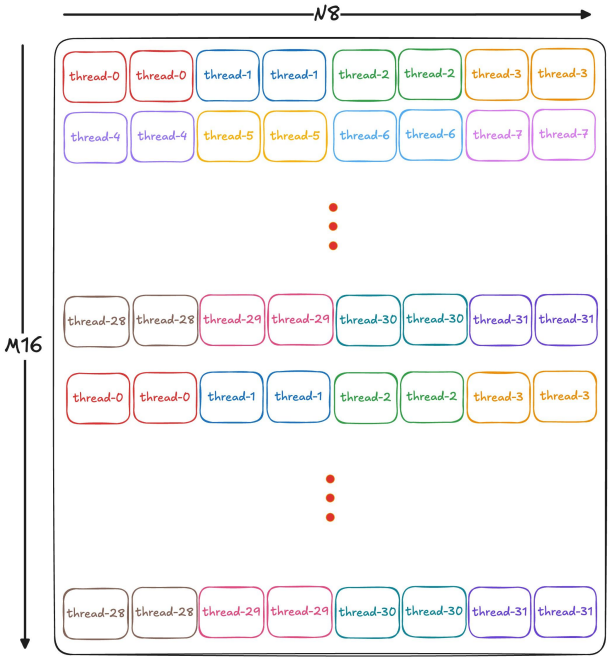
19.3　如何使用 PermMNK

在 issue [2] 中，Cris Cecka[6]以 fp64 mma 来作为实例以方便可视化，与上述我们给的例子本质是一样的。为展示方便，我们仍然以其在该 issue 中例子上进行分析。注意 fp64 只用于可视化：其 C fragment 里 thread0 本来就有两个连续的 double（16 B），置换并不增加它的 store 字长；store 字长的收益属于 4 字节累加器（STG.64 → STG.128）。首先，在常规的用法中，我们指定 permMNK 为一个普通的 tile 时，对应代码如下：

```
TiledMMA tiled_mma = make_tiled_mma(SM80_8x8x4_F64F64F64F64_TN{},
                                    Layout<Shape<_1,_1,_1>>{},     // AtomLayout
                                    Tile<_8,_16,_8>{});            // Tiler
```

我们观察 mma 的执行过程与 thread 中存放 C 的 reg 数据，可以如 Fig.2 所示：

[1] https://zhuanlan.zhihu.com/p/1937145378446226159
[2] https://link.zhihu.com/?target=https%3A//github.com/NVIDIA/cutlass/discussions/1345
[3] https://link.zhihu.com/?target=https%3A//github.com/ccecka
[4] https://zhuanlan.zhihu.com/column/c_1973695125898150444
[5] https://zhuanlan.zhihu.com/column/c_1946580149119202264
[6] https://link.zhihu.com/?target=https%3A//github.com/ccecka



知乎 @竹熙佳处

图 19.1: Figure.1 16x8x16 FP16 mma 指令 C 矩阵对应的 thread-value 映射关系。可以看到每个 thread 会得到 C 矩阵中特定位置的数据

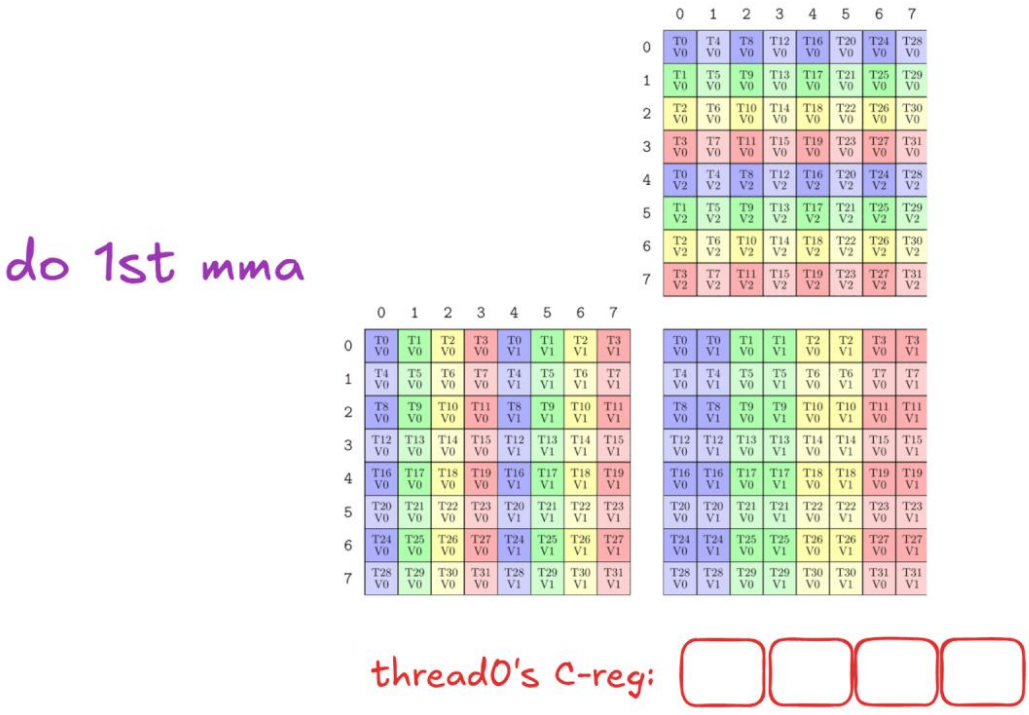


图 19.2: Figure.2 常规调用 2 次 mma，thread0 拿到的 C 矩阵数据对应坐标为 {(0, 0), (0, 1), (0, 8), (0, 9)}，无法作为连续的数据写出

如我们所预期的，每个 thread 拿到的数据在写出时是不连续的。原因即是我们遵循 mma 的标准做法：对 B 矩阵进行了连续的读取，并做了两次连续的 mma。

但是，如果我们可以**让两条 mma 指令，不处理 A & B 矩阵在物理上连续的数据块，而是交错式的取数据，计算的结果也交错写出到 C 矩阵中；然后再执行第二条 mma，依旧是读交错的位置，得到的结果交错写入 C reg 矩阵**。这个过程如 Fig.3 所示：

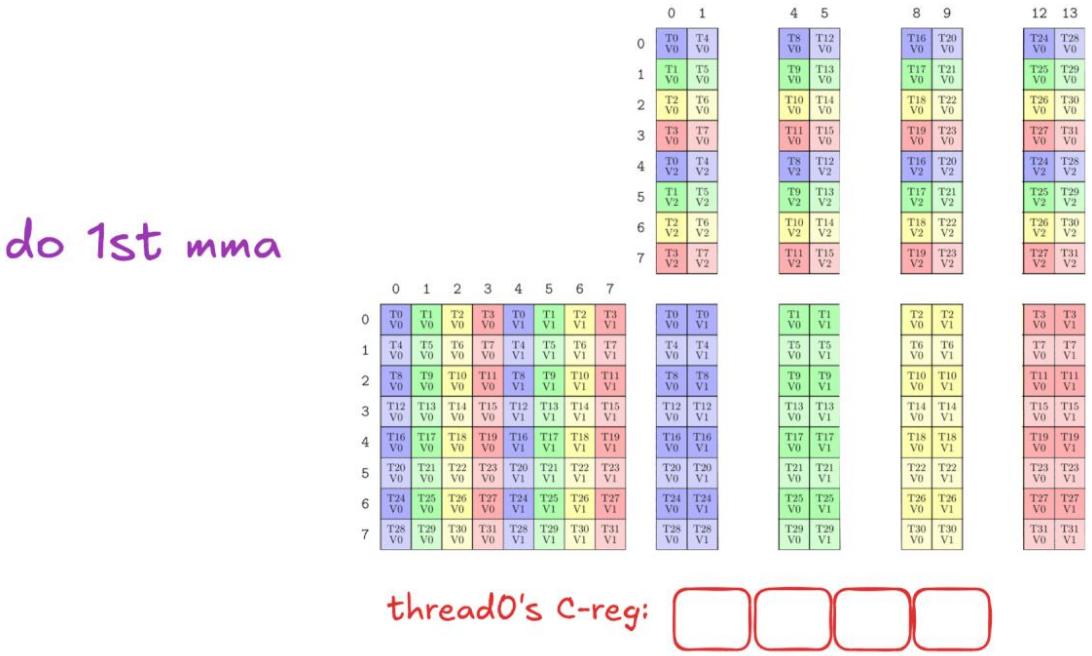


图 19.3: Figure.3 交错调用 2 次 mma，thread0 拿到的 C 矩阵数据对应坐标为 {(0, 0), (0, 1), (0, 2), (0, 3)}，可以作为连续的数据写出

我们发现，通过这样计算出来的 C 矩阵，各个 thread 能够拿到连续的 4 个数。以本节 fp64 画图为例，$4\times8\,\mathrm{B}=32\,\mathrm{B}$，需要两次 STG.128；换成 fp32 累加器，$4\times4\,\mathrm{B}=16\,\mathrm{B}$ 正好一次 STG.128（fp16 的 4 个值只有 8 B，对应 STG.64）。那么这个写法，对应到 permutationMNK 的参数如下，其就是在 N 维度做了重排：

```
TiledMMA tiled_mma = make_tiled_mma(SM80_8x8x4_F64F64F64F64_TN{},
                                    Layout<Shape<_1,_1,_1>>{},     // AtomLayout
                                    Tile<_8,                       // Permutation on M, equivalent to
    8:1, identity
                                    Layout<Shape <_2,_4,_2>,
                                           Stride<_1,_4,_2>>, // Permutation on N, size 16
                                     _8>{});                  // Permutation on K, equivalent to 8:1,
    identity
```

那么再考虑，用于 s2r copy 的 LDSM 指令在这个时候是否还可以使用？笔者认为是可以的，因为虽然我们在 N 方向上有一些交错访问，但是交错的粒度，都是连续的 2*16 fp16 元素，因此 LDSM 仍然可以使用。不过，由于从 shared mem 中 load 的数据不再是紧密排列的了，常规的 swizzle BMS 参数需要重新调整一下才能做到 bank-conflict-free。

进一步的，我们思考，想要完成上述功能，除了 permMNK 之外，原本的 g2s -> s2r -> gemm 的 mainloop 代码还需要做什么改动吗？笔者认为应该是不需要的，我们观察 permMNK 在 tiled mma 的具体实现，其是通过 logical_divide 改动了 TV-layout，其作用会自然而然通过 partition 和 retile 的逻辑传递到 s2r tiled copy 上。

```
thrfrg_C(CTensor&& ctensor) const
{
  CUTE_STATIC_ASSERT_V(rank(ctensor) >= Int<2>{});
  // Reorder the tensor for the TiledAtom
  auto t_tile = make_tile(permutation_mnk<0>(),
                          permutation_mnk<1>());
  auto t_tensor = logical_divide(ctensor, t_tile);                 // (PermM,PermN)

```

```
  // ...
}
```

为了验证”permMNK 修改并不影响 ldmatrix 以及 mainloop 流程” 的想法，我们给出验证代码，有兴趣的读者可以尝试复现：

https://github.com/chrisHuxi/CuTe-coding-toolings/blob/feature/permMNK_tiled_mma/permMNK_example/perm_tiled_mma.cu

此外，有读者朋友 @6666[7]在评论区也给出了 CuTeDSL 的实现，发现在设置 permMNK 后再调用 ldmatrix 会产生报错，笔者初版判断是 CuTeDSL 本身的适配问题，待我们研究清楚后再更新结论到本文中，有熟悉 CuTeDSL 的读者也欢迎在评论区讨论。

19.4　总结

本文中，我们介绍了 Tiled mma 中 permutationMNK 参数的作用。其通过修改 TV-layout，做到对每个 thread 负责输出 C 矩阵的位置进行微调，从而赋予了 load / store 时更多的灵活性。从这个例子我们可以看到，CuTe 的抽象确实足够完备，但高度的抽象也隐藏了诸多细节，确实不易理解，希望我们的梳理能够帮助大家进一步加深对 CuTe 的理解。

19.5　Reference

[1] 写给大家看的 CuTe 教程：tiled mma[8]
[2] [QST] What is PermutationMNK in TiledMMA in CUTLASS 3.4 changes?[9]
[3] 写给进阶开发看的 CuTe 教程[10]
[4] 写给大家看的 CuTe 教程[11]

[7] https://www.zhihu.com/people/940468f485e92a6af2c696581a440e60
[8] https://zhuanlan.zhihu.com/p/1937145378446226159
[9] https://link.zhihu.com/?target=https%3A//github.com/NVIDIA/cutlass/discussions/1345
[10] https://zhuanlan.zhihu.com/column/c_1973695125898150444
[11] https://zhuanlan.zhihu.com/column/c_1946580149119202264

第二十章　写给大家看的 CuTe 教程：async pipeline

作者：竹熙佳处（知乎专栏「CuTe 教程」）　原文：https://zhuanlan.zhihu.com/p/2024832248386528275

20.1　动机

在之前的文章 [1][2] 中，我们介绍了 Hopper 架构引入的 TMA 来提升数据搬运的能力。TMA 让我们能以一条指令搬运一整块 Tile 数据到 Shared Memory，mbarrier 则提供了约束异步操作的顺序的能力。

由于本文介绍的 async pipeline 围绕 mbarrier 构成，我们首先高度精简地回顾一下：

> mbarrier 是一个 64-bit 的 shared memory 变量，内部包含 phase、expect arrival count、expect transaction bytes 等字段。当实际的 arrival count 和 transaction bytes 都达到预期值时，phase 位翻转，正在 wait 这个 barrier 的操作即可继续执行。更多细节的可以参考：

写给大家看的 CuTe 教程：TMA Copy

构建于 TMA & mbarrier 两大设计之上，针对 gemm / attention 这类 tensor core kernel，NVIDIA 在 Hopper 架构中推出了 Warp Specialization 编程范式，并在后续架构延续下来。以 gemm 为例来说明：

- 让一小部分 Warp 专门做 Producer，只负责向 TMA 发送搬运指令，写 A/B tile 数据到 Shared Memory；
- 让绝大部分 Warp 专门做 Consumer，只负责从 Shared Memory 读数据做 mma。

Producer 和 Consumer 各自独立运行，天然就具备了数据搬运与计算的重叠能力，同时也大大简化了编译器优化的压力。然而，这也引入了一个新问题：两者之间如何协调？具体地，我们需要解决两个问题：

- **Consumer 怎么知道数据已经 ready 了？**——不能在 Producer 还没搬完的时候就去读 Shared Memory。
- **Producer 怎么知道旧数据已经被用完了？**——不能在 Consumer 还在读 Shared Memory 的时候就覆盖写入下一轮数据。

如果只有一块 Shared Memory Buffer，那 Producer 搬完就得等 Consumer 算完，Consumer 算完又得等 Producer 搬下一轮，成为了串行模式，自然不是高效的。

因此，我们引入 multi-stage，即，准备多块（如，3 块）Shared Memory Buffer，生产者往 Stage 1 写数据的同时，消费者可以从 Stage 0 的 shared mem 读上一轮的数据，两者在不同的 Stage 上交替工作，形成 load 数据以及计算的 overlap。即，我们需要维护一个 stage-id 从 0 -> 1 -> 2 -> 0 -> ⋯的环形 buffer，我们在每一次 stage-id 变化时及时 arrive barrier，以让依赖这个 buffer 的 warp 可以解除 wait，继续执行。

朴素地，我们可以手动地维护 stage-id 的循环逻辑与 barrier 的状态，但是真正写过复杂 tensor core kernel 的读者，大概会发现，这不是一个好的选择，一个不留神就容易 hang 住，或是 arrive 时机不对导致报错或是结果异常。因此，我们可以构建一个结构体来管理 barrier 以及 multi-stage 的状态、协调生产者和消费者在各个 Stage 上的访问权限，此即 AsyncPipeline。

此外，AsyncPipeline 的适用范围不限于 TMA + MMA。MMA + Epilogue、Pingpong MMA 等场景同样可以用 Pipeline 来做交互，只要存在异步的“生产”和“消费”关系，AsyncPipeline 都能派上用场，因此我们在完成复杂 tensor core kernel（尤其是 sm100，引入了 umma 以及 tensor memory 后，tma -> umma -> tmem -> epilogue 这套流程需要维护的异步状态变得异常烧脑，更不要说 flash atten v4 这样引入 softmax + 两次 umma 的用法），AsyncPipeline 就更是不可或缺的基础组件。

因此，在本文接下来的内容中，我们结合 cutlass 中的具体实现，来了解其用法以及不可绕过的实现细节。

20.2　基本用法

我们首先来看一个简单的例子，借助其经典用法下 TMA + MMA，来了解 **AsyncPipeline 的基本用法**，然后逐一梳理各细节。

```
// ===== 初始化 =====
PipelineTmaAsync pipeline(shared_storage, params);
PipelineState pipe_read;
PipelineState pipe_write;

  // --- 生产者：搬数据 ---
  if (is_producer) {
    pipeline.producer_acquire(pipe_write);          // wait EmptyBarrier
    copy(tma_load.with(full_barrier(pipe_write)));  // issue tma copy
    ++pipe_write;                                   // advance to next stage
  }

  // --- 消费者：算数据 ---
  if (is_consumer) {
    pipeline.consumer_wait(pipe_read);               // wait FullBarrier
    gemm(acc, smem_A, smem_B);                       // issue mma
    pipeline.consumer_release(pipe_read);            // arrive EmptyBarrier
    ++pipe_read;                                     // advance to next stage
  }
```

20.3　AsyncPipeline：mbarrier 的封装

Async Pipeline 本质上是一个封装了 mbarrier 的结构体，并提供了对 barrier 操作的各项接口。那么，我们来推导一下这个结构体需要什么成员变量以及函数。首先我们思考，在 TMA + MMA 场景下，我们应该用多少个 mbarrier 来确保数据的使用不重不漏呢?

答案是：每个 Stage 都配备一对 barrier，分别用来约束：

- MMA warp 在此等待，直到 “TMA 已经 load 完数据到 buffer 了，MMA 可以使用了”，我们记为 **full barrier；**
- TMA warp 在此等待，直到 “MMA 已经使用完 buffer 里的数据了，TMA 可以 load 下一轮数据了”，记为 **empty barrier**。

所以，我们可以看到，Pipeline 的成员变量 `SharedStorage` 结构体如下：

```
struct SharedStorage {
  FullBarrier  full_barrier_[Stages];
  EmptyBarrier empty_barrier_[Stages];
};
```

除了 barrier 变量本身，这个结构体还提供对 barrier 操作的一系列函数，包括：

```
void producer_acquire(state);
// producer 等待当前 state 的 empty barrier 就位

void producer_commit(state, uint32_t bytes);
// 非 TMA pipeline 用于提交 full barrier; TMA load 场景下为 NOP（full barrier 由 expect_tx/complete 推进）

void consumer_wait(state);
// consumer 等待当前 state 的 full barrier 就位

void consumer_release(state);
// consumer 在发出当前 state 的数据消费后 arrive 对应的 empty barrier
```

我们逐个说明：

- **producer_acquire(state)**：**producer 等待当前 state 的 empty barrier 就位**，即，其在等待 consumer 将当前 stage 的 shared mem buffer 使用完毕。
- **producer_commit(state)**：**producer 在发出当前 state 的数据生产任务后 arrive 对应的 full barrier**。这一步在 TMA load 场景下是不需要的（当然写了也没事，会被编译为一个 NOP 操作），因为 TMA load 的 arrive 在 `copy()` 函数中就已经提交了；但在如 umma + load tmem 这类 pipeline 场景下则是必要的。
- **consumer_wait(state)**：**consumer 等待当前 state 的 full barrier 就位**，即，等待 producer 将当前 stage 的 shared mem buffer 填满。
- **consumer_release(state)**：**consumer 在发出当前 state 的数据消费后 arrive 对应的 empty barrier**。

有了这 4 个函数，我们实际上再回头来看我们的代码，就很清晰了。语义上，每个 warp 每次都是 acquire 到需要的资源之后，就开始执行（copy/gemm），执行完成后 release 当前的资源。

我们可以借助 Fig.1 来理解这个过程中是如何使用 AsyncPipeline 的：

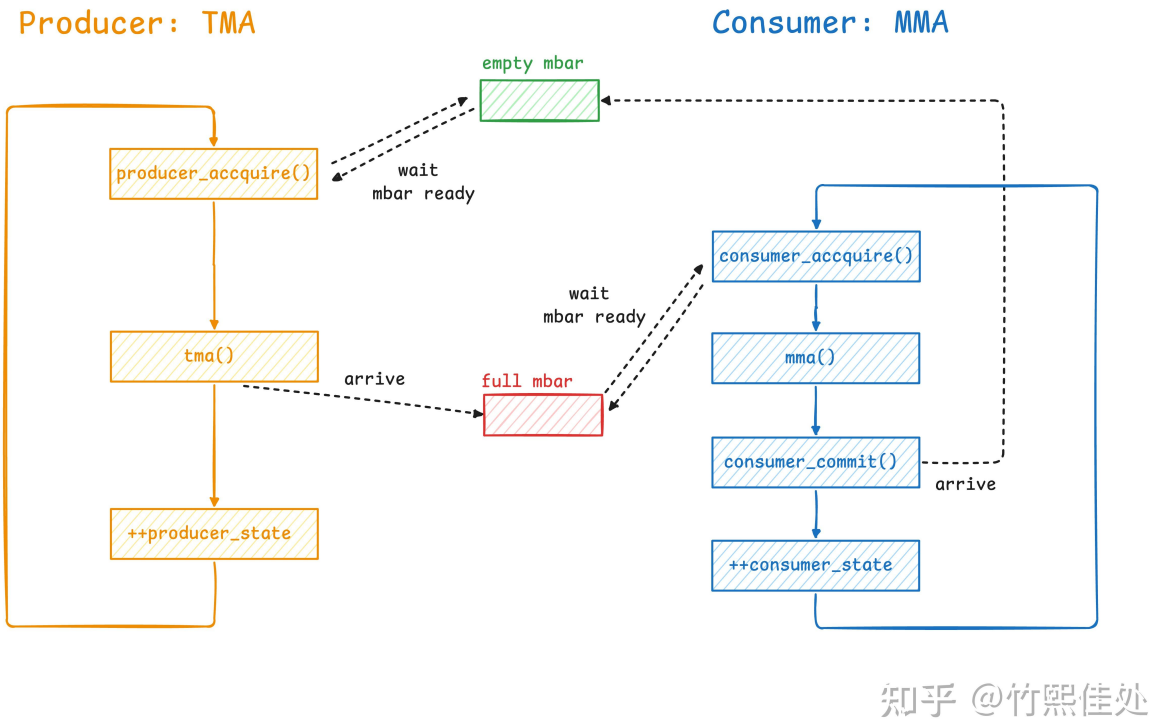


图 20.1: Figure 1. Producer 与 Consumer 通过 AsyncPipeline 交互示意图；每个 Stage 都配备一对 barrier，full & empty，分别用来表示 tma 已经填满当前 stage 的 shared mem buffer & mma 已经使用完当前 stage 的 shared mem buffer

此外，由于一些指令如 tma copy，是由单 thread 发起的，而如 epilogue 或是 softmax 这样的指令，仍然由多个 thread 共同完成。

这涉及到我们对 mbarrier 初始化，以及实际 arrive 的时候区别对待（即，arrive 一次还是多次），因此这就要求我们的 pipeline 带有身份信息，例如，“当前 warp 是 producer 还是 consumer?” 或是 “这个 thread 是用来发射 TMA 指令的 leader thread 吗?”。那么这段身份标识，就需要在 Pipeline 初始化的时候以 param 的方式传递给 pipeline 结构体。其核心代码可以展示为如下：

```
enum class ThreadCategory {
    NonParticipant,
    Producer,
    Consumer,
    ProducerConsumer
  };
```

```

  struct Params {
    uint32_t transaction_bytes = 0;
    ThreadCategory role = ThreadCategory::NonParticipant;
    uint32_t is_leader = 0;
    uint32_t num_consumers = 0; // Number of consumer threads
    uint32_t num_producers = 1; // Number of producer threads
    int initializing_warp = 0;
  };

// PipelineTmaAsync constructor
  template<class ClusterShape, class InitBarriers, class InitMasks>
  CUTLASS_DEVICE
  PipelineTmaAsync(SharedStorage& storage, Params params, ClusterShape cluster_shape, InitBarriers = {},
    InitMasks = {})
      : params_(params)
      , full_barrier_ptr_(&storage.full_barrier_[0])
      , empty_barrier_ptr_(&storage.empty_barrier_[0]) {
      // ...
  }
```

20.4　PipelineState：流水线的状态机

我们观察到，pipeline 结构体提供了对 mbarrier 的各种操作，但在 multi-stage 下，我们仍然需要手动映射当前的 i-k-tile、i-stage 以及预期的 barrier 状态，实际上仍然是比较烧脑的。笔者曾经因为历史原因维护过一套不借助 cutlass 的函数写的 sm90 flash atten 算子，各种 barrier 状态的转移分析和 debug 真是遭老罪了。

因此，CuTe 中提供了一个新的结构体来完成状态的管理：`PipelineState`。其核心定义如下：

```
// Circular Buffer Index + Associated Phase
// Assumes only one operation possible - i.e., ++
template<uint32_t Stages_>
struct PipelineState {
  static constexpr uint32_t Stages = Stages_;

  int index_ = 0;
  uint32_t phase_ = 0;
  uint32_t count_ = 0;

  CUTLASS_DEVICE
  PipelineState(int index, uint32_t phase, uint32_t count)
    : index_(index)
    , phase_(phase)
    , count_(count) {}

  CUTLASS_DEVICE
  void operator++() {
    if constexpr (Stages > 0) {
      ++index_;
      ++count_;
      if (index_ == Stages) {
        index_ = 0;
        phase_ ^= 1;
      }
    }
  }
};
```

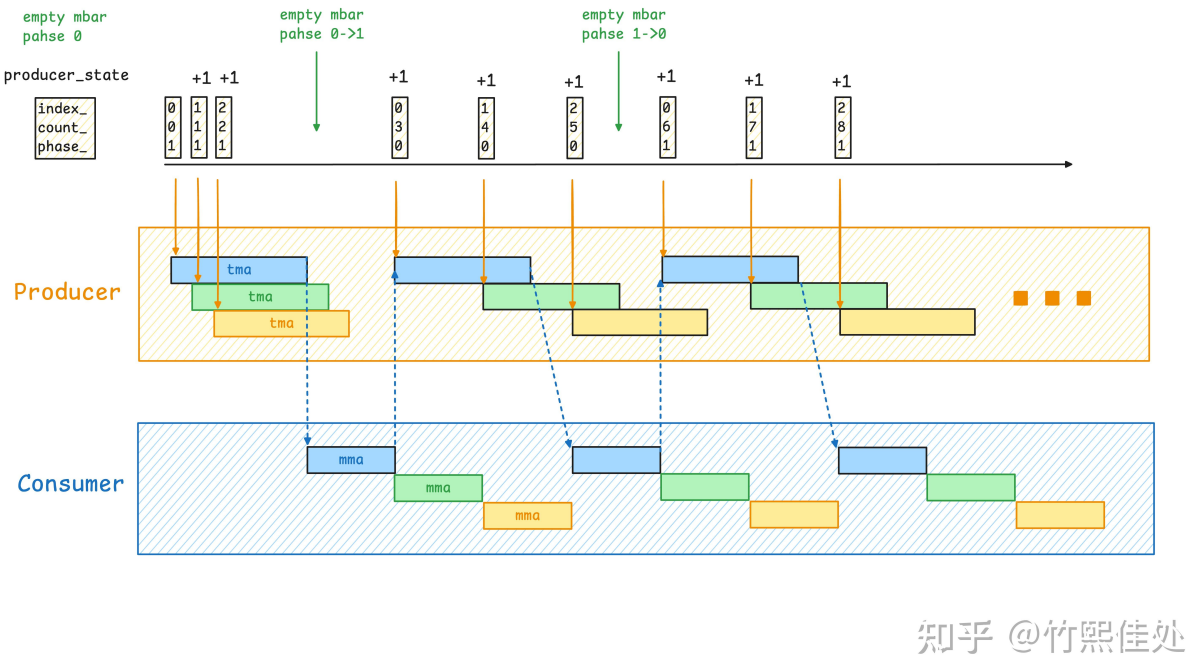
具体来说，PipelineState 只是维护了三个变量：

- count_：自流水线启动以来，总共经历的 k-tile id，这个值是一直增长的。
- index_：当前 stage 的 id，如在 3-stage 设定下，其就是 0->1->2->0->1 …的循环。
- phase_：当前 k-tile 下，我们等待的 mbarrier phase，只有 0 和 1 两个取值。若为 1，只有在 mbarrier phase 也为 1 的时候进行阻塞，mbarrier phase 为 0 的时候放行；反之则反。

index_ 和 count_ 的语义是较平凡的，关键在于 phase_。我们用一个 3-Stage 的 producer_state 的例子，随着 k-tile 的递增，来观察它们的变化规律。

这里我们需要提一下一个细节（感谢读者朋友 @ 最光阴[1] 的提醒），由于 empty barrier 在初始化时，phase 位为 0，而我们第一次发起 TMA 时，shared mem buffer 都是空的，也就是不需要阻塞，所以我们会将 producer_state 的状态初始化为 { index_ == 0, phase_ == 1, count_ == 0 }，否则 Producer 会等待一个无法解锁的 empty barrier，一上来就 hang 住。这个 phase_ == 1 在 producer_state 上是必要的，在 consumer_state 中则不必要。

我们将时间线展开如 Fig.2 所示：



知乎 @竹熙佳处

图 20.2: Figure 2. Producer 与 Consumer 在 k-tile 循环过程 & producer_state 中各变量变化情况；index 指向 stage-id，0,1,2,0,1,2 循环，count 随着 k-tile 进行不断递增，phase 则是每 3 次翻转一次。第一次发起 TMA 时，不需要阻塞，所以我们会将 producer_state 的 phase_ 初始化为 1

可以看到，从 0 开始，在每次完成 3 次 k-tile 后（即，所有 stage 的 shared mem buffer 都被用到了），phase_ 会发生一次位翻转 (0->1 或是 1->0)，这实际上代表着我们预期当前的 mbarrier 的状态应该发生了变化。因为在 multi-stage-buffer 场景下，同一个 stage 会被反复使用（第 0 轮用 stage 0，第 3 轮又轮到 stage 0），如果没有 phase 状态的标记，硬件就无法区分” 这是新一轮的通知” 还是” 上一轮留下来的残余状态”。

phase 的巧妙之处在于，它让 barrier 的状态具备了代际区分能力，而无需在每轮开始前显式清理 barrier——硬件自动通过 phase 比对完成这一切。只有当 mbarrier 本身的 phase 位和我们在 PipelineState 里的这个需要等待的 phase_ 不一致时，wait 之后的代码才能被放行。因此，通过这样一个结构体，我们就能很轻松的完成 multi-stage 下 mbarrier 状态的管理。

[1] https://www.zhihu.com/people/7bb7911bb19edeb4bacade3815513a47

20.5 总结

Mbarrier 的引入，正式将 CuTe kernel 转变为 warp-specialized + producer & consumer 范式，然而多个 producer & consumer 的设定，叠加 multi-stage buffer，让 mbarrier 的状态维护变得极为复杂。

为了解决这个问题，cutlass 提供了 AsyncPipeline 结构体，提供了对 full_barrier 和 empty_barrier 操作的封装。

进一步地，我们在使用 AsyncPipeline 时，通常会同时再为 producer & consumer 分别定义一个 PipelineState，用以管理 k-tile & i-stage & 预期的 phase 三者的映射关系。

具体来说，我们只需在每次循环后将 PipelineState 作为参数传给 AsyncPipeline 的对应函数（acquire/commit/release），在每次循环结束前 ++PipelineState，就能让维护状态的复杂度在封装好的函数中消化掉。

至此，我们已经梳理完 CuTe 中写好一个 tensor core kernel 所需的基本概念，包括 tiled_copy[3]，tiled_mma[4]，tma_copy[1] 以及 async_pipeline，因此接下来的文章，我们来尝试用这些组件搭建一个 SOTA 级别的 kernel（e.g gemm / attention @ RTX5080）。

第三部分

NVidia GPU 指令集架构（reed）

第二十一章　NVidia GPU 指令集架构-前言

作者：reed（知乎 @reed-84-49，专栏「CUDA 高性能编程」）　原文：https://zhuanlan.zhihu.com/p/686198447

2017 年的 ACM 图灵奖颁给了 John L. Hennessy 和 David A. Patterson 表彰他们在计算机架构开拓性的贡献。在他们共同撰写的文章“计算机架构的新黄金时代”中，详细介绍了指令集架构的发展和未来的机遇点。此处我们引用文中的指令集架构（Instruction Set Architecture）定义：“Software talks to hardware through a vocabulary called an instruction set architecture (ISA)“，和 DSA（Domain Specific Architecture）的优势：1. 针对特定领域更高效的并行模式，2. 对内存层级更有效的利用，3. 某些场景可以使用更低的精度，4. DSL（Domain Specific Language）可以更好的暴露硬件能力。

对于高性能计算（High Performance Computing）和以深度学习（Deep Learning）为核心承载的人工智能，NVidia GPU 在算力输出方面扮演着极其重要的角色。NVidia GPU 可以认为是图形和深度学习领域的 DSA 架构，即 Domain 为 Graphic 和 Deep Learning 的 Specific Architecture，自然地前文提到的 DSA 架构的优势 NVidia GPU 也都有享用。指令集是软件和硬件沟通的“词汇”，我们必须用这套语言体系来和计算机硬件进行沟通，将我们的算法逻辑变换成硬件可理解的“词汇”，硬件通过这套词汇便可以得到我们所需要的算法逻辑结果。如图 1 所示，指令集体现了硬件能力的暴露，通过指令集暴露了硬件提供的加法计算、减法计算和乘法计算能力，图中的硬件没有提供除法计算能力，则软件在遇到除法问题时需要用其他指令来模拟实现。

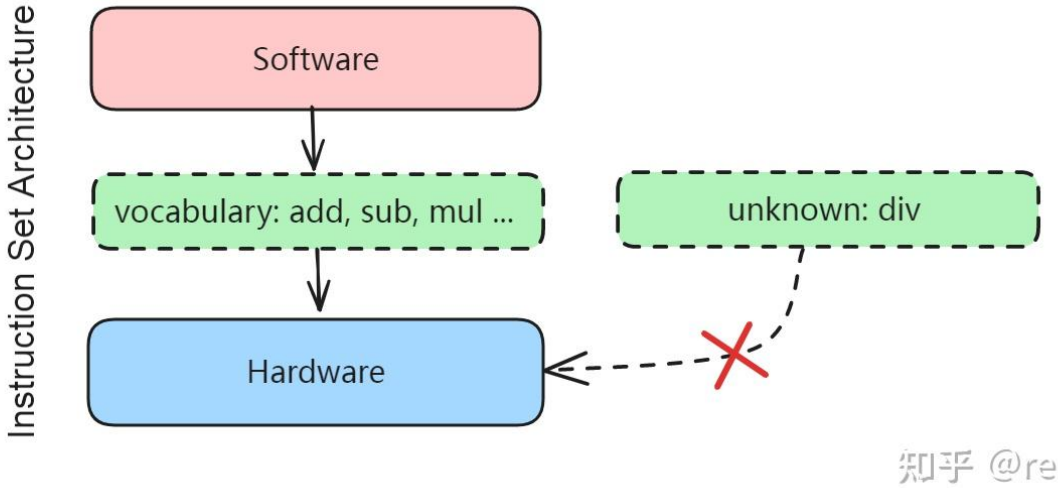


图 21.1: Figure-1. Instruction Set Architecture

指令集架构是硬件能力的暴露，了解指令集架构能够让我们更清楚的知晓硬件所提供的基础能力，更好的辅助我们选择硬件友好的算法，同时了解指令集架构可以让我们选择更高效的指令，继而提升软件运行效率。

对于 NVidia GPU 而言，其软件部分的核心语言为 CUDA，硬件架构的指令在不同代际是不同的（如 Tesla，Fermi，Kepler，Maxwell，Pascal，Volta，Turing，Ampere，Hopper，Blackwell），本文将基于 Ampere 架构进行介绍。图 2 展示了 NVidia GPU 由软件编程到硬件可执行的编译流程，其中 CUDA 是用户可编程语言，使用该语言编写的程序可以通过 NVCC 编译器编译成 PTX 指令，PTX 指令是硬件无关的（实际上也有版本）用于屏蔽硬件差异，PTX 指令通过 PTXAS 汇编工具可以汇编成 SASS 指令，SASS 指令可以交由硬件执行，SASS 指令是硬件相关的。我们所要研究的指令集则指 SASS 指令这一层次，因为该层次的指令直接能被硬件执行，是硬件能力的最基本抽象。

值得注意的是，NVidia 并没有官方介绍 SASS 的文档，只在介绍 cuda binary utilities 的时候简要提及，本文内容多为利用 cuobjdump 反汇编 libcublas、libcublasLt、libfft 等官方库、利用 ncu 分析实际用例得到，同时和 PTX 文档相互对应印证，其中也不乏猜测和臆断的部分。

本系列文章将以指令分类的形式介绍 NVidia Ampere GPU 的指令集架构，同时在介绍具体指令时会结合实际场景分析潜在的性能风险和优化点。在文章结构方面主要有包含寄存器部分，该部分分为通用寄存器、特殊寄存器、Uniform

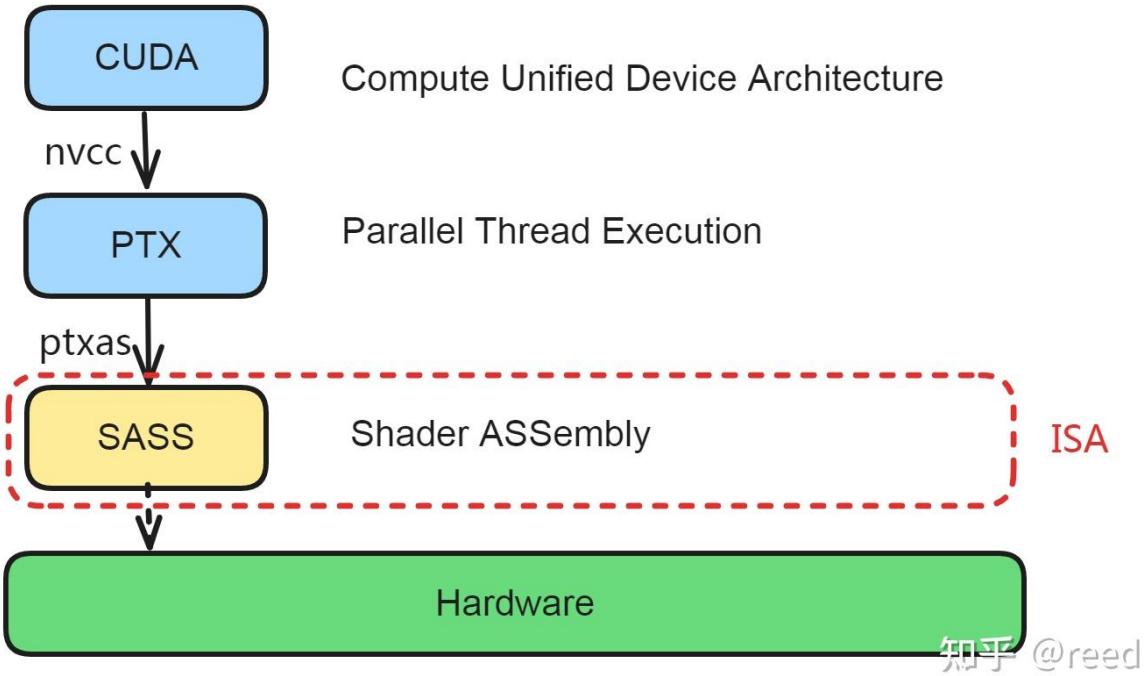


图 21.2: Figure-2. NVidia GPU Compilation and Execution Phase

寄存器、Predication 寄存器；常量 Cache 部分，整数操作指令、浮点操作指令、BIT 操作和逻辑指令、特殊函数指令；分支和控制指令；数据加载和存储指令；Warp Level 指令；Atomic 指令。希望读者了解完该系列文章后能够回答类似如下问题："SASS 中 PRMT 指令很多，是如何产生的，如何优化?"，"有了 half 类型为什么还需要 half2 类型?"，"整数除法是如何实现的?"，"浮点除法的效率如何?"，"Flash Attention 中的快速 EXP 指数是怎么回事，在什么场景下结果会有问题?"。

本文为该系列的前言，后续部分我们将分不同章节重点介绍如下内容

- 通用寄存器
- 特殊寄存器
- Uniform 寄存器
- Predication 寄存器
- 常量 Cache
- 整数操作指令
- 浮点操作指令
- BIT 操作和逻辑指令
- 特殊函数指令
- 分支和控制指令
- 数据加载和存储指令
- Warp Level 指令
- Atomic 指令

总结

指令集架构是软件和硬件沟通的“词汇”，熟悉并了解这些词汇（ISA）才能更好的将软件问题通过这些指令映射到硬件上，同时了解 NVidia GPU ISA 也能帮助我们设计我们自己的 GPU、NPU ISA。

参考

https://amturing.acm.org/byyear.cfm
https://dl.acm.org/doi/10.1145/3282307
https://docs.nvidia.com/cuda/cuda-binary-utilities/index.html

第二十二章　NVidia GPU 指令集架构-寄存器

作者：reed（知乎 @reed-84-49，专栏「CUDA 高性能编程」）　原文：https://zhuanlan.zhihu.com/p/688616037

指令集是软件和硬件沟通的“词汇”[1]，在这个沟通过程中，具体的输入输出参数体现为寄存器，作为芯片上最基础的存储机构，了解它是学习具体指令的基础。本文将重点介绍 NVidia GPU 上的寄存器体系。针对不同的场景和目的这些寄存器包括：通用寄存器、特殊寄存器、Predicate 寄存器、Uniform 寄存器。文章结构方面本文首先通过介绍 Load-Store 架构引入寄存器然后介绍了寄存器表示的程序状态和 GPU 中延迟隐藏，之后分别介绍了 NVidia GPU 中的各类寄存器。

22.1　Load-Store 架构

Load-store 架构，也叫做寄存器-寄存器架构，是计算机指令集体系架构中的重要形式。该架构中，所有的计算类指令的源操作数和目的操作数都必须是寄存器。内存和寄存器之间的通信通过独立的 Load 和 Store 指令完成。Load-store 概念是 RISC（Reduced Instruction Set Computer）架构的基本概念之一。NVidia Ampere 及其之前的 GPGPU 架构基本符合 Load Store 架构的定义（有一点不符合的是常量内存），由于 GPU 为了提高内存的访问效率和数据局部性，内存层次相较于传统的 CPU 架构会多一些，所以其 Load/Store 指令也区分为全局内存（global memory）、共享内存（shared memory）、寄存器溢出或者局部动态寻址数组引入的 Local Memory 以及针对 Tensor Core 数据搬运的指令 ldmatrix。整体 Load、Store 指令和非 Load、Store 指令的操作数情况如下表格所示

指令	类型	目标操作位置	源操作位置
LDG	Load	寄存器	全局内存
STG	Store	全局内存	寄存器
LDS	Load	寄存器	共享内存
STS	Store	共享内存	寄存器
LDL	Load	寄存器	局部内存
STL	Store	局部内存	寄存器
LDSM	Load	寄存器	共享内存
非 Load/Store 指令	算数指令	寄存器	寄存器

Load-Store 架构是一种在许多现代处理器设计中广泛采用的数据访问模型，特别是在精简指令集计算机（RISC）体系结构中。这种架构的主要优势包括：

1. 简化的指令集：Load-Store 架构通过限制指令集只对寄存器进行操作，大大简化了指令集的设计和编码。这使得处理器的设计更加直观，易于理解和实现。
2. 高效的流水线设计：由于所有的内存访问都是通过 Load 和 Store 指令完成的，这使得处理器的流水线设计更加简单和高效。处理器可以独立地优化计算和内存访问操作，减少资源冲突和提高吞吐量。
3. 编译器优化：显式的 Load 和 Store 指令使得编译器更容易进行优化，如指令重排、冗余消除等，以减少不必要的内存访问次数，从而提高程序的运行效率。
4. 提高内存访问效率：通过显式的 Load 和 Store 指令，可以更有效地管理内存访问，减少内存带宽的浪费，并提高内存访问的效率。
5. 寄存器的高效利用：由于 Load-Store 架构鼓励使用寄存器来存储数据，这有助于减少对慢速内存的访问，从而提高程序的执行速度。
6. 清晰的指令格式：Load-Store 架构通常采用三地址格式，这意味着大多数指令只涉及三个操作数（源操作数、目标操作数和立即数），这使得指令格式更加一致和易于处理。

[1] https://zhuanlan.zhihu.com/p/686198447

7. 并行处理能力：由于 Load-Store 架构的简单性和高效的流水线设计，其特别适合于实现并行处理，这对于现代多核和众核处理器来说是一个重要的特性。

8. 更好的错误隔离：在 Load-Store 架构中，由于内存访问和计算操作是分开的，这有助于在发生内存访问错误时更好地隔离错误，因为错误不会影响到计算操作。

总的来说，Load-Store 架构通过其简单性、高效的流水线设计和编译器优化能力，为现代处理器提供了高性能和高能效的计算环境。这种架构在现代微处理器设计中非常流行。

从 Hopper 架构开始 Tensor Core 相关指令方面引入了 wgmma 指令，此计算类指令可以直接读取共享内存上的数据进行计算，而不必要求操作数必须在寄存器上，其则突破了 Load Store 架构的约束，但对于 vector（SIMT）类的计算依然遵循。

22.2　寄存器表达的程序状态机

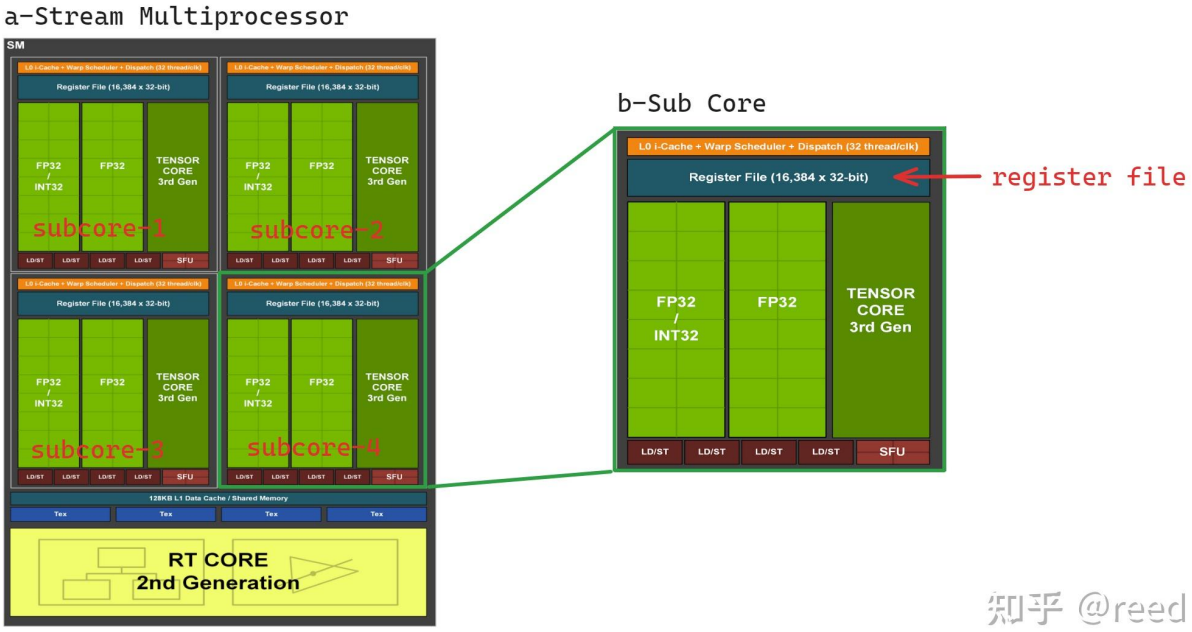


图 22.1: Figure 1. Register File in Sub-Core of SM

图 1 为 NVidia Ampere 的 SM（Stream Multiprocessor）的架构描述，可以看到一个 SM 包含 4 个 Sub Core（或者称为 Sub Partition），每个 Sub Core 中包含 16,384 个 32bit 的寄存器（通用寄存器）。我们知道 CUDA GPU 中的调度单位为一个 warp，每个 warp 包含 32 个 lane，这 32 个 lane 共享同样的代码，所以他们的寄存器在各自的线程看来也是同样的，将 16,384 个寄存器分配给 32 个 lane 我们可以得到 512 组寄存器。如图 2-a 所示，我们可以认为 Sub core 上的寄存器是一个二维结构横向为 32 个 lane，纵向为 512 组。当 SM 执行某个 kernel 时，会将这些寄存器分配给不同的 warp（warp 组成 block），图 2-b 展示了每个 warp 分配 4 个寄存器时，各个 lane 得到的本地的寄存器的名称和不同的 warp 组，对于每一个需要被执行的 warp，寄存器文件按照 kernel 所需要的寄存器数目分配给不同的 warp。sm_70 及以后按每 warp 256 个 32-bit 寄存器分配，即每线程 8 个（老架构可能按 4 个）；示例中每个线程看到 R0-R3，是图 2-b 为讲解画的 4 寄存器小例子，不是架构粒度。

这种寄存器分配模式构成了 CUDA 中最重要的延迟隐藏机制，那就是程序的执行状态都在寄存器中被记录和表示，warp 调度单元选择一个可以执行的单元，利用执行单元（如图 1 中的 FP32 单元）读取特定 warp 对应的寄存器中的数据，将结果写入到这些寄存器中。如某个 warp 遇到了 Load 指令需要较多 cycle 才能获取到数据，而 warp 调度器则可以切换到其他数据已经 ready 需要进行计算任务的 warp 执行，即换一组寄存器表的状态即可（类似于 CPU 的线程调度）。如此便做到了执行单元只有一份，但是表达程序运行状态的存储单元却有多份，并且这种切换执行是十分轻量的，其和 CPU 中的超线程是类似的：一份执行单元，多份寄存器状态。通过 warp 切换来达到延迟隐藏的目的。物理硬件上寄存器文件一般使用 SRAM（static random access memory）实现。

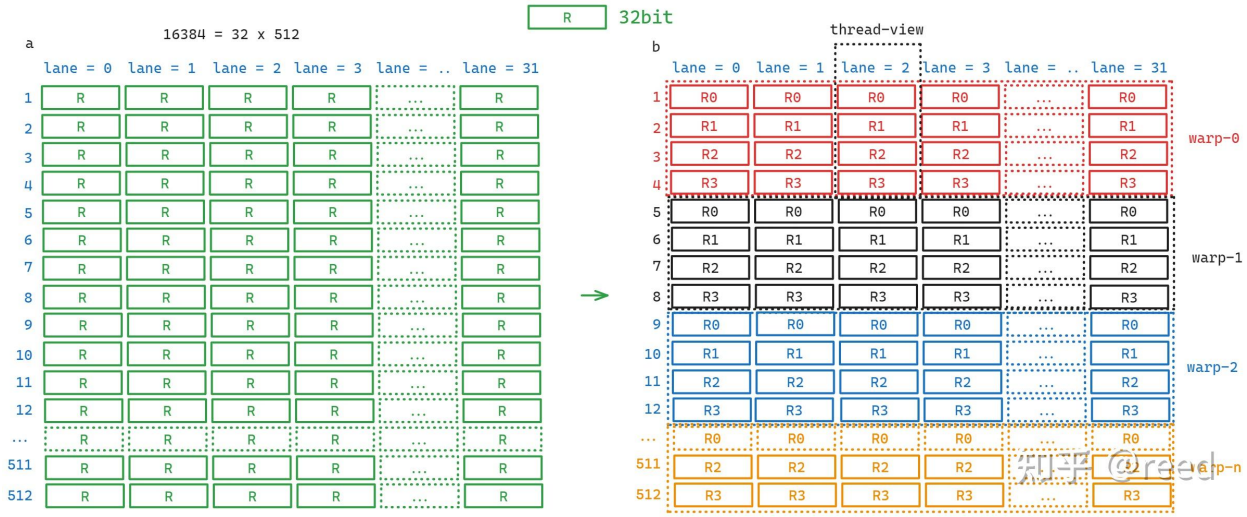


图 22.2: Figure 2. Register File and its Warp View and Thread View

22.3 通用寄存器

上面提到的寄存器文件即为通用寄存器，通用寄存器是 CUDA 中的重要存储结构，其也是 GPU 上最高效的存储结构，通用寄存器可读可写，非 Load/Store 指令只能对寄存器进行操作，即源操作数和目的操作数都为寄存器。单个寄存器的位宽为 32bit。每个线程所私有的寄存器最多可以为 255 个，在 SASS 表示中，以 R 为前缀，表示为 R0-R255。由于寄存器的位宽为 32bit，有时候算法需要更宽的数据存储结构时，则采用连续的多个寄存器组来完成，在 SASS 编码中只体现首寄存器编号，其余的寄存器不体现在编码中。如`F2F.F64.F32 R4, R2;` 表示 32bit 浮点数到 64bit 浮点数的类型转换 (float to float: float32 to float64)，其中目标寄存器需要使用两个连续的寄存器 R4R5 来存储 double 值，源寄存器 R2 存储 float 值。另外约定 R255 寄存器为常零寄存器，在 SASS 中表示为 RZ（Register ZERO）。

```
R0, R1, R2, R3, ..., R251, R252, R253, R254, R255(RZ)
```

22.4 特殊寄存器

特殊寄存器（Special Register）一般是只读的，用于标识该执行单元的定位信息，如线程号，线程块号等，需要通过特定的指令来读取这些寄存器，常见的特殊寄存器如下，其中`SR_TID`表示 cuda thread block 内的线程 id 即 cuda 编程中的 threadIdx，`SR_CTAID`表示线程块 id 即 cuda 编程中的 blockIdx，还有其他的获取硬件 SM id、时间信息等的特殊寄存器，如下

```
SR_TID.X, SR_TID.Y, SR_TID.Z,
SR_CTAID.X, SR_CTAID.Y, SR_CTAID.Z,
SR_VIRTUALSMID, SR_LANEID, SR_LEMASK, SR_LTMASK, SR_GEMASK,
SR_CLOCKLO, SR_CLOCKHI
SR_GLOBALTIMERLO, SR_GLOBALTIMERHI
SRZ
SR_PM0-7
SR_SMEMSZ
```

具体的寄存器意义，可以参考 PTX 文档中的 Special Register 章节。

22.5 Predicate 寄存器

Predication（谓词化执行）是 GPU 架构中用于消除和控制分支的一种技术，与 CPU 中的分支预测（branch prediction）不同。在 GPU 中，predication 通常用于控制线程束（warp）中线程的执行流程。Predicate 寄存器的优势主要体现在以下几个方面：

1. 提高分支效率：在 GPU 中，Predicate 寄存器允许线程束中的各线程根据谓词取值来决定是执行还是跳过某些指令。这种方式可以减少每个线程单独进行分支决策的开销，从而提高分支处理的效率。
2. 减少分支开销：由于 Predicate 寄存器的存在，GPU 可以在一个统一的控制下管理线程束的执行路径，这减少了每个线程单独处理分支的需要，从而降低了分支操作的总体开销。
3. 优化指令流水线：通过 predication，GPU 可以更好地管理指令流水线，减少分支指令及其控制开销（如分歧管理指令（如 BSSY/BSYNC）的派发与额外的取指（instruction fetching））。
4. 提升并行执行效率：在 GPU 的并行计算中，线程束中的线程通常会执行相同的指令流。Predication 寄存器允许线程束作为一个整体来响应分支指令，这样可以保持线程束内部的同步性，减少由于分支分叉导致的执行效率下降。
5. 简化编程模型：对于程序员来说，Predicate 寄存器提供了一种更为简单的方法来控制线程束的行为，而不需要在代码中显式地处理复杂的同步和分支逻辑。
6. 增强硬件的灵活性：通过使用 Predicate 寄存器，GPU 的硬件设计可以更加灵活地处理分支密集型的程序，特别是在处理大量并行线程时，可以有效地管理线程束的执行状态。
7. 减少能耗：由于减少了分支控制与分歧管理的开销，Predicate 寄存器有助于降低能耗，这对于功耗敏感的移动和嵌入式设备尤为重要。

总的来说，Predicate 寄存器是 GPU 架构中用于提高分支处理效率和优化线程束执行的关键技术。它通过简化分支控制逻辑，减少了分支控制的开销，并提高了 GPU 在执行并行计算任务时的性能和能效。具体的一个分支代码在不使用 predicate 和使用 predicate 寄存器后的效果如图 3 所示：

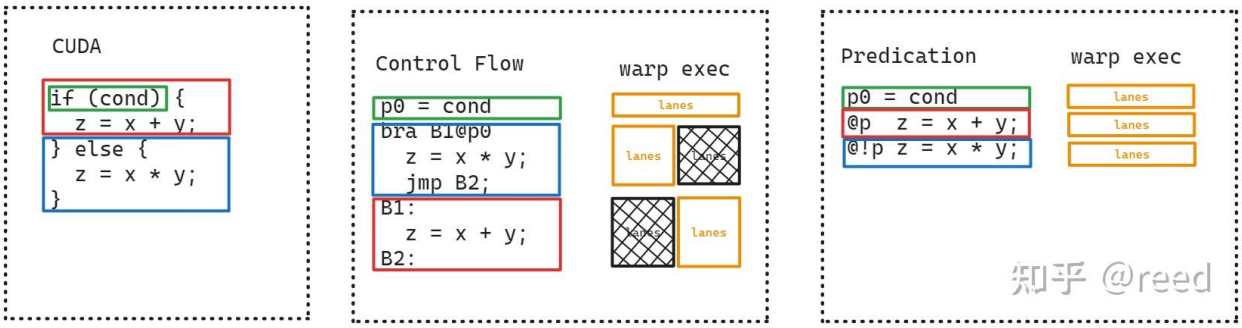


图 22.3: Figure 3. Predication avoid pipeline stall

具体地，Predicate 寄存器主要有两个作用：一个是作为指令的执行条件放在指令的开始，如`@P6 FADD R5 R5 R28;`和`@!P1 FADD R11 R11 R17;`用以指示该条指令为条件指令，只有当 @ 后的 predicate 运算结果为 True 时才会执行，反之为 False 时，该指令不产生执行副作用；二是 predicate 可以作为操作数参与特定指令的运算，如`FMNMX R9 RZ R6 !PT;`。Predicate 寄存器为每个线程私有，每个线程最多可使用 8 个，在 SASS 表示中名称以 P 为前缀，表示为 P0-P7，其中 P0-P6 为常规的读写 Predicate 寄存，P7 为常真寄存器，即它始终为 True，SASS 中标示为 PT（Predicate True）：

```
P0, P1, P2, P3, P4, P5, P6, P7(PT)
```

22.6　Uniform 寄存器

以上的通用寄存器和 predicate 寄存器在 SIMT 编程模式看都是线程私有的寄存器，有些场景一个 warp 内的所有 lane 会执行完全相同的逻辑或者做 reduce 等功能（CUDA 对应`__reduce_sync`类函数），NV 提供了 Uniform 寄存器、Uniform Predicate 和相应的指令来完成 Warp Level 的公共计算。使用 Uniform 寄存器可以减少对私有寄存器的使用量，继而可以减少 warp 对通用寄存器的使用，使得 SM 上有机会运行更多的 warp 提升并发度，同时由于 Warp Level 不需要向量化的执行单元，也能减少整体芯片功耗。在 SASS 层面，Uniform 寄存器以 UR 作为前缀，单个 warp 最多可用 64 个，Uniform Predicate 以 UP 作为前缀，单个 warp 最多可用 7 个，和通用寄存器、Predicate 类似，最后一个寄存器 UR63 为常零寄存器，SASS 中表示为 URZ（Uniform Register ZERO），类似地，也有常真 Uniform Predicate UPT（Uniform Predicate True）。

Uniform 寄存器表示如下：

```
UR0, UR1, UR2, UR3, ..., UR60, UR61, UR62, UR63 (URZ)
```

Uniform Predicate 表示如下：

```
UP0, UP1, UP2, UP3, UP4, UP5, UP6, UP7(UPT)
```

22.7 总结

寄存器是 GPU 中重要的存储结构，GPU 通过提供大量的寄存器实现高并发的延迟隐藏模型，对于单线程而言使用较少的寄存器可以提升 SM 上能并发的 Warp 数，提升并发度，同时 NV 限制了单个线程能使用寄存器的上限为 255 个 32bit；NV 针对线程块索引等提供了特殊的寄存器用于获取逻辑坐标；Predicate 寄存器可以实现小块代码段的更高效的流水线效果。Uniform 寄存器可以实现 warp level 一致的计算逻辑和 reduce 功能，其可以减少通用寄存器的使用提升能效。

22.8 参考

https://eng.libretexts.org/Bookshelves/Computer_Science/Programming_Languages/Introduction_to_Assembly_Language_Programming%3A_From_Soup_to_Nuts%3A_ARM_Edition_(Kann)/04%3A_New_Page/4.04%3A_New_Page

https://en.wikipedia.org/wiki/Load%E2%80%93store_architecture

https://en.wikipedia.org/wiki/Predication_(computer_architecture)

https://www.nvidia.com/content/PDF/nvidia-ampere-ga-102-gpu-architecture-whitepaper-v2.pdf

https://course.ece.cmu.edu/~ece740/f13/lib/exe/fetch.php?media=onur-740-fall13-module7.4.2-predicated-execution.pdf

https://docs.nvidia.com/cuda/parallel-thread-execution/index.html#special-registers

第二十三章　NVidia GPU 指令集架构-Load 和 Cache

作者：reed（知乎 @reed-84-49，专栏「CUDA 高性能编程」）　原文：https://zhuanlan.zhihu.com/p/692445145

前文介绍了 NVidia GPU 指令集架构[1]中的寄存器部分[2]，对于一个 GPU 程序而言，这些寄存器数据最初来自于外部存储结构，如何将数据从外部存储结构搬运到寄存器，以及在搬运过程中经过哪些 Cache 对程序的效率有重要影响。本文将围绕数据搬运展开，重点介绍 NVidia GPU 的存储层次和 Cache 层级，以及这些存储层级之间的数据搬运指令和 Cache 控制指令。文章结构方面，首先介绍了 NVidia GPU 上的计算单元和存储层级，然后介绍了 Cache 和 Shared Memory 机构，再后介绍了 Load 和 Cache 相关的指令以及一些特殊的 Cache 和预取行为，最后对文章进行了总结。

23.1　NVidia GPU 的计算单元和存储层级

GPU 是一个高度并行的设备，其上装配了大量的计算单元，为了更好的组织这些计算单元，更充分的利用数据局部性（Spatial Locality）和支持数据规约和同步能力，GPU 采用层次化的组织形式。整体而言，在 Ampere 及其之前的架构在计算单元组织方面可以分为三个层次：最小的计算机构为 SubCore 其负责线程束（warp）的执行，4 个 SubCore 组成一个 SM（Stream Multiprocessor），多个 SM 组织成 GPU 设备（device），根据设备设计规格的不同 SM 数目是其最直观的区别，如数据中心级 A100 和消费级 GeForce RTX3080、GeForce RTX3090 卡都是 Ampere 架构，在计算单元层面的最大区别就是所装配的 SM 数目。就计算单元具体而言（如图 1），SubCore 中包含有核心计算单元 Tensor Core 用以完成矩阵计算、CUDA Core 用以完成向量化的浮点数和整数的乘加运算、SFU（special function unit）特殊函数单元用以完成 sin、exp2、sqrt、rcp 等超越函数，同时 SubCore 内有程序状态核心存储机构寄存器文件（Register File），还有具有广播语义的常量内存（图中标记为 constant cache），SubCore 内还有用于外部数据加载和存储用的 Load Store Unit。SubCore 中还有 warp scheduler、branch unit、FP64 等单元（他们对于理解指令集架构没有特别明显的作用，我们就没有单独标出）。对于存储层次而言，SubCore 内有寄存器和 Constant Cache，SM 内有 4 个 SubCore 共享的 L1 Cache 和 Shared Memory。所有的 SM 通过交叉开关（CrossBar）共享 L2 Cache。分 Slice 的 L2 Cache 通过 Memory Controller 和外部存储（独立的 Die）HBM（high bandwidth memory）或者 GDDR（Graphic DDR SDRAM）相连。数据中心级卡 A100 和消费级 GeForce RTX 3080 的另一个重要不同则是内存介质技术（HBM vs GDDR）。

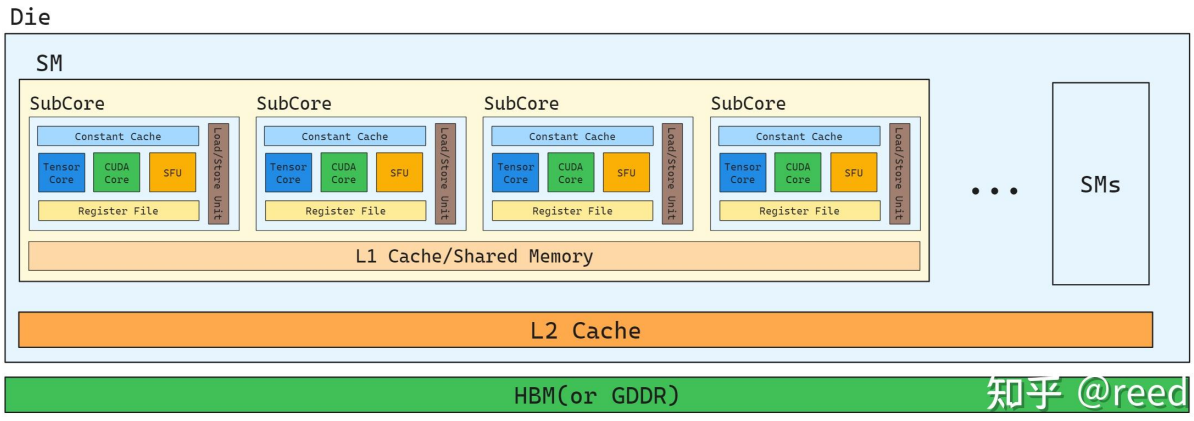


图 23.1: Figure-1. NVidia GPU 存储层级和 Cache 层级

软件的是对物理硬件的抽象描述，对于 CUDA 编程而言，如图 2，我们基本上可以将物理的 SubCore 映射到 CUDA 编程中的 warp；将硬件上的 SM 映射为 CUDA 编程中的 thread block；将多个 SM 组成的 Device 映射为 CUDA 编程

[1] https://zhuanlan.zhihu.com/p/686198447
[2] https://zhuanlan.zhihu.com/p/688616037

中的 grid。只是软件在向硬件映射的时候多了调度的逻辑。即一个物理的（physical）subcore 可以运行多个 warp，一个 SM 可以运行多个 thread block，有限的 SM 组成的 device 可以运行远超其硬件数目的 grid。

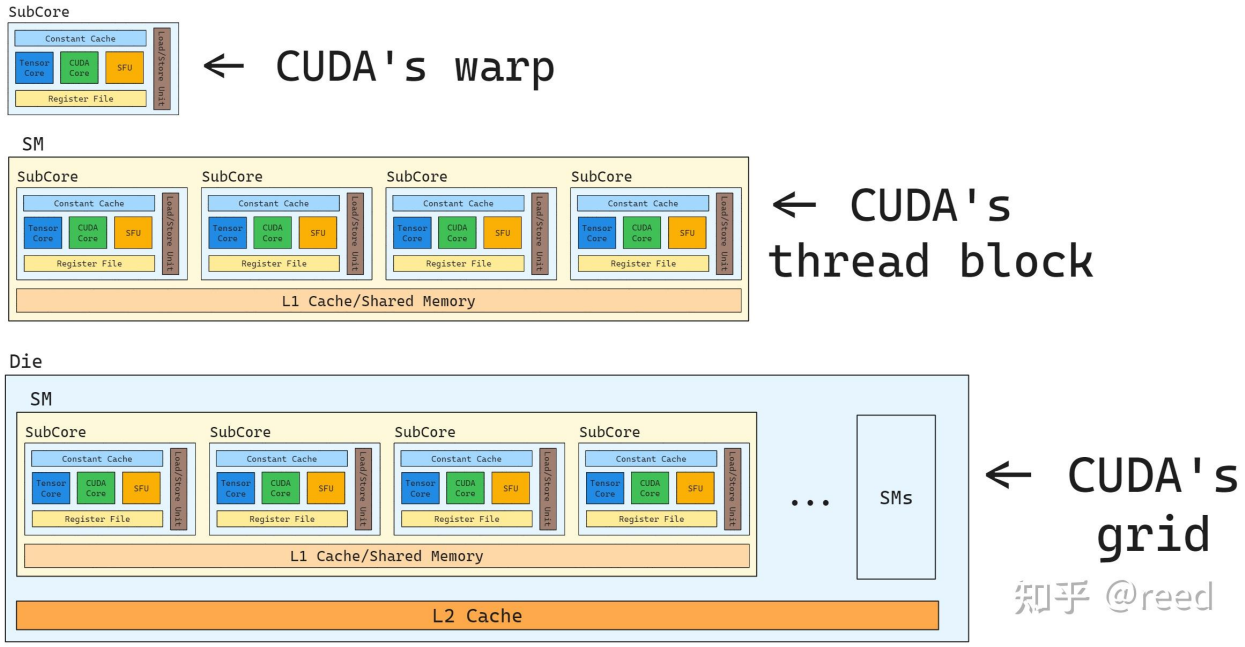


图 23.2: Figure-2. Hardware and its Software Abstraction

23.2 GPU 中的 Cache 和 Shared Memory 机构

Cache 是时间局部性和空间局部性的一条重要的实践方案，也是 20 世纪 90 年代研究最多的课题。对于高度追求并行处理的 GPU 而言，Cache 是其重要的存储结构，其能极大的提升程序效率。GPU 编程中（CUDA 编程），数据最初被存储在全局内存中（global memory），而核心的计算发生在 SubCore 中的计算单元内。前文提过 GPU 是 Load-Store 架构（寄存器-寄存器架构），所以计算单元只能访问 SubCore 内的寄存器（和 constant cache）。如果需要外部数据，必须通过 Load 指令将数据加载到寄存器中，GPU 利用局部性原理在 global memory 和寄存器之间设置了两层 Cache 机构：L2 Cache 和 L1 Cache。其中 L2 Cache 在 A100 下为 40MB，其数据被所有 SM 共享。L1 Cache 被装配在每一个 SM 中，被 SM 中的 4 个 SubCore 共享。当 SubCore 中的 Load Store Unit 产生对全局内存的数据访问请求时，L1 Cache 会查看当前请求在之前是否被请求过，如果之前请求过该数据并且没有被清除掉，那么 L1 Cache 命中可以直接返回该数据。如果 L1 Cache 之前没有被请求或者数据被清除导致 miss，则 L1 Cache 会对 L2 Cache 发送请求，此时如果 L2 Cache 命中，则立即返回，只有两级 Cache 都 miss 的情况下才会对全局内存产生请求。L1/2 Cache 的带宽和访问 Latency 相比 HBM 要好很多，适当的调整数据访问逻辑，提升数据局部性可以更充分的利用各级 Cache，极大的提升程序效率。

Cache 是不可编程的存储空间，其命中和清除逻辑由硬件自行控制，有些时候程序书写着可以更精准的管理共享数据，并且可以选择更合适的时机对数据进行更新和同步，为此 NVidia GPU 在 SM 级别提供了可编程的 Shared Memory 存储机构，它是一片可寻址的地址空间，同时提供了 Load Store 以及同步数据可见性的能力，这样当有一段数据需要反复使用时，则用户可以显式的加载到共享内存中，然后反复读取，避免了对低层级内存的访问压力。从 Fermi 架构开始，L1 Cache 和 Shared Memory 共享同一份后端存储空间，只在前端做 tag 命中和地址判定，提供了相对灵活的配置能力，用户根据具体的使用场景决策将这份空间多分给 L1 Cache 还是自己手动可以控制的 Shared Memory。

结合上面的介绍我们不难发现，从不同的维度去看数据加载有不同的形式：

1. 软件概念：Global -> Shared -> Register
2. 物理概念：HBM(Global) -> SRAM(L2 Cache) -> SRAM(L1 Cache) -> SRAM(Register File)
3. 片上概念：OffChip(Global) -> OnChip(L2 Cache) -> OnChip(L1 Cache) -> OnChip(Register File)
4. 共享层次：SMs(Global) -> SMs(L2 Cache) -> SM(L1 Cache) -> SM's SubCore(Register File)

23.3 数据 Load 指令

数据加载相关的指令整体如下：

```
LD, LDG, LDS, LDSM, LDL
```

其中 LD 指令（LoaD），是通用的（编译器在编译时无法推导地址空间类型的数据加载），如果编译时可以明确的知晓地址空间类型则使用有类型的加载指令 LDG（LoaD Global memory），LDS（LoaD Shared memory），LDSM（LoaD Shared Matrix），LDL（LoaD Local memory）等，具体地

全局内存到寄存器：

```
LDG.类型.向量.Cache控制.L2预取
```

加载全局内存地址中的数据到寄存器，通过 modifier 可以配置加载时的数据宽度，如 8bit 数据，16bit 数据，128bit 数据等。同时可以配置各层级的 Cache 的 bypass 等情况，也可以配置是否对数据向 L2 中预取。单就指令而言向量化的数据加载（或叫大字长加载）如 LDG.128 是 NVidia GPU 支持的最大的加载指令，一条指令可以加载 128bit 数据，对于同等规模的数据使用更宽的加载指令可以减少 warp 对指令的调度次数，减少调度开销，减少 MIO queue 的事务数，避免由于 queue 满而造成阻塞。除了单指令位宽，更高效的数据加载还需要考虑合并访存（关于合并访存的优势我们后续会专门出一个“软件优化的硬件解释”系列中详细介绍）。

全局内存到共享内存：

```
LDGSTS, LDGDEPBAR, DEPBAR.LE SB0, 0x1
```

异步读取数据全局内存并将结果存储到共享内存 LDGSTS（LoaD Global memory STore Shared memory），可以实现不经过寄存器的全局内存到共享内存数据搬运，可以减少寄存器的使用和依赖，它在矩阵计算中，尤其 Multi Stage 的矩阵计算中有重要作用，可以参考 cute 之 GEMM 流水线[3]的异步拷贝章节。同时该指令要结合 Barrier 设置和等待指令（LDGDEPBAR，DEPBAR）协同使用。另外该指令在加载数据时可以指定是否在 L1 进行 Cache，和对 L2 进行数据预取。

共享内存到寄存器：

```
LDS.类型.向量化，LDSM.块.转置
```

LDS 的 modifier 可以设置数据位宽信息，和 LDG 类似高位宽的指令可以减少 warp 指令的调度数减少 MIO queue 中的事务数目，防止 queue 满引起的阻塞。LDSM 为 warp 级协作指令，完成共享内存到寄存器的数据加载，进而将这些寄存器 feed 给 Tensor Core 指令完成矩阵计算，更细节的介绍可以参考 cute 之 Copy 抽象[4]和 ldmatrix 指令优势介绍[5]。

局部数组和寄存器溢出：

```
LDL
```

目前认为有三种情况可以引入 Local Memory：1. 当线程计算需要局部数组，并且数组的下标不能被编译时计算时；2. 单线程的寄存器使用数目超过 255；3. 访问 kernel 中的常量数组（如局部 const 数组）时使用了不能被编译时确定的索引（此时若编译器无法将其折叠为立即数或常量表，可能退化为 Local Memory）。Local Memory 是 CUDA 编程中的一个概念，它的物理实体是全局内存中的一段。当上面情况发生时，每一个线程都会被分配一段全局内存来作为数据空间，由于数据需要对全局内存进行读写，一般而言对于线程数比较多的场景，其开销很大，除非万不得已，应该尽量能避免 Local Memory 的使用。

23.4 广播语义的常量 Cache

除了 Load 全局内存和共享内存，我们提到了 SubCore 中有 constant cache 机构，虽然它的名字是 cache，但其本质是有名对象（通过地址标识）和寄存器更类似，它提供了一种广播语义，即 warp 内的所有线程都访问同一个数据时，它的访问速度和寄存器一样快，所以其可以被直接编码在指令操作数中。同时我们知道 kernel 的参数需要广播给所有的

[3] https://zhuanlan.zhihu.com/p/665082713
[4] https://zhuanlan.zhihu.com/p/666232173
[5] https://www.zhihu.com/question/600927104/answer/3029266372

执行线程，所以其也是使用 constant cache 实现，可以实现高效的广播语义。另外我们可以实现 device 端的可编程常量（`__constant__ __device__ int a;`）。当 warp 内的线程访问同一个 constant 位置时，其是确定的 latency 的（和访问寄存器一样），但是当不同的线程访问的位置不同时，则是串行效果，是一个变 Latency 指令，所以 SASS 层级提供了 LDC 指令，用以实现不同线程访问不同的 constant 位置。

23.5 寄存器 reuse 和 Prefetch

除了以上常规的存储机构和 Cache，在计算单元流水线中，已经加载进计算单元流水线的数据也可以复用，其体现为寄存器的 reuse，我们可以把它当做寄存器 cache，它可以减少寄存器带宽压力一定程度上降低功耗，如

```
R1.reuse
```

除了前面提到的 Load 指令可以做伴随的数据预取，SASS 还提供了显式设置 L1/2 Cache 的预取指令，如（CCTL = Cache ConTroL）

```
CCTL.E.PF2
```

23.6 总结

本文介绍了 NVidia GPU 的内存层次和 Cache 机构和指令集架构中的 Load 和 Cache 相关指令，了解这些指令能够更好的理解硬件在数据搬运时的行为，充分且合理的利用这些指令和 Cache 可以提升数据的搬运效率，指导数据搬运相关的优化。

23.7 参考

https://www.qidian.com/book/1031795831/

超标量处理器设计: 9787302347071: 姚永斌: Books: https://www.amazon.com/%E8%B6%85%E6%A0%87%E9%87%8F%E5%A4%84%E7%90%86%E5%99%A8%E8%AE%BE%E8%AE%A1-%E5%A7%9A%E6%B0%B8%E6%96%8C/dp/B00JFJTI2I/ref=monarch_sidesheet

https://pc.watch.impress.co.jp/docs/column/kaigai/1275220.html

https://pc.watch.impress.co.jp/video/pcw/docs/1275/220/p3.pdf

reed：cute 之 GEMM 流水线[6]

reed：cute 之 Copy 抽象[7]

tensorcore 中 ldmatrix 指令的优势是什么？[8]

[6] https://zhuanlan.zhihu.com/p/665082713

[7] https://zhuanlan.zhihu.com/p/666232173

[8] https://www.zhihu.com/question/600927104/answer/3029266372

第二十四章　NVidia GPU 指令集架构-浮点运算

作者：reed（知乎 @reed-84-49，专栏「CUDA 高性能编程」）　原文：https://zhuanlan.zhihu.com/p/695667044

计算机算数[1]是计算机工程的一个重要分支，现代计算类的软件多是构造在浮点运算之上的。了解浮点数和浮点运算对于我们理解计算类任务，提升计算精度和效率有很大的帮助。NVidia GPU 上 Tensor Core 和 CUDA Core 都提供了浮点计算能力，本文只专注 CUDA Core 部分（后续有专门章节介绍 Tensor Core 指令集），重点介绍了计算机上的浮点数相关知识和 NVidia GPU 上的浮点数相关指令。文章结构方面，首先介绍了浮点数的表示：浮点数基础、圆整、特殊值、denormal 信息；然后介绍了 NVidia GPU 上的浮点数相关指令：加法和乘法、乘加运算、随路取负和绝对值、低精度指令、超越函数、除法。

24.1　浮点数的表示

24.1.1　浮点数基础

计算机中数据的存储采用的是二进制形式，这决定了在有限位数的表示情况下，其只能表示有限的、离散的数据。对于整数而言，这种离散是自然的，如图 1 中的整数（Integer），我们可以用二进制的 b00000101 表示十进制的数字 5。对于计算机中表示实数（Real Number）而言，我们也可以使用图 1 中定点数的表示形式，即固定小数点位置（Fixed Point），这样小数点（图示箭头）左侧的部分表示整数部分，右侧部分表示小数部分。图中的定点数表示 $\frac{1}{2}+\frac{1}{8}=0.625$，但这种定点表示等价于将数轴均匀的划分，精度确定但是数据范围有限。计算机科学家们在此基础之上提出了浮点数的表示方法，通过浮动小数点（floating point），提高数据表示范围。

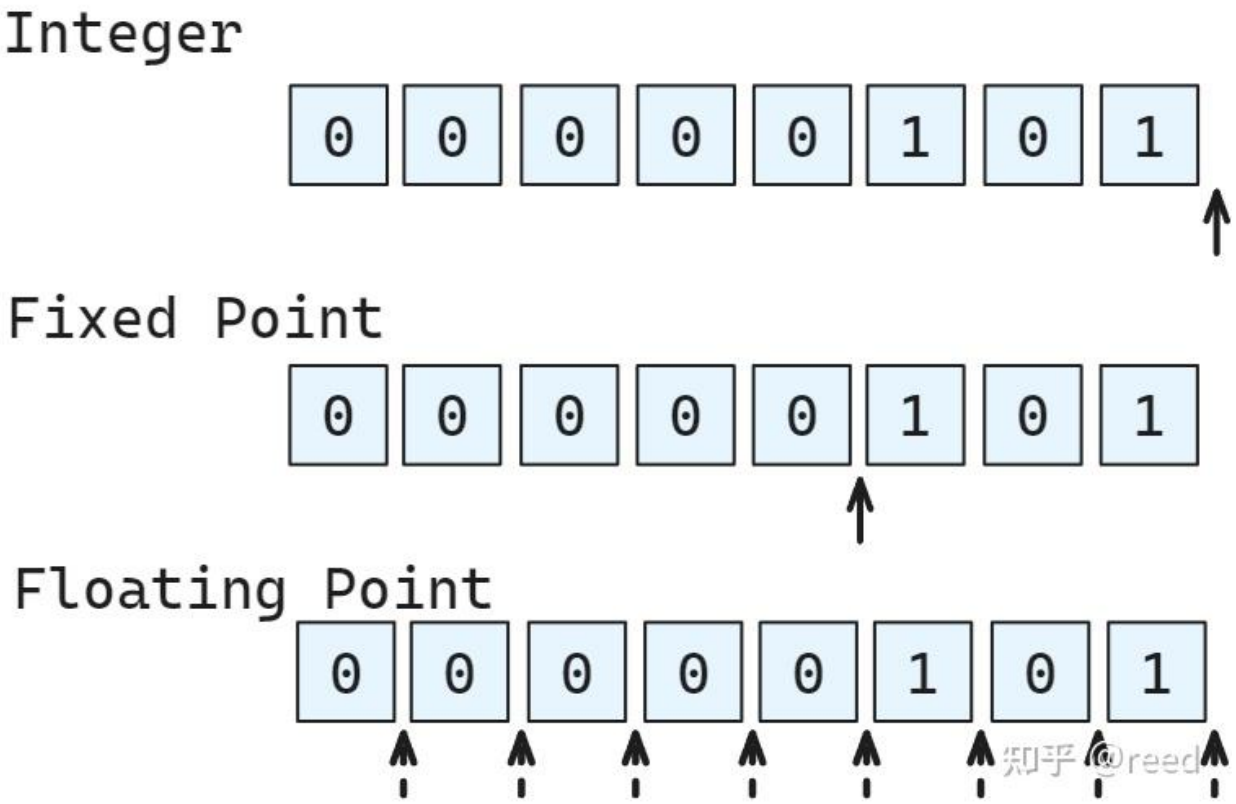


图 24.1: Figure 1. Integer, Fixed Point and Floating Point Number

具体地，如图 2 所示，对于 32bit 浮点数，其中 1bit 为符号 bit，0 表示正数，1 表示负数；8bit 的指数位，表示浮点数中点的位置，可以作为无符号的整数来进行取值 k，其表示带偏置的指数，k - 127 为去偏置后的实际指数，用以实

[1] https://ieeexplore.ieee.org/xpl/conhome/1000125/all-proceedings

际的浮点数计算，8bit 无符号数表示的有效范围为 0 到 255，即 [0，255]，减去偏置项 127 后，指数范围为 [-127, 128]，其中全零和全一用以表示特殊的数字，有效的指数为 [-126, 127]；指数位之后为尾数位（或者叫有效位）用以表达浮点数中的有效数字，在表示实际数值是会在尾数之前固定的添加一个数字 1 用以规则化数据（normal），确保数据的第一位是 1，从而确保了有效数据的位数为 24 位。

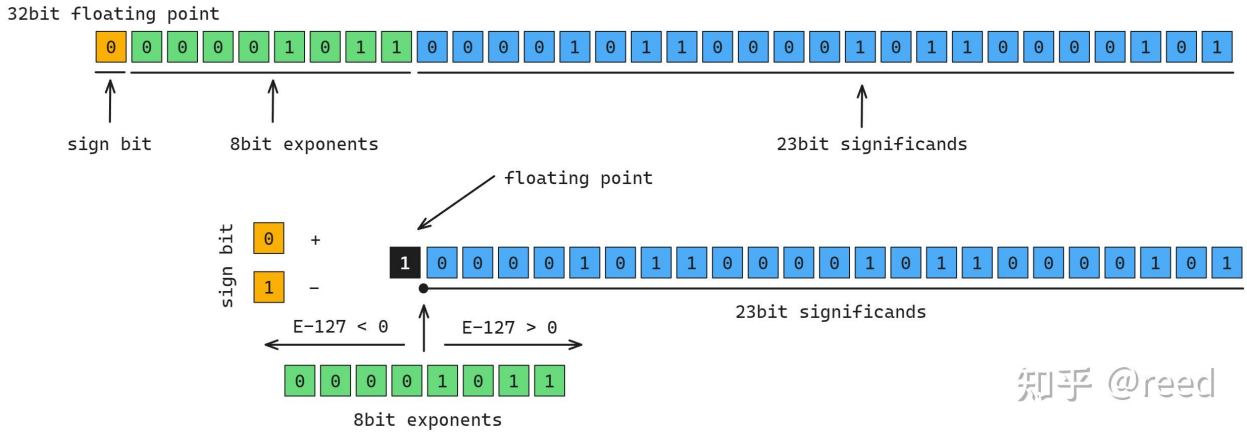


图 24.2: Figure 2. 32bit floating point bit details

浮点数的数值为 $v = (-1)^{sign} \times 2^{(E-127)} \times 1.S$ ，具体的如图 2 中，符号位为 0，表示 v 为正数，指数位为 b00001011 十进制为 11，去偏置后的实际指数为 11-127 = -116，尾数为前面添加 1 的一个小数，二进制表示为 1.00001011000010110000101，十进制表示为 1.043137192726135253906 25。将指数作用到尾数上后（即将含前缀 1 的尾数的小数点左移 116 位），得到的十进制数据为 1.25563072223456437348238306129524010182326911326927391629018127862382758763715173699893057346343994140625×10-35。具体的可以参考https://float.exposed/0x05858585。

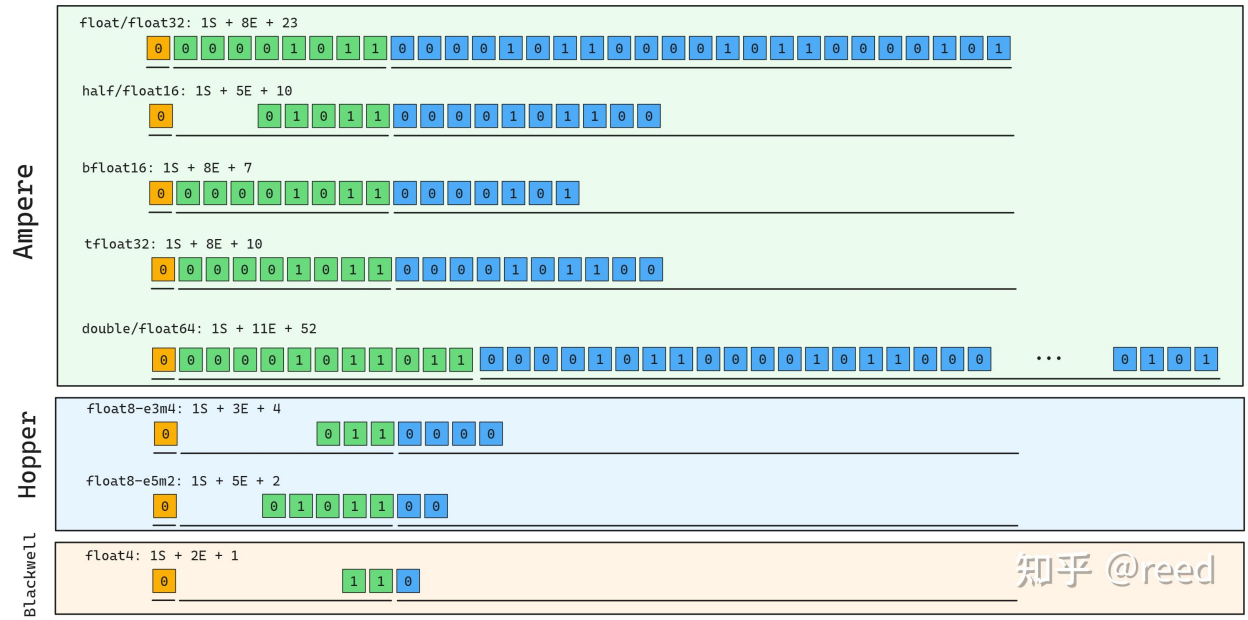


图 24.3: Figure 3. various floating point with various sign exponents significands bits

通过上面的描述我们知道浮点数中通过 E 指数来调整小数点的位置，通过尾数来表示数据的有效数值。E 的容量越大则表示的数据范围越大，尾数的容量越大则表示有效位数越大，数据精度越高。针对不同场景所需求的的数据范围和精度的平衡，Ampere 中支持五种类型的浮点数：float32、half（也叫 float16）、bfloat16、tfloat32 和 double 精度，hopper 中添加了 float8 类型，根据指数和尾数的多少分为 E5M2 和 E4M3 子类型，blackwell 架构中添加了 fp4 类型，具体的各类型的指数和尾数的位数如图 3 中所示。

24.1.2　浮点数的圆整

由于浮点数是实数的近似表示，IEEE-754 标准在定义浮点数上的四则运算（加减乘除）时为了获得更高的精度则定义了圆整方案，其中有：向最近圆整，向零圆整，向正无穷圆整（up）和向负无穷圆整（down）。一般情况下，四则运算采用最近圆整其能满足标准约束的 ULP 误差。将浮点数向整数转换时，应用向正无穷圆整则实际为 ceiling 函数，向负无穷圆整则为 floor 函数。在一些高精度除法等计算时，需要对数据做特殊的修正，此时也会使用到特定的圆整方式。

24.1.3　浮点数特殊值

在介绍浮点数的指数表示时，其中全 0 和全 1 是被保留的。其中指数全 1（尾数非全 0）的浮点数被解释为 NaN（Not A Number），它是浮点数中一类特殊的数值。如零除以零在数学上是无意义的，但是我们必须能用浮点数表示这种无意义，这时候这个计算结果即可以表示为 NaN（有些编程语言则实现为异常，对于 GPU 而言，作为异步设备计算不产生异常），类似地对负数开平方在实数解空间下也是无意义的，其结果也表示为 NaN，还有其他类似的指数等运算也可以产生 NaN 结果。同时 NaN 有一定的传递性，如果某个计算中间产生了 NaN 很容易使得后续的结果也为 NaN，如 NaN + 1.0 = NaN，NaN x 0.0 = NaN。这也是编程中对浮点数表示不熟悉而容易犯的错误，认为一个未初始化的数据乘以 0 就一定可以可以得到 0。其实不然，因为这个未初始化的数据如果是 NaN 其结果就不是 0 了。NaN 具体的还可以分为 Quiet 和 Signaling，在此我们不再赘述，有兴趣的可以参考 IEEE-754 标准的介绍。

当指数全 1，尾数为全零时，根据符号位的不同则定义了正无穷（+infinity）和负无穷（-infinity）。正负无穷也是一种相对特殊的表示，但是定义在这上面的数学计算则满足数学规则，如 exp(-infinity) = 0。

恰当的使用 NaN 和 +Infinity，-Infinity 可以简化和 unify 一些数学计算而避免一些分支结构。

24.1.4　浮点数的 denormal

前文提到针对尾数表示会在前面固定的添加一个 1（规则化，normal）确保了数据有效位数为 24 位，对于 E 指数为全 0 时，这时在计算尾数时不再添加这个固定的 1，那么尾数的表示可能包含 0 开头的数据，从而减少了数据的有效位数，这样的数据称为非规则浮点数（denormal 或者 subnormal），同时全零的 E 指数记为 2^{-126} （而非 2^{-127} ），这样其可以表示较小的 0 附近比 normal 数更小的浮点数，加密了 0 附近的点的数据表示，对于 0 附近的绝对值特别小的数据能更好的表示。

图 4 展示了浮点数中的 0，normal 数，subnormals，无穷和 NaN 时指数和尾数的情况。

24.2　NVidia GPU 浮点运算指令

24.2.1　加法和乘法

NVidia GPU 上实现了 IEEE-754 标准的加减乘指令和不同的圆整类型，具体地，加法（FADD = Float Add）和乘法（FMUL = Float MULtiply）指令如下，可以实现 float32 数据类型的加法和乘法，同时采用默认的 Nearest 圆整方式，对于 double 精度，则有 DADD(Double Add) 和 DMUL(Double MULtiply)；

```
FADD R0 R1 R2;     // R0 = R1 + R2  with round to NEAREST
FADD.RZ R0 R1 R2;  // R0 = R1 + R2 with round to ZERO
FADD.RP R0 R1 R2;  // R0 = R1 + R2 with round to POSITIVE(+Infinity)
FADD.RM R0 R1 R2;  // R0 = R1 + R2 with round to MINUS(-Infinity)
FMUL R0 R1 R2;     // R0 = R1 * R2 with round to NEAREST
FMUL.RZ R0 R1 R2;  // R0 = R1 * R2 with round to ZERO
FMUL.RP R0 R1 R2;  // R0 = R1 * R2 with round to POSITIVE(+Infinity)
FMUL.RM R0 R1 R2;  // R0 = R1 * R2 with round to MINUS(-Infinity)

DADD R0 R2 R4;     // R0.64 = R2.64 + R4.64 with round to NEAREST
DADD.RZ R0 R2 R4;  // R0.64 = R2.64 + R4.64 with round to ZERO
DADD.RP R0 R2 R4;  // R0.64 = R2.64 + R4.64 with round to POSITIVE(+Infinity)
DADD.RM R0 R2 R4;  // R0.64 = R2.64 + R4.64 with round to MINUS(-Infinity)
DMUL R0 R2 R4;     // R0.64 = R2.64 * R4.64 with round to NEAREST
DMUL.RZ R0 R2 R4;  // R0.64 = R2.64 * R4.64 with round to ZERO
DMUL.RP R0 R2 R4;  // R0.64 = R2.64 * R4.64 with round to POSITIVE(+Infinity)
DMUL.RM R0 R2 R4;  // R0.64 = R2.64 * R4.64 with round to MINUS(-Infinity)
```

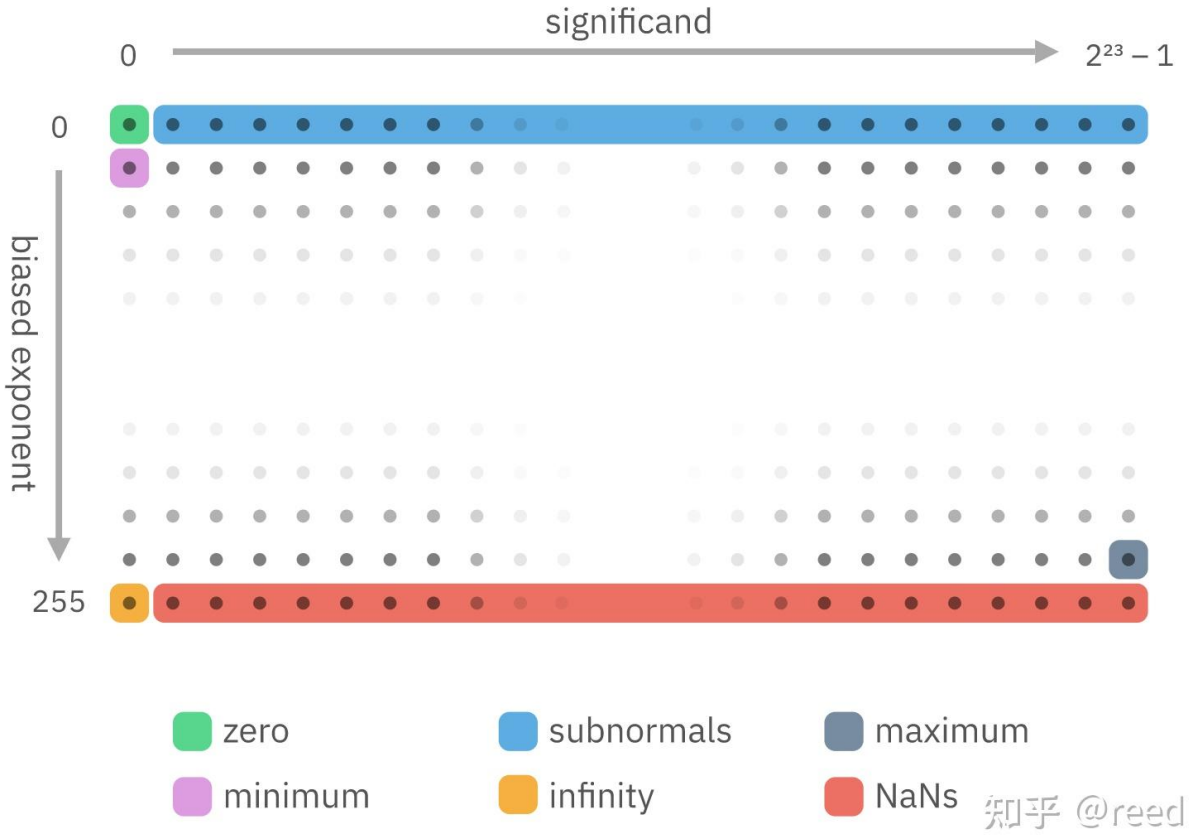


图 24.4: Figure 4. All the special values of floating point(引用自参考 1)

对于忽略 denormal 数据的处理也有对应的 modifier（FTZ = flush to ZERO）等

```
FMUL.FTZ R3, R4, R5 ;
```

24.2.2 乘加 FMA（Fused Multiply Add）

对于乘加运算而言，可以采用两条指令实现，如上面提到的 FMUL，然后对乘法的结果调用 FADD 进行累加，IEEE-754-2008 标准提出了 FMA 的计算标准（Fused Multiply Add），即可以一条指令完成 d = a x b + c 的运算，同时中间计算结果的精度是无限精度的，只在最后环节进行圆整操作，其可以提供相较于 FMUL + FADD 组合指令更高的精度。同时 FMA 指令的 latency 和 FMUL 或 FADD 是一样的，所以单条 FMA 指令的 FLOP 吞吐是 FADD 或 FMUL 的两倍，但指令吞吐相同（每周期仍只发射一条 FP32 指令），具体地，单精度浮点指令为 FFMA（Float Fused Multiply Add），双精度浮点指令为 DFMA（Double Fused Multiply Add）它们各自的不同圆整 Modifier 如下，

```
FFMA R0, R1, R2, R3;  // R0 = R1 * R2 + R3 with round to NEAREST
FFMA.RZ R0, R1, R2, R3;  // R0 = R1 * R2 + R3 with round to ZERO
FFMA.RP R0, R1, R2, R3;  // R0 = R1 * R2 + R3 with round to POSITIVE
FFMA.RM R0, R1, R2, R3;  // R0 = R1 * R2 + R3 with round to MINUS

DFMA R0, R2, R4, R6;  // R0 = R2 * R4 + R6 with round to NEAREST
DFMA.RZ R0, R2, R4, R6;  // R0 = R2 * R4 + R6 with round to ZERO
DFMA.RP R0, R2, R4, R6;  // R0 = R2 * R4 + R6 with round to POSITIVE
DFMA.RM R0, R2, R4, R6;  // R0 = R2 * R4 + R6 with round to MINUS
```

24.2.3　随路取负和绝对值

对于减法而言，NVidia 的指令集架构中并没有独立的减法指令，而是在 FADD 中随路对数据取负实现，即 d = a - b 可以写做 d = a + (-b)，同时 FADD 指令中随路做对 b 的取负操作：

```
FADD R0 R1 -R2;     // R0 = R1 - R2  with round to NEAREST
FADD.RZ R0 R1 -R2;  // R0 = R1 - R2 with round to ZERO
FADD.RP R0 R1 -R2;  // R0 = R1 - R2 with round to POSITIVE(+Infinity)
FADD.RM R0 R1 -R2;  // R0 = R1 - R2 with round to MINUS(-Infinity)
```

如此在 FADD 指令上附加对操作数取负能力，单个指令即可以实现浮点数的加法和减法功能，并且圆整能力保持不变。除了对第二个操作数取负号，NVidia 的浮点运算指令集基本上可以实现对任意操作数取负。形如 $d = \pm a \pm \pm b$ ，$d = \pm a \times \pm b$ ，$d = \pm a \times \pm b \pm c$ 的形式的操作都可以通过 FADD、FMUL、FFMA 指令随路对操作数取负而单指令完成。甚至 NVidia 指令集中没有直接提供取负的指令，而是通过 d = 0 - d 来 unify 的实现；除了对操作数随路取负，这些浮点指令也可以随路取绝对值操作。如`d = abs(a) * abs(b) - abs(c);`可以通过单条 FMA 指令实现，如下

```
FFMA R7, |R0|, |R7|, -|R6|;
```

24.2.4　低精度浮点指令

half、bfloat16 是 NVidia 提供的 16bit 的浮点数类型，连续的两个 16bit 数据可以合并成一个 32bit 的数据（寄存器宽度），该连续的两个 half 数形成 packed 类型 half2 和 bfloat162 类型。因为寄存器宽度是 32bit 的，所以 NVidia 的计算指令是针对 packed 类型而提供的，即便是操作单个 half 数，也会使用针对 packed 类型的指令，只是会在操作数上做重复和选择。具体的针对 packed half 即 half2，提供了加法、乘法和乘加指令（HADD2，HMUL2，HFMA2.MMA），这些指令就不再提供不同的 round mode（只有 round nearest），而是提供了饱和（SAT，最终结果截断到 [0.0, 1.0]）和整流（RELU）的 Modifier，同时该指令也提供了随路 abs 和取负能力。对于 packed bfloat16 即 bfloat162，没有提供 ADD 和 MUL 指令，只提供了`HFMA2.BF16_V2`指令，加法乘法等都是使用该 FMA 实现的，同时该指令支持必要的随路取负和 abs 能力，modifier 方面不如 half2 类型支持的丰富，只提供了 RELU 能力。

```
HADD2 R7, R0, R7;
HMUL2 R7, R0, R7;
HADD2.SAT R7, R0, R7;
HMUL2.SAT R7, R0, R7;

HFMA2.MMA R7, R0, R7, R6;
HFMA2.MMA.SAT R7, R0, R7, R6;
HFMA2.MMA.RELU R7, R0, R7, R6;
HADD2 R7, |R2|, -RZ.H0_H0;
HADD2 R7, -R2, -RZ.H0_H0;
HADD2 R7, R0, -R7;

HFMA2.BF16_V2 R7, -RZ.H0_H0, RZ.H0_H0, |R2|;
HFMA2.BF16_V2 R7, R0, 1, 1, R7;
HFMA2.BF16_V2 R7, R0, R7, R6;
HFMA2.BF16_V2.RELU R7, R0, R7, R6;
HFMA2.BF16_V2 R7, R0, R7, -RZ.H0_H0;
HFMA2.BF16_V2 R7, -RZ.H0_H0, RZ.H0_H0, -R2;
HFMA2.BF16_V2 R7, R7, -1, -1, R0;
```

24.2.5　类型转换

对于类型转换方面各个类型相互转换的指令如下表格，有些类型直接没有单指令转换的能力，需要借助中间类型来实现，在表格中表现为 Multi-Instr(多条指令)

转换成->	float64	float32	half	bfloat16
float64	/	F2F.F32.F64	F2F.F16.F64	Multi-Instr
float32	F2F.F64.F32	/	F2FP.PACK_AB	F2FP.BF16.PACK_AB
half	Multi-Instr	HADD2.F32	/	NA
bfloat16	Multi-Instr	PRMT	NA	/

24.2.6 超越函数

对于浮点数的运算，还有一些特殊的函数，如 exp 指数，log 函数，三角函数（sin、cos 等），倒数 rcp，平方根 sqrt 等，这些函数在 NVidia 指令集架构中是利用特殊函数单元实现（SFU = special function unit），其底层实现是将这些函数分段然后在某一个小段内使用二次函数进行逼近实现（y = ax^2 + bx + c），对于不同区间段通过提供不同的 a，b，c 系数则可以产生相对高精度的结果。也就是说这类特殊函数是通过查表得到系数 a/b/c，然后利用二次函数得到最终结果。常见的指令如

```
MUFU.EX2 R7, R6 ;
MUFU.SIN R7, R7 ;
MUFU.COS R7, R7 ;
MUFU.LG2 R7, R0 ;
MUFU.RCP R7, R6 ;
MUFU.RSQ R5, R2 ;
MUFU.SQRT R7, R13 ;
MUFU.TANH R11, R4 ;
```

这些函数的精度都是偏低的，对于高精度的需求 NVidia NVCC 编译器会通过软件的形式对上面结果进行修正，得到更高精度的结果。如果我们对上述计算要求的精度不高可以使用这些函数的快速版本，一般是在函数前加__前缀，如`sinf()`的快速版本`__sinf()`。也可以在编译选项中启用`use_fast_math`选项来全局使能。

24.2.7 除法

前面介绍了四则运算中的加法、减法、乘法（以及融合的乘加运算），对于除法运算，NVidia 指令集中没有提供满足 IEEE 标准的除法指令，而是通过低精度的特殊函数单元提供的 MUFU.RCP 倒数能力结合泰勒展开利用 FADD、FMUL、FFMA 指令进行高阶修正得到，这些高阶修正算法可以通过分析 SASS 指令得到，如 normal 数的除法的 5 阶 Taylor 修正大概如下

```
MUFU.RCP R8, R5 ;                                   /* 0x0000000500087308 */
                                                           /* 0x000e220000001000 */
FADD.FTZ R10, -R5, -RZ ;                            /* 0x800000ff050a7221 */
FFMA R3, R8, R10, 1 ;                               /* 0x3f80000008037423 */
FFMA R12, R8, R3, R8 ;                              /* 0x00000003080c7223 */
FFMA R3, R7, R12, RZ ;                              /* 0x0000000c07037223 */
FFMA R8, R10, R3, R7 ;                              /* 0x000000030a087223 */
FFMA R11, R12, R8, R3 ;                             /* 0x000000080c0b7223 */
FFMA R7, R10, R11, R7 ;                             /* 0x0000000b0a077223 */
FFMA R3, R12, R7, R11 ;                             /* 0x000000070c037223 */
```

除了前面的计算类指令外，浮点数还有相关的控制流，比较大小，Normal 数判断，取整等，它们共同协作完成浮点数相关的计算任务

```
FSETP
FMNMX
FCHK
FRND  FRND.CEIL  FRND.F16.FLOOR  FRND.F64.FLOOR  FRND.FLOOR
```

24.3　总结

本文回顾了浮点数的表示和圆整等基础信息，介绍了 NVidia GPU 指令集架构中的浮点数的相关运算，继而介绍了浮点相关的加法、乘法、乘加指令和特殊函数对浮点数的操作，以及随路的 abs 能力和取负能力，同时介绍了各种数据类型直接的相互转换指令，了解这些指令能够使我们更清晰的了解硬件对浮点数计算的支持情况，指导我们对算法的设计和数据类型的选择（如 FFMA 替换 FMUL + FADD，HALF2 packed 类型替换单元素类型）。

24.4　参考

Exposing Floating Point –Bartosz Ciechanowski[2]
https://ieeexplore.ieee.org/xpl/conhome/1000125/all-proceedings
https://iremi.univ-reunion.fr/IMG/pdf/ieee-754-2008.pdf
https://web.ece.ucsb.edu/~parhami/pubs_folder/parh02-arith-encycl-infosys.pdf
https://en.wikipedia.org/wiki/Floating-point_arithmetic
https://docs.oracle.com/cd/E19957-01/800-7895/800-7895.pdf
CUDA Toolkit Documentation[3]

[2] https://ciechanow.ski/exposing-floating-point/
[3] https://docs.nvidia.com/cuda/cuda-math-api/group__CUDA__MATH__INTRINSIC__SINGLE.html

第二十五章　NVidia GPU 指令集架构-整数运算

作者：reed（知乎 @reed-84-49，专栏「CUDA 高性能编程」）　原文：https://zhuanlan.zhihu.com/p/700921948

前文我们介绍了 NVidia GPU CUDA Core 上的浮点运算指令 (https://zhuanlan.zhihu.com/p/695667044)，CUDA Core 除了提供浮点能力外还提供了整数运算能力，整数运算能力在整个计算体系中扮演着至关重要的作用：如数据处理方面的统计、排序、计数、地址计算、索引；算法实现中的加密计算和验证。在大语言模型背景下，低比特的数据量化表示（如 4bit 量化）也是在对整数的进一步延伸，了解整数的表示以及其之上的运算是我们进行设计和优化的基础。本文将介绍 NVidia GPU 指令集架构中的整数运算指令。文章结构方面，首先介绍常用的整数类型和 NVidia GPU 的支持情况，然后介绍有符号整数的补码表示及其衍生的算术位移和逻辑位移的概念，然后文章具体介绍了 NVidia GPU 的整数相关的指令，最后对文章进行了总结。

25.1　常用整数类型及其表示

在计算机编程中，常用的整数数据类型有以下几种：

`int8_t`、`int16_t`、`int32_t`、`int64_t`、`uint8_t`、`uint16_t`、**`uint32_t`**、`uint64_t`，它们占用的字节数和数据范围如下表：

符号	类型	字节数	最小值	最大值
有符号	`int8_t`	1	-128	127
有符号	`int16_t`	2	-32768	32767
有符号	`int32_t`	4	-2147483648	2147483647
有符号	`int64_t`	8	-9223372036854775808	9223372036854775807
无符号	`uint8_t`	1	0	255
无符号	`uint16_t`	2	0	65535
无符号	`uint32_t`	4	0	4294967295
无符号	`uint64_t`	8	0	18446744073709551615

25.2　NVidia GPU 对不同整数类型的支持

NVidia GPU 针对不同的计算精度需求提供了不同的整数精度和类型支持，具体地，在 CUDA 编程方面，它支持以上整数类型（`int8_t`、`int16_t`、`int32_t`、`int64_t`、`uint8_t`、`uint16_t`、**`uint32_t`**、`uint64_t`）的计算，在指令层面，由于 NVidia GPU 的寄存器为 32bit，以加法为例对于小于等于 32bit 的计算，都使用同样的指令实现（IADD3 指令），对于大于 32bit 的加法，则通过两条指令实现，分别计算低 32bit 加法结果和高 32bit 的加法结果，同时在计算低 32bit 的结果时，使用 Predication 寄存器 (https://zhuanlan.zhihu.com/p/688616037) 保存其进位情况，在计算高 32bit 时同时附带上低 32bit 的进位结果。

如图 1 所示，GPU 编程虽然语言层面支持不同的整数数据类型和精度，但硬件和指令集层面这些不同的类型加法都使用同一个加法器（如图 1 中的 Adder），其为一个 32bit 的整数加法器其通过 carry-0 可以指定低位的进位情况同时通过 carry-1 表示加法的向上进位情况。对于 8bit、16bit、32bit 的加法，carry-0 默认为 0，8bit、16bit 的数据在寄存器都被扩展为 32bit，直接采用 32bit 的加法器进行计算，在存储结果时对结果进行截断即可（只保留低 8bit、16bit）。对于 64bit 数据的加法，需要两阶段完成（编译器生成两条 32bit 加法指令），第一阶段计算低 32bit 的加法，和前面的计算过程类似，只需要 carry-0 设置为 0 进行计算，同时在计算过程中将 carry-1 保存在 Predication 寄存器中，第二阶段计算高 32bit（即 32-63bit），同时将第一阶段的 carry-1 设置为第二阶段的 carry-0 以确保进位被正确传递。

由于指令集方面只有 32bit 的计算指令，所以在 GPU 程序中采用低 bit 的整数计算并没有计算性能优势，反而低 bit 会引入截断指令，继而降低程序效率。对于大语言模型而言低 bit 量化可以减少内存使用量和数据 IO 开销，更多的是利用其存储空间优势，而非计算优势。

图 25.1: Figure 1. 单一加法器支持不同精度的整数加法

25.3　有符号整数的补码表示和算术位移

在计算机中为了简化了计算机内部的运算过程，统一加法和减法的处理，统一有符号和无符号运算，提高计算效率，在表示有符号数据时采用了补码的形式。计算机体系中数据都是通过多 bit 的 0、1 来表示，GPU 也不例外。为了更好的表达有符号数据，可以在加法的基础上引申出负数的表示。如一个有符号的 8bit 的整数 123，则其用原码表示为 0b01111011，如图 2 所示，我们在加法意义上可以定义-123，使得它和 123 加起来后可以得到 0（有符号 8bit 表示为 0b00000000），那么如何定义-123 可以满足这种加法逻辑呢，那就是先让 123 + ? = 0b11111111，然后两边同时加 1 即可以在向上溢出的意义下满足全 0。为了能得到全 1 我们可以针对 123 的二进制表示每个 bit 取反，然后把后面的 +1 也作用在-123 的二进制表示上即可，这也就是教科书上的" 取反加一"。这种取反加一的表示码称为补码（Two's Complement），即能够补（相加）为 0 的表示码，这样符号位也可参与运算，如此便形成了高位第一位（Most Significant Bit）表示符号位 0 表示正数，1 表示负数。

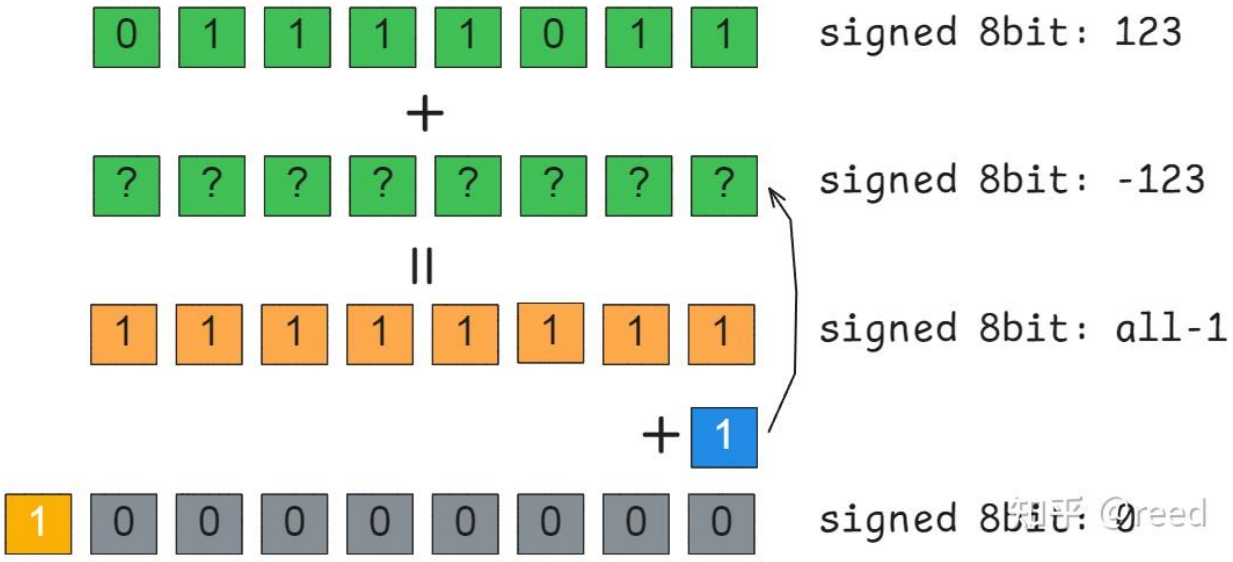


图 25.2: Figure 2. 有符号数的补码表示

由于 MSB 表示符号，在进行比特级左右移时（Left or Right Shift），是否保留符号位产生了逻辑位移（Logical Shift）和算术位移（Arithmetic Shift）：

逻辑位移不考虑数的符号位，只是简单地将每一位向左或向右移动指定的位数，移出边界外的位将被舍弃，新的位置上会填充 0。

- 逻辑左移：所有位向左移动，右边空出的位用 0 填充。
- 逻辑右移：所有位向右移动，左边空出的位用 0 填充。

逻辑位移通常用于无符号数的运算，比如乘以 2 的幂次方（左移）或除以 2 的幂次方（右移）。
算术位移则考虑数的符号位，主要用于保持数值的符号不变。

- 算术左移：与逻辑左移相同，所有位向左移动，右边空出的位用 0 填充。
- 算术右移：所有位向右移动，但左边空出的位会根据原数的符号位填充，若原数为正，则填充 0；若原数为负，则填充 1。

算术位移通常用于有符号数的运算，尤其是除法运算时，它能够保持数值的正负号不变。
在 NVidia GPU 指令方面，逻辑位移和算术位移统一为 SHF 指令，通过有符号和无符号类型作为 Modifier 来实现对符号位的保留与否。针对大模型中的低 bit 量化，通常采用的是算术位移，同时附加上其他的逻辑操作实现低 bit 的数据合并和表示。

25.4 NVidia GPU 整数指令集

NVidia GPU 指令集架构提供了针对整数的各种计算和操作，该章节分类介绍各个计算相应的指令。值得注意的是不同的 GPU 架构整数计算和浮点计算的吞吐有很大差别 (https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#arithmetic-instructions)，选择合适的计算类型能有效的提升程序性能，切记不要简单的以为整数效率比浮点高。

25.4.1 整数加法

整数加法通过`IADD3`（Integer ADD 3 operand）实现（`d = a + b + c`），一般形式如下，其中 8bit、16bit、32bit 的加法共用如下形式，针对 64bit 的加法，示例中 P0 为输出寄存器，类型为 Predication Register，其功能如图 1 中的 carry-0，用以输出低 32bit 计算时的进位情况，然后将这个进位带给高 32bit 的计算`IADD3.X`，通过这两条指令实现 64bit 的加法。整数减法通过随路取符号实现；将某个加数随路取负、其余加数置零（如 `IADD3 R7, RZ, -R0, RZ`），即可实现取相反数的能力（negate）`d = -a`。

```
// 8bit, 16bit, 32bit
IADD3 R7, R0, R7, RZ ; // R7 = R0 + R7 + RZ

// subtraction
IADD3 R7, R0, -R7, RZ ;

// 64bit R6-7 = R4-5 + R6-7
IADD3 R6, P0, R4, R6, RZ ;  // (R6, P0) = R4 + R6 + RZ; P0 indicate carry
IADD3.X R7, R5, R7, RZ, P0, !PT ; // R7 = R5 + R7 + RZ + P0;
```

25.4.2 整数乘法

指令集没有提供独立的整数乘法，而是以乘加合并的形式提供（Integer Multiply ADd），提供形如 `d = a x b + c` 的整数计算能力，除了标准能力外，该指令还以 Modifier 的形式提供了，只算高 32bit 的 HI；通过 a、b、c 给特定值实现`MOV(a = 0, b = 0)`，`ADD`语义`(b = 1)`，`SHL`语义（`b = 2,4,..`），以及 64bit 的支持 WIDE，常见的带 Modifier 指令如下，对于 c 可以随路取负实现 `d = a x b - c` 的计算语义：

```
IMAD
IMAD.HI
IMAD.HI.U32
IMAD.IADD
IMAD.IADD.U32
IMAD.MOV
IMAD.MOV.U32
```

```
IMAD.SHL
IMAD.SHL.U32
IMAD.U32
IMAD.U32.X
IMAD.WIDE
IMAD.WIDE.U32
IMAD.WIDE.U32.X
IMAD.X
```

25.4.3　整数位移

位移（SHFit）通过 Modifier 指定移动方向向左（Left）或者向右（Right），同时通过 U32、U64 等指定类型和或逻辑位移，通过 S32、S64 指定类型和或算术位移。

```
SHF.L.U32
SHF.L.U32.HI
SHF.L.U64.HI
SHF.L.W.U32
SHF.R.S32.HI
SHF.R.S64
SHF.R.U32.HI
SHF.R.U64
```

25.4.4　整数除法和取余

NVidia GPU 指令集架构中并没有提供独立的整数除法指令，而是先将整数转换为浮点数，通过 SFU（Special Function Unit 提供的倒数指令，参看浮点计算中的除法（https://zhuanlan.zhihu.com/p/695667044））完成除法计算，然后再转换为整数进行结果修正。对于除数为编译时常量的，编译器会使用数学变换将该除法变换成更简单的位移和乘法指令（https://gmplib.org/~tege/divcnst-pldi94.pdf），cutlass 中也有类似实现（https://github.com/NVIDIA/cutlass/blob/main/include/cutlass/fast_math.h#L364）。

25.4.5　整数转浮点数

对于整数和浮点数类型的转换（Integer to Float），默认为 int32 转换为 float32，指令集也提供了相应的功能，通过 Modifier 提供更细节的修饰，涉及整数到 float16（F16）、float64（F64）类型，同时源操作数也区分是有符号和无符号，和低精度 (S8、S16、S64、U16、U32、U64)，以及 Round Positive（RP）：

```
I2F
I2F.F16
I2F.F64
I2F.F64.S64
I2F.F64.U32
I2F.F64.U64
I2F.RP
I2F.S16
I2F.S64
I2F.S8
I2F.U16
I2F.U32
I2F.U32.RP
I2F.U64
I2F.U64.RP
```

25.4.6　整数转 Packed 整数

指令集除了提供整数和浮点数的转换，还提供了整数和 packed 整数的转换（Integer to Integer Packed）来避免使用更复杂的逻辑和位移实现，如下

```
I2IP.S8.S32.SAT
```

25.4.7 整数绝对值

原生提供了绝对值计算指令 IABS（Integer ABSolute）

```
IABS
```

25.4.8 整数点积

针对低精度整数的点积运算（Integer Dot Product），对于深度学习场景，针对特别小的 channel，利用该指令可以实现较好的效果：

```
IDP.2A.LO.U16.U8
IDP.4A.S8.S8
```

25.4.9 整数最大最小

指令集提供整数两个数据比较大小的操作（Integer MiNimum and MaXimum），同时选取出其中的大数或小数，避免使用条件分支实现，同时提供了针对数据类型 Unsigned Integer 的 Modifier，对于 32bit 及以下数据都可以使用该指令实现，对于 64bit 数据使用`ISETP`实现：

```
IMNMX R7, R0, R7, !PT ; // when PT return R7 = min(R0, R7), when !PT R7 = max(R0, R7)
IMNMX.U32
```

25.4.10 整数比较产生条件

对于更复杂的比较和逻辑（Integer SET Predicate），常见的整数相关的条件如下，组合它们可以得到更复杂的控制逻辑：

```
ISETP.EQ.AND      ISETP.EQ.U32.OR  ISETP.GE.OR.EX       ISETP.GT.AND.EX      ISETP.GT.U32.OR   ISETP.LT.
    AND      ISETP.LT.U32.AND.EX  ISETP.NE.OR.EX
ISETP.EQ.AND.EX   ISETP.EQ.XOR     ISETP.GE.U32.AND     ISETP.GT.OR          ISETP.LE.AND      ISETP.LT.
    AND.EX   ISETP.LT.U32.OR      ISETP.NE.U32.AND
ISETP.EQ.OR       ISETP.GE.AND     ISETP.GE.U32.AND.EX  ISETP.GT.OR.EX       ISETP.LE.OR       ISETP.LT.OR
           ISETP.NE.AND         ISETP.NE.U32.AND.EX
ISETP.EQ.OR.EX    ISETP.GE.AND.EX  ISETP.GE.U32.OR      ISETP.GT.U32.AND     ISETP.LE.U32.AND  ISETP.LT.
    U32.AND  ISETP.NE.OR
ISETP.EQ.U32.AND  ISETP.GE.OR      ISETP.GT.AND         ISETP.GT.U32.AND.EX  ISETP.LE.U32.OR   ISETP.LT.OR
    .EX    ISETP.NE.OR
```

25.4.11 Tensor Core 整数指令

另外 NVidia GPU 通过 Tensor Core 提供了算力更高的整数类型矩阵乘法指令，我们会在后面章节更详细的介绍：

```
IMMA.16816.S8.S8
IMMA.16832.S8.S8.SAT
IMMA.8816.S8.S8.SAT
```

25.5 总结

本文介绍了编程中常用的整数类型和 NVidia GPU 的统一加法器逻辑对它们的支持情况，同时介绍了有符号数的补码表示以及其衍生出的算术位移和逻辑位移的区别，最后文章详细介绍了 NVidia GPU 针对整数运算的各种指令，了解整数的表示和计算逻辑以及底层指令能帮助我们选择合适的计算精度，其对更高阶的优化有重要的指导意义。

25.6　参考

https://gmplib.org/~tege/divcnst-pldi94.pdf

https://github.com/NVIDIA/cutlass/blob/main/include/cutlass/fast_math.h#L364

https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#arithmetic-instructions

reed：NVidia GPU 指令集架构-寄存器 (https://zhuanlan.zhihu.com/p/688616037)

reed：NVidia GPU 指令集架构-浮点运算 (https://zhuanlan.zhihu.com/p/695667044)

第二十六章　NVidia GPU 指令集架构-比特和逻辑操作

作者：reed（知乎 @reed-84-49，专栏「CUDA 高性能编程」）　原文：https://zhuanlan.zhihu.com/p/712356884

前面文章我们介绍了 NVidia GPU(https://zhuanlan.zhihu.com/p/686198447) 中常用的浮点运算 (https://zhuanlan.zhihu.com/p/695667044) 和整数运算指令 (https://zhuanlan.zhihu.com/p/700921948)，除了浮点和整数运算外，比特和逻辑操作也是重要的基础运算。比特操作（Bit Operation），即位操作，是计算机科学中对二进制位进行操作的运算，主要包括单 bit 的位与（AND）、位或（OR）、位非（NOT）、位异或（XOR）和多 bit 的计数等操作（如 bit reverse 等）。位操作应用广泛且至关重要，其在数据压缩与优化存储、加密算法等方面有重要应用。NVidia GPU 针对 bit 操作在指令集层面提供了丰富的支持。本文将重点介绍 NVidia GPU 指令集层面的 bit 操作能力。文章结构方面，首先介绍了无类型的寄存器表示，然后依次介绍了简单的指令：POPC, FLO, BREV, SGXT, BMSK，最后文章详细介绍了功能复杂的 PRMT 和 LOP3 指令。

26.1　无类型的寄存器表示

计算机中的数据是二进制表示的，对于 NVidia GPU 体系中一个寄存器可以存储 32 位，也就是说寄存器中是 32 个 0 和 1 的序列 (https://zhuanlan.zhihu.com/p/688616037)，如图 1 所示，一个寄存器表示 32bit 的 0 和 1，寄存器只是 0/1 序列表示，本身没有数据类型。只有当具体的指令处理这些数据时候会根据指令的语义来解读该序列的 0/1 表示。如当该寄存器作为有符号加法（IADD3）指令 (https://zhuanlan.zhihu.com/p/700921948) 的输入时，指令会把该寄存器的最高位（MSB）作为符号位来处理；如果将该寄存器给浮点的乘法指令（FMUL）(https://zhuanlan.zhihu.com/p/695667044)，则浮点数处理单元会把最高位解释为浮点数的符号位，紧接着 8bit 解释成指数位，再其后的 23bit 解释为尾数。

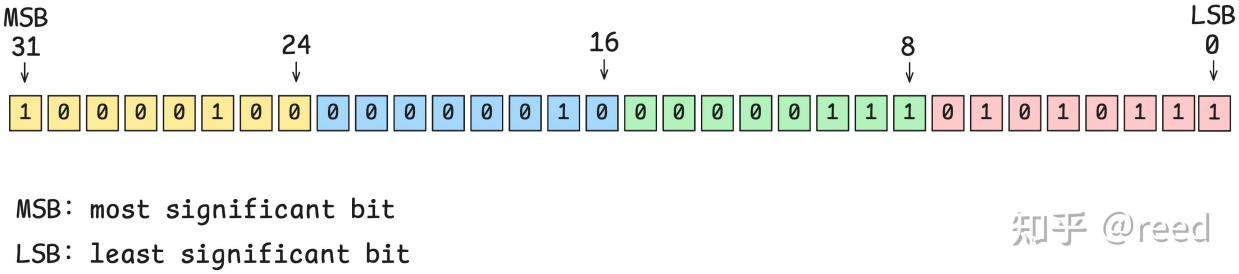


图 26.1: Figure 1. NVidia GPU 中 32bit 寄存器

也就是说寄存器只存储数据，并没有具体的数据类型其代表的值的概念，bit 操作同样的不关注操作数的类型，只针对其中的每一位的 0 和 1 进行处理。

26.2　POPC 指令

每一个 bit 位存储着 0 或 1，NVidia GPU 提供了 POPC（POPulation Count）指令，它可以数出 32bit 寄存器数据中 bit 为 1 的个数，图 2 展示了该指令的功能。

具体的指令如下，其表示计数 R2 寄存器中 bit-1 的个数，将其结果存储在 R7 中：

```
1 POPC R7, R2;
```

CUDA device API 提供了__popc(unsigned int)来封装该指令，针对 64bit 的数据，虽然 api 提供了__popcll(uint64_t)类的函数，但是其内部会对高 32bit 和低 32bit 分别调用 POPC 指令，然后将高低 bit 的 POPC 结果通过整数加法求和得到。

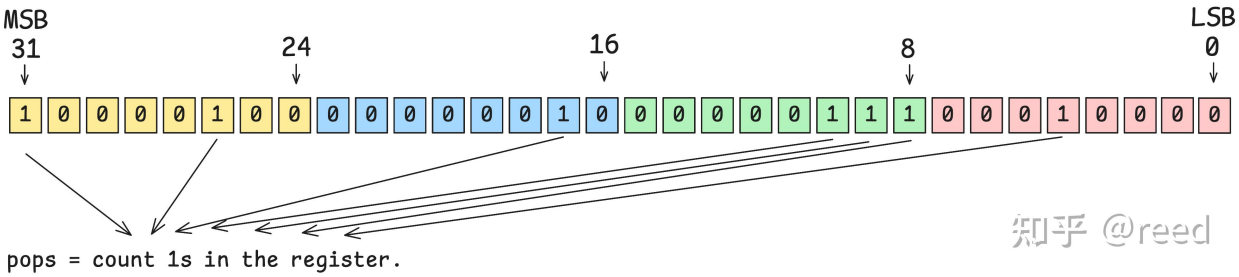


图 26.2: Figure 2. Population Count 指令功能

26.3　FLO 指令

FLO（First Leading One）指令，从高位（MSB）向低位（LSB）查找第一个为 1 的位置（index），最低位的位置为 index 记为 0，向高位依次增加，最高位为 31，如果输入 bit 全为 0，则返回 0xFFFFFFFF，输入 bit 全为 1 则返回 31。

```
FLO.U32 R0, R2 ;
```

指令集只提供 32bit 的计算能力，针对 64bit 的计算任务，通过多条指令组合出 64bit 的计算逻辑，同时该指令还针对有符号数据提供了区分正数和负数的处理逻辑，正数则找从高位开始找第一个 1，负数则从最高位开始找第一个 0，同时该指令还提供了 SH modifier，该 modifier 可以返回将前面的 index 调整到符号位置所需要的左移量，整体指令如下:

```
FLO R7, R2 ; // signed find leading one
FLO.U32.SH R7, R2 ;
FLO.SH R7, R2 ;
```

该指令在 PTX 层面通过bfind触发。

26.4　BREV 指令

BREV指令实现比特逆序的功能（Bit REVerse），实现 32bit 数据的高低位交换。

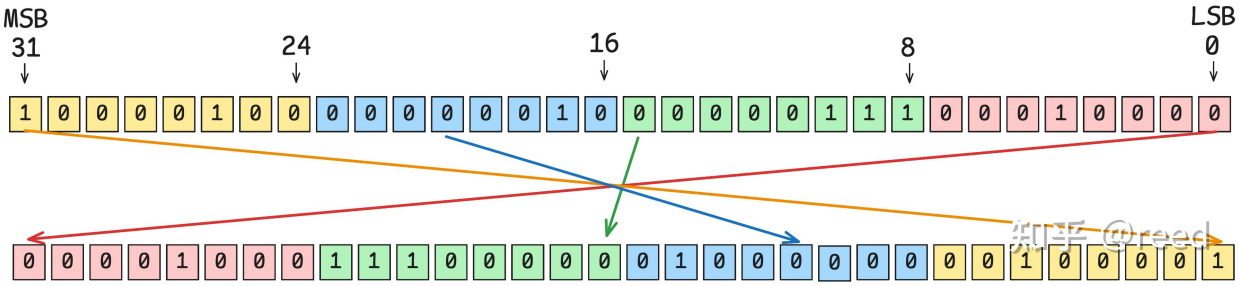


图 26.3: Figure 3. Bit Reverse 指令功能

```
BREV R7, R2 ;
```

针对 64bit 的数据 reverse 功能，可以通过两条 BREV 指令实现。

26.5　SGXT 指令

SGXT（SiGn eXTend）指令，实现特定 bit 位数据的符号位扩展，

```
SGXT R7, R0, R7 ;  // nbit = R7
```

其中输入操作数 R7 指定操作数当做多少 bit 的数据（如 4bit 数据，其中最高位为符号位），以上指令可以将 R0 的第 R7 - 1bit 扩展到填充至高 32bit。其也可以通过前面介绍的算术左移和右移 (https://zhuanlan.zhihu.com/p

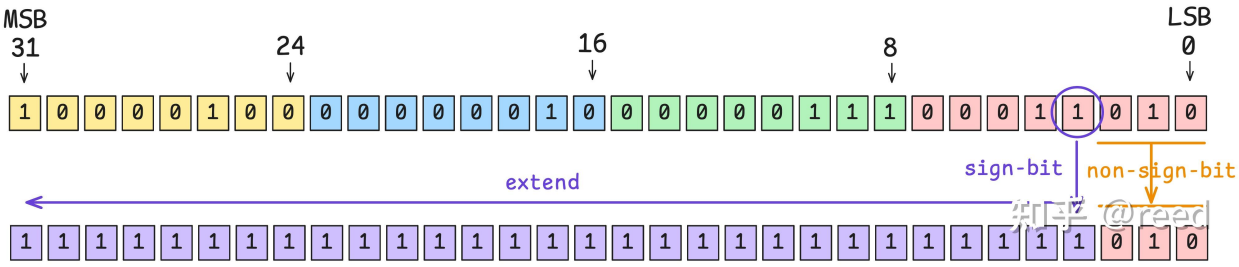


图 26.4: Figure 4. Sign Extend 指令功能

/700921948) 实现，具体的如下

```
R7 = (R0 << (32 - R7)) >> (32 - R7)
```

通过位移实现需要的操作指令会更多，效率方面相较于 SGXT 也会更低。

26.6 BMSK 指令

BMSK（Bit MaSK）指令可以返回一个 32bit 的数据，从其从 start 位置开始连续填充 mask 个 1，其余位置位 0，如图 5 所示，

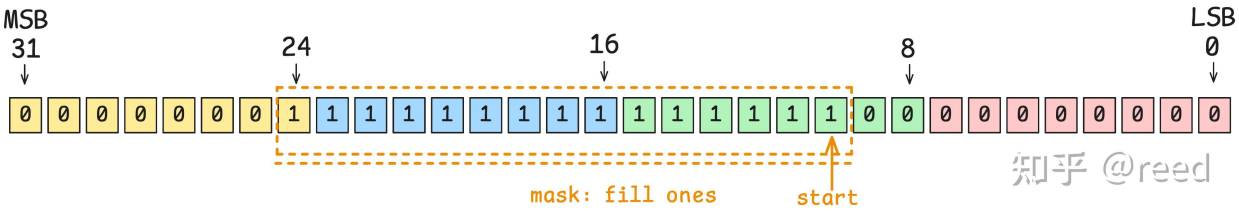


图 26.5: Figure 5. Bit Mask 指令功能

```
BMSK R2 R0 R1; // start: R0, mask: R1;
BMSK R5, R5, 0x3 ;
```

26.7 PRMT 指令

PRMT（PeRMuTe）指令可以实现按照 8bit（一 byte）为基础单位，根据 selector 指定的位置数据重新选择和排列。如下指令示例，该指令接受三个输入 Ra、Rb、Rsel，产生一个 Rd。如图 6 所示，Ra 每 8bit 构成一个最小的 byte 单位，从低到高被编号为 b0, b1, b2, b3; Rb 每 8bit 表示一个最小单位，从低比特位向高位依次被标记为 b4, b5, b6, b7。Rsel 寄存器用来指定 b0-7 的编号，即从 b0-b7 中选择 4 个字节（可以重复）重新输出到一个 32bit 的寄存器中。Rsel 需要选择 4 个编号来完成 32bit 的输出，每个字段占用 4bit，其中每个字段的低 3bit 表示 operand a/b 中选取的 byte 的编号，4bit 中的高 bit 表示该选择的编号位置是原值复制还是重复符号位（0 表示原值复制，1 表示填充符号位）。也就说结果中的高 8bit（24-31bit）是由 Rsel 中的 12-15bit 决定的（s3=b0），其中 12-14bit 用来决策选择 operand a/b 中的第几个 byte，如图所示，其 bx=0b000=0 表示选择 operand a/b 中的 b0 段，且最高位（15bit）位 0 表示原值复制，这样 operand a b0 位置的数据则复制到输出位置（24-31bit），类似地，输出位置的 16-23bit 由 Rsel 的第二个 4bit（s2=0b111=7）控制，表示选择 b7，即 operand b 中的 b7 所对应的 8bit 数据，同时该控制 Rsel 4bit 中的最高位为 0 表示复制原值，输出位置的 8-15bit 来自 operand a 的 b1 位置，输出位置 0-7bit 由 Rsel 的 s0（s0=0b1011）控制，其中低 3bit 表示来自于 b 的序号，为 3，高 1bit 表示复制符号位，则其综合表示复制 b3 的符号位，且将该符号位填充输出的 8bit，结果上表现为 Rd 的 0-7bit 为全 1。

```
// PRMT Rd Ra Rb Rsel;
PRMT R7 R4 R5 R6;
```

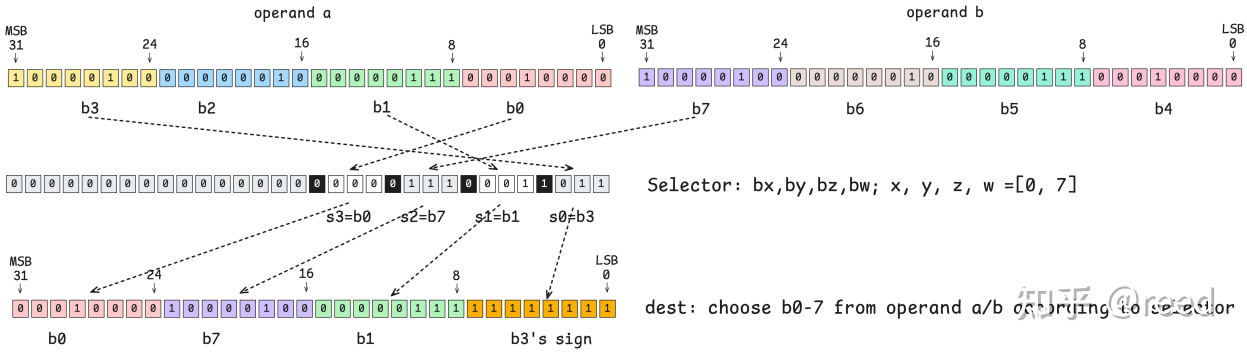


图 26.6: Figure 6. Permute 指令功能

PRMT 指令还提供了其他的 modifier 来简化上面的操作，但是功能方面和上面是一致的，此处就不再赘述，更详细的 modifier 的介绍可以参考 PTX 文档。

26.8　LOP3 指令

`LOP3`（Logical OPeration on 3 inputs）指令实现三个操作数的按位的 bit 运算。如三个操作数的位与`d = a & b & c;`更复杂的 `d = a & b ^ (~c);` 等。该指令可以实现三操作数的位逻辑运算自然地其也可以实现两个操作数的逻辑位运算和一个操作数的取反等操作，指令形式如下，其中第一个 R7 为输出，R0、R7、R6 为三个输入操作数，0xf 为查找表的表项 (记为 imm)，!PT 暂时不考虑：

```
LOP3.LUT R7, R0, R7, R6, 0xf, !PT ;
```

该指令是通过查表（Look Up Table）方式实现，所以其指令都有 LUT modifier。查表法执行逻辑为：依次提取三个输入操作数 A、B、C 的特定 bit 位（如从 0bit 到 31bit），每次可以得到一个 3bit 的输入数据，如图 7 中的 ABC 中的黑框所示（0b001 序列），将该 3bit 数据输入给 LookUp Table, 在查找表中找到该三 bit 对应的行，然后找到 immLut 对应的位，即可得到一个 1bit 的输出值（图示为黄色 1），将该值放置在输出 D 的对应 bit 位置即可（如图中下半部分斜箭头所示）。遍历输入操作数 ABC 的所有 bit，便可以通过查表 immLut 得到所有的 D 对应位置的输出。

我们不难发现，查找表的本质为三个输入 bit 的全排列，每一种排列形式都确定一个输出 bit 值这样三操作数的任意逻辑结果便可以表示，不难发现图中的 Ta = 0xF0, Tb = 0xCC, Tc = 0xAA，将 immLut 设置为 Ta、Tb、Tc 的逻辑组合结果便可以得到 3bit 组合逻辑到一个输出空间的全映射。如我们要计算三个操作数的与运算（`A & B & C`），本质上是要求当输入 3bit 数据为 0b000 到 0b110 时输出 0b0，输入为 0b111 时输出 0b1，以此类推便可以得到 3bit 所表达的 8 个数的所有的映射值，即为 immLut 的值。对于更少的输入数的场景，该查找表方法依然有效，如我们要计算一个 32bit 数据的取反，我们可以把这个数作为 ABC 的任意一个操作数，则有三种计算路径，如 `D = ~A`, `D = ~B`, `D = ~C`, 这样我们分别对 Ta、Tb、Tc 取反可以得到不同的立即数查找表，分别是`immLut = 0x0F`,`immLut = 0x33`, `immLut = 0x55` , 这三种路径都可以实现对输入的取反，具体细节如下表

	方法一	方法二	方法三
路径操作数	A	B	C
公式	`D = ~A`	`D = ~B`	`D = ~C`
T 值计算	`~Ta`	`~Tb`	`~Tc`
ImmLut	0x0F	0x33	0x55
伪调用形式	`lop3(x, 0, 0, 0x0F)`	`lop3(0, x, 0, 0x33)`	`lop3(0, 0, x, 0x55)`
指令示例	`LOP3.LUT R7, R0, RZ, RZ, 0x0f, !PT ;`	`LOP3.LUT R7, RZ, R0, RZ, 0x33, !PT ;`	`LOP3.LUT R7, RZ, RZ, R0, 0x55, !PT ;`

以上三个方法都可以实现对特定数的取反，更复杂的操作可以通过对 Ta、Tb、Tc 作用得到。对 Ta、Tb、Tc 进行常见的一元二元和三元操作得到对应的 immLut 的值如图 8 所示，

反汇编常用的 cuda 库得到的 immLut 值和出现的频率和逻辑表达（未化简）情况如图 9 所示，

同时该 LOP3 指令还可以将输出结果与、或上一个额外的 Predication 值得到一个 Predication 输出来增强该指令的功能，具体的可以参考 PTX 文档和反汇编常见库得到。和 LOP3 类似，NVidia GPU 提供了`PLOP3`指令，用于计算多个 1bit Predication 的逻辑组合。

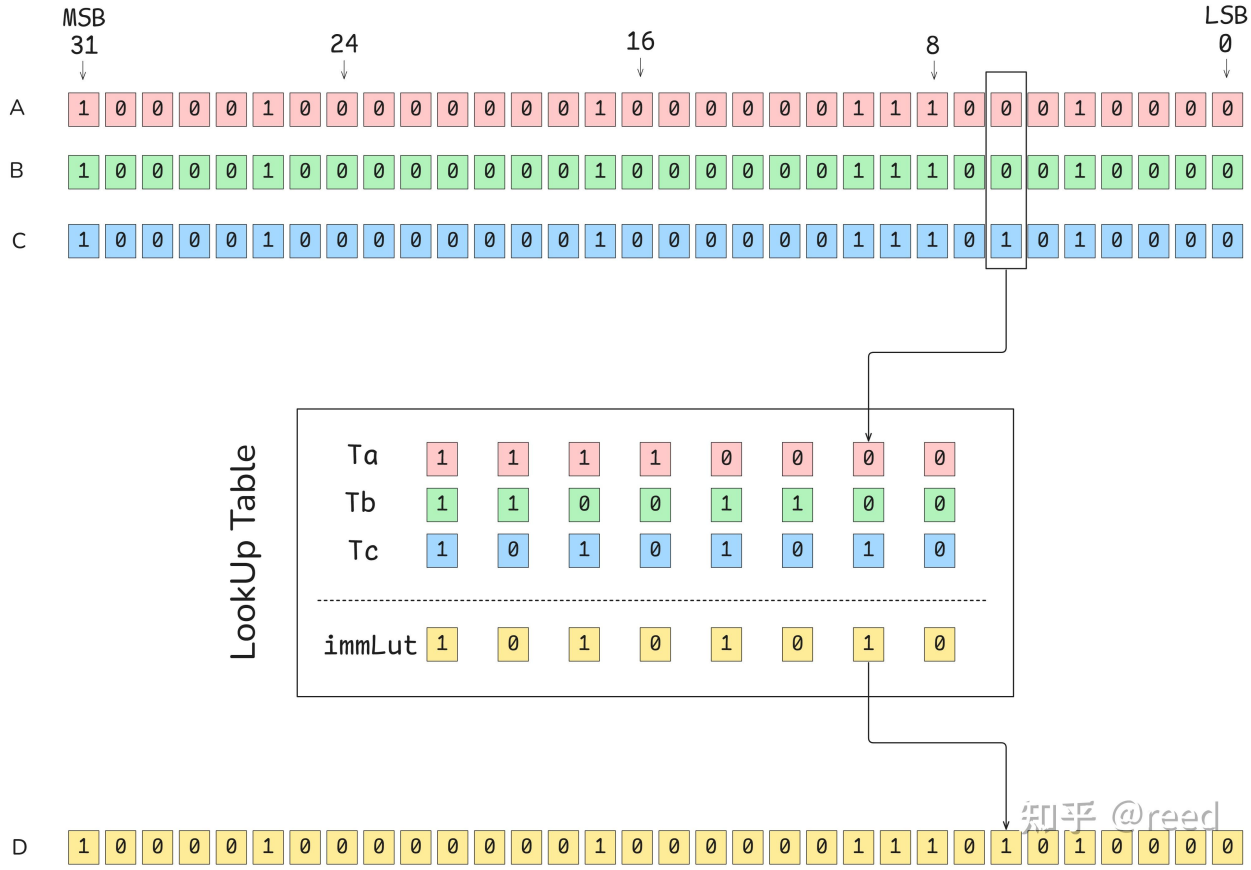


图 26.7: Figure 7. Logical Operation on 3 inputs 指令功能

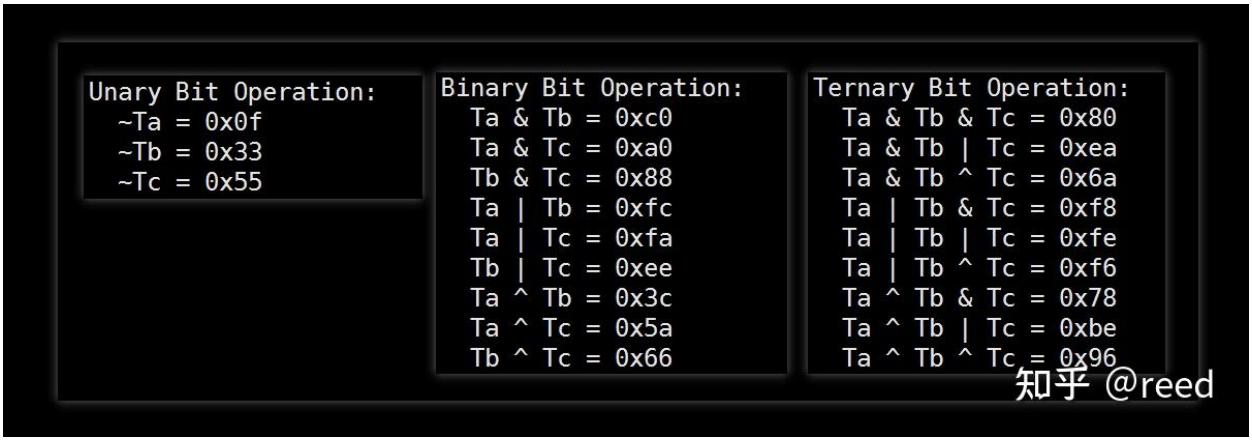
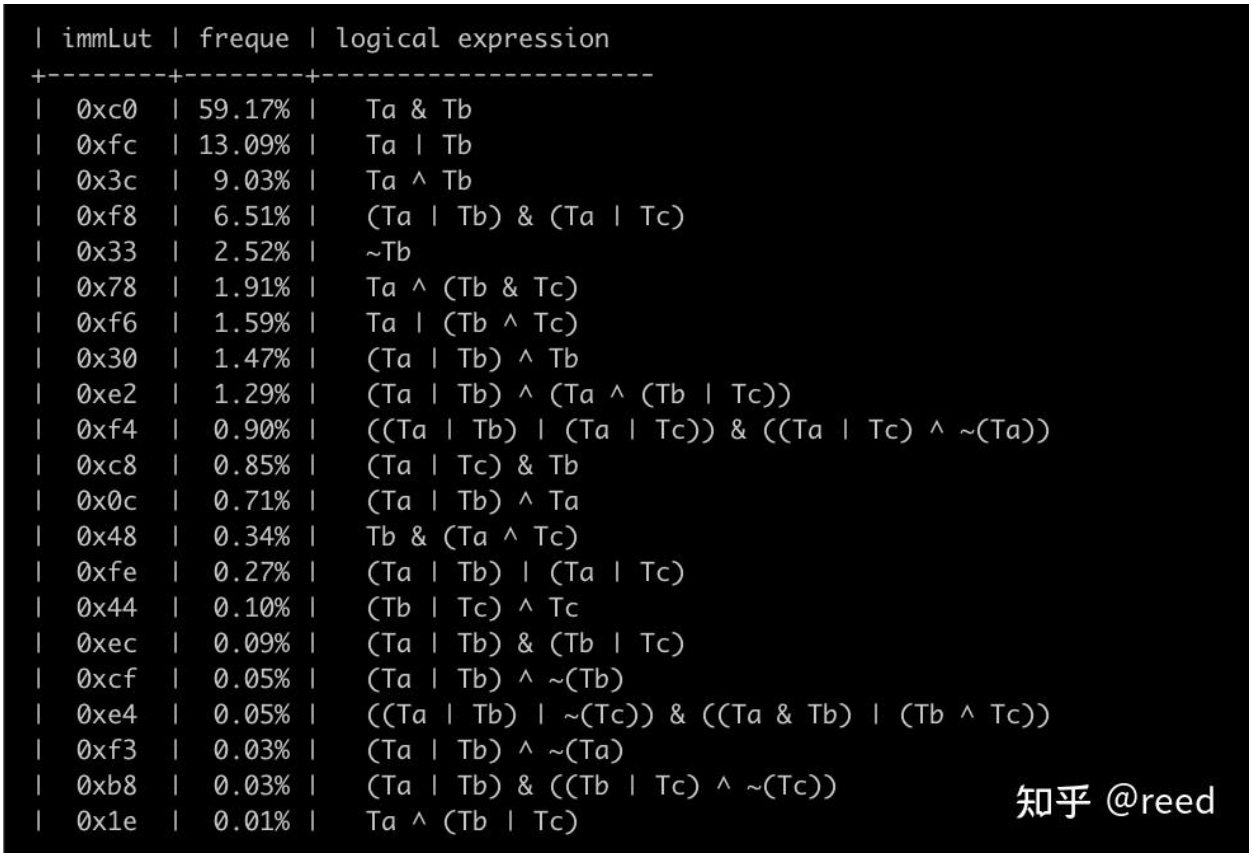


图 26.8: Figure 8. 常用的 bit 逻辑对应的 immLut 值



26.9　总结

本文以图示的形式介绍了 NVidia GPU 常用的 bit 操作指令如 POPC, FLO, BREV, SGXT, BMSK，同时详细介绍了相对复杂的 PRMT 和逻辑 LOP3 指令，了解这些指令的功能和计算对特定的优化有一定的指导意义，其可以帮助我们在遇到类似计算需求时选择合适的指令，提升运算效率。

26.10　参考

reed：NVidia GPU 指令集架构-前言 (https://zhuanlan.zhihu.com/p/686198447)

reed：NVidia GPU 指令集架构-浮点运算 (https://zhuanlan.zhihu.com/p/695667044)

reed：NVidia GPU 指令集架构-整数运算 (https://zhuanlan.zhihu.com/p/700921948)

reed：NVidia GPU 指令集架构-寄存器 (https://zhuanlan.zhihu.com/p/688616037)

PTX ISA 8.5(https://docs.nvidia.com/cuda/parallel-thread-execution/index.html#data-movement-and-conversion-instructions-prmt)

PTX ISA 8.5(https://docs.nvidia.com/cuda/parallel-thread-execution/index.html#logic-and-shift-instructions-lop3)

第二十七章　NVidia GPU 指令集架构-Warp 级和 Uniform 操作

作者：reed（知乎 @reed-84-49，专栏「CUDA 高性能编程」）　原文：https://zhuanlan.zhihu.com/p/712357647

前面文章我们介绍了 NVidia GPU 的 Load 指令 (https://zhuanlan.zhihu.com/p/692445145)、浮点计算指令 (https://zhuanlan.zhihu.com/p/695667044)、整数计算指令 (https://zhuanlan.zhihu.com/p/700921948)、比特和逻辑操作指令 (https://zhuanlan.zhihu.com/p/712356884)，逻辑上这些指令都是以单线程的模式工作的。除此之外，NVidia GPU 提供了 warp 级别的指令，这些指令不再是单线程的视角，而必须以 warp 作为整体来理解和执行，同时 NVidia GPU 提供了 warp 级别的 Uniform 寄存器和对应的指令集来提升 GPU 的效率。本文重点介绍了 NVidia GPU 上的 Warp 级别的指令和 Uniform 类的指令。在文章结构方面，本文首先介绍了 SIMT 模型和 Tensor 计算，然后介绍了 Warp Level 指令：DMMA、HMMA、IMMA、SHFL 等，接着介绍了 Uniform 指令，最后对本文进行了总结。

27.1　SIMT 体系下的 Tensor 计算

NVidia GPU 体系结构提供了单指令多线程的执行模型（SIMT: Single Instruction Multiple Thread），前端语言体现为 CUDA，用户在编写 CUDA 程序时，整体的思考模型是：

- 将一个可并行的问题划分为规格更小的原子的计算单元；
- 使用 CUDA 中单个线程完成以上计算任务；

在该过程中，专注的是局部的、原子的单个线程的执行逻辑。通过配置 Thread Block、Grid Block 来实现单个线程计算逻辑的横向扩展，以便构成更大规模的计算逻辑。图一中 Problem 部分展示了原始问题划分成单个计算任务的过程，这些子任务在 CUDA 中体现为线程的执行任务，也是 CUDA Kernel 的核心逻辑，这些计算逻辑可以更近一步地组织成线程块（Thread Block），线程块有一定的局部性，其中的线程可以做必要的同步和数据交换，线程块间相互独立，多个线程块可以继续组织成计算网格（Grid Block）。线程网格中的线程块会被调度到硬件上执行，线程网格整体构成的规格可以比硬件规格大很多，这些任务可以分批次的利用有限的硬件单元来完成计算，使得有限的硬件单元可以处理逻辑上无限的任务。

虽然 CUDA 编写时，思考和书写的是单个线程的计算逻辑，但在硬件执行时，真正被调度的最小单位是 Warp（线程束），其由从零开始的连续的 32 个线程组成，这 32 个线程会同时开始和结束，即锁步执行 (lock step)，逻辑上它们共享着相同的执行进度。在 Volta 之前的架构这个执行同步是被硬件约束的，这 32 个执行线程共享同一个 PC（Program Counter），在 Volta 及其之后的架构中各个线程有了自己独立的 PC，它克服了在一些竞争算法实现中锁步带来的死锁问题，但这并不是说这些线程可以自由且高效的独立的运行，只是说它们可以不锁步，在这个竞争区之后还会设立同步点进行同步操作，来确保 Warp 可以高效且同步的执行。

在 SIMT 意义下，可以认为每一个线程有自己独立的、私有的存储空间（划分后的寄存器），如图二所示，lane-0、lane-1、lane-31 表示 Warp 中的第一、二和最后一个线程，这些 lane 逻辑概念上包含了自己独立的寄存器状态。执行时，调度器选定一个可以运行的 warp，得到其运行的指令，如图示的`FADD R3 R2 R1`，来执行一条浮点数加法`R3 = R2 + R1`任务，将任务发送给执行单元（图示 SIMD EU），执行单元则能够并行地完成 32 个 lane 的`R3 = R2 + R1`任务，执行结束后，每一个 lane 中有自己独立的 R3 的结果，lane 之间相互没有影响，逻辑上可以认为每一个 lane 是独立执行的。

我们可以看到，在 SIMT 框架下寄存器资源是按照线程划分的，每一个线程都持有一组寄存器资源，用于表示该线程的计算和执行状态，并且逻辑上线程相对独立，它们之间不会相互影响，寄存器资源的使用也限定在单个线程内部。这种模型简单且高效，传统的 CUDA 类的编程都是在该线程私有寄存器框架下完成的。

随着深度学习的爆发和其对算力的强力需求，同时为了对抗 Google 的 TPU，NVidia 从 Volta 架构开始提供了 Tensor Core 计算单元，Tensor Core 计算单元可以在更短的指令周期内完成特定规格的矩阵乘法计算，极大的提高了 GPU 的算力能力，支持了深度学习对于矩阵类计算的需求，并且依赖这种优势迅速占领市场。在编程方面 NVidia 也相

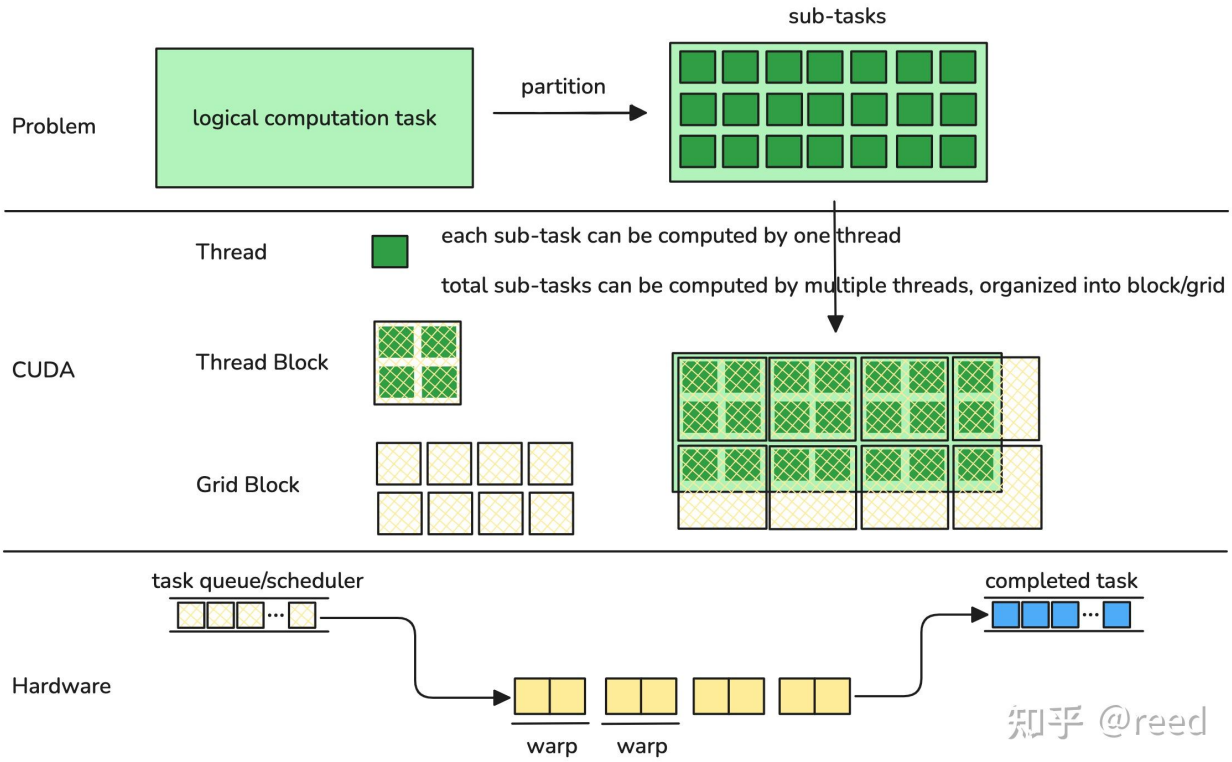


图 27.1: Figure 1. Partition Problem into small task then feed it to Hardware

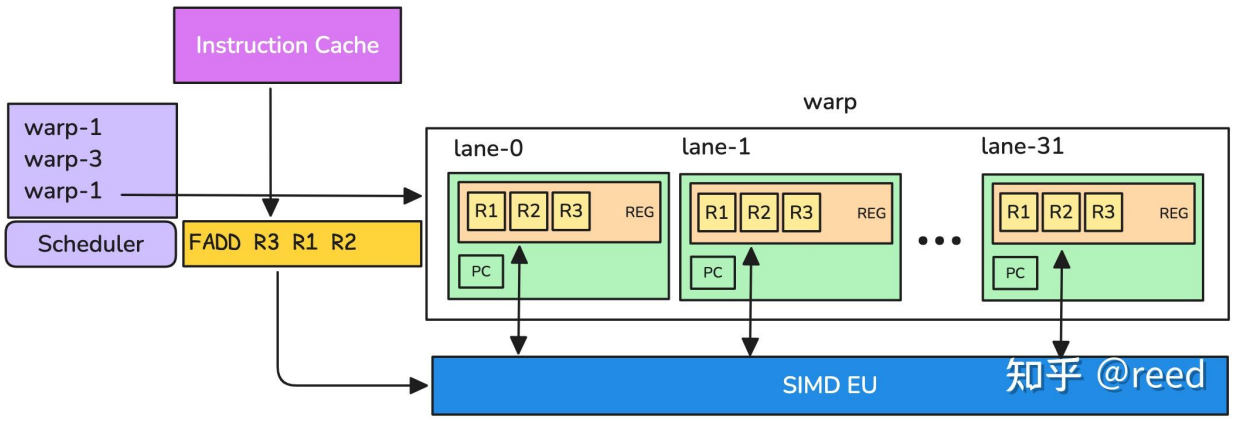


图 27.2: Figure 2. Lane holds Register and is an isolated logical unit

应的在 SIMT 的编程模型上扩展出相应的编程能力。如图 3 所示，Tensor Core 的引入，让 SIMT 的模型不再像之前那样：每一个 lane 有独立的存储和简单的执行逻辑。Tensor Core 执行时要求 warp 内的所有的 lane 都需要参与，并且每个 Lane 只提供一部分数据作为输入，这些数据整体参与运算，运算结果会再次分发到各个 lane 的寄存器。这种计算引入了 lane 间的数据共享和交换。这对于硬件模型是平凡的，因为寄存器文件本身只有一份，所谓的各个 lane 私有寄存器也是只是一种逻辑映射；但是对于软件的 SIMT 抽象或多或少是不平凡的，因为这类指令突破了单个线程完成自己线程内的私有、独立计算的逻辑，单线程的思考模型已经不再具备意义，这时需要同时考虑所有的线程提供数据，每一个 lane 只是这个计算中的一个部分（fragment），经过 Tensor Core 进行计算，计算的结果再次填充给各个 lane 的寄存器中，这种模式下各自 lane 的输入输出不在具有 lane 内的局部性，他们之间相互影响，即 lane-i 的输入会影响 lane-j 的输出（$i \neq j$）。由于该类指令要求 warp 内的所有 lane 同时参与，所以本文将该类型相关的指令称为 Warp-Level 指令。

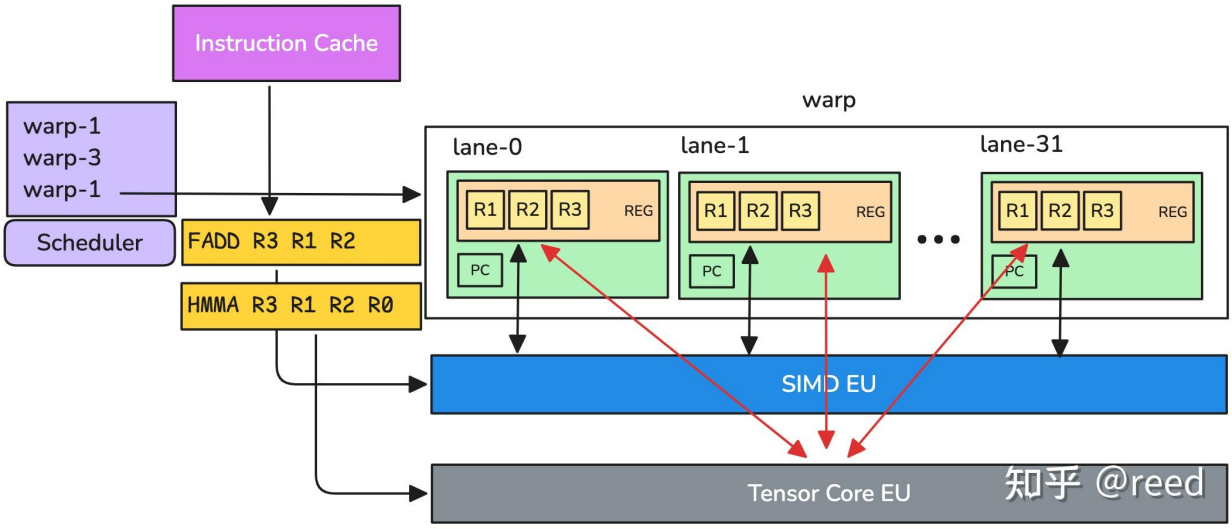


图 27.3: Figure 3. Tensor Core is not a trival SIMT, it shares all the Register resources among lanes

如图三所示，对于 FADD 指令，其为传统的 SIMT 模型，逻辑上可以认为每一个 lane 都有自己独立的寄存器空间，都可以进行独立的计算，并且这个计算在 lane 之间相互独立。对于 HMMA 类指令，其为 Warp Level 的指令，其要求 warp 内的所有 lane 都参与，每个 lane 提供计算所需要的一部分数据，写出的结果会由 lane 所有的输入决定，而不是 lane 内私有的数据部分决定。

27.2 Warp Level 指令

常见的 Warp Level 的指令有 Tensor Core 所提供的核心的矩阵计算

```
DMMA, HMMA, IMMA
```

27.2.1 DMMA 指令

其中 DMMA 表示 Double Matrix Multiply Accumulate，其可以完成双精度（double）的矩阵乘累加运算，图三展示了 DMMA 指令计算的逻辑空间，其中 lane-10 所提供的寄存器，Tensor Core 可以完成如图中`D = AB + C`的矩阵计算，其要求该矩阵的 A 来自于 lane-0 到 lane-31，每一个 lane 中的寄存器在 A 中的排布如图所示，形成一个 8x4 的矩阵，其中 lane-10 提供的寄存器在 A 矩阵中为第 2 行，第 2 列（坐标从 0 开始），同样的 lane-10 提供了 B 矩阵的寄存器，在 B 中表示为第 2 行，第 2 列，经过 Tensor Core 计算，lane-10 得到来计算结果为 D 矩阵的第 2 行，第 4、5 列。根据矩阵计算的定义我们知道 D 中 T10 的两个元素需要的数据为当前行和列，并不是完全来自于 lane-10 自己，而别的 lane 也参与来该计算。

在 NVidia Ampere 架构下具体的 DMMA 指令示例如下：

```
DMMA.884 R64 R96 R90 R64
```

其中 884 表示矩阵指令实现的的矩阵乘法的 mnk 分别为 8，8，4，即 $D_{8\times8} = A_{8\times4}B_{4\times8} + C_{8\times8}$ 。该指令中输出操作数 R64 表示输出矩阵 D，同时约定 R64, R65, R66, R67 四个连续的寄存器表示两个输出的 double 数据，R96，R97

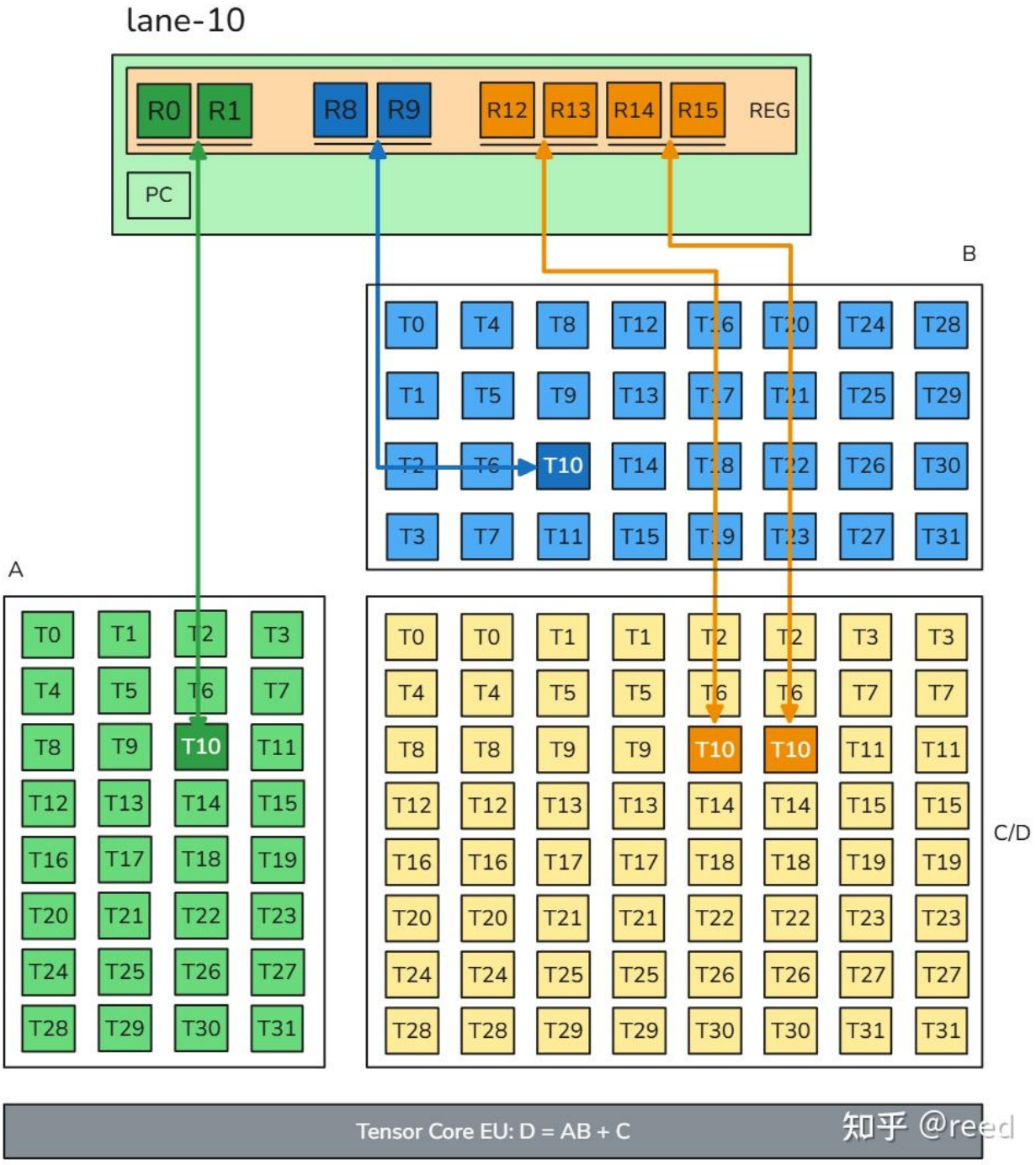


图 27.4: Figure 4. Tensor Core DMMA instruction and its register distribution

两个连续的寄存器共同存储 A 矩阵中的一个 double 数据，R90, R91 表示 B 矩阵中的 double 数据。另外常见的带有 Modifier 的指令还有如下，分别表示对 A 矩阵取负数，对 B 矩阵取负，对 AB 都取负；reuse 表示寄存器复用减少寄存器 io 压力。

```
DMMA.884 R64 -R96 R90 R64
DMMA.884 R64 R96 -R90 R64
DMMA.884 R64 -R96 -R90 R64
DMMA.884 R64 R96.reuse R90 R64
DMMA.884 R64 R96 R90.reuse R64
```

NVidia GPU 针对上面的寄存器排布提供了特定的LDSM指令，对该指令的探讨可以参考问题：ldmatrix 指令的优势 (https://www.zhihu.com/question/600927104/answer/3029266372)，具体用例可以参考：cute 之 Swizzle(https://zhuanlan.zhihu.com/p/671419093)。

27.2.2 HMMA 指令

Half Matrix Multiply Accumulate 半精度类型的矩阵乘累加指令，具体的指令有

```
HMMA.16816.F16
HMMA.16816.F32
HMMA.16816.F32.BF16
HMMA.1688.F16
HMMA.1688.F32
HMMA.1688.F32.BF16
HMMA.1688.F32.TF32
```

其中 16816 和 1688 Modifier 表示 Warp Level 的指令所支持的矩阵的 mnk 规格，具体地，16816 表示 m 为 16，n 为 8，k 为 16，1688 表示 m 为 16，n 为 8，k 为 8。类型 Modifier 表示具体的类型，其中 F16 表示所有的矩阵（ABCD）都是 Half 类型，F32 表示求和器（Accumulator）为 float32 类型，即 C、D 为 float32 类型，A、B 矩阵为 Half 类型；F32.BF16 表示 C、D 为 float32 类型，A、B 为 bfloat16 类型；F32.TF32 表示 C、D 为 float32 类型，A、B 为 tfloat32 类型。具体的类型可以参考 NVidia GPU 指令集架构介绍的浮点数章节 (https://zhuanlan.zhihu.com/p/695667044)。具体的寄存器排布可以参考 PTX 手册中的相关章节 (https://docs.nvidia.com/cuda/parallel-thread-execution/#warp-level-matrix-multiply-accumulate-instructions)。

27.2.3 IMMA 指令

Integer Matrix Multiply Accumulate 整数类型的矩阵乘累加指令，常见的指令有

```
IMMA.16816.S8.S8
IMMA.16832.S8.S8.SAT
IMMA.8816.S8.S8.SAT
```

和 DMMA、HMMA 类似，16816Modifier 表示矩阵乘累加时的 mnk 的规格，整数类型的矩阵乘法指令 Accumulator 为 int32 类型。Modifier 中的 S8.S8 表示 A、B 矩阵为 Signed Int8，即有符号的 8bit 数据。SAT modifier 表示饱和处理，即当 accumulator 加法溢出 int32 的表示范围时会使用 int32 的最大值（向上溢出时）和最小值表示（向下溢出时）。A、B 除了有符号 8bit 数据类型，还可以是无符号的 8bit 数据、有符号的 4bit 数据、无符号的 4bit 数据以及 1bit 的数据，对应的 mnk 的规格也会有调整，具体的支持规格和数据排布要求可以参考 PTX 手册或 cute 中的 MMA 抽象 (https://github.com/NVIDIA/cutlass/blob/main/include/cute/arch/mma_sm80.hpp)。

27.2.4 SHFL 等

除了矩阵乘累加类指令，Warp Level 的指令还有

```
SHFL
VOTE/VOTEU
REDUX
WARPSYNC
```

SHFL 表示 SHuFfLe，可以实现 Warp 内不同 lane 间的寄存器数据的交换，这些指令对于实现排序、规约等功能具有重要作用，具体的指令有

```
SHFL.BFLY SHFL.DOWN SHFL.IDX SHFL.UP
```

这些 Modifier 可以实现 lane 号异或、向下、指定 index、向上形式的 lane 之间的寄存器数据交换。在 CUDA 中体现为__shfl_sync类函数，具体的语义可以参考 CUDA 编程手册相关章节 (https://docs.nvidia.com/cuda/cuda-c-programming-guide/#warp-shuffle-functions)。

VOTE 类指令提供了 Warp 内的 lane 的 bool 级别规约能力，即每一个线程提供一个 1bit 的 bool 值，在 Warp 的 lane 内规约 Any 或者 All，同时将规约后的结果同步给所有线程。规约 All 表示只有当 warp 内的所有 lane 都成立时规约结果为 True；规约 Any 表示线程中只要有一个结果为 True，则最终结果为 True，该指令多用于统计等类场景中，VOTEU 指令除了 VOTE 的功能外还会合并上 Warp Level 的 Uniform 寄存器的作用 (https://zhuanlan.zhihu.com/p/688616037)。

除了规约 bool 值，NVidia Ampere 架构还提供了对整数数值的规约能力，具体的指令为REDUX，其常用的带 Modifier 为

```
REDUX.MAX.S32
REDUX.MIN.S32
REDUX.OR
```

上述指令可以实现 Warp 内 lane 的有符号 32bit 整数的最大值规约和最小值规约，和异或。即如下语义

```
int32_t ret = Vilane0;
for (ilane = 1; ilane < 32; ++ilane) {
  ret = max(ret, Vilane); // min, xor()
}
```

该指令在 CUDA 端可以通过__reduce_add_sync类函数触发，也可以通过 PTX 中的redux.sync指令触发，更多的类型和详细操作可以参考 CUDA 编程手册中的 Warp Reduce 函数章节 (https://docs.nvidia.com/cuda/cuda-c-programming-guide/#warp-reduce-functions)。

从 Volta 开始 lane 可以分裂执行，其可以解决竞争情况下锁步造成的死锁问题，但是如果都以独立的形式运行，效率会受很大影响，所以 NVidia GPU 的指令集架构也提供了WARPSYNC来实现 warp 内线程的同步，即该指令可以等待前面分裂执行的 lane，使 lane 的执行对齐。提高后续指令的效率。

27.3　Uniform 指令

在 NVidia GPU 介绍寄存器 (https://zhuanlan.zhihu.com/p/688616037) 时，我们介绍了 Uniform 寄存器，该寄存器是 Warp Level 的，即所有的 lane 共享这些状态，不需要每一个 lane 独立存储。如图五，Uniform 寄存器（如图示 UR1，UR2 等）被分配到不同的 warp 中，这些单元不同于 lane 级别的通用寄存器，Uniform 寄存器表示 warp 的状态而非 lane 的状态，是 warp 内所有 lane 的公共状态，对于指令集而言，也提供了相应的操作指令来完成这些寄存器的操作，由于是 warp level 的，所以其表达的是控制和执行流而非核心计算，NVidia 针对 Uniform 指令集只提供了整数和逻辑相关指令，并没有提供浮点等类型的指令，并且这部分指令用户无法通过 CUDA/PTX 编程来触发和控制。在 AMD 体系下该类型的寄存器称为 Scalar 寄存器，也明确表明了该类型的计算单元为 Scalar 计算单元，但在 NVidia 体系下对该部分没有暴露编程能力，而是保留给编译器。当代码执行流分析时，如果路径上有 warp 一致的公共计算和控制逻辑，编译器会自动决断该部分是否使用 Uniform 寄存器和 Uniform 指令控制。

使用 Uniform 寄存器和 Uniform 指令可以一定程度上减少通用寄存器的使用，继而提高 kernel 的 Occupancy 并发度，最终提高 kernel 的执行效率。经过编译器生成的常用的 Uniform 指令如下

```
UFLO UIADD3 UIMAD UISETP ULDC ULEA ULOP3 UMOV UPLOP3 UPOPC UPRMT USEL USGXT USHF
```

这些指令名称表现为整数指令、比特和逻辑指令之前添加 U 前缀，具体的含义可以参考该系列文章之前介绍整数指令 (https://zhuanlan.zhihu.com/p/700921948)、比特和逻辑运算的部分 (https://zhuanlan.zhihu.com/p/712356884)。

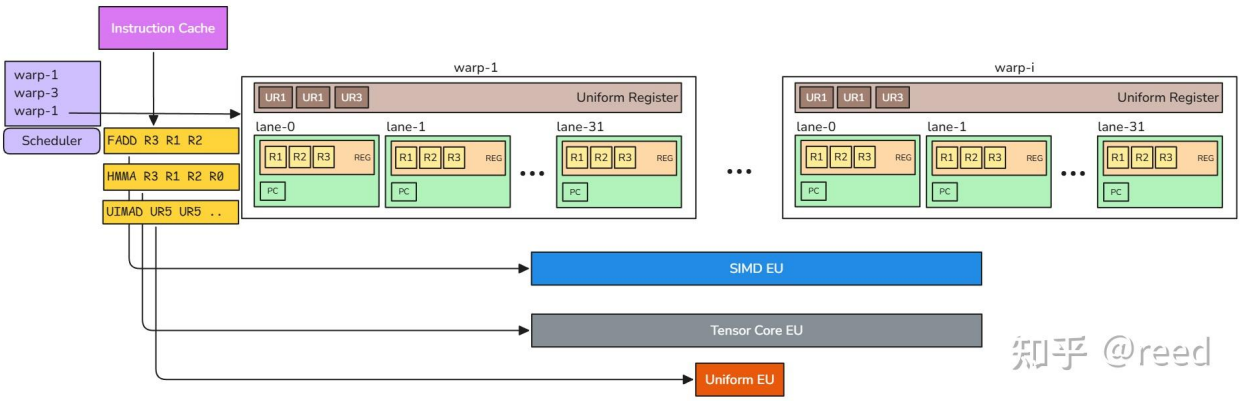


图 27.5: Figure 5. Uniform Register and Uniform Instruction

27.4 总结

本文介绍了 SIMT 编程体系，介绍了资源划分和执行的关系，在此基础上介绍了 NVidia GPU 的 warp level 的指令，它们需要 warp 内的所有 lane 协作来表示数据，共同完成相关指令的计算。详细介绍了 DMMA 等矩阵乘累加指令对 lane 的数据表示和协作逻辑。最后文章介绍了 warp 的状态表示 uniform 寄存器和相关指令。通过本文可以帮助读者更好的理解 SIMT 体系下 Tensor 类协同指令是如何工作的。

27.5 参考

reed：NVidia GPU 指令集架构-浮点运算 (https://zhuanlan.zhihu.com/p/695667044)

reed：NVidia GPU 指令集架构-寄存器 (https://zhuanlan.zhihu.com/p/688616037)

reed：NVidia GPU 指令集架构-整数运算 (https://zhuanlan.zhihu.com/p/700921948)

https://docs.nvidia.com/cuda/parallel-thread-execution/#warp-level-matrix-multiply-accumulate-instructions

https://github.com/NVIDIA/cutlass/blob/main/include/cute/arch/mma_sm80.hpp

https://docs.nvidia.com/cuda/cuda-c-programming-guide/#warp-shuffle-functions

https://docs.nvidia.com/cuda/cuda-c-programming-guide/#warp-reduce-functions

tensorcore 中 ldmatrix 指令的优势是什么？(https://www.zhihu.com/question/600927104/answer/3029266372)

reed：cute 之 Swizzle(https://zhuanlan.zhihu.com/p/671419093)

第二十八章　NVidia GPU 指令集架构-程序控制和原子操作

作者：reed（知乎 @reed-84-49，专栏「CUDA 高性能编程」）　原文：https://zhuanlan.zhihu.com/p/712357443

在前面的文章中，我们详细介绍了 NVIDIA GPU 中的浮点计算 (https://zhuanlan.zhihu.com/p/695667044)、整数计算 (https://zhuanlan.zhihu.com/p/700921948)、位操作 (https://zhuanlan.zhihu.com/p/712356884) 以及 Warp 级协同计算 (https://zhuanlan.zhihu.com/p/712357647)，这些构成了 GPU 的核心计算能力，为图形渲染和 AI 计算提供了强大的支持。此外，我们还探讨了 GPU 的寄存器 (https://zhuanlan.zhihu.com/p/688616037)、加载/存储单元和缓存机制 (https://zhuanlan.zhihu.com/p/692445145)，它们负责数据的存储和传输，确保计算单元能够高效地获取和处理数据。

然而，计算和数据搬运只是 GPU 功能的一部分，程序控制逻辑同样是不可或缺的。程序控制逻辑负责协调计算流程，确保程序按照预期的逻辑执行。尽管 GPU 是一个高度并行的系统，但在许多情况下，我们仍然需要局部或全局的串行逻辑来完成程序控制和数据同步。为此 NVIDIA GPU 提供了一系列原子操作指令，以支持这些需求。

本文将重点介绍 NVIDIA GPU 的程序控制逻辑和原子操作指令。首先，我们将从编程语言层和底层指令的对应逻辑开始介绍，逐步深入到指令层的控制流，探讨 Predicate、SEL、BRA 等指令在程序控制中的作用，然后我们介绍了原子指令 ATOM、ATOMG、ATOMS 以及更高效的 RED 指令，最后文章对全文进行了总结。

28.1　高级编程语言和底层指令

在高级编程语言（如 CUDA C）中，控制流结构，如 if、for、while、switch、goto、function call、return 被广泛使用（如图 1），这些结构可以为高级编程语言提供丰富的控制和表达能力，底层硬件指令在实现时会结合 GPU 多线程、高并行的硬件特征提供更底层和原子的控制能力。具体地，NVidia GPU 提供了 Predicate 能力，Select、Branch、Call、Return、Exit 等相关指令来完成程序控制（如图 1 所示）。

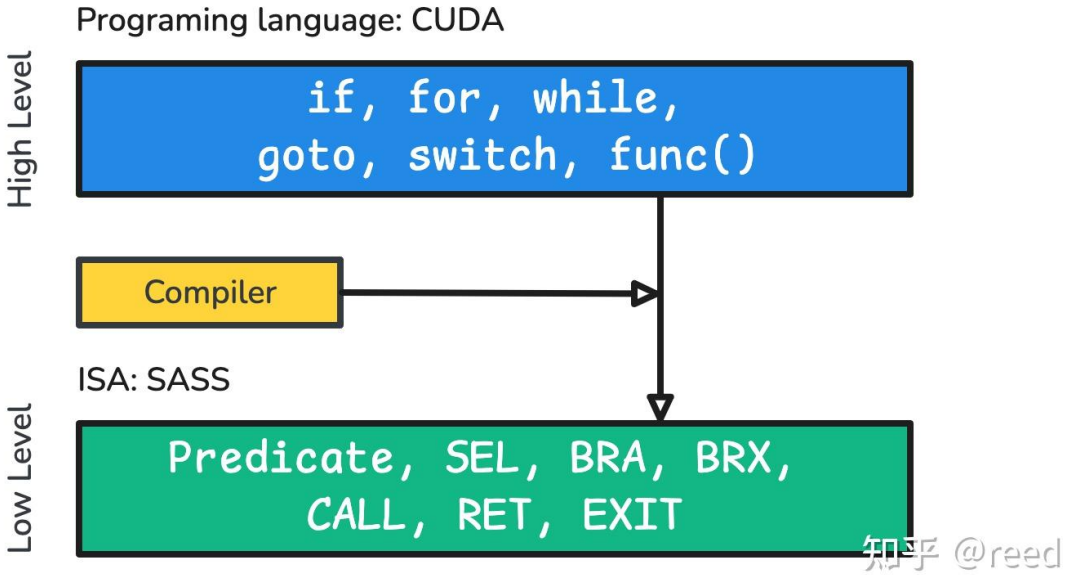


图 28.1: Figure 1. High level CUDA and Low Level SASS

利用或组合这些指令可以表达出高层语言的控制流结构。值得注意的是，高层语言和底层指令并不是一一对应，同一个高层语言的逻辑可以对应底层指令的多种组合形式，如有些 if 逻辑可以通过 Predicate 实现，可以通过 SEL 实现，也可以通过 BRA 实现，但是针对不同的场景效率会有很大的差别，编译器会结合具体的上下文给出更好的指令选择。

28.2 指令层的控制流

Predicate 实现控制流

如下 CUDA 代码展示了 if 控制流，在 lower 到指令集时，一般会使用 Predicate 来完成控制，

```
  if (c > 0) {
    v = __sinf(f);
  } else {
    v = __cosf(f);
  }
```

上述 CUDA 代码对应的指令表示

```
@!P0 MUFU.COS R0, R8 ;
@P0 MUFU.SIN R0, R8 ;
```

从上面可以看到高层编程语言的 if 控制流在底层指令中是通过 Predicate 来实现。

选择指令（SEL）

CUDA 编程语言中的 if 语句，如果满足某种约定且足够简单，则编译器可以从后端的指令集中选择 SELect 指令，如下 CUDA 语句

```
  if (c > 0) {
    v = 1;
  } else {
    v = 2;
  }
```

则编译器生成指令的一种可能的行为如下：

```
MOV R7, 0x1 ;
...
SEL R7, R7, 0x2, P0 ;
```

其中 P0 为 Predicate 寄存器，其表达了`c > 0`这个条件，如果条件成立，则选择 R7, 否则选择立即数 0x2 (即整数 2)。SEL 指令在 SIMT 意义下可以避免跳转的开销，提升执行效率。

分支指令（BRA，BRX，BRXU）

branch 分支指令可以实现指令的跳转，如 CUDA 代码中的 for 循环，

```
  float v = 0.f;
#pragma unroll 1
  for (int i = 0; i < n; ++i) {
    v += ff;
  }
```

则可以通过分支跳转指令实现，一种可能的指令序列如下

```
/*0070*/                   IADD3 R3, R3, 0x1, RZ ;
/*0080*/                   FADD R0, R0, c[0x0][0x184] ;
/*0090*/                   ISETP.GE.AND P0, PT, R3, c[0x0][0x180], PT ;
/*00a0*/             @!P0 BRA 0x70 ;
```

其中 IADD 语句表示整数加一，对应于 CUDA 中的 i++; FADD 语句表示浮点加法，对应于 CUDA 中的`v += ff`;ISETP 语句表示根据整数比较结果来设置 Predicate 寄存器，其中比较的条件为 GE(Greater than and Equal) ，对应于 CUDA for 语句中的 `i < n` 的取反条件表示。`@!P0 BRA 0x70 ;` 表示 P0 条件不成立时则跳转到 0x70 位置执行。通过 BRA 跳转语句实现了 for 的循环能力。

除了以上的给定地址的分支跳转，NVidia GPU 指令集中还提供了动态的跳转指令 BRX，其指令实例为`BRX R6 -0x7960`形式，它根据寄存器的值进行更动态的分支跳转能力，在 switch 语句实现中有可能出现该指令。同时还提供了根据 Warp Level 的 Uniform 寄存器值进行动态跳转的指令 BRXU，命令形式如 `BRXU UR34 -0xb200`。

函数调用和返回（CALL，RET）

在 CUDA 中可以通过`__device__`定义 device 函数，这些 device 函数可以在 global 函数中进行调用，对于比较小的

函数，编译器一般会对此类函数进行 inline 优化，使其成为主函数的一部分，减少函数调用的指令的使用，但是有时候依然会使用到函数调用指令，在 NVidia GPU 中，常见的函数调用指令如下，Modifier 有 REL/ABS 和 NOINC：

```
CALL.REL 0x60f0;
CALL.REL.NOINC 0xfd00;
CALL.ABS.NOINC R2;
```

其中 Modifier REL 表示相对调用，ABS 表示绝对调用，NOINC 表示 PC 值不改变。函数返回常见的指令为

```
RET.ABS R18 0x20;
RET.REL.NODEC R80 0x0;
```

由于 NVidia 并没有公开其函数调用的 ABI 约定，并且 GPU 是一个拥有大量寄存器的设备，其很难在调用时把大量的寄存器都 save 下来，并且大部分函数都在一个编译单元内部，其有很大的优化空间，并且这部分内容更多的是编译器约定的范畴，此处就不做过多解读。

线程退出（EXIT）

由于 GPU 是一个多线程设备，大部分情况下不同的线程会执行同样的指令，有些时候其中的某些线程并不需要工作，其可以提前退出，NVidia GPU 也提供了相应的指令

```
EXIT ;
```

执行了该指令的线程并不会因为同 warp 的线程没有退出而执行其它指令，但值得注意的是，虽然 warp 内的某些线程执行了 EXIT 指令但是如果 warp 中仍有 active 线程，那么这些线程在执行 SYNC、BAR 等指令时行为应当是正确的。和其他指令类似，该指令也是可以被 Predicate 的，如

```
@P2 EXIT
```

执行逻辑分裂与合并

由于 GPU 是多线程模型，在某些场景下需要一部分线程执行一个基本块，另一部分线程执行另外的基本块，这样可以确保逻辑正确，但是长期分裂执行会导致效率低下，NVidia GPU 提供了相应的指令来显式的设置同步点，以达到高效的目的，具体的有如下指令实例：

```
BSSY B0, 0x150
BSYNC B0 ;
@!P0 BREAK B2 ;
```

其中 BSSY 表示设置同步点指令，BSYNC 为同步等待指令，BREAK 为打破同步点指令，这些指令对于处理复杂的嵌套条件有重要作用，同时 NVidia 并没有对用户开放这部分能力，这部分能力由编译器裁决，更详细的信息可以参考 NVidia 的专利 (https://patents.google.com/patent/US11847508B2)。

除此之外，NVidia GPU 中常见的控制相关的指令还有 NOP(No Operation) 和 LEPC(Load Effective PC) 等指令：

```
NOP;
LEPC R96;
```

28.3　原子操作

前面我们介绍的计算类指令逻辑上要么是单个线程独立工作，要么是 Warp Level 协同工作，它们具有很好的局部性和独立性。在一些应用中，除了各个线程独立的计算之外，还需要在 Block 上进行规约或者计数操作，并且这些操作要求在逻辑上是原子执行的，为此 NVidia GPU 指令集架构中引入了 Atomic 类指令，其主要包含

```
ATOM, ATOMS, ATOMG, RED
```

其中 ATOMS 和 ATOMG 分别是在 Shared Memory 空间和 Global Memory 空间上进行原子操作，ATOM 指令没有具体的地址空间限定为 generic 的原子操作，如果编译器可以显式的推断出地址空间则使用对应的指令，如果不能推断出地址空间则可以使用通用的指令，这时硬件在运行时裁决是哪一个地址空间。NVidia GPU 通过 Modifier 来实现不同的运算的，如加，最小，异或、与、CAS 等（具体支持的操作可以参考图二 AtomicALU 部分），也通过 Modifier 来实现不同的作用 Scope。常用的指令形式有

```
ATOMS.ADD  ATOMS.ARRIVE.64 ATOMS.MIN.S32 ATOMS.POPC.INC.32
ATOM.E.ADD.STRONG.GPU  ATOM.E.ADD.STRONG.SYS  ATOM.E.CAS.STRONG.GPU  ATOM.E.EXCH.STRONG.SYS
ATOMG.E.ADD.64.STRONG.GPU ATOMG.E.ADD.STRONG.SYS    ATOMG.E.CAS.STRONG.GPU
ATOMG.E.ADD.STRONG.GPU    ATOMG.E.CAS.64.STRONG.GPU ATOMG.E.EXCH.STRONG.SYS
```

具体的 ATOM 类指令的形式如下，其语义为将 R9 的值原子的加在全局地址空间 [R4.64] 上，并返回加之前的值，获取值和加是原子的不能被其它操作打断。

```
ATOMG.E.ADD.STRONG.GPU PT R5 [R4.64] R9
```

有时候我们只需要原子操作并不需要返回值，NVidia GPU 提供了更轻量的原子指令 RED，如图 2，

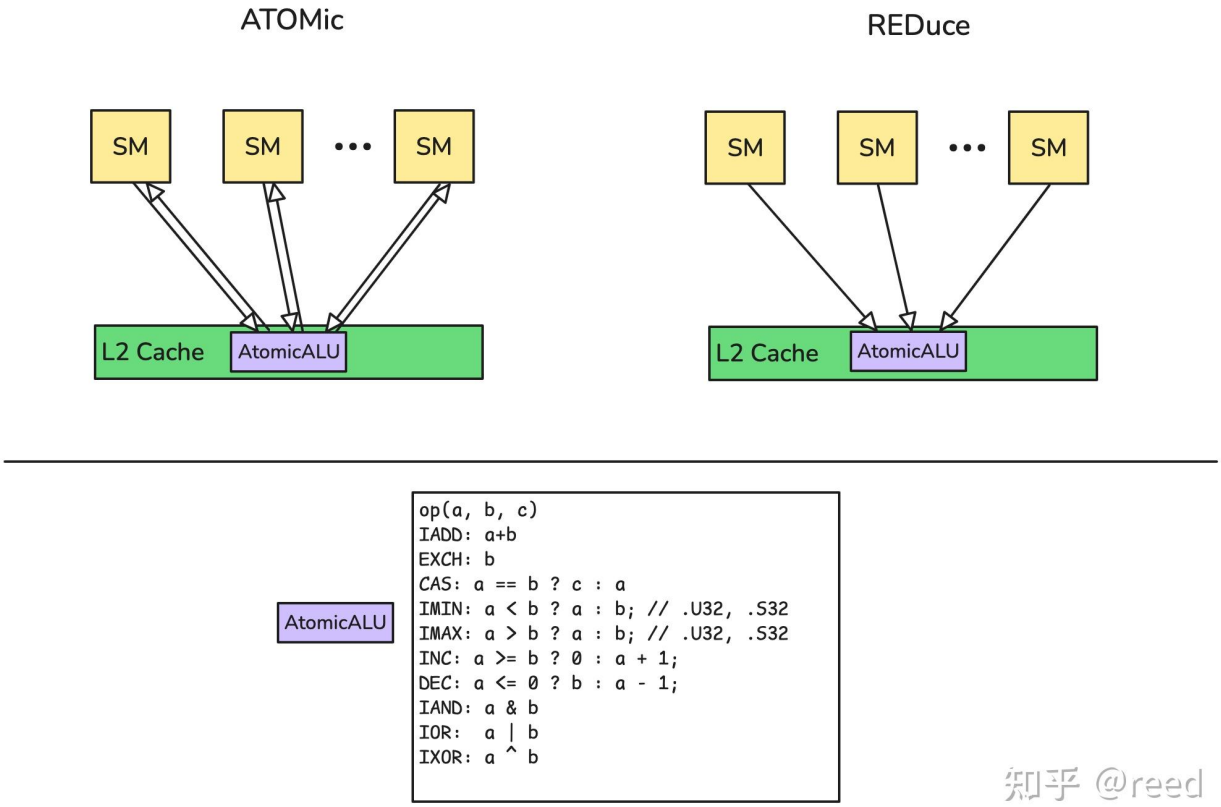


图 28.2: Figure 2. ATOM and RED Operation Difference and Supported Operation

ATOM 指令需要获取读取的值，RED 不需要。针对最终的内存而言，RED 和 ATOM 的副作用一致。由于 RED 不必返回原子操作之前的值，效率也自然的会更高，具体的差异可以参看 NVidia 的专利 (https://patents.google.com/patent/US7627723B1)。

```
RED.E.ADD.64.STRONG.GPU         RED.E.ADD.STRONG.GPU           RED.E.OR.STRONG.GPU
RED.E.ADD.F32.FTZ.RN.STRONG.GPU RED.E.MAX.S32.STRONG.GPU
RED.E.ADD.F64.RN.STRONG.GPU     RED.E.MIN.S32.STRONG.GPU
```

具体的指令形式为

```
RED.E.MAX.S32.STRONG.GPU [R4.64] R7
```

从 RED 的指令形式我们可以看到其相较于 ATOM 指令，只需要输入数据的寄存器和操作的地址数据，并不会返回数据，从而也减少了返回数据的开销，提升了指令的效率。

28.4 总结

本文介绍了高级编程语言中的控制流和底层的程序控制的指令，指出了它们之间可能的多映射关系，针对不同的场景有不同的优化和映射方式，同时针对底层程序控制指令进行了详细介绍：Predicate 实现控制流、SEL 选择指令，BRA

分支指令、函数调用 CALL 和返回 RET 指令，指出 EXIT 退出指令并不是简单的退出而需要考虑其他的 BARRIER 的一致性行为和并行执行引入的分裂和合并逻辑。同时文章介绍了原子指令，和无需返回值的 RED 类指令，了解这些指令能够更好的了解高并行硬件和相关编译器设计。

28.5　参考

reed：NVidia GPU 指令集架构-浮点运算 (https://zhuanlan.zhihu.com/p/695667044)

reed：NVidia GPU 指令集架构-整数运算 (https://zhuanlan.zhihu.com/p/700921948)

reed：NVidia GPU 指令集架构-比特和逻辑操作 (https://zhuanlan.zhihu.com/p/712356884)

reed：NVidia GPU 指令集架构-Warp 级和 Uniform 操作 (https://zhuanlan.zhihu.com/p/712357647)

reed：NVidia GPU 指令集架构-寄存器 (https://zhuanlan.zhihu.com/p/688616037)

reed：NVidia GPU 指令集架构-Load 和 Cache(https://zhuanlan.zhihu.com/p/692445145)

Do switch statements require gmem reads for the jump table?(https://forums.developer.nvidia.com/t/do-switch-statements-require-gmem-reads-for-the-jump-table/179914/4)

https://forums.developer.nvidia.com/t/the-calling-process-of-device-function/287822

https://docs.nvidia.com/cuda/cuda-binary-utilities/index.html

https://patents.google.com/patent/US11847508B2

https://patents.google.com/patent/US7627723B1

第四部分

GPU 架构 · 内存 · Attention 精度

第二十九章　GPU 内存模型之 Proxy

作者：reed（知乎 @reed-84-49，专栏「CUDA 高性能编程」）　原文：
https://zhuanlan.zhihu.com/p/2080760925481592736

C++ 内存模型用 happens-before 表达跨线程的数据交接：release 与 acquire 配对后，release 之前的普通写入可以由 acquire 之后的读取安全观察。编译器与硬件负责把这一语言级关系映射到目标平台；访问方法本身不会作为独立维度暴露给程序。

GPU 为提高数据搬运效率引入了多套硬件通路。除通用 LD/ST 单元外，TMA 按描述符独立搬运 global 与 SMEM 之间的数据，Tensor Core 也有直接访问 SMEM 的操作数通路。这些通路各有队列与执行单元，PTX 不假定它们天然共享同一个访存顺序，而是把访问方法的差异抽象为 proxy。

由此，release/acquire 不能单独跨越不同 proxy 建立顺序；跨 proxy 数据边必须先由 proxy fence 或指令规定的隐式机制建立 causality。proxy fence 的方向由 from → to 固定，与 `.release` / `.acquire` 无关。若两个操作还分属不同线程，再由 release/acquire 与 scope 建立线程间同步。线程关系与访问方法因此是正交的——同一线程先后发出的两个操作，只要分属不同通路，程序序也不能替代 cross-proxy ordering。

mbarrier 进一步把“事务完成”与“内存可见性”拆成两件相互独立的事：计数决定完成，同步语义决定可见性。异步流水线的同步因此可以分解为依次确认的四步：Q1 用计数机制确认异步事务完成，Q2 确定消费方所经的 proxy 以及它必须看到的内容，Q3 按正向交接的普通内存端点所跨的线程范围选择 scope，Q4 判断缓冲区的反向复用是否跨 proxy。其中 Q2 同时覆盖可见性与 cross-proxy ordering，tcgen05 等异步指令族在这一步附加各自的 execution ordering；Q4 在同 proxy 时保留相应的完成与 execution ordering，跨 proxy 时再补反向 fence。

行文上，先从 C++ 内存模型出发，引入 proxy 的硬件基础与 morally strong 三条件，说明线程与访问方法两个维度为什么正交；再梳理 fence、barrier 与 mbarrier 的分工；随后逐一走过四种典型数据边，最后给出同步判定的四步流程与总结。

29.1　C++ 内存模型

29.2　契约、实现与数据竞争

内存模型规定一个线程的写在什么条件下能够被另一个线程观察。它描述可见性与顺序，不规定操作发生的物理时刻；“程序顺序 = 执行顺序 = 可见顺序”在现代系统中并不成立。内存模型是契约而非机制，编译器、流水线与缓存一致性协议共同实现这份契约。

三层的职责划分如图 1 所示：编译器负责把语言级的顺序要求翻译成指令与屏障，并放弃契约所禁止的优化；流水线与 store buffer 负责跨地址的访存顺序以及 fence 的语义；缓存一致性协议则按每个地址维持单写多读与数据值不变式。

图中三根箭头指向的三张卡片，正是契约的三个实现者，卡片下方的红字标出各自管不到的部分——每一层都只兑现契约的一部分，任何一层都无法独力提供跨线程可见性。一致性协议维护单一地址的副本状态，但不规定不同地址写入的相对可见顺序，这一层顺序需要由 fence 建立；流水线可以约束本地发射顺序，却不能替代缓存一致性协议的传播职责。语言模型与硬件内存模型之间还隔着编译器翻译：目标指令序列提供的保证不得弱于语言契约，硬件模型越弱，编译器需要插入的真实同步指令越多。

对本文讨论的普通对象读写，C++ 数据竞争的判据是：两个潜在并发的操作访问同一内存位置，且至少一个为写，称为冲突；若至少一个操作不是原子操作，二者又没有 happens-before 关系，则构成数据竞争，程序行为未定义。

潜在并发 + 冲突 + 至少一方非原子 + 无 happens-before = 数据竞争。

29.3　happens-before

忽略本文不讨论的 `memory_order_consume` 后，happens-before 可由两部分合成并取传递闭包：

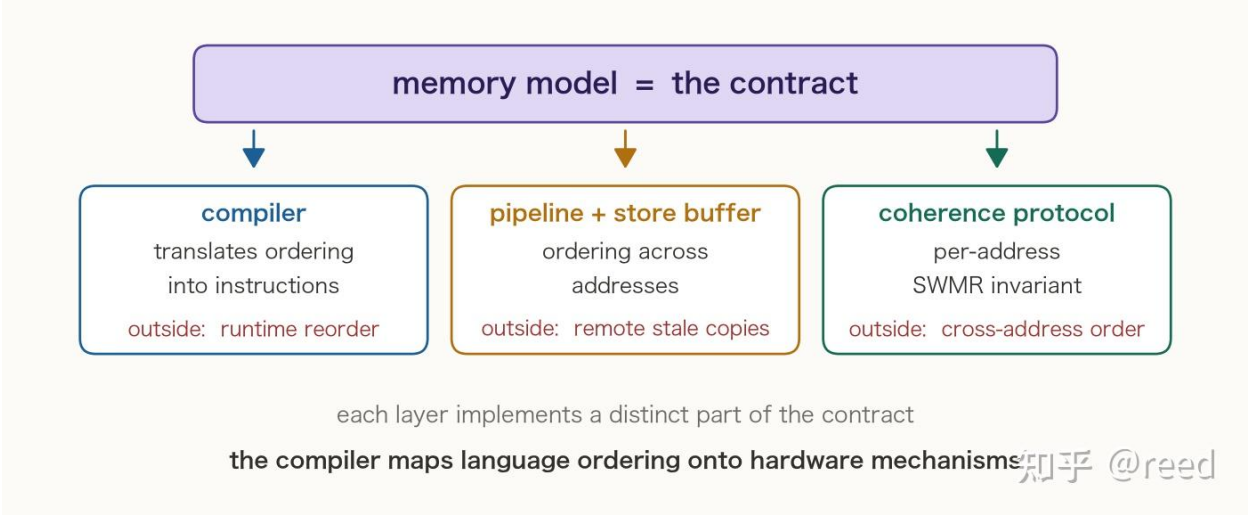


图 29.1: Figure 1. The contract and its three implementations

```
happens-before = (sequenced-before or synchronizes-with) 的传递闭包
```

其中 sequenced-before 是单线程内的程序顺序，它指的是语义上的先后，而非指令发射的先后；synchronizes-with 则是跨线程的那条边，由 release 与 acquire 配对、mutex、thread join 等操作建立。

二者的分工由此清楚：单线程内部由 sequenced-before 确定抽象机中的先后关系；一旦跨越线程，程序顺序不再延伸过去，普通共享数据的交接必须由 synchronizes-with 建立 happens-before。

happens-before 约束后续操作可以观察哪些副作用；对普通共享数据而言，它是跨线程无竞争交接的前提，原子对象的取值还需满足各自的 modification order。由于该关系可传递，一条 release/acquire 边可以覆盖 release 之前的多次写入，无需为每个数据对象单独建立同步。

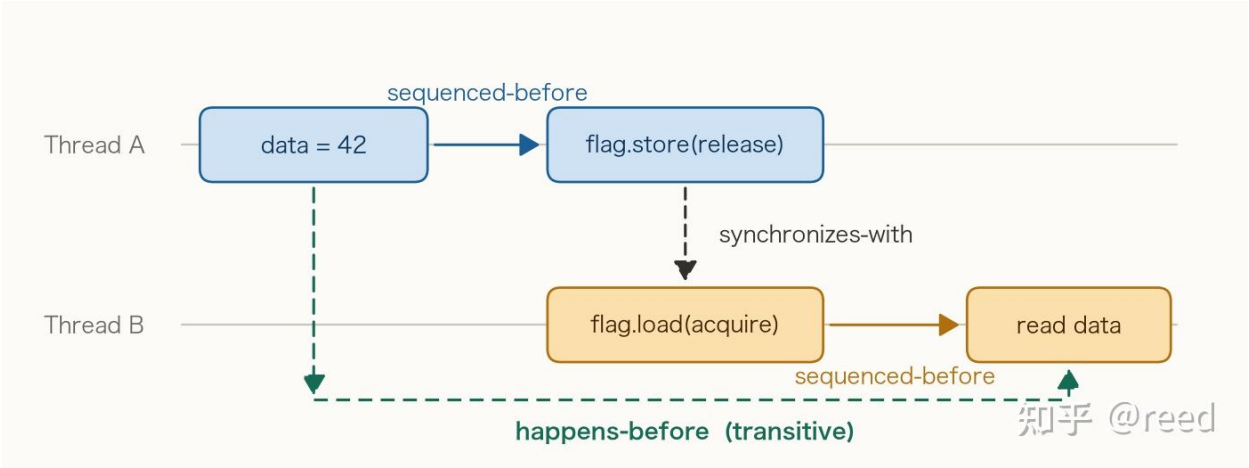


图 29.2: Figure 2. Composing happens-before from sequenced-before and synchronizes-with.

图中两条实线是各自线程内的 sequenced-before，中间那条虚线是由 release/acquire 建立的 synchronizes-with；三段首尾相接，才把线程 A 的 data = 42 排到线程 B 的读取之前。缺了中间那条跨线程边，两侧的程序序无法拼成一条 happens-before。

29.4 acquire / release / relaxed

C++ 有六种 `memory_order`，常用四种，核心区别在约束方向：

relaxed 只保证原子对象自身的原子性与 modification order，不对周围访问增加同步约束；release 与 acquire 典型地分别用于 store 和 load，前者约束其前序读写，后者约束其后续读写，原子 RMW 与 fence 也可以携带相应语义；所有 `seq_cst` 操作在此之上构成单一总序，通常不需要使用这一更强约束。

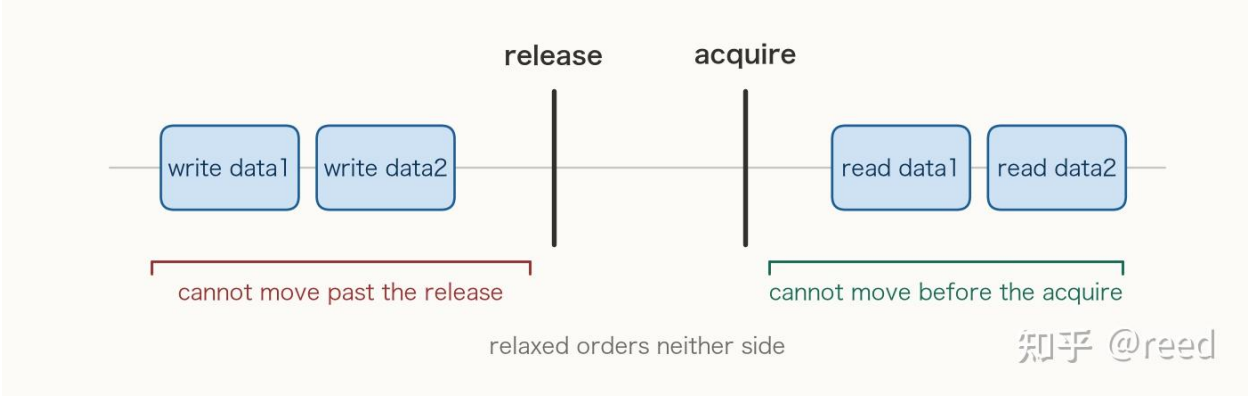


图 29.3: Figure 3. A release orders what precedes it; an acquire orders what follows it.

```
// 线程 A（生产者）
data = 42;
flag.store(true, std::memory_order_release);

// 线程 B（消费者）
while (!flag.load(std::memory_order_acquire)) {}
assert(data == 42);                              // 一定成立
```

把 release 换成 relaxed 后，data 的非原子读写之间没有 happens-before，程序产生数据竞争并进入未定义行为；问题并非仅仅是某次 assert 失败。

方向性为：release 约束其前序访问，acquire 约束其后续访问。

29.5　fence

fence 约束同一线程中位于其前后的访存顺序，不要求其它线程到达某个执行点。单独的 fence 不构成跨线程同步边；跨线程传递数据仍需一次可被对端观察的通信操作，例如作用于同一原子对象的写入与读取。fence 负责约束通信操作两侧的顺序，通信操作负责把可见性关系连接到另一线程。

在实现层面，编译器需要禁止跨越 fence 的相关重排，硬件需要等待前序访存满足目标内存模型要求。具体指令和开销由目标体系结构决定；这层抽象足以对应 PTX 中 thread fence 与 release/acquire pattern 的作用。到了 GPU，还要处理一个 C++ 模型中不存在的维度：访问方法本身也成为内存模型必须显式定序的对象。

29.6　Proxy 的硬件基础

29.7　Coherence 与 consistency

coherence 与 consistency 描述不同层次的问题（Nagarajan、Sorin、Hill、Wood，A Primer on Memory Consistency and Cache Coherence）。coherence 按单一内存位置定义，维护该地址副本的单写多读与数据值不变式；consistency 面向多个地址，规定不同线程可以观察到的跨地址顺序。coherence 约束单一地址的写入传播，consistency 约束多个地址之间的可见顺序。

对本文而言，关键变化是 PTX 不假定不同访存方法共享同一个一致视图。DMA 类单元与线程 LD/ST 可以访问同一地址，却经由不同队列和端口；跨方法顺序必须由内存模型显式建立，proxy 正是对此的抽象。

29.8　Proxy 与访存方法

C++ 内存模型不把访问方法作为独立维度暴露给程序。GPU 则在单个 SM 上提供多套物理独立的访存通路：通用 LD/ST 单元（线程执行 `ld/st/atom`）、TMA 引擎（执行 `cp.async.bulk*` 的 DMA 式搬运单元）、Tensor Core 操作数通路（`tcgen05.mma` 直接从 SMEM 取 A/B）、TMEM 通路（`tcgen05.ld/st`）。

这些通路分别优化不同的数据路径：TMA 将地址计算与边界处理交给硬件并允许搬运与计算重叠，Tensor Core 直接从 SMEM 获取操作数，TMEM 为累加器提供独立于寄存器堆的存储。各通路拥有独立的队列与缓冲，其中几条还落在同一块物理 SRAM 上、抵达路径彼此独立；PTX 不假定它们共享统一的访存视图，而是将访问方法的差异抽象为 proxy。

PTX 列出的非 generic 访问方法包括 texture / surface、alias、async、tensormap 与 fabric。本文主要涉及 generic 与 async：普通 ld、st、atom、red 和 mbarrier 指令属于 generic proxy，`cp.async.bulk*` 以及 `tcgen05.mma` / `tcgen05.cp` 对 SMEM 的访问属于 async proxy。用不同虚拟地址访问同一位置虽然仍采用 generic 方法，也按跨 proxy（`.alias` proxy）处理。

29.9 四个基本构件

异步搬运的完成经由单元到单元的消息路径回报。线程提交 TMA 时只递交描述符（tensormap + 坐标）就返回，搬运由引擎独立完成，完成后按 `mbarrier::complete_tx::bytes` 对屏障做一次事务计数（tx-count）归减。这条回报路径不属于线程指令流，线程因此无法凭自身的程序序判断搬运何时结束。

scope 定义同步直接覆盖的线程集合。`.cta` 覆盖与当前线程同属一个 CTA 的线程，`.cluster`、`.gpu` 与 `.sys` 再依次扩大到同一 cluster、整个 device 与系统范围。同一 SM 可以同时驻留多个 CTA，因此 `.cta` 不是 SM scope。PTX 只承诺相应线程集合中的可见性与顺序，具体需要刷新、失效或穿越哪些缓存层级由实现决定。这一点对普通写同样适用：一次写可以先在较小的 scope 内可见，再传播到更大范围，程序员需要确定的是同步必须覆盖哪些线程。

mbarrier 是 SMEM 中的 8 字节状态对象。`.layout::v0` 与 `.layout::v1` 都维护 pending arrival count、tx-count 与 primary / conditional phase；arrive-on 和 `complete_tx` 分别递减前两个计数，满足当前 phase 的完成条件后对象自动进入下一 phase。v1 额外维护 payload report，并允许异步操作阻止 conditional phase 推进；本文只使用默认的 v0，其 primary 与 conditional phase 同步前进。mbarrier 位于 SMEM，因此 cluster 内其它 CTA 可以通过 DSMEM（distributed shared memory，cluster 内 CTA 之间可互相寻址的 SMEM）地址执行 arrive；`test_wait` / `try_wait` 仅支持本 CTA 的 mbarrier 对象，因为等待操作还涉及本 SM 上线程的挂起与恢复。

fence 建立内存模型要求的顺序。线程 fence 约束同一线程在指定 scope 内的访存顺序；proxy fence 则在两种访存方法之间建立 cross-proxy causality order。PTX 只定义这种顺序及其可见性效果，具体硬件可能表现为等待队列排空、阻止后续发射或同步内部状态。

与 C++ 抽象机相比，PTX 额外暴露了两个选择：同步直接覆盖哪些线程，以及访问是否跨 proxy。前者由 scope 表达，后者由 proxy fence 处理；二者分别对应线程维度和访存方法维度。下一节说明，这两个维度为什么是正交的。

29.10 Proxy 与线程的正交

PTX 用 causality order 描述跨操作的因果先后，它在功能上承担类似 C++ happens-before 的角色，但正式定义还包含 PTX 特有的 proxy-preserved base causality 与 observation order，二者并不等同。release/acquire pattern 是跨线程建立 synchronizes-with 的一种形式，morally strong 是这类 synchronizes-with 成立时的端点条件，也是多数 PTX 内存模型公理使用的关系条件；其中 strong 描述操作的内存语义与 scope，不表示执行时延或硬件强度。

PTX 规定，两个操作要构成 morally strong 必须同时满足三个条件：二者要么由同一线程按程序序执行，要么各自都是 strong 操作且 scope 互含对方线程；二者必须经由同一个 proxy；若都是内存操作，则必须完全重叠（PTX ISA，Memory Consistency Model: Morally Strong Operations）。

这三个条件的地位并不相同。第一个条件有两种满足方式，同一线程按程序序执行即可满足；第二个与第三个条件则在所有情形下都必须满足。这一差别决定了 proxy 与线程是两个正交的维度：即便两个操作出自同一个线程、在程序中一前一后，只要它们经由不同的 proxy，morally strong 依然不成立。程序序仍然存在，但不足以建立跨 proxy 的 causality 与内存可见性；这一额外维度在 C++ 模型中没有对应物，因为那里不把访问方法暴露给程序。

synchronizes-with 要求 release pattern 的首操作与 acquire pattern 的末操作 morally strong，因此可以得到：

release/acquire 单独不能为跨 proxy 的两个操作建立 causality order。

C++ 与 PTX 对 data race 的正式定义并不相同：C++ 关注潜在并发的非原子冲突是否缺少 happens-before，PTX 则要求冲突访问既没有 causality order、又不 morally strong。proxy 出现在 PTX 的第二个条件中；只有在 cross-proxy causality 也未建立时，冲突访问才形成 PTX data race。

一个 `mbarrier.try_wait` 上的 `.acquire` 本身不是 proxy fence，不能单独建立 generic 与 async 之间的 ordering。对 TMA 写入而言，completion 已经提供 async→generic 的隐式 proxy ordering（详见下文 “TMA 完成语义”），成功的 acquire wait 再把后续 generic 读取排在完成之后，因此不需要额外执行显式 `fence.proxy.async`。

proxy fence 建立的是 cross-proxy causality，而 release/acquire 建立的是跨线程 synchronizes-with。若操作出自同一线程，只需补足缺失的 cross-proxy ordering；若操作还分属不同线程，则两类关系都要成立。async 指令在请求完成前即把控制权返回给执行线程，generic 与 async 通路因此可以并行推进；cross-proxy ordering 约束的正是这两条通路之间的可见顺序。

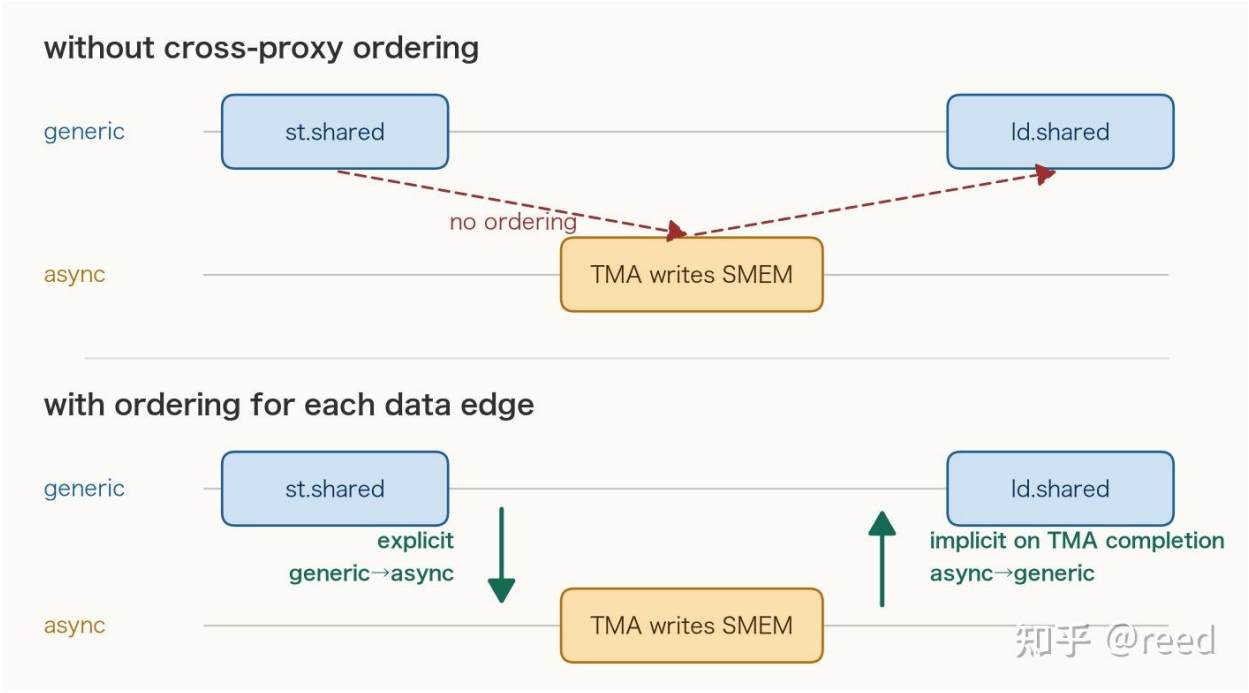


图 29.4: Figure 4. Cross-proxy ordering must be established in the direction of each data edge.

上图是无 fence 的情形：同一块地址被 generic 线与 async 线交替访问，两次访问之间不存在任何排序关系。下图为每条数据边各建立一个方向的 cross-proxy ordering 后，两个方向分别被定序；其中 async→generic 由 TMA 完成隐式提供，generic→async 才需要显式 fence。

29.11　GPU 上的同步原语与可见性

29.12　三类原语的分工

fence、barrier 与 mbarrier 分别解决访存定序、线程会合和事件或事务完成三个问题，职责如图 5 所示。

CUDA barrier 同时提供执行同步与内存可见性保证。`__syncthreads()` 首先等待线程块内所有未退出线程到达同一同步点；barrier 完成后，参与线程在 barrier 之前发起的普通内存访问（ld / st / atom / red，不含 TMA、cp.async、tcgen05 等异步事务）已经完成，并对这些线程从 barrier 返回后的访问可见。异步事务的完成必须由 mbarrier 或相应的 `*.wait_group` 另行确认。因此，barrier 的作用包含两个不可分割的部分：线程在执行进度上会合，以及会合前的普通内存效应对参与线程可见。PTX 的 `bar.sync` 提供相应语义。

mbarrier 与前两者的区别在于把 arrival 与 wait 拆开：以 expected arrival count 定义每个 phase 需要满足的 arrival 数量，并可同时跟踪 tx-count；实际参与线程由控制流决定，同一线程也可以贡献多个 arrival。当参与线程分别 arrive 并等待同一 phase 时，mbarrier 可以实现 split-phase 线程会合。仅等待异步事务完成或资源状态的 mbarrier 用法不自动代表某个线程集合已经会合，而普通 barrier 也不提供 TMA 的事务完成跟踪。

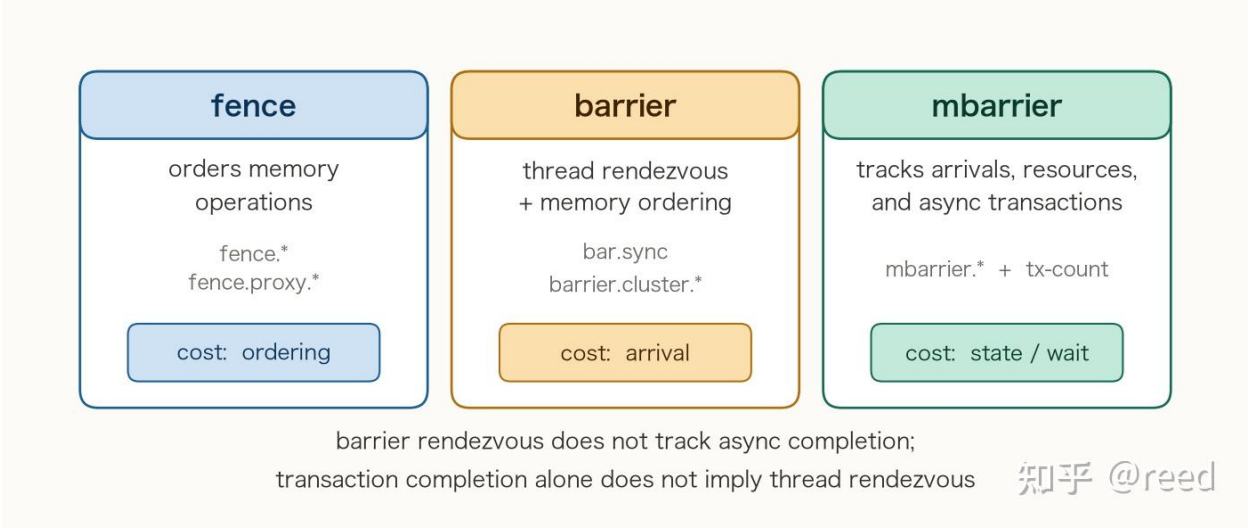


图 29.5: Figure 5. Three primitives for three distinct problems

29.13 Scope 作为同步条件

release 的 scope 必须包含 acquire 线程，acquire 的 scope 也必须包含 release 线程；跨 CTA 的普通数据交接因此要求两侧都使用 `.cluster`，单边升级不能建立所需关系。

scope 与地址空间相互独立。`.shared::cluster` 表示 mbarrier 对象或数据位于可由 cluster 访问的 SMEM 地址空间，`.scope::cluster` 表示同步效果覆盖 cluster 内线程；访问远端地址不自动把同步 scope 升为 cluster，访问本地地址也不代表 CTA scope 必然充分。判断依据始终是 release 与 acquire 的执行端点及需要传递的内存效应。

29.14 Fence 家族

PTX 里 fence 有四类：

```
// 1) 线程 fence：约束本线程访存之间的顺序
fence{.sem}.scope;
fence.acquire.sync_restrict::shared::cluster.cluster; // 只针对 shared::cluster 空间
fence.release.sync_restrict::shared::cta.cluster;

// 2) 操作 fence：只针对某一类操作（.op_restrict = { .mbarrier_init }）
fence.mbarrier_init.release.cluster;

// 3) proxy fence（双向）：为两条 proxy 线互相定序，不带 scope
fence.proxy.proxykind; // 例：fence.proxy.async.shared::cta

// 4) proxy fence（单向）：只约束一个方向，带 scope
fence.proxy.to_proxykind::from_proxykind.release.scope;
fence.proxy.to_proxykind::from_proxykind.acquire.scope [addr], size;
fence.proxy.async::generic.release.sync_restrict::shared::cta.cluster;
fence.proxy.async::generic.acquire.sync_restrict::shared::cluster.cluster;
```

`fence.proxy.to::from` 的方向始终由 from → to 决定。因而 `fence.proxy.async::generic.release` 与 `fence.proxy.async::generic.acquire` 都建立 generic→async 的单向 proxy ordering；`.release` 与 `.acquire` 决定的是这两条 fence 如何组成跨线程的 release/acquire sequence，并不会把 proxy 方向反转。无需区分方向时可以使用双向 `fence.proxy.async`。

`.sync_restrict` 是可选限定符，一旦出现就与 `.sem` 强绑定：`.sync_restrict::shared::cta` 只能配 `.release`，`.sync_restrict::shared::cluster` 只能配 `.acquire`，且此时 `.scope` 必须为 `.cluster`，fence 的作用范围被限制在对应地址空间的对象上；省略 `.sync_restrict` 时 fence 作用于所有状态空间。因此像 acquire + `sync_restrict::shared::cta` 这样的组合并不合法。规范没有解释这条不对称从何而来，但官方示例给出了用法上的分工：release 侧要发布的是本 CTA SMEM 里的对象，acquire 侧要覆盖的是 cluster 内可寻址的对象，其中包含远端 CTA 的 SMEM。

29.15　mbarrier 的 sem / scope 规则

两项语法约束：`.sem` 与 `.scope` 必须成对出现；默认值 arrive = `.release.cta`、wait = `.acquire.cta`。

取值上，`mbarrier.arrive` 的 `.sem` 可取 `.release` 或 `.relaxed`，`mbarrier.test_wait` 与 `try_wait` 的 `.sem` 可取 `.acquire` 或 `.relaxed`，两者的 `.scope` 都可取 `.cta` 或 `.cluster`。

wait 侧不提供 release 语义，因此 PTX 不接受 wait 上的 release 限定符。

地址空间与 scope 的独立性在指令上很直接：`mbarrier.arrive.release.cta.shared::cluster.b64` 访问的是远端 mbarrier，但 release 效果仍限定在本 CTA。

远端 CTA 第一次访问 mbarrier 之前，初始化结果还需要通过 `fence.mbarrier_init.release.cluster` 发布到 cluster；该 fence 保护的是 barrier 状态的初始化，而不是 barrier 所跟踪的数据。

29.16　完成状态与内存可见性

mbarrier 提供两种相互独立的保证，如图 6 所示。

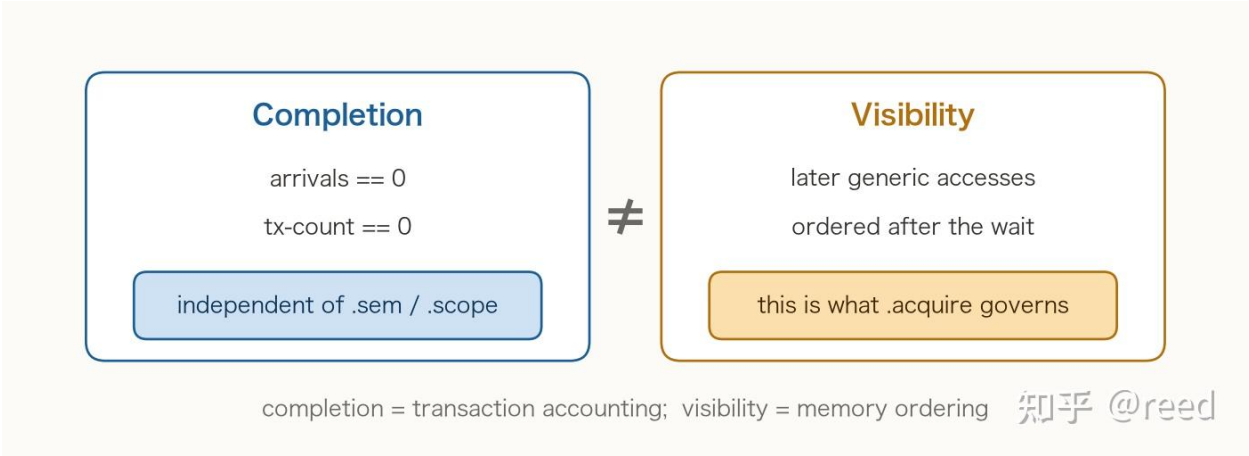


图 29.6: Figure 6. A barrier's two independent guarantees: completion by counting versus visibility by ordering.

完成状态由 mbarrier 计数决定。在本文使用的 `layout::v0` 中，phase 翻转要求 pending arrivals 与 tx-count 两者都归零。所有被跟踪的异步操作字节都完成后，`try_wait` 才返回 true；这一计数条件与 wait 上的 `.sem` / `.scope` 无关。

内存可见性由同步语义决定。`.acquire` 约束 wait 返回后本线程的后续 generic 访问。

因此，仅跟踪异步字节是否完成时不需要额外内存语义；后续 generic 读取还需要被约束在 wait 成功之后时，才需要 acquire。两类保证分开后，modifier 可以按数据边逐项确定。这也是后文 Q1（完成）与 Q2（可见性）相互独立的原因。

29.17　TMA 完成语义

PTX 规定，`cp{.reduce}.async.bulk` 完成后伴随一条隐式的 generic-async proxy fence：一旦完成被观测，其 async 写入结果即对 generic proxy 可见，因此该 async→generic 数据边无需再执行双向 `fence.proxy.async`。后续 generic 读仍需由 wait 上的 `.acquire` 或独立的 `fence.acquire` 约束在完成之后（PTX ISA，cp.async.bulk 与 Memory Consistency Model）。

这条 fence 是规范挂在 completion 机制上的，而不是由 release/acquire 推导出来的，因此它并不与 morally strong 的同 proxy 要求冲突：跨 proxy 的 causality 依然需要有东西来建立，只不过这一次由指令规范预先给定。哪些异步操作的完成带这条隐式 fence，要按各自的规范条目确认；不能因为存在这类例外，就指望 release/acquire 去覆盖其它跨 proxy 数据边。

这条隐式保证只有 async→generic 方向。若同一缓冲区随后由新的 TMA 覆写，本轮 generic 读 → 下一轮 async 写仍需单独建立 generic→async ordering。其它异步写也必须按各自规范判断，不能直接套用 TMA 完成时的隐式保证。

PTX 给出的 canonical acquire pattern 把 fence 放在成功分支，避免在重试路径上重复执行：

```
cp.async.bulk.mbarrier::complete_tx::bytes.relaxed.sys.b128 [dst], [M], size, [barrier];
mbarrier.try_wait.relaxed p, [barrier];
@p fence.acquire;
```

官方示例里对本地 SMEM 的 acquire 也采用同一写法：relaxed wait 加一条独立的 acquire fence。

```
mbarrier.try_wait.relaxed.cluster.shared::cta.b64 complete, [addr], phase;
fence.acquire.sync_restrict::shared::cluster.cluster;
```

29.18 典型数据边

以下选取四种典型场景：同 proxy 消费、generic 读取、跨 CTA 的 mbarrier 交接和异步响应缓冲区的跨 proxy 复用。每条数据边都按同一套四步判定：完成（Q1）、消费方的 proxy 与可见性（Q2，含 tcgen05 等指令族的 execution ordering）、普通内存同步的 scope（Q3）与反向复用（Q4）；这些关系不能互相替代，四步的完整口径见后文“同步判定”。

29.19 TMA 写入与 tcgen05.mma 消费

TMA 写入与后续 async-proxy 消费位于同一 proxy，因此这条数据边不需要 proxy fence；mbarrier 的 tx-count 负责完成跟踪（Q1）。wait 上的 acquire 约束的是 wait 之后的 generic 访问，而 `tcgen05.mma` 属 async proxy，其 ordering 不来自 acquire（Q2）。wait 成功后要发起异步 tcgen05 操作时，PTX canonical sequence 用 `tcgen05.fence::after_thread_sync` 将其排在此前的 mbarrier execution-ordering operation 之后（PTX ISA，tcgen05 Memory Consistency Model: Canonical Synchronization Patterns）。

这里出现的是两种不同的定序：memory ordering 决定一次写在什么时候对读方可见，execution ordering 决定两条指令在各自的发射通路上谁先谁后。对这条数据边，TMA completion、tx-count 与同 proxy 规则共同保证 async 写入已经完成，relaxed wait 只负责观测完成状态；若后续改由 generic proxy 读取，还需要 acquire 约束后续访问。tcgen05 另需 execution ordering，因此这条专用 fence 不能被同一个 mbarrier wait 取代。mbarrier 的 phase 翻转是 SMEM 中的一次计数事件，`try_wait` 返回只保证“计数已达成”；而 tcgen05 的操作数与执行走的是 Tensor Core 独立的发射通路，它并不自动排在这次计数事件之后。`tcgen05.fence::after_thread_sync` 正是把随后的 tcgen05 操作显式排到先前那次 execution-ordering operation 之后，补上这条 wait 覆盖不到的顺序。这一位置是按指令族填的：mbarrier 负责完成跟踪，并可通过 acquire 传递普通内存效应；凡是自带独立发射通路的异步指令族，还要在这里补上属于自己的 execution ordering，具体指令由各自的规范给出。

以 Blackwell 2SM（`cta_group::2`）主循环为例，两个 CTA 各发一条 TMA 搬运各自的半份 tile 到本地 SMEM，MMA 由 CTA pair 共同执行。以下 PTX 仅用于表达 issuer、完成信号与 execution ordering 的关系：省略 `tcgen05.mma` 的部分强制操作数，示例假设每个 CTA 只有一个参与线程，不作为可直接汇编的完整程序。CTA 内有多个线程时，`barrier.cluster.arrive` 必须由所有未退出线程各执行一次，否则 barrier 无法完成。

```
// p_cta_leader: 每个 CTA 恰好一个 elected thread
// p_pair_issuer: CTA pair 中恰好一个 tcgen05 issuer; 本例选择 CTA0 的 elected thread
// %r_mbar: 本 CTA 的 mbarrier 的 shared 地址

// CTA0 elected thread 登记两条 TMA 的总事务字节数
@p_pair_issuer mbarrier.arrive.expect_tx.relaxed.cta.shared::cta.b64
               _, [%r_mbar], tx_bytes_total;
barrier.cluster.arrive.relaxed;
barrier.cluster.wait;

// 清除 peer bit: cta_group::2 的完成信号按 CTA-Pair 归并，统一记到 CTA0
and.b32 %r_mbar_pair, %r_mbar, 0xFEFFFFFF;

// 每个 CTA 仅由 elected thread 发起一条 TMA 写入本地 SMEM
@p_cta_leader cp.async.bulk.tensor.2d.shared::cta.global
 .mbarrier::complete_tx::bytes.cta_group::2.L2::cache_hint
```

```
 [smem], [tmap, {c0,c1}], [%r_mbar_pair], hint;

// CTA0 的 pair issuer 等待并发起一次 tcgen05.mma
@!p_pair_issuer bra mmaDone;
waitLoop:
mbarrier.try_wait.parity.relaxed.cluster.shared::cta.b64 P, [%r_mbar], phase;
@!P bra waitLoop;
tcgen05.fence::after_thread_sync;
tcgen05.mma.cta_group::2.kind::f16 [taddr], adesc, bdesc, idesc, enable-input-d;
mmaDone:
```

29.20　完成信号与发起粒度

多 CTA 协同还包含两个与 sem / scope 无关的约束：完成信号的目标 CTA 与 MMA 的发起粒度。

完成信号路由由 mbarrier 地址决定，只有持有该 mbarrier 的 CTA 能够执行 wait。在 Blackwell 上，`cta_group::2` 的完成信号落到 CTA-Pair 中的哪个 CTA，取决于 mbar 所在 SMEM 属于该 pair 的偶数号还是奇数号 CTA——mbarrier 既可以与目标 SMEM 同属一个 CTA，也可以位于其 peer-CTA。因此 CUTLASS 主动把 peer bit 清 0，让两个 CTA 的完成信号统一记到 CTA0。

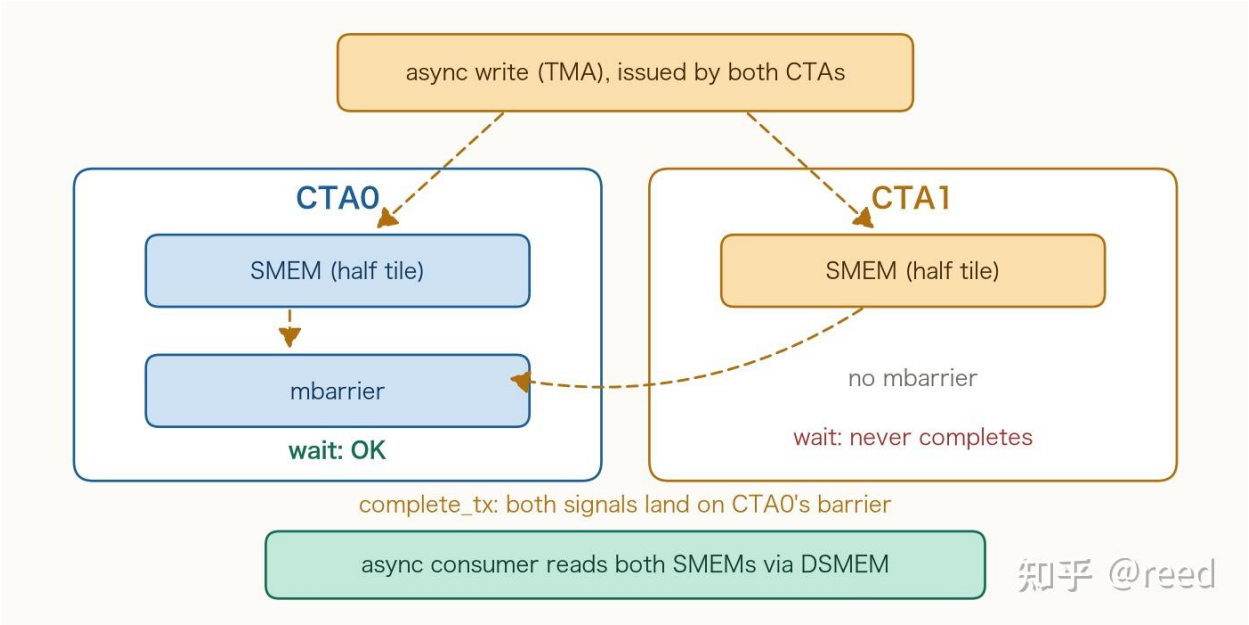


图 29.7: Figure 7. Routing of an async write's completion signal: only the CTA holding the barrier may wait.

图中两个 CTA 各把自己那半份 tile 写入本地 SMEM，但完成信号都被引导到 CTA0 的 mbarrier；CTA1 没有屏障，只有持有屏障的 CTA0 能够 wait，随后异步消费者经 DSMEM 读取两份 SMEM。

指令发起粒度要求 `cta_group::2` 操作由 CTA pair 中的单个线程发起，且 peer CTA 必须处于 active、未退出状态。若一个 warp 的 32 个线程都执行该指令，就不满足“单线程发起”的规范要求，其行为不被 PTX 定义，不应依赖任何具体表现。

29.21　TMA 写入与 generic 读取

数据由线程使用 generic load（`ld.shared` / `ldmatrix` / `ld.global`）消费时，wait 成功后需要 acquire（Q2 落在 generic load 分支）。scope（Q3）取决于建立 release/acquire 关系的两个端点是否跨 CTA；数据地址位于本地还是远端是判断端点位置的一项依据，但不是唯一条件。

```
waitLoop:
mbarrier.try_wait.parity.shared::cta.b64 P, [mbar], phase; // 默认 = .acquire.cta
@!P bra waitLoop;
```

```
ld.shared.v4.b32 {r0,r1,r2,r3}, [smem]; // 或 ldmatrix.sync.aligned.m8n8.x4.shared.b16
```

可以采用以下两种写法：

```
// (i) 单条指令即可（默认 .acquire.cta）
mbarrier.try_wait.parity.shared::cta.b64 P, [mbar], phase;

// (ii) relaxed wait + success-path acquire fence
mbarrier.try_wait.parity.relaxed.cluster.shared::cta.b64 P, [mbar], phase;
@!P bra waitLoop;
fence.acquire.sync_restrict::shared::cluster.cluster;
```

TMA 向本 CTA 的 SMEM 写入、且消费者也在本 CTA 时，`.acquire.cta` 足以覆盖本地读取；若消费者依赖另一 CTA 的普通写入，或通过 DSMEM 读取对端数据，则 release 与 acquire 都必须使用 `.cluster`。TMA 的 complete-tx 已带 `.release.cluster`（这是 `cp.async.bulk.tensor` 规范固定的语义，不由前述 arrive/wait 默认值决定），普通线程的 mbarrier arrive 则需要显式选择与 consumer 匹配的 scope。

作为反向数据边（Q4）的对照，generic 写入随后由异步操作读取时，程序序仍不能替代 proxy ordering。TMA 完成已经提供 async→generic 的隐式 proxy ordering，因此该方向不需要额外的 `fence.proxy.async`；反向的 generic→async 数据边仍需显式建立：线程先用 `st.shared` 把数据写进 SMEM，再让异步操作经 async proxy 读这块 SMEM 时（例如把它作为 TMA store 的源，或作为 tcgen05 读 SMEM 的输入），需要执行 `fence.proxy.async.shared::cta`，即 CUTLASS 的 `fence_view_async_shared()`。

写入与异步读取由同一线程按程序序发起，但二者分别位于 generic 与 async proxy，morally strong 仍不成立。该场景不涉及跨线程同步，只需要建立 generic→async 的 proxy ordering，直接说明了 proxy 与线程维度相互独立。

需要区分的一点是：TMA 使用的 tensormap 描述符本身不属于这条 SMEM 数据边，它需经 `fence.proxy.tensormap::generic` 一类机制发布，不能与上面这条 `fence.proxy.async.shared::cta` 混为一谈。

29.22 跨 CTA 的 mbarrier 交接

mbarrier 对象可以位于远端 CTA 的 SMEM（`.shared::cluster`），但 wait 只能作用于本地 mbarrier 对象——PTX 的远端地址支持矩阵中，`.shared::cluster` 只支持 `arrive / arrive_drop / expect_tx / complete_tx`，其余（含 init / `test_wait` / `try_wait`）一律 Not supported，对远端对象执行 wait 不被 PTX 支持、行为未定义（PTX ISA，§9.7.14.16.11 mbarrier Instructions）。scope 是否升级，取决于是否通过 mbarrier 在跨 CTA 的执行端点之间交接普通内存效应；只传递计数、phase 或资源状态时不需要 cluster 级 release/acquire。

在 2SM empty barrier 中，MMA 消费完成后 producer 复用 SMEM buffer，odd CTA 远程 arrive 到 even CTA 的 barrier（CUTLASS `umma_arrive_2x1SM_sm0`，`cutlass/arch/barrier.h`）：

```
// (i) MMA 的完成跟踪：由 commit 直接登记到 empty barrier，同时带生产侧 ordering
// .cta_group 必选，且须与同一 kernel 内 tcgen05.mma 的 cta_group 一致
tcgen05.commit.cta_group::2.mbarrier::arrive::one.shared::cluster.b64 [%r_mbar_pair];

// (ii) odd CTA 远程 arrive 到 even CTA 的 barrier（CUTLASS 中即 umma_arrive_2x1SM_sm0）
mbarrier.arrive.shared::cluster.b64 _, [%r_mbar_pair];  // %r_mbar_pair 同前，已清除 peer bit
                                                        // 未写 sem / scope，默认 = .release.cta
```

两种写法的职责不同。`tcgen05.commit.mbarrier::arrive::one` 让 mbarrier 跟踪此前异步 `tcgen05.mma` 的完成，并同时提供生产侧所需的 execution ordering；第（ii）种只贡献一次 arrive，必须由已经确认完成的线程发出。若异步操作是 `tcgen05.ld` 这类需要先 wait 再交给对端的场景，PTX canonical 给出的是 `tcgen05.wait` 加 `tcgen05.fence::before_thread_sync`，再接 `mbarrier.arrive`：这条 fence 只补 execution ordering，不承担完成跟踪，不能与 commit 互换。mbarrier 在这里仅传递“SMEM buffer 可以复用”这一资源状态，不交接普通内存数据，因此不需要 cluster 级 release/acquire；现有默认语义可以工作，显式 relaxed 也足以表达这条计数关系。

这条边同时给出了反向复用（Q4）的另一种情形：`tcgen05.mma` 对 SMEM 的读取与下一轮 TMA 的写入都发生在 async proxy 内部，反方向的复用因此不跨 proxy，不需要 proxy fence，但仍须由上述 specialized execution ordering 把消费完成排到复用信号之前。只有当本轮消费改为线程的 generic 读取时，反向边才跨 proxy，也才需要下一节讨论的 release proxy fence。

当生产者需要把本 CTA 的普通内存写交给对端 CTA 的 waiter 时，构成 DSMEM 数据交接，release 与 acquire 必须两侧同时使用 `.cluster`：

```
mbarrier.arrive.release.cluster.shared::cluster.b64 _, [remAddr]; // 生产者
mbarrier.try_wait.parity.acquire.cluster.shared::cta.b64 P, [bar], phase; // 消费者
```

单边升级无效。CuTe DSL 对这一取舍有一段直接的注释（mbar.py:437-442），大意是：远程 arrive 在形式上应当使用 cluster scope 以获得跨 CTA 的内存模型排序；保留 CTA 默认值是因为现有协议只依赖屏障计数，且 cluster scope 在那里实测更慢；做 DSMEM 数据交接时应显式传 CLUSTER。

29.23　异步响应缓冲区复用

缓冲区在本轮由 generic proxy 读取、下一轮由 async proxy 覆写时（Q4），需要 generic→async 的 release proxy fence，把本轮读取排在下一轮异步写之前。这条 fence 不能移到 wait 上，因为 wait 的 acquire 只约束其后的访问。TMA 完成时附带的隐式 fence 只覆盖 async→generic 的相反方向，因此缓冲区复用时同样需要这一 release fence。

Cluster Launch Control（CLC，运行时把待处理的 CTA 动态分派给 cluster 的机制）流水线中的 `clc_info[stage]` 是一块复用的 16 字节响应缓冲区：`try_cancel` 异步写入响应，线程等待 mbarrier 后用 `ld.shared.b128` 读取 handle，再由 release proxy fence 把本轮读取排在下一轮 `try_cancel` 之前。

该 fence 必须保留在 generic 读与下一轮 async 写之间。该缓冲区被两个 proxy 交替访问：`try_cancel` 在 `.multicast::cluster::all` 下以 weak async-proxy writes（未定序的 async proxy 写）把响应写到 cluster 内每个 CTA 各自的本地 SMEM（PTX ISA，`clusterlaunchcontrol.try_cancel`），`ld.shared` 读则属 generic proxy，跨 proxy 时 morally strong 不成立。如果缺少 generic→async 的 release proxy fence，下一轮新的 async 写可能覆盖上一次 `ld.shared` 仍在读取的值，读到撕裂的 handle、解出错误的 ctaid，最终可能表现为错误的 tile 分配。

该 fence 约束的是“本次 generic 读 → 下次迭代 async 写”。wait 上的 `.acquire` 只约束 wait 之后的访问；这里需要 release 语义，将 fence 之前的 generic 读排在 fence 之后的 async 写之前。

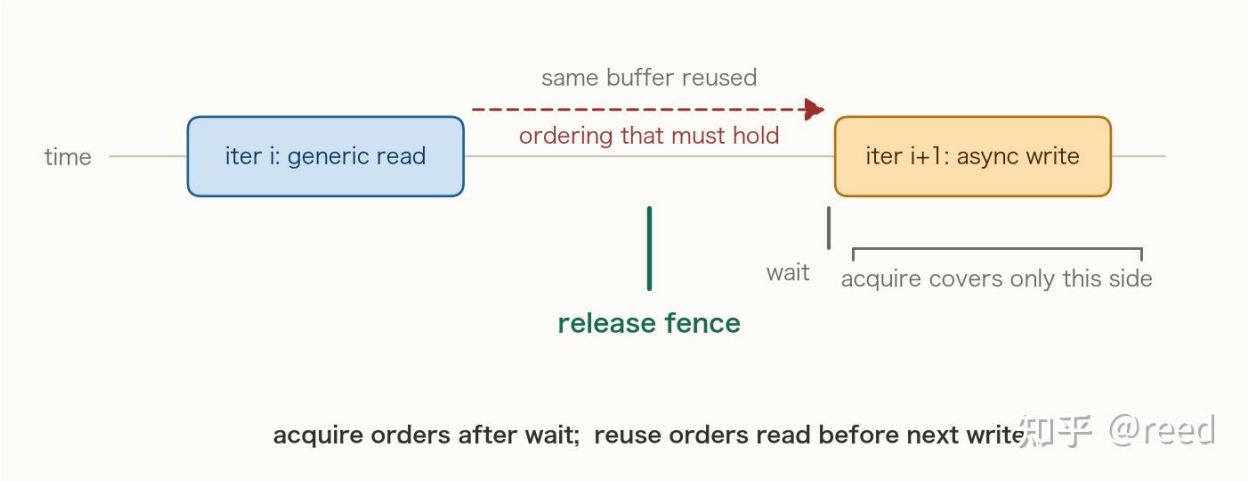


图 29.8: Figure 8. A reused async response buffer: why the release fence cannot be moved onto the wait.

图中红色虚线是必须保持的“本轮读 → 下轮写”顺序，绿色竖线（release fence）压在两者之间；右侧的 wait 及其 acquire 只笼罩它之后的一段，无法覆盖到下一轮的写，这正是 fence 不能挪到 wait 上的原因。

29.24　CLC 官方示例中的同步关系

官方示例的关键几行：

```
waitLoop:
 mbarrier.try_wait.parity.acquire.cluster.shared::cta.b64 complete, [mbar], phase;
 @!complete bra waitLoop;
 ld.shared.b128 handle, [addr];
 // ↓ 保护“本次 generic 读”先于“下次 async 写”
```

```
fence.proxy.async::generic.release.sync_restrict::shared::cta.cluster;
barrier.cluster.arrive.relaxed;
barrier.cluster.wait;
// 与上一轮各 CTA 的 release proxy fence 配对；方向仍为 generic→async
fence.proxy.async::generic.acquire.sync_restrict::shared::cluster.cluster;
```

该示例包含三个独立约束。第一，`mbarrier.try_wait.acquire.cluster` 既等待 CLC 响应完成，也阻止随后的 generic `ld.shared` 越过 wait。第二，本轮读完 handle 后，release proxy fence 建立 generic→async ordering；cluster 会合后的 acquire proxy fence 与各 CTA 的 release sequence 配对，把上一轮各 CTA 的 generic 读排在当前迭代的 async 写之前。两条单向 fence 的 proxy 方向相同，区别在跨线程同步中的 release/acquire 角色。第三，multicast 下 `try_cancel` 由 elected CTA 发起，却写入每个 CTA 各自的本地 SMEM，因此这对单向 fence 使用 `.cluster` scope。

29.25 同步判定

同步选择分成两层。第一层确定需要建立的关系：本线程访存顺序使用 fence，线程集合会合使用 barrier，事件、资源状态或异步事务完成使用 mbarrier；若同一条数据边跨 proxy，还要按实际方向增加 proxy ordering。图 5 给出三类原语的职责边界，图 9 则把一条异步数据边的判定拆成依次确认的四步。

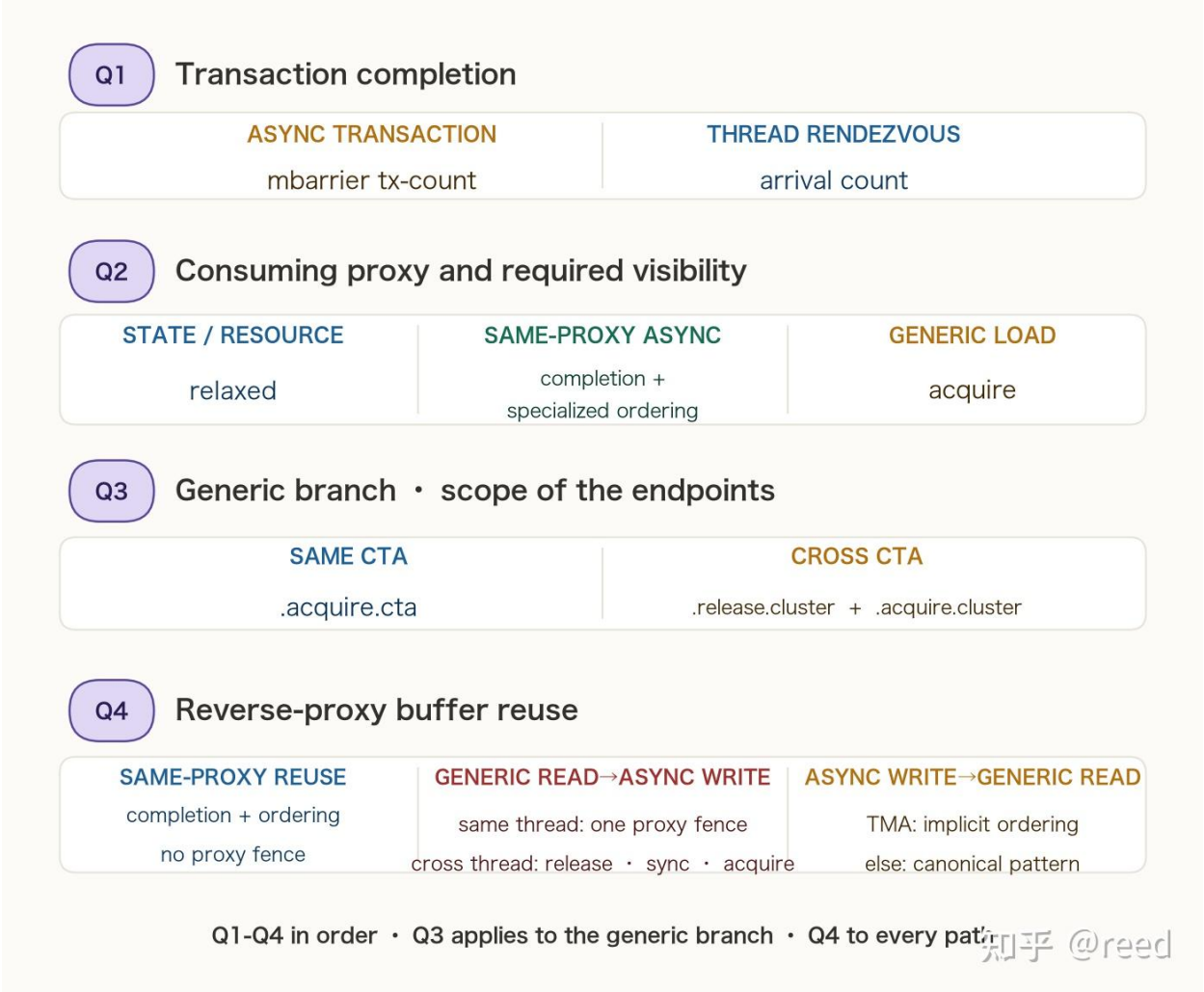


图 29.9: Figure 9. Four checks for ordering an async data edge.

Q1 完成：异步事务或事件的完成由计数机制判定——TMA 用 tx-count，线程会合用 arrival count，`try_wait` 返回 true 即代表当前 phase 的计数条件已满足；这一步与 sem / scope 无关。

Q2 消费方的 proxy 与可见性：这一步确定 wait 成功后由谁、经哪个 proxy 消费，以及消费方必须看到的内容。只传递 phase 或资源状态时 relaxed 足够；同 proxy 的异步消费者（如 `tcgen05.mma`）依赖完成计数，并按指令族补 specialized execution ordering 及其 scope（如 canonical 里的 relaxed wait 加 `.cluster`，再接 `tcgen05.fence::after_thread_sync`）；

generic load 则需要 acquire。

Q3 普通内存同步的 scope：仅当数据边需要通过 release/acquire 传递普通内存效应时单独判断，由两个同步端点是否跨 CTA 决定 `.cta` 还是两侧 `.cluster`。异步指令族自身的 execution-ordering scope 已在 Q2 中处理。

Q4 反向复用：独立作用于所有分支，先看本轮消费所经的 proxy。若消费也走 async proxy（如 `tcgen05.mma` 直接读 SMEM），反向边是 async→async，同 proxy，不需要 proxy fence；但仍须由 `tcgen05.commit`（或 `tcgen05.fence::before_thread_sync` 加显式完成等待）把消费的完成与 execution ordering 一起排到复用信号之前。若本轮是 generic 读、下一轮由 async proxy 覆写，反向边才跨 proxy：同一线程使用一条 generic→async proxy fence；不同线程则由 reader 侧 release proxy fence、线程间同步和 writer 侧 acquire proxy fence 组成，两条单向 fence 的 proxy 方向都为 generic→async。至于 async 写 →generic 读方向，`cp.async.bulk*` 的完成已提供隐式保证，其它异步操作按各自的 canonical completion pattern 处理。

四步之外，还有两点贯穿始终：mbarrier 对象可以位于远端 CTA 的 SMEM，但 `test_wait` / `try_wait` 只能访问本 CTA 的 mbarrier；地址空间、同步 scope 与 proxy 方向三者互不蕴含，需要分别确认。

29.26　常见错误

Q1 的错误集中在完成机制与对象归属。非持有 CTA 对远端 mbarrier 执行 wait 不受 PTX 支持；`bar.sync` 与 `barrier.cluster.wait` 不跟踪 TMA 事务；`barrier.cluster` 只要还有未退出线程没有 arrive 就无法完成；`cta_group::2` 操作也不能由多个线程重复发起。

Q2 / Q3 的错误集中在 scope 与地址空间。跨 CTA 传递普通内存效应时，生产侧缺少 release、只升级 wait 不能建立同步；`.shared::cluster` 是地址空间，不等于 `.scope::cluster`；省略 `fence.mbarrier_init.release.cluster` 会让远端 CTA 观察到未发布的 barrier 状态。语法上，`.sem` 与 `.scope` 必须成对出现，`.acquire` 也不能和 `sync_restrict::shared::cta` 组合。

Q2 / Q4 的错误集中在 proxy、execution ordering 与缓冲区复用。`fence.proxy.to::from` 的方向始终由 from → to 决定，不随 release/acquire 反转；acquire 不能替代 proxy fence；wait 后直接发起 tcgen05、省略 `after_thread_sync` 会缺少 specialized execution ordering；generic 读取后的缓冲区若由 async proxy 复用，也不能省略反向 proxy fence。不同虚拟地址访问同一位置仍可能形成 `.alias` proxy，不能按普通 generic 同址访问处理。

29.27　总结

内存模型规定可见性与顺序，具体保证由编译器、执行流水线和缓存一致性机制共同实现。PTX 在此基础上进一步暴露线程集合与访问方法两个维度：scope 描述同步覆盖的线程，proxy 描述内存访问方法。morally strong 的三条件说明这两个维度正交——跨 proxy 数据边需要单独建立 causality order，即便操作出自同一线程也不例外；若操作还分属不同线程，则同时满足 release/acquire 与 scope 条件。

mbarrier 的 phase 由 arrival count 与 tx-count 推进，完成状态与内存可见性相互独立。只传递资源状态时可以使用 relaxed；交接普通内存数据时，需要按执行端点选择 sem 与 scope。TMA 完成附带的隐式 fence 只覆盖 async→generic 方向，generic 读到下一轮 async 覆写仍需 generic→async ordering。

fence 约束访存顺序；CUDA barrier 同时完成 collective rendezvous 与参与线程间的普通内存定序，但不跟踪 TMA 等异步事务；mbarrier 将 arrival 和 wait 拆开，可用于线程子集的 split-phase 会合，也可跟踪资源状态或异步事务完成，并按 sem / scope 选择是否传递普通内存效应。

落到具体一条数据边，判定就是图 9 的四步：Q1 确认事务完成，Q2 确定消费方的 proxy 与可见性，并按异步指令族补 execution ordering，Q3 按普通内存同步端点的线程范围选择 scope，Q4 判断缓冲区的反向复用是否跨 proxy；同 proxy 时保留指令族所需的完成与 execution ordering，跨 proxy 时再补反向 proxy fence。

29.28　参考

Hans-J. Boehm, Sarita V. Adve, Foundations of the C++ Concurrency Memory Model, PLDI 2008 https://www.cs.toronto.edu/~pekhimenko/courses/csc2231-f17/Papers/C++Consistency.pdf

Vijay Nagarajan et al., A Primer on Memory Consistency and Cache Coherence, 2nd Edition https://directory.doabooks.org/handle/20.500.12854/97841

NVIDIA, Parallel Thread Execution ISA 9.3: Memory Consistency Model; fence; bar; mbarrier; cp.async.bulk; tcgen05; `clusterlaunchcontrol.try_cancel` https://docs.nvidia.com/cuda/parallel-thread-execution/index.html

NVIDIA, CUDA Programming Guide: Synchronization Functions; Asynchronous Barriers; Asynchronous Data Copies https://docs.nvidia.com/cuda/cuda-c-programming-guide/

NVIDIA CUTLASS, main: `include/cute/arch/barrier.h`, `include/cute/arch/copy_sm90_desc.hpp`, SM100 collective implementations https://github.com/NVIDIA/cutlass

LLVM, NVPTX Usage: mbarrier, proxy fence and tcgen05 intrinsics https://llvm.org/docs/NVPTXUsage.html

cute 之 Hopper MBarrier https://zhuanlan.zhihu.com/p/1962636004235153810

第三十章　Blackwell 双 die 拓扑：SM 与地址的 die 映射及跨 die 访存代价

作者：reed（知乎 @reed-84-49，专栏「CUDA 高性能编程」）　原文：
https://zhuanlan.zhihu.com/p/2051639750541546085

单 die 面积有两条硬约束——光刻机一次曝光的 reticle limit（约 800 mm^2）划下面积墙，良率又随面积超线性下降使得大 die 成本陡峭。把一颗大芯片拆成两颗较小的 die 再用高带宽互联缝合是绕开这两条约束的标准做法，代价是引入一道 die 边界：原本片内均匀的访存被分成”本地”与”跨边界”两种。

Blackwell 是 NVIDIA 第一代沿这条路线、把多颗 die 封装为单一逻辑 GPU 的数据中心架构。一颗 Blackwell GPU 内部是两颗 reticle 尺寸的 die，由 NVIDIA 的片间高带宽互联 NV-HBI（NVIDIA High-Bandwidth Interface）连接，对主机呈现为单一设备、单一地址空间、单一逻辑 L2。这套封装内的结构与多路 CPU 服务器上的 NUMA（Non-Uniform Memory Access）同构：一颗 die 连同它本地的 HBM 对应一个 socket，NV-HBI 对应 socket 间的 UPI / Infinity Fabric——从某个 SM 访问本 die 的 HBM，与访问另一颗 die 的 HBM，代价不应相同。

本文围绕三件事展开：确定每个 SM（Streaming Multiprocessor）归属哪颗 die；确定任意一个虚拟地址背后的字节由哪颗 die 持有；并量化当二者错配、访问跨越 die 边界时所付出的代价。我们所用的全部手段都是常规 CUDA 代码，不借助任何特权接口或硬件文档之外的信息。

行文上，我们首先介绍双 die 拓扑及其硬件特点；之后给出测量单次冷 HBM 访问延迟的原语，并先用它把每个 SM 标定到所在 die；继而反过来以标定好的 SM 为量尺，反推出地址到 die 的三层确定性结构，它可以从虚拟地址闭式求解；随后考察跨卡推广性；最后基于 SM 与地址这两套 die 归属构造亲和与反亲和访问，测出二者的只读带宽差并解释其放大机理，再讨论走向真实工程的适用范围与局限，对全文进行总结。

30.1　一、Blackwell 双 die 的拓扑与硬件特点

Blackwell 的双 die 拓扑如图 1 所示。两颗对称的 die 各含 4 个 GPC（Graphics Processing Cluster）、约 74 个 SM、2 个 L2 partition 和 4 个内存控制器，各自连接 4 个 HBM3e（High Bandwidth Memory）堆栈、提供约 4 TB/s 的本地带宽；两颗 die 间由 NV-HBI 互连，聚合带宽不低于 10 TB/s，整颗芯片峰值带宽约 8 TB/s（HGX 机型 7.7TB/s，NVL72 机型 8TB/s）。本文的测量在 B200 上进行，其规格为合计 148 个 SM、8 个 GPC、4 个 L2 partition（是 Hopper 的两倍）、8 个 HBM3e 堆栈。

值得强调的是 L2 的位置：4 个 L2 partition 并非集中一处，而是每颗 die 各占两个，与该 die 的内存控制器同侧——也就是说 L2 是按 die 切分的。当一颗 die 上的访问需要另一颗 die 所缓存或所拥有的数据时，无论是取数本身还是维持两侧 L2 的一致性，相关流量都必须经 NV-HBI。因此 NV-HBI 承载的不只是远端 HBM 的读写，还包括跨 die 的缓存一致性通信，它是这颗封装内 NUMA 唯一的跨域通路。

这套拓扑有一个反直觉的带宽关系：NV-HBI 的聚合带宽（$\geq$ 10 TB/s）高于单颗 die 的本地 HBM 带宽（$\approx$ 4 TB/s），这让带宽在 Blackwell 上不是一个方便的观测量。在 CPU NUMA 上，跨 socket 访问既增延迟又降带宽，把数据绑到远端 socket 跑个带宽测试即可暴露远端；而在 Blackwell 上，NV-HBI 比单 die 的本地带宽还宽，只要访存大致均摊在两颗 die 上，无非是两组内存控制器各出一半力，聚合带宽并不会因为”跨了 die”而明显下降。要想用带宽把 die 边界显示出来，就必须人为控制每次访问落在哪颗 die 上，而 CUDA 并没有提供按 die 申请内存的接口——这道障碍要到下一节才着手解决。相比之下，单次访问的延迟在本地与远端之间存在可测差异，即跨 die 多出的那一跳——因此本文以延迟而非带宽为主信号，这是整套方法论的起点。

顺带一提：NVIDIA 把双 die 的细节封在驱动之内、对程序员仅暴露”一颗逻辑 GPU”的抽象，并未提供查询 SM 到 die 或地址到 die 的接口。本文的全部 die 归属信息都靠用户态微基准反推。

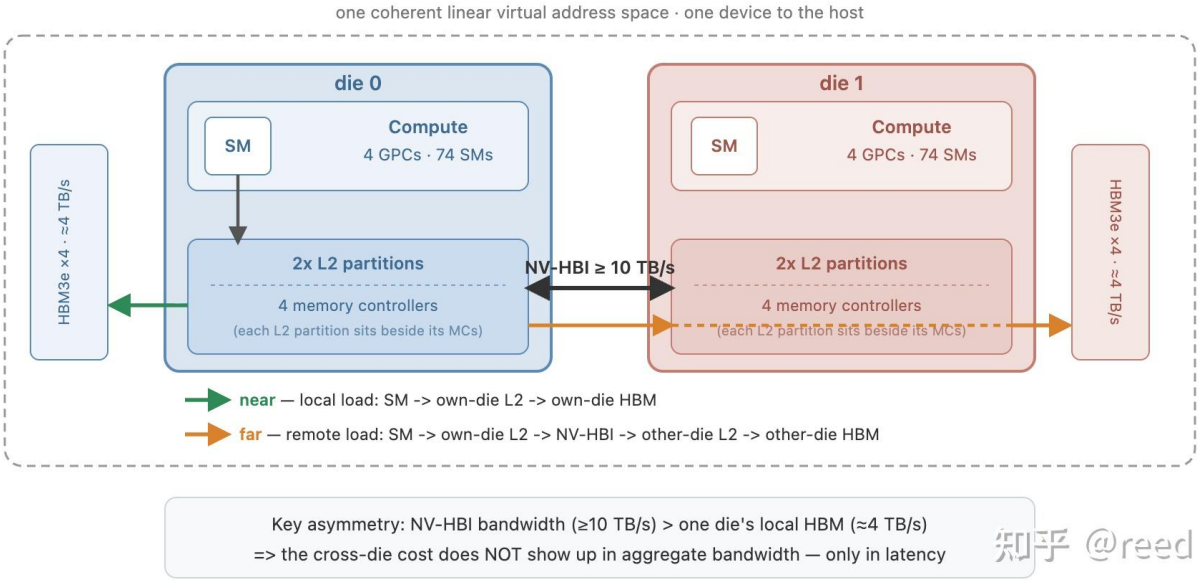


图 30.1: Figure 1. Blackwell single-package 2-die NUMA topology.

30.2 二、测量原语：一次干净的冷 HBM 访问延迟

既然可观测的信号是延迟，整套方法就建立在一个原语之上：测准从某个指定 SM 发出、到某个指定地址的单次冷 HBM 访问延迟。这个原语本身就是后文所有结论的量具，如图 2 所示。要让它真正”测准”，前三件事必须同时解决：

1. 冷访问。测的必须是真正的冷 HBM 访问，而非缓存命中。每次计时前先流式扫过远超 L2 容量的无关数据以驱逐缓存，并令计时核只执行一次访问、不做预热，保证第一跳就是冷访问。

2. 不被预取或访存重叠掩盖。由单个线程串行执行依赖式访问——每一拍的地址依赖上一拍的返回值——所测周期即真实的 load-to-use 延迟。

3. 归属到确定的物理 SM。通过申请超过单 SM 一半容量的动态共享内存，保证一个 SM 上只驻留一个线程块，再令该块读取自身 `%smid` 上报，延迟即归属到该 SM。

第四件事是单位稳定性：GPU 频率随 DVFS（Dynamic Voltage and Frequency Scaling）波动，因此主度量始终采用对频率不变的 cycles/hop，并在每段测量前后各测一次实际频率以剔除存疑样本。

计时核心可浓缩为：

```
int v = 0;
int64_t t0 = clock64();
do {
  v = ldg_cv(input);
} while (v < 0);          // 强制 load 完成（retire）后才出循环
int64_t t1 = clock64();
output[0] = v;            // sink 写，阻止编译器消除计时
```

这里的 do-while + sink 写是关键：没有它们，ptxas 会把第二次 clock64 排到 LDG 发射之后立即执行，而 LDG 是异步 retire 的，计时只会捕获到 LDG 发射（issue）的时刻，而非数据真正返回（retire）的时刻。加上二者，t1 - t0 才干净地报告周期级的冷 HBM 延迟。

30.3 三、SM 到 die：干净的 74/74 划分

反推的出发点是一个足够小的地址：小到一次 16 B 的访问，其背后的字节必然完整落在某一颗 die 上，不会横跨 die 边界。固定这样一个地址，用全部 148 个 SM 各自访问它一次，每个 SM 测得的冷延迟就只取决于它与这个地址是否同 die——同 die 的 SM 走本地通路，跨 die 的 SM 要多走一趟 NV-HBI。

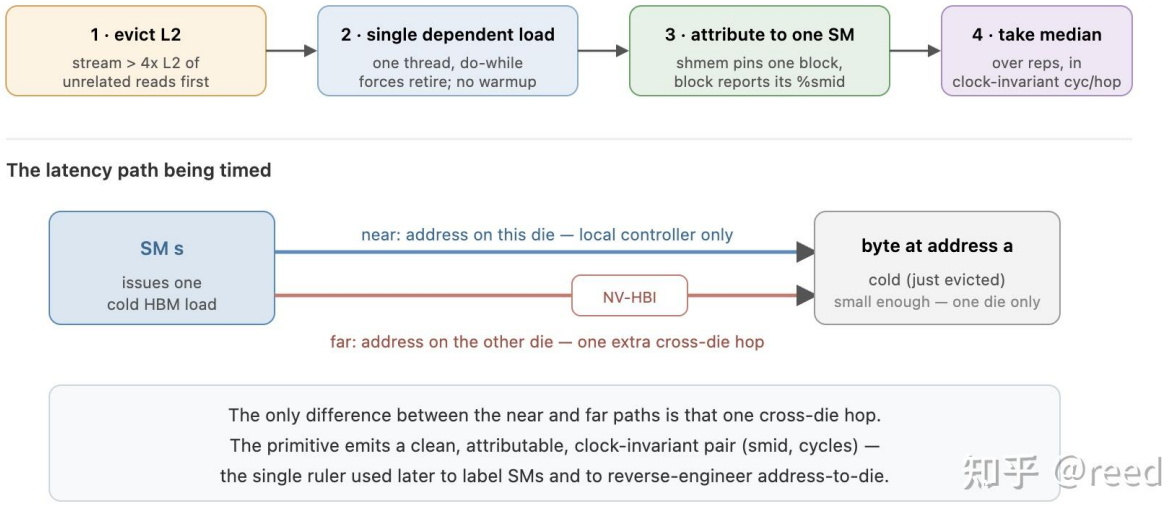


图 30.2: Figure 2. Measuring one cold-HBM access latency (SM s to address a).

把 148 个延迟画成直方图，结果如图 3：两条不重叠的 band，跨 die 比同 die 多约 +400 cyc（约 +220 ns），每条各含 74 个 SM。快的一条是” SM 与该地址同 die”，慢的一条是” SM 需跨 NV-HBI 才能取到”。零重叠意味着对这个特定地址而言，die 归属是确定的、可二值判定的。

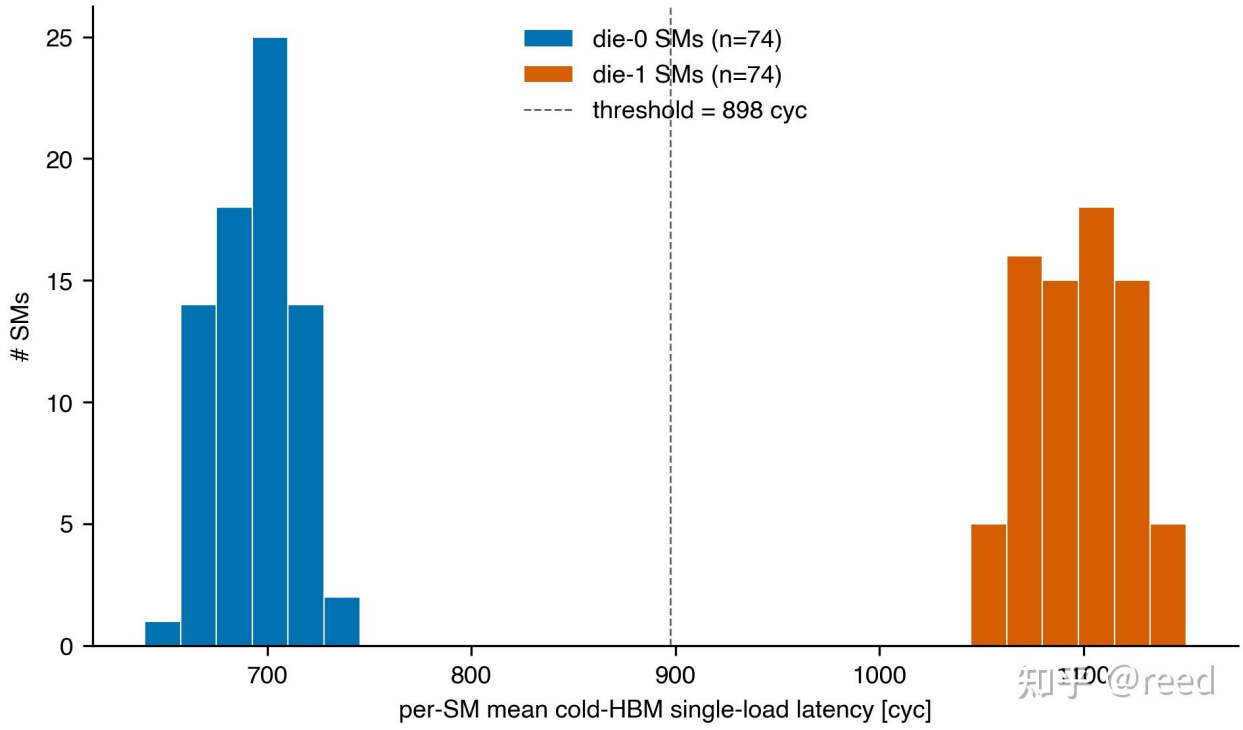


图 30.3: Figure 3. SM-to-die latency histogram —two non-overlapping bands, 74 / 74.

单个地址给出一次两组划分，还不足以断定这个二分就是 SM 的固有属性——它有可能只是该地址的偶然。为此换一批彼此独立、落在不同 die 上的地址，各自重做同一探测：每个地址都给出同样清晰的两组 band，而且同一个 SM 在所有地址上始终落在与之相对的同一组里（它对本 die 地址快、对另一 die 地址慢，从不摇摆）。这说明被探出的二分不依附于某个具体地址，而是 SM 自身的稳定属性，即它所在的 die。由此这张 SM 到 die 的标签表是确定且可复用的——对任意一个新地址，只要做一次单 SM cold load 探测，就足以读出该地址在哪颗 die，进而所有 SM 都拿到自己的 die 标签。

把 SM 到 die 标号按物理 smid 排布，得到图 4 的映射。它是确定的、周期性的、由架构固定：在前若干 smid 区间内每 16 个 SM 为一行、行间模式一致，越过中部边界后模式发生规律性旋转。这张映射可以一次性导出，并以一张常量表的形式供后续 kernel 在入口直接查表使用。

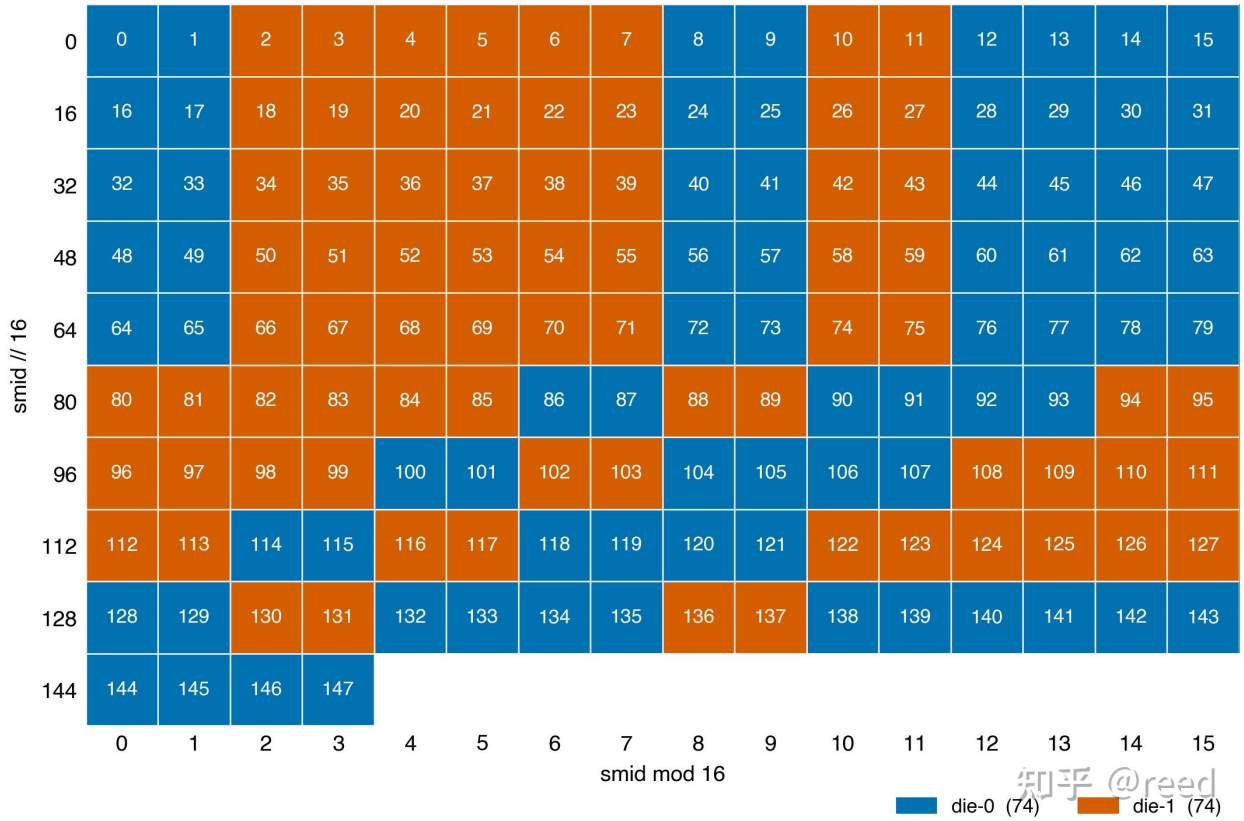


图 30.4: Figure 4. SM-to-die map (each cell is one of 148 SMs, labeled with its smid).

30.4 四、地址到 die：一个三层确定性结构

有了 SM 到 die 的标签表，量尺与被测对象就互换了：现在反过来，用一颗已知 die 的 SM 去探测内存——它对某地址的延迟落在哪条 band，就直接读出该地址在哪颗 die。沿连续地址滑动这把量尺，便能勾勒出内存的 die 归属如何随地址变化。结果并非杂乱无章，而是一个三层、完全确定的结构，可以从地址 + 一个 per-allocation 的常量闭式求解：最底层是 die 分配的原子粒度，中间层是 2 MiB 块内由地址位奇偶决定的图样，最上层是块的极性如何随分配排布。下面把这三层粒度记作 G0、G1、G2（G 取 granularity），自底向上逐层拆开。

G0：4 KiB 是 die 分配的原子单位。用一颗已知 die 的 SM，以 16 B 步长细扫一段地址（图 5）：die 标签的边界精确落在 4 KiB 处，且是单步翻转——偏移 4095 与 4096 之间延迟差直接由 +400 cyc 跳到 −400 cyc，没有过渡区。每个 4 KiB 段整段属于同一颗 die，跨过 4 KiB 边界即翻 die。

G1：2 MiB polarity tile，由 8 个地址位的奇偶决定。把尺度放大到 256 MiB，每 4 KiB 取一个样本得到图 6：横轴是该样本在 2 MiB 块内的位置（512 个 4 KiB 段），纵轴是 128 个连续的 2 MiB 块。图中可以同时看到两件事——4 KiB 段的 die 图样确实以 2 MiB 为周期重复（每一行内部呈现同样形态的细密交错），而相邻行之间只在两种互补的图案间交替。我们把这样一个 2 MiB 块称为 chunk。每个 chunk 内恰好 256 段属 die-0、256 段属 die-1，且归属服从一个确定的 8 bit hash：

```
mask 0x1EF000 = bits {12,13,14,15, 17,18,19,20}  (bit 16 被跳过)
die_intra(addr) = popcount(addr & 0x1EF000) & 1
```

我们在 128 个 chunk、65 376 个干净样本上验证，符合率 100%。沿一个 chunk 内 512 个段的序号看过去，die 标签就是该序号在这 8 位上的 popcount 奇偶——一条 Thue-Morse 样的交替序列。用地址位奇偶做这种细粒度交织，其效果（也是这类交织 hash 的惯常用途）是把任意规则步长的访问尽量均摊到两颗 die 上：固定的访存 stride 不会与 die 划分共振、把访问系统性地压到单颗 die，从而避免某种特定 pattern 让负载在两颗 die 间严重倾斜。同样的”打散”思路在更大尺度上由 G2 延续：G1 在 chunk 内打散 4 KiB 段，G2 在 chunk 之间打散 polarity，两层叠加让任意 stride 的访问都难以持续偏向单颗 die。这个不对称的掩码——4 个连续位、跳过 bit 16、再 4 个连续位——有一个鲜明的后果：相距 64 KiB 的两个地址落在同一颗 die（popcount 不变），要相距 128 KiB 才翻 die（bit 17 翻转）。bit 16 的跳过本身是个尚未完全解释的现象，最多只能说”很可能是某种 HBM / FBPA 条带 hash 在 64 KiB 处相消”。每个 chunk

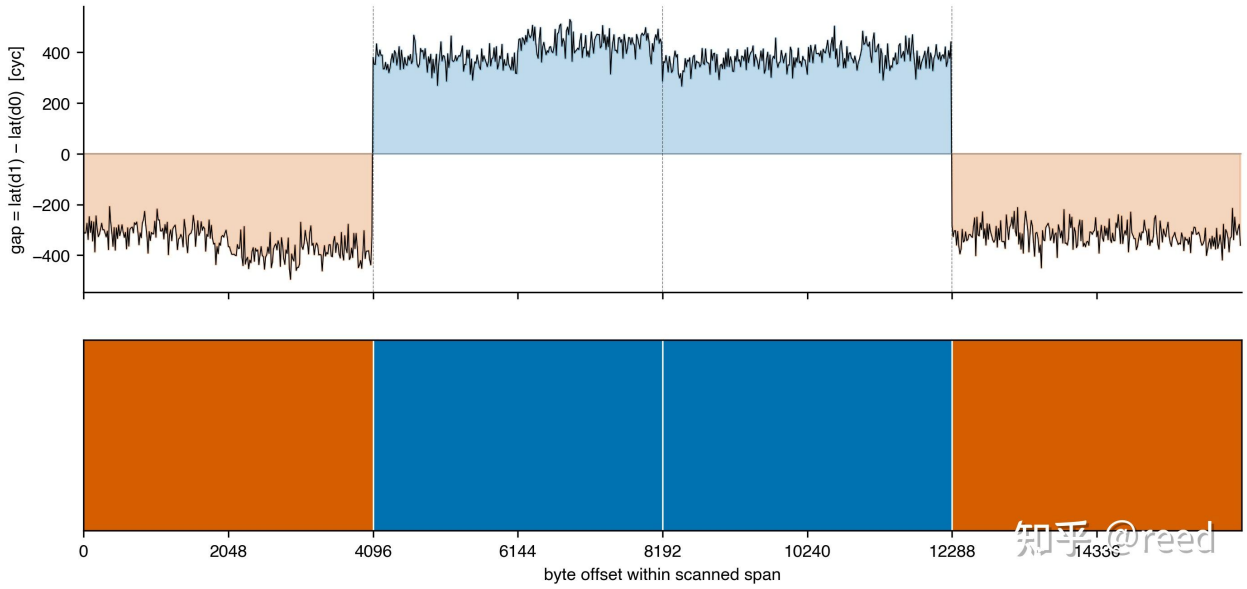


图 30.5: Figure 5. 4 KiB is the atomic die-assignment unit —single-step boundary.

只有两种互补的内部排布，二者恰好互为按位取反——我们用一个 1-bit 的 polarity（极性）记录每个 chunk 落在哪一种：polarity = 0 表示 chunk 内 seg 0 在 die-0、整块按 base 图样；polarity = 1 是它的取反副本。整块的 die 图样即 `die_intra(addr) ^ tile_polarity`。

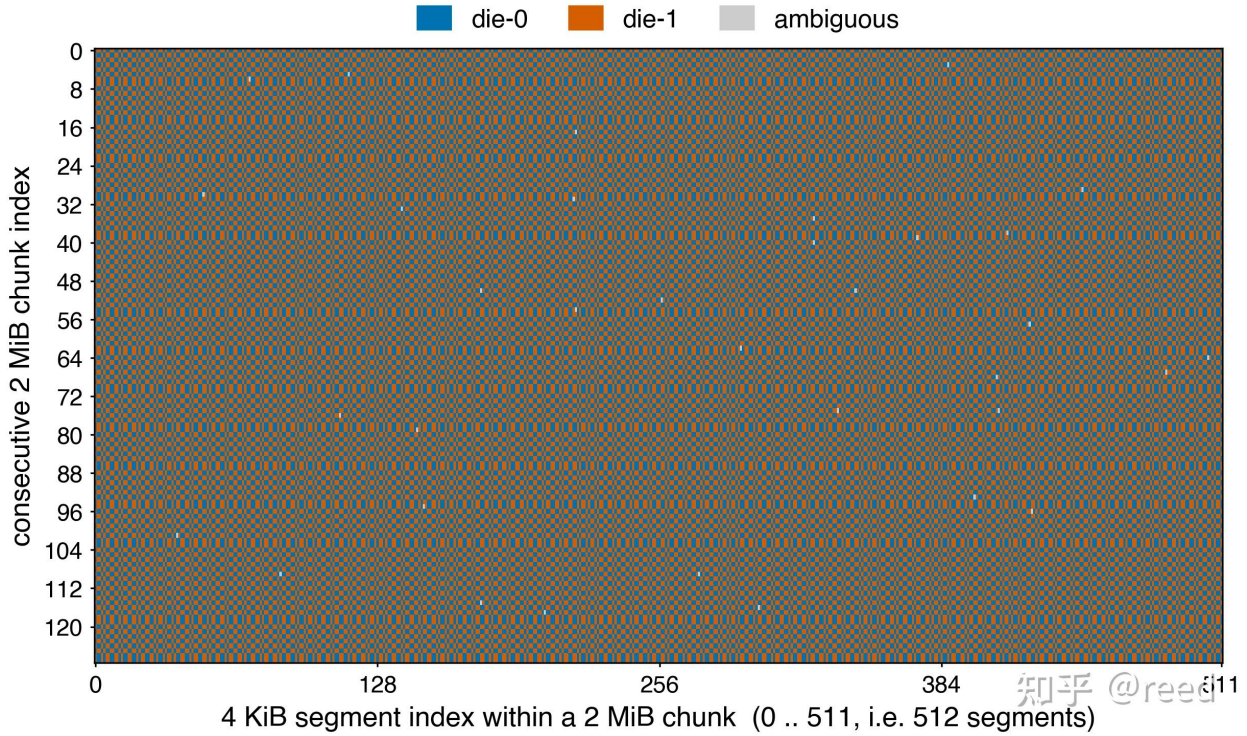


图 30.6: Figure 6. Per-2 MiB chunk pattern —two complementary layouts, 256/256 split.

G2：chunk 的 polarity 取自一条固定序列，分配只决定从哪里切入。G0、G1 已经把一个 chunk 内部的 die 图样确定为 popcount(addr & 0x1EF000) & 1，只差每个 chunk 的那一位 polarity。把一大段 cudaMalloc 出来、逐个 chunk 探测 polarity，会发现这一位并不是每次分配现掷的硬币，而是早已排好的一条固定序列（下称 canonical 序列；后文将看到，它在判据里就是那张 4096 位的 chunk_polarity[] 表，加上 8 GiB 反相位的扩展）：cudaMalloc(4 GiB) 得到的 2048 位 polarity，与依次 cuMemCreate(2 MiB) 申请 2048 个块得到的 2048 位逐位一致。这说明 polarity 不属于某次分配，而是物理 chunk 池本身的属性——池里每个物理块的 polarity 是固定的，分配只是按顺序从池中取块，因而必然复刻池的 polarity 排列。注意”按顺序”这一点是关键：同一段 VA 在两次分配中得到的不一定是同一段物理 chunk，因

此 polarity 序列也未必相同（具体规律见图 7）。

剩下的唯一自由度，是一次分配从这条序列的哪里切入。它只由分配从池中第几个块开始取决定，这个起点就是全局 cursor。图 7 用几组分配模式验证了这个模型：顺序分配时，cursor 每次前进一个分配的大小（E1、E2）；cudaFree 把取走的块还回池中，下一次同尺寸分配优先复用刚空出的位置，于是反复 alloc/free 会在少数几个起点之间轮转——对 ≥ 1 GiB 的分配实测为两个 zone 交替（E3、E4）。换言之，只要拿到一次分配的起点 offset，整段地址的 polarity 就能照着 canonical 序列查出来。

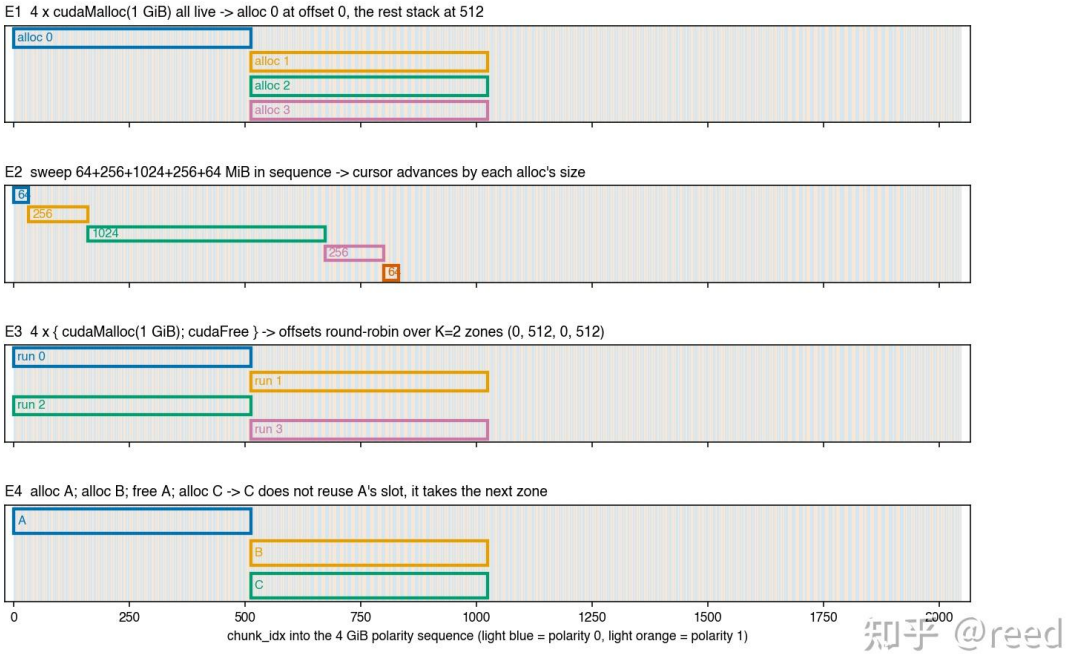


图 30.7: Figure 7. Each allocation enters one canonical sequence at a cursor offset; free + realloc round-robins K=2 zones.

三层合起来，就是一个对任意地址都成立的闭式判据。先做几个记号：把内存按 2 MiB 切成连续的块，第 k 个 2 MiB 块的位置记作 chunk_idx = k（即该块从分配基址起的 2 MiB 偏移序号；对地址 addr，它就是 `((addr - alloc_base) >> 21) + 0`，其中 O 是该分配进入序列的起点）。判据是：

```
die(addr) = popcount(addr & 0x1EF000) & 1       // G0+G1: 8 位地址奇偶
          ^ chunk_polarity[chunk_idx mod 4096]  // G2: 512 字节常量
          ^ (chunk_idx >> 12) & 1               // G2: 8 GiB 反相位
```

三项各自来源不同：

第一项是 G0、G1 在 8 个地址位上的闭式算术，对任意 VA 立即可算。

第二项 chunk_polarity[] 是这条 polarity 序列的核心。它是一张只有 4096 位（= 512 字节）的常量表，每位描述一个 2 MiB chunk 的极性，覆盖连续 8 GiB（= 4096 个 chunk）；G1 里的 tile_polarity 即第 c 个 chunk 在这里查到的那一位异或上 8 GiB 反相位，等价于 `chunk_polarity[c mod 4096] ^ ((c >> 12) & 1)`。它没有更短的闭式描述——4096 位的具体内容须靠测量得出，但所有更大尺度的结构都从这 4096 位推出：100 GiB 实测显示，`pol[c] == chunk_polarity[c mod 4096] ^ ((c >> 12) & 1)` 在 51200 个 chunk 上有 51182 匹配（剩余 18 位是探针噪声），即整段 100 GiB 严格服从这条公式。

第三项就是上面那条公式里的 `(chunk_idx >> 12) & 1`：每越过一个 8 GiB 边界，整张 chunk_polarity 表”翻转一次”——前 8 GiB 用 base、后 8 GiB 用 ~base、再后 8 GiB 又回到 base，如此正负正负地铺满整个内存空间。这就是为什么 4096 位的 base 表加一位奇偶，足以描述 B200 全部 192 GiB 内存里所有 chunk 的极性。

这张 4096 位常量是 per-arch 的：跨卡测得 8191/8192 一致（剩 1 位是探针噪声），不是 per-card；本仓库的 `polarity_scan` 工具一次性扫出后即可作为代码内常量永久使用。每次分配进入序列的起点 O 则是 per-allocation 的：分配入口对 buffer 的前 ~32 个 chunk 各做一次单 SM 探测、与常量表做最佳匹配即可定出，整个过程不到一秒。

至此地址到 die 完全确定：先做 8 位掩码 popcount，再查 512 字节表的一位，最后与 8 GiB 奇偶位异或，这三步操作的结果就是该地址所在的 die。

至此 SM 到 die、地址到 die 两套映射都已建立。下一节先验证它们的跨卡推广性，第六节再用它们做实测。

30.5　五、跨卡：结构普适，标号逐卡

这套结构能否跨卡复用，关系到映射推导的成本能否一次性摊销。在同一张卡上重复测量，每次都得到完全一致的 SM 到 die 映射，对固定设备它是确定性的。在多张 B200 上分别跑同样的 `smid_map` 探测后会发现：双 die 的结构本身普适——每张卡都呈现严格的 74/74 划分与同量级的跨 die 延迟差；但具体的 smid 到 die 的标号逐卡不同，如图 8 所示，差异既可能微小、也可能涉及大多数 SM。

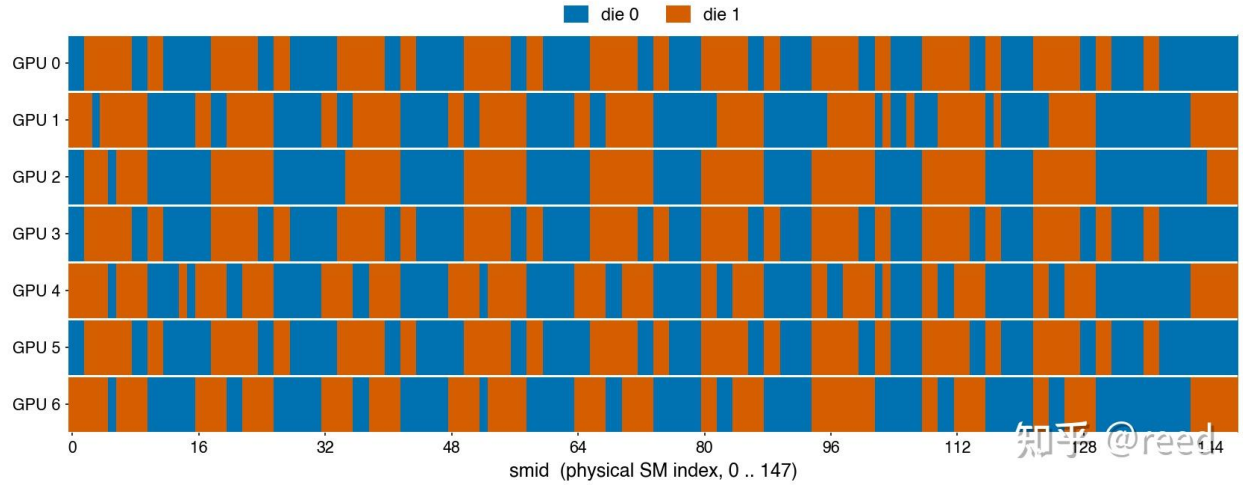


图 30.8: Figure 8. SM-to-die labels across multiple B200s —the 74/74 partition is universal, but the smid-to-die labels vary per device.

这在直觉上是合理的：双 die 的划分是硅片层面的硬件常量，而 smid 只是运行时暴露物理 SM 的逻辑编号，很可能在制造或固件阶段为良率与 binning（如屏蔽个别失效 SM）而设定，NVIDIA 并未文档化。其工程后果是明确的：SM 到 die 映射不能跨卡假定一致，必须按设备各自推导——即便两张卡碰巧一致，也不能据此推广。好在同卡推导一次即稳定，多 GPU 作业由每个进程各自推导本地映射即可。

30.6　六、die 亲和放置与只读带宽

把前面两套 die 归属——SM 在哪颗 die、某段地址在哪颗 die——合到一起用于优化，最直接的形式是让每个线程块只访问它所在 die 上的数据。

要干净地验证这一点，需要一段 die 归属已知、且最好处处一致的内存。借助第四节的闭式判据可以直接构造：用 VMM（Virtual Memory Management）接口逐个申请 2 MiB 物理块、探测其 polarity，只保留 polarity = 0 的块拼成一整段 global 内存。这样整段内存的 polarity 恒为 0，闭式判据退化为 `die(addr) = popcount(addr & 0x1EF000) & 1`——不查表就能算出任一地址在哪颗 die，且每个 4 KiB 段的 die 归属在整段内存里都按同一规则排布。

在这段内存上跑一个只读的聚合 kernel：每个线程流式扫过分给它的那些 4 KiB 段、把读到的字节累加进寄存器，工作集远超 L2、不写回，因而带宽被压到饱和、GB/s 直接反映读带宽。kernel 配置为 `<<<148, 1024>>>`（用 200 KiB 动态共享内存占满 SM，强制每个 SM 只驻留 1 个块，从而 148 个块铺满 148 个 SM）。我们只改”哪个块去读哪些段”这一张对应表，构造三种配置：

```
die-affinity:  每个 SM 只读本 die 的段（用第三节的 SM 到 die 表 + 闭式判据配对）
cudaMalloc:    内存换成默认 cudaMalloc（polarity 不再恒为 0），块仍读自己那一段，
               于是约一半的段随机落在跨 die
die-anti:      每个 SM 只读对侧 die 的段，每条 LDG 都跨 NV-HBI
```

三种配置共用同一个 kernel 二进制，grid、block、寄存器数、指令数、搬运字节数全部相同，唯一的变量是每条 LDG 实际打到本 die 还是对侧 die。结果如图 9、下表：

映射	输入带宽	占 8 TB/s 峰值 (* 实际 7.7TB/s)
die-affinity	6226 GB/s	77.8%
cudaMalloc	5800 GB/s	72.5%
die-anti	4396 GB/s	55.0%

affinity 对 anti = +42% 带宽，而 cudaMalloc 干净地落在中段，正是随机极性在段级混合同 die / 跨 die 所预期的位置。这一收益完全来自把计算与数据放在同一颗 die，代价仅是 kernel 入口一次 SM 到 die 查表。

这个差并非来自单次访问变快。第一节已经指出，访存一旦在两颗 die 间均摊，聚合带宽就看不出 die 边界；affinity 与 anti 的差之所以出现，是因为它们经过的路径不同。两种配置在 HBM 控制器侧的负载是对等的——affinity 让各 die 的 SM 读各自 die 的段，anti 让各 die 的 SM 全部读对侧 die 的段，但两侧的 HBM 控制器组都各承担一半总流量；差异只在中间这一跳：anti 配置下每条 LDG 都要经 NV-HBI、并在请求方与归属方两颗 die 的 L2 各做一次 sector 查找（下文 ncu 段落的 1.5×/2× 放大正是这条路径的直接指纹）。

决定饱和吞吐的不是单次延迟，而是每个资源的排队：由 Little 定律，吞吐 ≈ 在飞请求数 ÷ 有效服务时间。anti 配置下每条请求的有效服务时间因跨 die 多一跳路径与额外的 L2 查找而上升，且 NV-HBI 聚合带宽（标称 ≥ 10 TB/s）虽然超过单 die 本地带宽，但被双向流量同时占用。两者叠加，使得一个空载下并不显眼的单次延迟差，在饱和下被放大成可观的带宽差。跨 die 惩罚的本质，不是单次访问慢了多少，而是同一份请求被两次占用了沿途的资源，至于哪一环（NV-HBI 链路、两侧 L2 partition、还是再下游）先成为瓶颈，本文未做穷尽对比；带宽收益则是把延迟探测给出的 die 映射用于回避这种占用之后的结果。

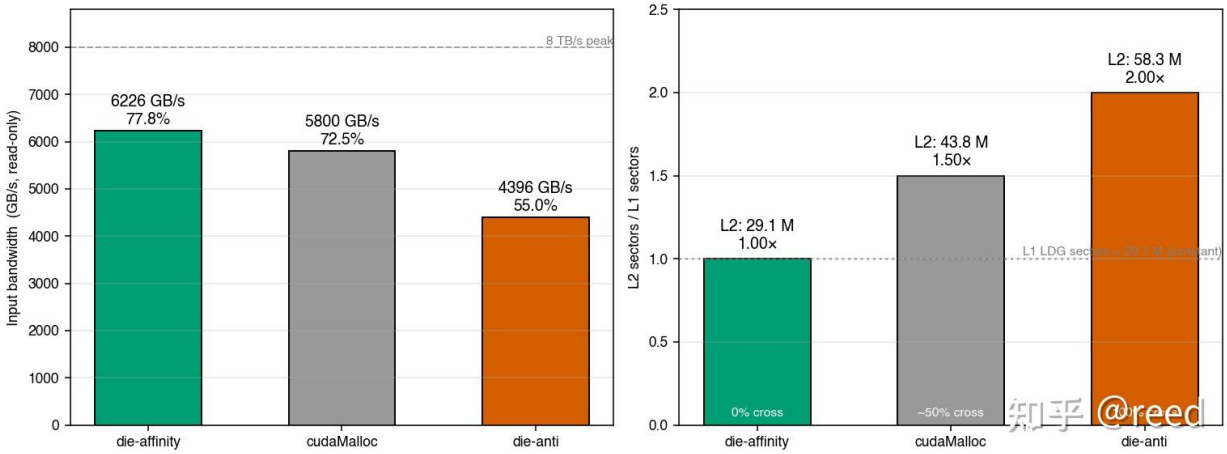


图 30.9: Figure 9. Affinity / cudaMalloc / anti bandwidth, and the L2 sector amplification from ncu.

ncu 的指标进一步确认了 NV-HBI 的角色。三种配置下 SM 侧指标（指令数、warp 占用、scoreboard 压力）完全相同，DRAM 实际读字节也相同；唯一随跨 die 比例变化的是 L2 sector 放大：L1 在每种配置下都发出相同的 sector 读，而 L2 服务的 sector 数恰好是 1× / 1.5× / 2×：

```
L2 sectors / L1 sectors = 1 + cross_die_fraction
  affinity ( 0% 跨die): 1.00×
  cudaMalloc (~50%)   : 1.50×
  anti (100%)         : 2.00×
```

最自然的解读是：每条越过 die 分界线的 LDG 要在两颗 die 各自的 L2 partition 各做一次 sector 查找——请求方 die 的 L2（未命中、转发）和归属方 die 的 L2（未命中、下到 HBM）。这就把 SM 到内存的路径定性为：SM 接的是本 die 的 L2，L2 才是判定” 这条 line 不归我、转发出去” 的单元；换言之，一次跨 die 访问要顺着” SM、本 die L2、NV-HBI、远 die L2、HBM” 这条链走完——NV-HBI 不是挂在旁边的总线，而是把两颗 die 的 L2 缝合成一块分布式缓存的链路。需要提醒的是：本节为了让 “每个 SM 唯一对应一个 block，读自己 die 那一份” 这件事干净成立、把所有差异都归因到 die 边界本身，选了 148 × 1024 + 200 KiB 动态共享内存这种 “每 SM 仅驻留 1 个 block” 的形态，它不

是带宽最优的，提高 occupancy 或使用 TMA 可以把绝对带宽再次提高，但本节的解释机理（NV-HBI 路径上的双 L2 partition 占用、L2 sector 1× / 1.5× / 2× 放大）对那些形态同样成立。

30.7　七、工程可用性与局限

放置本身不依赖调度器：每个块在入口读自身 `%smid`、查出所在 die、再选对应 die 的数据区，调度顺序不可预知也无妨。即使不做精细的逐块亲和，仅把数据均分到两颗 die 轮流使用，就已显著优于所有访问倾斜到单颗 die 的退化情形（这并不需要谁主动制造——一个对 die 拓扑无感的分配器加上恰好与 polarity 共振的访存 stride 就够了）：均分让两颗 die 的 HBM 同时出力，倾斜则受限于单 die 带宽。精细亲和是在此之上再逼近峰值。

要落到真实工程，有两处待解决。其一在分配侧：前文测的是 cudaMalloc / cuMemCreate，而业务代码通常经框架内存池（如 PyTorch caching allocator）、cudaMallocAsync 或 unified memory 拿到显存——这些接口下 polarity 序列与 cursor 是否同样确定，本文未单独验证；可行的折中是把 die 感知下沉到分配器——底层按 die 维护两个内存池、对上层仍暴露常规接口，分配时按调用方所在 die 选池。其二在地址侧：地址到 die 虽可闭式求解，但仍需先确定某段数据在哪颗 die——或由分配器在分配时记录，或用 canonical 序列加偏移探测推算，或如第六节用 VMM 拼出 polarity 恒为 0 的内存。如何最低成本地拿到地址归属，是走向通用的关键一环。

几处本文未覆盖、不应当作已解决：sub-1 GiB 分配的 zone 数 K、内存压力与多进程下的行为、以及第七节提到的非 cudaMalloc 分配路径（PyTorch caching allocator、cudaMallocAsync、unified memory）下 polarity / cursor 是否同样确定。最后，整套方法成立的前提是硬件确实以足够粗的粒度按 die 划分内存；若未来某代在更细粒度上交织、跨 die 惩罚低于可分辨阈值，则 die 感知放置不再有意义，应转而以聚合带宽为优化目标——本文的测量流程本身也能识别这种情形。

30.8　总结

本文把”Blackwell 跨 die 访问究竟有何代价”这一难以直接观测的问题，构建为一条可测量、可证伪的链路：

- 以一个测准单次冷 HBM 访问延迟的原语为量具，先固定一个必落在单一 die 的小地址、用 148 个 SM 各探一次，把每个 SM 标定到所在 die；再反过来以已知 die 的 SM 为量尺去扫地址，反推内存的 die 归属——这条自举链是整套方法的骨架；
- SM 到 die 是干净的 74/74：两条零重叠的延迟 band，且换多个独立地址复测划分始终一致，说明它是 SM 的稳定物理属性而非寻址假象；
- 地址到 die 是一个三层确定性结构 `die(addr) = popcount(addr & 0x1EF000) & 1 ^ chunk_polarity[chunk_idx mod 4096] ^ (chunk_idx >> 12) & 1`——4 KiB 原子段、2 MiB polarity tile（bit 16 被跳过，64 KiB 同 die / 128 KiB 翻 die）、以及一张 4096 位（512 字节）per-arch 常量加 8 GiB 反相位，整段 192 GiB 内存的 die 归属由它完全描述；
- 双 die 结构跨卡普适，但 smid 标号逐卡不同，SM 到 die 必须逐设备推导；
- 一个空载下约 +220 ns 的单次延迟差，在饱和下经排队效应放大为 +42% 的聚合读带宽差，ncu 中表现为 L2 sector 放大 1× / 1.5× / 2× = 1 + 跨 die 比例，表明 NV-HBI 位于 L2 fabric 之上。

就方法而言，最关键的两点是：在这套封装内 NUMA 上，可观测的主信号是延迟而非带宽；SM 到 die 映射必须逐设备推导。就落地而言，分配粒度与地址归属是两处尚待打磨的环节。此外还有一个本文未能测到的二阶收益：基准全程是冷的（L2 命中率 ≈ 0），而跨 die 访问会瞬态在两颗 die 的 L2 各占一个 line。极限来看，全跨 die 下每条 cache line 都在两边各占一个槽位，die 亲和等价于把 L2 有效容量翻倍——有待一个工作集驻留的 kernel 去量化。

30.9　参考

- Dissecting the NVIDIA Volta GPU Architecture via Microbenchmarking https://arxiv.org/abs/1804.06826

- Microbenchmarking NVIDIA’s Blackwell Architecture: An in-depth Architectural Analysis https://arxiv.org/abs/2512.02189
- John L. Hennessy, David A. Patterson. Computer Architecture: A Quantitative Approach https://dl.acm.org/doi/book/10.5555/1999263
- NVIDIA Blackwell Architecture Technical Brief https://resources.nvidia.com/en-us-blackwell-architecture
- NVIDIA Blackwell Datasheet https://www.primeline-solutions.com/media/categories/server/nach-gpu/nvidia-hgx-h200/nvidia-blackwell-b200-datasheet.pdf
- CUDA C++ Programming Guide —Virtual Memory Management https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#virtual-memory-management
- Nsight Compute —Metrics Reference（lts___t_sectors / dram___bytes_read） https://docs.nvidia.com/nsight-compute/ProfilingGuide/index.html
- Non-uniform memory access —Wikipedia https://en.wikipedia.org/wiki/Non-uniform_memory_access
- Little’s law —Wikipedia https://en.wikipedia.org/wiki/Little's_law
- Thue-Morse Sequence https://mathworld.wolfram.com/Thue-MorseSequence.html

第三十一章　FP8 Attention 中的精度优化：逆序计算与缩放因子选择

作者：reed（知乎 @reed-84-49，专栏「CUDA 高性能编程」）　原文：
https://zhuanlan.zhihu.com/p/2038747362596737621

随着 LLM 模型规模的持续增长，低精度计算逐渐成为提升训练和推理吞吐的核心手段。Hopper 架构在 Tensor Core 上首次提供了对 FP8（包括 E4M3 与 E5M2）的原生支持，FlashAttention-3 在此基础上把 attention 算子的中间结果也整体下放到 FP8。但 FP8 与 BF16 / FP16 在数值表示上的差距是数量级的，E4M3 仅用 3 比特描述尾数，相对精度只有 $2^{-3} = 12.5\%$，比 FP16 差了一到两个数量级。FP16 / BF16 时代被认为无关紧要的实现细节，例如 KV block 的迭代方向、softmax 之后 P 的量化缩放因子取值，到了 FP8 下都会被显著放大并影响最终精度。

本文围绕 Hopper 上的主流 FP8 FlashAttention 布局（O 累加器全程 FP32、P 显式 cast 为 E4M3）讨论这两个问题，给出对应的工程修法：KV block 逆向迭代与 $S = 256 = 2^8$ 的静态量化 scale。文章结构上，首先回顾 FP8 E4M3 的数值结构以及 Attention Sink 在正向迭代中如何使 P 矩阵塌陷到 subnormal 区域之下，从机理上引出逆向计算策略；然后从 IEEE 754 浮点数学与 E4M3 数轴几何两个角度证立 $S = 256$ 是最优解；最后对照主流 FP8 attention 实现的具体选择，并给出对照实验。

需要事先指出的是，Attention Sink 也有训练侧的解决路径，例如 learnable sink token、clipped softmax 等，本文走的是另一条正交方向，即在 sink 已经存在的预训练模型上从 kernel 层规避 FP8 量化引入的精度退化，不需要重新训练。

31.1　Attention Sink 与 P 的精度退化

Self-Attention 是 Transformer 的核心算子，

$$O = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V.$$

实际实现中 K、V 沿序列维度切分为若干 block，通过 Online Softmax 维护 running max 和 running sum，逐 block 迭代完成精确归一化。Hopper 上 FP8 FlashAttention 的精度策略是 Q、K 在第一次矩阵乘前已是 FP8，矩阵乘累加器与输出 O 全程保持 FP32，softmax 之后的 P 是 FP32，在送入 $P \cdot V$ 之前需要显式 cast 为 FP8 E4M3。P 是链路中真正发生 FP8 量化的张量，本文讨论的精度问题都围绕它展开。

31.1.1　FP8 E4M3 的精度结构

E4M3 的位宽布局为 1 位符号、4 位指数、3 位尾数；最大正值 $448 = 7 \times 2^6$，最小正规格化数 2^{-6}，最小正非规格化数 $2^{-9} \approx 1.95 \times 10^{-3}$。

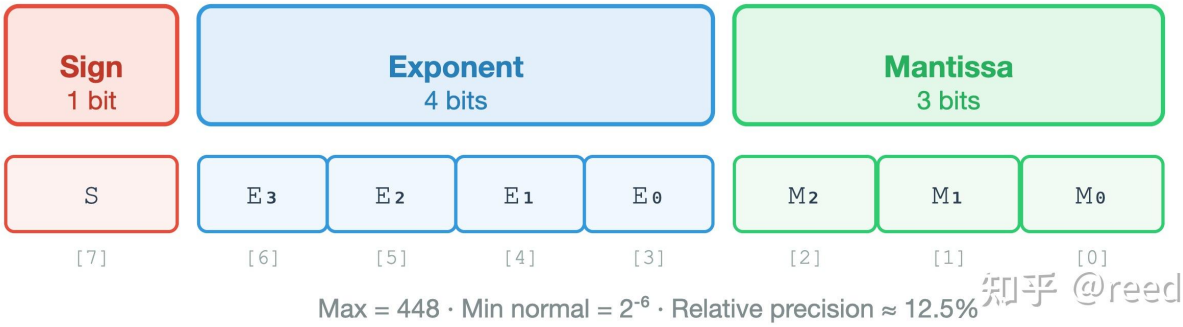


图 31.1: Figure 1. FP8 E4M3 Bit Fields Layout

3 位尾数意味着任意一段 binade $[2^n, 2^{n+1})$ 内只有 $2^3 = 8$ 个均匀分布的可表示值，相对精度 $2^{-3} = 12.5\%$。E4M3 的字段定义如图 1 所示，正半轴可表示值的分布如图 2 所示。

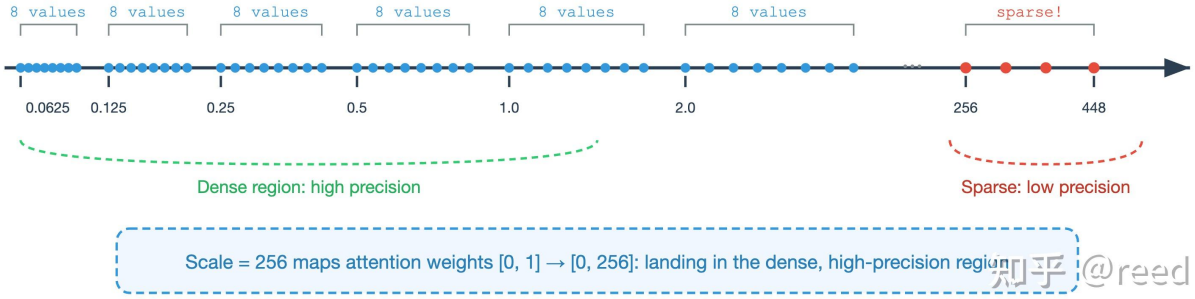


图 31.2: Figure 2. FP8 E4M3 Representable Values (Positive Axis)

E4M3 的可表示值绝对间距（LSB）随 binade 上行成倍变大，到 $[256, 512)$ 这一段 LSB 已经达到 32。一个连续实数被量化到 FP8 时，其量化误差直接由它落入的 binade 决定。下表给出后文几何分析会反复用到的几个关键 binade。

binade	LSB	半 LSB（最坏量化误差）
[2^-9, 2^-6)（subnormal）	2^-9	2^-10
[64, 128)	8	4
[128, 256)	16	8
[256, 512)（含 max_normal = 448）	32	16

31.1.2 Attention Sink 与 P 塌陷

Attention Sink 是经过充分训练的 Transformer 中一个被广泛观察到的现象，序列开头的若干 token 会获得异常高的注意力权重，对应到 logit 矩阵 $S = QK^T/\sqrt{d_k}$ 上则表现为序列首部若干列的 score 显著大于其他位置。多项实证研究表明，主流预训练 LLM 中的 sink 强度 $\Delta = S_{\text{sink}} - S_{\text{normal}}$ 在 context 长度数千量级时大致落在 $\Delta \in [6, 13]$ 区间内。

下面分析 Online Softmax 在正向迭代时如何与 sink 相互作用。每处理一个新的 KV block 走五步：(i) 算局部 score S_{local}；(ii) 更新全局最大值 $m_{\text{new}} = \max(m_{\text{old}}, m_{\text{local}})$；(iii) 算修正因子 $\alpha = \exp(m_{\text{old}} - m_{\text{new}})$；(iv) 算局部概率 $P_{\text{local}} = \exp(S_{\text{local}} - m_{\text{new}})$ 并 cast 为 FP8 E4M3；(v) 更新 FP32 累加器 $O_{\text{new}} = \alpha \cdot O_{\text{old}} + P^{\text{fp8}}_{\text{local}} \cdot V^{\text{fp8}}_{\text{local}}$。

在标准的正向迭代中，sink 位于 Block_0，第一步就把 m_{global} 推到 Δ 量级。后续正常 block 的局部 score 大致分布在 $[-1, 1]$，因此从第二步开始 $P_{\text{local}} \sim \exp(-\Delta)$。只要 $\Delta \gtrsim 9$，$\exp(-\Delta) \lesssim 10^{-4}$ 已经低于 E4M3 subnormal 下界 2×10^{-3}，P 中所有非 sink 位置的元素在 cast 时被舍入为零，最终 $P^{\text{fp8}} \cdot V$ 的结果只剩 sink token 一列的贡献，正常 token 的注意力信息在 cast 这一步整体丢失。

值得指出的是，这一精度退化的根源不在累加器（O 全程在 FP32 上累积），而在 P 这张需要 cast 到 FP8 的中间张量上；同时 P 塌陷是 sink 强度的临界现象，弱 sink 下 cast 仍能保留相对结构，只有当 sink 强到把 P 推到 subnormal 之下才出现整列归零的灾难。本文的实验在 $\Delta_{\text{sink}} = 12$ 下进行验证，对应主流预训练模型中较为典型的 sink 量级。

31.2 逆序计算策略

针对正向迭代下由 Attention Sink 引发的 P 塌陷，一个直接而有效的解决方案是将分块迭代顺序反转：$\text{Block}_n \to \text{Block}_{n-1} \to \cdots \to \text{Block}_0$。Online Softmax 在结合律下与迭代顺序无关，无限精度下结果严格等价，逆序不改变算法的数学正确性，差异只体现在有限精度下舍入误差的累积方式。

逆序对 P 塌陷的规避机理如下。逆序迭代从 Block_n 开始，由于序列中绝大多数 token 的 attention score 处于一个相对均匀且温和的范围内，前 n 步迭代过程中 m_{global} 始终维持在一个较小的水平（如 $m_{\text{global}} \approx 0.6$），对应的 P_{local} 元素分布在 $[\exp(-2), 1] \approx [0.14, 1]$ 区间内，完全落在 E4M3 normal 区域，cast 误差只来自正常的 round-to-nearest，没有 subnormal 截断。当迭代行进到最后一步遇到 sink 时，m_{global} 才发生一次大跳变，此时之前累积的 O_{old} 需要乘以一个极小的修正因子 α，但 O 全程在 FP32 上累积，FP32 的 23 位尾数足以保留 O_{old} 各分量之间的相对结构；同时这一

步的 P 来自 sink block 自身，元素值整体接近 1，cast 到 E4M3 后保持高精度。逆序将 P 塌陷到 subnormal 之下这一灾难性事件完全规避，将精度退化压缩到最后一步可控的 $\alpha \cdot O_{\text{old}}$ 缩放上。

具体到实现，逆序计算的修改非常简洁，仅需将 KV block 的迭代循环方向反转：

```
# 标准 FlashAttention (Forward Order)
for j in range(0, num_kv_blocks):           # 0, 1, 2, ..., n
    # online softmax + P cast to FP8 + P_fp8 @ V

# 逆序 FlashAttention (Reverse Order)
for j in range(num_kv_blocks - 1, -1, -1):  # n, n-1, ..., 1, 0
    # 算法逻辑完全一致
```

需要指出的是，FlashAttention 系列（FA2 / FA3 / FA4）的 KV 主循环本身就采用逆序方向，但其源码注释给出的动机是 mask phase 拆分与寄存器节省，与精度无关。本节给这条已经存在的工程默认补上一条独立的精度依据，使它可以被单独地施加到 hpc-ops、TensorRT-LLM XQA 等仍走正向 K-loop 的实现上。

31.3　缩放因子 256 的选取

除迭代顺序外，FP8 Attention 中另一个直接影响精度的实现选择是 P 在 cast 之前的静态缩放因子 S。完整的计算路径为

$$O = \frac{(P \cdot S)_{\text{fp8}} \cdot V_{\text{fp8}}}{S}.$$

S 的选择决定整条链路的精度上限。本节从 IEEE 754 浮点数学与 E4M3 数轴几何两个角度证立 $S = 256$ 是最优解，并就实现层面对第三条的具体落地做一点澄清。

31.3.1　二次幂缩放使乘除运算 bit-exact

在 IEEE 754 浮点数体系中，乘以或除以 2^k 是精确操作，其只改变 exponent 字段、不动 mantissa 字段，不引入任何舍入误差。$S = 2^k$ 时整条链路上的 $P \cdot S$ 与 O/S 都是 bit-exact 的，整条公式中只有 P 到 FP8 的 cast 这一步引入误差，其余所有缩放与反缩放都位精确。当 S 取非二次幂（如 250、300、448）时则不然，每一次 $P \cdot S$ 与 O/S 都是真正的 FP32 浮点乘除，每次额外引入约 2^{-23} 量级的尾数舍入误差。值得注意的是，E4M3 的最大正值 $448 = 7 \times 2^6$ 并不是二次幂，主流实现采用的 amax/448 类取值不享受这条性质。

31.3.2　dp(S) 锯齿波在二次幂中锁定 S = 256

仅满足二次幂这一条件还不能唯一锁定最优 S，需要再引入一层”最大表示误差”分析。对 P 中任意 $p \in [0, 1]$，乘 S 之后落在 $[0, S]$ 内某条 binade 上，cast 到 FP8 时被舍入到该 binade 的可表示值，反映回原域的最坏量化误差为

$$dp(S) = \frac{\max\limits_{x \in [0,\, \min(S,\, 448))} \text{LSB}_{\text{E4M3}}(x)}{S},$$

即 $[0, S]$ 内最稀疏 binade 的 LSB 除以 S。它给出 P 中任一元素在 cast 后能保证的最坏精度。把 $dp(S)$ 在 $S \in [2, 2048]$ 上画出来，便得到图 3。

曲线的形状揭示了 S 选取的全部几何结构：（i）每个 binade $[2^k, 2^{k+1})$ 内 $dp(S) = 2^{k-3}/S$ 单调下降，跨过二次幂边界后跳变并重新下降，形成锯齿波；（ii）每个二次幂边界 $S = 2^k$ 处 $dp(2^k) = 2^{-4} \approx 0.0625$，对所有 k 都是同一个值，构成下包络，任意非二次幂 S 的 $dp(S)$ 严格大于 2^{-4}、最坏可达 2^{-3}（一倍差距）；（iii）$S > 448$ 时 amax 越过 max_normal 被 clamp，$dp(S) = 1 - 448/S$ 随 S 上升迅速劣化。

由此从图 3 可以读出 $S = 256$ 的最优性：在 $S \le 448$ 不溢出约束下，二次幂候选 $\{2, 4, \ldots, 128, 256\}$ 都给出最低的 $dp = 2^{-4}$，落在下包络上；主流实现采用的 $S = 448$ 落在 binade $[256, 512)$ 内部，$dp = 32/448 \approx 0.0714$，比 $S = 256$ 高约 14%。落在下包络上的二次幂候选不止一个，最大的那个 k 还附带一个额外好处：能落入 E4M3 normal 区的 P 小值范围最广。E4M3 normal 区下界为 2^{-6}，反映回原域为 2^{-6-k}，S 越大该阈值越低。$S = 256$ 把该下界推到 $2^{-14} \approx 6.1 \times 10^{-5}$，比 $S = 128$ 的 1.22×10^{-4} 低一倍。

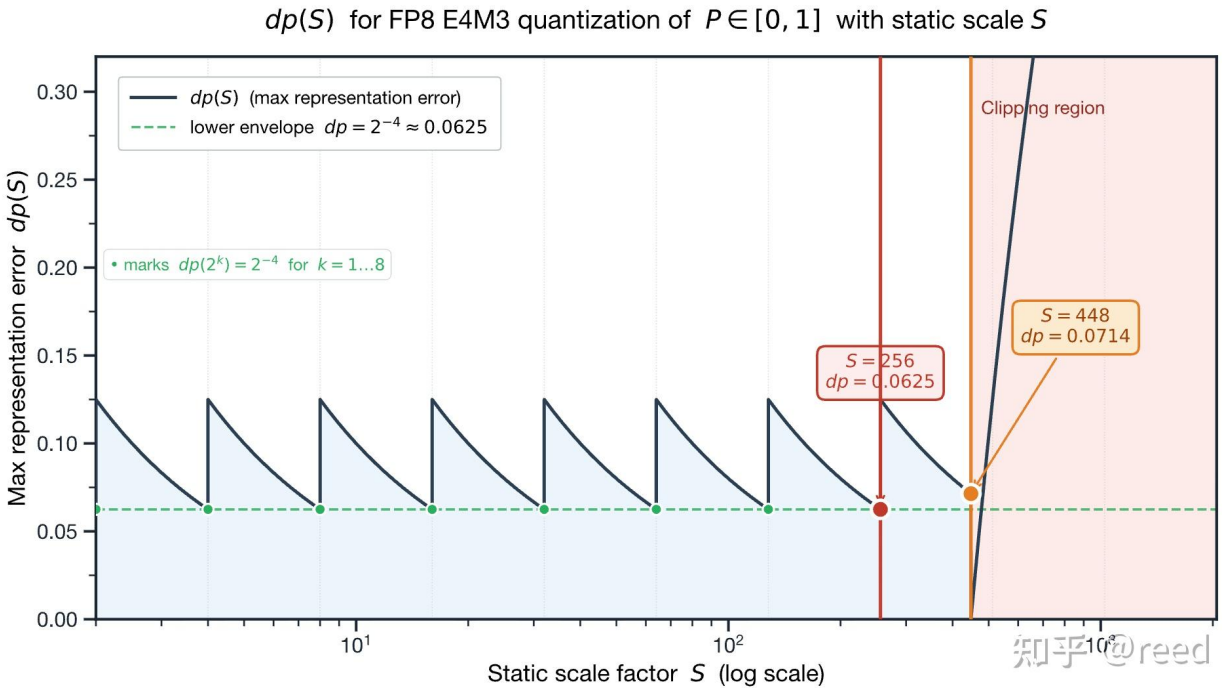


图 31.3: Figure 3. FP8 Quantization Error Analysis

S=256 可以最小化最大误差，S=448 可以最小化 subnormal，优先级上" 最大误差" 在前、" 保护 subnormal" 在后：因为 $O = P \cdot V$ 的求和结构使 amax 量级元素的误差以高权重传递到最终输出，而 subnormal 量级元素自身数值小、对 O 的贡献本就被压低。据此先用 $dp(S)$ 下包络这条硬条件淘汰所有非二次幂候选（包括 $S = 448$），再在通过的二次幂候选中取最大 k，得到 $S = 256$ 这一同时兼顾两项目标的唯一最优解。

31.3.3 实现层补充：精度优势就是工程优势的全部

第三条仅作为对实现层的澄清。Hopper SASS 没有直接计算 e^x 的指令，只有计算 2^x 的多功能单元指令 MUFU.EX2，因此主流 FlashAttention 实现把 $\log_2 e$ 折进 `softmax_scale`、用 $\exp_2$ 计算 softmax；P 在 cast 之前要乘上 S，落在实现上就是给 MUFU.EX2 的输入项再附加一个常数 $\log_2 S$，finalize 阶段则乘上 $1/S$。

需要说明的是，GPU 上浮点乘法（包括 FFMA）本身已经非常高效，乘以 256 与乘以 1/256 在指令开销上和乘以任意其他浮点常数没有差别——不存在" 2^k 走整数 bit-shift 比浮点除法快" 这类硬件 trick。S 取 2^k 与取 448 在性能上等价，唯一的差别就是上面理由一里讲过的那一条：当 $S = 2^k$ 时 S 和 $1/S$ 都是 IEEE 754 下精确可表示的浮点常数、且对 finite 数做乘法是 exponent-only 操作，不引入任何舍入；而 1/448 不是 IEEE 754 精确可表示的浮点数，对应的乘法每次都引入约 2^{-23} 量级的 round-to-nearest 舍入。也就是说，$S = 2^k$ 在实现层面相对非二次幂的全部优势就是精度上的优势，不存在额外的指令级好处。

将以上几条性质合在一起，$S = 256$ 是同时满足下列三条条件的唯一取值：(i) 二次幂使 $\times S$ 与 $\times(1/S)$ 都是 IEEE 754 下精确可表示的常数、对 finite 数做浮点乘法是 exponent-only 操作不引入任何舍入；(ii) 落在 $dp(S)$ 锯齿波下包络 2^{-4} 上，最大表示误差最小；(iii) 在 $S \le 448$ 不溢出约束下取最大值，最大化 normal 区对小值的覆盖。$S = 128$ 浪费一半 normal 区动态范围；$S = 512$ 越过 max_normal 触发 clipping；$S = 448$ 同时违反 (i) 与 (ii)，与 $S = 256$ 在 $dp(S)$ 上有约 14% 的差距，反映到平方阶的输出 MSE 上约为 $(1.14)^2 - 1 \approx 30\%$。

31.4 主流实现对比与实验验证

31.4.1 主流 FP8 Attention 实现的设计选择

下表汇总几个主流 FlashAttention 系列实现及外部 FP8 attention kernel 在 KV 顺序与 P scale 上的具体选择，均从源码中实际读出。

具体来看：

实现	KV 顺序	P 量化 scale	2^k	累加器
FA2 / FA3 (BF16)	逆向	—(P 不 cast)	—	FP32
FA3 / FA4 (FP8)	逆向	S = 256 = 2^8	是	FP32
Tencent hpc-ops	正向	S = 1（直接 cast）	是	FP32
FlashInfer	逆向	S = 448（贴 max_normal）	否	FP32
TensorRT-LLM XQA	正向	S = 448（贴 max_normal）	否	FP32
SageAttention2	正向	S = 448（per-block）	否	FP32
SageAttention2++	正向	S = 112（FP16 累加器约束）	否	FP16
本文	逆向	S = 256	是	FP32

- FlashAttention 系列。逆向 K-loop 从 FA2 起就已存在，FA3 / FA4 沿用；$S = 2^8$ 的 P-scaling 由 FA3 引入并被 FA4 在 SM100 路径上原样保留。两条选择源码注释给出的动机均与精度论证无关：逆向是为 mask phase 拆分与寄存器节省，$S = 2^8$ 的注释只说" use more of the FP8 range to reduce underflow"，即把 P 从 $[0, 1]$ 抬到 $[0, 256]$ 以减少下溢，但并未对" 为何取 2^8 而不是其他 2^k "给出几何或精度论证。这两条工程默认在 FP8 + Attention Sink 场景下精度上的好处是本文关注的副作用。

- 直接 cast 路线（hpc-ops，等价于 $S = 1$）。其 prefill kernel 对 softmax 之后的 P 直接 cast 为 E4M3，不引入显式缩放因子；K-loop 正向，P×V 用 FP32 累加器，O 在 epilogue 乘 v-scale 后 cast 为 BF16 写回 HBM。该实现对 sink 非常敏感，sink 让正向第一步把 m_{global} 钉死后，后续 P 元素整体落到 subnormal 之下被 cast 归零。

- 贴 max_normal 路线（FlashInfer）。softmax 之后对 P 乘 E4M3 max_normal（即 448）后 cast 到 FP8，再在 epilogue 中除回，等价于 $S = 448$；K-loop 逆向，与其 BF16 路径完全同构，纯属沿用 FA3 模板。

- 贴 max_normal 路线（TensorRT-LLM XQA）。NVIDIA TensorRT-LLM 的 generation 阶段使用自研的 XQA kernel，与 FA3 / FlashInfer 完全独立。K-loop 正向，P 在 cast 前乘固定常数 448，后续在 epilogue 反 scale。源码注释中给出 scale 选择的设计哲学：" 把 softmax 输出值域 $[0, 1]$ 的上界 1 直接映射到 E4M3 满量程 448"。

- 贴 max_normal 路线（SageAttention2 / 2++）。SageAttention-2 对 P 采用 per-block 静态 $S = 448$；SageAttention-2++ 将 P×V 累加器换成 FP16，由溢出约束 $|32 \cdot PV| \le 65504$ 反推出 $P_r = 112$、$V_r = 4.5$，实质等价于 $S = 112$。

$S = 256$ 与 $S = 448$ 是两个独立的设计选择，对应两条不同的设计逻辑：256 由" 二次幂、$dp(S)$ 下包络、不溢出取最大 k "三条几何条件唯一锁定；448 由" 贴 max_normal、对任意 amax 不溢出" 的工程兼容性单条考量得出。

31.4.2 实验：两项优化的有效性验证

为定量验证两项优化的具体效用，构造一组对照实验：以" Forward + $S = 1$" 作为基准（对应 hpc-ops 当前实现），与" Reverse + $S = 1$ "（仅施加优化 1）、" Forward + $S = 448$ "（amax/448 路线）、" Forward + $S = 256$ "（仅施加优化 2）、" Reverse + $S = 256$ "（两项优化叠加）四种配置在同一组合成 attention 输入上对比 MSE。实验在 Hopper FP8 layout 下模拟，注入 $\Delta_{\text{sink}} = 12$ 的 Attention Sink，序列长度从 512 扫到 8192，每个配置重复 5 次取均值。

图 4 给出完整的对照结果，可以观察到四个事实：

1. 基准严重退化。Forward + $S = 1$ 的 MSE 比其他四种配置高 1~3 个数量级，正是 P 元素被 sink 推到 subnormal 之下后 cast 归零的直接体现。

2. 逆序与加 scale 各自独立修复了基准。Reverse + $S = 1$ 与 Forward + $S = 448$ 都把 MSE 从 10^{-4} 量级压回 $10^{-7} \sim 10^{-6}$ 量级，长序列上落到同一数量级。逆序通过控制 m_{global} 轨迹避免 P 落入 subnormal，加 scale 则把 P 整体抬高远离 subnormal 下界，两条路径对修复 P 塌陷是等价的。

3. $S = 256$ 严格优于 $S = 448$。长序列上 Forward + $S = 256$ 比 Forward + $S = 448$ 低约 30%（`seq_len`=8192 上 1.4×10^{-7} vs 1.8×10^{-7}），与 $dp(S)$ 几何预测一致。

4. 两项优化叠加与单独 $S = 256$ 几乎重合。Reverse + $S = 256$ 与 Forward + $S = 256$ 差距均在 1% 以内，落在同一精度下界 $dp = 2^{-4}$ 上。这从实验上印证了两项优化修复的是同一个机理（subnormal 塌陷），叠加不再带来乘性增益。

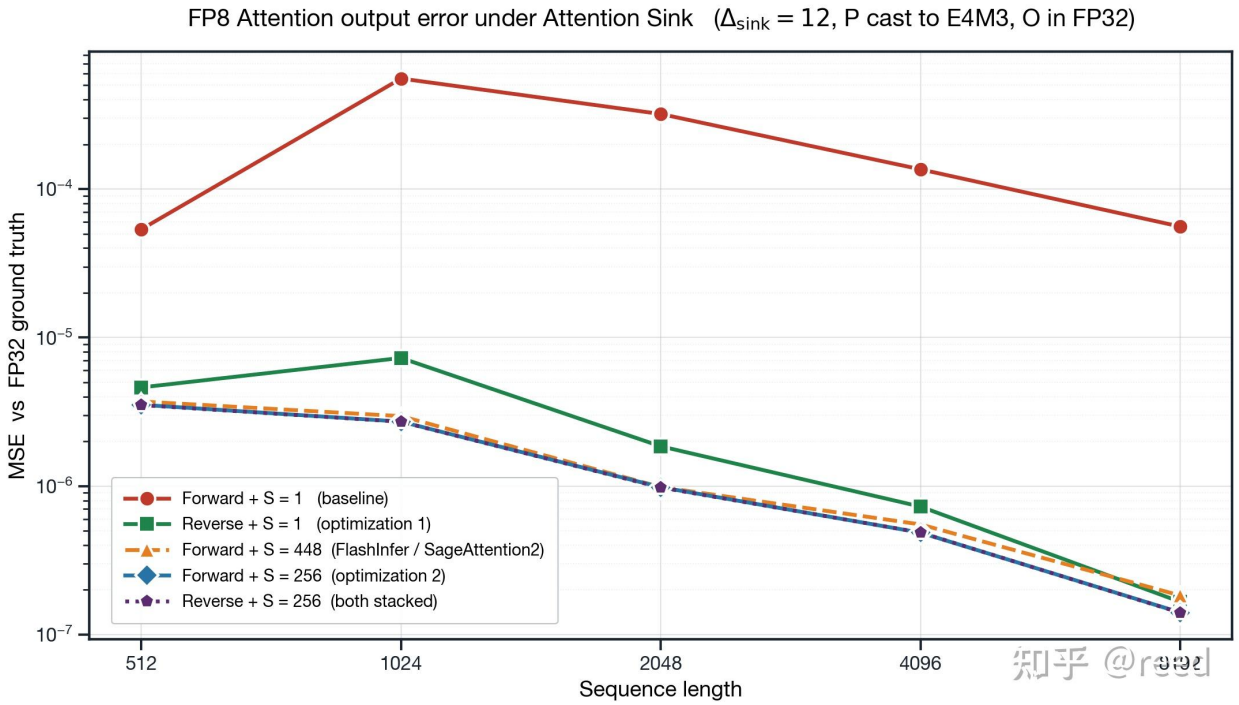


图 31.4: Figure 4. FP8 attention output MSE under Attention Sink

综合而言，逆序计算主要在 P scale 不足以保护的实现路径下发挥作用（hpc-ops 的 $S = 1$、TensorRT-LLM XQA 的正向 K-loop），单独施加即可把 MSE 改善 1~3 个数量级；$S = 256$ 在已有 scale 保护的路径下相对 $S = 448$ 这类非二次幂取值给出约 30% 的进一步改善。两项优化机理不同、可独立施加，叠加饱和到同一下界。两者实现成本均极低，前者反转 for 循环方向，后者把 scale 从 1/448 改为 1/256。

31.5 总结

本文从 FP8 E4M3 的数值结构出发，分析了 Hopper FP8 Attention 主流实现路径下的两个核心精度问题，并分别给出对应的工程优化。第一个问题是 Attention Sink 在正向迭代中引发的 P 塌陷，本文给出 KV block 逆序迭代作为解决方案；第二个问题是 P 在 cast 时的缩放因子选取，本文从 IEEE 754 浮点数学与 $dp(S)$ 锯齿波几何两个角度证立 $S = 256 = 2^8$ 是 $S \le 448$ 不溢出约束下的唯一最优解。最后通过对照实验定量验证了两项优化的有效性，并对照了 FlashAttention-3 / 4、FlashInfer、TensorRT-LLM XQA、Tencent hpc-ops、SageAttention2 / 2++ 等主流实现在 KV 顺序与 P scale 这两项实现选择上的具体取值。

31.6 参考

NVidia GPU 指令集架构-浮点运算 https://zhuanlan.zhihu.com/p/695667044
https://en.wikipedia.org/wiki/Binade
https://github.com/Dao-AILab/flash-attention
https://github.com/flashinfer-ai/flashinfer
https://github.com/NVIDIA/TensorRT-LLM
https://github.com/Tencent/hpc-ops
https://github.com/thu-ml/sageattention

第三十二章　CUDA shared memory 避免 bank conflict 的 swizzling 机制解析

作者：frankshi（知乎 @frankshi-38）　原文：https://zhuanlan.zhihu.com/p/4746910252

32.1　背景

Cuda shared memory 按照 4 字节一个 bank，总共 32 个 bank（128 字节）来组织，其 store 和 load 操作在一定情况下存在 bank conflict 的情况：

- 不同的线程访问同一 bank 的不同 address 时就会出现 bank conflict。
- bank conflict 只发生在同一个 warp 的不同线程间。
- 如果多个线程访问 shared memory 的相同 bank 的相同 address，实际效果是 broadcast，非 bank conflict。
- bank conflict 只发生在 shared memory 的读写操作上，global memory 的读写操作不会有 bank conflict 产生。

  在某些情况下，bank conflict 的发生是在 warp 中处于同一个 phase 的不同 threads 之间，这里的 phase 是指 warp 的 32 个线程操作 shared memory 时，分多个 phase，每个 phase 的参与线程不一样。比如 ldmatrix 指令`ldmatrix.sync.aligned.x4.m8n8.shared.b16`，该指令从共享内存加载数据到线程寄存器，操作分 4 个 phase，在 phase0 阶段，thread0 7 操作共享内存，在 phase1 阶段，thread8 15 操作共享内存，以此类推。

  bank conflict 会导致 warp 被 stall，冲突较多会对整个 pipeline 的耗时会有较大的影响。

  当发生 bank conflict 时，warp 需要额外的一个 cycle 来重新提交 shared memory 的访问指令到 LSU 单元，该指令需要在 MIO 中排队，这种排队会导致访问延迟增加，此时 warp 可能处于等待数据返回的状态，warp state 标识为 **Stall Short Scoreboard**。如果 MIO 队列满，此时 warp 先需要等待 MIO 队列处于非空的状态，此时 warp state 标识为 **Stall MIO Throttle**。

  解决 bank conflict 的主要有下面几种：

- padding。
- 转置存储（矩阵乘法的优化措施中，其中的一个典型操作是将从 global memory 读取的数据转置存储到 shared memory 中，本文中不做详细解释）。
- swizzling 机制。

本文以矩阵转置这个 kernel 的实现为例，通过几种实现方法的对比，来解释 swizzling 机制是如何避免 bank conflict。

32.2　矩阵转置基于 shared memory 的 naive 实现

下面是基于 shared memory 进行矩阵（矩阵大小：1024*2048）转置的一个 naive 实现：

```
const int M = 1024; //矩阵行
const int N = 2048; //矩阵列
const dim3 block_size(32, 32);
const dim3 grid_size(N/32, M/32);
matrix_trans_shm<<<grid_size, block_size>>>(dev_A, M, N, dev_B);
```

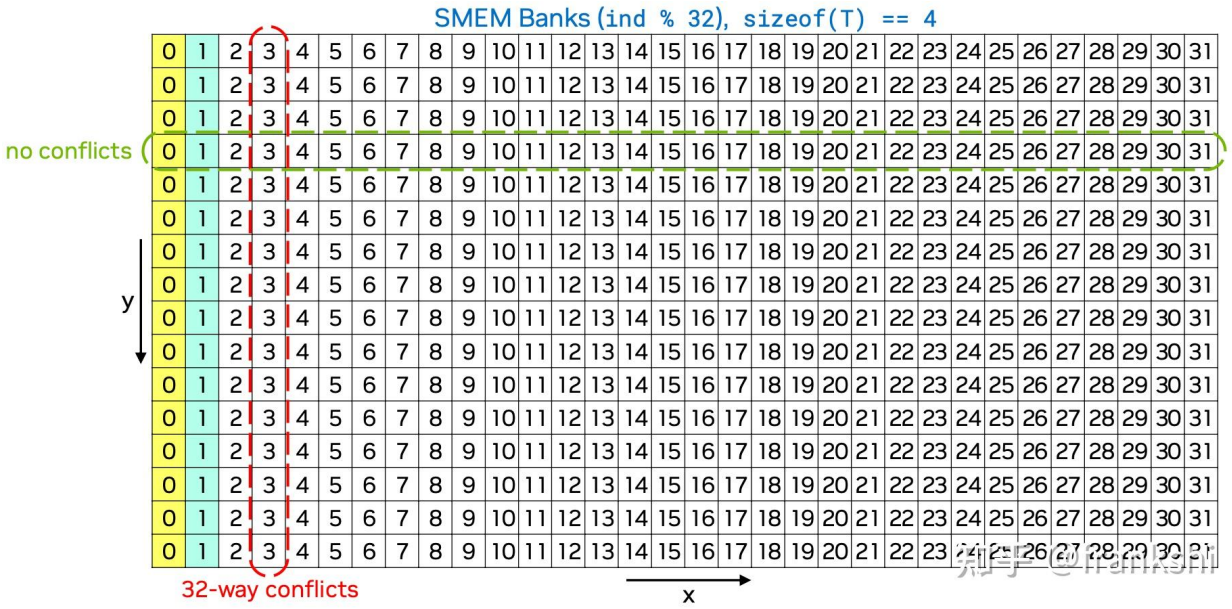
kernel 函数的实现如下：

```
// 转置前的矩阵存储在dev_A中，矩阵大小为M*N，转置后的数据存储在dev_B中
__global__ void matrix_trans_shm(int* dev_A, int M, int N, int* dev_B) {
  int row = blockIdx.y * blockDim.y + threadIdx.y;
  int col = blockIdx.x * blockDim.x + threadIdx.x;

  // 每个block处理32*32的矩阵块
  __shared__ int s_data[32][32];

  if (row < M && col < N) {
    // 从全局内存中加载数据，转置后写到共享内存中
    s_data[threadIdx.x][threadIdx.y] = dev_A[row * N + col];
    __syncthreads();
    int n_col = blockIdx.y * blockDim.y + threadIdx.x;
    int n_row = blockIdx.x * blockDim.x + threadIdx.y;
    if (n_col < M && n_row < N) {
      // 从转置后的共享内存按行写到全局内存结果中
      dev_B[n_row * M + n_col] = s_data[threadIdx.y][threadIdx.x];
    }
  }
}
```

上述代码中声明的 shared memory 数组`s_data[32][32]`，其中每个元素大小为 4 字节，刚好对应一个 bank，而一行 32 个元素刚好对应完整的 32 个 bank。数组中每个元素的对应的 bank 编号如下图所示：

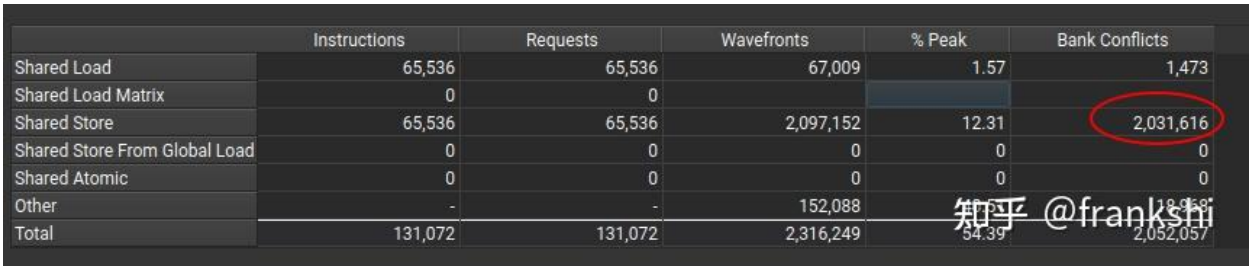


上图中只显示了前 16 行信息，后续行省略。图片从这里截取：Advanced Performance Optimization in CUDA - NVIDIA GTC 2024。

warp 是 SM 的基本执行单元，一个 block 内相邻的 32 个线程划分为一个 warp，一个 warp 内的 32 个线程按照 SIMT 的模式来执行指令。在上述代码中，每个 thread 从 global memory 读取一个元素后，转置存储到 shared memory 中，对应到 warp 层面的实际的操作是：

- warp 先从 global memory 读取连续的 32 个元素（Coalesced access to global memory）。
- warp 将读取的元素转置后写入到 shared memory 的同一列中。

可以看到，warp 内的 32 个线程在写数据到 shared memory 数组中时，对应的同一列，这些地址全部落到相同的一个 bank 中，也就是 32-way bank conflicts。通过 profile 的数据，可以看到 shared store 的 bank conflicts 达到了 2,031,616 次。



	Instructions	Requests	Wavefronts	% Peak	Bank Conflicts
Shared Load	65,536	65,536	67,009	1.57	1,473
Shared Load Matrix	0	0			
Shared Store	65,536	65,536	2,097,152	12.31	2,031,616
Shared Store From Global Load	0	0	0	0	0
Shared Atomic	0	0	0	0	0
Other	-	-	152,088	[illegible]	[illegible]
Total	131,072	131,072	2,316,249	54.39	2,052,057

知乎 @frankshi

说明：将转置后的数据从 shared memory 写入到输出内存时，是按**行**进行的 load 操作，理论上 warp 不应该存在 bank conflicts，但上面的截图显示仍然有 **1473** 次冲突，推测这个 load 冲突跟我们的这里的读取逻辑无关。

这个 2,031,616 次冲突的计算如下：

- 每个 warp 的 32 个线程，第一个线程的写不会冲突，其他的 31 个线程的写都会冲突，因此每个 warp 的 bank conflicts 的次数为 32-1=31。
- 每个 block 有 32 个 warp，总的 block 数为 (M*N)/(32*32)= (1024*2048)/(32*32) = 2048。
- 所有 block 累计的 bank conflicts 为 2048(block)*32(warp)*31(conflicts) = 2031616。

类似的 wavefronts 的数值为 2,097,152 的计算如下：

- 由于 bank 冲突，32 个线程的写都需要在 MIO 中排队，也就是需要分 32 次访问，每次访问对应一个 wavefront(cycle)，如果没有冲突，32 个线程只需要在 MIO 中排队一次，此时耗费的 wavefronts 数为 1。
- 所有 block 累计的 wavefronts = 2048(block)*32(warp)*32(wavefronts) = 2097152。

关于 wavefront 的概念：A *wavefront* is the maximum unit that can pass through that pipeline stage per cycle. If not all cache lines or sectors can be accessed in a single wavefront, multiple wavefronts are created and sent for processing one by one, i.e. in a serialized manner.

A wavefront is described as a (work) package that can be processed at once, i.e. there is a notion of processing one wavefront per cycle in L1TEX. Wavefronts therefore represent the number of cycles required to process the requests, while the number of sectors per request is a property of the *access pattern* of the memory instruction for all participating threads. For example, it is possible to have a memory instruction that requires 4 sectors per request in 1 wavefront. However, you can also have memory instruction having 4 sectors per request, but requiring 2 or more wavefronts.

32.3 使用 Padding

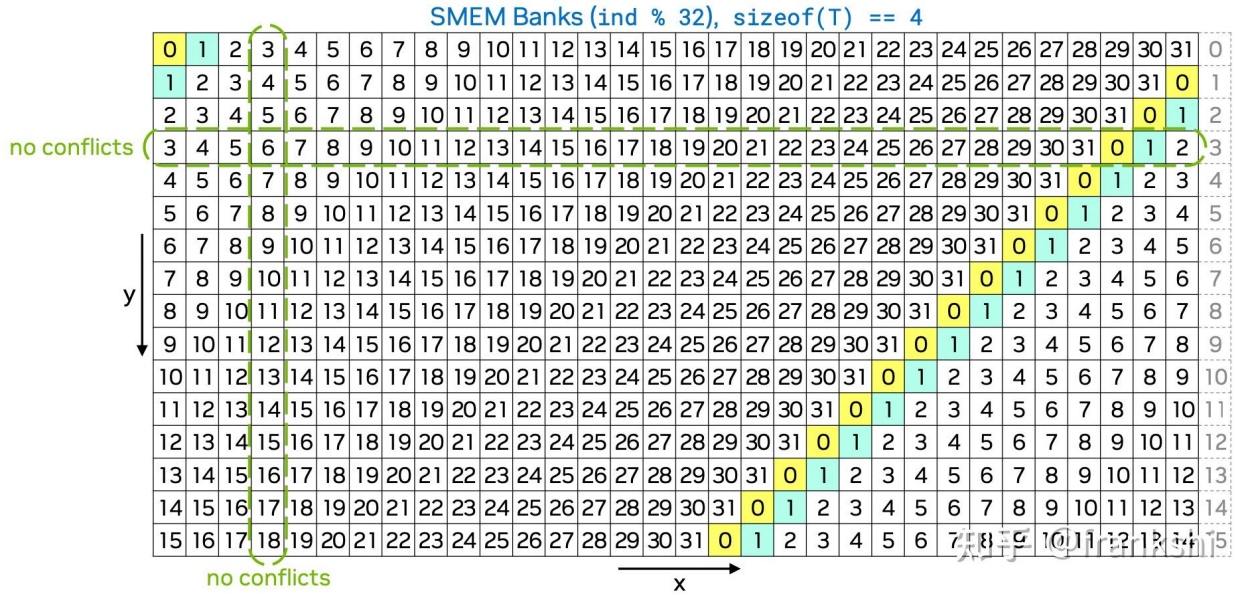
避免的 bank conflict 的一种方法是对 shared memory 使用 padding，通过在尾部 padding 一个元素，数组变为`s_data[32][33]`，这样相同列的不同行的元素的 bank 值不再一样，在转置时就避免了 bank 冲突。如下图所示：

新的代码如下：

```
__global__ void matrix_trans_shm_padding(int* dev_A, int M, int N, int* dev_B) {
  int row = blockIdx.y * blockDim.y + threadIdx.y;
  int col = blockIdx.x * blockDim.x + threadIdx.x;

  // 每个block处理32*32的矩阵块，尾部padding来避免bank conflict
  __shared__ int s_data[32][33];

  if (row < M && col < N) {
    s_data[threadIdx.x][threadIdx.y] = dev_A[row * N + col];
    __syncthreads();
    int n_col = blockIdx.y * blockDim.y + threadIdx.x;
```




```
12      int n_row = blockIdx.x * blockDim.x + threadIdx.y;
13      if (n_col < M && n_row < N) {
14        dev_B[n_row * M + n_col] = s_data[threadIdx.y][threadIdx.x];
15      }
16    }
17  }
```

profile 的结果如下图所示，可以看出 shared store 的 bank conflicts 的数目变为 0。

	Instructions	Requests	Wavefronts	% Peak	Bank Conflicts
Shared Load	65,536	65,536	75,699	2.61	10,163
Shared Load Matrix	0	0			
Shared Store	65,536	65,536	65,536	0.57	0
Shared Store From Global Load	0	0	0	0	0
Shared Atomic	0	0	0	0	0
Other	-	-	143,933	[illegible]	[illegible]
Total	131,072	131,072	285,168	9.84	20,976

知乎 @frankshi

padding 的缺点有：

- 可能降低 SM 的 occupancy。由于每个 SM 的可使用的 shared memory 有限，如果每个 block 使用的共享内存增加，则 SM 内最大可并发的 block 数目减少，导致资源不能被充分利用，一些计算资源被闲置。
- 地址访问对齐问题。需要仔细考虑 padding 的大小来避免地址不对齐的问题，比如访问 shared memory 时可能是向量化的访问，比如`int4`访问，也就是每次访问 4 个 int，即 16 字节，那每次访问的地址必须是 16 字节对齐的，对于`int s_data[32][33]`这种 padding 方式，第二行元素的起始地址就是非 16 字节对齐，会导致 kernel 执行出错。

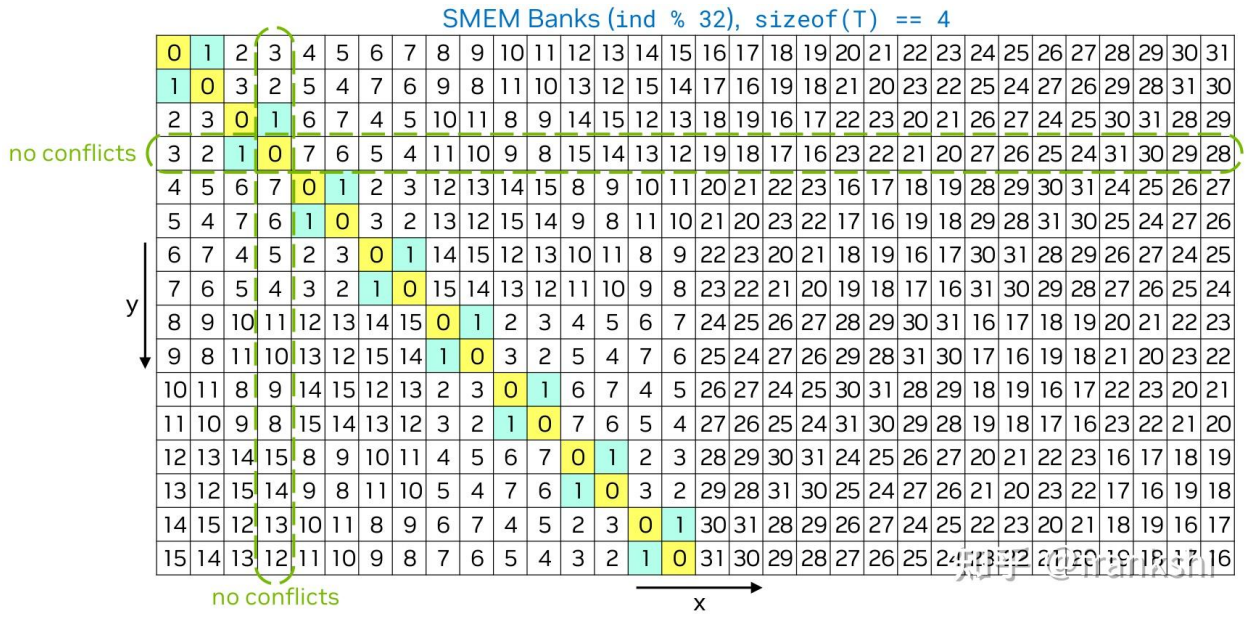
32.4 swizzling 机制

swizzling 是在不额外分配内存的情况下，通过将 shared memory 的数据进行重排来避免 bank conflict。下面是重排后的一个示例：

在上面的图中，每个元素有唯一的位置信息 (x, y) ，x 和 y 分别表示列和行，并且假设从 0 开始。重排后，每一行内的数据集合保持不变，但行内元素的相对位置进行重排，比如对于 $y = 1$ ，x 原来的索引顺序是 [0, 1, 2, 3, ..., 30, 31]，而新的索引顺序是 [1, 0, 3, 2, 5, 4, ..., 31, 30]。从上图可以看到，每一行/列的各个元素的 bank 值不再一样，这样就避免了 shared memory 在 load/store 操作时的 bank 冲突。

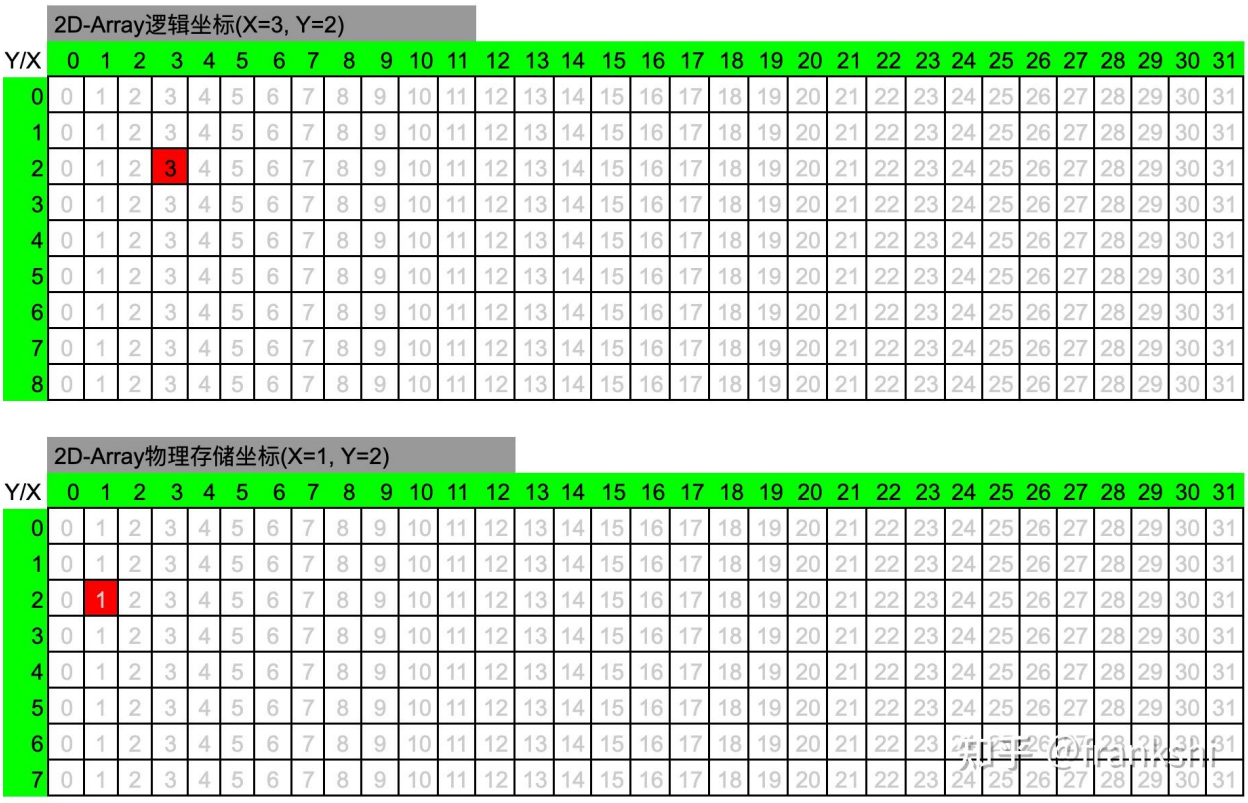
32.4.1 4.1 逻辑位置和物理位置

这里引入两个概念，逻辑位置和物理位置：



- 逻辑位置表示元素在矩阵中的逻辑坐标。
- 物理位置表示其对应元素在实际存储数据的 shared memory 中的位置坐标。

当我们说读取矩阵的第 2 行第 3 列的元素，这里 $(x=3, y=2)$ 就表示逻辑位置，而真正读取数据的时候，我们需要从实际存储数据的 shared memory 中对应的位置 $(x=1, y=2)$ 去读取数据（$x_p = x_l \oplus y_l = 3 \oplus 2 = 1$，$y_p = y_l = 2$），这里逻辑坐标被映射到了 shared memory 中的物理坐标，如下图所示：



在我们大部分的代码中，逻辑位置和物理位置是相同值，而在 swizzling 机制中，这两者不一样，存在一个映射关系：

$$(x_p, y_p) = f(x_l, y_l)$$

上面的公式中 (x_p, y_p) 表示物理存储坐标，(x_l, y_l) 表示逻辑坐标，x 和 y 取值都是整数。这个映射必须满足：

- 映射是一一对应的关系，即不能是多个逻辑坐标映射到同一个物理坐标或者一个逻辑坐标映射到多个物理坐标，否则就可能存在数据丢失或者重复的问题。

- 映射后的 x 和 y 的取值范围分别与映射前一致，否则可能会导致需要更多的 shared memory 容量，比如通过乘法映射也能满足一一映射关系，但映射后的值空间远大于映射前的范围。

32.4.2 4.2 swizzling 中的映射函数

对于 swizzling 机制，逻辑坐标 (x_l, y_l) 和物理坐标 (x_p, y_p) 的映射关系如下：

- $y_p = y_l$ ，物理行与逻辑行一致。
- $x_p = x_l \oplus y_l$ ，物理列等于逻辑行和逻辑列的异或值。

公式中的 $\oplus$ 表示异或操作，异或操作除了满足交换律和结合律之外，还有下面的一些性质：

- 性质一：$x \oplus x = 0$ 即两个相同的值进行异或，结果仍然为 0。
- 性质二：$x \oplus 0 = x$ 即任何值与 0 进行异或，结果等于自身。
- 性质三：如果 $x_1 \neq x_2$ ，则 $x_1 \oplus x_2 \neq 0$ ，即两个不同的值异或，结果一定不为 0。
- 性质四：如果 $x_1 \oplus x_2 \neq 0$ ，则 $x_1 \neq x_2$ ，即如果两个值异或的结果不为 0，则两者的值一定不同。

下面将证明我们采用的这样映射满足 4.1 节中的条件。为了简化证明过程，我们在证明之前先附加下面的两个假设：

- 条件一：x 和 y 取值范围一样。
- 条件二：x 和 y 的取值从 0 开始，最大取值为 2 的整数幂减去 1，比如取值范围：[0, 31]。

实际的应用中，可能有些场景不满足上面的两个假设，但采用的映射函数核心思想与上面的映射类似，都是基于异或操作，但会针对不同的场景（向量化访问或者多 phase 访问）进行一些调整，在「TMA 中的 swizzling 机制」一节中有相关的案例。

32.4.3 4.3 一对一映射

由于映射前后的坐标保持行不变，我们只需要证明，同行内两个不同列在映射后仍然保持列不同，即可证明坐标映射是一对一。下面我们将证明同一行 (y 相同) 的不同列 (x 不同) 在映射后的 x 值也不相等。假设：$(x_{l1}, y), (x_{l2}, y)$ 为同一行内的两个不同列的逻辑坐标，映射后的物理坐标为 $(x_{p1}, y), (x_{p2}, y)$ 则有：

- $x_{p1} = x_{l1} \oplus y$
- $x_{p2} = x_{l2} \oplus y$
- $x_{l1} \neq x_{l2} => x_{l1} \oplus x_{l2} \neq 0$

此时将 x_{p1} 与 x_{p2} 进行异或操作有：

$$x_{p1} \oplus x_{p2} = (x_{l1} \oplus y) \oplus (x_{l2} \oplus y) = (x_{l1} \oplus x_{l2}) \oplus (y \oplus y) = (x_{l1} \oplus x_{l2}) \oplus 0 = (x_{l1} \oplus x_{l2}) \neq 0$$

根据异或的性质四得出：$x_{p1} \neq x_{p2}$ ，即同一行的两个不同逻辑坐标 x 值在映射后的物理坐标 x 值也不一样。下面图显示 $y = 3$ 的行内映射后的 x 值各不一样：

反之，从映射后的物理坐标 x_p 也能很容易的计算得到映射前的逻辑坐标 x_l ，只需要将其与 y 再进行异或：

- $x_p \oplus y = (x_l \oplus y) \oplus y = x_l \oplus (y \oplus y) = x_l \oplus 0 = x_l$

Y/X	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
1	1	0	3	2	5	4	7	6	9	8	11	10	13	12	15	14	17	16	19	18	21	20	23	22	25	24	27	26	29	28	31	30
2	2	3	0	1	6	7	4	5	10	11	8	9	14	15	12	13	18	19	16	17	22	23	20	21	26	27	24	25	30	31	28	29
3	3	2	1	0	7	6	5	4	11	10	9	8	15	14	13	12	19	18	17	16	23	22	21	20	27	26	25	24	31	30	29	28
4	4	5	6	7	0	1	2	3	12	13	14	15	8	9	10	11	20	21	22	23	16	17	18	19	28	29	30	31	24	25	26	27
5	5	4	7	6	1	0	3	2	13	12	15	14	9	8	11	10	21	20	23	22	17	16	19	18	29	28	31	30	25	24	27	26
6	6	7	4	5	2	3	0	1	14	15	12	13	10	11	8	9	22	23	20	21	18	19	16	17	30	31	28	29	26	27	24	25
7	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	23	22	21	20	19	18	17	16	31	30	29	28	27	26	25	24
8	8	9	10	11	12	13	14	15	0	1	2	3	4	5	6	7	24	25	26	27	28	29	30	31	16	17	18	19	20	21	22	23
9	9	8	11	10	13	12	15	14	1	0	3	2	5	4	7	6	25	24	27	26	29	28	31	30	17	16	19	18	21	20	23	22
10	10	11	8	9	14	15	12	13	2	3	0	1	6	7	4	5	26	27	24	25	30	31	28	29	18	19	16	17	22	23	20	21
11	11	10	9	8	15	14	13	12	3	2	1	0	7	6	5	4	27	26	25	24	31	30	29	28	19	18	17	16	23	22	21	20
12	12	13	14	15	8	9	10	11	4	5	6	7	0	1	2	3	28	29	30	31	24	25	26	27	20	21	22	23	16	17	18	19
13	13	12	15	14	9	8	11	10	5	4	7	6	1	0	3	2	29	28	31	30	25	24	27	26	21	20	23	22	17	16	19	18
14	14	15	12	13	10	11	8	9	6	7	4	5	2	3	0	1	30	31	28	29	26	27	24	25	22	23	20	21	18	19	16	17
15	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16

知乎 @frankstar

32.4.4 4.4 映射前后的取值范围保持不变

还是以同一行（y 相同）的不同列（x 不同）在映射后的取值范围为例来说明。由于映射前的 x 的最大值为 2 的整数幂次方减 1，对应的二进制表示的所有 bit 均为 1，并且 x 和 y 的取值范围一致，故在该范围内，任意两个数异或后，最大可能值仍然为 2 的整数幂次方减 1。比如，对于 (x, y) 取值范围均为 $[0, 31]$ 来说，假设 y 取值为 3，x 在 0~31 中，一定有：

- 某个 $x = 3$ ，满足 $x \oplus y = 0$ 。
- 另外一个 $x = 28$ ，满足 $x \oplus y = 31$ 。

另外，由于映射后的元素两两不一样，所以可以得出映射后的范围也是在 0~31 之间分布，即映射前后取值区间保持不变。

32.4.5 4.5 列内任意两行在映射后的 x 值不同

现在我们将证明同一列（x 相同）的不同行（y 不同）在映射后的 x 值也不相等。假设：$(x, y_{l1}), (x, y_{l2})$ 为同一列内的两个不同行的逻辑坐标，映射后的物理坐标为 $(x_{p1}, y_{l1}), (x_{p2}, y_{l2})$ ，则有：

- $x_{p1} = x \oplus y_{l1}$
- $x_{p2} = x \oplus y_{l2}$
- $y_{l1} \neq y_{l2} => y_{l1} \oplus y_{l2} \neq 0$

x_{p1} 和 x_{p2} 的两者异或值为：

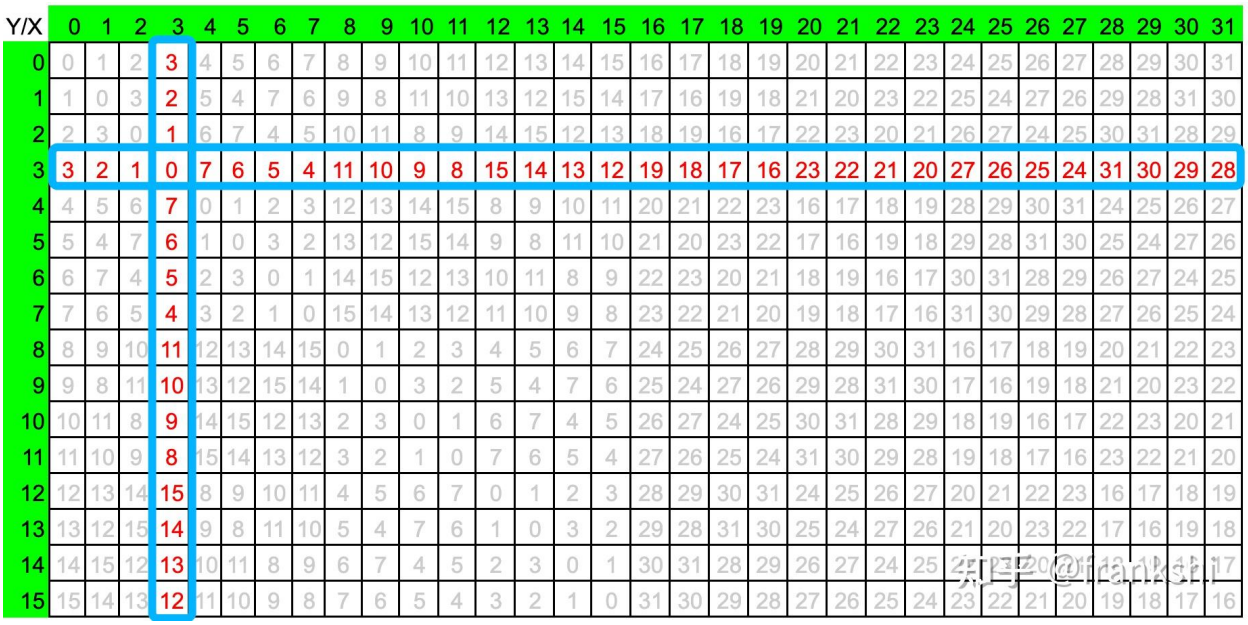
$$x_{p1} \oplus x_{p2} = (x \oplus y_{l1}) \oplus (x \oplus y_{l2}) = (x \oplus x) \oplus (y_{l1} \oplus y_{l2}) = 0 \oplus (y_{l1} \oplus y_{l2}) = (y_{l1} \oplus y_{l2}) \neq 0$$

根据异或的性质四得出：$x_{p1} \neq x_{p2}$ ，即同一列的两个不同逻辑坐标 y 值在映射后的物理坐标 x 值也不一样。另外，也很容易证明，映射后的某个行坐标值系列与对应的某个列值序列完全一致，比如：$y = 3$ 的行与 $x = 3$ 的列的值序列完全一致，如下图所示（下图只显示前 16 行）：

从上面的证明过程看，在映射后，任意行和任意列内都不存在 bank 冲突了。

32.4.6 4.6 swizzling 机制的代码应用实践

矩阵转置采用 swizzling 处理后的代码如下：




```cpp
__global__ void matrix_trans_swizzling(int* dev_A, int M, int N, int* dev_B) {
  int row = blockIdx.y * blockDim.y + threadIdx.y;
  int col = blockIdx.x * blockDim.x + threadIdx.x;

  __shared__ int s_data[32][32];

  if (row < M && col < N) {
    // 从全局内存读取数据写入共享内存的逻辑坐标(row=x,col=y)
    // 其映射的物理存储位置位置(row=x,col=x^y)
    s_data[threadIdx.x][threadIdx.x ^ threadIdx.y] = dev_A[row * N + col];
    __syncthreads();
    int n_col = blockIdx.y * blockDim.y + threadIdx.x;
    int n_row = blockIdx.x * blockDim.x + threadIdx.y;
    if (n_row < N && n_col < M) {
      // 从共享内存的逻辑坐标(row=y,col=x)读取数据
      // 其映射的物理存储位置(row=y,col=x^y)
      dev_B[n_row * M + n_col] = s_data[threadIdx.y][threadIdx.x ^ threadIdx.y];
    }
  }
}
```

代码中关于使用 swizzling 机制的代码逻辑如下：

- 块中的每个线程从 global memory 读取一个元素后，在转置前，对应 shared memory 的逻辑为坐标(x = threadIdx.x, y = threadIdx.y)，进行转置后存储的逻辑坐标为(x = threadIdx.y, y = threadIdx.x)。
- 对于一个 warp 内的 32 个线程，threadIdx.y值一样，threadIdx.x不同，导致存储到 shared memory 中的位置是相同列不同行中，从而导致 bank conflict。
- 此时，应用 swizzling 进行映射后的，转置存储时，行保持不变仍然为threadIdx.x，新的列为 threadIdx.x ^ threadIdx.y。
- 读取时采用类似的机制，将逻辑坐标转换为物理存储坐标，再从 shared memory 的对应物理位置去读。

Profile 结果如下：
图中显示 shared memory store 时 bank conflicts=0，但 load 时还有 bank conflict，推测跟 swizzling 无关。

	Instructions	Requests	Wavefronts	% Peak	Bank Conflicts
Shared Load	65,536	65,536	72,568	2.53	7,032
Shared Load Matrix	0	0			
Shared Store	65,536	65,536	65,536	0.57	0
Shared Store From Global Load	0	0	0	0	0
Shared Atomic	0	0	0	0	0
Other	-	-	143,620		
Total	131,072	131,072	281,724	9.84	17,532

知乎 @frankshi

32.5 TMA 中的 swizzling 机制

TMA：Tensor Memory Accelerator，参考这里。

矩阵乘法中，为了减少内存访问指令数目，通常通过向量化的访问指令来实现，比如`int2`/`int4`，`float2`/`float4`等，即一次访问 8 字节或 16 个字节。这种情况下，数据 chunk 的大小就不是 4 字节，跟单个 bank 的大小就不一样，此时仍然有 bank 冲突的现象存在，只是发生冲突的 bank 号不是连续分布，而是间隔分布，比如 bank 为 0，4，8 等等。为了避免冲突，可以按 chunk 为单位进行 swizzling，其核心的处理思想仍然是使用异或操作，下图是一个示例：

2D-Array按chunk = 16Bytes 进行swizzling

Bank	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
Y/X	0				1				2				3				4				5				6				7			
0	0				1				2				3				4				5				6				7			
1	1				0				3				2				5				4				7				6			
2	2				3				0				1				6				7				4				5			
3	3				2				1				0				7				6				5				4			
4	4				5				6				7				0				1				2				3			
5	5				4				7				6				1				0				3				2			
6	6				7				4				5				2				3				0				1			
7	7				6				5				4				3				2				1				0			

知乎 @frankshi

下面说明 TMA 中进行 swizzling 的处理过程：

- TMA 以 128 字节单位为 1 个 segment，以 16 字节为 1 个 chunk。
- 通过对 segment 内的 chunk 进行 swizzling，来实现不同 segment 间的 bank conflict free。

TMA 假设使用的 shared memory 的数组的形式是`T array[][NX]`（T 可以是 1/2/4Bytes 的数据类型）。硬件要求行字节数不超过 swizzle 宽度，本章为简化后续映射，取等号：

- `NX * sizeof(T) <= SWIZZLE_SIZE`；下面的推导假设取等号。
- `SWIZZLE_SIZE` 必须是 32，64 或者 128 三个值之一。

下面的计算过程假设`SWIZZLE_SIZE=128`字节，对于数组中逻辑坐标是`[y][x]`的元素，通过 TMA 映射后，在物理存储数组中的坐标为`[y_swz][x_swz]`。跟之前的映射类似，物理坐标`y_swz`与逻辑坐标 y 保持一致，`x_swz`的计算过程如下：

- 计算 16 字节的 chunk 块的索引
- `i16 = (y * NX + x) * sizeof(T) / 16`。`y16 = i16 / 8`。`x16 = i16 % 8`。
- 计算这 16 字节的 chunk 块的 swizzling 的索引
- `y16_swz = y16`，即 chunk 块的行保持不变。`x16_swz = y16^x16`，即 chunk 块的列为逻辑行和逻辑列的异或值。
- 计算 chunk 块的某个 x 元素的映射后的索引
- 先计算 chunk 块在行内的偏移，然后计算 chunk 块中的元素在 chunk 块中的偏移，两者之和即为偏移。`x_swz = x16_swz * 16 / sizeof(T) % NX + x % (16 / sizeof(T))`。

Swizzled indices of 16-byte chunks		
32-byte	64-byte	128-byte
0 1	0 1 2 3	0 1 2 3 4 5 6 7
0 1	0 1 2 3	1 0 3 2 5 4 7 6
0 1	1 0 3 2	2 3 0 1 6 7 4 5
0 1	1 0 3 2	3 2 1 0 7 6 5 4
1 0	2 3 0 1	4 5 6 7 0 1 2 3
1 0	2 3 0 1	5 4 7 6 1 0 3 2
1 0	3 2 1 0	6 7 4 5 2 3 0 1
1 0	3 2 1 0	7 6 5 4 3 2 1 0

知乎 @frankshi

对于不同的SWIZZLE_SIZE进行坐标重排后的其效果如下：

由于SWIZZLE_SIZE为 128 字节时，刚好对应了 32 个 bank 的大小，此时在 chunk 层面的进行的 swizzling 映射不会改变 chunk 所在的行，只会改变列。如果SWIZZLE_SIZE不为 128 字节时，映射后有可能改变行，比如对于`float4 array[15][4]`这样的数组，SWIZZLING_SIZE = 64字节，对于数组中超过第 8 行的那些数据，第 8 行后的 chunk 块的前 4 个和后 4 个的相对位置已经发生变化，如下图所示：

对SWIZZLING_SIZE不为 128 字节的情况，chunk 的处理逻辑与上面的处理过程类似，但从 chunk 到单个元素的转换时，需要做些调整。

32.6 Cutlass 中的 swizzling 使用

Cutlass 使用 tensor core 的进行矩阵乘法时，比如对于m16n8k16的 fp16 的矩阵乘法，需要分 4 个 phase 从 shared memory 分别加载 4 个 8x8 的 fp16 的矩阵数据到线程寄存器中，此时的 bank conflict 只局限同 phase 的 8 个线程内，可以只考虑针对这 8 个线程需要访问的 shared memory 来进行 swizzling，从而避免 bank conflict，感兴趣的读者可以去找资料学习。

32.7 参考资料

- Advanced Performance Optimization in CUDA - NVIDIA GTC 2024 https://www.nvidia.com/gtc/session-catalog/?search.h1topich1pppacceleratedcomputingtoolstechniquesp=1699467437927003C0hc#/session/1695395019805001m1oR
- https://www.nvidia.com/en-us/on-demand/session/gtcsj20-s21745/
- https://leimao.github.io/blog/CUDA-Shared-Memory-Swizzling/
- https://zhuanlan.zhihu.com/p/710337546

2D-Array按chunk = 16Bytes 进行swizzling

Bank	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
Y/X	0				1				2				3				4				5				6				7			
0	0				1				2				3				4				5				6				7			
1	1				0				3				2				5				4				7				6			
2	2				3				0				1				6				7				4				5			
3	3				2				1				0				7				6				5				4			
4	4				5				6				7				0				1				2				3			
5	5				4				7				6				1				0				3				2			
6	6				7				4				5				2				3				0				1			
7	7				6				5				4				3				2				1				0			

2D-Array: float4 Array[15][4]

Y/X	0	1	2	3
0	0	1	2	3
1	0	1	2	3
2	1	0	3	2
3	1	0	3	2
4	2	3	0	1
5	2	3	0	1
6	3	2	1	0
7	3	2	1	0
8	0	1	2	3
9	0	1	2	3
10	1	0	3	2
11	1	0	3	2
12	2	3	0	1
13	2	3	0	1
14	3	2	1	0
15	3	2	1	0

知乎 @frankshi